

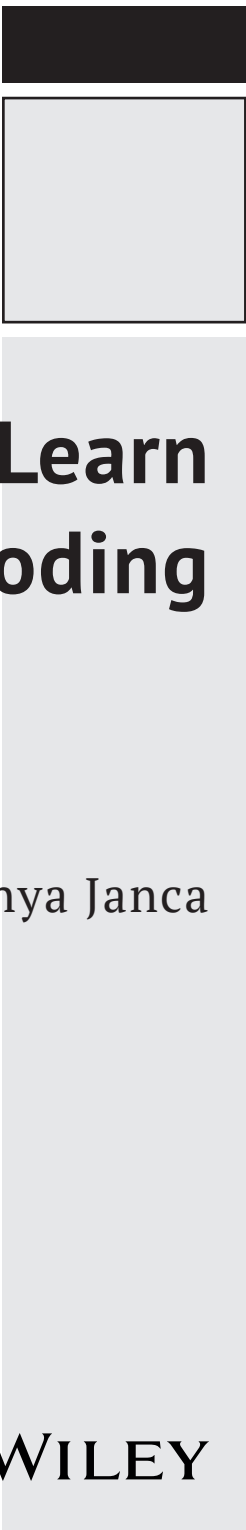
Таня Янца

Алиса и Боб учатся безопасному кодированию



Таня Янца

Алиса и Боб учатся безопасному кодированию



Alice & Bob Learn Secure Coding

Tanya Janca

WILEY

Алиса и Боб учатся безопасному кодированию

Таня Янца



Москва, 2025

УДК 004.42
ББК 32.973
Я65

Янца Т.

Я65 Алиса и Боб учатся безопасному кодированию / пер. с англ. А. А. Слинкина. – М.: ДМК Пресс, 2025. – 410 с.: ил.

ISBN 978-5-93700-388-1

Эта книга – путеводитель по безопасному кодированию, содержащий как советы, применимые к любому техническому стеку, так и более специальные рекомендации для десятка языков программирования, нескольких технологий и всех известных классов уязвимостей. Она призвана вселить в каждого программиста чувство ответственности за качество создаваемых им программных продуктов.

Книга адресована начинающим программистам, желающим сделать карьеру разработчика программного обеспечения или члена команды по безопасности.

УДК 004.42
ББК 32.973

All rights reserved. This Translation publish under license with the original publisher John Wiley & Sons, Inc.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-39417-170-5 (англ.)
ISBN 978-5-93700-388-1 (рус.)

© 2025 by John Wiley & Sons, Inc.
© Перевод, оформление, издание,
ДМК Пресс, 2025

Я посвящаю эту книгу моему самому любимому человеку.

*Спасибо, что ты останавливал мир на время,
которого мне хватило, чтобы написать эту книгу.*

Я чувствую, что с твоей поддержкой могу все.

Содержание

От издательства	15
Об авторе	16
О технических редакторах	17
Благодарности	18
Вступительное слово	19
Введение	21
Часть I ОБЩИЕ РЕКОМЕНДАЦИИ	23
Глава 1 Введение в основы безопасности	24
Предполагайте, что все прочие системы и данные небезопасны	24
Триада CIA	25
Принцип наименьших привилегий	27
Безопасные умолчания и вымощенные дороги	29
Предполагайте возможность взлома, создавайте планы действий в чрезвычайных ситуациях	30
Нулевое доверие	31
Эшелонированная оборона	31
Безопасность цепочки поставок	32
Безопасность через неясность	32
Уменьшение поверхности атаки	33
Беспроblemная безопасность	33
После отказа закрыть, затем откатиться	34
Комплаенс, законы и нормативные акты	34
Каркасы безопасности	36
Учимся на ошибках и делимся уроками	38
Обратная совместимость (и сопутствующие ей потенциальные риски)	38
Моделирование угроз	39
Трудности обновления	39
Повторное тестирование исправлений новых дефектов	

безопасности	40
Упражнения	41
Глава 2 В начале пути	43
Применяйте безопасный жизненный цикл разработки ПО	43
Используйте современный каркас и все имеющиеся в нем средства безопасности	44
Проверка входных данных	45
Кодирование вывода	48
Примеры кодирования вывода	50
Контекст HTML.....	50
Контекст JavaScript	50
Параметризованные запросы и ORM	51
Аутентификация и идентификация	53
Авторизация и управление доступом.....	55
Модели управления доступом	56
Логические методы управления доступом (реализация).....	56
Управление сеансами.....	57
Управление секретами.....	58
Управление паролями	60
Безопасность связи (только криптография и HTTPS).....	62
Защита конфиденциальных данных.....	63
Заголовки, связанные с безопасностью.....	65
Новые функции заголовков, связанных с безопасностью.....	66
Заголовки запроса метаданных	66
Заголовок Content Security Policy.....	66
Strict-Dynamic	67
Trusted-Types.....	67
Ранее рассмотренные заголовки, связанные с безопасностью	67
Заголовок Content-Security-Policy.....	68
HTTP Strict-Transport-Security	68
X-Frame-Options	68
X-Content-Type-Options.....	68
Permissions-Policy.....	69
Expect-CT	69
Referrer-Policy.....	69
Расширение для фиксации открытого ключа HTTP (HPKP)	69
X-XSS-Protection	69
Еще несколько новых заголовков	69
Политика одного источника	70
COEP: Cross-Origin Embedder Policy.....	70
COOP: Cross-Origin Opener Policy.....	71
CORP: Cross-Origin Resource Policy	71
CORS: Cross-Origin Resource Sharing.....	72
CORB: Cross-Origin Read Blocking.....	72

Безопасные cookie.....	73
Обработка ошибок.....	74
Упражнения	75
Глава 3 Улучшение	77
Безопасность баз данных.....	78
Четыре взгляда на защиту баз данных.....	78
Управление файлами.....	81
Загрузка файлов на сервер	83
Ваш исходный код	84
Управление памятью (переполнение буфера, стека, строки и целого числа).....	85
Как избежать переполнения?	87
Сериализация и десериализация	88
Конфиденциальность (пользователя, гражданина, заказчика, работника).....	89
Ошибки.....	92
Протоколирование, мониторинг и уведомления.....	95
Закрытие при сбое	96
Блокировка ресурсов.....	97
Привлечение менеджеров паролей	98
Криптографическая практика.....	98
Строго и слабо типизированные языки	100
Строго типизированные языки	100
Слабо типизированные языки.....	101
Предметно-ориентированная разработка	102
Языки, безопасные относительно памяти	103
Упражнения	104
Глава 4 Достижение цели	105
Безопасное проектирование	106
Сколько безопасности (при проектировании) «достаточно»?	108
Управление зависимостями и безопасность цепочки поставок	110
Безопасность зависимостей.....	110
Проверяем, безопасно ли использовать зависимости	111
Безопасность цепочки поставок.....	111
Безопасные умолчания	114
Безопасные умолчания для пользователей.....	115
Безопасные умолчания для разработчиков.....	116
Код, удобный для чтения и аудита	118
Важные функции выполняются в надежных системах	121
Что такое «ненадежная» система?.....	121
Что такое «важные функции»?.....	122
Собираем все вместе	122
Черные и белые списки.....	123

Почему черные списки – это плохо?.....	123
Как создать белый список?.....	123
Безопасные конфигурации.....	124
Проверка имени хоста.....	125
Повторно используемый код	126
Безопасные системные вызовы	127
Смягчение последствий.....	128
Комментарии и другая документация.....	128
Комментарии	128
Документация	130
Запрос согласия пользователя	131
Проверки целостности, подписание кода и неизменяемые сборки ...	132
Неизменяемые сборки.....	134
Избегать грубой силы.....	135
Элементы безопасности.....	136
О расширенных привилегиях	137
Обслуживание безопасности.....	138
Оплата технического долга	139
Упражнения	141
Часть I – итоги	143
Контрольный список общих рекомендаций по безопасному кодированию	143
Часть II КОНКРЕТНЫЕ РЕКОМЕНДАЦИИ	151
Глава 5 Рекомендации, зависящие от технологии	152
Рекомендации по безопасности API	152
Рекомендации по безопасности мобильных приложений.....	159
Рекомендации по безопасности WebSocket.....	163
Рекомендации по бессерверным вычислениям.....	164
Рекомендации по безопасности IoT	165
Упражнения	167
Глава 6 Популярные языки программирования	169
JavaScript.....	169
HTML и CSS	174
Конкретно о HTML5.....	176
Python	178
SQL.....	180
Node.js.....	184
Java	187
Сериализация в Java	191
TypeScript	193
C#.....	194

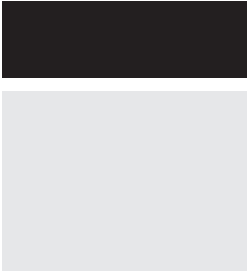
PHP	198
C/C++	203
Заключение	206
Упражнения	207
Глава 7 Популярные каркасы.....	208
Веб и JavaScript	208
Express	208
React.js	210
Angular.....	213
jQuery.....	217
Vue.js	219
Другие каркасы и библиотеки.....	221
.NET (Core).....	221
Ruby on Rails	226
Spring и Spring Boot	232
Flask	234
Упражнения	238
Глава 8 Категории уязвимостей.....	239
Просчеты проектирования (логические просчеты).....	240
Как такое возможно?.....	241
Риск.....	241
Предотвращение.....	242
Дефекты в коде (ошибки реализации).....	242
Как такое возможно?.....	243
Риск.....	243
Предотвращение.....	243
Переполнения и другие проблемы при работе с памятью.....	244
Переполнения	244
Чтение за границей буфера.....	245
Страничные отказы.....	245
Использование после освобождения.....	246
Неинициализированные переменные	246
Утечки памяти.....	246
Как такое возможно?.....	247
Риск.....	247
Предотвращение.....	247
Внедрение кода: проблемы интерпретатора и компилятора	249
Как такое возможно?.....	249
Риск.....	249
Предотвращение.....	250
Проблемы с входными данными.....	251
Как такое возможно?.....	251
Риск.....	252

Предотвращение	252
Проблемы аутентификации и идентификации	252
Как такое возможно?	252
Риск	253
Предотвращение	253
Проблемы авторизации и доступа	254
Как такое возможно?	254
Проблемы конфигурации и реализации	254
Как такое возможно?	255
Риск	255
Предотвращение	255
Мошеннические транзакции	256
Как такое возможно?	256
Риск	257
Предотвращение	257
Атаки воспроизведением	257
Как такое возможно?	258
Риск	258
Предотвращение	258
Пересечение границ доверия	259
Как такое возможно?	260
Риск	260
Предотвращение	260
Проблемы при работе с файлами	260
Как такое возможно?	260
Риск	261
Предотвращение	261
Проблемы при работе с объектами	261
Отличительные особенности ООП	262
Десериализация и другие проблемы при работе с объектами	264
Как такое возможно?	264
Риск	264
Предотвращение	264
Проблемы управления секретами	265
Как такое возможно?	266
Риск	266
Предотвращение	266
Состояния гонки и проблемы хронометража	267
Как такое возможно?	267
Риск	268
Предотвращение	268
Проблемы при работе с ресурсами	270
Как такое возможно?	270
Риск	271
Предотвращение	271

Попадание в неизвестное состояние	271
Как такое возможно?	272
Риск	272
Предотвращение	272
Упражнения	273
Часть II — итоги	274
Контрольный список рекомендаций по технологическим аспектам безопасного кодирования	274
Контрольный список рекомендаций по безопасному кодированию для языков и каркасов	275
Сводка уязвимостей, на которые следует обращать внимание	277
Часть III БЕЗОПАСНЫЙ ЖИЗНЕННЫЙ ЦИКЛ РАЗРАБОТКИ СИСТЕМЫ	279
Глава 9 Требования	280
Начало проекта: общий обзор действий по обеспечению безопасности	280
Составление графика и планирование проекта	281
Требования к безопасности	282
Упражнения	284
Глава 10 Проектирование	285
Моделирование угроз	286
Паттерны и концепции безопасного проектирования	288
Изображение архитектуры на доске	289
Изучение потоков данных	289
Пользовательские истории о безопасности	290
Упражнения	291
Глава 11 Кодирование	293
Обучение	293
Организации	295
Физические лица	296
Анализ кода	297
Статические анализаторы первого и второго поколения	297
Безопасные ограждения	299
Плагины IDE и другие наставления	299
Проверка безопасности зависимостей	300
Как решить, какие зависимости следует обновить или заменить?	301
Поиск секретов и управление ими	302
Динамическое тестирование (DAST)	303
Упражнения	305

Глава 12 Тестирование.....	306
Тестовое покрытие и хронометраж	307
Выбор между глубиной и широтой покрытия.....	308
Сканирование инфраструктуры	308
Производственная и менее ответственные среды	308
Охват.....	309
Хронометраж.....	309
Ручное тестирование	311
Автоматизированное тестирование	313
Фаззинг	315
Интерактивное тестирование приложений на безопасность (IAST).....	315
Программы вознаграждения за найденные ошибки	316
Результаты тестирования	318
Действия по результатам тестирования	319
Заключительные мысли.....	321
Упражнения	321
Глава 13 Выпуск и развертывание.....	322
События безопасности в системе CI/CD	323
Прерывание сборки.....	324
Сканирование секретов.....	325
Статический анализ	325
Динамический анализ	325
Анализ состава программы.....	326
Линтинг	326
Сканеры инфраструктуры как кода	326
Обеспечение безопасности самого конвейера CI/CD.....	327
Поддержание целостности сборки	330
Одобрение выпуска группой безопасности.....	331
Упражнения	331
Глава 14 Сопровождение	333
Мониторинг, уведомление и наблюдаемость.....	334
Блокировка и экранирование	336
Брандмауэры веб-приложений (WAF).....	337
Сети доставки содержимого (CDN)	337
Самозащита приложения на этапе выполнения (RASP).....	338
Виртуальное латание	338
Шлюзы API	338
Замечание специально для исследователей данных	339
Непрерывное тестирование	340
Инциденты безопасности.....	341

Непрерывность бизнеса и планирование аварийного восстановления	343
Упражнения	345
Глава 15 Заключение	346
Хорошие привычки.....	346
Ваша обязанность	349
Сколько будет достаточно?.....	350
Безопасное использование искусственного интеллекта	352
Непрерывное обучение.....	354
Станьте ответственным за безопасность	356
Все на борт	357
Переход в группу безопасности.....	357
Устройство на работу в области безопасности в другой организации	358
Заключение	363
Часть III – итоги.....	366
Контрольный список действий, связанных с безопасностью, на каждом этапе SDLC	366
Приложение А. Ресурсы	369
Приложение В. Ответы к упражнениям.....	385
Предметный указатель	401



От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

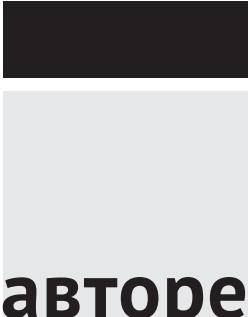
Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательства «ДМК Пресс» и Wiley очень серьезно относятся к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

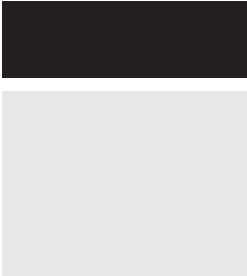
Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.



Об авторе

Таня Янца, она же SheHacksPurple, – автор бестселлеров «Alice and Bob Learn Secure Coding», «Alice and Bob Learn Application Security» и карточной игры AppSec Antics. За 28 лет карьеры в ИТ она стала лауреатом бесчисленных премий (включая звание пожизненного выдающегося члена OWASP и хакера года), выступала с докладами по всей планете и является плодовитым блогером. Таня обучила тысячи разработчиков программного обеспечения и специалистов по безопасности в своих онлайн-академиях (We Hack Purple и Semgrep Academy) и на практических семинарах. В рамках контртеррористической деятельности обеспечивала безопасность выборов 52-го генерал-губернатора Канады и обезопасила бесчисленное количество приложений. Таня Янца считается общепризнанным международным авторитетом по безопасности программного обеспечения.



О технических редакторах

Барре Дийкстра 25 лет занимается разработкой ПО на различных ролях – от практического программирования до руководства.

За это время он пришел к выводу о необходимости уделять больше внимания безопасности приложения, от кода до архитектуры и процесса. В настоящее время он часто оказывает поддержку клиентским командам по таким вопросам, как моделирование угроз, конфиденциальность и безопасность по определению, а также работает с группами обеспечения безопасности, занимающимися разработкой.

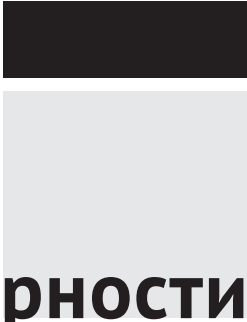
Помимо работы в сфере разработки ПО, Барре играет в CTF*, работает волонтером в Голландском институте обнаружения уязвимостей (Dutch Institute for Vulnerability Disclosure – DIVD), который принимает участие в организации одного из крупнейших соревнований по CTF, и несколько семестров преподавал на курсах подготовки бакалавров в области методов взлома и обратного проектирования.

Фабьен Ло Рикко – системный инженер с богатым 15-летним опытом в области конструирования ПО, системной инженерии и технической поддержки. Специализируется на обеспечении безопасности с упором на предотвращение потери данных (data loss prevention – DLP) и применяет свой практический опыт для защиты конфиденциальной информации.

Занимаясь на протяжении карьеры обслуживанием малых предприятий, он наработал разносторонний набор навыков, покрывающих весь спектр ИТ – от технической поддержки до проектирования архитектуры систем. Он активно выступает за ПО с открытым исходным кодом и применяет как проприетарные, так и открытые решения для проектирования надежных и экономичных систем, ориентированных на потребности бизнеса.

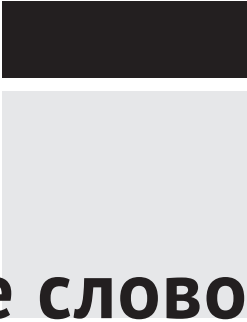
Франциска Пассинг – старший инженер-эксплуатационник в Дуолинго. Раньше работала в отделах инженерии внутренних платформ и обеспечения безопасности в таких компаниях, как Mapbox, Microsoft и др. Является опытным докладчиком, организатором технических сообществ и активным членом феминистского хакерского клуба Heart of Code. В настоящее время проживает в Берлине, где любит попивать кофе и участвовать в триатлонах. Связаться с ней можно через сайт <https://franka.tech>.

* Capture the Flag (захвати флаг) – командная игра по компьютерной безопасности. – Прим. перев.



Благодарности

Выражаю глубокую благодарность трем моим выдающимся техническим редакторам (Фабьену Ло Рикко, Франке Шмидт и Барре Дийкстра), чья непоколебимая поддержка, конструктивная критика и преданность были для меня стимулом на протяжении всего времени написания второй моей книги. Их воодушевление, настойчивость и стремление к совершенству не только повысили качество работы, но и вдохновили меня на достижение новых высот на моем писательском пути. Я в неоплатном долгу перед ними за их бесценный вклад и за то, что они находились со мной больше двух лет, сопровождая на всех подъемах и спусках процесса написания книги. Мне не хватает слов, чтобы в должной мере отблагодарить их за наставления, опыт и веру в проект.



Вступительное слово

С технической точки зрения сейчас отличное время быть инженером-программистом. Когда я была юным ботаном, зачитывалась научно-фантастическими романами и смотрела ранние эпизоды фильмов «Доктор Кто» и «Звездный путь: новое поколение», меня очаровывали технологии, демонстрируемые на экране. В том новом будущем у нас были приложения и приспособления для решения любой проблемы.

25 лет промелькнули быстро, и кажется, что мы сейчас живем в век необычайных изобретений. Программы проникли чуть ли не в каждую часть нашей жизни, от работы до свободного времени, а создаем их мы, инженеры.

Системы, которые мы пишем, объединяют нас и помогают сотрудничать и делиться информацией в невиданных ранее масштабах. Программы также помогают нам определять состояние здоровья, управлять роботами, которые помогают преодолевать неблагоприятные окружающие условия и даже ездят вместо нас по улицам наших городов. Однако при такой зависимости от программ и преимуществ, которые они предлагают, мы несем общую ответственность за то, что эти программы могут делать, и за их воздействие на людей, данные и системы.

В начале моей карьеры разработчика создаваемые мной системы (в области финансов и налогообложения) были не такими ответственными, как теперь. Эти системы обычно использовались только с 9 утра до 5 вечера с понедельника по пятницу, а если они вдруг переставали работать, то это, конечно, было неудобно, но компенсировалось многочисленными ручными системами.

Нынешние программы так глубоко вплетены в нашу жизнь, что последствия инцидентов и проблем значительно многообразнее и – в некоторых случаях – намного опаснее.

Именно поэтому настоящая книга и вся Танина работа по безопасному кодированию так важны. Мы, разработчики программного обеспечения, стоим на распутье. Мы обязаны следить за тем, чтобы наши программы и системы были высочайшего качества и отвечали самым строгим стандартам. Мы несем ответственность за баланс производительности, масштабируемости, удобства использования, наблюдаемости и доступности – всего того, благо-

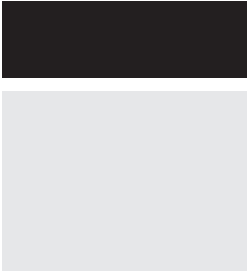
даря чему наши приложения работают как задумано, когда это необходимо, обслуживая максимально возможное количество людей.

Самое время добавить сюда еще и безопасность.

В этой книге Таня постаралась сделать безопасное кодирование доступным разработчикам ПО на все случаи жизни и во всех программных экосистемах. Благодаря ее легким для понимания примерам и простому, прагматичному подходу даже самые занятые команды разработчиков смогут приступить к работе над безопасностью своего кода с учетом сложившегося у них цикла разработки, культуры и выбора языка. Изобилующая примерами и врезками, эта книга станет удобным справочником, который вы захотите вручить своей команде.

Если мы собираемся строить это чудесное пронизанное программами будущее вместе, то надеюсь, что сможем уделить время тому, чтобы сделать каждый следующий день немного безопаснее, чем предыдущий, а каждую строку кода будем рассматривать как возможность улучшить качество и безопасность своих систем.

Лаура Майн Белл,
генеральный директор и основатель компании SafeStack



Введение

Прямо сейчас наша индустрия требует от программистов больше, чем когда-либо прежде: я уверена, все вы видели статьи, призывающие вас стать 10-кратным инженером, свободно владеть несколькими языками программирования и освоить все аспекты разработки полного цикла. Но одно четкое и непререкаемое ожидание выделяется из общего ряда: программист обязан создавать безопасные приложения. И не имеет значения, что это не обсуждалось в вашем курсе информатики, что команда «безопасников» ничем не может помочь и что у вас нет подходящего инструментария. Ваши пользователи, коллеги и организация в целом рассчитывают, что вы будете поставлять надежное и безопасное программное обеспечение.

Эта книга научит вас писать более безопасный код. Точка. Это набор инструментов, который позволит вам стать первым среди равных, разрабатывать непробиваемые приложения и обеспечивать безопасное хранение данных. Прочитав эту книгу, вы сделаете гигантский шаг к тому, чтобы стать старшим инженером-программистом или войти в группу безопасности.

В отличие от многих учебников эта книга не раз заставит вас улыбнуться. Возможно, вы сочтете, что ее материал удивительно легко понять и впитать. Как бывший профессиональный массовик-затейник, к тому же страдающий дислексией, я старалась сделать эту книгу приятной для чтения и как можно более простой для понимания. Прибегая к шуткам и различным приемам обучения (анalogии, узнаваемые истории, сочувствие к Алисе и Бобу, диаграммы, повторение сложных абстрактных концепций и т. д.), я стремилась написать книгу, которую хотела бы иметь в начале своей карьеры.

Эта книга – путеводитель по безопасному кодированию, содержащий как советы, применимые к любому техническому стеку, так и более специальные рекомендации для 10 языков программирования, 9 каркасов, нескольких технологий и всех известных мне классов уязвимостей. Это действительно *много* информации. Материала достаточно, чтобы создать отличное руководство по безопасному кодированию для вашего места работы.

Это не просто книга, это интерактивное путешествие. На протяжении 2025 и 2026 годов я буду каждый месяц проводить прямые трансляции для глав книги. На этих мероприятиях мои (многознающие) друзья будут вместе со

мной обсуждать каждую главу, включая ответы в конце. Предусмотрены также вопросы и ответы.

Я писала эту книгу не для себя, а для вас. Когда я только начинала узнавать о безопасности, это была мука. Те немногие ресурсы, которые удавалось найти, мало на что годились. Я решила сделать своей личной миссией помогать переходу на безопасные приложения и совершенствовать подходы к безопасности ПО в нашей индустрии. Но со временем я поняла, что если хочу добиться значительных изменений, то должна обратить внимание на разработчиков. И вот я здесь, любезные читатели.

Я написала эту книгу еще по одной причине и подозреваю, что по той же причине вы ее и купили. Я хочу, чтобы и я сама, и дорогие мне люди могли пользоваться программами *безопасно*. Я хочу, чтобы продуктам, которые производит наша индустрия, можно было *доверять*. На данный момент это не так; все мы слышали о бесконечных утечках данных, об инцидентах безопасности и другом вреде или ущербе, который был нанесен в результате эксплуатации небезопасных приложений. И пока мы ждем, что наши правительства предпримут коллективные действия, я пытаюсь поднять планку. Этой книгой я надеюсь вселить в каждого программиста чувство ответственности за безопасность создаваемых им программных продуктов.

Каждым в жизни движет что-то свое. Лично для меня очень важно творить добро. Делясь своими знаниями и возвращая сообщества разработчиков, преданных созданию безопасного ПО, я надеюсь оградить множество людей от вреда. Но для этого мне нужна ваша поддержка. Готовы отправиться в путешествие вместе со мной?

Я приглашаю вас погрузиться в эту книгу и воспользоваться содержащимися в ней знаниями для создания поистине достойных восхищения программ.

Общие рекомендации

Глава 1. Введение в основы безопасности

Глава 2. В начале пути

Глава 3. Улучшение

Глава 4. Достижение цели

Введение в основы безопасности

Эта глава посвящена фундаментальным концепциям безопасности, вещам, которые, с точки зрения всякого «безопасника», должен знать каждый профессионал в сфере ИТ. Возможно, у вас не было и не будет возможности применить их все в своей работе, но, понимая их, вы сможете создавать более надежные и защищенные системы.

Предполагайте, что все прочие системы и данные небезопасны

Быть может, самый важный урок всей этой книги – научиться по умолчанию не доверять компьютерным системам, данным и пользователям. Люди вообще по природе своей весьма доверчивы¹. С точки зрения создания общества с его законами, безопасностью и порядком в целом хорошо, что большинство людей изначально готовы выказать доверие. Это часть того, что заставляет наше общество работать.

Поскольку люди, вообще говоря, склонны к доверию, мы и при проектировании компьютерных систем закладываем в них *подразумеваемое доверие*. Это значит, что, вместо того чтобы автоматически проверять факты, наши компьютерные системы предполагают, что они должны быть истинны. Последствия могут быть катастрофическими.

КЛАССНЫЙ ВОРИШКА

Учась в колледже, Алиса привыкла сохранять свои рабочие файлы на общем для всей школы сетевом диске. Если бы ее компьютер сломался, то она смогла бы взять оттуда резервную копию. На втором году обучения она делала презентацию для курса по программной инженерии, которую готовила для группового учебного проекта. Когда на уроке ее группа готовилась представить свои результаты, одноклассница по имени Ева² попросила ее дать возможность провести презентацию своего проекта первой. Алиса ничего не заподозрила и согласилась. Ева вышла к доске и представила все находки Алисы как свои собственные! Она даже воспользовалась Алисиными слайдами, только на титульной

странице поставила свое имя. Алиса подумала: «Да что ж это такое? Как это могло случиться?!» Все одноклассники аплодировали и были очень довольны Евой. Алиса думала, что, сохраняя свои файлы на общем школьном диске, она обеспечивает их безопасность. У нее и в мыслях не было, что одноклассница сможет украсть ее работу и представить как свою собственную или что кто-то будет копаться в ее папках на общем диске. Алиса *доверяла априори* и обожглась на этом.

При попытке создать безопасную систему нужно взять за важнейшее правило никогда ничему не доверять. Это может означать многофакторную аутентификацию для защиты от атак на учетные данные, перепроверку данных, полученных из базы, на предмет правильности формата и размера или выполнение аутентификации и авторизации через интерфейс прикладного программирования (API) при вызове бессерверного приложения, пусть даже оба были созданы в вашей компании.

Приведем несколько примеров подразумеваемого доверия.

- Зонирование при проектировании сети: после того как кто-то вошел в зону, он может получить доступ к любой системе в этой зоне без необходимости пересекать брандмауэр или повторно аутентифицироваться.
- Прием входных данных от пользователя без проверки с последующим использованием для построения SQL-запроса, URL-адреса для перенаправления или для принятия какого-нибудь другого решения в системе.
- Предоставление API для доступа из интернета без организации перед ним шлюза или какого-нибудь другого механизма аутентификации и авторизации, так что обратиться к нему может кто угодно – включая ботов.

Если, прочитав эту книгу, вы запомните только что-то одно, то пусть это будет следующий совет: при проектировании любой системы подразумеваемое доверие должно быть сведено к минимуму. Проверяйте все, включая входные данные, принимаемые решения, данные и интеграцию с другими системами. Выполняйте более одной проверки, если возможно, и обязательно, когда ставки высоки (совсекретная информация, системы, требующие высокой доступности, и т. д.). Всегда предполагайте, что другие системы и данные потенциально небезопасны.

Триада CIA

Триада CIA – это три вещи, обязательные в большинстве мировых систем ИТ-безопасности. Задача группы кибербезопасности – защищать конфиденциальность, целостность и доступность (Confidentiality, Integrity и Availability) систем и данных, на охрану которых она поставлена. Группа безопасности защищает не только CIA, но эти аспекты часто считаются главными. В этой книге мы будем рассматривать многое другое, в том числе приватность, физическую безопасность, подлинность и многоуровневую защиту.

СОВЕРШЕННО СЕКРЕТНАЯ ИНФОРМАЦИЯ

Боб когда-то выполнял совершенно секретную работу для правительства Канады. Касалась она антитеррористической деятельности, и это все, что ему было разрешено говорить о своей работе. Сохранение секретных данных в безопасности считалось задачей высочайшей важности, и Боб очень серьезно относился к методам защиты данных, которым его учили. Когда Боб проходил обучение в Канадском центре кибербезопасности, его спросили, чем гриф «совершенно секретно» отличается от грифа «секретно» и других. Ответ был таким: «Если эти данные окажутся в распоряжении противника, это может нанести вред Канаде как нации. Потенциально это может привести к гибели сотен людей и привести к провалам в различных других сферах контртеррористической деятельности. Эта информация могла бы не только причинить ущерб государству, но и помешать раскрытию различных заговоров, способных привести к смерти десятков граждан, а в худшем случае даже к успешному государственному перевороту. Ваша важнейшая задача, какой бы сложной, изнурительной и трудной она ни была, — предотвратить попадание этой информации в чужие руки. Защищайте ее буквально ценой собственной жизни». Хотя Боб редко говорил об этом с друзьями и семьей, все они прекрасно знали, как Боб относится к важности своей работы в интересах Министерства юстиции.

Примечание

- **Конфиденциальность:** состояние, в котором информация является секретной или приватной³.
- **Целостность:** внутренняя согласованность или отсутствие повреждений электронных данных³.
- **Доступность:** готовность к использованию или получению в любой момент времени³.

КОГДА КРИТИЧЕСКАЯ ИНФРАСТРУКТУРА ВЫХОДИТ ИЗ СТРОЯ

9 июля 2022 года сеть канадской телекоммуникационной компании Rogers вышла из строя⁴. В Канаде всего три крупные телекоммуникационные компании, а остальные пользуются их линиями связи или арендуют их. Из-за этой аварии население значительной части страны в течение нескольких дней не могло оплачивать счета, заходить в сеть и даже звонить по телефону 911. В фокусе внимания кибербезопасности обычно находятся три вещи: конфиденциальность, целостность и доступность (CIA). Как видите, когда сеть Rogers вышла из строя, пострадала *доступность* на значительной части территории страны, а следовательно, авария стала *проблемой безопасности*. Если всего одна человеческая ошибка может отключить аварийные службы для большей части страны на несколько дней, значит система *небезопасна*. Хотя сама авария не была вызвана кибератакой, она повлияла на одну из компонент триады CIA и, значит, была проблемой безопасности. Отсутствие планов на случай чрезвычайных ситуаций в таких системах *также* является проблемой безопасности. Это не критика в адрес Rogers, а реальный пример того, что безопасность является *частью* управления качеством, а также важности многоуровневой защиты, планирования непрерывности бизнеса и аварийного восстановления.

Хотя выше мы привели определение *целостности*, давайте еще немного поговорим о смысле этого слова. Часто, говоря о «целостном» характере»

³ В английском языке слово *integrity* означает также «честность, профессиональная этика, безукоризненная репутация». — Прим. перев.

человека, мы имеем в виду, что ему можно доверять: на него можно положиться, зная, что он всегда принимает «верное» решение. В компьютерных системах целостность имеет похожий смысл; если система и ее данные целостны, то мы можем использовать их для принятия решений в уверенности, что эти решения будут хорошими. Эти данные уже были проверены, они заслуживают доверия.

Теперь представьте, что вы врач и пользуетесь компьютером, чтобы рассчитать дозу лекарства в зависимости от веса, роста, медицинских показателей пациента и других факторов. Назначая пациенту препарат, вы *предполагаете*, что дозировка правильная, – информация, сообщенная вам компьютером, должна помочь, а не навредить пациенту. Поступая так (доверяя результатам системы), вы предполагаете, что система и ее данные *целостные*. Представьте, какой ужас испытал бы врач, если бы компьютер выдал неверный рецепт и пациент пострадал бы от этого. Для одних систем целостность данных критична (в том числе при назначении медицинских препаратов), для других – не так важна (скажем, если в рецепте торта предполагается положить одно яйцо вместо двух, то конец света точно не наступит). При создании системы очень важно знать, какие из компонентов CIA наиболее важны, и проектировать систему и ее тесты, помня об этом.

Принцип наименьших привилегий

Принцип наименьших привилегий (PoLP) относится к концепции информационной безопасности, когда пользователю предоставляется минимальный уровень доступа (или прав), необходимый ему для выполнения своей работы.

— CyberArk⁵

Хотя из этой цитаты вроде бы следует, что принцип наименьших привилегий распространяется только на пользователей, на самом деле это не так; он относится к любому лицу или предмету, который может обладать доступом или привилегиями, включая компьютерные системы, в частности программное обеспечение и ИИ. Принцип наименьших привилегий также говорит, что доступ должен предоставляться на время, когда он необходим, и ни секундой дольше.

БОЛЬШЕ НИКАКИХ ПРАВ АДМИНИСТРАТОРА

Алиса помнит, как группа безопасности преподала ей первый урок по концепции наименьших привилегий. Ей было сказано не использовать привилегии администратора на своей рабочей станции. Алисе это не понравилось. Она пыталась объяснить, что, будучи руководителем высшего звена, она нуждается в административных привилегиях для установки найденных в интернете программ, не дожидаясь разрешения со стороны группы

технической поддержки. «Безопасники» сообщили ей, что она ни в коем случае не должна устанавливать программное обеспечение, не полученное от компании, и более того – что, в частности, по этой причине эту привилегию у нее отбирают. Ей объяснили, что уже несколько руководителей стали жертвами фишинга, а из-за того, что они обладали привилегиями администратора и не установили никакой защиты от вредоносного ПО, возникло множество проблем. Ей дали понять, что она, как руководитель, является заманчивой целью для противников, так как обладает большими возможностями в компании. Ее также заверили, что свою работу она прекрасно сможет выполнять и без этой привилегии, а если ей понадобится какая-то программа, которой у нее нет, ее достанут – нужно только попросить. Алиса не получила большого удовольствия от этой беседы, но было видно, что «безопасники» не отступят, поэтому она сказала «Что ж, хорошо» (как будто у нее был другой выбор).

Публикуя или развертывая новую программу или добавляя пользователя в сеть или иную электронную систему, мы должны решить, какой объем прав ей (ему) предоставить и на какое время. Если дать доступ ко всему, то у субъекта появится возможность нанести вред любому компоненту системы. Вред может быть случайным или намеренным. Может также случиться, что учетная запись обычного пользователя была взломана злоумышленником, а затем использована для нанесения ущерба вашей сети, приложению или всему тому, к чему у него есть доступ.

Но если мы ограничим права и привилегии субъекта, предоставив доступ только к тому, что необходимо для выполнения работы, то ущерб, который он может нанести, будет существенно меньше. Это относится ко всем электронным системам, в том числе и программным! Рассмотрим несколько примеров.

Пример 1. Уолтер – отец близнецов-подростков, которым в полночь надлежит быть дома. В доме Уолтера установлены электронные замки, которые перестают распознавать их отпечатки пальцев в пять минут первого. Это значит, что если дети пропустили «комендантский час», то они должны будут звонить в дверь и тем самым сообщать родителем, что припозднились. Однако родители могут приходить домой в любое время. Привилегии родителей по доступу в свой дом иные, чем у детей, у которых их меньше. Уолтеру так нравится!

Пример 2. Вы разрабатываете графический интерфейс пользователя (GUI) для своего приложения, три API, которые выполняют различные службы от имени интерфейса, и одну базу данных, в которой хранятся все ваши данные. Для приложения создана одна служебная учетная запись в сети. В идеале ваши API могут вызываться только от имени учетных записей служб, а все остальные вызовы должны игнорироваться. Ваша база данных должна принимать только подключения от ваших же API, принадлежащих вашему приложению, использующему учетную запись вашей службы, а также от администратора базы данных (DBA). Тогда если другое приложение или сетевой объект попытается отправить команды вашей базе данных или обратиться к вашим API, то ничего не получится. Вы могли бы дополнительно защитить системные данные, задав наименьший необходимый объем привилегий для работы с данными, например: только выборка или операции создания-чтения-обновления-удаления (CRUD), но не операции, доступные владельцу базы данных (DBO).

Пример 3. На вас возложен контроль доступа к лаборатории, в которой ведутся секретные исследования. Вы выдаете каждому сотруднику личную карточку доступа с фотографией, которую нужно просканировать, чтобы попасть в различные лаборатории. Вы также настраиваете доступ к различным шкафам в системе, но разрешаете сотрудникам проникновение только в те помещения, к которым у них есть доступ. Например, одним ученым разрешен доступ к химикатам, а другие могут использовать очень специальное измерительное оборудование. У каждого сотрудника есть доступ только к тому, что ему нужно для работы, и ни к чему более. Когда сотрудник увольняется или прекращает выполнять прежнюю роль, доступ отзывается.

Пример 3, расширенный. В охраняемых вами лабораториях хранятся чрезвычайно секретные и опасные химикаты и проводятся эксперименты с грифом «совершенно секретно». В этой ситуации налицо одновременно физические и политические риски, поэтому вы повышаете уровень безопасности и уменьшаете объем привилегий, предоставленных каждому сотруднику. Для прохода в здание сотрудник должен предъявить свою нагрудную бирку и пройти сканирование тела и вещей на наличие оружия; затем сотрудник входит в клетку, где его визуально осматривает один из охранников. Для попадания в некоторые помещения необходимы карточки доступа двух человек, а не одного. Доступ в особо важные помещения возможен только при условии многосторонней авторизации.

Безопасные умолчания и вымощенные дороги

Если у компьютерной системы есть какая-то конфигурация, то есть и значения по умолчанию, они являются частью поставленной системы. Часто пользователи оставляют неизменными заводские параметры по умолчанию, открывающие доступ ко всей системе. Это означает, что большинство пользователей так и будут их использовать, даже если они не оптимальны в части конфиденциальности или безопасности. Многие поставщики программного обеспечения извлекают из этого выгоду, продавая значения параметров по умолчанию, предоставляя их другим организациям или как-то иначе употребляя во зло, чем нарушают конфиденциальность своих пользователей.

СЕКРЕТНЫЕ СИГНАЛЫ

Алиса привыкла использовать Telegram для отправки сообщений друзьям, потому что думает, что этот мессенджер очень безопасен и поддерживает сквозное шифрование. В один прекрасный день приятель сообщил ей, что шифрование производится только на пути от клиента к серверу, а на сервере данные дешифрируются. Все эти годы она верила, что ее сообщения зашифрованы и находятся в безопасности, даже от сотрудников Telegraph! Теперь она чувствует себя идиоткой и решила перейти на мессенджер Signal.

Но мы, «повернутые» на безопасности ИТ-профессионалы, можем тоже воспользоваться этим и настроить параметры по умолчанию, так чтобы они были безопасными. Если путь наименьшего сопротивления сделать безопасным, то все окажутся в выигрыше. Если можно сделать самый простой способ достичь чего-то также и самым безопасным, то мы скорее получим желаемые результаты! Всюду, где возможно, прокладывайте вымощенную дорогу, которая будет вести пользователя по самому безопасному пути.

Предполагайте возможность взлома, создавайте планы действий в чрезвычайных ситуациях

Концепция *предполагайте возможность взлома* означает, что планировать, проектировать и реагировать нужно так, как будто система уже взломана или будет взломана в будущем. Под взломом может пониматься несанкционированное проникновение в систему, кража данных, проникновение в здание и т. д.

Примерами могут служить запуск процедуры реагирования на инцидент в момент, когда от программы вознаграждения за найденные ошибки или от программы согласованного раскрытия информации поступили сведения о серьезной уязвимости; использование многоуровневой защиты на случай, если один уровень преодолен, или мониторинг всей внутренней сети на предмет подозрительной активности.

Планирование действий в чрезвычайных ситуациях очень похоже на «предположение возможности взлома» с тем отличием, что включает не реагирование на событие, а только его планирование. Проектирование систем с несколькими уровнями защиты, включающее планы действий в чрезвычайных ситуациях и поддержание в актуальном и выполнимом виде планов обеспечения непрерывности бизнеса (BCP) и планов аварийного восстановления (DR), – вот некоторые способы, которые мы, ИТ-профессионалы, можем применять для составления планов на случай аварии.

ПЛАНИРОВАНИЕ ДЕЙСТВИЙ В ЧРЕЗВЫЧАЙНЫХ СИТУАЦИЯХ В РЕАЛЬНОЙ ЖИЗНИ

Реальный пример планирования действий в чрезвычайных ситуациях дает пандемия COVID-19, достигшая пика в 2020 году. Вспомните, как быстро организации перешли на работу из дома, на продажу своей продукции онлайн и на развертывание систем, обеспечивающих безопасность своих сотрудников. Некоторые организации возглавили это движение, и очень может статься, что для этого они ввели в действие свои планы BCP. И хотя данных не было в открытом доступе, я (автор) очень хотела бы посмотреть на отчеты о финансовых затратах при наличии и отсутствии в организации планов BCP и DR и о том, сколько денег, времени и человеческих жизней было спасено в результате планирования на будущее.

Нулевое доверие

Концепция *нулевого доверия* противоположна предполагаемому или подразумеваемому доверию. Она является продолжением сказанного в первом разделе этой главы: «Предполагайте, что все прочие системы и данные небезопасны». Предполагаемое доверие означает, что после начальной проверки мы доверяем системе, человеку или учетной записи, как правило, на постоянной основе. Например, кому-то вошедшему в сеть разрешен доступ ко всему в этой сети, или человеку, вошедшему в здание, разрешено заходить в каждую комнату и на каждый этаж.

В реальности люди обычно склонны доверять, но все зависит от социальной ситуации. Представьте, что вы застали гостя, приглашенного на обед, шарящим в вашей аптечке; это было бы странно и насторожило бы вас. Вы, вероятно, спросили бы его, что он тут делает, или попросту выгнали из дома. Но компьютеры социальных ситуаций не понимают, так что мы должны научиться их заранее.

Нулевое доверие означает, что никакого предполагаемого или подразумеваемого доверия нет и быть не может. По умолчанию доступ к любой системе запрещен. Если человек не предоставил доступ явно, ответом будет «нет». Реализуя принцип нулевого доверия, мы должны внимательно следить за тем, чтобы все было настроено правильно, иначе система работать не будет. Применение такой концепции требует немалых усилий, но если все сделано правильно, то она очень эффективна.

Эшелонированная оборона

Эшелонированная оборона – простейшая из всех концепций безопасности; она просто означает, что уровней обороны больше одного. Некоторые люди ставят только замок на входной двери, это один уровень обороны. Другие ставят замок, охранную систему, видеокамеры и заводят сторожевую собаку. Это эшелонированная – многоуровневая – оборона. Мы поступаем так, потому что в нашей броне часто имеются слабые места или неизвестные бреши; несколько уровней могут преградить путь нападающим. В идеале вы сами решаете, сколько уровней обороны необходимо, исходя из ценности системы и (или) данных, подлежащих защите, а также риска, что системе или данным будет причинен ущерб.

Безопасность цепочки поставок

Цепочка поставок – это все необходимое для создания продукта. Допустим, вы производите томатный соус для продажи в бакалейных лавках. Вам понадобятся томаты, специи, соль, (возможно) сахар и банка, в которую все это будет сложено. Наверное, вы также захотите наклеить на банку красивую этикетку, т. е. понадобится сама этикетка и клей. Чтобы наладить постоянное производство продукции, вы должны организовать регулярную поставку всех этих вещей, причем некоторые из них должны быть свежими (из гнилых томатов вкусного соуса не приготовишь).

Этикетка, банка и клей сами по себе являются продукцией, а значит, у каждого из них есть своя цепочка поставок. Если перечислить все без исключения элементы, а затем включить их в общий список, то получится *цепочка поставок* для вашего продукта.

Безопасность цепочки поставок означает защиту всей цепочки ингредиентов. Лично я даже не знаю, что входит в клей, но я точно знаю, что без него этикетка отвалится. Каждый элемент цепочки поставок необходимо физически защищать; нам не нужны ни битые банки, ни ядовитый соус.

В случае программного обеспечения это означает, что использование любой зависимости, включенной вами в свое приложение, должно быть безопасным. Если эти зависимости вызывают другие библиотеки, пакеты или программы, то и они должны быть безопасны! Сюда же относятся все инструменты, использованные при создании ПО, в том числе интегрированная система разработки (IDE), серверы для тестирования, ваша система управления версиями и т. д. В этой книге мы будем говорить о том, как со всем этим справляться, но пока достаточно просто понимать, что любой компонент, библиотека, каркас или пакет, которые вы используете для сборки своей программы, является частью цепочки поставок.

Безопасность через неясность

Эта концепция чаще всего применяется к сравнению открытого кода с проприетарным, который считается более безопасным. На мой взгляд, любая программа может содержать дефекты и просчеты, а ее безопасность определяется системами, которые призваны обнаруживать, исправлять и предотвращать проблемы. И тем не менее отсутствие исходного кода в публичном пространстве затрудняет решительному противнику его анализ на предмет уязвимостей. Если вы не раскрываете SSID (имя) своей домашней сети Wi-Fi, то можете помешать случайному наблюдателю вломиться в нее, подобрав пароль, но решительно настроенный и обладающий нужными знаниями противник ее все равно найдет. Хотя «безопасность через неясность» никогда не должна быть *единственным* средством обороны системы, она все же может

поспособствовать защите интеллектуальной собственности и использоваться в качестве одного из нескольких слоев более полного плана обеспечения безопасности.

Уменьшение поверхности атаки

Если с какой-то частью вашего приложения можно взаимодействовать, значит, ее можно и атаковать. Все, что может стать целью атаки, является частью *поверхности атаки*. Если нам удастся уменьшить поверхность атаки, то у злоумышленников будет меньше возможностей нанести ущерб, а значит, и меньше шансов на успех.

Говоря об уменьшении поверхности атаки, часто забывают упомянуть, что неиспользуемый код в конечном итоге обычно удаляется, но, пока он существует, нередко является небезопасным! Например, если в меню существует пункт, которым уже никто не пользуется, следует *удалить код из приложения*, а не просто закомментировать пункт меню. Со временем эту неиспользуемую возможность, скорее всего, перестанут тестировать и обновлять, потому что она не находится в списке вещей, с которым сверяются сотрудники отделов контроля качества и пентестеры. Про нее забывают, а значит, ее безопасность оказывается под вопросом. Есть и другой плюс – уменьшая размер кодовой базы, мы делаем ее более доступной для понимания и сопровождения.

Примечание Важный урок: шансы на то, что неиспользуемый код небезопасен, гораздо выше, поэтому мы должны удалять его из приложений.

Беспроблемная безопасность

Человек всегда найдет способ сделать свою работу, мы – существа решительные. Когда группа безопасности устанавливает какое-то средство, мешающее людям выполнять свои обязанности, даже самый технически безграмотный пользователь найдет способ его обойти – такова уж человеческая природа. Когда пользователи находят лазейку, часто оказывается, что такой способ использования системы менее безопасен. Лучше, когда группа безопасности делает все возможное для того, чтобы принимаемые ей защитные меры не затрудняли работу конечным пользователям. Спросить мнение пользователей и отрегулировать защитные механизмы так, чтобы пользователи могли нормально работать, – вот способ добиться желаемого (безопасной работы пользователей) с минимумом конфликтов. *Обратная связь имеет значение.*

Примечание Приведем несколько примеров лазеек, с помощью которых пользователи обходят неудобные или не позволяющие работать политики безопасности: пользователи записывают пароли на бумажках, которые приклеивают к монитору, потому что

«безопасники» заставляют их менять пароли раз в 90 дней; приложение запрещает делать снимки экрана, поэтому пользователи фотографируют экраны и делятся информацией с коллегами; сканирование кода плохо настроенным приложением статической проверки безопасности (static application security testing – SAST), которое дает больше ложных срабатываний, чем истинных, что приводит к отключению его в конвейере, из-за чего код попадает в систему управления версиями вообще непроверенным. Список можно продолжать до бесконечности.

После отказа закрыть, затем откатиться

Когда во время транзакции возникает ошибка, важно откатить ее и начать заново. Если мы что-то открыли (например, подключение к базе данных), то должны закрыть. Если мы предоставили доступ, то должны его отозвать. Мы поступаем так, чтобы избежать состояний гонки, атак с хронометражем и других потенциальных уязвимостей. Чтобы вернуться в начало и проделать все заново, на современном компьютере нужно очень мало усилий и времени. Но если оставить ошибку или отказ без внимания, то система может оказаться в неизвестном состоянии. И именно в таком состоянии нас чаще всего поджидают уязвимости: когда следующее действие нашей системы непредсказуемо. Поэтому всегда лучше после отказа закрывать, выполнять откат и начинать заново.

Комплаенс, законы и нормативные акты

Комплаенс: действие или процесс в соответствии с пожеланием, требованием, предложением или указанием, повиновение.

— Словарь Merriam-Webster

За пределами США и Евросоюза не так уж много законодательных актов, относящихся к кибербезопасности, и, насколько я поняла из поисков в интернете, ни один из них не касается исключительно безопасности программного обеспечения. Однако в США и в ЕС есть несколько законов, очень близко относящихся к ПО: Федеральный закон о модернизации информационной безопасности 2014 года (FISMA), закон о преемственности и подотчетности медицинского страхования 1996 года (HIPAA), закон об уведомлении в случае нарушения безопасности 2003 года (действует только в штате Калифорния), закон ЕС о кибербезопасности, директива о безопасности сетей и информационных систем (директива NIS), закон о цифровой операционной устойчивости (DORA) и Общий регламент ЕС по защите персональных данных 2018 года (GDPR)⁷.

Тем не менее в сфере информационной безопасности существует несколько самозванных регулирующих органов, которые выпускают регламенты и стан-

дарты, которым мы можем следовать. Хотя многие в отрасли согласны с тем, что безопасность и комплаенс не одно и то же, адресованное пользователям требование придерживаться передовых практик в области безопасности может оказаться весьма полезным для стимулирования организаций к разработке и поддержанию более надежной программы обеспечения безопасности.

Далее приведен неполный список требований комплаенса и стандартов в области информационной безопасности. Нет необходимости заучивать его наизусть, но знать о существовании стоит, это поможет вам лучше справиться со своей работой, если в будущем организация должна будет соответствовать каким-то из этих требований.

- Стандарт защиты информации в индустрии платежных карт (Payment Card Industry Data Security Standard – PCI DSS)⁸. Для компаний, которые обрабатывают платежи по кредитным картам и другие формы онлайн-платежей.
- Специальные публикации NIST. Для любых информационных систем NIST публикует различные стандарты.
- Регламент по кибербезопасности департамента финансовых услуг города Нью-Йорка (NY DFS). Для финансовых организаций в США.
- Закон о цифровой операционной устойчивости (Digital Operational Resilience Act – DORA) – это регламент Евросоюза, направленный на усиление информационной безопасности финансовых институтов, включая банки, страховые и инвестиционные компании. Этот регламент вступил в силу 17 января 2025 года. Цель DORA – гарантировать, что финансовый сектор Европы сможет сохранить устойчивость в случае значительных нарушений операционной деятельности.
- Стандарты Международной организации по стандартизации (ISO). Система, призванная помогать организациям в управлении всей программой информационной безопасности.
- Общий регламент по защите персональных данных (GDPR). Ориентированный на конфиденциальность регламент, имеющий целью защитить потребителей.
- Закон о преемственности и подотчетности медицинского страхования (HIPAA) и закон о применении медицинских информационных технологий в экономической деятельности и клинической практике (HITECH). Направлены на защиту персональных данных о здоровье граждан США.
- Федеральный закон об управлении информационной безопасностью (Federal Information Security Management Act – FISMA). Еще один американский регламент, призванный защитить государственные учреждения от компрометации данных⁹.
- Закон штата Калифорния о защите конфиденциальности потребителей (California Consumer Privacy Act – CCPA). Защита прав потребителей, проживающих в Калифорнии, на тайну частной жизни.

Каркасы безопасности

Несколько организаций создали каркасы безопасности, стремясь упростить определение, реализацию и сопровождение системных процессов. Ниже приведен неполный перечень каркасов, о которых вы, возможно, захотите узнать больше в путешествии по миру безопасного кодирования и безопасных приложений.

Неудивительно, что на первом месте стоят три каркаса, разработанных в рамках OWASP (Open Web Application Security Project – открытый проект по безопасности веб-приложений): каркас знаний о безопасности (Security Knowledge Framework), модель зрелости DevSecOps (DevSecOps Maturity Model – DSOMM) и модель зрелости контроля качества ПО (Software Assurance Maturity Model – SAMM).

Security Knowledge Framework¹⁰ – свободное ПО с открытым исходным кодом, в котором стандарт проверки безопасности приложения OWASP (Application Security Verification Standard – ASVS) используется в качестве основы, чтобы помочь разработчикам в написании безопасных приложений. В нем приводятся примеры безопасного кодирования на нескольких языках, чтобы программист мог найти самый безопасный способ сделать то, что ему нужно.

OWASP DSOMM – каркас, иллюстрирующий различные меры безопасности, которые особенно легко применить, если используется методология разработки ПО DevOps; эти меры упорядочены по важности.

OWASP SAMM¹¹ – каркас, помогающий оценить степень зрелости безопасного жизненного цикла разработки системы (secure system development life cycle – S-SDLC). OWASP SAMM не зависит от технологии или языка программирования; он просто помогает улучшить ваш SDLC и другие процессы, чтобы конечное ПО гарантированно было безопасным.

OWASP предлагает еще несколько каркасов, в том числе каркас тестирования веб-приложений. Он также включает весьма обширную документацию по безопасности приложений, свободно доступную на сайте <https://owasp.org>. Самый известный ресурс OWASP называется Top Ten Security Risks to Web Applications (Десять наибольших рисков для веб-приложений) (поклонники называют его просто «the Top Ten»), он расположен по адресу <https://owasp.org/www-project-top-ten/>.

Примечание Советы разработчикам ПО обязательно должны быть своевременными, конкретными и точными. Когда-то давно я работала в группе безопасности, насчитывавшей больше 20 штатных сотрудников, в обязанности которых входила в основном оценка рисков по контрольным спискам. Когда разработчик спрашивал совета по своему приложению, «безопасники» давали ответы, повергавшие меня в шок. Архитектор программы спрашивал, нельзя ли уже сейчас получить список, сверяясь с которым он сможет в конце SDLC проверить безопасность приложения, чтобы он мог правильно конструировать его с самого начала. Ему отвечали, что «это будет сбивать с толку» и в просьбе отказывали; видимо, критерии безопасности были большим секретом.

Один разработчик спросил, как безопасно управлять идентификатором сеанса в своем приложении; ему отправили ссылку на сайт NIST с указанием прочитать все материалы (больше 1000 страниц) и таким образом научиться строить безопасные приложения. Я пыталась объяснить группе безопасности, что это, по сути дела, то же самое, что отправить картинку со средним пальцем разработчикам, задававшим вопросы, – бесполезно и оскорбительно. Никто не согласился со мной. Уволилась я оттуда довольно быстро – это место мне не подходило.

Следующим в списке является NIST, который выпускает много каркасов наряду с другими весьма полезными публикациями; NIST – потрясающий онлайн-ресурс. Специальная публикация NIST (SP) 800-53¹² – каркас безопасности приложений, в котором описаны рекомендуемые методы управления рисками для программного обеспечения. Основной упор в этой публикации сделан на тестировании ПО и рекомендованных для этой цели инструментах. В самые последние обновления включены новые инструментальные средства, такие как Interactive Application Security Testing (IAST – интерактивное тестирование безопасности приложений) и Run-time Application Self Protection (RASP – самозащита приложения на этапе выполнения).

Тот же NIST выпустил каркас кибербезопасности (Cybersecurity Framework – CSF), где на титульной странице заявлено, что он предназначен для улучшения критической инфраструктуры, но на самом деле он применим к большинству ИТ-систем. В этом каркасе упор сделан на управление рисками и их оценку, а не на специфику безопасного кодирования или проектирования.

Еще одна группа, о которой следует знать, – Центр интернет-безопасности (Center for Internet Security – CIS), который составил список критических практик безопасности (Critical Security Controls)¹³. Ранее известный под названием «SANS Top 20», этот список – самое полезное приобретение в области кибербезопасности для крупной организации. Он включает рекомендации для обеспечения безопасности всей организации, а не только отделов ИТ и разработки ПО. Эта группа также регулярно выпускает документы «CIS Benchmarks» – не зависящие от поставщика руководства по конфигурированию безопасности, а равно предварительно укрепленные образы инфраструктуры. CIS отдает приоритет инфраструктуре, а не заказным приложениям.

Следующие две организации близки к комплаенсу, но технически являются независимыми. Впрочем, обычно их рекомендации внедряют, чтобы доказать отработанность мер безопасности сторонним организациям: заказчикам, инвесторам и деловым партнерам.

Сертификаты ISO 27001 и ISO 27002 Международной организации по стандартизации (ISO) считаются международным стандартом проверки программы кибербезопасности¹⁴. Быть может, это и не каркас как таковой, но в этих документах имеются пошаговые руководства по приведению дел в порядок с точки зрения управления рисками.

Документ Service Organization Control (SOC) Type 2, или SOC2, – это список из 60 элементов безопасного управления клиентскими данными¹⁵. Внедрить указанные в этом каркасе элементы трудно, но когда организация может утверждать, что «соответствует SOC 2», это является для нее и ее продукции отличной рекомендацией.

Хотя, конечно, существуют и другие каркасы, относящиеся к кибербезопасности, ни один из них не связан с безопасностью ПО. Таким образом, на момент написания книги OWASP и NIST занимают лидирующие позиции в предоставлении ориентированных на ПО каркасов безопасности.

Учимся на ошибках и делимся уроками

Хотя эта тема, строго говоря, не относится исключительно к безопасности, многие проблемы кибербезопасности проистекают от недостатка знаний, информации или обучения. Если бы каждая группа безопасности поставила себе целью широко распространять информацию в организации, после того как исправлен дефект, выявлен просчет или проведен анализ по результатам расследования инцидента безопасности, мы, как отрасль, наверное, реже становились бы свидетелями провалов. Коль скоро проблемы, такие как кража данных и т. п., возникают столь часто, мы должны усердно работать, чтобы избежать повторения ошибок.

Обратная совместимость (и сопутствующие ей потенциальные риски)

Как всякий, кто предпочитает использовать свои персональные устройства как можно дольше, я ценю обратную совместимость и ту экономию, которую она предлагает пользователю и его банковскому счету. Вместе с тем дело не обходится без рисков. Старение программы происходит очень неприятно: чем дольше она находится на виду, тем больше потенциального злонамеренного внимания привлекает. Иногда уязвимости обнаруживаются немедленно, а иногда для этого требуется время, и чем больше времени предоставлено, тем вероятнее, что уязвимости будут найдены. По мере старения программ, оборудования и их зависимостей что-то перестает поддерживаться, и это тоже влечет за собой негативные последствия (прекращается тестирование, не выпускаются обновления безопасности и т. д.). Решая, сколько времени вы собираетесь поддерживать обратную совместимость для своего продукта, принимайте во внимание безопасность и то, сможете ли вы и дальше гарантировать безопасную эксплуатацию конечному пользователю.

Моделирование угроз

Под моделированием угроз понимается процесс идентификации угроз системе, которую вам поручено создать или сопровождать. После выявления потенциальных угроз анализируется, нужно ли принимать меры по их купированию (некоторые угрозы незначительны или крайне маловероятны, поэтому их нет необходимости устранять, чтобы поддержать желаемый уровень безопасности). В процессе анализа кода и тестирования безопасности следует искать проявления ваших моделей угроз и попытаться доказать, что система адекватно защищена от них.

Моделирование угроз может оказаться невероятно формальным видом деятельности, требующим объемной документации, процессов и времени. Но может быть и неформальным, например мозговым штурмом с записями. Независимо от способа исполнения эта процедура почти всегда открывает глаза, высвечивая ряд потенциальных проблем, о которых вы раньше не задумывались. Мы еще вернемся к этой теме в третьей части книги.

ПЛАНИРОВАНИЕ КОНФИДЕНЦИАЛЬНОСТИ

Много лет назад, когда Алиса была гораздо моложе, она мечтала о карьере кинозвезды. Она пробовалась и получала роли в нескольких фильмах небольших независимых кинокомпаний, у нее даже образовалось маленькое сообщество поклонников. Продолжалось это недолго, но она наслаждалась, сколько могла. Во время этого краткого ралли более зрелая и опытная актриса отвела ее в сторонку и объяснила, что теперь, когда она «на виду у публики», ее модель угроз изменилась. «Что такое модель угроз?» – спросила Алиса. Женщина ответила, что по большей части она будет получать благожелательное внимание, но что любой уровень известности привлекает также негативное внимание, поэтому очень важно защищать свою персональную информацию: адрес проживания, дату рождения, номер социального страхования и вообще все, что другие могут использовать, чтобы найти и выдать себя за нее. Алиса восприняла этот совет серьезно и, став старше, очень внимательно следила за тем, какими фотографиями и информацией делится в сети. Иметь поклонников – это здорово, но она вовсе не хотела, чтобы кто-то из них неожиданно возник у нее на пороге без приглашения!

Трудности обновления

Применение обновлений к телефону или установленным на компьютере программам называется *исправлением*. Когда мы обновляем свою операционную систему, могут возникать проблемы в установленных программах, требующие обновить старые версии, которые в противном случае перестают работать. Если такая программа имеет зависимость, а та – свои собственные зависимости, то, возможно, придется выполнить несколько раундов обновления – это сложно, трудно и долго. Теперь представьте себе

обновление инфраструктуры языка программирования для своей заказной программы. Иногда это означает, что нужно выгрузить некоторые зависимости или больше не вызывать некоторые функции; изредка (обычно когда вы отстали на несколько версий) может потребоваться полное переписывание приложения! После завершения этой работы придется провести обширное тестирование, чтобы убедиться, что приложение по-прежнему работает как задумано и что не было внесено новых ошибок (связанных с безопасностью и не только). Имейте это в виду, когда слышите от кого-то «Просто обнови ее»; обычно это куда больше, чем кажется на первый взгляд.

Работая с компаниями над их программами обеспечения безопасности приложений, я чаще всего слышу такую жалобу: «Разработчики не устраняют уязвимости, о которых мы сообщаем». Если копнуть чуть глубже, то обычно обнаруживается одна из двух проблем: либо нагрузка на разработчиков непомерно велика и у них просто нет времени на исправление ошибок, либо процесс выпуска новых версий мучителен, трудоемок и неэффективен. Сколько раз я говорила клиентам: «Главная угроза безопасности вашего ПО – неспособность выпускать исправления за разумное время. Мы должны отладить этот процесс как можно быстрее». Любые действия, которые вы можете предпринять для оптимизации или автоматизации этого процесса, помогут вашей организации реагировать на проблемы безопасности быстрее и станут хорошим вложением времени и денег.

НЕРАБОТАЮЩАЯ КОНФИДЕНЦИАЛЬНОСТЬ

Боб всегда любил свой MacBook и операционную систему Apple MacOS, но вышедшее в 2019 году обновление Catalina MacOS принесло с собой кучу новых «особенностей» конфиденциальности, из-за которых все поломалось! На следующий день у него была запланирована презентация на работе, но чтобы соответствующая программа могла получить доступ к его папкам, приходилось возиться около 10 мин; сказать, что он расстроился, – ничего не сказать! Тем вечером он попытался сделать видео на своем персональном компьютере, и Catalina нанесла еще один удар! Ему пришлось ставить совершенно новую программу видеозаписи; Open Broadcaster Software (OBS) вообще отказалась работать. Несколько недель ушло на то, чтобы привести персональный компьютер в нормальное состояние, к которому он привык. Боб испытал сильнейшее сочувствие к людям, зарабатывающим на жизнь установкой обновлений!

Повторное тестирование исправлений новых дефектов безопасности

Исправляя дефект, мы изменяем код или пишем новый. И очень может стать, что по ходу дела внесем новые дефекты. Среди которых могут оказаться и дефекты безопасности. Никто не делает этого сознательно, но рано или

поздно с каждым случается. Нет смысла ругать за допущенные ошибки, все мы люди, все мы человеки. Но после любых изменений мы всегда обязаны заново протестировать код, чтобы по возможности не оказаться в такой ситуации. В идеале повторное тестирование должно быть автоматизировано. С появлением технологии DevOps с ее конвейерами непрерывной интеграции и непрерывной доставки и развертывания (CI/CD) сделать это стало проще, чем когда-либо раньше. Более того, сегодня существуют сотни инструментов, который можно очень быстро настроить для повторного тестирования своего приложения, и среди них имеются свободные и с открытым исходным кодом. Читайте дальше, и вы узнаете, как тестировать свои приложения, не будучи экспертом по безопасности.

Примечание KISS – Keep It Simple, Silly (будь проще, глупышка). Сложность – враг безопасности. Система лучше работает, когда она простая. Это относится ко всему, включая разрабатываемые вами приложения. Чем сложнее код, тем проще надеть в нем ошибок и тем труднее его сопровождать. Уменьшите сложность – и написанное вами программное обеспечение станет безопаснее.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Почему важно заново тестировать код после исправления ошибки?
2. Почему важно (хотя бы пытаться) поддерживать в актуальном состоянии каркасы программирования, с помощью которых мы создаем заказное ПО?
3. Какой каркас безопасности кажется вам наиболее полезным и почему?
4. Какова ваша личная модель угроз? (Примечание: на этот вопрос нет неправильных ответов. Его смысл в том, чтобы вы идентифицировали потенциальные угрозы в своей жизни и подумали, можно ли их купировать.)
5. Приведите один пример «предполагаемого доверия». Необязательно связанного с ИТ.
6. Подумайте о какой-нибудь одной новой вещи, связанной с безопасностью, которая, как вам кажется, могла бы принести пользу вашей команде на работе, в школе или в личной жизни. Поделитесь этой информацией с подходящими людьми. Она может быть взята из этой книги или из любого другого места – главное, чтобы, на ваш взгляд, она могла кому-то помочь чувствовать себя в большей безопасности.
7. Приведите пример принципа наименьших привилегий и объясните, как бы вы его применили. Опишите потенциальные последствия в случае неприменения принципа наименьших привилегий к данной ситуации или системе.

8. Приведите пример потенциально безопасного умолчания или вымощенной дороги, который вы хотели бы внедрить.
9. Приведите пример безопасности через неясность.
10. Приведите примеры ситуаций, в которых «отказ безопасен», «следует закрывать в случае отказа» или «откатывать транзакцию, и объясните, как это защитило бы пользователей системы.

https://t.me/IT_Portal

В начале пути

Безопасное кодирование сродни мышечной памяти: чем чаще упражняешься, тем легче становится. У вас входит в привычку проверять все поступающие в приложение данные. Вы всегда используете только параметризованные запросы. Вы просите у кого-то из «безопасников» помощи в классификации и разметке данных, пока в один прекрасный день не осознаете, что можете сделать и самостоятельно.

Начиная с этой главы, мы будем говорить о рекомендациях по безопасному кодированию, которые относятся к большинству языков программирования и каркасов. Информация в этой главе – начало нашего путешествия по миру безопасного кодирования.

Применяйте безопасный жизненный цикл разработки ПО

Если бы я могла дать только один совет вашей компании, команде или лично вам, то посоветовала бы применять *безопасный* жизненный цикл разработки системы. Что я имею в виду? Что добавлять действия по обеспечению безопасности нужно как часть процессов, которые вы соблюдаете при разработке ПО. Это могло бы означать, что, приступая к проекту, следует составить список требований, включающий требования к безопасности, а затем на этапе проектирования заняться моделированием угроз совместно с ответственным за безопасность приложений и менеджером по продукту. В стандартные процессы можно включить целый ряд действий, которые в итоге приведут к выпуску более безопасной программы. В части III мы поговорим об этом подробнее.

ПЕРВЫЙ ДЛЯ БОБА БЕЗОПАСНЫЙ ЖИЗНЕННЫЙ ЦИКЛ РАЗРАБОТКИ СИСТЕМЫ (SDLC)

Боб помнит тот первый раз, когда группа безопасности захотела «добавить кое-что» в его проект. На установочном совещании с места поднялась женщина и сказала: «Здравствуйте, меня зовут Алеша, и я буду вашим ресурсом по безопасности на протяжении всего проекта. Я помогу вам выполнить все требования, которые мы предъявляем к проекту. Если вам понадобится помощь или возникнут вопросы, обращайтесь ко мне». Боб был поражен! Обычно группа безопасности держала дистанцию, но эта дама вела себя иначе. И она сдержала слово! На протяжении всего проекта она постоянно была рядом, проверяла и помогала. Иногда Боб задавал вопросы, и если она не знала ответа, то не стеснялась в этом признаться и тогда организовывала мозговой штурм, на котором они вместе исследовали проблему. Она никогда не отказывала Бобу и его команде в помощи. Для Боба это стало другим и *лучшим* опытом. Впоследствии Боб узнал, что такая тактика имеет название «модель партнерства», и это стало его излюбленным способом работы с группой безопасности.

Используйте современный каркас и все имеющиеся в нем средства безопасности

Следующий совет – по возможности использовать проверенный временем, современный, считающийся отраслевым стандартом каркас программирования. Иногда у нас нет выбора, и мы вынуждены пользоваться тем же, чем пользуется кто-то другой в организации. Но если выбор есть, то выбирайте каркас тщательно и, если возможно, остановитесь на том, что имеет наилучшую репутацию в плане безопасности.

Работая с каркасом программирования, всегда старайтесь использовать встроенные в него средства безопасности, а не изобретать собственные. Причина в том, что каркасы хорошо протестированы – и не только их авторами, но и программистами, которые используют их с утра до вечера. При таком обширном тестировании и активном использовании они с большей вероятностью будут безопасны, чем код, написанный одним программистом за своим рабочим столом.

РАСПОЛЗАНИЕ РАМОК ПРОЕКТА С ТОЧКИ ЗРЕНИЯ АЛИСЫ

В колледже Алиса посещала курс программирования и для сдачи экзамена должна была выполнить большой проект вместе с одноклассниками. Один из них решил «написать собственный парсер XML». Он сказал, что так будет проще, чем использовать уже существующий. Оказалось, что *не* проще. На самом деле работа над парсером так и не была доведена до конца, и остальной команде пришлось найти другой и включить его в проект в последний момент, когда сроки уже поджимали. Целых четыре месяца он твердил, что все «почти готово» и что *его код* «куда лучше» парсера XML, выбранного командой. Алиса не понимала: зачем он так поступил?

Двигаясь по карьерной лестнице к должности руководителя, каковым и является сейчас, она снова и снова сталкивалась с этим явлением: программисты думают, что справятся лучше, чем целая команда преданных своему делу профессионалов, создавших продукт, который разработчик хотел написать сам, вместо того чтобы купить. Они будут раз за разом изобретать колесо, которое обойдется в целое состояние! Сопровождение доморощенных приложений вообще никогда не учитывалось в смете, а уж сколько было споров с руководством!

Проверка входных данных

Теперь перейдем к совету по *кодированию*. Когда в наше приложение поступают данные, мы должны убедиться, что это именно то, чего мы ожидаем, и что использовать их безопасно; это называется *проверкой входа*. Входные данные могут поступать откуда угодно: от пользователя, который ввел их в поле ввода, от API или из базы данных, из загруженного на сервер файла, из кем-то измененного скрытого поля или из введенного вручную параметра URL. Если у пользователя есть возможность изменить входные данные, то вы обязаны предполагать, что они могли быть изменены. Все входные данные могут представлять опасность для приложения, намеренно или нет. Может, это пользователь, случайно опершийся на клавиатуру, а может, и злоумышленник с дурными намерениями. Поскольку мы не умеем ни читать мысли, ни предвидеть будущее, самый простой способ защититься от таких ситуаций – всегда проверять безопасность данных, прежде чем делать с ними что-то в программе.

Для этого мы хотим для начала проверить, что на вход подано *то, чего мы ожидаем*. Делать это нужно в надежной системе (на сервере, а не в браузере, где работает JavaScript-код) и при этом использовать белый список (значений, образцов, форматов, которые разрешены и которых мы ожидаем), а не черный список (все, что, как мы опасаемся, может причинить вред приложению). Заметим, что мы *также* можем проверить вход во фронтальном JavaScript-коде, но не *вместо* проверки на сервере, а лишь в дополнение к ней.

Давайте поговорим об этом чуть подробнее, потому что если вы усвоите эту мысль, то ваши приложения станут значительно безопаснее. Правильная и тщательная проверка входа может исключить множество уязвимостей в любой программе.

Прежде всего что вы ожидаете получить? Допустим, дату в формате MM/DD/YYYY.

В Visual Basic for Applications мы можем проверить, является ли нечто датой, воспользовавшись функцией `IsDate`¹⁶:

```
Dim MyVar, MyCheck
... Код, получающий дату от пользователя ...
MyCheck = IsDate(MyVar)
' возвращает true, если это действительно дата, иначе false
```

Затем мы, вероятно, захотим проверить, что дата находится в разумном диапазоне. В предположении, что мы запрашиваем у человека день рождения, ожидается, что дата будет раньше сегодняшней, но не будет отстоять от нее больше, чем на 150 лет.

Совет Межсайтовый скриптинг, часто называемый XSS, – очень опасная и весьма распространенная уязвимость в веб-приложениях. Она возникает, когда браузер удается обманом заставить интерпретировать код противника, как если бы он был частью приложения, после чего браузер атакует компьютер конечного пользователя. Ограничения JavaScript – единственное, что стоит на пути ужасных вещей, которые могут стать следствием этой уязвимости, например: установка клавиатурного шпиона на компьютер пользователя, включение его веб-камеры и запись на нее или установка вредоносной программы. Об этой и многих других уязвимостях мы будем говорить в главе 8.

В ASP.NET Core и C# код¹⁷, эквивалентный приведенному выше, выглядел бы так:

```
DateTime MyDate = new DateTime();
// присваивает значение по умолчанию 01/01/0001 00:00:00

MyDate = ... код получения даты от пользователя ... ;

If (MyDate.TryParse(text, out dateTime)) // это дата19
{
    if (MyDate >= DateTime.Today) // сегодня или позже
    {
        Console.WriteLine("Invalid date of birth. Please try again.");
    }
    else if (MyDate <= DateTime.Now.AddYears(-150);)
    { // Минус 150 лет от сегодняшней даты18
        Console.WriteLine("Invalid date of birth. Please try again.");
    }
    else
    {
        // остальной код приложения, дата хорошая
    }
}
else
{
    Console.WriteLine("Invalid date of birth. Please try again.")
}
```

Совет Мы всегда проверяем и кодируем, но подвергаем санированию и экранированию, только если требуется принять потенциально опасные символы.

Мы проверили, что это дата и что она раньше сегодняшней, но не больше чем на 150 лет. Тем самым мы проверили, что получили *то, что ожидали*, а значит, использовать это безопасно.

В главах 6 и 7 мы будем рассматривать популярные языки программирования и каркасы и тогда приведем дополнительные примеры и расскажем, на что обращать внимание.

Обычно мы можем просто проверить данные и либо отклонить их (если проверка не прошла), либо принять. К сожалению, иногда жизнь оказывается сложнее, и проверки недостаточно.

Если вы *должны* принимать специальные символы, то нужно быть внимательнее. К числу других мер защиты относятся санирование, экранирование и кодирование (именно в таком порядке). Отметим, что в веб-приложениях проверка и кодирование обязательны.

Совет Оригинал этой книги написан на английском языке, но мир-то пользуется и другими. Решая, какие символы принимать, следите за тем, чтобы не заблокировать алфавиты, которые могут потребоваться пользователям.

- **Санирование** – это удаление или замена проблемных символов. Примерами проблемных символов в JavaScript являются < и >. Причина в том, что они используются в начале и в конце тега <script>, которым противник мог бы попытаться воспользоваться для проведения XSS-атаки (межсайтового скриптинга). Их можно просто удалить, а если вы хотите знать, что они присутствовали, то замените все открывающие и закрывающие угловые скобки чем-то еще (например, буквой или безопасным специальным символом, скажем тильдой).
- **Экранирование** – это добавление символа обратной косой черты (\) перед специальными символами до сохранения. Это лишает следующий символ специального смысла. Если впоследствии информацию подать на вход интерпретатору, то он не придаст экранированным символам никакой специальной семантики.
- **Кодирование** похоже на экранирование тем, что лишает символы специального смысла, но эта операция продлевается со всем текстом, подвергнутым кодированию, а не только с одним символом. Однако этим различием дело не исчерпывается. Читайте дальше!

В идеале следует проверять как можно больше входных данных, а к санированию и экранированию прибегать, только если необходимо принять специальные или потенциально опасные символы. Но кодировать информацию, отображаемую на экране веб-приложения, нужно всегда.

Рассмотрим другой пример. Представьте, что вы пишете на PHP и хотите проверить какие-то данные, введенные в поисковое поле. Получить-то поисковый запрос от пользователя просто, но как понять, что это именно то, что вы ожидаете?

Следующий код – отредактированная версия кода с сайта w3Schools²⁰:

```
<?php
// определить переменные и инициализировать их пустыми значениями
$name = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = validate_input($_POST["name"]);
    //получить "name" из формы, отправить на проверку
}
```



```
function validate_input($data) {  
    $data = trim($data);           // обрезать пробелы и табуляторы  
    $data = stripslashes($data);   // удалить знаки обратной косой черты  
    $data = htmlspecialchars($data); // преобразовать специальные символы  
    return $data;                 // вернуть безопасные данные  
}  
?>
```

Несколько мыслей о входных данных. Любые данные, подаваемые на вход приложения или API, могут быть вектором атаки. Часто ошибки, допущенные при проверке данных, связаны с тем, что разработчик не воспринимает их как вход и пропускает при проверке.

Например, допустим, что вы пользуетесь хранимой процедурой, чтобы получить данные из базы; они ведь не могут быть вредоносными, правда? Не правда. К сожалению, если кто-то другой не произвел надлежащий контроль при помещении полученных от пользователя данных в базу, там могла быть сохранена XSS-атака, только и ждущая, что вы ее извлечете и попадетесь на крючок.

Мы также хотим быть уверены, что извлеченные данные правильного типа (проверяйте это, не предполагая, что тип именно такой, как вам говорят) и что диапазон полученных данных логичный и допустимый; убедитесь, что значения ожидаемы и разумны. Нам не нужен человек в возрасте 500 или –5 лет.

Еще одно место, куда разработчики часто забывают поместить надлежащие проверки безопасности (проверку входа, аутентификацию, авторизацию и т. д.), – API, который вызывает другой API или бессерверное приложение. Зачастую разработчик предполагает, что данные, поступающие от им же написанного API или приложения, можно безопасно использовать. Но что, если приложение было скомпрометировано? Что, если API был плохо запрограммирован? Что, если кто-то подменил ваше приложение своим собственным (маловероятно, но все же возможно)? Есть много ситуаций, когда подразумеваемое доверие может стать причиной уязвимостей в программе. Памятуя об этом, всегда проверяйте данные перед использованием, откуда бы они ни были получены.

Кодирование вывода

При выводе чего-то на экран программист выбирает формат отображения. Обычно выбор зависит от визуальной привлекательности, удобства для пользователя (UX) и технического стека (HTML? JavaScript? Windows-приложение? Серверная система, имеющая только текстовую консоль?). Но при работе с веб-приложениями, работающими в браузере, беспокоиться нужно не только о внешнем виде.

Браузеры выполняют ваше веб-приложение, как Windows исполняет локальные Windows-приложения, Linux – приложения, написанные для Linux,

iOS – приложения для Apple iPhone, Android – приложения для Android, а MacOS – приложения для Mac. Разница между веб-приложениями и большинством других видов ПО в том, что 1) они в основном просматриваются через интернет или еще какую-то сеть (а не запускаются в том операционной системе, которая работает на вашем устройстве), 2) они интерпретируются сразу, а не сохраняются в виде исполняемого файла, который затем устанавливается в операционную систему и ей исполняется, и 3) они нуждаются в браузере (именно в него встроен интерпретатор!). Веб-приложение может исполняться разными браузерами, а между ними имеются, пусть и небольшие, функциональные различия.

Исполняя веб-приложение, браузер интерпретирует его динамически. Он читает всю HTML-разметку, все CSS-таблицы стилей и весь JavaScript-код (не путайте с Java), интерпретирует их и организует ваше взаимодействие с приложением. Код интерпретируется по мере того, как пользователи выполняют различные задачи, причем интерпретация производится в различных контекстах: HTML, JavaScript или CSS.

Иногда в процессе интерпретации браузер запутывается и по ошибке принимает введенные пользователем данные за код. Данные обрабатываются, сохраняются, изменяются и используются для принятия решений, но они никогда не должны исполняться или интерпретироваться, как если бы являлись частью кода приложения. Если входные данные ошибочно приняты за код и интерпретированы браузером, то мы имеем межсайтовый скриптинг (XSS).

Если вы слышали о XSS, то и замечательно! Это значит, что вы уже знаете по меньшей мере об одном из распространенных рисков для веб-приложений. Если же нет, то знайте, что XSS считается опасностью для веб-приложений (для всех), и случается это куда чаще, чем хотелось бы любому специалисту по безопасности. Но если мы экранируем данные, прежде чем вывести их на экран, то в большинстве случаев эту проблему удастся избежать.

Чтобы предотвратить XSS, лучше всего проверять входные данные, кодировать выходные, использовать заголовок Content-Security-Policy и в идеале еще и проводить анализ кода и тестирование безопасности, чтобы убедиться, что все указанные выше меры предосторожности внедрены правильно. Но если у вас есть время, деньги и (или) влияние только для одной защитной меры с целью предотвращения XSS, то пусть это будет кодирование вывода.

При этом важно производить кодирование с учетом текущего контекста. Если вы находитесь внутри HTML-тега, то следует выполнять HTML-кодирование. Если вы внутри CSS, то это и будет вашим контекстом. Бывает и так, что один тег находится внутри другого, и тогда приходится кодировать дважды (я понимаю, что писать такой код не самое приятное и интересное дело, но поверьте мне – предотвращение XSS поможет вам избежать огромных неприятностей в будущем).

Примечание Всегда пользуйтесь средствами кодирования, предоставляемыми каркасом, а не пишите их самостоятельно.

Примеры кодирования вывода

Большинство моих примеров кодирования против XSS взято из шпаргалки по предотвращению XSS²¹, входящей в состав проекта Open Worldwide Application Security Project (OWASP). Проект Cheat Sheets Project – первый ресурс, с которого я всегда начинаю, когда привожу примеры или даю советы. Я горячо рекомендую всю серию Cheat Sheet и преклоняюсь перед людьми, которые сопровождают этот кладезь отличных советов. Дополнительные сведения см. по адресу https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.

Контекст HTML

Допустим, вы хотите вставить переменную между двумя HTML-тегами. Это контекст HTML и, значит, кодировать нужно для HTML (т. е. заменить литеры символьными мнемониками, не имеющими специального смысла). Вы могли бы написать код на JavaScript, который вручную заменит каждую литеру < эквивалентной ей HTML-кодированной мнемоникой <. Можно написать целый скрипт, который будет заменять каждую литеру. Но большинство разработчиков не пишут HTML-разметку с нуля, а пользуются .NET, PHP или чем-то подобным. Поэтому посмотрим, как в этой ситуации сгенерировать безопасный код на HTML и JavaScript.

Совет Если вы помещаете переменную между HTML, CSS или JavaScript-тегами, то обязаны ее проверить, возможно, санировать или экранировать, а затем закодировать. Эта ситуация чревата рисками, действуйте осторожно.

.NET

В каркасах .NET и .NET Core имеется удобная функция `HtmlEncode`, которая делает именно то, что нам нужно²²:

```
public static string HtmlEncode( string s )
```

PHP

В PHP вывод нужно кодировать вручную, как в следующем примере кодирования HTML:

```
string htmlentities ( string $string [, int $flags = ENT_COMPAT |
ENT_HTML401 [, string $encoding = ini_get("default_charset") [, bool
$double_encode = true ]]] )
```

Контекст JavaScript

JavaScript-код при кодировании превращается в Юникод, т. е. < переходит в \u003c²³.

.NET

По счастью, в каркасе .NET имеется функция `HttpUtility.JavaScriptStringEncode`, которая выполняет необходимое кодирование. Спасибо, MSFT!

RНР

К сожалению, в RНР нет готового решения, приходится писать код самостоятельно. Справиться с нелегкой задачей кодирования проще всего, используя только те каркасы, которые уже умеют это делать. Значительная часть разработчиков пользуется такими JavaScript-каркасами, как Vue.js, Angular или React, которые берут эту задачу на себя. Они автоматически кодируют вывод, производят экранирование и выполняют множество других действий, связанных с безопасностью, так что вам об этом и думать не нужно. По возможности старайтесь не писать код с нуля, а пользоваться современными каркасами программирования.

Параметризованные запросы и ORM

Когда я только начинала заниматься разработкой ПО, мне и в голову не приходило рассматривать базу данных как интерпретатор моего кода. Когда я с гордостью писала свои первые строки SQL-кода, я и не думала, что их можно использовать для причинения зла, а не только для выборки записей, предъявляемых пользователям. Я выступала в роли программиста, и все мое внимание было занято решением задач в интересах своих клиентов – созданием программ, которые обслуживали их и их потребности. Тогда я еще не боялась, что мои запросы будут использованы против меня, моего приложения и пользователей моих приложений. Как же я ошибалась!

Когда я работала на канадское федеральное правительство, у нас был этический хакер (он же пентестер), который проводил презентацию для моей группы разработки. Он показал экран входа в систему, который выглядел так же, как в одном из наших веб-приложений. И сказал: «Сейчас я за минуту войду в это приложение, не вводя пароля. Да и столько-то времени нужно лишь потому, что я еще и говорю». Я подумала: «Ну, да, болтай...», но он это сделал!

Я хочу рассказать, как он действовал. Сначала он сказал: «Как вы думаете, что стоит за этим экраном? Запрос, примерно такой:

```
cmd = "select * from table users where username like '%" + usernamevar + "%'";
```

Теперь представьте, что я ввел имя пользователя, 'SheHacksPurple'. До SQL-сервера оно дойдет в таком виде:

```
select * from table users where username like '%SheHacksPurple%'
```

Сервер вернет все имена пользователей, содержащие строку 'SheHacks-Purple'. Но что, если я помешу в `username` не имя, а *код*? Что-нибудь такое, что всегда возвращает `true`? Ну например:

```
username = " SheHacksPurple ' or 1=1 --";
```

Тогда отправленный SQL-серверу код будет выглядеть так:

```
select * from table users where username like '%SheHacksPurple' or 1=1 --'
```

Эта команда ищет записи, в которых имя пользователя равно 'SheHacks-Purple', или... любую другую запись в таблице базы данных! Потому что условие `1=1` всегда истинно. А это означает, что ваш код, проверяющий, существует ли имя пользователя в базе данных, будет возвращать `true` всегда, даже если такого пользователя нет».

Показав нам это, он объяснил, что то же самое можно было бы сделать для поля пароля, – и вот мы вошли приложение. Затем он рассказал, что этот тип атак называется внедрением SQL-кода (SQLi) и что существует несколько типов внедрения: внедрение команд, внедрение LDAP-кода и даже внедрение NOSQL-кода (для языков запросов, отличных от SQL). Он сказал, что это уязвимость номер один в веб-приложениях и что прямо сейчас она существует на самых разных сайтах по всему интернету! Я была ошарашена. Этот урок стал первым шагом, приведшим к решению сменить карьеру разработчика ПО на профессионала по безопасности.

Примечание Добавление в конец двух дефисов (--) означает, что SQL-команда кончилась и все последующие символы можно игнорировать. Этот прием часто используется в атаках такого типа (SQLi), чтобы упростить атакующий код. Зачем стараться сделать код всего запроса синтаксически корректным, если можно просто отрезать кусок после первого же уязвимого параметра?

Мы могли бы предотвратить эту атаку, воспользовавшись сочетанием проверки входа (первый уровень защиты) и параметризованных запросов (второй уровень) вместо подстановки строки в SQL или какой-то другой язык запросов к базе данных, в том числе NOSQL.

Возможно, вы думаете: «Ну ничего страшного. Я-то могу написать свой код входа в систему очень аккуратно. У меня все будет хорошо». К сожалению, внедрение кода – очень опасная уязвимость, которую не следует недооценивать. Одно лишь внедрение SQL-кода – и противник сможет украсть все ваши данные, изменить или удалить данные и даже (в комбинации с другими уязвимостями) захватить контроль над вашим SQL-сервером, так что окажется внутри сети, за брандмауэром. Уязвимости, связанные с внедрением кода, позволяют организовать чрезвычайно опасные атаки против приложений, операционных систем и сетей.

ПЕРВЫЙ ОПЫТ БОБА РАБОТЫ С ХРАНИМЫМИ ПРОЦЕДУРАМИ

Боб помнит, как группа безопасности запретила его группе разработки использовать «встраиваемый SQL-код» и велела, начиная с сегодняшнего дня, перейти на хранимые процедуры. Кто-то должен был объяснить ему, что под *встраиваемым SQL-кодом* понимается тот факт, что он сам создает SQL-команду (в которую включены полученные от пользователя данные), а затем отправляет ее серверу базы данных для выполнения. Группа безопасности хотела, чтобы он вместо этого использовал параметризованные запросы. Это значит, что он должен вызвать из своего приложения функцию, хранящуюся в базе данных, и передать ей данные в виде параметра. Боб не понимал, в чем разница: *кому какое дело, где именно написан запрос?* «Безопасники» растолковали ему, что, когда данные передаются хранимой процедуре в виде параметра, сервер лишает их всех суперсил и гарантирует, что данные остаются только данными, так что атаки типа внедрения невозможны. И хотя Боб так по-настоящему и не понял, как сервер ухитряется это делать, все сработало! После полного перехода на хранимые процедуры команда прогнала все свои тесты безопасности, и результатом Боб остался доволен.

Аутентификация и идентификация

Под аутентификацией понимается процедура, посредством которой компьютер или система проверяет, что пользователь действительно является тем, кем представляется. Задача в том, чтобы убедиться, что вы подлинный пользователь, а не самозванец. Когда кто-то подходит к вашему автомобилю или звонит в дверь квартиры, вы обычно смотрите на него, определяете, кто это, и решаете, хотите ли вы его впускать. Люди со слабым зрением иногда опознают человека по голосу, а бывает, что мы проверяем документы, чтобы решить, следует ли доверять человеку. Большинство людей не пускают незнакомцев в дом, в машину или в офис. Но у компьютеров нет органов чувств, с помощью которых можно было бы узнать человека; им нужна для этого другая система, механизм или устройство. Тот цифровой механизм, которым они пользуются, и называется аутентификацией.

АУТЕНТИФИКАЦИЯ И АВТОРИЗАЦИЯ: ДВА ЧРЕЗВЫЧАЙНО ВАЖНЫХ МЕХАНИЗМА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ

Часто можно слышать, как профессионалы в области безопасности произносят AuthN и AuthZ, полагая, что все вокруг знает, о чем они говорят и что означают эти странные сокращения. Но больше вам не придется гадать, любезные читатели!

- AuthN – сокращение от «authentication» (аутентификация). Так компьютер спрашивает «Вы кто?» и решает, знает он вас или нет.
- AuthZ – сокращение от «authorization» (авторизация). Так компьютер спрашивает «А вас тут ждали?» и решает, предоставлять доступ или нет.

В общем случае компьютерная система сначала аутентифицирует, а потом авторизует пользователя.

Примечание Акроним AAA означает «authentication, authorization, accounting» (или auditing), т. е. «аутентификация, авторизация и учет» (или аудит). Иногда еще добавляют букву I (Identity – идентификация), так что получается IAAA. Основное добавление здесь – аудит аутентификации и авторизации: *акт проверки того, что доступ используется правильно.*

Идентификация – это когда человек (пользователь) или система предъявляет свой идентификатор в сети или иной цифровой системе, который позволил бы отличить его от других пользователей и систем. Когда вы что-то делаете в сети или другой цифровой системе, и та должна проверить, кто вы такой, она проверяет ваш идентификатор (точно так же, как вашу личность удостоверяют при входе в аэропорт, поликлинику и т. д.). У всего в сети имеются идентификаторы, которые хранятся в системе управления идентификаторами. Даже у веб-приложения или API есть идентификатор.

Самая широко используемая система идентификации – Windows Active Directory (или просто AD). Из других имеющихся на рынке систем упомянем Okta, Microsoft Azure Active Directory, Auth0 и SolarWinds.

Примечание Акроним SSO означает «single sign-on» – единая точка входа. При использовании SSO пользователь выполняет процедуру входа, когда начинает работать с системой, а затем ему предоставляется доступ к другим системам, являющимся частью системы SSO. Впервые я приняла участие в команде, создававшей такую систему, когда работала в секретариате Совета казначейства канадского федерального правительства. У нас было несколько приложений, которыми могли пользоваться все государственные служащие, и мы не хотели, чтобы им приходилось входить по 20 раз на дню. Мы дали им возможность входить в систему один раз с помощью SSO, после чего приложения передавали друг другу токен, содержащий идентификатор пользователя, незаметно для него. Пользователю для этого ничего не нужно было делать, вследствие чего SSO можно считать прекрасным примером *беспроблемной безопасности.*

Итак, мы теперь знаем, что такое аутентификация и идентификация, осталось понять, как их обезопасить. В идеале лучше купить систему, чем создавать собственную. Такие системы невероятно сложны, и если вы станете делать собственную, то с большой долей вероятности допустите ошибку. Представленные на рынке системы очень тщательно протестированы, и все же иногда в них обнаруживаются ошибки. Другая связанная с безопасностью причина, по которой такие системы покупают, заключается в том, что это первая и самая важная линия обороны, а значит, тестировать ее будут чаще всего (и в недобрых целях). А причина, не относящаяся к безопасности, заключается в том, что купить систему обойдется дешевле, чем создавать аналогичную сопоставимого качества. В большинстве случаев затраты времени на ее создание и сопровождение не окупятся.

Примерами поставщиков платформ SSO могут служить Facebook, Google, Microsoft, LinkedIn и X (бывший Twitter). Из примеров SSO, которые можно купить и внедрить самостоятельно, упомянем Duo, Indaptive, Okta, NetIQ, PingOne и OneLogin.

Подробнее о том, как написать свою систему, см. в книге «Alice and Bob Learn Application Security» (глава 4, стр. 94–98), а также документ OWASP Authentication and Identity Cheat Sheet (https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html).

Авторизация и управление доступом

Под авторизацией понимается процедура, посредством которой компьютер решает, предоставить ли вам цифровой доступ или права. Управление доступом – это технический механизм, который дает или ограничивает доступ. Мы поговорим о том, что это значит, но запомните главное – по возможности следует покупать систему для этой цели, а не создавать ее самостоятельно. Если это невозможно, но в каркасе, с которым вы работаете, есть средства для реализации такой системы, воспользуйтесь ими, но к построению системы с нуля прибегайте только в самом крайнем случае.

Примечание Купить, заимствовать, создать – в таком порядке. Всегда старайтесь купить систему, которая делает то, что вам нужно, и только если на рынке нет решения, удовлетворяющего всем требованиям, прибегайте к заимствованию. Под заимствованием понимается повторное использование кода, созданное другой командой, другой организацией или с применением средств, имеющихся в каркасе. В крайнем случае, когда ничего другого не остается, мы создаем систему с нуля, т. е. пишем все ее части самостоятельно. Приоритеты расставлены именно так, чтобы сэкономить время и деньги, но также и потому что готовые системы и встроенные в каркас средства обычно написаны с большим вниманием к безопасности, а значит, и вам будет спокойнее. Это сделка, в которой обе стороны выигрывают.

Взгляните на рис. 2.1. Разрешено ли Алисе видеть эту папку? Может ли она редактировать ее содержимое? Существуют *модели* доступа, и существуют *реализации* доступа (логические методы управления доступом)²⁴.

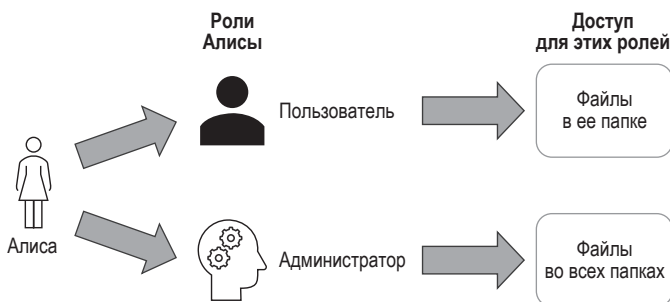


Рис. 2.1. Управление доступом для Алисы

Модели управления доступом

Хотя разных моделей много, наиболее хорошо известны следующие четыре.

В случае **мандатного управления доступом** (mandatory access control – **MAC**) грифы секретности (секретно, совершенно секретно, несекретно) назначаются людям, ресурсам, системам и другим субъектам, а доступ разрешен только имеющим такой же или более высокий уровень авторизации (например, если у вас есть допуск с совершенно секретным материалам, то вам разрешено просматривать совершенно секретные документы, относящиеся к проектам, на которые вы назначены, и к работе, которую вы выполняете). Эта модель часто используется в государственных, и особенно военных учреждениях.

При **ролевом управлении доступом** (role-based access control – **RBAC**) доступ предоставляется в зависимости от роли, которую субъект играет в организации. Например, если вы администратор базы данных (**DBA**), то получите доступ ко всем или к некоторым серверам базы данных. А если вы не **DBA**, то у вас не будет доступа ни к какому серверу БД.

При **избирательном управлении доступом** (discretionary access control – **DAC**) создателю системы, документа или программы предоставляется полный контроль над ним и право предоставлять доступ другим субъектам. Обычно это используется в онлайн-офисных инструментах типа Office 365 и G-Suite; вы создаете документ, а затем лично решаете, кому дать доступ к своему творению. **DAC** применяется почти повсеместно. Это самая распространенная форма управления доступом в системах потребительского класса (например, Windows и Linux).

При **управлении доступом на основе правил** (rule-based access control – **RBAC** или **RB-RBAC**) доступ предоставляется динамически на основе критерия, заданного системным администратором. Например, работникам дневной смены может быть разрешен доступ к их системам ежедневно с 8:00 до 18:00, но только к устройствам с IP-адресами из заданного диапазона. Или же доступ может быть немедленно прекращен, когда статус работника изменяется на «уволен». Примерами такого вида управления доступом являются брандмауэры.

Логические методы управления доступом (реализация)

Списки управления доступом (access control list – **ACL**) – это списки идентификаторов в системе, которым разрешен доступ к объектам в сети или в пределах систем. Взгляните на рис. 2.2. Может ли Таня редактировать этот документ? Нет. Может ли она добавить комментарий? Да. Отлично, теперь она будет добавлять комментарии.

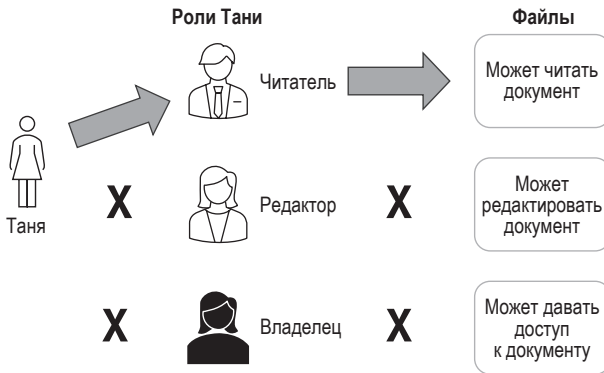


Рис. 2.2. Управление доступом для Тани

Групповые политики позволяют членам различных групп (электронных списков) выполнять определенные действия (это часть ОС Windows). Групповые политики можно применять к пользователям, системам или группам.

Биометрия, ПИН (персональные идентификаторы), пароли и токены – другие логические элементы управления, которые могут использоваться для предоставления или ограничения доступа.

Управление сеансами

Когда пользователь перемещается по вашему приложению, нажимает кнопки, щелкает по ссылкам и другим частям приложения, все это поведение отслеживается с помощью сеанса, технические детали которого обычно хранятся в файлах cookie. Мы делаем это для того, чтобы приложение могло запоминать действия пользователя в системе. Если он выбрал пару красивых туфель и положил их в корзину, то при переходе к следующей странице пользователь хочет, чтобы приложение запомнило цвет и размер обуви. А при нажатии кнопку «Купить» (вот же счастье!) пользователь хочет, чтобы приложение вспомнило все, что он положил в корзину. И на протяжении всего этого времени приложение должно знать, что это один и тот же пользователь, и не путать его с другим IP-адресом, пользователем и вообще самозванцем. Это и называется *сеансом*.

Сеансы не были присущи интернету изначально. Когда интернет только проектировался, никто из его создателей даже не думал, что он превратится в то, что мы имеем сегодня. Поскольку сеансов поначалу не было, они не встроены ни в протокол TCP/IP, ни в какой-либо другой из основополагающих элементов интернета.

Если в вашем каркасе имеются средства управления сеансами, пользуйтесь ими. Если нет, то можете передавать идентификатор сеанса и другую информацию от клиента (браузера) серверу и обратно в безопасном cookie. Подробные инструкции, как это сделать, имеются в книге «Alice and Bob

Learn Application Security», глава 4, раздел «Управление сеансами», а также в документе OWASP Session Management Cheat Sheet (https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html).

Управление секретами

Секреты – это цифровые учетные данные для аутентификации (как компьютер доказывает, что он такое). Секреты используются компьютерными системами для доступа к другим системам, данным, сетям и т. д. Секреты используются не людьми, а компьютерами, чтобы аутентифицировать себя другим компьютерам. Самый известный тип секрета – пароль, но есть и другие: ключи, секреты API, токены, хеши, сертификаты и т. д. Когда специалист по безопасности говорит об «управлении секретами», он имеет в виду способ обработки секретов. Как они хранятся? Когда срок хранения секрета истекает, как он замещается новым? Средство управления секретами решает эти задачи за вас.

В те дни, когда я работала программистом, эти вещи не изучались ни в школе, ни на работе. Я хранила все строки подключения (к базам данных) в коде. Да и вся моя группа так делала. У нас была одна строка подключения к производственной системе, другая – к среде разработке и обычно еще несколько для других сред. И все они хранились прямо в коде удобства ради. Мы закомментировали ненужные в данный момент, чтобы не потерять пароль. А затем мы помещали все это в репозиторий кода, где любой член команды мог увидеть секрет. И никто из нас даже не думал, что все это можно сделать лучше.

Спустя несколько лет я стала младшим пентестером. Я узнала, что существуют самые разные места, где можно хранить свои секреты, и что можно купить программы, которые сделают это правильно. Я также узнала, что хранить такие вещи в коде категорически запрещено! Я вспоминала свои прежние привычки, приобретенные в бытность разработчиком, и дала себе слово, что, обнаружив секреты в коде, буду сообщать плохие новости терпеливо и доброжелательно.

К сожалению, те, кто ищет, неизменно продолжают находить секреты в коде. В 2021 году я нашла действующие пароли к онлайн-банку в ходе всего лишь *одной* выполненной в том году оценки безопасности. Я уже даже не работала пентестером и, хотя обладаю кое-каким опытом в этой области, экспертом себя не считаю. Я просто сделала клиенту одолжение. А это значит, что разработчики так и не уяснили, что существует лучший и более простой способ управления секретами, чем хранение их в коде.

Первый шаг к управлению секретами – поискать секреты во *всем* коде во всей организации, и делается это на удивление легко и дешево. В интернете есть много бесплатных сканеров секретов, большинство из них напоминают команду Linux GREP тем, что используют регулярные выражения для поиска строк, которые выглядят как секреты. Если строка обладает высокой энтро-

пией и (или) высокой случайностью, то это может быть секрет. Если имеется переменная с именем `key`, `password` или `Secret`, которой присваивается значение, то это значение может быть секретом. Такие инструменты могут проанализировать огромный объем кода за очень короткое время и выдают достойные внимания результаты. Внимательно просмотрите их и запишите все настоящие секреты. Если возможно, настройте регулярное выражение, так чтобы оно не возвращало ложноположительных результатов, а затем сохраните его.

Примечание Существуют платные программы поиска секретов, в которых применяются более тонкие алгоритмы, а кроме того, проверяется, являются ли секреты «действующими». Они сэкономят вам время и дадут более точные результаты, но это стоит денег. Решайте, что для вас ценнее: время или деньги.

Затем купите систему для управления секретами. Знаю, знаю – вы программист и можете написать ее *сами*. Но поверьте мне – лучше купить. Скорее всего, это обойдется дешевле и почти наверняка будет безопаснее. Прошу вас, не пишите такую систему самостоятельно, если только не работаете в компании, которая специализируется на изготовлении продуктов для обеспечения безопасности; и даже в таком случае это потребует значительных усилий.

Приобретенную систему управления секретами нужно развернуть. Можете нанять себе в помощь компанию, у которой покупали, или сделать это самостоятельно. Но кто бы это ни делал, проследите за тем, чтобы для каждой системы был отдельный сейф (экземпляр). Не следует хранить секреты всех приложений, разрабатываемых и эксплуатируемых в компании, в одном сейфе – ведь если какое-то приложение будет скомпрометировано, то оно сможет аутентифицироваться для входа в сейф и украсть пароли всех остальных приложений. Каждому приложению или системе необходим свой сейф.

Теперь поместите все свои ценности в систему управления секретами! Измените все найденные действующие секреты и поместите в сейф их новые версии. Больше нигде не храните копии секретов. Ваше приложение сможет аутентифицироваться и получить секрет – так же как и все остальные приложения, настроенные для работы в этом режиме. Человек вообще не должен прикасаться к секретам, и хранить их где-либо еще не следует. Если человеку все же понадобится копия секрета для чего-то, ротируйте секрет, используйте, а затем сохраните новую версию в сейфе и нигде более. Копия секрета может потребоваться, например, если секрет был кому-то раскрыт, скомпрометирован как-то иначе или если вышла новая версия API, нуждающаяся в создании нового ключа.

Приложения могут получать доступ к секретам программно, посредством API или даже в конвейере CI/CD.

Совет Под ротацией секрета понимается генерирование нового секрета с последующим сохранением его в системе управления секретами. Старый выбросьте – и радуйтесь жизни!

Совет Системы управления секретами, поддерживающие несколько ключей подписания (например, алгоритм разделения секрета Шамира²⁵), также пользуются преимуществами конфиденциальности и соответствия Общему регламенту по защите данных (GDPR) благодаря уничтожению или отзыву ключей. *Криптографическое уничтожение* (shredding) – это удаление ключа шифрования данных как средство «удаления» самих данных (они становятся недоступными, а это все равно что удаленными)²⁶. Такая тактика достижения комплаенса с GDPR довольно часто встречается в системах на основе последовательности событий: удаление ключа пользователя уничтожает всю информацию, позволяющую установить его личность (personally identifiable information – PII). Уничтожение ключа делает дешифрирование невозможным, т. е. доступность данных необратимо утрачивается.

Управление паролями

В этом разделе мы будем рассматривать пароли, с помощью которых пользователи аутентифицируются в приложениях. Нас сейчас не интересует, что люди делают со своими паролями, мы обсудим, как приложение и база данных должны хранить пароли и управлять ими для доступа пользователей к созданному вами ПО.

Прямо сейчас в даркнете выставлены учетные данные на продажу. Учетные данные – это пара, состоящая из имени и пароля пользователя, которая может быть успешно применена для доступа хотя бы к одной системе. И прямо сейчас значительная доля пользователей на планете страдает, не будучи способна запомнить великое множество паролей, а потому практикует *повторное использование паролей*. Это означает, что одна и та же пара (имя пользователя, пароль) используется для входа в несколько систем. А коль скоро так поступает много людей, преступники знают, что, купив большую партию утекших или украденных паролей, они смогут использовать многие из них для аутентификации на большом числе сайтов. Ну и, конечно, используют украденные учетные данные, чтобы обокрасть незадачливых пользователей, а заодно совершить и другие преступления. Понятно, что мы хотим уберечь наших пользователей от этой ужасной судьбы.

Примечание Попытка использовать несколько пар учетных данных на одном и том же сайте, чтобы взломать его, называется *атакой с подстановкой учетных данных*. Очень часто пользователи добавляют цифры в конец пароля, например «Password1» или «Winter2025», и преступники об этом знают. Когда злоумышленник модифицирует пароль, увеличивая или уменьшая число в конце, мы имеем *атаку с распылением паролей*, или *радужную атаку с подстановкой учетных данных*. Слово «радужная» – отсылка к радужным таблицам, которые используются для вскрытия алгоритмов хеширования.

Решить эту проблему можно несколькими способами. Один из них – не использовать пароли вовсе. Вариантов тут несколько, в частности *беспарольные* системы (т. е. такие, в которых для аутентификации используется что-то взамен пароля). Примером может служить ваша система идентификации

в сети, которая «опознает» пользователей при попытке доступа в систему и молчаливо пускает их. Хранящийся в устройстве сертификат тоже можно использовать для беспарольного входа, равно как отпечатки пальцев, распознавание лица и т. д. К сожалению, многие пользователи игнорируют этот вариант, даже когда он предлагается.

Другой метод входа без пароля – *ключи доступа*, в которых для идентификации используется биометрия (чаще всего отпечаток пальца). Дополнительные сведения см. по адресу <https://passkeys.dev/>. Постепенно пользователи привыкают к ключам доступа, и это прекрасная новость для энтузиастов безопасности.

Запомните, что чем меньше неудобства для пользователя, тем лучше. Но если вы все-таки вынуждены использовать пароли, то нужно знать несколько вещей, которые предотвратят кражу и последующее неправомерное использование учетных данных.

Самое главное – создавайте систему, в которой пароли пользователей хранятся в виде хешей, а не в зашифрованном виде. Хеширование – это односторонний криптографический метод, т. е. вы никогда* не сможете восстановить по хешу исходное значение. С другой стороны, зашифрованное значение можно дешифровать (если известен ключ и алгоритм) и, значит, восстановить. Вы, наверное, подумали: «Как это – не иметь возможности восстановить пароль пользователя? А как же, не зная пароля, я решу, пускать его или нет?»

Примечание **Никогда***: на момент написания книги мощности современных компьютеров недостаточно для обращения алгоритмов хеширования. Но в постквантовую эру ситуация может измениться.

Допустим, что пользователь уже задал свой пароль. Когда он впоследствии входит в систему, введенный пароль хешируется тем же алгоритмом, что при первоначальном задании пароля. После этого два хеша сравниваются. Если они совпадают, то система пускает пользователя, иначе предлагает попробовать еще раз.

Итак, мы знаем, что значение должно храниться в хешированном, а не в зашифрованном виде. Что дальше? *Подсаливание*. Это означает добавление случайного значения к паролю (в начале или в конце) и последующее хеширование. В базе данных хранятся как случайное значение, так и хеш²⁷. Когда пользователь вводит свой пароль, вы добавляете к нему соль, применяете к результату алгоритм хеширования и сравниваете то, что получилось, с хранимым хешем.

Совет В сети полно сайтов, содержащих заранее созданные радужные таблицы для взлома украденных паролей; это гораздо проще, чем вы думаете. Подсаливать пароли абсолютно необходимо!

Дополнительный уровень безопасности, помимо хеширования и подсаливания, называется *перчением*. (Я не шучу. Можете сами побалагурить насчет добавления соли и перца в свое блюдо AppSec.) Криптографический перец

похож на соль в том смысле, что тоже является случайным и добавляется к паролю до хеширования, но он считается секретом и не хранится вместе с паролем в базе данных. Его можно хранить в программе управления секретами или в другом безопасном месте, например в аппаратном модуле безопасности. В каждой организации есть только одно значение перца, и оно используется для всех пользователей. Держите его под замком!

У использования перца есть несколько недостатков. С административной точки зрения это масса хлопот. Потеряв перец, вы должны будете заставить поменять пароль всех без исключения пользователей. Мало того, рано или поздно вы должны будете ротировать перец, и это тоже большая работа, даже если она планируется заранее. Комбинация хеширования с подсаливанием считается «достаточно безопасной» для большинства организаций, но когда нужен более высокий уровень защиты, то можно добавить и перцы.

Безопасность связи (только криптография и HTTPS)

Все, что мы передаем через интернет, должно быть зашифровано. Прошли дни медленного интернета и компьютеров с мизерным объемом памяти. Благодаря Фонду электронных рубежей (Electronic Frontier Foundation – EFF) все мы можем получить бесплатные сертификаты для любого сайта на сайте Let's Encrypt (<https://letsencrypt.org/>). Поэтому я, как и многие другие профессиональные «безопасники», настоятельно рекомендую обслуживать веб-приложения и API только по протоколу HTTPS. Каждый запрос и каждый ответ должны быть зашифрованы последней стабильной версией протокола Transport Layer Security (TLS).

Но это еще не все – важно использовать продукты, которые шифруют данные. Если вы не работаете в компании, которая специализируется на разработке продуктов для шифрования, то не пытайтесь писать такие программы самостоятельно. Это *очень трудно*, и потому заниматься этим должны опытные специалисты. Понимаю, что начинаю напоминать заевшую пластинку.

ДОМОРОЩЕННОЕ «ШИФРОВАНИЕ»

Боб помнит, как принял участие в игре «Захвати флаг» (CTF) с ребятами, с которыми познакомился на встрече OWASP. Первым заданием было «дешифровать» доморощенный шифр. Автор задания дважды применил к паролю кодировку base64 вместо настоящего шифрования. Боб видел, как ребята из его команды скопировали значение на сайт, который его декодировал; это заняло меньше минуты! Надо же, Боб раньше и помыслить не мог, насколько очевидны для других незашифрованные значения... Он раз и навсегда решил не позволять своим разработчикам самим писать код для такой ответственной задачи.

Совет Даже если вы создаете собственную систему настоящего шифрования (а не кодирования), хакеры все равно смогут вскрыть ее! Часто на соревнованиях CTF участникам предлагаются системы «доморощенного шифрования», и они умудряются взломать их. Шифрование – трудная задача, оставьте ее специалистам.

Защита конфиденциальных данных

Общая проблема безопасности, свойственная любому приложению или компьютерной системе, – непреднамеренная выдача конфиденциальной информации. Такая ситуация называется *раскрытием конфиденциальных данных, кражей данных* (если данные были украдены) или *утечкой данных* (если данные попали в чужие руки случайно). Но как бы ни называть, это опасно и может нанести серьезный ущерб физическим лицам, компаниям и государственным органам.

Ниже приведен неполный перечень видов конфиденциальных данных.

- Идентифицирующие персональные данные (*personally identifiable information* – PII) – все, что может быть использовано для кражи или раскрытия вашей личности, например: полное имя, дата рождения, домашний адрес, номер социального страхования, номер или изображение водительских прав, место работы и т. д.
- Информация о состоянии здоровья: есть ли у вас какие-либо заболевания, как они отражаются на вас, принимаете ли вы какие-то лекарства и т. д.
- Интеллектуальная собственность: рукописи книг, фильмы, пьесы, копии произведений искусства (живописных, музыкальных и т. д.) и вообще все, что может быть юридически защищено авторским правом.
- Бизнес-аналитика и другие коммерческие секреты, например: стратегическое направление развития, проекты, планы и т. д. – все, что может нанести ущерб бизнесу, если будет раскрыто публично или попадет в распоряжение конкурентов.
- Сведения с грифом «для служебного пользования», «секретно» и «совершенно секретно», например контртеррористические планы, еще не обнародованные государственные стратегии, результаты исследований и т. д.
- Информация о заказчиках: что они покупали, когда, сколько потратили денег, какими услугами пользовались и т. д. Как правило, заказчики хотят конфиденциальности от организаций, с которыми ведут дела, даже если купили всего пару штанов.
- Все, что в случае раскрытия, может вызвать смутнение, потерю денежных средств, физическую опасность или причинить еще какой-то ущерб.

Первое, что нужно сделать для предотвращения раскрытия конфиденциальных данных по ошибке, – вообще не собирать те данные, которые нам не нужны (это называется *минимизацией данных*). Если мы разрабатываем супер-пупер-приложение-напоминалку, то нужны ли нам дата рождения пользователя, его домашний адрес и полное имя? Пожалуй, нет. Отказавшись от сбора ненужной информации, мы уменьшим поверхность атаки, поскольку становится меньше данных, нуждающихся в защите.

Второй шаг – классифицировать и пометить данные, чтобы каждый, кто к ним обращается, знал об уровне секретности. В результате с данными будут обращаться и защищать их согласно классификации.

Под классификацией данных понимается определение их уровня конфиденциальности. То, что классифицировано как «несекретно» (в США) или «для общего доступа» (в Канаде), может быть раскрыто публично, и это не причинит никакого ущерба; такими данными можно беспрепятственно делиться. Но если стоит гриф «для служебного пользования» (США) или «охраняется» (Канада, уровни А, В или С), то всякий, кто пользуется данными, сразу поймет, как с ними следует обращаться.

Пометка означает ровно это: присоединение к данным какой-то метки. Это может быть дополнительное поле в базе данных, наклейка на CD, DVD или жестком диске, пометка контейнера в облачном хранилище и т. д. Основной смысл пометки – сделать ее максимально очевидной для других.

УНИЧТОЖЕНИЕ СОВСЕКРЕТНОЙ ИНФОРМАЦИИ

В самом начале карьеры Алиса работала в одном совершенно секретном учреждении. Там на CD и DVD наклеивали этикетки «совершенно секретно», но все равно, переставая быть нужными, они подвергались уничтожению. Алиса рубила их на мелкие кусочки в бумагорезке и складывала в сейф. А раз в неделю все это отправлялось на сжигание. Ей также довелось работать в месте, где вместо бумагорезки использовали «ящик для сжигания». Иногда данные настолько ценные или секретные, что физическое уничтожение – единственный способ обеспечить их безопасность.

Следующий шаг защиты конфиденциальных данных – соблюдать правила и инструкции, принятые в организации. Если для вас не предусмотрено должностной инструкции или она кажется вам слишком ограничительной и вообще «неподходящей», то примите во внимание следующие рекомендации.

Рекомендации по защите конфиденциальных данных.

- Соблюдайте все регламенты, законодательные акты, комплаенс, политики и правила, применимые к вашим данным, например: GDPR (Евросоюз), Закон о защите персональной информации и электронных документах (PIPEDA – Канада) или Закон штата Калифорния о защите конфиденциальности потребителей (CCPA – США).
- Не сохраняйте конфиденциальные данные в журналах. Это касается полных номеров кредитных карт, американских номеров социального страхования (SSN), канадских номеров социального страхования (SIN), имен, дат рождения и домашних адресов пользователей и т. д. Если сомневаетесь, спросите «безопасников».
- Шифруйте *все данные* (не только конфиденциальные) в процессе передачи и при хранении. Если данные особо конфиденциальные, то стирайте память клиентской машины, в которой они хранились, когда необходимость в них отпадает.

- Храните данные не дольше чем необходимо. Если есть возможность, ставьте автоудаление после крайней даты. Например, если некоторые данные нужно хранить 7 лет, то задайте автоудаление по истечении этого срока. Встраивайте такие возможности в систему с самого начала.
- Не помещайте конфиденциальные данные в скрытые поля, параметры URL-адреса и прочие места, где их может увидеть конечный пользователь. Такие места не следует использовать для хранения и передачи конфиденциальных данных даже в зашифрованном виде.
- Никогда не кешируйте конфиденциальные данные.
- Проверяйте идентификатор пользователя (аутентификация) и наличие у него прав доступа (авторизация), перед тем как разрешить доступ к данным, а затем повторно выполняйте авторизацию для каждого нового запроса (страницы или записи). В особо секретных системах заново аутентифицируйте пользователя с регулярным интервалом (от 15 мин до нескольких дней, проконсультируйтесь с лицами, ответственными за безопасность приложений).
- Для особо секретных данных иногда можно использовать систему токенизации, которая преобразует данные в зашифрованные токены и раскрывает истинные значения тогда, когда это необходимо. Значения остаются зашифрованными до момента, когда они понадобятся (часто говорят «своевременно»). Программисты реализуют эту идею с помощью группы безопасности или стороннего подрядчика; в общем случае не предполагается, что это должно быть сделано своими силами, без привлечения экспертов.
- Протоколируйте все запросы к конфиденциальным данным и поднимайте тревогу в случае нескольких неудачных попыток или любого потенциально опасного или необычного поведения пользователя.
- Ведите мониторинг системы на предмет аномалий или любых признаков вредоносных программ. Обычно такую систему внедряет группа безопасности, но они могут обратиться к вам за помощью.
- Следуйте всем рекомендациям по кодированию, приведенным в этой книге. Работайте там, где к безопасности информационных систем относятся серьезно. Выбирать место работы с учетом отношения к информационной безопасности и наличия развитой культуры безопасности, пожалуй, является лучшим способом вырасти в грамотного специалиста по безопасному кодированию.

Заголовки, связанные с безопасностью

HTTP-заголовок – это поле HTTP-запроса или ответа, в котором передается дополнительный контекст и метаданные о запросе или ответе.

HTTP-заголовки сообщают браузеру о том, как разработчик хочет видеть обработку запросов и ответов в веб-приложении.

— Таня Янца (это я!)

В своей первой книге «Alice and Bob Learn Application Security» я рассматривала все связанные с безопасностью HTTP-заголовки, существовавшие в то время (конец 2020 года). С тех пор добавилось несколько новых, а функции уже имеющихся расширились. Для тех, кто читал мою первую книгу, я приведу здесь их краткое описание, а остальные могут найти более подробные сведения в сети или в моей книге.

Новые функции заголовков, связанных с безопасностью

Новые функции были добавлены, чтобы мы могли яснее выражать свои намерения о том, как браузер должен относиться к безопасности. Они предоставляют разработчику и приложению больше контекста и возможностей управления, чтобы сделать приложения более безопасными и надежными.

Заголовки запроса метаданных

Эти новые заголовки запроса позволяют браузеру присоединить к запросу дополнительный контекст, который поможет серверу решить, стоит ли выполнять запрос. Эти решения могут быть приняты на сервере или «подняты» на уровень сети доставки содержимого (CDN) или обратного прокси-сервера (Web App Firewall, WAF или Run-time Self Protection, RASP). Заголовки этого типа призваны предотвратить потенциально вредоносные действия от имени пользователя, запрошенные с внешних сайтов и (или) из скриптов (межсайтовый скриптинг). С их помощью мы добавляем информацию об источнике и контексте запроса, чтобы сервер мог использовать ее при принятии решения:

- Sec-Fetch-Site: определяет, какой сайт сгенерировал запрос;
- Sec-Fetch-Mode: определяет предполагаемый режим или тип запроса;
- Sec-Fetch-User: определяет, был ли запрос инициирован действием пользователя;
- Sec-Fetch-Dest: определяет предполагаемого получателя запроса.

За дополнительными сведениями обращайтесь по адресу <https://w3c.github.io/webappsec-fetch-metadata>²⁹.

Заголовок Content Security Policy

Этот заголовок не новый, но в нем появились новые возможности, достойные упоминания: `strict-dynamic` и `trusted-types`³⁰. Они и рассматриваются в этом разделе.

Strict-Dynamic

Политика безопасности содержимого (content security policy – CSP) позволяет перечислить ресурсы и скрипты приложения, чтобы помешать злоумышленникам добавить свои собственные. Раньше программисты часто использовали одноразовое число (случайное значение, которое показывало, что скрипт разрешено исполнять), но это было довольно сложно, да и правильно построить его нелегко. При использовании одноразовых чисел в сочетании с CSP если вы не контролируете какие-то сторонние скрипты или ваши скрипты не порождают дочерние скрипты, то добавить одноразовое число не получится, и все ваши тщательно взлелеянные планы летят насмарку. С появлением параметра `strict-dynamic` в одноразовом значении нуждаются только скрипты в заголовке ответа, и это позволяет скриптам создавать дочерние скрипты (которым не нужно собственное одноразовое число). Хотя кое-какую работу проделать все же придется, провести рефакторинг веб-приложения под применение CSP не так уж трудно. Подробное руководство по использованию CSP см. по адресу <https://csp.withgoogle.com>³¹, а о том, как протестировать правильность реализации CSP, – по адресу <https://csp-evaluator.withgoogle.com>³².

Trusted-Types

XSS-атаки на основе DOM труднее всего предотвратить, обнаружить и обезвредить. Параметр `trusted-types` позволяет разработчику создать типы переменных, которым XSS не страшен, а затем использовать CSP, чтобы разрешить использовать в приложении только эти доверенные типы. Когда режим `trusted-types` включен, приложение может использовать только эти типы, все остальные типы переменных запрещены.

Для создания доверенного типа мы создаем политику доверенных типов, включающую проверку. Затем эта политика используется для создания объектов доверенных типов, и, наконец, политика доверенных типов активируется с помощью заголовка CSP. Подробнее о доверенных типах см. <https://github.com/w3c/trusted-types>³³.

Отдельное спасибо Лукасу Вайксельбауму, его группе в Google, Майклу Коппманну и конференции Sec4Dev 2022 (Вена, Австрия), которые просветили меня насчет этих двух добавлений в CSP и доверенных типов. Следите за Лукасом, если хотите узнавать о новациях в браузере и обновлениях безопасности в Chrome.

Ранее рассмотренные заголовки, связанные с безопасностью

Ниже приведены краткие пояснения к различным заголовкам безопасности, имеющимся в распоряжении разработчиков веб-приложений; все они были подробно рассмотрены в моей первой книге «Alice and Bob Learn Application Security»³⁷.

Заголовок Content-Security-Policy

Заголовок Content-Security-Policy требует перечислить все ресурсы (скрипты, изображения и т. д.), которые вы хотите использовать в своем приложении, но которые не принадлежат вашему домену. Это предотвращает XSS и атаки с помощью фреймов. Реализовать этот заголовок труднее всего, но зато это самая действенная форма защиты. Проконсультируйтесь с группой безопасности приложений.

Следующая закрытая реализация CSP блокирует вообще все:

```
Content-Security-Policy:default-src'self'; block-all-mixed-content;
```

Если вы планируете включить в свое приложение что-то из других доменов, то обязательно объявите это в заголовке Content-Security-Policy. Не объявленное работать не будет. Ниже показаны наиболее употребительные конфигурации, но вообще-то ими дело не исчерпывается:

- object-src= плагины (в качестве источников допустимы элементы <object>, <embed> и <applet>);
- style-src= стили (CSS – каскадные таблицы стилей);
- img-src= изображения;
- media-src= видео и аудио;
- frame-src= фреймы;
- font-src= шрифты;
- plugin-types= ограничивает типы допустимых к исполнению плагинов.

HTTP Strict-Transport-Security

Заголовок Strict-Transport-Security (HSTS) разрешает использование только HTTPS (перенаправляет с HTTP на HTTPS). В следующем примере режим задан на один год:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

X-Frame-Options

Этот заголовок запрещает сайтам показ чужих сайтов во фреймах на вашем сайте. Он работает для старых браузеров, которые не поддерживают заголовок CSP:

```
X-Frame-Options: SAMEORIGIN (разрешается показывать во фреймах сайты из своего домена)
X-Frame-Options: DENY (запрещены любые фреймы)
```

X-Content-Type-Options

Запрещает автоматическое определение типа содержимого и заставляет вас объявлять типы. Также останавливает сниффинг-атаки:

```
X-Content-Type-Options: nosniff
```

Permissions-Policy

Этот заголовок запрещает или разрешает различные средства HTML 5. По умолчанию подразумевается запрет, если некоторое свойство не обязательно:

```
Permissions-Policy: camera 'none'; microphone 'none'; speaker 'none';
vibrate 'none'; geolocation 'none'; accelerometer 'none'; ambient-
light-sensor 'none'; autoplay 'none'; encrypted-media 'none'; gyroscope
'none'; magnetometer 'none'; midi 'none'; payment 'none'; picture-in-
Picture 'none'; usb 'none'; vr 'none'; fullscreen *
```

Expect-CT

Уже после выхода моей первой книги этот заголовок объявлен нереконмендуемым. Не пользуйтесь им.

Referrer-Policy

Пользуйтесь этим заголовком, чтобы избежать разделения страниц, ранее просмотренных пользователями, и для обеспечения конфиденциальности пользователей:

```
Referrer-Policy: origin
```

Расширение для фиксации открытого ключа HTTP (НРКР)

Не используйте. Этот заголовок не рекомендуется и опасен³⁴.

X-XSS-Protection

Этот заголовок объявлен нереконмендуемым и несет с собой небольшой риск. Лучше не использовать³⁴.

Совет Используйте все возможные заголовки безопасности. Относитесь к ним как к ремню безопасности в своей машине: он помогает только в случае аварии, но уж если что-то случилось, то вы возблагодарите бога, что накинули его. Стоит к ним привыкнуть, как перестаешь замечать, но в один прекрасный день они могут спасти вас (и ваше приложение!).

Еще несколько новых заголовков

Эти заголовки появились в 2021 году, но я называю их «новыми», потому что до сей поры их почти никто не использовал, т. е. они *новые для вас*. Заголовки и другие средства браузера, связанные с безопасностью, со временем меняются, реагируя на новые типы атак и рисков. Все заголовки, описанные в этом разделе (за исключением CORS, который существовал до 2021 года), создавались, чтобы предотвратить такие атаки по побочным каналам, как Specter и Meltdown.

Политика одного источника

Политика `same-origin` – это средство браузера, которое налагает ограничения на то, как документы и скрипты из одного домена могут взаимодействовать с ресурсами и скриптами из другого домена³⁵. Оно было создано в попытке прекратить атаки веб-приложений друг на друга. Оно включено и по умолчанию активировано во всех современных браузерах. Эта политика запрещает исполнение в приложении скриптов, происходящих из другого источника. *Источником* по определению является сочетание схемы URI, домена и номера порта. Если все три одинаковы, то скрипт может быть выполнен. Если хотя бы один компонент различается, то скрипт запрещено выполнять, если только он не указан в составе `CORS`³⁶. Политика `same-origin` не предотвращает загрузку других ресурсов страницы, только скриптов.

Как человек, тесно связанный с безопасностью, я очень довольна, что эта политика по умолчанию активирована!

Чтобы понять правила конфигурирования следующих трех заголовков безопасности, нужно уяснить себе смысл некоторых терминов.

На одном сайте означает, что все находится в одном домене, включая поддомены. Например, <https://shehackspurple.ca/> и <https://newsletter.shehackspurple.ca/> находятся на одном сайте, но не в одном источнике.

Источник относится к домену, схеме и порту. Если все три компонента для двух URL-адресов одинаковы, то оба URL имеют одинаковый источник (порт, поддомен/домен и схему). Например, <https://shehackspurple.ca/blogs/article1> и <https://shehackspurple.ca/blogs/article2> находятся на одном сайте и в одном источнике.

Однако <https://shehackspurple.ca/blogs/article1> и <https://newsletter.shehackspurple.ca/article1> находятся на одном сайте, но в разных источниках (из-за различия поддоменов). Если вы хотите обобществить ресурсы обоих сайтов, то либо не должны использовать ни один из следующих заголовков безопасности (и смириться с тем, что другие сайты тоже смогут получить доступ к вашим ресурсам, а также с атаками по побочным каналам и не только), либо использовать следующие заголовки безопасности и явно разрешить общий доступ.

COEP: Cross-Origin Embedder Policy

Этот заголовок определяет, как ресурсы из разных источников разрешается встраивать в документ. Все ресурсы из разных источников должны быть явно перечислены в заголовках `CORS` или `CORP` (подробнее о них ниже), иначе они будут заблокированы. По умолчанию подразумевается значение `unsafe-none`, которое позволяет брать ресурсы откуда угодно без необходимости явно перечислять их. Слово `unsafe` употреблено не без причины!

`require-corp` разрешает ресурсы из того же источника (вашего домена), а также все ресурсы из другого источника, явно помеченные как «загружаемые» (чтобы пометить ресурс как загружаемый, его нужно перечислить в одном из заголовков `CORS` или `CORP`).


```
Cross-Origin-Embedder-Policy: (unsafe-none | require-corp);
report-to="default"
```

* Все эти заголовки поддерживают отчетность с помощью API отчетности. Если вы используете REPORT-URI или напрямую пользуетесь API, то знаете, как заголовки безопасности защищают ваше приложение.

COOP: Cross-Origin Opener Policy

Этот заголовок не дает другим процессам изучать вашу память путем выведения ее из группы контекстов просмотра (Browsing Context Group). Достигает он этого посредством изоляции вашего приложения и его информации. Заголовок гарантирует, что вы не разделяете контекст ни с какой другой страницей, потенциально враждебной. Но вы можете определить уровни сосуществования с другими страницами, оставаясь в рамках COOP.

Same-origin означает, что ваше приложение разделяет контекст (а значит, потенциальный доступ к данным, хранящимся в памяти) с другими страницами, которые находятся в вашем источнике *и* такими, что их заголовок COOP настроен на тот же источник. Это означает, что если вы откроете новое окно, в котором отображается страница не из того же источника, что ваше приложение, то оно будет открыто в новой группе контекстов просмотра (и тем самым ваше приложение будет защищено от разделения памяти и чего бы то ни было, хранящегося в вашем источнике или доступном из него).

Если вы хотите разделить свой источник со всплывающим окном, то можете сделать это явно, задав режим same-origin-allow-popups.

И последний режим для этого заголовка – unsafe-none, это режим работы приложения по умолчанию в случае, когда этот заголовок не задан. В нем ваша страница находится в той же группе контекстов просмотра, что и все остальные страницы с неустановленным заголовком COOP. Небезопасным (unsafe) он называется не зря!

```
Cross-Origin-Opener-Policy: (same-origin | same-origin-allow-popups |
unsafe-none); report-to="default"
```

CORP: Cross-Origin Resource Policy

Этот заголовок требует, чтобы были явно разрешены (перечислены) все ресурсы, которые можно загружать из другого источника. Вы также можете определить, как именно разрешено загружать эти ресурсы.

Режим same-site самый строгий, страницы из разных поддоменов не могут загружать ресурсы друг друга.

Режим same-origin задается для большинства сайтов, потому что в нем ресурсы сайта разделяются со всеми его поддоменами.

В режиме cross-origin разрешено все, его рекомендуется задавать только для сайтов, которые должны предоставлять ресурсы другим сайтам. В таком режиме работают сети доставки содержимого, а для большинства других сайтов он нежелателен.

```
Cross-Origin-Resource-Policy: (same-site|same-origin|cross-origin)
```


CORS: Cross-Origin Resource Sharing

Заголовок CORS существует уже много лет, и никто даже не подумает назвать его «новым». Вместе с тем он тесно связан со всеми другими «новыми» (выпущенными в 2021 году) заголовками безопасности. CORS говорит серверам, на которых хранятся активы, каким источникам (домен, порт и схема) разрешено эти активы загружать. CORS можно использовать, чтобы помешать посторонним загружать ваши активы, например запретить другим людям показывать на своих сайтах ваши изображения, защищенные авторским правом.

CORS определяет, как загружать активы в зависимости от типа запрашиваемого ресурса. Он либо делает ровно то, что вы попросили (например, загружает изображение), либо посылает запрос HTTP Options (чтобы объяснить сложную часть: например, если в запросе используется редко применяемый метод HTTP, скажем delete, trace или head), а вслед за ним оригинальный запрос, который вы имели в виду. Первый запрос, содержащий http options, называется *предполетным* и посылается лишь иногда (когда ваш запрос считается «сложным»).

Access-Control-Allow-Origin: domain-this-resource-can-be-shared-with

CORB: Cross-Origin Read Blocking

Заголовок безопасности CORB активирован по умолчанию в версиях браузера Chrome, начиная с 68. Другие браузеры пока что находятся в разных стадиях готовности. Он лучше всего известен среди немногочисленных злополучных разработчиков, которые получали леденящее душу сообщение «blocked resource».

Cross-Origin Read Blocking (CORB) blocked cross-origin response MY URL with MIME type application/json.

Этот заголовок блокирует запросы к ресурсам, которые запрашивались как изображения. Это значит, что ваше приложение не удастся обманом заставить принять JSON, HTML или XML как изображение (которого вы и ожидали), – без этого заголовка такое иногда возможно. Заголовок блокирует загрузку потенциально вредоносного содержимого, если оно отличается от того, что вы ожидали, поэтому атака пресекается на корню.

Этот заголовок защищает следующие типы содержимого: HTML, JSON и XML.

Чтобы заголовок работал правильно, нужно сконфигурировать заголовки CORS и CORP, а также установить x-content-type-options в режим no-sniff. Чтобы защитить эти три типа данных, CORS должен быть задан так, чтобы предотвращать загрузку из большинства или всех источников, т. е. параметр allow-origin может быть равен всему, кроме *. Пожалуйста, не используйте *; это не только сводит на нет все остальные меры защиты, но и открывает эти три типа данных для атак по побочным каналам.

X-Content-Type-Options: nosniff

и

Access-Control-Allow-Origin: URL-of-desired-origin | definitely-not-*

* В прежних версиях Chrome CORB либо активируется вместе с функцией изоляции сайтов, либо может быть активирован вручную путем запуска Chrome со следующим флагом в командной строке:

--enable-features=CrossSiteDocumentBlockingAlways

Безопасные cookie

Ниже мы очень кратко объясним различные параметры безопасности файлов cookie (подробнее они были рассмотрены в моей первой книге³⁷). Прошу вас, используйте их все.

Флаг **Secure**: гарантирует, что cookie будет передаваться только по протоколу HTTPS:

Set-Cookie: Secure; (плюс прочие параметры)

Флаг **HttpOnly**: запрещает доступ к cookie из JavaScript и тем самым уменьшает урон от XSS:

Set-Cookie: HttpOnly; (плюс прочие параметры)

Expires: определяет предельный срок хранения cookie, после которого он удаляется. Ниже установлено хранение до 1 января 2021 года:

Set-Cookie: Expires=Mon, 1st Jan 2021 00:00:00 GMT; (плюс прочие параметры)

Max-age: определяет предельный срок хранения cookie, после которого он удаляется. Ниже установлено хранение на 1 час:

Set-Cookie: Max-Age=3600; (плюс прочие параметры)

Примечание Используйте либо **Expires**, либо **Max-age**, но не оба сразу.

Domain: если вы хотите использовать cookie и в других доменах:

Set-Cookie: Domain=app.NOTshehackspurple.ca; (плюс прочие параметры)

Path: ограничивает доступ к cookie определенным частям (путям) вашего домена:

Set-Cookie: path=/YourApplicationsPath; (плюс прочие параметры)

Same-site: запрещает вашему приложению использовать файлы cookie из других доменов.

Set-Cookie: same-site=Strict; (плюс прочие параметры)

Предпочтительный режим

Set-Cookie: same-site=Lax; // блокирует только POST-запросы, разрешает ссылки, блокировка XSS

Совет Классифицируйте данные в файлах cookie, выделяя конфиденциальные. По возможности добавляйте метки и документируйте их. Чем более конфиденциальны данные в cookie, тем аккуратнее нужно с ними обращаться и тем скорее следует их удалять.

Обработка ошибок

Если вы читаете эту книгу и занимаетесь разработкой ПО, то, конечно, знаете, что ошибки приложения нужно «обрабатывать». Под «обработкой» программной ошибки понимается наличие кода, который реагирует на возникновение ошибки в программе или в системе и в идеале выдает сообщение пользователю, протоколирует событие и, если необходимо, посылает уведомление группам безопасности и (или) разработки.

Если мы не будем обрабатывать ошибки, то приложение окажется в неизвестном состоянии, что может привести к краху, зависанию, системным ошибкам, странному поведению и, возможно, уязвимости. Будучи разработчиком, ответственно относящимся к безопасности, вы хотите избежать падения в неизвестное состояние, потому что это именно то, за чем охотится любой злоумышленник. Именно тогда, когда приложение ведет себя неожиданно, возможна кража данных, расширение привилегий и другие страсти.

Например, если не «отловить» ошибку, то приложение может «упасть». Если в программе имеется предложение case без ветви else, перехватывающей все необработанные варианты, и все ветви when ложны, то приложение не знает, что делать дальше:

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN conditionN THEN resultN
    ELSE result
END;
```

Если ветви ELSE нет вообще или есть, но пуста, то приложение остается в растерянности, и это может привести к неожиданным результатам.

Писать код обработки ошибок для каждой функции или блока кода утомительно, но это поможет избежать проблем на этапе тестирования или, хуже того, после сдачи в эксплуатацию. Добавление глобального обработчика исключений гарантирует страховочную сетку на случай, если вы что-то упустили. Но вне зависимости от того, как спроектировали обработку ошибок, очень важно тщательно тестировать, чтобы ничего не пропустить.

Если ошибка возникла, когда выполнялась транзакция, обязательно откатите ее и начните заново. Частично выполненная транзакция – еще одна ситуация, где могут притаиться уязвимости.

Наконец, обрабатывая ошибки, никогда не позволяйте пользователю видеть сообщения, сгенерированные системой, а всегда заменяйте их собственными. Ошибки, полученные от базы данных или веб-сервера, могут содержать сведения, ценные для противника, например номер версии, уровень исправлений, синтаксис и другую системную информацию. В свое собственное сообщение включайте как можно меньше технических деталей – ровно столько, чтобы пользователь понял, что делать дальше.

Совет Всегда выключайте отладочный режим в производственной версии программы! Так вы случайно не выплеснете на экран распечатку стека и другие пугающие пользователя, но очень ценные для противника сообщения об ошибках.

На прошлой неделе Боб работал с программой, которая «грохнулась». Она показала ему системную ошибку и заставила вспомнить соревнование CTF, в котором он участвовал год назад. Его команда нашла способ заставить приложение выдать сообщения об ошибках SQL-сервера, введя мусор в поле пароля. Один из членов команды прекрасно разбирался в SQL и, увидев это сообщение, изменил атаку с внедрением SQL-кода. Он сказал, что сообщения об ошибках были обратной связью от сервера, позволившей ему исправить синтаксис. Благодаря сообщению команда смогла успешно пройти экран входа в систему и захватить флаг! И хотя Боб был рад победе, в глубине души он волновался насчет приложений в офисе. С тех пор он предельно ясно наставлял все команды, с которыми работал, никогда не показывать пользователям системные сообщения об ошибках – делиться такой информацией слишком опасно.

В этой главе мы только начали путешествие в мир безопасного кодирования и заложили фундамент, на котором возведем здание книги. Мы обсудили безопасный жизненный цикл разработки системы, использование современных каркасов и встроенных в них средств обеспечения безопасности, а также ценность таких подходов. Мы рассмотрели несколько типов базовых механизмов безопасности, включая проверку входных данных, кодирование вывода, аутентификацию, авторизацию и управление сессиями. Внедрив все, что вы узнали к этому моменту, вы встанете на путь создания достойных приложений! Но еще предстоит многому научиться, чтобы создавать надежные и безопасные программы. Вперед!

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Данные, поступающие в API непосредственно из вашего веб-приложения, проверять не нужно. Правда это или ложь?
2. Сколько раз пользователь должен входить в систему, реализующую SSO?

3. В каком порядке производятся авторизация и аутентификация? Почему порядок важен?
4. Назовите три типа секретов, которые вы стали бы хранить в программе управления секретами.
5. В современных браузерах политика Same-Origin активирована по умолчанию. Правда это или ложь?
6. В современных браузерах политика Cross-Origin-Opener активирована по умолчанию. Правда это или ложь?
7. Приведите две причины, по которым не следует выводить системные сообщения об ошибках прямо на экран.
8. Приведите две причины, по которым необходимо проверять ошибки и обрабатывать их в приложениях.

Улучшение

Теперь, когда вы вступили на путь становления безопасного кодировщика, давайте продолжим расширять свои знания. Мы расскажем о конфиденциальности (privacy), о технологиях, которые вы, вероятно, будете использовать (базы данных, файлы и память), и о том, как принимать решения, способные предотвратить атаки в будущем.

В этой главе мы подготовимся к защите от «среднего противника», а не от особенно продвинутого (например, государства). При работе с клиентами я, как специалист по безопасности, всегда хочу знать, к какому уровню безопасности они стремятся. В какой именно безопасности они нуждаются *на самом деле*? Обычно в начале беседы клиент говорит, что хочет максимально возможной безопасности, но это может оказаться чрезвычайно дорого и трудоемко. Мы же не хотим потратить миллион долларов на защиту того, что стоит всего несколько тысяч. И не хотим ограничивать свою свободу действий настолько, что едва можно пошевелиться, если только такой уровень безопасности не является абсолютно необходимым. Обычно через несколько минут мы приходим к пониманию того, что клиенту нужно, а затем начинаем планировать, как этого достичь.

Когда вы пишете приложения для своего работодателя, в идеале группа безопасности должна очертить требования к безопасности проектов, систем и данных. Если так оно и есть, то вы можете использовать информацию в этой книге, чтобы удовлетворить их требования. Если же по какой-то причине это не так, то, возможно, вам придется решать, какого уровня безопасности «достаточно». Попытайтесь использовать приведенные здесь сведения, чтобы наилучшим образом защитить создаваемые вами системы и данные, которые в них хранятся.

Безопасность баз данных

В информатике базой данных называется организованный набор данных, хранящийся в компьютерной системе в электронном виде. Для хранения, извлечения и изменения данных можно использовать систему управления базами данных (СУБД). Выражение «база данных» часто употребляется применительно к любой СУБД, системе баз данных или ко всему приложению, ассоциируемому с базой данных.

— «Википедия»³⁸

На тех из нас, кто работает в сфере информационной безопасности, возложена обязанность защищать (физическую и цифровую) информацию и информационные системы организации. Информационно-технологическая (ИТ) безопасность призвана защищать только цифровые активы, и в этой книге мы обычно говорим именно об ИТ-безопасности, а не о защите физических активов. Сфера ее ответственности распространяется на весь ИТ-департамент, все цифровые документы, сети, приложения и ассоциированные с ними базы данных. Каждый бит цифровой информации – наша забота, и нам необходима *ваша* помощь, чтобы защитить ее надлежащим образом. В этом разделе мы будем говорить о безопасности базы данных.

Совет Под *укреплением* понимается защита системы от вторжения. Это означает использование безопасных конфигурационных параметров: применение последних исправлений, отключение всех протоколов, служб и средств, которые не нужны для нормального функционирования системы, да и вообще любые меры, которые делают систему неприступной и безопасной. Для большинства систем имеются руководства по укреплению, которые можно запросить у поставщика. Следуйте приведенным в них рекомендациям!

Четыре взгляда на защиту баз данных

Прежде всего уточним, что базы данных – это находящиеся в сети серверы, на которых установлено программное обеспечение управления базами данных и хранятся данные.

Сначала поговорим о серверах. Нам нужно укрепить и просканировать сервер баз данных, убедившись, что все исправления безопасности установлены и конфигурация отвечает рекомендациям. Обычно поставщик, у которого вы приобретали программное обеспечение базы данных, предоставляет и руководство по укреплению. Впоследствии мы должны периодически сканировать сервер с помощью средства оценки уязвимости (часто оно называется VA-сканером), чтобы поддерживать сервер в безопасном состоянии. Эта обязанность, как правило, возлагается на группу безопасности.

Совет Чтобы избавить себя от необходимости устанавливать исправления и обслуживать свой сервер баз данных, можете арендовать управляемую базу данных у одного из публичных поставщиков облачных услуг.

Следующий шаг – изучить сеть, в которой находится сервер. Она публичная? Надеюсь, что нет! Есть ли в ней различные «зоны», защищенные брандмауэрами? Следует ли она модели «нулевого доверия»? Обычно этими вопросами занимаются совместно группа технического обслуживания сетей и группа безопасности.

Теперь мы опускаемся на уровень программного обеспечения баз данных, которым управляют администраторы баз данных (DBA), часто прибегая к советам группы безопасности (эти «безопасники» повсюду суют свой нос, не так ли?). ПО баз данных тоже можно сканировать с помощью инструментов безопасности, чтобы удостовериться в применении надлежащего укрепления. Подробнее прочитать о тестировании, сканировании и защите сервера баз данных можно в книге «Alice and Bob Learn Application Security», глава 6³⁹.

ПОБЕДНЫЕ ОТКАТЫ

Боб помнит время, когда он только начинал работать программистом в Канадском федеральном правительстве. На первом месте работы начальник настаивал, чтобы он создавал скрипт отката для любого скрипта, передаваемого DBA. Он обязан был прогнать свой скрипт на тестовой базе данных, затем откатить его и проверить, что база восстановлена в первоначальном состоянии. В то время Боб считал, что это перебор, но DBA в том учреждении были одними из лучших, с которыми ему доводилось работать. Они относились к своей работе очень серьезно, чем снискали его уважение. Боб продолжал придерживаться этой практики на протяжении всей своей карьеры программиста, и это не раз выручало его!

Последний шаг – дело для вас и группы безопасности. Для защиты данных в базе их нужно классифицировать и пометить, а затем обращаться с ними в соответствии с грифом секретности. Правила обращения с конфиденциальными данными распространяются на доступ, хранение, манипулирование и резервное копирование. Иногда эти правила также предписывают не собирать определенные данные, не выделять части данных и вообще не относиться к ним иначе, чем к прочим, в силу их классификации. Они также устанавливают частоту резервного копирования данных, условия отката (который также называют *восстановлением*) и даже сроки удаления данных. В резервном копировании данных мало толку, если вы хотя бы один раз не попробовали восстановиться из копии. Правила также описывают, нужно ли хранить конфиденциальные данные в отдельной таблице, в зашифрованном или токенизированном виде и т. д.

Перечислим еще некоторые важные способы защиты данных.

- Данные должны шифроваться в процессе передачи и при хранении. Используйте последнюю доступную версию TLS и придерживайтесь определенных в вашей организации предписаний в части длины клю-

ча и допустимых алгоритмов шифрования. Примечание: не пытайтесь писать собственные методы шифрования.

- Включите средства протоколирования и мониторинга, чтобы можно было узнать, какие запросы предъявлялись базе данных.
- Создавайте роли доступа с минимальными привилегиями, например роли, допускающие только чтение (там, где возможно), или роли, дающие доступ только к определенным таблицам.
- Учетная запись, от имени которой производится доступ к базе данных, должна иметь длинный и уникальный пароль. По возможности применяйте двухфакторную аутентификацию (иногда употребляются акронимы 2ФА или МФА).
- Некоторые поставщики облачных услуг разрешают временные учетные данные без пароля (например, роли управления идентификацией и доступом [IAM]); это еще один безопасный вариант.
- Когда ваши приложения и другие ИТ-системы обращаются к базе данных, они должны использовать учетную запись службы (сетевой идентификатор, уникальный в данной системе), а не физического лица. Пароли для этой учетной записи должны храниться в инструменте управления секретами, а доступ к ним должен производиться из программы без участия человека.
- Не храните информацию, принадлежащую базе данных, вне этой базы. Не делайте копий для себя или для других, если это не является проектным требованием. Никогда не храните копии на своей локальной машине.
- Не используйте копию производственных данных ни в какой другой среде (разработки, контроля качества, приемочного тестирования и т. д.). Производственные данные не должны использоваться для контроля качества, тестирования, разработки – вообще ни для чего, кроме работы в производственном режиме. Вместо этого используйте замаскированную версию данных или генерируйте выборочные данные для рабочих сред.
- Никогда не делитесь ни с кем ни под каким предлогом паролями или учетными записями.
- Не просматривайте данные в производственной базе. Разработчики не должны видеть данные заказчиков, если это не входит в их служебные обязанности. Это вопрос конфиденциальности, а не безопасности.
- Не оставляйте компьютер разблокированным и подключенным к базе данных в свое отсутствие. Это вроде бы должно быть очевидно, но упомянуто здесь не без причины.

Совет Под *маскированием* понимается перемешивание. Информация та же самая, но в поля одной записи помещены значения из другой записи. Если первоначально запись содержала поля

First Name = Tanya, Last Name = Janca, DOB = April 1, 1980

то после маскирования она могла бы стать такой:

First Name = Tanja, Last Name = Reznor, DOB = Jan 1, 1999

Это уже не запись о реальном человеке, но тестировать систему можно, потому что данные не мусорные, форматирование в полном порядке. Но информация не точна, а именно это нам и нужно, чтобы помешать использовать реальные производственные данные в неблагоприятных целях.

Совет Процедура маскирования данных должна исключать обратное проектирование с целью идентификации человека; это значит, что никакая идентифицирующая личностная информация не должна остаться на своем месте. Меняйте все поля, а не только некоторые.

АХ, ЕСЛИ БЫ МЫ ТЕСТИРОВАЛИ ПРОЦЕДУРУ ОТКАТА!

Много лет назад в компании, где работала Алиса, произошла крупная авария на сети, и всех отправили по домам. Ведь без компьютерной сети и доступа к интернету работать все равно было невозможно. На следующий день ИТ-департамент сообщил, что из-за какого-то «глюка» было потеряно три дня работы всей компании! Руководитель ИТ сказал Алисе, что для восстановления изменений, произведенных за три дня во всех личных системах и общих папках, потребуется примерно месяц. Алиса была вне себя! Она спросила, почему так долго, и руководитель объяснил, что они никогда раньше не пробовали восстанавливать данные из резервной копии и применять процедуру отката. Они начинали с чистого листа и не были уверены, что вообще что-то получится. Алиса рассвирепела: «Зачем вообще делать резервные копии, если из них нельзя восстановиться? Зачем платить людям, которые делают неработающие копии? Почему не уволить их и не сэкономить деньги?» После таких заявлений Алису почему-то перестали любить в ИТ-департаменте...

БОБ ДАЕТ ВЗЯТКИ

Раньше программистам Боба приходилось отправлять заявку со своими SQL-скриптами группе DBA, чтобы те выполнили их в производственной системе. Иногда это занимало много времени, Боб видел, что администраторы уходят на обед, и очень расстраивался из-за необходимости ждать. Как-то раз Боб лакомился печеньками в столовой. Увидев это, DBA сказал: «Я бы тоже съел печенюшку». Боб, конечно, поделился, и неожиданно его заявка оказалась в начале очереди и была выполнена немедленно. С тех пор Боб стал держать коробку с печеньками в ящике стола, чтобы на выполнение его скриптов уходили часы, а не дни, пусть даже подкуп – это «неправильно».

Управление файлами

Ваши файлы – желанная добыча для злоумышленников. Файлы на вашем веб-сервере, на других сетевых серверах, на USB-флешках, на резервных дисках, на вашем личном компьютере, в облаке и даже файлы, хранящиеся в базе данных. Изображения, скрипты, служебные записи, электронная почта, видео, рукопись вашей следующей книги. Да назовите что угодно – и, скорее всего, найдется кто-то желающий это украсть, изменить или вернуть вам за выкуп.

Для создаваемых нами программ, особенно подключенных к интернету, это потенциальный вектор атаки. Даже программы, которые не дают прямого доступа к файлу, будучи плохо написаны, могут непреднамеренно открыть путь к недостаточно защищенным файлам в вашей сети. В этом разделе мы обсудим, как защитить файлы, которые являются частью ваших приложений или хранятся на ваших веб-серверах. Предполагается, что защитой файлов, не связанных с вашими программами и недоступных из них, занимается группа безопасности.

Примечание Отдельное замечание о вашем коде: ваш исходный код имеет огромную ценность, и вы должны защищать и его тоже! Всегда сохраняйте изменения в системе управления версиями (или репозитории) и соблюдайте соответствующие инструкции, действующие в вашей организации. Если это закрытый, проприетарный код, за написание которого вам платят, то не храните его копии в других местах, в том числе в публичных репозиториях, на своем домашнем компьютере и т. д. Это интеллектуальная собственность, и конкуренты вашей компании прыгали бы от радости, обнаружив копию там, где ее быть не должно.

Прежде всего вы должны использовать какой-то вид управления доступом к файлам, если они размещены на вашем веб-сервере, файловом сервере или управляемом сервере или если к ним имеется доступ из вашего веб-приложения или через API. Все файлы на веб-серверах и файловых серверах должны быть доступны только для чтения и не иметь права выполнения. В идеале на веб-серверах и файловых серверах также должны быть установлены инструменты управления приложениями (раньше они назывались белыми списками приложений) и мониторинга целостности файлов. Проконсультируйтесь по этому поводу с группой безопасности!

Файлы, полученные от пользователей (например, фотография для профиля) в идеале следует хранить не на веб-сервере, а на файловом сервере, защищенном брандмауэром (на случай, если файл окажется опасным). Прежде чем сохранять файл на сервере, вы должны просканировать его и, если он выглядит подозрительно, отклонить. Кроме того, именно ваше приложение должно выбирать имя, местоположение, тип файла (должен быть список разрешенных расширений файла, все остальные отвергаются) и другие связанные с файлом детали. Файл должен быть помечен грифом секретности. Следует также ограничивать размер файла и проверять, что указанный пользователем тип файла совпадает с реальным⁴⁰. О том, как обеспечить безопасность загружаемых файлов, многое можно прочитать в документе OWASP File Upload Cheat Sheet (https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html).

Примечание Центр безопасности коммуникаций Канады (Communications Security Establishment of Canada – CSE) создал бесплатный инструмент с открытым исходным кодом для проверки загруженных файлов на вредоносность, он называется AssemblyLine. Он не передает никакой информации ни правительству Канады, ни какой-либо другой организации. Можете скачать его по адресу <https://cyber.gc.ca/en/assemblyline>.

Помимо всего прочего, группа безопасности должна вести мониторинг доступа к файлам, регулярно создавать резервные копии, следить за тем, чтобы документы передавались и хранились в зашифрованном виде⁴², разрешать доступ к файлам только аутентифицированным и авторизованным учетным записям пользователей и запрещать копирование файлов за пределы сервера. В идеале должна быть установлена система управления доступом к файлам, отвечающая всем этим требованиям. Эпизодические аудиты также повышают степень уверенности в безопасности, поскольку могут вскрыть какие-то ошибки или аномалии.

Загрузка файлов на сервер

Загрузка файлов подробно рассматривалась в книге «Alice and Bob Learn Application Security», глава 4 «Безопасный код». Но поскольку это чрезвычайно рискованная операция, поговорим о ней еще раз!

Если программа разрешает загружать файл, то пользователь может загрузить *все что угодно*. Вредоносную программу, zip-бомбу (файл, при распаковке которого создается другой zip-файл – и так рекурсивно, пока сервер не «упадет»), противозаконные изображения, защищенный авторским правом материал и т. д. Если вы сначала аутентифицируете и авторизуете пользователей (заставляя их выполнить вход в приложение), то шансы на загрузку чего-то вредоносного значительно уменьшаются, но все равно такое случается. Разрешая загрузку файлов, нужно принимать меры предосторожности.

Самый простой способ сделать это безопасно – воспользоваться сторонним плагином, библиотекой или инструментом, который хорошо зарекомендовал себя и тщательно проверен на безопасность. Если вы поищите в сети или просканируете его средством анализа состава программы (software composition analysis – SCA), то часто сможете сказать, есть ли в библиотеке известные уязвимости. Точно так же вы должны проверять все библиотеки, которыми пользуетесь в своих приложениях (мы будем говорить об этом в третьей части книги).

Если вам *необходимо* написать свой собственный инструмент, то соблюдайте следующие рекомендации.

- В идеале ограничьтесь только определенными типами файлов. По возможности избегайте файлов типа PDF и EXE; тогда проблем у вас побавится.
- Проверяйте тип файла (и не используйте для этой цели расширение, а читайте заголовок файла) и его размер, прежде чем сохранять.
- Перед сохранением сканируйте файл на вредоносность. Подумайте об использовании программы AssemblyLine (<https://cyber.gc.ca/en/assemblyline>).

- Не позволяйте пользователям определять местоположение файла. Решать должны вы, а не пользователь.
- Переименуйте файл, не оставляйте имя, указанное пользователем.
- Сохраняйте файл на файловом сервере, а не на веб-сервере, в контейнере или PaaS-системе.
- Позаботьтесь о том, чтобы сервер, на котором сохраняются файлы, не имел привилегий выполнения, подвергался регулярному резервному копированию и мониторингу и был окружен брандмауэром (не выпускающим вредоносные файлы за пределы сервера).
- Резервные копии также должны храниться на сервере, допускающем только чтение, с аналогичным брандмауэром.
- Ведите мониторинг файлового сервера и его резервной копии на предмет неразрешенного доступа или аномального поведения.
- Дополнительные сведения см. в книге «Alice and Bob Learn Application Security», глава 2 «Загрузка файлов», и в документе OWASP File Upload Cheat Sheet. Шпаргалки OWASP всегда являются отличной отправной точкой.

Ваш исходный код

Как уже было сказано, необходимо защищать также свой исходный код. Его могут украсть, переместить не туда или каким-то иным образом нарушить ваши права на интеллектуальную собственность. Проводя аудит безопасности приложений в разных компаниях, я не раз обнаруживала код в публичных репозиториях на GitHub, код, единственная копия которого хранилась на чьем-то персональном компьютере (и была стерта), код, который редактировался прямо на производственном сервере (без создания копии где-либо еще), и, хуже того, исходный код, который так никогда и не был восстановлен после утраты, так что у владельцев оставался только двоичный файл.

Чтобы защитить свой исходный код, вы должны делать в основном то же, что и для защиты файлов: обязательно поместить его в систему управления версиями, настроить управление доступом, создавать резервные копии, вести мониторинг доступа к коду и (по возможности) настроить уведомления о необычном поведении. Группа безопасности может сделать и больше, например мониторить всю сеть на предмет странного утекания из нее данных (это называется *экспфильтрацией*), заводить файлы-канарейки (ложные файлы, которые «звонят домой», если украдены), оплачивать работу сторонних служб, сообщающих вам о том, что ваш исходный код украден, и многое другое.

Совет Если вы защищаете мобильный исходный код, то должны также минимизировать и обфусцировать его, чтобы защитить от обратного проектирования. Код для мобильных приложений работает на устройстве, которое контролируется пользователем, а не всякий пользователь уважает ваше желание держаться подальше от вашего исходного кода. Подробнее об этом в главе 5!

Совет Если вам поручена защита приложения, для которого в наличии только двоичный файл (и нет исходного кода), то все равно имеются кое-какие варианты! Можно использовать инструменты статического анализа первого поколения (они выполняют символические вычисления, а не анализ потока и сравнение с антипаттернами), которые называются также SAST, чтобы оценить степень опасности файла, а затем поставить перед ним брандмауэр веб-приложений (WAF) и выделить в отдельный сегмент сети (что-то вроде рва вокруг замка). Это защитит его от очевидных атак, уменьшит ущерб остальной сети в случае, если когда-то файл будет успешно атакован, и позволит оценить, какой риск он представляет для организации.

УТРАЧЕННАЯ РАБОТА

Когда Алиса руководила командой разработчиков, у руководителя другой группы, Чеда, было в подчинении 11 консультантов, которые создавали формы для SharePoint. Команда Алисы настроила Team Foundation Server (TFS), на котором группа Чеда должна была хранить весь свой код. Группа Чеда создала свою собственную сеть, чтобы не следовать правилам компании в области ИТ (это называется *теневым ИТ*). Алиса говорила, что так нельзя, но Чед все равно сделал по-своему. Чед говорил, что будет делать резервную копию систем каждую ночь, поэтому «Не волнуйся, все будет нормально». Но однажды Алиса увидела, как Грейс (руководитель еще одной группы) идет мимо ее комнаты и тащит большой сервер. «Грейс, извини, это что?» Грейс ответила, что это TFS-сервер, который Алиса подготовила для Чеда, что он несколько недель был отключен от сети и, поскольку никто его не использовал, она его отформатировала! Алиса побежала к Чеду, который сказал, что несколько недель был в офлайне, но не хотел ее беспокоить. Она спросила, когда он последний раз делал резервную копию, и Чед признался, что до этого «ни разу руки не дошли». И хотя какие-то части удалось восстановить из копий на локальных машинах, большая часть работы, которую 11 консультантов делали в течение 5 месяцев, была утрачена. Алиса была вне себя!

Управление памятью (переполнение буфера, стека, строки и целого числа)

Первое собрание OWASP, которое я посетила, было посвящено семинару по переполнению буфера⁴³. Я уже 14 лет как окончила колледж и успела позабыть, как работает стек и управление памятью. В современных языках и каркасах памятью управляют *за вас*, но программировать я училась на языках C и C++, где эту работу приходится делать самостоятельно. Я должна была вручную освобождать память и убирать мусор и столько намучилась с этим, будучи подростком, что поняла, что хочу иметь дело только с языками более высокого уровня, которые управляют памятью за меня.

Переполнение означает выход за пределы области, выделенной для переменной. Переполниться может буфер, строка, целое число или стек.

Предположим, что вы создали поисковое поле и отвели под переменную 5 символов. Если пользователь введет шесть символов, то куда денется лишний? В языке с небезопасным управлением памятью он окажется в остальной памяти компьютера! Но в остальной памяти хранится и работающая

программа. Разумеется! Все команды, исполняемые компьютером, также находятся в памяти. На рис. 3.1 показано, как это работает.

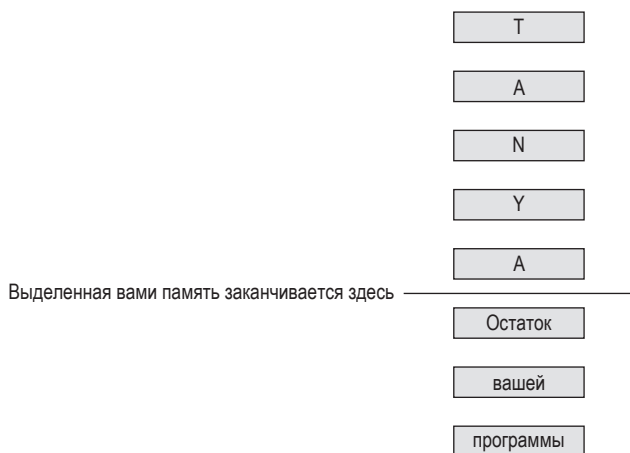


Рис. 3.1. Правильно сохраненная переменная

Когда переменная переполняется, лишние данные попадают в остальную память. Если при этом была изменена часть памяти, в которой находится указатель стека, то программу можно перенаправить в другое место. Указатель стека говорит компьютеру, где искать следующую команду. Компьютер переходит по адресу в указателе стека и выполняет команды, пока программа не завершится или не окажется в непредсказуемом состоянии с поврежденной памятью (программа «падает»).

Когда злоумышленник эксплуатирует переполнение буфера, он намеренно перезаписывает указатель стека. Он помещает в память свои команды, заставляя компьютер их выполнить, а эти команды как раз и являются атакой. Подсунутые злоумышленником команды называются *шелл-кодом* и могут быть использованы для перехвата управления веб-сервером. Это значит, что противник может выполнять любые команды на веб-сервере, и работать они будут с теми привилегиями, которые веб-сервер предоставил вашему приложению (что позволит ему читать и изменять информацию и другие приложения, хранящиеся на устройстве). Атакующий может делать все, что его душе угодно, причем удаленно. Такая ситуация называется удаленным выполнением кода (*remote code execution – RCE*), и это самое страшное, что может случиться с веб-приложением. На рис. 3.2 показано, как это работает.

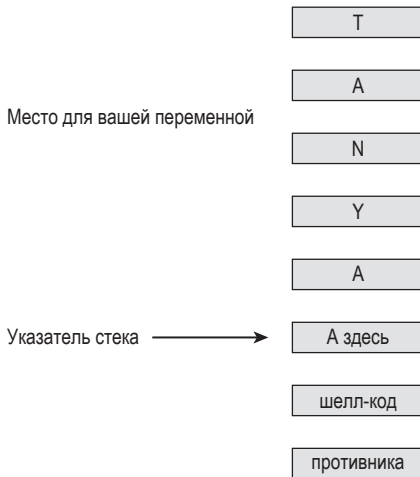


Рис. 3.2. Неправильно сохраненная переменная

Как избежать переполнения?

Теперь, узнав о переполнениях и связанных с ними уязвимостях, вы, наверное, хотите любой ценой избежать их. Самый простой способ – избегать языков программирования и каркасов, лишенных средств безопасного управления памятью. Например, используйте Rust вместо C и C++.

Что делать, если выбора нет? Например, если имеется унаследованное приложение, написанное на C или другом небезопасном языке, и вы должны с ним работать. А времени, денег или разрешения руководства переписать его нет, так что приходится мириться. Давайте рассмотрим некоторые рекомендации.

Предотвращение переполнений

- Проверяйте все входные данные. Проверяйте границы всех переменных.
- Тщательно тестируйте потенциальные ошибки управления памятью.
- Проводите критический анализ кода, уделяя особое внимание проблемам при работе с памятью.
- Применяйте фаззер для тестирования всех возможных входных данных.
- Избегайте функций, которые не выполняют проверку выхода за границы буфера (например, в C используйте `fgets()` вместо `gets()`)⁴⁴. Дополнительные советы на эту тему см. в табл. 3.1⁴⁶.
- Или используйте автоматическое средство, которое выполняет вместо вас проверку выхода за границы буфера.
- Используйте управляемые буферы и строки вместо простых массивов и указателей (см. примеры в табл. 3.1 ниже)⁴⁵.

- Пользуйтесь неисполняемым стеком (т. е. механизмом предотвращения выполнения данных [DEP]). В этом случае стек, а иногда и другие структуры данных помечаются как области, в которых запрещено выполнять код. Это значит, что злоумышленник по-прежнему может внедрить код эксплойта в стек, но выполнить его не сумеет⁴⁶.
- Пользуйтесь операционной системой с рандомизацией распределения адресного пространства (address space layout randomization – ASLR). Это значительно уменьшает риск переполнения буфера; злоумышленник не сможет перезаписать стек и перехватить управление вашей программой⁴⁶.
- Вставляйте в стек перед адресом возврата канареечные значения⁴⁴ и следите за тем, чтобы они не перезаписывались. Если они перезаписаны, то программа останавливается.

Таблица 3.1. Более безопасные варианты

Предпочитайте	Избегайте
<code>std::string</code>	<code>char*</code>
<code>std::vector</code>	<code>int array[15]</code>
<code>std::cin</code>	<code>scanf</code>
<code><iostream></code>	<code><cstdio></code>
<code>Strlcpy, strcpy_s</code>	<code>strcpy</code>
<code>Strlcat, strcat_s</code>	<code>strcat</code>
<code>snprintf/sprint_s</code>	<code>printf/sprintf</code>
<code>fgets</code>	<code>gets</code>

ОШИБКА НА ЕДИНИЦУ

В школе Боб занимался программированием и написал карточную игру на С. Его программа все время падала, он распечатал ее и показал учителю, мистеру Фергюсону. Оказалось, что он начал счетчик 52 карт с 1, а не с 0, и когда доходил до последней карты, программа падала. Спустя несколько дней учитель вернул 10 страниц кода Бобу и объяснил, в чем ошибка. Боб был счастлив иметь учителя, который с такой страстью относился к преподаванию программирования, что мог выдержать 500 вопросов Боба. До окончания школы Боб превзошел со своим любимым учителем все три уровня программирования.

Сериализация и десериализация

Сериализация – процесс преобразования объекта в поток байтов для его хранения или передачи в память, базу данных или в файл⁴⁸. Обратный процесс называется *десериализацией*.

Сериализация позволяет сохранить состояние объекта, а затем воссоздать его точно в таком же виде после сохранения или передачи. Объект может быть чрезвычайно сложным, и если его нужно передать в другой домен с помощью веб-службы, сквозь брандмауэр, то сериализация гарантирует, что он не изменится в пути. Существует много ситуаций, в которых программисту необходимо сериализовать объект. JSON и XML наиболее распространенные форматы сериализации, хотя это можно сделать и в .NET, и в других языках и каркасах.

К сожалению, процесс десериализации может быть опасным. Десериализуемый объект может содержать вредоносный код, который затем атакует систему, в которой был десериализован. Если злоумышленник сумеет подменить отправленный вами объект чем-то вредоносным, то вам не поздоровится. Ситуация, когда сериализованный объект принимается из источника, который вы не контролируете, порождает уязвимость, называемую *небезопасной десериализацией*.

Как вы, наверное, догадались, простейший способ избежать десериализации – вообще никогда ничего не сериализовать. Однако же эта процедура в ряде случаев полезна, поэтому посмотрим, как можно снизить риски.

Если вам приходится принимать сериализованные объекты, то позаботьтесь о том, чтобы злоумышленник не мог их подменить своими. Это значит, что сначала объект нужно зашифровать или подписать, затем передать по месту назначения, там дешифровать или проверить подпись, а потом десериализовать. Если вы шифруете объект своим собственным ключом, то должны быть уверены, что он не подвергался манипуляциям. Если вы проверяете подпись, то должны точно знать, откуда она взялась, и быть уверены в ее целостности.

Однако если объект отправлен из места, которое вы не контролируете, и если есть возможность, что кто-то мог внести в него изменения в процессе передачи, или если он получен от пользователя, или если у вас нет возможности зашифровать и подписать его самостоятельно, а затем дешифровать или проверить подпись, то вы оказываетесь в ситуации, когда нужно принимать значительно больше мер предосторожности. Это означает, что сериализованный объект поступил из *ненадежного источника*, и вы попали в ситуацию небезопасной десериализации.

Подробно о том, как защищать приложение в случае приема сериализованных объектов из ненадежных источников, см. в книге «Alice and Bob Learn Application Security», глава 5, «Десериализация»⁴⁸, а также в шпаргалке OWASP по небезопасной десериализации⁴⁹.

Конфиденциальность (пользователя, гражданина, заказчика, работника)

При создании программного обеспечения обычно вам вручают список требований, чтобы вы знали, что делать. Но некоторые требования подразуме-

ваются неявно, а не сформулированы явно, что может приводить к путанице. Часто такие вопросы, как требования к конфиденциальности и безопасности, подразумеваются, а это значит, что они не обязательно соответствуют ожиданиям пользователей, клиентов или заказчиков. Чтобы не оказаться в такой ситуации, лучше заранее и явно формулировать, чего вы хотите, причем как можно детальнее.

В последние годы конфиденциальность* вышла на передний план во многих технологических дискуссиях. Потребители и общественность хотят знать, что мы (технологические компании) делаем с их данными. Хотя я писала об этом в книге «Alice and Bob Learn Application Security», думаю, что тема заслуживает дополнительного обсуждения.

Уединение: состояние, когда человек находится один, за ним никто не наблюдает и никто не прерывает.

— Оксфордский словарь⁵⁰

Давайте уточним это определение. Находясь в онлайне, мы можем сидеть в комнате в одиночестве, но если наши персональные данные в это время где-то сохраняются и продаются, а за нашим поведением наблюдают и (или) нас прерывают бесконечными всплывающими окнами и уведомлениями насчет файлов cookie, то это уединение или все-таки нет? По мне, так нет.

Уединение – это о самоопределении и достоинстве. Не только о данных.

— Абигайл Дубинецки⁵¹

Уединение – это также о способности делать выбор наедине с собой, не подвергаясь притеснениям, соблазнам, унижениям и даже насилию. Этот аспект никогда не был виден в обществе так отчетливо, как сейчас, когда большие онлайн-группы ненавистников повсюду используют различные технологии для преследования, издевательств, избиения и даже убийства тех из нас, кого они считают «неприемлемыми». Уважая право наших пользователей на неприкосновенность частной жизни, мы также защищаем их жизнь.

Примечание Будучи разработчиком ПО, вы можете возразить, что если заказчик требует добавить в программу функциональность, которая нарушает конфиденциальность пользователей, то, выполняя его требование, вы «просто делаете свою работу». Да, вы выполняете свою работу. Но *вместе с тем* вы соучаствуете в нарушении права поль-

* Наиболее точный перевод слова *privacy* (если не считать кальки «приватность») является и самым длинным – «неприкосновенность частной жизни». Краткости ради в основной части текста он заменяется неточным, неоднозначным, но принятым в информационной безопасности термином «конфиденциальность». Но в приведенном ниже определении из Оксфордского словаря речь идет об обиходном значении слова. Хочется надеяться, что читатель по контексту поймет, о чем идет речь. – *Прим. перев.*

зователя на неприкосновенность частной жизни. Если вы читаете эту книгу, то, вероятно, безразлично относитесь к безопасности и, наверное, имеете какое-то мнение о конфиденциальности. Я не предлагаю вам уволиться и сжечь здание (пожалуйста, воздержитесь от поджога), а пытаюсь убедить вас в том, что, быть может, имеет смысл поупираться и объяснить заказчику, как его желание может отразиться на конечных пользователей. Иногда легкое подталкивание или просто открытое высказывание своего мнения на собрании может полностью изменить ход проекта.

Данные пользователя, гражданина, заказчика и даже работника могут быть персональными и (или) конфиденциальными по своей природе. Персональная идентифицирующая информация (PII) – это данные, которые могут идентифицировать пользователя и упростить злоумышленникам кражу его личности. Персональные медицинские данные (PHI) – это сведения о состоянии здоровья, которые могут быть использованы для увеличения страховых платежей, препятствования в получении работы, преследований, шантажа и даже доведения до самоубийства. Многие типы информации не представляют опасности сами по себе, но в сочетании становятся угрозой.

Говоря о конфиденциальности, следует также помнить, что сегодня потребители пекутся о ней больше, чем когда-либо прежде. И хотя доля пользователей, готовых отказаться от продукта или услуги из-за проблем с конфиденциальностью, все еще мала, каждый год она возрастает. Это удар по вашему продукту сравнительно с другими, и, пожалуй, это не стоит того, чтобы рисковать репутацией в не столь уж отдаленном будущем.

Совет Многие потребители не до конца понимают последствия различных продуктов и сопровождающих их услуг для конфиденциальности. Поскольку мы создаем эти продукты, наша обязанность – учитывать их последствия для прав пользователей и защищать тех, кто не понимает, на что соглашается.

НЕОЖИДАННОЕ ОБОБЩЕНИЕ

Когда Боб учился в колледже, он и три его соседа по общежитию часто пользовались одним компьютером для выхода в интернет. Принадлежал он Бобу, а у остальных своих компьютеров не было, поэтому они пользовались им вместе. Однажды один из приятелей решил просмотреть кеш, когда Боб вышел из комнаты. А потом начал задавать Бобу всякие неудобные вопросы о том, что тот делает в интернете. Боб сказал соседу, что просматривать чужой кеш неприлично и не его дело, чем Боб занимается в интернете! Он указал бывшему приятелю на дверь, а затем уселся и сам стал просматривать кеш. Боб был удивлен тем, что увидел: спортивные сайты, группа «Nine Inch Nails», фестивали еврейских фильмов, всевозможный «контент для взрослых» – чего там только не было. С тех пор он всегда включает автоудаление кеша и ставит на компьютер пароль. Больше никакого обобщения!

Совет Не собирайте данные, которые вашей системе *не требуются*! Это не только уменьшит поверхность атаки, но и защитит конфиденциальность пользователей. В такой сделке нет проигравших! Если вы хотите больше узнать об этом предмете, поинтересуйтесь инженерией конфиденциальности (Privacy Engineering)⁴¹, новой важной отраслью информатики и управления информацией.

Ошибки

Некоторые ошибки являются также инцидентами безопасности. Иногда сочетание ошибок или повторение какой-то ошибки заданное число раз или ошибка, повторяющаяся многократно в течение короткого промежутка времени, может *стать* инцидентом безопасности. Умение определить, что является, а что не является инцидентом безопасности, отличает опытного сотрудника от начинающего. Чтобы познакомиться с этой тематикой, прочитайте этот раздел, а затем обсудите с группой безопасности моделирование угроз (подробнее об этом мы будем говорить в третьей части книги). Это позволит вам научиться уверенно выявлять потенциальные инциденты безопасности.

Примечание Говоря о «состоянии» приложения, мы имеем в виду множество переменных, описывающих поведение системы в конкретный момент времени⁵². Говорят, что система *обладает состоянием*, если она умеет запоминать предшествующие события или взаимодействия с пользователями; запомненная информация и составляет *состояние* системы⁵³. Приложение находится в *неизвестном состоянии*, если программист не учитывал возможность такой ситуации при проектировании и кодировании, т. е. приложение не знает, что делать дальше. В таких случаях приложение ведет себя неожиданно, что часто ведет к уязвимостям и другим проблемам с безопасностью.

Любое приложение должно «отлавливать» и «обрабатывать» ошибки. Если мы не «отлавливаем» ошибки, то приложение может аварийно завершиться, зависнуть или перейти в неизвестное состояние, что может стать причиной его непредсказуемого поведения. Специалисты по безопасности ни в коем случае не хотят наблюдать неизвестное состояние приложения – разве что при тестировании на безопасность или исследовании уязвимостей, потому что именно так многие уязвимости и обнаруживаются. Отлавливание ошибок гарантирует, что вы не утратите контроль над программой, предоставив обработку ваших ошибок базовому серверному ПО.

Отловив ошибку, ее нужно *обработать*. Обработка подразумевает, что вы принимаете решение, что делать в данной ситуации, а затем проводите свои решения в жизнь. Обычно это сводится к отправке пользователю сообщения об ошибке (которое ни в коем случае не должно совпадать с системным сообщением), после чего приложение выполняет действия, необходимые для выправления ситуации. Если приложение находилось в процессе выполнения транзакции, то обычно ее следует откатить и начать заново. Если оно пыталось отправить форму, то следует восстановить предыдущее состояние формы. Решение о том, что делать в случае ошибки, принимать вам и клиенту, в идеале оно должно еще следовать инструкциям по безопасному кодированию, написанным вашей группой безопасности.

Совет Большинство ситуаций, в которых ошибки становятся проблемами безопасности, можно избежать, установив жесткие ограничения на темп в каждой части приложения. Если мы предотвратим возникновение проблемы, то и не нужно будет поднимать тревогу.

Ниже приведены условия, при которых ошибки становятся проблемами безопасности⁵⁴.

- Кто-то пытается войти чаще, чем 10 раз за 10 с или 100 раз в минуту. Человек так быстро печатать не может. Вы можете ослабить или усилить это ограничение.
- Кто-то раз за разом некорректно обращается к вашему API. Какое количество раз и какая частота являются основанием для тревоги, решать вам и владельцу продукта.
- Кто-то повторно обращается к вашему API с запросами неправильного формата, но при каждом обращении формат немного изменяется. Это может быть злоумышленник, перебирающий ваши API (чтобы найти их все) или пытающийся вручную вызвать API в качестве составной части атаки или подготовки к ней.
- С одного и того же IP-адреса в течение очень короткого времени создается несколько новых учетных записей в одной и той же системе.
- С одного и того же IP-адреса пытаются создать несколько учетных записей с разными адресами электронной почты, и ни один адрес не верифицирован.
- С одного и того же IP-адреса пытаются несколько раз изменить пароль, но по ссылке восстановления пароля ни одного перехода не было. Похоже на попытку взломать чужую учетную запись.
- Платеж несколько раз подряд не проходит. Похоже, что кто-то украл кредитную карту и теперь пробует подобрать ПИН-код наудачу. В любом случае это проблема.
- Кто-то неоднократно пытается перейти на несуществующие страницы. Несколько ошибок HTTP 400 подряд пахнут перебором каталогов.
- Попытки загрузки файлов отклоняются вашими защитными механизмами и проверками. Если так происходит для нескольких файлов подряд, есть повод насторожиться.
- Любой случай отката транзакции заслуживает протоколирования, но если пришлось откатываться два или три раза, то, возможно, имеет место инцидент безопасности или какая-то атака.
- Если вы пытались ротировать или обновить идентификатор сеанса (поскольку он существует уже давно и клиенту необходим новый), но клиент все равно подсовывает вам старый идентификатор, или если приложению подсовывают идентификатор сеанса, который вы никогда не выпускали, то, очевидно, имеет место проблема безопасности.
- Повторяющиеся ошибки сервера в ответ на действия пользователя или несколько разных ошибок в диапазоне 400 или 500. Быть может, это не значит ничего, а быть может, является признаком каких-то злонамеренных действий. Ошибки HTTP-сервера, возникающие больше одного раза, – повод для расследования.

- Необоснованные всплески потребления процессора или памяти. Особенно если вы не проводили интенсивного нагрузочного тестирования и (или) тестирования производительности и не было никаких внешних событий, объясняющих всплески (например, поступления в продажу билетов на концерт популярной группы, дня выборов или того факта, что вы известный журналист, в который раз прогневавший группу хактивистов).
- Ошибки аутентификации при доступе к базе данных, к API и т. п., которые вообще-то должны работать нормально. Если ваше приложение не может аутентифицироваться в других системах, налицо проблема.
- Любые аномалии, которые кажутся странными, необычными или неожиданными, заслуживают изучения. Быть может, поднимать тревогу и не стоит, но по меньшей мере их следует запротоколировать и поделиться своими находками с командой.

Приведенный выше список не исчерпывающий.

Ошибки, протоколирование и оповещение должны являться неотъемлемой частью системы⁵⁴. В идеале должна быть одна служба для протоколирования, ошибок и оповещений или три разные системы (по одной для каждого компонента). У одного приложения не должно быть нескольких систем обработки ошибок; все они должны обрабатываться единообразно, в идеале одними и теми же функциями. Анализ проекта и кода помогут гарантировать, что все члены команды действуют по одному и тому же плану и используют одинаковые функции для обработки ошибок. Когда все единообразно, жизнь разработчиков и «безопасников» заметно упрощается.

В журналы никогда не следует помещать конфиденциальную информацию, в том числе РП, РНП, полные записей пользователей, полные идентификаторы сеансов (для отладки достаточно последних четырех цифр) и т. д. Если журналы попадут в руки злоумышленника, они не должны дать ему материал для последующих атак.

Совет Экспортируйте системные исключения, пользуясь внешним средством прослеживания ошибок, например Bugsnag или Rollbar, чтобы знать, сколько было исключений конкретного типа, и мониторить новые по мере поступления.

При проектировании системы протоколирования важно тестировать отвечающую за это часть приложения на устойчивость к *внедрению в журнал* – уязвимости, позволяющей злоумышленнику помещать в журнал записи по собственному усмотрению или вставлять в журналы вредоносное содержимое⁵⁵.

И последнее замечание об ошибках: в любом приложении должен быть глобальный обработчик исключений, который перехватывает все системные ошибки, не перехваченные ни одним из ваших предложений «catch». Он перехватывает то, что упустил остальной код. Теоретически, если все сделано идеально, он никогда не должен вызываться. Но ведь страховочные сетки для воздушных акробатов ставятся не просто так! Ваше приложение должно

выдавать какое-то уведомление при срабатывании глобального обработчика исключений. Не обязательно это проблема безопасности, но тем не менее вы упустили что-то такое, что должно было быть перехвачено, и разобраться в проблеме следует при первой возможности.

Протоколирование, мониторинг и уведомления

Боб напоминает, что всякий раз, как в бытность разработчиком ПО он пытался включить в приложения протоколирование, его останавливали со словами «слишком накладно». Много лет спустя он принимал участие в расследовании инцидента безопасности в одном из своих приложений. Когда расследователи попросили посмотреть журналы, он отвечал: «А у нас их нет. Мне сказали, что это слишком дорого». Расследователи были шокированы: «А что же тогда нам расследовать? Как мы можем узнать, что случилось, без журналов?» У Боба не нашлось ответа. Не имея журналов, расследователи инцидента не знают ни что произошло, ни источника инцидента, ни как предотвратить подобное в будущем. Выходит, что журналы важны не только разработчикам, но и группе безопасности.

Веб-приложения, API, базы данных, бессерверные приложения и т. д. – все они нуждаются в журналах. У разработчиков могут быть свои соображения о том, что записывать в журнал, но в дополнение группа безопасности должна попросить о включении следующей информации.

- Все действия, связанные с безопасностью. Кто-то пытается войти? Протоколируйте. Кто-то пытался войти, но потерпел неудачу? В журнал. Проверка входных данных не прошла десять раз подряд? Обязательно в журнал! А затем еще и уведомление отправьте.
- Все ошибки, системные исключения и вообще любые типы отказов приложения.
- Если сработал глобальный обработчик ошибок, запротоколируйте это и отправьте уведомление.
- По возможности форматируйте записи в журнале так, чтобы их можно было загрузить в SIEM (информационно-аналитическую систему безопасности, с которой обычно работает группа безопасности сети).
- В запись журнала включайте следующую информацию: временную UTC-метку, IP-адрес источника, что произошло, закончилась операция успешно или была откачена, системную информацию и сообщения об ошибках, в какой части приложения произошла ошибка (URL-адрес), информацию, соотносимую с пользователем (имя пользователя, почтовый адрес, идентификатор) и все прочее, что запрашивает группа безопасности.

- Не включайте персональную идентифицирующую информацию, номера кредитных карт, номера социального страхования и другую конфиденциальную информацию. Избегайте комбинации нескольких полей, которые в совокупности могут стать персональной идентифицирующей информацией.
- Никогда не протоколируйте тела запросов или объектов целиком. Пользуйтесь белыми списками, чтобы помещать в журнал только необходимое.
- Протоколируйте все ситуации, перечисленные выше при обсуждении условий, при которых ошибки становятся проблемами безопасности. Для каждой из них имеет смысл не только запротоколировать, но и отправить уведомление.

Лучше протоколировать больше, чем меньше, только не надо включать конфиденциальную информацию. Если не уверены, проконсультируйтесь с группой безопасности. Если они не знают, задайте вопрос в канале Slack фонда OWASP. Членство в онлайн-обществе OWASP на платформе Slack бесплатно, и там полно экспертов, обожающих советовать людям, как избежать проблем с безопасностью.

По возможности пользуйтесь стандартной библиотекой протоколирования во всех своих приложениях, поскольку это позволит быстрее разбирать журналы.

Замечание о сроке хранения журналов: если вы постоянно удаляете журналы, то их может не оказаться именно в тот момент, когда возникнет необходимость. Инциденты безопасности происходят не по расписанию, и чем дольше вы храните журналы, тем дольше сохраняете возможность расследовать странные ситуации. Но я понимаю, что хранение стоит денег, а журналы по природе своей конфиденциальны. Так что постарайтесь найти наилучший баланс между затратами, сроками хранения данных и потребностями исследователей.

Закрытие при сбое

Когда в процессе выполнения возникает ошибка, важно откатить ее и начать заново. Такой подход называется *закрытием при сбое* в противоположность *оставлению открытой при сбое*, когда мы разрешаем проблематичной транзакции продолжить выполнение. В последнем случае есть риск, что транзакция окажется неполной, некорректной, продублированной или случится еще что-то ужасное и (или) непредвиденное.

Хотя откат транзакции и повтор транзакции в приложении может показаться болезненной операцией, это куда меньше работы, чем в случае потенциального инцидента безопасности. Неправильно или лишь частично выполненная транзакция может стать причиной самых разнообразных уязвимостей, кражи данных, состояний гонки, а то и чего-нибудь похуже. Всегда

откатывайте незавершенную транзакцию и повторяйте ее сначала, даже если «тут, похоже, все может быть нормально». Поверьте мне, не стоит оно риска.

Блокировка ресурсов

Впервые я устроилась на работу программистом на полную ставку в крохотной компании в городке Каната в провинции Онтарио в Канаде, было это в 1990-х годах. Мне было 19 лет, и я не могла поверить, что мне платят 12 долл. в час за то, что я целый день пишу код! И я тогда пристрастилась слушать музыку во время работы! Я писала на языке программирования очень низкого уровня, создавая текстовые серверные системы, а всего нас в компании было четверо, включая основателя. Я работала полный день летом и на полставки в течение учебного года. Моего босса звали Билл, и я считала его Богом Программирования. Он всегда знал ответ. *Он был крут*⁵⁶.

Как-то раз я запустила программу и изменила что-то в базе данных, но, проверив после обеда, обнаружила, что измененная мной запись *снова* изменилась. Но изменила ее не я. Понятное дело, я спросила Билла, знает ли он, что случилось.

Билл: «Ты заблокировала запись, когда изменяла ее?»

Я: «Мммм, нет... А зачем? Что за блокировка?»

Тогда Билл улыбнулся своей всезнающей улыбкой и преподавал мне новый урок!

Он сказал: «Представь, что ты сохраняешь документ, с которым работаешь, в нашей общей папке. Ты его открыла и редактируешь. Но что, если я тоже его открою? И тоже начну редактировать? Откуда компьютеру знать, какую версию сохранять? И что будет, если ты сохранишь свою и закроешь ее, а потом я сохраню свою и тоже закрою? Чья версия окажется на нашем общем диске?» Как обычно, урок Билла посеял смятение в моем мозгу.

Если вы когда-либо пытались открыть файл, а в ответ система сообщала, что файл используется, то это потому, что он был *заблокирован*. В наши дни большинство баз данных делают это по умолчанию; пока вы выполняете транзакцию, одна или несколько записей (или в нашем случае файл) блокируются на все время транзакции, а затем разблокируются (становятся доступными для редактирования).

Со времен 1990-х годов кое-что изменилось. Теперь файлы по большей части блокируются сами или допускают объединение изменений в режиме реального времени. Если вам доводилось пользоваться Google Docs до собрания или во время него и вы наблюдали, как изменения мелькают перед глазами, то это как раз и было объединение изменений в действии! Для файлов базы данных вам, скорее всего, такие изыски не понадобятся, но все равно нужно гарантировать, что они заблокированы на время редактирования. Если вы работаете на языке низкого уровня или со старой базой данных, то, возможно, блокировать записи, файлы и другую информацию придется вручную. Но помните, что, хотя современные СУБД обычно берут этот труд на себя (благодаря команде начала транзакции), так бывает не всегда.

За дополнительными сведениями о состояниях гонки, о важности блокировки записей и о закрытии при сбое обращайтесь к разделу «Состояние гонки» главы 5 книги «Alice and Bob Learn Application Security», а также к главе 8 этой книги.

Привлечение менеджеров паролей

Менеджеры паролей – это программы, которые управляют вашими паролями, т. е. шифруют их, хранят, логически организуют и даже копируют в веб-формы. Они могут позаботиться о том, чтобы пароли были уникальными, случайными и длинными и даже сообщить, если узнают о том, что ваш пароль утек в составе украденных данных, так что его нужно срочно изменить. Некоторые даже исполняют роль аутентификатора, автоматически отвечая на запросы многофакторной аутентификации. Я большая поклонница менеджеров паролей, поскольку считаю, что они облегчают жизнь и помогают людям безопасно вести себя в интернете.

Разрабатывая приложение, имеющее экран входа с полями для ввода имени пользователя и пароля, *пожалуйста*, разрешайте пользователям копировать данные в эти поля. Это необходимо для правильной работы менеджеров паролей. В прошлом бестолковые специалисты по безопасности заклинали против такой практики, полагая, что возможность вставки в поле пароля как-то упростит злоумышленникам подбор паролей. На самом же деле это только побуждало людей не использовать очень длинные и сложные пароли, поскольку вводить-то их приходилось вручную, что порождало кучу ошибок и озлобленность. В результате конечные пользователи на сайтах, запрещающих вставку, выбирали менее сложные пароли, которые было легко взломать. Это не та сделка, в которой нет проигравших.

Чтобы избежать этого, вам всего-то и нужно *не* запрещать вставку в поле пароля. Пожалуйста, сделайте такое одолжение. Спасибо!

Совет Задавайте разумные и реалистичные правила для длины и сложности паролей. Не позволяйте выбирать пароль длиной всего 8 символов, этого мало. Минимальная длина должна быть равна 12 или даже 16 символам. И не навязывайте странных правил, регулирующих сложность, которые трудно соблюсти. Уверена, вы не раз в недоумении глядели на экран задания пароля, пытаясь понять, какое из 12 правил вы не выполнили. «Специальный символ? Есть. Но не один из этих четырех? Есть. Заглавная буква? Есть. Отсутствие повторяющихся символов? Вот же черт». При любой возможности старайтесь, чтобы пользователям легко было вести себя правильно.

Криптографическая практика

Если вы когда-либо работали в крупной компании или государственном учреждении, то, наверное, знаете о существовании большой группы безопасности, которая написала инструкции по применению шифрования, криптогра-

фии и всех прочих вещей, относящихся к безопасности. А от вас требовалось скрупулезно выполнять эти инструкции.

Но если в том месте, где вы работаете, за вас этого никто не сделал, то придется сделать вам самим!

Начнем с начала: *криптография* – это область математики, которая изучает вопрос об изменении формы данных. Вообще говоря, криптография используется для защиты данных и систем, в которых они используются. *Шифрование* – это двусторонний криптографический метод, который позволяет сделать данные непонятными и непохожими на свое исходное представление; но впоследствии шифрование можно обратить (это называется *дешифрованием*), приведя данные к первоначальному виду. *Хеширование* – это односторонний криптографический метод, который изменяет форму данных, так что восстановить исходное представление невозможно. Вы никогда не сможете получить оригинальную информацию по хешированному значению (хотя некоторые атаки на хешированные значения все же возможны). Именно хеширование, а не шифрование следует использовать для хранения паролей.

Вообще говоря, специалисты по информационной безопасности рекомендуют шифровать все данные в процессе передачи (в том числе передаваемые из браузера пользователя на веб-сервер и обратно) и при хранении (в базе данных, на файловом сервере и т. д.). Для наиболее ответственных систем и данных шифровать можно также данные в памяти и стирать их сразу после завершения операций с ними. Следует использовать последнюю поддерживаемую версию протокола Transport Layer Security (TLS) с нескомпрометированными алгоритмами (это исключает MD5, SHA1 и т. д.; полный перечень см. в криптографических стандартах и наставлениях NIST) и длинным ключом (128, 192 или 256 бит для симметричных алгоритмов).

Хотела бы я сказать, что являюсь специалистом по шифрованию и знаю ответы на все вопросы. Но увы... Зато я знаю специалистов и могу направить вас к ним, будучи уверена, что вы получите хороший совет, который останется таковым еще много лет после публикации этой книги. Вот мои рекомендации актуальных и заслуживающих доверия источников по криптографии:

- Cryptographic storage: Open Worldwide Application Security Project (OWASP) Cheat Sheet Series⁵⁷;
- National Institute of Standards and Technology (NIST) Cryptographic Standards and Guidelines⁵⁸.

ЗНАТЬ БУДУЩЕЕ НЕ ДАНО

Когда я пишу этот текст, мы все еще живем в мире доквантовых вычислений. Когда квантовые вычисления станут реальностью, хешированные значения, возможно, удастся обратить. Никто не знает наверняка, произойдет ли это, и если да, то когда именно. Как случится, твитнете.

Совет Когда специалисты по информационной безопасности говорят «крипто», они имеют в виду криптографию, а не криптовалюты. Когда какой-нибудь чувак в баре уговаривает вас «вложиться в крипту», он имеет в виду криптовалюту. Если вы не заядлый игрок, держитесь от этого парня подальше.

Строго и слабо типизированные языки

Мы уже говорили о том, что если приложение оказывается в неизвестном состоянии, то это может привести к одной или сразу нескольким уязвимостям. Это значит, что если мы допускаем области (например, объявление переменных), где приложение может вести себя неоднозначно, то недалеко и до такого полного опасностей состояния. Если мы используем «слабо типизированный» язык программирования и не объявляем переменные или объявляем, но потом записываем в них черт знает что (и плевать на объявленный тип), то создаем в программе *область неоднозначности*. И это может впоследствии стать причиной проблем в приложении.

Строго типизированные языки

Строго типизированными называются языки, в которых типы всех переменных определены явно и задаются при их объявлении. Нельзя создавать собственные типы, а затем объявлять переменные этих типов; это запрещено. Если язык предоставляет типы `string`, `char`, `int`, `bool` и `float`, то это все, что есть в вашем распоряжении*. Это я о тебе, C++!

Примеры строго типизированных языков:

- C,
- C++,
- C#,
- Go,
- Java,
- Pascal,
- Python,
- TypeScript,
- Rust⁵⁹.

* Язык C строго типизированный, но разрешено приводить переменную одного типа к другому типу. Дополнительную информацию см. в совете ниже.

* Это, конечно полная чушь. Разумеется, в строго типизированном языке можно создавать свои типы и объявлять переменные этих типов. Оставим эту белиберду на совести автора. – *Прим. перев.*

Совет В большинстве строго типизированных языков запрещено приводить один тип к другому. Но не в С. В С можно «привести» целое к типу с плавающей точкой. Можно-то можно, но что произойдет потом, зависит от мнения компилятора о вашем коде и от контекста приведения. Не рекомендуется делать это дома, ребята!

Слабо типизированные языки

Слабо типизированные языки дают программистам большую свободу. Вы можете создавать собственные типы в любом формате, который только в состоянии вообразить*. Это вдохновляет, потому что возможности бесконечны! Но с точки зрения безопасности это означает, что нужно более внимательно анализировать код и тестировать новые типы, потому что так можно, ничего не подозревая, создать уязвимость. Ниже приведены примеры⁵⁹ слабо типизированных языков программирования:

- JavaScript,
- Perl,
- PHP,
- Ruby,
- Shell.

Следует отметить, что не все делят языки на строго и слабо типизированные. Некоторые предпочитают деление на *статически* и *динамически* типизированные. Поговорим и о них тоже.

Любой строго типизированный язык программирования может быть статически или динамически типизированным. В случае статической типизации компилятор определяет типы на этапе компиляции. Это прекрасно, потому что он может найти и сообщить об ошибках, а вы уж исправите их, прежде чем передавать свою программу в эксплуатацию. Динамическая типизация проверяется на этапе выполнения (когда приложение работает!); если компилятор найдет что-то неправильное, то попытается это игнорировать, но может и возбудить исключение, после чего ваша программа аварийно завершится. Но вместе с тем динамическая типизация дает программисту больше гибкости, что нравится многим из нас, зарабатывающих на жизнь созданием программного обеспечения.

Наверное, вы сейчас недоумеваете, к чему это Таня ведет? Выбирая язык программирования, помните о том, что каждый язык предлагает; это важно, чтобы знать, на что обращать особое внимание при тестировании и анализе кода. Я не призываю вас использовать или не использовать строго типизированные языки, но хочу, чтобы вы знали о последствиях и потенциальных рисках выбора инструментария. И устраняйте все ошибки, обнаруженные компилятором. Чем строже вы к себе относитесь, тем безопаснее будет конечная система.

* Что автор понимает под «форматом типа», остается только гадать. – Прим. перев.

Предметно-ориентированная разработка

Под предметно-ориентированной разработкой (Domain-driven Development – DDD) понимается проектирование объектов (в смысле объектно ориентированного программирования, ООП) вокруг реальных действий, потребностей, операций и т. д. бизнеса. Например, компания We Hack Purple продавала онлайн-курсы по запросу. Если бы мы проектировали академическую платформу для размещения этих курсов, то были бы объекты «курс», «студент» и «профессор». Эти объекты отражали бы реальную бизнес-модель: курсы, на которые записываются студенты, их оценки на экзаменах (сдал / не сдал), закончили они курс или нет и т. д.

Когда мы проектируем объекты на основе реальной бизнес-модели, управление идентификацией и доступом (IAM) становится интуитивно понятным. Очевидно, что объект «студент» не должен выставлять оценки ни себе, ни другим «студентам». Знания о предметной области диктуют правильные решения относительно доступа, разрешений и т. д. Именно так DDD способствует безопасности!

Когда для проектирования применяется методика DDD, нужно знать о нескольких компонентах: сущностях (объекты, содержащие подлежащие классификации данные), объектах-значениях (что-то вроде констант, с которыми можно сравнивать), событиях (которые можно проследживать и протоколировать), агрегатах (группы взаимосвязанных объектов предметной области; например, со студентом может быть ассоциирован агрегат всех прослушанных им курсов) и репозиториях (предоставляют авторизацию)⁶⁰.

Прекрасный пример приведен в выступлении Майкла Коппманна «Типизированная безопасность предотвращает уязвимости по построению» на конференции Sec4Dev 2022 в Вене, Австрия⁶¹. Принимая от пользователя адрес электронной почты, мы обычно хотим его проверить (чтобы убедиться, что он настоящий и не вредоносный). Но, выполняя проверку, мы, как правило, никак не помечаем поле или объект электронной почты, чтобы было видно, прошел он проверку или нет. Поэтому программист, не имеющий дополнительной информации, мог бы подумать, что почтовый адрес уже проверен, хотя на самом деле это не так, а затем принимать на этой основе решения, сохранять в базе данных или выводить на экран, непреднамеренно создавая уязвимость. В случае DDD мы можем создать два разных объекта: «адрес электронной почты» и «проверенный адрес электронной почты». Входную информацию от пользователя мы помещаем в объект адреса электронной почты, а затем выполняем для него проверку. Если она проходит, то адрес сохраняется в объекте проверенного адреса электронной почты. Когда это понятно, писать или анализировать код очень просто; ясно, что для принятия бизнес-решений следует использовать только проверенные адреса. И вы сразу же видите, когда по ошибке используются непроверенные входные данные. Если бы так проектировались все объекты, содержащие

данные, полученные от пользователей, то удалось бы устранить саму возможность многих уязвимостей, в том числе межсайтового скриптинга (XSS) и внедрения SQL-кода (SQLi). *Все в выигрыше!*

Главное в DDD – что вы можете включить безопасность в модель еще на этапе проектирования, и это интуитивно понятная часть процесса. Она не добавляется впоследствии, а естественно эволюционирует вместе с проектом. Все, что может помочь вам или другим людям в следовании по безопасному пути, в моих книгах считается выигрышем, и, если предметно-ориентированное проектирование и разработка кажутся вам и вашей команде привлекательными, призываю попробовать их в деле!

Совет При обсуждении DDD рано или поздно всплывает книга Эрика Эванса «Domain Driven Design»⁶². Прочитайте ее, если хотите узнать больше об этом предмете!

Языки, безопасные относительно памяти

В этой главе мы уже обсудили уязвимости из-за переполнения, почему их следует избегать и что самый простой способ это сделать – использовать языки, обеспечивающие безопасный доступ к памяти. А теперь я хочу еще немного поговорить на эту тему.

Выбирая язык, безопасный относительно памяти, вы должны быть готовы к тому, что старое приложение придется переписать, что работников нужно будет отправить на обучение и что вследствие необходимости осваивать новый материал работа над проектом может замедлиться. Но зато мы избегаем всех видов уязвимостей при работе с памятью, а они считаются критичными по степени риска. Голосуя за безопасность по умолчанию при выборе языка или каркаса, мы можем исключить несколько типов потенциальных ошибок, способных привести к серьезным проблемам с безопасностью впоследствии.

ПРИЗЫВ

Если вы читаете эту книгу, то, вероятно, считаетесь профильным специалистом у себя на работе. Это значит, что у вас есть возможность оказывать влияние на коллег, хорошо это или плохо. Но даже если вы пока еще только практикант, никто не мешает подумать вслух: «Быть может, стоит приложить дополнительные усилия для изучения Rust хотя бы для того, чтобы избежать этих чертовых переполнений». В большинстве культур молчание – знак согласия или одобрения всего того, что было сказано, решено или объявлено в вашем присутствии. Я надеюсь, что, прочитав эту книгу, вы будете более уверенно возвышать голос и предлагать мелкие (а быть может, и крупные) изменения сложившегося на работе порядка. И, в частности, будете со знанием дела рассуждать о выборе языка, безопасного относительно памяти, когда это возможно.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Существует ли убедительная причина для выбора языка, не безопасного относительно памяти? Да или нет?
2. Приложения, написанные на строго типизированных языках, всегда безопаснее написанных на слабо типизированных языках. Это так или не так?
3. Данные должны шифроваться в процессе передачи, во время хранения или в обоих случаях?
4. Запрет вставлять в поле пароль из буфера обмена затрудняет пользователям работу с менеджерами паролей, поэтому с большей вероятностью пользователи будут выбирать короткие, менее сложные пароли. Это так или не так?
5. Конфиденциальность, в частности, означает, что вас не тревожат, когда вы пребываете в одиночестве. Что за вами не следят в интернете. Что ваши персональные данные никто не продает без вашего согласия. Определенная таким образом конфиденциальность в настоящее время ежедневно нарушается многими сайтами и веб-приложениями. Это так или не так?
6. Базы данных необходимо защищать от разных опасностей, а не только от уязвимостей типа внедрения SQL-кода. Это так или не так?
7. Нам нет необходимости помечать наши данные после их классификации, потому что каждый должен с первого взгляда понимать, к какому классу они отнесены. Пометка избыточна и излишня. Это так или не так?
8. Исходный код является интеллектуальной собственностью, и вы должны принимать меры для его защиты. Это так или не так?
9. Десериализации можно избежать, если никогда не принимать сериализованные объекты из ненадежного источника. Это так или не так?
10. Совершенно безопасно использовать производственные данные в тестовой среде, потому что «мы доверяем своим сотрудникам». Это так или не так?

Достижение цели

Теперь, пополнив свои знания в области безопасного кодирования, вы готовы к *достижению* цели. Эта глава, содержащая советы по безопасному кодированию, сложнее, но по-прежнему вполне доступна.

В этой главе рассматривается не только кодирование; мы будем говорить о безопасном проектировании и концепциях (например, о создании безопасных умолчаний, о написании повторно используемого и удобочитаемого кода и о гарантиях принятия важных решений в надежных системах), применяемых для того, чтобы конечный продукт был надежным, крепко сбитым и безопасным.

Мы обсудим, как управлять зависимостями, избегать атак полным перебором и выполнять укрепление системы и проверку имени хоста. Мы поговорим о том, почему белые списки лучше черных, об оплате технического долга и о многом другом.

Основной упор в этой главе делается на более общих решениях, которые мы принимаем, чтобы создаваемые нами системы было удобно сопровождать и чтобы они оставались безопасными в течение длительного времени, а не сползали – медленно, но верно – к катастрофе⁶³. Весь пафос этой главы заключается в том, что лучше потратить время, силы и средства сейчас, избежав потенциальной катастрофы в будущем.

Если вы поговорите с людьми, отвечавшими за безопасность в организации, где случилась крупная утечка данных, то большинство скажут, что знали о проблемах за много месяцев, а то и лет до инцидента. Они сражались за выделение бюджета, который позволил бы оплатить технический долг или добавить проверки целостности, но сталкивались с отказом, зачастую неоднократным.

В этом вы, как лицо, занимающееся созданием и сопровождением программного обеспечения, можете оказать услугу группе безопасности. Когда решения только принимаются, вы можете сообщить относящуюся к делу информацию, которая поможет всей команде делать выбор, оптимальный для организации в краткосрочной и долгосрочной перспективе. Принимая участие в анализе технологий, которые будут применены в проекте, настаивая на добавлении в жизненный цикл разработки системы действий, связанных

с безопасностью, на всех этапах проекта, или требуя каждый месяц выделять время для оплаты технического долга команды, вы сражаетесь на стороне добра.

Я надеюсь, что эта глава поможет направить всю вашу организацию к более безопасному будущему.

Безопасное проектирование

Проектировать системы безопасно значит планировать их, стремясь сделать более надежными и безопасными для пользователей и данных.

Когда исследователи безопасности и пентестеры говорят о просчетах в бизнес-логике или в безопасности, они часто имеют в виду ошибки, допущенные при проектировании. *Дефект* (bug) безопасности возникает, когда мы сделали что-то не так в коде и тем самым создали уязвимость. Но есть еще и *просчеты* (flaw), означающие, что проблема возникла еще на этапе проектирования системы. Если вы продолжите прислушиваться к беседе исследователей и пентестеров, то в какой-то момент, наверное, услышите сегодешнего на то, что автоматические инструменты «решительно непригодны» для поиска просчетов в бизнес-логике, и это чистая правда. Потому-то у них и есть работа: люди гораздо лучше видят такие вещи, чем компьютеры.

ЭФФЕКТИВНОСТЬ GRC

Когда-то Боб работал в крупном государственном учреждении с гигантской командой GRC (управление рисками и комплаенсом). Он помнит, как они заставляли его и всю команду исполнять танцы с бубнами, прежде чем допустить систему к эксплуатации. Однажды человек из GRC принес ему документ на 80 страницах; это был архитектурный шаблон, который Боб должен был заполнить. Он показал его техническому руководителю, у того была аналогичная реакция. Откуда им было взять время на это, когда все и так были по уши загружены работой по проекту? Бобу было сказано, что это обязательно, поэтому пришлось выделить члена команды для заполнения документа. Когда проект подошел к концу, Боб выкроил время для работы с командой GRC, чтобы сократить документ, ускорить процесс и автоматизировать некоторые ответы. GRC, конечно, дело важное, но мы также хотим максимально эффективно распоряжаться временем, отведенным для работы над проектом.

ДУША КОМПАНИИ

В прошлом году Алиса работала в компании, которая была приобретена более крупной компанией. Команда GRC из родительской компании усердно работала с ее ИТ-департаментом над тем, чтобы привести документацию всех продуктов и процессов к тому же виду, что в родительской компании. Поначалу казалось, что это куча бессмысленной работы, но спустя 90 дней она начала понимать, зачем все делается: они поняли, как использовать код повторно, как предоставлять услуги друг другу, как лучше работать вместе и гарантировать соблюдение всех требований аудиторов. Всякий раз, как у кого-то из ее команды возникал вопрос, GRC либо уже знала ответ, либо подсказывала, с кем по-

говорить. В тот год аудит, который всегда был для Алисы жутким испытанием, они прошли без сучка без задоринки. Это было потрясающе! Раньше она никогда не относилась к GRC как к душе компании, но эта команда была «клеем», скрепляющим весь ИТ-департамент⁹¹.

Выявить и предотвратить просчеты проектирования можно, в частности, с помощью *моделирования угроз*, в ходе которого идентифицируются и документируются потенциальные угрозы одной или несколькими системам, а затем отыскивается (хочется надеяться!) способ смягчить наиболее тревожные из найденных проблем. Моделирование угроз может принимать разные формы: от очень формального процесса с кипами документации до простого мозгового штурма с последующими заметками. Как приступить к моделированию угроз, мы поговорим в третьей части книги.

Другой способ предотвратить просчеты безопасности – провести критический обзор архитектуры и (или) проекта. Этот процесс выглядит по-разному в разных организациях, но в общем случае рисуется какая-то диаграмма, задаются вопросы, ведутся обсуждения, создается документация, и в конечном итоге вырабатывается рекомендация о том, что следует добавить или изменить в проекте, чтобы сделать его значительно безопаснее. Процедура может быть чрезвычайно формальной и длительной или более быстрой, с меньшим объемом документации.

При проектировании системы мы можем обращаться к хорошо известным *парадигмам проектирования*, заведомо безопасным, надежным и отказоустойчивым. Следуя некоторой парадигме, мы получаем процесс, который гарантирует организованный и ответственный подход к решению задачи (в противоположность хаотичному построению ПО, которое в итоге оказывается непригодным для сопровождения и работает только в простых системах). Примерами парадигм проектирования являются управляемое данными, объектно ориентированное проектирование (ООП), процедурное, функциональное, предметно-ориентированное⁶⁴ и т. д.

УШЛЫЕ КОЛЛЕГИ

Боб сражался с требованиями команды GRC. Они постоянно просили его готовить гигантские документы с описанием архитектуры, но он едва успевал справляться с текущими задачами. Как-то раз он столкнулся с коллегой, Хейди, и спросил, как она ухитряется вовремя сдавать проекты и при этом еще снабжать всеми этими документами группу безопасности. Хейди по секрету поведала Бобу, что копирует шаблоны, вставляет имена своих проектов и складывает их в OneDrive, установив строгие ограничения доступа, чтобы никто не мог их увидеть. Потом она отправляет ссылку команде GRC и ждет, запросит ли кто-нибудь доступ. По ее словам, в большинстве случаев никто не запрашивает, так что она экономит время для работы над собственными проектами. А если кто-нибудь все-таки запрашивает, то она бросает все дела и начинает заполнять форму. Она сказала, что за все годы, что она так делает, заполнять формы пришлось всего два раза. Это сотни сэкономленных часов... Бобу идея врать коллегам не понравилась, он считал, что так нельзя. Но и мысль о том, что написанные им документы никто не читает, тоже не радовала. Он решил поговорить об этом с командой GRC.

Идеи безопасного проектирования можно встроить в системы, сделав их частью самого дизайна. Вот несколько примеров:

- предполагать возможность кражи данных;
- предоставлять наименьшие привилегии;
- нулевое доверие;
- эшелонированная оборона;
- уменьшение поверхности атаки;
- безопасность, удобная в использовании;
- безопасность через неясность (не столь эффективно, использовать осмоторительно);
- безопасность цепочек поставки.

Если при проектировании приложения вы закладываете минимально возможную поверхность атаки, то у противника будет меньше объектов для атаки. Если вы заботитесь о том, чтобы средствами безопасности было легко пользоваться, то найдется очень мало пользователей, желающих их обойти. Когда мы встраиваем средства безопасности в сам способ работы приложения, противникам становится очень трудно эксплуатировать уязвимости в наших системах.

Сколько безопасности (при проектировании) «достаточно»?

Важно решить, сколько сил и денег имеет смысл тратить на безопасность с учетом ряда факторов (см. перечень ниже). Мы не хотим оставлять свои системы слабыми и беззащитными, но точно так же мы не горим желанием «перебарщивать» с безопасностью, тратя настолько много денег, времени и ресурсов, что конечный продукт оказывается слишком дорогим для покупателя. Необходимо искать баланс между безопасностью и прибыльностью. Впрочем, когда на кону человеческие жизни, национальная безопасность или другие особые факторы (вещи, которые не измеряются деньгами), мы часто готовы встроить «больше» безопасности, основываясь на немонетарных критериях.

Факторы, учитываемые при решении о том, сколько безопасности достаточно

- Насколько велик риск эксплуатации этой системы в организации?
- Являются ли данные конфиденциальными? Насколько конфиденциальными? Представляет ли их утечка угрозу пользователям?

- Сталкивается ли организация с особо опасными угрозами (со стороны других государств, хактивистов, прямых коммерческих конкурентов и т. д.)? Или же мы имеем дело только со «стандартными» угрозами в интернете (хакерами-дилетантами, преступниками и пользователями, допускающими ошибки)?
- Достаточно ли у нас обученных людей, чтобы справиться с этой нагрузкой? Если нет, имеются ли средства для найма дополнительного персонала? Имеются ли обученные и подготовленные к этой работе люди, которых можно нанять?
- Достаточно ли у нас времени, чтобы поддержать новый процесс?
- Действительно ли этот процесс более важен и ценен для организации, чем какой-то другой? Мы хотим упорядочить выбранные действия в области безопасности по отношению их ценности к затратам денег, времени и ресурсов. Если затраты превышают ценность, лучше внедрить более сбалансированные подходы к безопасности.

В сфере безопасности всегда кажется, что работы больше, чем мы можем сделать, но рано или поздно нужно подвести черту. Если нужно выбирать, построить ли три очень подробных модели угроз для самых важных систем или быстро, но не так детально смоделировать угрозы для всех систем, то по какому пути пойти? Вы должны обсудить это с руководством и выбрать тот путь, который сулит организации максимальное снижение рисков.

ПЕРВОЕ АЛИСИНО ЗНАКОМСТВО С МОДЕЛИРОВАНИЕМ УГРОЗ

Фейт, новый сотрудник группы безопасности, пригласила Алису на заседание по моделированию угроз и сказала, что все новые проекты будут проходить эту процедуру. На заседании присутствовали люди из группы по безопасности приложений, Алиса (владелец продукта) и кто-то из технической группы. В основу моделирования угроз был положен подход Адама Шустэка на основе четырех вопросов⁶⁵. Оказалось на удивление легко! Алиса даже подумать не могла, что можно так быстро выявить проблемы и даже наметить их решения. С этой системой Алиса чувствовала куда большую уверенность, чем со старой, в которой нужно было заполнять бесконечные бумаги, никогда не получая полезного отклика. Впервые Алиса испытала чувство, что действия, направленные на обеспечение безопасности, могут иметь очевидную отдачу. Она привыкла к тому, что группа безопасности заставляет заполнять бумаги, а не ведет беседы, и новый процесс стал как глоток свежего воздуха.

Совет Испытайте на себе подход Адама Шустэка на основе четырех вопросов⁶⁵. Следующие вопросы, заданные по ходу моделирования угроз помогут выявить угрозы очень быстро!

- Над чем мы работаем?
- Что может пойти не так?
- Что мы собираемся с этим делать?
- Хорошо ли мы справились со своей работой?

Управление зависимостями и безопасность цепочки поставок

Цепочка поставок для продукта – это все необходимое для его изготовления; сюда входят инструменты и материалы. Чтобы произвести банку супа, нужны ингредиенты самого супа, все необходимое для его приготовления (упаковочная машина, большой котел, плита, что еще?), а также упаковочные материалы (банка и этикетка). Цепочка поставок для программного обеспечения включает язык программирования и каркас, зависимости (библиотеки, NuGet-пакеты, gem-пакеты Ruby, npm-пакеты, а также другие «включаемые файлы» или сторонние компоненты), систему непрерывной интеграции и доставки/развертывания (CI/CD) (если вы пользуетесь ей для выпуска кода), операционную систему, в которой работает приложение, среду IDE, в которой пишется код (а также ее расширения), репозиторий системы управления версиями и все прочее, без чего невозможно собрать и сопровождать программу.

Давайте отделим зависимости от остальной цепочки поставок и начнем с них.

Безопасность зависимостей

Моя подруга Рана Халиль⁶⁶ объясняет это так: представьте, что вы пригласили на вечеринку 10 гостей. А каждый из них пригласил еще десять человек. А те пригласили еще по 10 человек – и вот уже домашняя вечеринка вышла из-под контроля, и ваш дом полон людей, которые никогда раньше друг друга не видели. С программными зависимостями дело обстоит похоже: вы включаете пять зависимостей, но у каждой из них есть *собственные* зависимости (которые называются транзитивными – это все зависимости, кроме тех, что вы включили осознанно) и т. д. Вы-то думали, что включаете в приложение всего 5 библиотек, а на деле оказалось 72. И весь этот код, который не вы писали, теперь является частью вашего приложения и создает потенциальные риски для организации.

Совет В книге «Alice and Bob Learn Application Security», первой из серии про Алису и Боба, мы подробно говорили о цепочках поставок. Почитайте, если хотите углубиться в эту тему!

КАК LOG4J ЛИШИЛА ТАНЮ РОЖДЕСТВА

В декабре 2021 года казавшаяся совершенно невинной библиотека протоколирования на Java *Log4j* нарушила рождественские планы тысяч людей из-за того, что в ней была найдена критическая уязвимость, легко допускавшая эксплуатацию. Получившая название *#Log4Shell*, эта уязвимость породила хаос в интернете, ведь хакеру нужно было всего лишь скопировать строку символов в адресную строку браузера, чтобы проверить, уязвимо приложение или нет. Лица, отвечавшие за сопровождение открытого кода, трудились

в поте лица, чтобы поскорее выложить исправления, но тем временем многие организации подверглись резульативной атаке, которая обернулась неисчислимыми переработками, сердечными приступами и украденными данными. Проблема в том, что вся наша индустрия не сводится к штатным сопровождающим открытый исходный код; она зависит от неоплачиваемого труда (замечательных) волонтеров, для которых эта работа является *хобби*. Мы создаем профессиональные системы на основе труда любителей-энтузиастов. Наша индустрия должна найти способ поддержать эту работу (финансово и профессиональными ресурсами), особенно когда речь идет о гарантиях безопасности библиотек, которыми мы пользуемся каждый день.

Проверяем, безопасно ли использовать зависимости

За прошедшие десять лет индустрия безопасности приложений начала выпускать автоматизированные инструменты, которые определяют, какие зависимости безопасно использовать, а какие – нет. Такие приложения обычно называются инструментами *анализа состава программы* (software composition analysis – SCA), а иногда – инструментами *безопасности цепочки поставок ПО*. В большинстве из них хранится список заведомо непригодных зависимостей (содержащих общеизвестные или найденные самим поставщиком уязвимости), которые сравниваются с компонентами, включенными в ваше приложение.

Большинство представленных на рынке инструментов просто сообщают, присутствует ли в приложении уязвимая библиотека. Более современные средства SCA начинают сообщать о вызове из приложения конкретной библиотечной функции, содержащей уязвимость; это называется *достижимостью*. Вообще говоря, если вы не вызываете уязвимых функций, то и не создаете в своем коде уязвимости, допускающей эксплуатацию. И хотя это означает, что в какой-то момент уязвимую версию все же придется заменить более поздней, но все бросать и сломя голову бежать устранять ошибку нет необходимости. Обычно такие инструменты предлагают путь модернизации, показывающий, на какую версию библиотеки перейти, а некоторые даже создают от вашего имени запрос на включение изменений (pull request – PR) – вещь довольно полезная. Очень может статься, что по мере отработки эти инструменты научатся не только сообщать, допускает уязвимость в библиотеке эксплуатацию или нет, но смогут даже самостоятельно устранять уязвимости.

Безопасность цепочки поставок

Цепочка поставок ПО – это не только управление зависимостями. Все, что вы делаете или используете в процессе создания ПО, является частью цепочки поставок, и все это нужно обезопасить. Сюда входит:

- система, в которой вы пишете код, будь то управляемая песочница или ваш ноутбук; ее необходимо защитить от вирусов, вредоносного ПО

и несанкционированного доступа с помощью (как минимум) регулярного резервного копирования;

- сеть и инфраструктура, которую ваша компания использует для размещения ПО, необходимого вам на этапе развертывания. Вы должны обезопасить ее перед развертыванием и затем следить за ее безопасностью уже после того, как приложение передано в эксплуатацию. Это относится к каждому пункту данного списка;
- все используемые системы взаимодействия (Slack, Microsoft Teams и т. д.) и отслеживания ошибок (Jira, Bugzilla и т. д.);
- инфраструктура сборки программы, например сервер разработки или песочница, в которой вы экспериментируете с новыми технологиями;
- ваша IDE (в которой вы пишете код), все подключенные к ней плагины и способ ее настройки;
- место, где вы храните свой код (часто называется репозиторием или системой управления версиями); необходимо регулярно делать его резервные копии и защищать от вредоносного ПО и несанкционированного доступа со стороны пользователей. В идеале должно быть несколько территориально разнесенных резервных копий, поскольку эта интеллектуальная собственность обладает исключительной ценностью для организации. Если вы пользуетесь поставщиком облачных услуг или решения SaaS (программное обеспечение как услуга), то поинтересуйтесь его политиками и процессами резервного копирования;
- место (менеджер пакетов или сайт), откуда вы берете сторонние компоненты (библиотеки) и (или) контейнеры, должно быть надежным источником;
- путь, по которому библиотеки и (или) контейнеры поступают в вашу среду сборки, должен быть безопасным (шифроваться в процессе передачи). Чтобы убедиться в том, что полученные копии подлинные, можете выполнять проверку целостности (контрольной суммы или подписи);
- цепочка поставки может включать оборудование, особенно если вы создаете продукты для интернета вещей (IoT);
- все конфигурации, используемые в процессе разработки, также являются частью цепочки поставки. Подумайте, сколько сил было потрачено на настройку среды разработки; это все время конфигурирования;
- ваша система CI/CD должна быть защищена от изменения настроек (например, отключения тестов безопасности) людьми, не имеющими на это права, и где-то в безопасном месте должна находиться ее резервная копия (если вы потеряете CI/CD, то на ее повторную установку и конфигурирование потребуется время);
- все сторонние API, вызываемые из вашей системы, должны вызываться безопасно (шифроваться в процессе передачи) и сами по себе должны заслуживать доверия (при подключении к ним должна выполняться

аутентификация и авторизация, а полученные данные должны быть проверены перед использованием);

- все сторонние данные, от которых зависит ваша система, должны шифроваться в процессе передачи, и для них должны выполняться те же действия, что для API (аутентификация, авторизация и проверка данных перед использованием);
- подписывайте созданный вашей командой код после сборки, но до отправки заказчиком. Тогда пользователи будут знать, что работают с правильной версией;
- на протяжении этого процесса вы, вероятно, будете выполнять одну или несколько форм статического и динамического анализа приложений, чтобы быть уверенными, что ваша команда не передает пользователям небезопасное ПО. Найденные во время тестов уязвимости, которые создают неприемлемые риски для организации, должны быть исправлены, после чего необходимо провести повторное тестирование;
- артефакты сборки (откомпилированный код, отчеты, журналы и т. д.) должны безопасно храниться и копироваться;
- по ходу дела необходимо проверять, что с лицензиями все в порядке. Хотя, строго говоря, лицензирование не является риском для кибербезопасности, это все же юридический риск для бизнеса, а потому проблемы следует своевременно разрешать;
- в какой-то момент необходимо проверять соблюдение действующих в организации политик конфиденциальности. Эта проверка по природе своей ручная;
- если выпускается совершенно новое приложение, API или другая программная система, то ее следует добавить в описание программных активностей. Если производится замена старой версии, быть может, в результате полной переработки, или если версия API меняется, скажем, с 1.338345 на 2.0, то позаботьтесь о том, чтобы предыдущая версия была надлежащим образом выведена из эксплуатации. В этот момент имеет смысл создать ведомость программных материалов (software bill of materials – SBOM);
- после передачи приложений в эксплуатацию необходимо наблюдать за ними, а создаваемые ими журналы загружать в какую-то систему мониторинга (например, SIEM – информационно-аналитическую систему безопасности).

Атаки на цепочку поставок встречаются на практике, когда злоумышленник (например, государство или организованная преступная группа) намеренно создает уязвимость в библиотеке, публикует ее в экосистеме ПО с открытым исходным кодом, а затем, когда жертва загрузит ее, атакует. Но это не частое явление; такие типы атак обычно запускаются, чтобы проникнуть в сильно защищенную организацию (например, государственное учреждение или частную компанию, обладающую чем-то необычайно ценным). Чаще

бывает так, что в библиотеке с открытым кодом обнаруживается непреднамеренная уязвимость, и злоумышленники пытаются атаковать с ее помощью столько организаций, сколько получится. Такие атаки обычно направлены не против какой-то конкретной группы или организации, поскольку это было бы дорого, сложно, опасно и потребовало бы слишком много времени.

Нацеленные атаки на цепочки поставок, о которых мы слышим на конференциях, читаем в блогах и смотрим в подкастах, обычно очень серьезные, уникальны, направлены против особо ценной цели и весьма изобретательны по характеру. Но атаки меньшего масштаба происходят постоянно, и любая команда, потерявшая свои данные, код или конфигурацию CI, может подтвердить, что успешная атака на цепочку поставок может приводить к опустошительным последствиям. Не допускайте, чтобы это случилось с вами или с вашей командой: обеспечивайте безопасность систем в момент их создания и включайте шаги защиты во все последующие процессы.

Предупреждение Если ваша компания находится в процессе слияния или поглощения, то злоумышленник может увидеть подходящий момент для нанесения удара. Атака на меньшую или хуже защищенную организацию в попытке проникнуть в более крупную в прошлом срабатывала много раз. В ходе объединения двух организаций будьте особенно внимательны к выполнению всех процедур безопасности.

ЗАЩИЩАЙТЕ СВОЙ РЕПОЗИТОРИЙ

Ожидая лифт, Алиса краем уха услышала разговор двух программистов о «полной катастрофе» и не смогла удержаться – стала подслушивать! Они говорили о том, как начальник дал недавно поступившему на работу студенту (практиканту) право чтения и записи в репозиторий кода, и студент удалил оттуда *все*! Начальник был занят восстановлением из резервной копии, а свою группу отправил «попить кофейку или поболтать где-нибудь», пока сам предавался горю в офисе. О боже!

Безопасные умолчания

Что такое «безопасный по умолчанию»?

Безопасными по умолчанию называются продукты, которые безопасно использовать прямо из коробки, вообще или почти не внося изменений в конфигурацию, и которые могут работать без дополнительных затрат, например на многофакторную аутентификацию (МФА), сбор и протоколирование свидетельств потенциальных вторжений и управление доступом к конфиденциальной информации.

— *Cyber Security and Infrastructure Security Agency (CISA)*

При разработке ПО есть два способа сделать его «безопасным по умолчанию»: (1) встроить безопасные параметры по умолчанию (максимально

безопасные из возможных) в продукты, создаваемые для пользователей, или (2) настроить среду разработки, так чтобы предлагаемые нам варианты разработки – те, которые «проще всего» выбрать, – были одновременно и *самыми безопасными*. Таким образом, и разработчики, и конечные пользователи будут с наибольшей вероятностью чаще всего выбирать более безопасные пути. Иногда эту методику называют вымощенной дорогой⁶⁷.

Начнем с пользователей!

Безопасные умолчания для пользователей

Будучи проектировщиками программного обеспечения, мы обязаны защищать своих пользователей от угроз, а иногда угрозами являются они сами. Не все пользователи нашего ПО технически подкованы, терпеливы и осознанно относятся к тому, как используют программы. Даже я сама иногда бездумно «соглашаюсь» с условиями обслуживания, потому что спешу, не могу набраться терпения или предполагаю, что «все будет хорошо». Пользователи часто не изменяют параметры по своей инициативе, если нет такой необходимости. А раз так, то если мы зададим безопасные параметры по умолчанию, то маловероятно, что пользователи найдут время изменить их. Я понимаю, что это смахивает на манипуляции, но в данном случае считаю их оправданными; для нас это лучшая возможность защитить наших пользователей, менее технически грамотных и хуже осведомленных в вопросах кибербезопасности.

Ниже приведены примеры безопасных настроек, сделанных нами в интересах пользователей.

- Предлагать многофакторную аутентификацию, а в особо ответственных системах делать ее обязательной.
- Проверять, что заданные ими учетные данные не совпадают с украденными комбинациями электронная почта + пароль и что не используются совсем уж очевидные пароли (типа «Password1!» или «Winter2025»).
- Требовать задания секретного вопроса и ответа или создания аварийного комплекта на случай утраты пароля или иной компрометации учетной записи.
- Сообщать пользователям, когда они в последний раз входили в систему и пытались ли они сменить пароль.
- Сообщать пользователям, с какого устройства они входили в систему, и давать возможность завершить сразу все сеансы.
- Сообщать пользователям о том, что они разрешили сторонним приложениям интеграцию с вашим приложением, и предлагать подумать, должно ли это разрешение по-прежнему действовать.
- Напоминать пользователям о том, что существуют другие пользователи, имеющие доступ к их учетной записи.

- Предлагать резервные способы доступа, например ввести другую электронную почту, принадлежащую тому же человеку, на случай если доступ к учетной записи будет заблокирован.
- Создавать правила работы с паролями, удобные для пользователей, особенно такие, которые допускают или поощряют использование менеджера паролей. Не просите пользователей менять пароль каждые 90 дней, просите изменить его, только если есть основания подозревать взлом. Разрешайте вставку в поля паролей и не заставляйте придумывать безумно сложные пароли (что только раздражает пользователей и редко повышает безопасность).
- Следить за тем, чтобы общие настройки (данные, изображения, местоположения и т. д.) по умолчанию были сведены к минимуму, и делать данные пользователя по умолчанию личными (например, для приложений социальных сетей).
- Следить за тем, чтобы система по умолчанию блокировала широко вещание, если только это не основное ее назначение (например, для Wi-Fi-маршрутизатора в кафе широко вещание обязательно).
- Предоставляя доступ другим пользователям, по умолчанию давать минимальные привилегии.
- Если можно предоставить как зашифрованные, так и незашифрованные каналы, по умолчанию предлагать зашифрованные.

Надеюсь, общую идею вы поняли. При каждой возможности выбирайте наиболее безопасные настройки по умолчанию.

Примечание Хотя вам (программисту, читающему эту книгу) может показаться, что вы нашли *обходной путь* решения проблемы, специалист по безопасности, скорее всего, объяснит, что ваша находка называется *эксплойтом*. Разработчики ПО довольно часто занимаются «хакингом», только мы называем это по-другому: поиск неисправностей.

Безопасные умолчания для разработчиков

С простой частью – обычными пользователями – мы разобрались, теперь поговорим о самых опытных из всех существующих пользователей: разработчиках ПО.

Разработчики – как вода: они без усилий проникают всюду, куда вы не хотели бы их пускать.

Сказано мной много-много раз на встречах с клиентами, которым я объясняла, что никакое техническое приспособление не помешает их разработчикам сделать то, что они хотят⁶⁸.

Мы хотим, чтобы разработчики ПО могли без труда сделать более безопасный выбор и при этом сохранить продуктивность. В идеале мы хотели бы еще, чтобы они при этом получали удовольствие от работы! Это значит,

что нужно помочь им писать более безопасные программы, создавая при этом как можно меньше препятствий. Один из таких способов – безопасные умолчания (вымощенные дороги). Ниже приведены примеры вымощенных дорог⁶⁹.

- *Управление секретами.* Создание правил хранения секретов, внедрение инструмента управления секретами, предоставление документации и обучение пользованию этим инструментом, а также создание в системе управления версиями точки подключения перед фиксацией, в которой код проверяется и отклоняется, если содержит что-то похожее на секрет. Если вы еще добавите ссылку на документ, в котором описывается, как *следует* обращаться с секретами, тем лучше.
- *Соблюдение политик безопасности.* Например, вы могли поручить своей облачной системе управления уровнем безопасности (cloud security posture management – CSPM) составить список примеров нарушения политик, упорядоченный по приоритету от «исправить немедленно» до «исправить, когда будет время». Быть может, у кого-то имеются административные права в системе «платформа как услуга» (PaaS), на которой размещено ваше приложение, но для него не задана МФА при доступе к сетевой учетной записи. Это пример упущения, требующего немедленного исправления.
- *Использование статической проверки приложения на безопасность* (static application security testing – SAST) в момент помещения кода в систему управления версиями, чтобы блокировать все известные антипаттерны (нерекомендуемые функции, ранее выявленные дефекты, код, нарушающий принятые в организации стандарты кодирования и т. д.).
- *Полезная документация.* По возможности создавайте короткие – не длиннее одной страницы – рекомендации по безопасности для каждого типа технологий, интеграции и использования инструментов. Важно, чтобы этим рекомендациям было легко следовать. Распространяйте и популяризируйте их среди разработчиков.
- *Самообслуживание.* Поскольку разработчики – опытные пользователи, они не привыкли ждать других, когда нужно решить техническую задачу. Чтобы они не тратили ценное время на ожидание, дайте им возможность все сделать самостоятельно. Пусть они сами запускают сканирование своего кода, предоставляют свой код для анализа, одобряют код друг друга, проверяют результаты, имеют доступ к золотым образам и т. д. Всякий раз, как вы избавляете их от ожидания «безопасников» и позволяете сделать работу в ваших интересах, выигрывают все.
- *Интегрированные среды разработки.* Если возможно, выбирайте инструменты безопасности приложений – статического (SAST) и динамического (DAST) тестирования на безопасность, анализа состава приложений (SCA), линтеры и т. д., которые имеют плагины для IDE, что даст

разработчикам возможность взять судьбу в свои руки путем тестирования и оценки кода еще до помещения в систему управления версиями.

- *Автоматические проверки* перед записью в систему управления версиями можно включить в IDE и в конвейер CI/CD. Можно также добавить еженедельное автоматическое сканирование репозитория кода на предмет обнаружения чего-то нового, пусть даже никаких изменений в код не вносилось. Иногда в библиотечном коде выявляется новая уязвимость, о которой вы должны знать, чтобы можно было смахнуть пыль со старого кода и модернизировать его.
- *Унификация технического стека*. Хотя попытки диктовать разработчикам, какие языки, каркасы, репозитории кода, средства отслеживания ошибок и прочие инструменты использовать, скорее всего, обречены на провал, чем успешнее вам удастся склонить их к стандартизации, тем проще будет обеспечить безопасность (и актуализацию инструментария). Если 100 % кода находятся в одной системе управления версиями, то группе безопасности нужно будет сканировать только один репозиторий, а найти подходящий для этого инструмент сравнительно легко. Попытка сканировать семь разных систем управления версиями, расположенных в 21 месте, делает работу группы безопасности бесконечно более сложной и может вынудить их покупать несколько инструментов, так что на другие меры обеспечения безопасности не хватит денег.
- *Предлагайте* всем разработчикам *пройти обучение* безопасной работе: как пользоваться переданными им инструментами безопасности, какие имеются процессы и что делать в случае инцидента безопасности. Тогда все будут знать, как себя вести, и со временем могут даже вымостить для себя собственные (безопасные) дороги.

Вы также можете спросить у разработчиков, какие задачи они хотели бы автоматизировать или выполнять самостоятельно, а затем подумать, как это осуществить. И раз уж мы заговорили об этом, пересматривайте свои собственные процессы хотя бы раз в год, чтобы понять, нельзя ли что-то ускорить, упростить или сделать более эффективным.

Код, удобный для чтения и аудита

Если вам когда-нибудь доводилось исправлять ошибку в чужом (или собственном) коде, то вы знаете, какой это кошмар. Наверное, прежде чем приступить собственно к исправлению, вы долго пытались понять, что вообще код делает. Потому-то и нужно писать удобный для чтения и аудита код, думая о том, кому придется работать с ним *после* вас. И хотите верьте, хотите нет, но спустя некоторое время человеком, испытывающим трудности при попытках понять ваш код, можете оказаться вы сами!

Начнем с определения удобочитаемости: другие люди смотрят на ваш код и без труда понимают, что вы хотели сделать. Если человек не понимает, что должен делать код, то как же он сможет безопасно внести в него изменения? Подумайте только, как сильно можно покалечить функцию, не понимая ее назначения.

Аудитопригодность кода – дело другое. Это означает, что код легко проанализировать с точки зрения безопасности и комплаенса. Это не то же самое, что удобочитаемость. В идеале вы должны оставлять комментарии вида «это средство безопасности делает то-то и то-то», чтобы аудитор знал, что вы пытаетесь сделать. Если из чтения кода и комментариев понятно, где находится каждое средство обеспечения безопасности и что оно должно делать, и если видно, что оно реализовано правильно, то цель аудитопригодности достигнута.

БОБ ОБНАРУЖИЛ НЕОЖИДАННЫЕ СЕКРЕТЫ

Боба попросили проанализировать приложение на предмет безопасности, прежде чем его компания приняла решение о приобретении компании, написавшей этот код. Ему дали всего несколько дней, так что работать пришлось быстро! Он прогнал для него инструменты SAST, DAST и SCA, и программа статического анализа обнаружила очень интересные результаты: реквизиты действующего банковского счета компании! *Обалдеть!* Боб немедленно вызвал автора приложения и попросил его сменить учетные данные (завести новые пароли), а затем помог ему перейти на использование инструмента управления секретами вместо зашивания их прямо в код. В тот день Боб действительно заработал свою зарплату – как тем, что нашел важную проблему безопасности, так и тем, что добросовестно сообщил о ней немедленно и помог исправить.

Итак, теперь мы понимаем, чего хотим достичь, но с чего же начать? Разберемся!

- Соблюдайте соглашения об именовании, прописанные в стандартах кодирования в вашей организации. Если таких стандартов нет, предложите своей команде создать их. Если все следуют одним и тем же соглашениям об именах переменных, функций, объектов и т. д., то понять ваш код будет намного проще.
- Добавляйте полезные комментарии, особенно о мерах безопасности, принятых для защиты пользователя и системы. Часто программисты помещают в начало функции комментарий, поясняющий ее назначение и прочие вещи, важные для преемников.
- Пробелы и форматирование. На любом семинаре по кодированию нам вбивают в голову обозначать циклы с помощью табуляторов, оставлять пустые строки между функциями и использовать пробелы, чтобы код было проще читать. В этом может помочь автоматизированный линтер!
- Будьте проще. Всюду, где возможно, избегайте сложного программирования. Иногда хочется продемонстрировать другим свои умения, но это неподходящее место. Чрезмерное усложнение означает, что другим программистам придется потратить больше времени, а значит, и боль-

ше денег на сопровождение написанных вами приложений. И кроме того, избыточная сложность может стать причиной непредвиденных уязвимостей, поэтому часто можно услышать, что «сложность – враг безопасности».

- Коллегиальное рецензирование. Попросите коллег проанализировать ваш код на предмет качества и безопасности. Это может быть официальной и формальной частью жизненного цикла SDLC или просто дружеской просьбой, когда вы чувствуете необходимость во второй паре глаз. Обратившись к младшему члену команды, вы заодно узнаете, легко ли читать ваш код. Если есть возможность, сделайте это формальной частью SDLC!
- Берите свою судьбу в собственные руки и проверяйте себя с помощью инструментов! Получите лицензии (хорошо бы от группы безопасности) на инструменты, которыми они пользуются для сканирования вашего кода, и делайте это сами. Средствами SAST и SCA пользоваться не трудно (как правило), и группа безопасности с радостью научит вас. Тогда вы сможете сами увидеть результаты, внести исправления, а *затем* передать свой код. Все в выигрыше!
- Оплата технического долга. Аудитор обязательно обратит внимание на технический долг, чреватый последствиями для безопасности. Обычно это неактуальные (отставшие на несколько версий) или заведомо уязвимые библиотеки и сторонние компоненты. Если вы можете не накапливать технический долг и гасить его часто, то меньше вероятность, что аудитор укажет на проблемы в этой области.
- Соблюдайте лицензионные соглашения! Добавляя сторонние компоненты, не забывайте прочесть лицензионное соглашение и не нарушайте его. Хотя это юридический риск, не связанный напрямую с безопасностью, любой квалифицированный аудитор отметит проблемы лицензирования в своем отчете⁷⁰.
- Юридическая чистота и комплаенс. Кто-то в вашей команде или организации должен контролировать, что требования к системе и ее дизайн не противоречат применимому к вам законодательству и нормативным требованиям (например, совместимость со стандартом безопасности PCI, законами о конфиденциальности штата Калифорния или регламентом GDPR)⁷⁰.
- Рефакторинг. Иногда имеет смысл переписать свой код – подвергнуть его рефакторингу. В этом случае выполните все девять описанных выше шагов, гарантирующих его удобочитаемость и аудитопригодность! Результатом рефакторинга обычно является код более высокого качества, так что, пользуясь этим списком и добавив толику решимости, вы сможете сделать из своего кода конфетку.
- Если есть возможность, включите в документацию рисунки, чтобы было понятнее. Иллюстрация дизайна, потоки данных, архитектура – все, что вы сочтете полезным, – поможет донести ваши мысли до читателя.

Совет Если вы не позаботитесь о техническом долге, то он позаботится о вас! На многие программные инциденты безопасности отреагировать будет сложнее, если накоплен большой технический долг. Если цикл выпуска версий короткий, технический долг низкий и есть возможность выпускать виртуальные заплатки (подробнее об этом ниже), то ваша команда будет компактной, эффективной и скорой на реагирование.

Желать, чтобы «код просто работал» – это нормально, но помните, что время, потраченное на написание кода, удобного для чтения, сопровождения и аудита, в долгосрочной перспективе окупится. Ваш код станет более безопасным и менее подверженным ошибкам. Да и коллегам будет приятнее с вами работать!

Важные функции выполняются в надежных системах

АЛИСА ПОЛУЧАЕТ УРОК НА ВСЮ ЖИЗНЬ

Алиса помнит, как когда-то с друзьями из колледжа пошла на соревнования «Захвати флаг». В ее команде была девушка-пентестер, которая по пути всем все рассказала. Они увидели перед собой экран входа в систему, и она ввела в поле пароля строку «or 1=1 --», но войти не смогла. Недопустимый формат. Она попробовала снова, на этот раз с помощью веб-прокси, и перехватила запрос, изменив поле пароля вручную *после* того, как его проверил JavaScript. Приложение приняло пароль и пустило нас в систему! Пентестерша объяснила, что это было внедрение SQL-кода и что если проверка производится только в JavaScript-коде, а не на сервере, то ее можно обойти с помощью веб-прокси. Одна женщина тут же встала с намерением уйти. Когда Алиса спросила, куда она собралась, та ответила: «В офис, куда же еще! Мне нужно проверить все мои приложения! О боже!» – и удалилась. В тот день все они узнали кое-что ценное!

Когда хакеры, этические и не очень, изучают ваши системы, они ищут слабые места и способы обойти встроенные средства безопасности. Один из способов – изменить или подменить системный код своим собственным (это называется *внедрением*), подставить свои значения переменных, URL-адреса, идентификаторы сеансов и т. д. Такая возможность у них появляется, когда важные функции выполняются в *ненадежных системах*.

Что такое «ненадежная» система?

Система, которую может изменить конечный пользователь или третья сторона, называется ненадежной (untrusted). Она ненадежна, потому что мы не контролируем ее полностью. Если систему может изменить только ее владелец (вы, программист), но не внешние пользователи или системы, то она надежна⁷¹.

Чтобы гарантировать надежность системы, мы должны соблюдать рекомендации (например, регулярно применять исправления и следовать руководству по укреплению), устранять найденные уязвимости, регулярно сканировать на предмет отсутствия новых уязвимостей и применять комбинацию средств мониторинга, протоколирования и (или) обнаружения.

Примером ненадежной системы может служить фронтальная часть веб-приложения, написанная на JavaScript и исполняемая в браузере. Злоумышленник может использовать веб-прокси для перехвата запросов и ответов, отправляемых и принимаемых JavaScript-кодом, а это значит, что конечный пользователь может манипулировать запросами и ответами. Если мы не будем ограничиваться проверкой значения на стороне JavaScript, а отправим его на сервер и проверим там, то система станет безопаснее, потому что сервер (в предположении, что он укреплен и содержит все исправления) является *надежной* системой. Сервер мы контролируем, и конечный пользователь не может им манипулировать.

Что такое «важные функции»?

Любой элемент безопасности (часть кода, выполняющая какую-то функцию обеспечения безопасности) является *важной* функцией; сюда относятся аутентификация, авторизация, изменение пароля, изменение привилегий или ролей в системе, шифрование и т. д. Любое выполнение транзакции, например платеж, обновление записи в базе данных, создание или удаление пользователя и т. д., – тоже важная функция. Любой механизм принятия решений в системе также считается важной функцией.

Собираем все вместе

Чтобы быть уверенным, что все важные функции выполняются только в надежных системах, лучше всего составить список всех надежных и ненадежных систем, с которыми имеет дело ваше приложение⁷¹, и четко обозначить их в проектных документах и на диаграммах. Убедитесь, что данные и запросы, полученные от ненадежных систем, всегда подвергаются тщательной проверке. Хотя тестировать интеграцию наших систем с другими нужно всегда, обращайтесь особое внимание на интеграцию с ненадежными системами.

Примечание Обход черного списка для злоумышленника часто является тривиальной задачей. И есть сайты, которые помогают обходить такие меры безопасности, например шпаргалка по обходу фильтров XSS от OWASP (XSS Filter Evasion Cheat Sheet)⁷³, которые помогают пентестерам расширять границы любого входного поля. Злоумышленники часто делятся друг с другом информацией о том, как лучше атаковать системы, подключенные к интернету. Чем больше информации о том, как нам защитить себя, публикуют такие группы, как OWASP, Def Con, Let's Talk Software Security и др., тем лучше мы все вместе сможем противостоять киберпреступности. Если возможно и разумно, распространяйте советы, данные и прочую информацию как можно шире, чтобы эффективнее противодействовать хакерам.

Черные и белые списки

В контексте проверки входных данных *черным списком* называется список символов, доменов или еще чего-то, которые *не* разрешены. А *белым списком* называется все то, что *разрешено*.

Вы, наверное, думаете: «Вот и славно! Составлю-ка я список всяких гадостей и объявлю его черным. Это же просто». Друзья мои, я тоже так думала, когда была разработчиком. А потом я стала пентестером, и не прошло и пяти минут, как научилась обходить черный список. И очень мне стало больно за приложения, которые я писала раньше.

Почему черные списки – это плохо?

Составить полный список всех символов, которые могут нанести урон вашему приложению, *трудно*. Есть очевидные, например <, > и, пожалуй, '. А как насчет), {, \ и ~ ? С ними все в порядке? Нужно ли проверять каждый символ, чтобы это выяснить? Пропустить-то легко, поскольку это придется делать для каждого входного поля. А множества допустимых символов для разных полей могут быть разными: сравните адрес электронной почты и имя человека. В первом случае обязательно должны присутствовать @ и точка, а во втором разрешены апострофы и дефисы. Много времени уйдет на то, чтобы перечислить все возможные недопустимые символы.

А теперь подумайте о хитрых хакерах. Что, если кто-то введет «%3C»? Запрет < не поможет – хакер обошел ваш черный список. URL-кодирование – один из нескольких способов, с помощью которых злоумышленник может попытаться обмануть черный список. Поэтому всегда следует использовать *белые списки*.

Как создать белый список?

Допустим, вы хотите создать поле для ввода имени пользователя. Обычно владелец продукта или представитель заказчика (человек, имеющий право определять, как должен работать продукт) решает, сколько места отвести для каждого поля. За документирование таких решений отвечает бизнес-аналитик (БА). Попросите БА согласовать с владельцем продукта минимальный и максимальный размер каждого поля, а также список допустимых символов. Возможно, вы захотите присоединиться к ним и объяснить свои резоны; можете показать им этот раздел книги. Если они скажут «Не имеет значения», то задайте пределы сами (и тогда можете быть очень строгим!).

Если вам поручили выбрать множества допустимых символов для разных полей, то примите во внимание следующие рекомендации:

- самое строгое ограничение a–z/A–Z/0–9. Это все (латинские) буквы (строчные и заглавные) и числа. Их можно использовать для имен поль-

зователей и других полей, в которых заведомо не понадобятся другие символы;

- в поле имени обычно нужны одиночные кавычки, например O'Connor;
- для адресов электронной почты имеется регулярное выражение, используемое во всем мире, см. https://owasp.org/www-community/OWASP_Validation_Regex_Repository и <https://www.abstractapi.com/tools/email-regex-guide>.

Безопасные конфигурации

Ошибки при конфигурировании безопасности вот уже более десяти лет входят в 10 главных рисков для веб-приложений по версии OWASP⁷² – и не без причины. Часто систему взламывают не из-за допускающего эксплуатацию дефекта в программе, а потому что пользователь неправильно ее сконфигурировал. Иногда параметры по умолчанию слишком либеральны, и автор системы предполагает, что вы их ограничите. Иногда пользователи отменяют некоторые средства безопасности, чтобы какая-то часть приложения работала лучше. Иногда просто предполагают, что все будет хорошо, и не проверяют, достаточно ли укреплены параметры. Что же до злоумышленников, то им не важно, почему система сконфигурирована плохо; важно лишь, сколько вреда они смогут причинить в результате этого.

Простейший способ избежать этой проблемы – следовать руководству по укреплению всех систем, которыми вы пользуетесь дома и на работе, а затем проверять параметры примерно раз в год. Это не простая работа, она не приносит удовольствия и не на виду, но она *важна*. Это часть гигиены безопасности. Это как процесс ежедневной чистки зубов – он не выглядит таким уж привлекательным, но получаемой в результате красивой улыбкой (и здоровыми зубами) можно гордиться. Именно ежедневная гигиена безопасности гарантирует, что ваша организация будет и *далее* чувствовать себя защищенной.

Если вы покупаете конфигурируемый программный продукт, например SharePoint или JIRA, то с ним обязательно идет руководство по укреплению. Следуйте его указаниям! Оно поможет оградить ваши системы от взлома. Предупреждение: чем строже меры безопасности, тем труднее работать с системой, а значит, вам, вероятно, придется идти на компромиссы с другими командами. В этом нет ничего плохого. Только следите за тем, чтобы (1) компромиссы не заставляли вас просыпаться по ночам от страха и (2) все изменения и их причины были документированы. Это может быть просто электронное письмо, чрезмерный формализм не обязателен. Но если когда-то случится взлом, то вы можете заглянуть в документацию и понять, «почему мы это сделали». Знать об этом в разгар устранения чрезвычайной ситуации полезно.

Любая купленная вами система попадает в эту категорию, будь то домашний модем или маршрутизатор, сотовый телефон или умные часы, а также все умные устройства интернета вещей. Если в них есть программа и если они изготовлены для потребителя, то обычно имеется меню для задания параметров. Изучите его и постарайтесь выбрать самые безопасные параметры. Если вы не уверены, что означает параметр, воспользуйтесь поисковиком.

Совет Если вы пользуетесь социальными сетями, то проверяйте не только параметры безопасности, но и параметры конфиденциальности, а заодно и все прочие. Они изменяются, иногда совсем немного. А иногда изменения выглядят так... будто кто-то нарочно пытается обойти выбранные вами ранее механизмы защиты. Может показаться, что это чрезмерно круто, но я как-то удалила весь свой аккаунт в Facebook, а затем создала новый (и не раз), чтобы скрыть все мои старые данные. Демонстрировать такую же «крутизну» вам вовсе необязательно, но подумайте о том, сколько информации о вас хранится в интернете и хотите ли вы и дальше выставлять эту информации напоказ. *Это говорит дама, имеющая тысячи подписчиков в социальных сетях...

Совет Если вы живете в Евросоюзе, то можете запросить полный экспорт данных из большинства продуктов (благодаря регламенту GDPR). Изучение собственной персональной информации бывает очень интересным занятием!

Проверка имени хоста

Подключаясь к серверу, убедитесь, что это именно тот сервер, который вы имели в виду. При подключении желательно использовать протокол TLS (Transport Layer Security версии 1.2 или выше на момент написания книги), чтобы передаваемые данные шифровались. Но мы также хотим быть уверены, что общаемся именно с тем, с кем думаем, что общаемся, а для этого служат сертификаты. Мы должны проверить, что сертификат выпущен надежным удостоверяющим центром (УЦ) и что он выписан на имя хоста, которое мы ожидаем увидеть⁷⁴. Это называется *проверкой имени хоста*.

Мы так жаждем проверить, с кем общаемся, чтобы избежать атаки с *человеком посередине*, или *манипулятором посередине*, когда посторонний субъект прослушивает наш «разговор». Кто-то вклинился между вами и другой стороной и читает все, что передается в обе стороны. Это может быть частью «прослушивания сетевого трафика», но еще хуже, потому что каждый запрос и каждый ответ проходят через злоумышленника. По очевидным причинам это очень плохо – вы раскрываете конфиденциальные данные.

Бывает, что разработчики ПО проверяют сам факт наличия сертификата, но забывают проверить, кто его выпустил и для какого хоста. Проверка того, что сертификат выпущен настоящим УЦ на имя ожидаемого вами хоста, поможет избежать этой атаки. Если сомневаетесь, возбуждайте иск!

Повторно используемый код

Считается хорошей практикой писать код, который можно использовать повторно; зачем снова и снова изобретать колесо, если и так уже полно колес самых разных видов? Но на практике разработчики часто торопятся скорее передать продукт в эксплуатацию, а написание качественных повторно используемых функций, которые могли бы пригодиться другим командам, оказывается не на первом месте.

В идеале повторно используемый код должен быть⁷⁵:

- гибким, чтобы его можно было легко использовать в другом приложении;
- совместимым с различным оборудованием, API, бессерверными приложениями и другими технологиями, которые вы используете;
- безопасным. В нем не должно быть ошибок и дефектов, которые могут повлиять на безопасность или надежность других приложений.

БИБЛИОТЕКА БЕЗОПАСНОГО КОДА

Алиса помнит, как одна из ее ведущих программистов объясняла ей, что у них есть нечто, именуемое «библиотекой безопасного кода», и очень гордилась собой. Она сказала, что в каждом новом проекте они используют шаблон, полный кодировщиков, валидаторов, экранов входа, обработчиков паролей и еще всякого разного! Что, тестируя на проникновения шаблон и повторно используемые функции вместо одного приложения за раз, они могут исправить ошибки сразу во всех своих приложениях, работающих в организации. Она похвасталась, сколько денег сэкономила компании – и все благодаря повторно используемому коду! Алиса пометила себе сказать несколько теплых слов начальнику этой дамы...

Из преимуществ повторного использования кода отметим экономию времени и денег, снижение рисков (потому что вы уже протестировали этот код и знаете, что он безопасен), упрощение сопровождения и т. д.

Но, как вы наверняка догадались, у повторного использования кода есть и минусы. Самый большой из них – осведомленность; просто дать знать другим командам о том, что они могут пользоваться новой написанной вами функцией, гораздо труднее, чем мне казалось вначале. Чтобы повторное использование стало обыденностью, а не одиночным явлением, нужно приложить много организационных усилий.

Чтобы повторное использование приносило пользу⁷⁶, нужно обеспечить следующие условия:

- найти и использовать код просто;
- интероперабельность: повторно используемый код работает со всеми платформами, языками и каркасами, являющимися частью технологического стека;

- строгий присмотр (рецензирование и тестирование кода как минимум);
- какая-то форма управления изменениями, позволяющая узнать, что было изменено, когда, кем и почему. За неимением лучшего можно использовать систему управления версиями и хорошие комментарии;
- версионирование для публикации изменений в библиотеке и доведения их до пользователей. Следует избегать автоматических обновлений, которые могут привести к сбоям в работе, и постараться сделать применение обновлений максимально простым;
- масштабируемость. Ваш код должно быть возможно использовать как в крохотном, так и в гигантском приложении; он должен работать с возможно большим числом систем. Если вы хотите создать библиотеку или повторно используемый код, позаботьтесь о том, чтобы он был *безопасным*, для чего применяйте знания, полученные из этой книги.

Безопасные системные вызовы

Обращение из программы к операционной системе называется *системным вызовом*⁷⁷. Обычно наши приложения работают в какой-то операционной системе: Microsoft Windows, Linux, macOS, Android, Apple iOS и т. д. Если мы хотим, чтобы операционная система сделала для нас какую-то работу, то должны попросить ее об этом посредством системного вызова.

Операционные системы обладают невероятными возможностями; они умеют делать практически все, на что способно программное обеспечение. Они могут делать больше, чем наши приложения: запускать другие приложения, останавливать процессы, связываться с другими серверами и т. д. Чрезвычайно – критически – важно защищать системные вызовы от злоумышленников.

Самый очевидный способ защитить системные вызовы – никогда их не делать, но это не всегда возможно. В предположении, что вы все-таки должны обращаться к системе, чтобы ваша программа работала правильно, следующий вариант – никогда не допускать, чтобы полученные от пользователя данные прямо или косвенно использовались для выполнения системного вызова (*никогда* не подставляйте пользовательские данные в параметры системного вызова). Если вам необходимо, чтобы действия или данные пользователя были частью системного вызова, то не копируйте их напрямую, а сравните с белым списком входов. Если есть совпадение, то можете продолжать (но текст берите из белого списка, а не из пользовательских данных). Если нет, отклоните и начните снова.

Любой системный вызов должен выполняться в надежной системе (на сервере, а не на компьютере пользователя), где вы полностью контролируете то, что делаете.

Смягчение последствий

Ко всему прочему, вы должны установить защитные программы на том сервере, где выполняется системный вызов. Например, файловые серверы не должны иметь возможности выполнения кода. На веб-сервер должно быть запрещено устанавливать новые программы. На всех серверах должны быть установлены антивирусы, системы предотвращения и обнаружения вторжений (IPS/IDS), инструментальные средства контроля приложений и система мониторинга целостности файлов (FIM).

В этой области можно сделать и больше, если вы пишете код очень низкого уровня, но большинство программистов в мире пишут веб-приложения. Если вы пишете низкоуровневые программы, то прочитайте следующую статью, в которой описываются дополнительные способы защиты системных вызовов: C. M. Linn, M. Rajagopalan, S. Baker, C. Collberg, S. K. Debray, J. H. Hartman «Protecting Against Unexpected System Calls»⁹².

Комментарии и другая документация

Комментарии и документация – еще один способ сообщить что-то другим людям, работающим над проектом, – сейчас или в будущем. Если вы пишете кратко и информативно, то вашему преемнику будет приятно работать. Обратное тоже верно. Давайте поговорим о том, как делать это правильно.

Комментарии

Комментарии в коде объясняют, о чем мы думали и что должен делать код. В идеале мы разбрасываем комментарии по всему коду, чтобы помочь программисту, который придет нам на смену (иногда это мы сами). Часто функцию начинают комментарием, сообщаящим читателю, для чего она предназначена, а в неочевидных местах размещают дополнительные комментарии.

Ниже приведены плюсы оставления комментариев в коде⁷⁸.

- Комментарии объясняют преемнику, как работает часть кода и почему она написана именно так. В результате код проще сопровождать, а исправление в нем ошибок занимает меньше времени.
- Комментарии поясняют, как вы мыслили, когда писали код, что в идеале сделает работу с ним более приятной для следующего программиста. Возможно, это звучит странно, но код, с которым очень трудно работать, не способствует удержанию сотрудников. Люди могут попросту не хотеть работать с вами в одном проекте.
- Комментарии напоминают, о чем вы думали. Со временем все забывается.

- Есть инструменты, которые умеют создавать документацию из комментариев, что экономит ваше время (пример: `pydocs`).

Но у оставления комментариев в коде есть и минусы.

- Если комментариев *слишком* много, то они могут вызывать пресыщение или раздражение – например, когда комментариев больше, чем кода.
- Устаревшие или неправильные комментарии могут вводить в заблуждение. Представьте, что вы исправили ошибку, но комментарии не обновили, и теперь они не отражают истинное положение вещей. Так вы отправите следующего программиста искать вчерашний день. Это никому не нужно.
- Сбивчивые комментарии могут... сбивать с толку. Не каждый умеет хорошо формулировать свои мысли.

Помимо комментариев, старайтесь по возможности писать самодокументированный или самообъясняющий код. Например, если вы назовете переменную `math1`, то преемник, наверное, предположит, что она будет использоваться для каких-то математических операций. Это полезно, но не слишком.

Чтобы толку было больше, вы могли бы назвать ее `intAverageHeight`, сказав тем самым, что переменная, во-первых, целая, а во-вторых, содержит средний рост (кого, мы не знаем).

Еще лучше было бы назвать переменную `intAverageStudentHeight`, тогда мы будем знать, что она содержит средний рост студентов (быть может, будет еще и переменная `intAverageTeacherHeight`, содержащая средний рост преподавателей). А свою функцию вы могли бы назвать `AverageHeight`, потому что она вычисляет средний рост группы людей – студентов, преподавателей или еще кого-то.

Соглашения об именовании, поясняющие, что вы хотите сделать, – отличный способ сделать код легким для чтения.

ДЖЕННИФЕР

Однажды Боба приняли на работу в теневой ИТ. Поначалу он этого не понял, но вины его в том не было! Его наняло бизнес-подразделение, и он думал, что он и есть вся команда программистов. До него там работали еще два программиста, и одну из них звали Дженнифер. Впервые открыв одно из ее приложений, он понял, почему она тут больше не работает. Все ее переменные и функции назывались... `Jennifer!` Даже без префикса, обозначающего тип переменной (например, `strJennifer` хотя бы сделало понятным, что это строка). Видя перед собой функции `Jennifer1`, `Jennifer2` и `Jennifer3`, а затем переменные `Jennifer4`, `Jennifer5` и `Jennifer6`, Боб уже через две секунды пришел в полное недоумение! Бобу пришлось переписать всю эту муру, чтобы извлечь из нее смысл и сделать пригодным для сопровождения. Он помнит, как объяснял начальнику, почему это заняло так много времени: он даже представить себе не мог, как высоко могут подниматься брови у начальника.

Примечание Хотите верить, хотите нет, но программисты раньше не оставляли комментарии, потому что боялись, что иначе их будет легче заменить другими. В 1980-х и в начале 1990-х годов встречались программисты, которые намеренно затрудняли сопровождение своего кода, чтобы застолбить за собой место. Работало это плохо.

Документация

С документацией дело обстоит сложнее. Очень легко перестараться (по сравнению с комментированием) и в итоге потратить на составление документации столько времени, что его не окупить никакой пользой от нее. Цель документации – сделать создание, сопровождение и обеспечение безопасности приложения быстрее, проще и дешевле. Если вы тратите вдвое больше времени на содержание документации в порядке, то что же остается от выигрыша? Очень важно держать объем документации в разумных рамках.

Начнем с преимуществ документирования приложений (это может быть описание архитектуры, план тестирования, дизайн-план и т. д.).

- Хорошо написанная документация позволяет быстро получить общее представление о приложении.
- Документация кода дает общее представление о коде и о его месте в системе в целом. Это может быть особенно полезно для новых разработчиков, старающихся понять, как устроена кодовая база.
- В документацию кода можно включать примеры и инструкции, которые помогут разработчикам использовать код правильно.
- Документация также поможет организовать долгосрочное сопровождение. Содержащая ясное объяснение назначения и порядка использования кода документация поможет предотвратить недопонимание и уменьшить риск ошибок.

Ниже приведены некоторые рекомендации по написанию документации.

- Будьте лаконичны. Документация должна быть по возможности краткой, но при этом содержать все важное. Вам платят не постранично (как правило), поэтому написание большого объема текста не принесет никакой финансовой выгоды. Старайтесь переходить к делу как можно быстрее.
- Язык должен быть ясным, чтобы его было легко воспринимать. На составление максимально ясного документа может потребоваться время, так что вам, возможно, придется написать несколько черновых вариантов. Но если вы добьетесь того, чтобы на чтение уходило в два раза меньше времени, а прочтут документ сотни людей, то ваши усилия позволят сэкономить время и деньги (а быть может, еще и спасут кого-то от головной боли).
- Если есть возможность, включайте рисунки, чтобы проиллюстрировать важные моменты (например, дизайн или поток данных). Люди воспринимают информацию по-разному, а изображения помогут разобраться быстрее.
- Чем меньше, тем лучше; пишите только ту документацию, которая, по вашему мнению, будет полезной (если только политики или правила не вынуждают вас действовать иначе).

- Не запрашивайте документацию, которая вам не нужна. Не включайте в документацию детали (например, списки IP-адресов, формы сторонних поставщиков, дату последнего теста на проникновение и т. д.), если в ней нет острой *необходимости*. Не тратьте чужое время на что-то, что только *можете* захотеть сделать позже. Вы, наверное, думаете: «Ну уж этого-то я никогда не сделаю», но за 27 лет моего пребывания в профессии я видела немало документации, которую запрашивали, но никогда не читали, а сколько на ее составление было потрачено часов работы инженеров – даже подумать невозможно.

РАЗУМНЫЕ КОММЕНТАРИИ

Алиса помнит, как посещала в колледже курс программирования, и профессор объяснял важность комментариев. Ей показалось, что это полезная вещь, и она решила, что будет стараться. Позже один сокурсник попросил ее просмотреть код, и Алиса *запуталась*. Логика казалась разбросанной по всему коду, и в конце концов она задала несколько вопросов. В итоге она убедила товарища добавить в разные места комментарии, и студент получил пятерку за свою работу. Алиса была очень довольна собой.

БОБ ВОЗВРАЩАЕТСЯ В ПРОШЛОЕ

Раньше Боб думал, что комментарии – это тупо: «Если следующий программист не сможет разобраться, то, наверное, он никчемный кодер». Так он считал, пока на втором году работы ему не пришлось исправлять ошибку в своем собственном коде. У него ушло 4 часа на то, чтобы понять, что же такое он хотел сделать, и только потом дело дошло до исправления ошибки. С тех пор он стал ценить комментарии!

Запрос согласия пользователя

У наших пользователей обычно* есть права. Если пользователь является потребителем, то он может выбрать, использовать нашу программу или нет. В большинстве стран вы обязаны запрашивать согласие пользователя на различные действия, совершаемые в приложении, например сохранение, предоставление в общее пользование, копирование их конфиденциальных данных, удаление данных или пользователей и т. д. Мы задаем такие вопросы, чтобы не огорчить своих пользователей. Мы спрашиваем даже тогда, когда у пользователя нет возможности увидеть, что мы делаем, – и не потому, что боимся нарушить закон, а потому что это *правильно*.

Вы должны спрашивать согласия пользователя в следующих ситуациях (как минимум):

- любая форма сбора или обработки персональных данных⁷⁹;
- создание или удаление учетной записи;
- изменение пароля;

- изменение контактной информации (включая имя пользователя и адрес электронной почты);
- предоставление доступа другим лицам;
- разделение данных пользователя со сторонним инструментом (включая вызов стороннего API, например Google Maps, и отправку ему данных пользователя в запросе);
- если вы собираетесь использовать данные способом, отличающимся от первоначально заявленного;
- продажа данных пользователя;
- обучение своего ИИ на данных пользователя.

Если сомневаетесь, получите у пользователя согласие.

Примечание *Иногда у пользователей *нет* прав. Канадская налоговая служба (Canadian Revenue Agency – CRA) заставляет граждан пользоваться своими услугами для оплаты налогов и выполнять другие налоговые законы. Услуги CRA, не предоставляемые через интернет, доступны только в рабочее время и обычно выливаются в многочасовое ожидание в очереди, чего большинство канадцев (включая автора этой книги) не могут себе позволить несколько раз в год. Поэтому большинство канадцев вынуждены (чтобы избежать финансовых мытарств) пользоваться онлайн-услугами CRA. К сожалению, в условиях обслуживания CRA написано, что она не несет юридической ответственности за утрату ваших конфиденциальных данных и за иной ущерб, причиненный в результате использования ее цифровых систем и служб. От такого обслуживания большинство из нас при нормальных обстоятельствах отказалось бы; с чего вдруг мы должны передавать наши самые ценные конфиденциальные данные (о наших финансах) организации, которая заявляет, что не несет ответственности за их утрату? Нормальный человек выбрал бы конкурирующее приложение или службу. Увы, у канадцев нет альтернативной налоговой службы, предлагающей разумные и справедливые условия обслуживания. На момент написания этой книги CRA потеряла персональную информацию тысяч канадцев (в их числе оказались и родители автора) в результате одного или нескольких взломов и является ответчиком по групповому гражданскому иску⁸⁰.

Проверки целостности, подписание кода и неизменяемые сборки

Когда в 2021 году вышел в свет документ OWASP о «новых» главных десяти рисках для веб-приложений, занудный автор этой книги была до крайности взволнована, обнаружив под номером 8 «Ошибки, связанные с целостностью программ и данных»⁸¹! Наконец-то кто-то указал, что мы не всегда можем доверять тому, что наши собственные сборки действительно являются нашими. В книге «Alice and Bob Learn Application Security» я говорила, что доверять нельзя никому, даже своей маме⁸², и я действительно так считаю. Как нам узнать, что откомпилированный нами код или собранный образ не были

модифицированы, если только мы не воспользовались ими сразу же? С помощью проверки целостности, вот как!

Когда мы собираем программу с помощью конвейера CI/CD, артефакты (откомпилированное приложение, контейнер, виртуальная машина (ВМ), отчет и т. д.) куда-то копируются. В идеале если это контейнер или ВМ, то он развертывается. Если это приложение, то оно развертывается в одном из таких контейнеров, виртуальных машин или в облачной PaaS (платформа как услуга). Но иногда артефакт где-то сохраняется, чтобы быть развернутым позже, особенно если применяется метод водопада или еще какая-то технология программирования, отличная от DevOps.

На протяжении большей части своей карьеры в программировании я зарабатывала по методу водопада, поэтому много раз наблюдала, как разработчики копируют свои приложения на разные серверы и обратно. Было бы проще простого подменить копию другой, и никто бы ничего не узнал. Именно тут в игру вступает проверка целостности.

Проверка целостности* может выступать в разных обликах. В случае людей она часто сводится к взгляду глаза в глаза и задаванию трудных вопросов: «Вы ели это печенье, милочка?» или «Вы что-нибудь пили сегодня вечером, сэр?». Глядя в лицо собеседнику и оценивая его ответы, часто можно сказать, говорит ли он правду. К сожалению, компьютеры не обладают интуицией, поэтому им нужны цифровые способы проверки целостности сборки, программы, сообщения и т. д.

Проверить целостность программы – значит убедиться в том, что налицо точная копия той программы, которую вы имеете в виду. Это можно сделать посредством⁸³ проверки контрольной суммы или цифровой подписи. Такая проверка часто производится в процессе установки программы, после скачивания (но до использования), во время CI/CD, на этапе развертывания и т. д.

Мы также можем защитить целостность цепочки поставок нашей программы⁸³.

- Убедитесь, что зависимости, используемые в программе, не содержат известных уязвимостей (а если содержат, то мы не вызываем уязвимых функций или уязвимость невозможно эксплуатировать при нашем способе использования).
- Убедитесь, что используемые зависимости взяты из надежного источника (заведомо безопасного репозитория, быть может, такого, который ваша организация контролирует или за пользование которым платит, – в отличие от менее строго контролируемых репозиторий типа Docker Hub).
- Убедитесь, что строго контролируется доступ к инструментам создания кода: репозиторию кода, конвейеру CI/CD и другим механизмам выпуска и развертывания программного обеспечения.

* В английском языке слово *integrity* означает не только «целостность», но и «честность, искренность». – Прим. перев.

- В этом отношении может также помочь процесс анализа кода и изменений конфигурации, но имейте в виду, что он ручной и медленный; планируйте с учетом скорости и простоты использования.
- Добавляйте в процесс автоматизированный анализ кода, например SAST. (Результат не гарантирован, потому что умелый злоумышленник может пропихнуть вредоносный код, как показал инцидент с компанией Solar Winds, но все равно это чрезвычайно полезно.)
- Если используется CI/CD, то в процесс доведения кода до этапа эксплуатации должно быть вовлечено несколько людей. Не должно быть так, что единственный человек может по собственной инициативе ввести программу в эксплуатацию. Это называется разделением обязанностей (один и тот же человек не должен контролировать весь процесс).
- У всех пользователей CI/CD и других программ публикации должна быть включена многофакторная аутентификация.
- Если бюджет позволяет, а риски высоки, то подвергните свою цепочку поставок тестированию на проникновение.

Примечание Алекс Доу из компании Mirai Security в Ванкувере провел потрясающую демонстрацию на конференции по безопасности SecTor в Торонто, провинция Онтарио, в 2021 году, в ходе которой скомпрометировал среду CI/CD DevOps более 20 раз на протяжении одного сеанса тестирования на проникновение. К счастью для нас, Алекс – хороший парень, который помогает сделать наш цифровой мир безопаснее, но эта презентация ясно показала, что взломать такие системы гораздо проще, чем большинство из нас хотело бы думать. Пожалуйста, относитесь к этим мощным системам с должным вниманием.

Неизменяемые сборки

Под *неизменяемостью* понимается невозможность изменения⁸⁴. После развертывания неизменяемой инфраструктуры она такой и остается. Мы не вносим в нее исправления, а заменяем уже исправленной версией.

Основное преимущество такого подхода⁸⁵ заключается в том, что мы можем откатиться в мгновение ока. Если при латании на живую что-то пойдет не так, то зачастую откатиться назад невозможно. Мы должны править работающую систему. В случае неизменяемых сборок у нас есть точная копия предыдущих версий, к которой мы в любой момент можем вернуться. Если мы вносили исправления в среду разработки и возникла ошибка, то мы просто откатываемся. Получив и тщательно протестировав работоспособную копию, мы можем перенести ее в производственную среду.

Еще одно преимущество неизменяемых сборок в том, что мы всегда *точно* знаем, что развернуто на наших серверах и что входит в состав сборки. Это идеальная инвентарная опись. И с точки зрения репликации все просто; нет нужды вручную копаться в измененных конфигурационных файлах, когда требуется создать еще 20 экземпляров контейнера.

Использование неизменяемых подписанных сборок – еще один способ обеспечить целостность наших программных систем.

Избегать грубой силы

Грубая сила: использование силы, стараний или властных полномочий в большом объеме вместо более эффективных, тщательно спланированных или точно направленных методов.

— Словарь Merriam-Webster⁸⁶

В атаках грубой силой*, или полным перебором, против программных систем используются средства автоматизация, иногда называемые *ботами* или скриптами. Большинство атак полным перебором не блещут остроумием, по крайней мере, так было раньше. Злоумышленники учатся, и некоторые приспособились к нашим мерам защиты и теперь проводят более изощренные атаки. API подвергаются атакам полным перебором чаще, чем веб-приложения или «фронтальные интерфейсы». Обычно атакующий пытается получить доступ к системе, в которой у него нет надлежащей учетной записи, но иногда просто хочет «положить» ваши серверы, чтобы причинить вред. В любом случае новость не из хороших.

Самый распространенный способ⁸⁷ предотвратить атаку полным перебором – заблокировать учетную запись после нескольких неудачных попыток доступа. К сожалению, если делать это неаккуратно, то злоумышленник сможет спровоцировать блокировку тысяч учетных записей, организовав тем самым атаку типа «отказ в обслуживании» (DoS).

Вторая, тоже распространенная мера – использовать ограничение темпа и (или) квотирование ресурсов, т. е. уменьшать темп запросов и (или) блокировать пользователя после определенного количества попыток. По видимости похожая на первый вариант, эта мера обычно реализуется с помощью технических средств, расположенных вне вашего приложения, например шлюза API, брандмауэра веб-приложений (WAF) или другого сетевого устройства.

Еще одна мера, предложенная OWASP⁸⁷, – делать цифровой отпечаток устройства и заблокировать его на основе файлов cookie браузера, IP-адреса или другой идентифицирующей информации. Вы должны отследить (с помощью cookie) браузер, который ранее был успешно использован для входа в систему. Если он уже использовался прежде, то, возможно, блокировать не нужно, достаточно некоторой задержки. Это зависит от профиля риска.

Из других мер отметим блокировку IP-адресов из определенных географических регионов (быть может, стран, не дружественных по отношению к вашей), IP-адресов известных прокси-серверов, дросселирование запро-

* В контексте безопасности ПО вместо термина «грубая сила» употребляется «полный перебор». Далее мы будем следовать этому соглашению. – Прим. перев.

сов (замедление их до такой степени, чтобы это мешало компьютеру, но не пользователю), квотирование ресурсов (разрешение не более заданного числа попыток, пусть даже замедленных), запрос второго фактора аутентификации (МФА), запрос другого значения того же фактора (обычно ответа на «секретный вопрос», но я этот механизм не рекомендую), добавление CAPTCHA, разрешение входа только с конкретных заранее одобренных IP-адресов, браузеров или устройств, а также предоставление уникальных страниц входа (для очень важных пользователей, например администраторов). В идеале следует настроить WAF или другой инструмент, который будет выполнять эти проверки и блокировки динамически и автоматически.

Надо сказать, что принудительное использование МФА для всех пользователей решает многие из описанных проблем. С точки зрения бизнеса это не всегда имеет смысл, но, если есть такая возможность, агитируйте за нее. МФА – самая главная мера защиты от подлога учетных данных, активируемая на стороне пользователя (следующая – использование длинных, сложных и уникальных паролей или ключей доступа).

Элементы безопасности

Элемент безопасности: защитная мера или контрмера, предписанная для информационной системы или организации, предназначенная для защиты конфиденциальности, целостности и доступности ее информации и отвечающая набору определенных требований к безопасности.

– Национальный институт стандартов и технологий (NIST)⁸⁸

Иными словами, элемент безопасности – это действие, направленное на принудительное внедрение политики безопасности. В случае программного обеспечения любой код, служащий обеспечению безопасности, является элементом безопасности. Примеры: принудительная аутентификация пользователя для входа в систему, запрос паролей, шифрование данных в процесс передачи и хранения – ну вы понимаете.

Элементы безопасности можно отнести⁸⁹ к следующим категориям: физические, технические и административные. Сами по себе элементы безопасности могут быть превентивными, обнаруживающими или корректирующими. Давайте рассмотрим все по порядку.

- **Физические элементы безопасности** – это замки на дверях, окнах и вашем шкафчике на уроках физкультуры. Сюда же относятся охранники, капканы и ворота. В эту категорию включаются также аппаратные устройства, например модуль аппаратной защиты (hardware security module – HSM) в ноутбук.
- **Технические элементы безопасности** – это программные средства, реализующие принудительную политику безопасности, например МФА (на

ум приходит менеджер паролей), брандмауэр и антивирусная защита конечных точек.

- *Административные элементы безопасности* – это политики, процедуры, инструкции, стандарты и т. д. Это правила, которые мы должны соблюдать, если хотим работать в системе, к которой они применимы (например, на вашем рабочем месте).

Вообще говоря, физические и технические элементы используются для контроля выполнения административных элементов. В идеале должен также существовать какой-то механизм отслеживания и уведомления (протоколирование, мониторинг и уведомление) во всех типах элементов безопасности, который гарантировал бы их соблюдение.

При чтении материалов на эту тему вы обязательно встретите превентивные, обнаруживающие и корректирующие элементы безопасности.

- *Задача превентивного элемента безопасности* – предотвратить нарушение до того, как оно произошло. Запирание двери, шифрование данных или создание политики, предотвращающей что-то ужасное, – примеры таких элементов.
- *Задача обнаруживающего элемента безопасности* – обнаружить, что нарушение уже произошло или происходит в данный момент. Примерами могут служить все виды протоколирования, мониторинга и уведомления, например автомобильная сирена. Парным административным элементом могло бы служить дерево обзвона (кому звонить в случае чрезвычайной ситуации) или процесс упорядочения реакций на инцидент.
- *Задача корректирующего элемента безопасности* – устранить последствия нарушения или атаки. Этот вопрос должен рассматриваться в тех частях процесса реагирования на инциденты, которые относятся к устранению последствий и восстановлению. С технической точки зрения речь идет о резервном копировании и откате, а с физической – о вставке окна взамен разбитого или перепрограммировании электронного замка.

Из других типов элементов безопасности упомянем отпугивающие, директивные (политики, инструкции), компенсирующие и восстановительные.

Если вы планируете создать формальную программу безопасности приложений (или вообще любую программу безопасности), то вам понадобятся все эти виды элементов безопасности.

О расширенных привилегиях

Будучи разработчиком ПО, вы, по существу, являетесь технологическим супергероем. Мало того что вы обладаете набором потрясающих умений, которые делают вас чрезвычайно могущественным, так еще в сетях и систе-

мах, с которыми вы работаете, вам обычно предоставлены дополнительные привилегии, права доступа и возможности. А значит, вы ценная мишень для злоумышленника. Если бы враг вашей организации получил доступ к вашим рабочим учетным записям, то он смог бы нанести огромный ущерб компании. Поэтому вы должны очень тщательно защищать свои учетные записи и привилегии.

Для этого есть два способа: использовать длинный, сложный и уникальный пароль (или ключ доступа) и включить многофакторную аутентификацию для всех важных учетных записей. Таким образом, вы предельно затрудняете их взлом.

Помимо этого, следует применять ко всем своим учетным записям принцип наименьших привилегий. Если вам не нужен доступ к чему-то, откажитесь от него. Если в каком-то инструменте отпала необходимость, откажитесь от него. Хотя отказ от доступа и инструментов временами ограничивает вас, в идеале следует отказываться только от того, что вам действительно не нужно для выполнения своих должностных обязанностей. Относитесь к этому как к весенней чистке своего цифрового отпечатка.

Сверх всего перечисленного рассмотрите возможность использования менеджера паролей и, возможно, второго фактора аутентификации. Менеджер паролей обеспечит безопасное и защищенное хранение всех ваших паролей. Если вы решите его использовать, то тот единственный пароль, который вы должны запомнить, – открывающий доступ к самому менеджеру, – должен быть необычайно длинным и сложным, и угадать его должно быть невозможно. У меня в качестве такового служит хохма. Да-да, вся хохма *целиком*. Вопрос и ответ*. Она длинная, потому что учетные записи, которые хранятся в моем менеджере паролей, для меня чрезвычайно ценны.

Совет Никогда не стоит недооценивать свою ценность для злоумышленников. Если кто-то просил вас дать свой пароль, заглядывал к вам через плечо или как-то оказывал на вас давление для получения доступа к вашим учетным записям, сразу сообщите об этом группе безопасности. Даже если неловко. Лучше решить вопрос немедленно, чем позволить ему разрастись до таких масштабов, что скрыть будет невозможно. Если сомневаетесь, на всякий случай смените пароли.

Обслуживание безопасности

Любую важную систему следует обслуживать должным образом. Мы отдаем свои машины в сервис, чтобы там проверили, что все компоненты, включая средства безопасности, работают нормально. Нередко сотрудники группы безопасности бывают настолько заняты, что у них не хватает времени ежедневно делать то, что мы хотим. И в результате мы забываем о надлежащем обслуживании своих инструментов.

* По-видимому, автор имеет вид что-то типа вопроса армянскому радио. А что, неплохая идея. – *Прим. перев.*

Крайне важно проверять и тестировать наиболее ответственные системы. Это брандмауэры, брандмауэры веб-приложений, системы обнаружения и предотвращения вторжений и прочие защитные системы, которые в идеале нужно проверять каждый год. Говоря об *обслуживании безопасности*, мы имеем в виду обслуживание систем обеспечения безопасности или средств безопасности, встроенных в наши системы. Мы хотим быть уверены, что они работают надлежащим образом, что в чрезвычайной ситуации у нас будет весь необходимый доступ и что исправления применяются вовремя, а конфигурация безопасна.

По возможности настройте ежегодное напоминание о необходимости тестирования подведомственных вам систем. Это не значит, что нужно проводить полное тестирование на проникновение, – это было бы слишком дорого и долго и нужно лишь тогда, когда речь идет об особо ответственной системе или потенциальные риски того требуют. В общем случае вы можете изучить последнюю версию руководства по укреплению системы и убедиться, что все рекомендации выполнены. Проверьте, что все, кому необходим доступ, имеют его, и что те, кому доступ больше не нужен, его лишены. Если предусмотрены какие-то аварийные учетные записи, то убедитесь, что они работают. Вы и так должны заботиться о том, чтобы ко всем системам регулярно применялись исправления, но эта ежегодная проверка – дополнительный шанс проверить, что все актуально и отвечает ожиданиям.

Хотя эта работа не особенно увлекательная, но это часть гигиены безопасности. Организация, у которой все системы находятся в актуальном состоянии, безопасно сконфигурированы, регулярно обновляются и вообще застегнуты на все пуговицы, находится в лучшем положении с точки зрения безопасности. В такой организации и количество инцидентов безопасности будет меньше, а их последствия не особенно катастрофичны. Считайте, что все это инвестиции в будущее вашей организации.

Оплата технического долга

На протяжении своей карьеры я много раз была свидетелем «накопления технического долга», обычно в форме инцидента безопасности, обошедшего в небольшое состояние, хотя простое применение исправлений могло бы его предотвратить. Мы заботимся о вещах, имеющих для нас значение дома, и должны заботиться о вещах, существенных для нас (и нашей организации) на работе. Технический долг – это (печальный) результат стремления разработчиков поскорее «выкатить» продукт, точно зная при этом, что впоследствии что-то придется переделывать. Работа небрежная, неполная или еще в каком-то отношении проблематичная, и они об этом знают, но *все равно решаются поставить заказчику*. Такие ситуации случаются в основном из-за давления со стороны коммерческого отделения организации.

Технический долг – это долг безопасности.

– Дэйв Льюис⁹⁰

Когда я перешла из группы разработки в группу безопасности, я сразу же стала замечать, что технический долг часто перерастает в проблемы безопасности. Я встречала приложения, которые были успешно атакованы вследствие уязвимости, которую мои средства сканирования выявили еще полгода назад. Я встречала системы, которые даже не тестировались перед сдачей в нещадную эксплуатацию. Я встречала системы, которые сдавались в огромной спешке, поэтому несколько известных уязвимостей не были устранены из-за «нехватки времени»; итог – бесславное уничтожение спустя всего несколько месяцев. Это накопление технического долга в действии, и это ужасно.

Под *оплатой технического долга* понимается возврат назад и починка углов, которые были срезаны в спешке. Это может означать устранение уязвимостей, обновление библиотек, рефакторинг кода и даже полное переписывание (но это уже крайность).

Избежать накопления технического долга можно несколькими способами. Один из них – следовать здоровой практике разработки ПО, избегая срезания углов. Точка. Однако отказывать людям, которые платят вам деньги, может оказаться вредно для вашего будущего трудоустройства, так что это не всегда рекомендуется, а иногда и просто невозможно. Другой способ избежать технического долга – рассказать о своих озабоченностях группе безопасности, если мы видим, как совершается, что нам совсем не нравится. Иногда это работает, но не всегда. Даже группа безопасности иногда бес-
сильна.

Некоторые организации разрешают оставлять замечание в проектной документации, и вы можете воспользоваться этим, чтобы отметить (заведомое) накопление технического долга. Вы можете не только попросить включить свои озабоченности в документацию, но и потребовать у коммерческого подразделения обязательств по срокам оплаты этого долга. Конечно, это проще сказать, чем сделать, и зачастую у одного разработчика не хватит влияния и власти в организации, чтобы протолкнуть нечто подобное.

Если подходы к начальнику или владельцу продукта ничего не дали, то иногда технический долг можно оплатить, не афишируя. В прошлом я несколько раз сама добавляла в крупные проекты небольшие изменения или поручала это сделать своим подчиненным, когда в других работах было затишье (например, во время летних отпусков). Мои начальники редко это замечали, а если и замечали, то я отговаривалась тем, что все мои проекты сдаются в срок и не выходят за рамки бюджета. Если это не та работа, за которую вашей команде платят заказчики, то вряд ли вас осудят за нее, при условии что вся остальная работа выполняется вовремя.

Совет Существуют инструменты оценки качества кода (иногда включаемые вместе с линтерами), которые помогают измерить технический долг и следить за ним. Наличие такого инструмента может сыграть важную роль, когда требуется убедить начальство в необходимости серьезного рефакторинга.

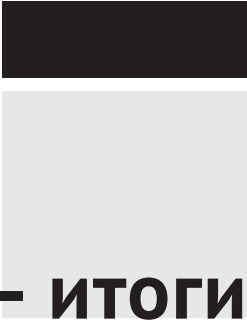
Если вы усердно работали над тем, чтобы избежать накопления большого технического долга, то уровень безопасности вашей организации, вероятно, будет выше, у вас будет меньше инцидентов безопасности, да и вообще количество стрессов в вашей жизни поубавится. Ей-богу, стоит постараться!

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Приведите один пример технического долга (реальный или фиктивный) и опишите, как вы подошли бы к его оплате.
2. Назовите одну причину, по которой важно защищать ваши рабочие учетные записи. Назовите один способ защитить свои рабочие учетные записи.
3. Приведите три примера элементов безопасности. Они могут быть физическими, техническими, административными и (или) превентивными, обнаруживающими или корректирующими. По вашему выбору. Могут быть даже одного вида.
4. Почему так важно избегать атак полным перебором? И своими словами расскажите, почему это трудно.
5. Своими словами объясните, почему важна целостность элемента программного обеспечения.
6. Своими словами объясните, почему согласие пользователя имеет значение.
7. Назовите одно преимущество документации.
8. Назовите одно преимущество комментирования кода.
9. Почему нужно защищать системные вызовы?
10. Назовите одно преимущество повторного использования кода. Назовите одну потенциальную проблему.
11. Почему нужно заботиться о безопасной конфигурации системы? Что может случиться плохого? Приведите один пример.
12. Объясните, что такое белый и черный список.
13. Объясните, что такое надежная система и почему важные функции должны выполняться только в надежных системах.

14. Почему важно, чтобы код был удобочитаемым?
15. Что такое безопасное умолчание? Что такое вымощенная дорога?
16. Приведите один пример безопасного умолчания и вымощенной дороги.
17. Является ли ваш конвейер CI/CD частью цепочки поставок ПО? Объясните свой ответ.
18. Всегда ли мы должны избегать использования зависимостей, содержащих уязвимости? Объясните свой ответ.
19. Когда и почему необходимо выполнять моделирование угроз чему-то?



Часть I – итоги

Контрольный список общих рекомендаций по безопасному кодированию

Если в вашей организации нет инструкции или стандарта по безопасному кодированию, то можете взять за основу этот документ. Хотя этот список не полон, он составлен так, что может быть применен к любой организации, занимающейся созданием большинства типов программного обеспечения. Если вам нужен очень высокий уровень уверенности в безопасности (банки, контртерроризм, совершенно секретные операции, системы, способные привести к гибели людей), то список может оказаться слишком либеральным. Исправляйте и дополняйте его по мере роста своей организации и накопления опыта.

1. Предполагайте, что все внешние системы и данные, с которыми должна взаимодействовать ваша система, небезопасны. Проверяйте все входные данные и подключения к вашим системам; отклоняйте данные и запросы, не отвечающие критериям.
2. Если возможно и разумно, применяйте следующие принципы при проектировании систем и управлении доступом к вашим системам: *наименьших привилегий, нулевого доверия, эшелонированной обороны, безопасности через неясность и уменьшения поверхности атаки.*
3. Применяйте все безопасные умолчания и вымощенные дороги, которые рекомендует ваша группа безопасности, если только не существует требований со стороны бизнеса, запрещающих это делать. В таком случае проработайте со своей группой безопасности альтернативное решение, а затем документируйте новое решение и его причину. Проинформируйте группу безопасности об исключении, если таковое имеет место. Возможно, от вас потребуются дополнительные разъяснения в письменной форме.

4. При проектировании систем предполагайте возможность взлома. Создавайте планы и процессы на случай чрезвычайной ситуации. Если есть возможность, отработайте такие ситуации на практике (это называется *штабными тренировками*).
5. В ваши обязанности входит обеспечение безопасности цепочки поставок создаваемого вами программного обеспечения. Это означает использование безопасных компонентов и проверки безопасности способа их использования, а также защиту вашей IDE, песочницы, системы управления версиями (репозитория кода), ноутбука или ПК и вообще всего того, чем вы пользуетесь при создании ПО.
6. При проектировании средств безопасности уделяйте особое внимание удобству пользования, чтобы обеспечить комплаенс, удовлетворенность пользователей и конкурентные преимущества.
7. Обнаружив ошибку или неожиданное поведение при выполнении транзакции, всегда откатывайте ее и выполняйте процедуру закрытия при отказе. Никогда не оставляйте транзакцию открытой после отказа. Никогда не пытайтесь восстановить часть транзакции. Всегда откатывайте ее целиком и начинайте заново.
8. Соблюдайте требования регуляторов (комплаенс), законы и регламенты, применимые к вашей организации, данным, заказчикам и стране (странам) ведения бизнеса. Запросите разъяснения у группы безопасности и юридического отдела в случае, если вы не уверены в предъявляемых к вам требованиях.
9. Если ваша организация следует каким-то каркасам безопасности, следуйте им и вы. Если не следует, то, возможно, вы захотите принять какой-то на вооружение, например рекомендации, изложенные в этом документе или в книге в целом.
10. По возможности распространяйте полученные знания как можно шире. Не страшно допустить ошибку. Страшно, когда одну и ту же ошибку совершаешь неоднократно. Не стесняйтесь и рассказывайте о своих ошибках коллегам, если это не запрещено.
11. Помните об обратной совместимости. Старайтесь искать баланс между удобством использования и сопутствующими рисками. Документируйте свои решения.
12. Если у вас есть возможность принять участие в сеансе моделирования угроз, сделайте это. Вы не только поможете сделать свои приложения более безопасными, но и научитесь чему-то новому, возможно, даже получите удовольствие. Самостоятельное изучение моделирования угроз – достойное занятие.
13. По возможности старайтесь, чтобы ваши каркасы и зависимости были актуальными, а к системам были применены все исправления. Если возможно, способствуйте упрощению и улучшению процессов применения исправлений на своей работе. Это часть вашей цепочки поставок.

14. Повторно тестируйте исправления ошибок и новые возможности после обнаружения проблем с безопасностью. Регрессионное тестирование проводится не только ради удобства пользования. Автоматизируйте как можно больше работы с помощью правил SAST и (или) автономных тестов.
15. Соблюдайте безопасный жизненный цикл разработки системы (S-SDLC). В идеале в вашей организации такой уже должен существовать. Если нет, последуйте рекомендациям в части III этой книги для создания своего собственного. Скорее всего, ваша группа безопасности не станет возражать против добавления дополнительных мер безопасности в SDLC при условии, что они не принесут вреда (особенно это касается DAST, веб-прокси, фаззинга против производственных систем или организации тестирования на проникновение без их ведома и одобрения). Большинство других мер безопасности разрешены, как правило, без надзора и согласования. Если в вашей организации нет группы безопасности, поговорите с главой ИТ-департамента или со своим начальником.
16. Если вам разрешено выбирать каркас программирования для проекта, то выбирайте современный и используйте все встроенные в него средства безопасности, а не пишите свои собственные. Всегда используйте последнюю стабильную версию каркаса.
17. Проверяйте все входные данные ваших приложений, поступающие из ненадежных источников. Если вы вынуждены принимать потенциально опасные символы, то экранируйте их, а не санируйте (не удаляйте), если это возможно. Если вы не уверены, что источник заслуживает доверия, считайте, что не заслуживает, и обращайтесь с ним соответственно.
18. Производите кодирование всего вывода на экран, а в идеале также данных, которые отправляете другим системам. Это позволит избежать межсайтового скриптинга.
19. Всегда используйте параметризованные запросы при отправке любых запросов базе данных, поскольку в этом случае параметры будут рассматриваться как данные, а не как код, подлежащий выполнению или интерпретации сервером базы данных. Это относится равным образом к базам данных SQL и NoSQL. В идеале следует также использовать каркас объектно-реляционного отображения (ORM), что дополнительно упростит предотвращение атак с внедрением кода на базы данных.
20. Если есть возможность, пользуйтесь готовыми системами, продуктами или встроенными в каркас средствами для управления аутентификацией, идентификацией, авторизацией, доступом и сеансами. Писать такие средства очень трудно, и остается много места для ошибок. Если вы испытываете искушение написать свой собственный механизм, то подумайте о том, сколько проблем с безопасностью имело место в Active Directory за годы работы, и это притом что у корпорации Microsoft

приблизительно миллион работников и подрядчиков, ее прибыль измеряется в триллионах долларов, и она является одной из старейших и самых уважаемых компаний по производству ПО в мире. Если уж для Microsoft это дело оказалось трудным, то, быть может, стоит оставить его специалистам?

21. Используйте для хранения системных секретов инструмент управления секретами и никогда не храните секреты в коде. Используйте средство сканирования секретов, чтобы найти, удалить и сменить все секреты, обнаруженные в кодовой базе. Затем используйте тот же инструмент при каждой новой записи в систему управления версиями, чтобы кто-то по оплошности не поместил секрет в код снова.
22. Пароли пользователей следует подсаливать, а затем хешировать перед сохранением. Перчение – дополнительный уровень безопасности, который вы можете добавить в зависимости от уровня безопасности, необходимого в вашей организации. Выполнение многократного хеширования (коэффициент работы) может уменьшить риск атаки с применением радужных таблиц и, вообще говоря, рекомендуется. Соль – не секрет, но она уникальна для каждого пользователя. Перец – секрет, но все пользователи системы будут использовать одно и то же значение перца. Храните перец в инструменте управления секретами, а соль в базе данных.
23. Все подключения к системам на основе веб-серверов должны производиться по протоколу HTTPS (зашифрованному), использовать нескромно прометированные алгоритмы, длинные ключи и соблюдать криптографические стандарты организации. Если в организации нет политики или стандарта на эту тему, присмотритесь к рекомендациям OWASP, OSSF, NIST или государственным стандартам. Если нет противопоказаний, выбирайте самые строгие рекомендации.
24. Классифицируйте и помечайте все данные, чтобы сделать защиту допускающей автоматизацию и более простой, а также подать визуальный сигнал о том, как обращаться с данными. Конфиденциальные данные следует шифровать в процессе передачи и хранения и в идеале отправлять в запросах и ответах с помощью безопасных cookie (или иным методом, в безопасности которого вы уверены). Документируйте потоки конфиденциальных данных и обязательно тестируйте их на безопасность. Следуйте всем применимым политикам, действующим в вашей организации в отношении защиты конфиденциальных данных.
25. Используйте все применимые заголовки безопасности. Дополнительные сведения о том, что это такое и как их конфигурировать, см. в главе 2 или в шпаргалке по адресу <https://newsletter.shehackspurple.ca/c/securityheaders>.
26. Используйте все параметры безопасности, применимые к файлам cookie, особенно флаги secure и httponly. Шпаргалку см. здесь: <https://newsletter.shehackspurple.ca/c/cookies>.

27. Права доступа следует проверять при каждом запросе. Это относится даже к API. Вы должны скрупулезно проверять авторизацию пользователей при обращении к каждой странице, функции или записи. Код, который это делает, особенно вызовы API, должен быть подвергнут тщательному анализу и тестированию.
28. Все ошибки следует перехватывать и должным образом обрабатывать. Их необходимо протоколировать и при необходимости отправлять уведомления. Шпаргалку см. здесь: <https://newsletter.shehackspurple.ca/c/errors>.
29. Безопасность баз данных подразумевает защиту серверов баз данных, сетей, в которых они размещены, программного обеспечения СУБД и хранимых данных. Все четыре стороны должны быть укреплены, протестированы, документированы и регулярно обновляться. Для них необходимо организовать мониторинг, протоколирование и защиту.
30. В ваши обязанности входит защита файлов, являющихся частью систем, которые вы проектируете, реализуете и сопровождаете. К ним относятся файлы в системе управления версиями, а также файлы, которые ваше приложение предоставляет в общее пользование, сохраняет или использует. Применяйте (и мониторьте) строгий контроль доступа, особенно осторожно относитесь к файлам из ненадежных источников, регулярно делайте резервные копии и храните файлы в зашифрованном виде.
31. Если есть возможность, пользуйтесь языками, безопасными относительно памяти. Если вы вынуждены использовать языки, не обладающие этим свойством, то во избежание переполнений – буфера, стека, строки и целого – принимайте следующие меры: тщательно проверяйте входные данные, не пользуйтесь функциями, которые заведомо могут приводить к переполнению (выходу за границы области), при тестировании безопасности уделяйте особое внимание этим местам кода, а также следуйте другим рекомендациям на эту тему в главе 3.
32. Никогда не десериализуйте объекты из ненадежных источников. Если этого никак не избежать, то применяйте контрмеры, описанные в главе 3, и рекомендации из шпаргалки OWASP на эту тему. Будьте предельно осторожны.
33. Проектируйте и работайте с системами, которые уважительно относятся к праву пользователей, граждан, заказчиков и работников на неприкосновенность частной жизни. Если вы не согласны с решениями своей организации в части конфиденциальности пользователей, задавайте вопросы, не молчите, возражайте. Пусть ваш голос говорит от имени пользователей. Я уверена, что в мире, где маркетологи подсматривают за самыми интимными моментами нашей жизни, пользователи скажут вам за это спасибо.
34. Во всех приложениях должны быть включены протоколирование, мониторинг и (когда уместно) уведомление. Даже для ваших API!

35. Избегайте потенциальных состояний гонки, для чего блокируйте ресурсы перед использованием. Примечание: большинство современных каркасов программирования делают это за вас!
36. Разрешайте вставку из буфера обмена в поля паролей, чтобы пользователям было проще использовать менеджеры паролей. Это относится ко всем полям паролей, в том числе и тем, что предназначены для повторного ввода пароля. Запрет вставки заставляет пользователей вводить пароль вручную, а значит, они будут придумывать менее сложные, короткие пароли, снижая тем самым уровень безопасности.
37. Используйте строго типизированные языки или позаботьтесь о том, чтобы всегда определять типы и единообразно использовать их во всем приложении. Не применяйте поэтическую лицензию к переменным; неоднозначность – враг безопасности и надежности программного обеспечения. Если вы решите остановиться на слабо или динамически типизированном языке, внимательно тестируйте его на потенциальные проблемы неоднозначности типов.
38. Предметно-ориентированная разработка – способ проектирования объектов, способствующий безопасности, и, если у вас есть возможность воспользоваться ей, радуйтесь! Хотя она не особенно популярна, но, безусловно, заслуживает внимания в силу интуитивной понятности.
39. Чтобы удостовериться в безопасности своих проектов, выполняйте моделирование угроз и рецензирование дизайна и применяйте безопасные концепции и парадигмы (процессы) проектирования. Затем привлечите эксперта, который вручную проведет тестирование безопасности (тестирование на проникновение или оценку безопасности).
40. По возможности предоставляйте безопасные умолчания для параметров, конфигураций и других значений, задаваемых пользователями. Если вы зададите значение, обеспечивающее очень высокую безопасность или конфиденциальность, то есть шанс, что пользователь оставит его в таком виде, а значит, будет лучше защищен.
41. Всегда старайтесь писать удобочитаемый (интуитивно понятный и хорошо документированный) и аудитопригодный (с комментариями, описывающими элементы безопасности) код.
42. Следите за тем, чтобы важные функции выполнялись в надежных системах (т. е. на сервере). Не делайте ничего важного во фронтальном JavaScript-коде, который можно обойти с помощью веб-прокси.
43. При проверке входных данных используйте белые списки вместо черных. Большинству злоумышленников не составит труда обойти черный список, к тому же поддерживать белый список гораздо проще.
44. Проверяйте конфигурацию безопасности всех подведомственных вам систем не реже раза в год. Следуйте руководству по укреплению. Если система достаточно важна, проводите тестирование на проникнове-

- ние. Никогда не оставляйте пароли по умолчанию, всегда заменяйте их более длинными, сложными и обязательно уникальными. Производите обновление систем в разумные сроки (не откладывая на недели или месяцы), особенно это касается исправлений безопасности.
45. При подключении к другим системам через интернет или любую ненадежную сеть проверяйте, что сертификат действителен и выписан именно тому хосту, к которому вы подключаетесь. Это проверка целостности.
 46. По возможности пишите повторно используемый код. Если собираетесь использовать код повторно, то проводите тестирование безопасности перед выпуском или повторным использованием. По прошествии времени повторяйте тестирование безопасности и в случае обнаружения и исправления ошибки вносите изменения во все места, где исправленный код используется.
 47. Проявляйте особую осторожность при выполнении системных вызовов и, если есть такая возможность, вообще избегайте их. Никогда не передавайте полученные от пользователя данные напрямую системному вызову.
 48. Оставляйте в коде полезные комментарии, так его будет проще понять и сопровождать. Документируйте приложения и проектные решения. Комментарии и документация помогают в большинстве форм тестирования безопасности, а иногда и при расследовании инцидентов.
 49. Запрашивайте согласие пользователя и документируйте его (в базе данных). Не предполагайте согласие по умолчанию, всегда запрашивайте явно.
 50. Проверки целостности могут предотвратить многие виды атак, поэтому не жалейте на них времени. Подписание кода и неизменяемые сборки – два метода, гарантирующие целостность систем.
 51. Предотвращение атак полным перебором на функциональность экрана входа в систему – всегда результат обдуманного баланса между защитой и удобством пользования. Проконсультируйтесь с бизнес-подразделением по вопросу разумных лимитов, а с группой безопасности – по поводу мониторинга и реагирования на имеющую место атаку. Тщательно протоколируйте все операции входа (успешные и неудачные). Отправляйте уведомление, если достигнуты лимиты, позволяющие предположить, что против вас идет атака. Тестируйте тщательнее.
 52. Группа безопасности (приложений) может попросить вас добавить в свои программные проекты элементы безопасности. Выполните эту просьбу. Напомним, что технические элементы безопасности бывают трех видов: превентивные, обнаруживающие и корректирующие.
 53. Используйте все имеющиеся превентивные меры, чтобы защитить *свои* учетные записи с расширенными привилегиями, в том числе многофакторную аутентификацию (МФА), длинные, сложные и уникальные

пароли, а также менеджер паролей. Для страховки зарегистрируйтесь на сайте haveibeenpwned.com. Незамедлительно меняйте пароли, заподозрив взлом.

54. Технический долг часто создает долг безопасности, а потому понижает уровень безопасности вашей организации. Более того, становится труднее реагировать на инцидент безопасности, если вы одновременно пытаетесь исправить ошибку или обновить зависимости. Усердная работа, направленная на предотвращение или оплату технического долга, не только упростит сопровождение ваших приложений, но и поможет сделать их безопаснее!

Хотя этот список длинный и содержит трудные задачи, усилия, затраченные на следование ему, помогут в создании надежных и защищенных приложений, которые трудно атаковать. Так вы защитите свою организацию и станете лучшим программистом. Желаю вам всяческих удач на пути к написанию более безопасного кода!

Искренне ваша,
Таня Янца

Конкретные рекомендации

Глава 5. Рекомендации, зависящие от технологии

Глава 6. Популярные языки программирования

Глава 7. Популярные каркасы

Глава 8. Категории уязвимостей

Рекомендации, зависящие от технологии

В этой главе мы отправляемся в путешествие, которое охватывает не одну, не две, а целых пять увлекательных тем: API, мобильные приложения, веб-сокеты, IoT-устройства и бессерверные вычисления. Готовьтесь укрепить свои приложения, потому что нам открывается удивительный мир обеспечения безопасности этих современных технологий!

Прочитав главу до конца, вы будете обладать целым арсеналом знаний о безопасности и сможете сделать приложения этих типов надежными, а главное, *безопасными*.

Рекомендации по безопасности API

В этой части книги мы будем рассматривать лучшие практики обеспечения безопасности Web API, уделив особое внимание REST-совместимым API, использованию протокола OpenAPI вместо SOAP (стремительно теряющего популярность)⁹³ или GraphQL (который набирает популярность, но пока еще составляет лишь малую долю от всех используемых в мире API)⁹⁴. SOAP и GraphQL в этой книге не рассматриваются.

Примечание Кое-что из описанного в этом разделе относится и к другим разделам этой главы. Эти материалы обозначены *. Применяйте их ко всем технологиям, а не только к тем, о которых читаете в этом разделе.

API – это приложения, которые предоставляют услуги другим приложениям, взаимодействуют от их имени и зачастую выступают в паре с фронтальным интерфейсом или каким-то файлом с документацией API (например, файлом определения OpenAPI). Хотя технически у API нет фронтального интерфейса, они все же постоянно сталкиваются с атаками из веба и являются новой мишенью злоумышленников⁹⁵. Поскольку API часто связаны с нашими данными и внутренними системами, а сами являются системами, от которых мы зависим, мы должны защищать их так же тщательно, как любое веб-приложение или иное программное обеспечение.

Две основные угрозы API – боты (автоматизированные скрипты в интернете, которые постоянно совершают атаки полным перебором или атаки типа «отказ в обслуживании») и некорректная авторизация (предоставленный по ошибке доступ к данным, функциям, системам и т. д.). В списке 10 главных угроз безопасности API, опубликованном OWASP в 2023 году, взлом управления доступом обсуждается в 4 из 10 случаев; это серьезная проблема. В этом разделе мы обсудим, как избежать этих и многих других угроз, а также как конструировать удобные для сопровождения и надежные API⁹⁶.

Рекомендации

- Первое, что нужно сделать, чтобы обезопасить свои API, – провести инвентаризацию⁹⁷ и убедиться, что должное внимание уделено безопасности всех конечных точек API (и действиям против них), а не только большинства. Вы должны включить документацию об их работе, типах принимаемых ими запросов, лимитах и т. д. Сошлитесь на документацию из инвентарной описи, чтобы ее было легко найти.
- Далее позаботьтесь о том, чтобы доступ к API был возможен только через шлюз. Используйте шлюз API для аутентификации и авторизации первого уровня, а также для ограничения темпа запросов, дросселирования и квотирования ресурсов. Также подумайте о включении контроля входных данных, если эта возможность является частью функциональности конкретного шлюза API⁹⁷. Примечание: вы все равно должны проверять входные данные самостоятельно; проверок на шлюзе недостаточно.
- Добавьте в свой код дополнительные уровни авторизации и управления доступом сверх предлагаемых шлюзом и тщательно протестируйте их. Проверяйте доступ к каждой странице и к каждому вызову функции⁹⁹. Составьте матрицу управления доступом, которая покажет, какие функции и средства разрешено вызывать каждому пользователю, а затем вручную проведите тестирование по этой схеме (автоматизированного тестирования для этого вида функциональности никогда не бывает достаточно). Повторяйте тестирование после выпуска каждой основной версии, а также любых изменений в способе работы с этой функциональностью в системе.
- Для аутентификации используйте существующие системы (OAuth или JWT, начиная с версии 2); не пишите свою собственную (тут очень легко наделать ошибок)⁹⁹.
- Используйте ограничение темпа⁹⁶. Ставьте лимиты на все, что можете. Также задавайте квоты ресурсов. Например:
 - тайм-аут на выполнение;
 - максимальный размер выделяемой памяти;
 - количество файловых дескрипторов;
 - количество процессов;
 - размер полезной нагрузки в запросе (например, загружаемого файла);

- количество запросов в расчете на одного клиента или ресурс (это также называется квотой ресурса);
 - количество записей на странице, возвращаемых в одном ответе.
- Соблюдайте практики безопасного кодирования (проверку, санирование и экранирование входных данных, кодирование вывода и параметризованные запросы).
 - Шифруйте данные в процессе передачи (и при хранении!).
 - Используйте эшелонированную оборону для аутентификации, чтобы избежать атак полным перебором (ограничение темпа) и с подстановкой учетных данных (сбор данных об угрозах⁹⁷, предотвращение использования украденных учетных данных пользователями, обязательная смена паролей после кражи учетных данных).
 - Для подключения к другим системам используйте только учетные записи служб, а не физических лиц (в том числе программистов). Настройте мониторинг и протоколирование учетных записей служб для обнаружения подозрительного поведения.
 - Обработывайте секреты из программы и храните их в средстве управления секретами, а не в своем коде.
 - Не возвращайте лишних данных, планируя игнорировать их на стороне клиента. Злоумышленники знают, что программисты так поступают для простоты, и нещадно эксплуатируют это упущение. Если данные не покидают сервер, то и о защите их на стороне клиента можно не беспокоиться. Применяйте к данным принцип наименьших привилегий⁹⁷.
 - Настройте максимальную длину ответа⁹⁸ на своем брандмауэре веб-приложений (WAF) для API. WAF служит для фильтрации запросов, поступающих веб-приложениям и от них, с целью защиты от атак. Эти WAF'ы создаются только для API, которые фильтруют запросы, адресованные API и поступающие от API, чтобы исключить потенциально вредоносный или недопустимый вывод (например, вряд ли вы хотите возвращать все содержимое базы данных). Задайте размер чуть больше максимального ответа от API, имевшего место в предыдущем месяце (после проверки приемлемости данных). Один мегабит для изображения – мало, но 1 Мбит для JSON – очень много, и вряд ли вы хотели вернуть столько пользователю. Это может предотвратить эксфильтрацию (проще говоря, кражу) конфиденциальных данных.
 - В процессе аутентификации⁹⁹ следуйте рекомендациям по входу в систему (используйте подсолненные и хешированные хранимые пароли, предлагайте пользователям многофакторную аутентификацию, отслеживайте украденные учетные данные, требуйте задания сложных и длинных паролей) и блокируйте атаки полным перебором, отслеживая неудачные попытки входа (как для быстрых, так и для медленных атак).
 - Дополнительно защищайте данные путем их классификации и пометки, чтобы к ним всегда относились с должными предосторожностями.

- Защищайте параметры API⁹⁹ от изменения пользователями (атак), используя только строгие типы данных (целые или булевы, а не «fun(x, y)» или еще какой-то придуманный вами тип) и отклоняя все неправильно форматированные запросы (даже если можете их обработать, не делайте этого). Не принимайте параметры, которых ваша функция не запрашивала. (Если в запросе есть лишние параметры, отклоняйте запрос целиком – это атака. Не пытайтесь его обработать.)
- Избегайте подробных сообщений об ошибках, содержащих конфиденциальную информацию. Системная информация, данные о крахе и трассы стека никогда не должны включаться в сообщения об ошибках. Будьте расплывчаты, эти сообщения должен читать только компьютер. На системные ошибки вы должны быть заранее готовы отреагировать, а не передавать объект ошибки дальше с последующим созданием пользовательского сообщения на основе его содержимого⁹⁹.
- Используйте ориентированный на API линтер (в идеале плагин для IDE)¹⁰⁰ и (или) плагин статического анализа, который помогает искать в коде ошибки безопасности или отсутствующие элементы безопасности¹⁰¹. Использование инструмента, который помечает ошибки красными извилистыми линиями и дает рекомендации по улучшению кода, помогает писать высококачественный код. Пользуйтесь – не пожалеете.
- Вовремя и полностью выводите старые API из эксплуатации, когда выпускаете новую версию. Старые версии API не следует оставлять работающими или подключенными к активным системам, если их мониторинг и сопровождение прекращены. Можно оставлять старые системы в целях обратной совместимости, но тогда их следует безопасно и активно обслуживать.
- Шифруйте все запросы и ответы API (не только данные). Передавайте все только по HTTPS и никогда по HTTP⁹⁹.
- Постоянно улучшайте и проверяйте безопасность своих API⁹⁹. Каждую неделю прогоняйте анализ состава программы (SCA) и статическое тестирование безопасности приложений (SAST) для API в репозитории кода, а также динамические и (или) интерактивные тесты работающих API. В случае обнаружения серьезной ошибки в одном API проверяйте, нет ли ее также в других. Изучите журналы, чтобы понять, была ли она эксплуатирована, а затем создайте охранный механизм с помощью пользовательских правил SAST, чтобы предотвратить ее повторное появление.
- Убедитесь в безопасности дизайна своей системы, выполняя для нее моделирование угроз и применяя методы безопасного проектирования, рассмотренные ранее в части I книги. Выявление просчетов в бизнес-логике часто является ручным процессом, в этом может помочь хороший пентестер.
- Если ваши системы потребляют сторонние API, обязательно выполняйте проверку входных данных, их санирование (если принимать потен-

циально опасные символы необходимо), экранирование (если требуется) и кодирование вывода для всех принимаемых данных. Проверяйте, что получаете именно то, что ожидали, включая формат и тип, а если это не так, то без церемоний отклоняйте данные⁹⁶.

- Следует также¹⁰³ использовать отдельные ключи API для каждого приложения, удалять старые ключи API, хранить ключи в инструменте управления секретами, вовремя менять украденные ключи и периодически проводить аудит для гарантии того, что ключи не используются неподобающим образом.
- Чтобы избежать подделки запроса на стороне сервера (server-side request forgery – SSRF), нового пункта среди главных 10 угроз безопасности API по версии OWASP⁹⁶, принимайте только URL-адреса из белого списка одобренных, тщательно тестируйте код разбора URL-адресов, разрешайте доступ к внутренним ресурсам только через учетные записи служб (не позволяйте своему приложению запрашивать ресурсы у недопустимых или неожиданных серверов) и применяйте принцип нулевого доверия при проектировании сети (по умолчанию блокируйте все порты, учетные записи служб и т. д., если только со стороны бизнеса нет требования разрешить такое подключение)¹⁰².
- Протоколирование, мониторинг и уведомление должны быть включены⁹⁶. Защищайте журналы и обращайтесь с ними как с конфиденциальными и ценными данными, каковыми они и являются. Также убедитесь, что в журнал не попадают конфиденциальные данные, например идентифицирующие персональные данные (PII), медицинские персональные данные (PHI) или полные номера кредитных карт. Если хотите действовать на опережение, также проводите поиск угроз, просматривая свои журналы на предмет необычных и подозрительных действий.
- Блокируйте все неиспользуемые HTTP-методы (иногда они называются *глаголами*), например HEAD и TRACE. В большинстве API используются методы GET и POST, реже DELETE, PUT и UPDATE. Остальные методы, которые не используются, должны быть запрещены.
- Если в производственной среде включен протокол webdav (он предлагает еще больше HTTP-методов для использования во время разработки), то *выключите* его. Он не предназначен для работы в режиме эксплуатации и предоставляет злоумышленником дополнительные HTTP-методы для атаки на ваши API. Это пример применения принципа наименьших привилегий.
- Используйте сетку служб, если к вашим API производится *очень много* обращений и наблюдаются задержки или иные проблемы с их обслуживанием. Сетка служб не нужна, если имеется всего несколько API, но, когда количество обращений измеряется сотнями тысяч в день, она может оказаться полезной. Заодно вы получаете сквозное шифрование, не внося никаких изменений в код!

- Пишите стандарты кодирования и другие руководящие документы по безопасности для API в своей организации, в идеале включив в них пункты из этой части книги. В документах должно быть описано, какие протоколы API разрешены, а какие нет (например, OpenAPI разрешен, а SOAP и GraphQL не разрешены, если такова ваша политика). Широко распространяйте их, популяризируйте и обучайте им своих коллег.
- Следуйте всем советам из части I этой книги, применимым к API!

Совет Обычно я не привожу названий продуктов в своих книгах, но следующие бесплатны, поэтому надеюсь, что вы не будете возражать, любезные читатели. Плагин для IDE от компании 42Crunch – полусвободный линтер для API, который мне очень нравится, – его так легко использовать! Он вместе с вами работает над укреплением файла определения OpenAPI, чтобы сделать его надежным и защищенным. Добавление SAST-плагины для IDE от компании Semgrep и подключение его к бесплатному инструменту Semgrep Community позволит проверить ваш код, а красные извилистые подчеркивания покажут ошибки в коде API. Если ваш бюджет ограничен, то эти плагины – отличная (и бесплатная) отправная точка для путешествия в мир инструментальных средств разработки API. Далее в этой книге я назову еще несколько бесплатных инструментов.

ВЕЖЛИВЫЕ СООБЩЕНИЯ ОБ ОШИБКАХ

Бобу как-то довелось присутствовать на тесте на проникновение, и это было захватывающе. Поначалу пентестер посылал запросу новому API, разработанному командой Боба, и он гордился собой, читая очень вежливые сообщения об ошибках, которые система возвращала в ответ на некорректные запросы. Но, прочитав парочку сообщений, пентестер, широко улыбнувшись, сказал: «Благодаря этим полезнейшим сообщениям об ошибках я сумел скорректировать атаку на синтаксис и успешно провел атаку с внедрением кода!»

Боб был ошарашен: как такое могло случиться? Пентестер показал ему последовательность отправленных запросов. Первый был очень плохо отформатирован, но благодаря сообщению об ошибке второй был уже гораздо ближе к истине. Через пять или шесть попыток код был успешно внедрен, и база данных заснула на 30 с, демонстрируя, что все сработало. Боб канадец и гордится хорошими манерами, поэтому он думал, что подробные и полезные сообщения об ошибках – неотъемлемая часть высококачественного приложения! Но пентестер объяснил, что API вызываются *машинами*, а человек должен видеть эти сообщения только на этапе разработки и тестирования. Все остальное время их видят только компьютеры, если, конечно, не вмешается злоумышленник. «Но ведь ты не хочешь быть вежливым со злоумышленником, правда, Боб? Ты ведь не хочешь ему помогать. Поэтому делай так, чтобы твои сообщения об ошибках не выдавали ничего. Некорректный запрос, попробуйте еще раз. И этого достаточно». Боб был шокирован. Он чувствовал себя таким глупым! Когда позже он объяснил это своей команде, все все поняли и больше никогда такой ошибки не допускали.

НЕ ТАКОЙ УЖ ДИНАМИЧНЫЙ DAST

Алиса отвечала за безопасность одной из порученных ей на работе систем. Ее спросили, чего она хочет: провести тест на проникновение или воспользоваться автоматическим инструментом для тестирования API, которые разрабатывала ее команда. Вкрадчивый продавец, отряженный поставщиком, сообщил ей, что инструмент динамического тести-

рования безопасности приложений (DAST) мог бы протестировать ее API не хуже пентестера и что она может использовать его для тестирования всех API во всей организации. Алиса подумала, какой она молодец, что купила систему, способную протестировать все на свете, вместо того чтобы платить кучу денег пентестеру за тестирование всего одного API. Когда команда Алисы протестировала первый API, все прошло без сучка без задоринки – никаких ошибок безопасности. Алиса ликовала, что ее команда сделала такой отличный API! Они протестировали второй, снова все прошло отлично. Алиса искренне гордилась своей командой и домой пошла в приподнятом настроении.

Через несколько дней человек, отвечавший за DAST-сканирование, подошел к Алисе с озабоченным выражением лица. «Алиса, я протестировал 27 твоих API и *ни в одном* не нашел *ни единой* ошибки. Не знаю, как бы это сказать... но такого просто не может быть. Мы только начали разрабатывать API, мы не проходили курса безопасности, и я не хочу сказать ничего плохого, но мне кажется, что этот инструмент не работает». Продолжение следует...

ДОРОГОСТОЯЩИЕ ЛЖИВЫЕ ОБЕЩАНИЯ

Алиса позвонила поставщику DAST и спросила, почему их инструмент не нашел никаких ошибок в ее API. Ей сказали, что ее файл определения API был неправильно отформатирован, и нужно внести в код несколько мелких изменений. Алиса и вся команда долго общались с поставщиком, снова и снова изменяя файл определений (схемы) API, чтобы заставить DAST правильно взаимодействовать с API. После долгой возни он наконец нашел парочку ошибок, но разработчики негодовали, что пришлось внести столько изменений в код только для того, чтобы «эта ее хренова штука заработала». Когда Алиса сложила все временные затраты, получилось, что они потратили почти все время, отведенное ей на исправление ошибок безопасности в проекте! И не было никакой возможности потратить столько же времени на все остальные API, чтобы к ним можно было применить этот инструмент. В итоге ей пришлось запросить дополнительный бюджет, репутация ее при этом пострадала, а пентестера нанимать все равно пришлось. Он нашел несколько вещей, который DAST пропустил, и написал очень полезный отчет. Поставщик все это время настаивал, что инструмент «работает с API», несмотря на ужасные полученные результаты. Алиса чувствовала себя неловко из-за того, что поверила обещаниям продавца, не убедившись сначала, что инструмент делает то, что ей нужно. Ее проект вышел за рамки бюджета и был сдан с опозданием на два месяца. Какое унижение! Отныне она будет проводить проверку концепции (POC), настаивать на том, чтобы установить инструмент, тщательно протестировать его и, только убедившись, что он работает, как рекламируется, что-то подписывать.

ПИВО И ОШИБКИ

Боб экстрасверт и любит пиво, а значит, он готов выпить кружечку с любым, кто пригласит. Как-то раз он пошел по пиву с пентестером, который сказал ему, что в тот день он тестировал вторую версию API и по наитию вызвал версию 1, чтобы проверить, ответит ли она. Ответила! Тогда он начал ее тестировать и обнаружил *катастрофу*. Он нашел *столько ошибок безопасности!* Он сообщил об этом нанявшему его менеджеру, и тот был впечатлен донельзя! Он рассказал Бобу, что это его «секретное оружие»: всегда искать старые версии тестируемого API, потому что если такие удастся найти, то это почти всегда удача. Боб сделал себе заметку – проверить, что все старые версии API, написанные *его* командой, деактивированы! Отличный совет!

В этом разделе мы рассмотрели рекомендации не только для API, но и для систем многих других типов, которые встретятся нам в этой главе. Не забывайте применять советы, помеченные *, ко всем технологиям в этой главе, а не только к API!

Рекомендации по безопасности мобильных приложений

Мы будем здесь обсуждать платформенные и гибридные мобильные приложения, а не веб-приложения, которые притворяются мобильными, но на самом деле являются просто ярлыком в вашем телефоне, ведущим на веб-приложение в интернете (эта технология называется прогрессивными веб-приложениями [PWA]). Читая этот раздел, помните, что значительная часть мобильной функциональности предоставляется веб-API, и применяйте полученные ранее знания к API, на которых основаны мобильные приложения.

Рекомендации

- Мобильные приложения часто декомпилируют в процессе обратного проектирования (чтобы украсть ваш IP-адрес или узнать, как атаковать ваше приложение), поэтому обфусцируйте и минимизируйте код, чтобы максимально затруднить эту задачу¹⁰⁴. На рынке есть инструменты, которые умеют это делать; самостоятельно трудно сделать это правильно. А если вы сделаете плохо, то обфускацию легко будет обратить, а тогда и делать этого не стоило.
- Используйте эмулятор¹⁰⁴ или реальные устройства (несколько смартфонов разных типов), чтобы посмотреть, как приложение будет работать на разных устройствах. Тестируйте на возможно большем числе устройств. Это относится к тестированию не только безопасности, но и любой функциональности.
- Хранилища для мобильного приложения и для обычного веб-приложения могут сильно отличаться. Мы часто хотим сохранять данные на телефоне, чтобы не загружать их снова и снова по сети и тем уменьшить взимаемую с нашего пользователя плату, а заодно сократить задержку (время, в течение которого пользователь ждет завершения загрузки данных). Памятуя об этом, согласуйте с группой безопасности, (1) что хранить данные локально на телефоне можно и (2) как, с их точки зрения, это лучше делать. Всегда храните данные в зашифрованном формате. Тщательно тестируйте функциональность, чтобы убедиться в том, что она защищена от вмешательства пользователя.
- Не храните конфиденциальные данные в следующих местах¹⁰⁴:
 - в журнале приложения;
 - на серверах третьей стороны;

- в кеше клавиатуры;
 - в механизме IPC;
 - на устройстве пользователя во время взаимодействия.
- Конфигурируйте автоматическое удаление локально хранимых данных¹⁰⁴ по истечении определенного промежутка времени, определяемого бизнес-требованиями и принятой в организации политикой информационной безопасности.
 - * Никогда не храните ключ шифрования вместе с данными, которые им зашифрованы¹⁰⁴.
 - Запрашивайте только те разрешения на устройстве, которые необходимы приложению для работы. Иногда разработчики запрашивают кучу всего, просто потому что могут, но это также означает, что если в один прекрасный день ваше приложение будет скомпрометировано, то разрешения можно будет обратить против пользователей. Ограничивая свои запросы, мы ограничиваем потенциальный ущерб в случае инцидента (это пример принципа «предполагайте возможность взлома», о котором мы говорили ранее).
 - Если это укладывается в рамки вашего бюджета, разверните средства обнаружения несанкционированного вмешательства, которые уведомят о том, что кто-то пытается атаковать ваше мобильное приложение¹⁰⁵.
 - Продолжительность сеансов в мобильном приложении дольше, чем в большинстве приложений для ПК¹⁰⁵, поэтому управление сеансами приобретает особое значение. Для защиты сеанса следуйте рекомендациям, изложенным в части I книги, и плюс к тому:
 - используйте для идентификации сеансов не идентификаторы устройств, а токены, которые можно отозвать в случае кражи или утери устройства;
 - предусмотрите удаленное стирание данных;
 - предусмотрите удаленный выход и при этом уничтожайте сеанс;
 - предусмотрите автоматическое завершение сеанса на случай, если пользователь не инициировал выход, прежде чем закрыть приложение.
 - Шифруйте все хранящиеся локально на устройстве файлы, включая ваш исходный код¹⁰⁵. Всегда используйте «последнее и лучшее» (текущий отраслевой стандарт), когда речь заходит о шифровании.
 - Используйте сертификаты для подписания кода. Это подтверждает целостность кода приложения конечному пользователю¹⁰⁵.
 - * Подключайтесь к API только после аутентификации и авторизации¹⁰⁵.
 - * В различных областях вашего приложения, скорее всего, необходимы различные уровни авторизации. Проверяйте авторизацию для каждой страницы, средства или функции (в зависимости от конфиденциальности).

- * Если вы можете позволить себе реализацию биометрии, ключей доступа или сторонней службы аутентификации, то обычно это более безопасно, чем реализация схемы «имя пользователя + пароль». Пользуйтесь такой возможностью¹⁰⁵.
- * Предлагайте пользователям двух- или многофакторную аутентификацию, если можете себе позволить.
- *Безопасно* храните конфиденциальные данные в памяти, стирая их после использования¹⁰⁵. Удостоверьтесь, что действительно стерты сами данные, а не просто указатель на них, так что данные по-прежнему находятся в невыделенной области памяти, откуда их легко может достать злоумышленник. Будьте особенно осторожны с ключами шифрования!
- Будьте осторожны с хранением чего-либо в памяти устройств на Android, потому что обращение с памятью в таких устройствах не такое, как на обычных веб-серверах¹⁰⁶. Возможно, вам придется заполнить флеш-память мусорными данными, чтобы гарантировать, что она действительно стерта. Примечание: это может замедлить работу приложения и телефона; будьте внимательны и тщательно тестируйте.
- Не используйте универсальный уникальный идентификатор (UUID) для идентификации учетной записи, он присваивается телефону на всю жизнь, но телефоны меняют владельцев¹⁰⁷. Это вопрос конфиденциальности и безопасности.
- * Будьте осторожны при отслеживании и обработке географических данных. Не будьте навязчивым преследователем, отслеживайте только те данные, которые вам действительно *необходимы* (см. обсуждение выше). Также имейте в виду, что информация о местоположении попадает в ваши системы телеметрии, журналы и, возможно, еще в какие-то системы. Старайтесь соблюдать баланс между конфиденциальностью пользователя и потребностями бизнеса.
- Компилируйте настройки приложения *в код*, а не храните их в локальном файле, например plist, XML или SQLite. Внешние файлы не всегда надежно защищаются операционными системами телефона¹⁰⁸.
- Если данные приложения конфиденциальны, то следите за тем, чтобы они не попали в резервные копии устройства (если только приложение не шифрует их перед резервным копированием). В противном случае вы создаете утечку собственных данных¹⁰⁹.
- * Как и в предыдущем совете, не кешируйте конфиденциальную информацию приложения на телефоне.

Рекомендации для IOS

- Отключайте режим автоматического исправления для любой конфиденциальной информации, а не только для паролей. IOS запоминает все нажатия нецифровых клавиш (в кеше клавиатуры) для автоматического исправления опечаток, а для конфиденциальных данных это проблема безопасности¹¹⁰!

- Отключайте копирование и вставку всех конфиденциальных данных, кроме паролей (последнее – чтобы можно было воспользоваться менеджером паролей).
- Не пользуйтесь WebViews и NSString во избежание фрейминга и перехвата кликов¹¹¹.
- Пользуйтесь функциональностью связки ключей очень осторожно, в частности обдумывайте выбор класса защиты и избегайте резервных копий iTunes¹¹².
- Блокируйте кеширование моментальных снимков приложения, поскольку это очень близко к кешированию конфиденциальной информации¹¹³.
- Реализуйте средства обеспечения безопасности памяти, чтобы избежать атак с переполнением буфера, а именно: автоматический подсчет ссылок (ARC), рандомизацию распределения адресного пространства (ASLR) и защиту от разбиения стека¹¹⁴.
- Явно отключайте кеширование запросов и ответов по протоколу HTTP(S) путем задания свойства cachePolicy объекта NSURLRequest¹¹⁵.
- Реализуйте транспортную безопасность приложения (App Transport Security – ATS) для гарантий безопасных подключений к своим серверам¹¹⁶.

Рекомендации для Android

- Задавайте права доступа к файлам приложения, так чтобы их нельзя было прочитать вне приложения, и дополнительно шифруйте их. Например, не задавайте разрешение на чтение для всех, если нет *абсолютной уверенности* в том, что это правильно. Если все же задаете, то документируйте свое решение¹¹⁷.
- Используйте «намерения» и «широковещание» очень осторожно, проверяйте свои «активности» и следуйте рекомендациям по работе с «web-view». При работе с этими средствами Android очень легко допустить ошибку, пользуйтесь ими осмотрительно и тщательно тестируйте¹¹⁸.
- Взаимодействуйте с поставщиками контента очень осторожно. Не давайте им прав записи, если в этом нет острой необходимости¹¹⁹.
- Защищайте службы приложения от внешних обращений, блокируя их всеми возможными средствами (особенно путем установки и проверки разрешений)¹²⁰.
- Избегайте хранения кешированных фотографий, если изображения могут оказаться конфиденциальными. Они хранятся в NAND-памяти¹²¹.
- Не кешируйте объекты GUI, например конфиденциальные изображения. Если это необходимо, то затем выполняйте полный выход из приложения (чтобы стереть кеш) или каким-то иным способом добейтесь, чтобы объекты удалялись из кеша после завершения сеанса или выхода из приложения¹²².
- Подписывайте свой Android APK действительным сертификатом¹²³.

Помимо всего этого, я рекомендую прочитать шпаргалку OWASP по безопасности мобильных приложений: https://cheatsheetseries.owasp.org/cheatsheets/Mobile_Application_Security_Cheat_Sheet.html.

МАСТЕР СКРИПТОВ

Однажды, когда Алиса замещала руководителя группы безопасности, ушедшего в отпуск, случился крупный инцидент безопасности. Алисе пришлось поприсутствовать на разных встречах и услышать всю секретную информацию о том, что произошло. На одной из встреч исследователь сказала, что код был обфусцирован, но она написала скрипт де-обфускации. Алиса спросила, сколько времени это заняло, и услышала в ответ – между двумя встречами. Алиса была поражена! Исследователь сказала ей, что это была «не продвинутая, а простенькая обфускация». Очевидно, злоумышленник был не очень квалифицированным, и за пару часов они во всем разобрались. Но Алису очень впечатлила работа с этой командой!

Рекомендации по безопасности WebSocket

WebSocket – коммуникационный протокол, допускающий организацию одновременной двусторонней связи по одному TCP-соединению¹²⁴. Обычно он используется для передачи данных в режиме реального времени, например последних новостей. В протоколе HTTP не хранится информация о состоянии, но у веб-сокетов состояние есть, зато накладные расходы меньше. Веб-сокеты часто применяются для таких вещей, как чаты / обмен сообщениями, двунаправленная передача в реальном времени и «постоянные обновления», тогда как HTTP – в основном для доставки статического содержимого или доступа к REST API¹²⁵. Веб-сокеты также могут посылать данные в обоих направлениях по одному соединению *одновременно*.

Рекомендации

- Применяйте все, о чем узнали в этой книге.
- Используйте протокол безопасных веб-сокетов (Secure WebSocket Protocol – WSS) для шифрования подключений¹²⁶.
- Введите ограничения на темп, как и в случае API. Это избавит вас от несуразно больших счетов от поставщика облачных услуг и предотвратит разнообразные атаки на ресурсы¹²⁶.
- Следите за попытками перехвата управления и пользуйтесь политикой одного источника или CORS-заголовками безопасности (Cross-Origin Resource Sharing – совместное использование ресурсов между разными источниками), чтобы этого избежать¹²⁷. Ни то, ни другое средство по умолчанию не включены, вы должны включать их явно.
- Проверяйте оригинальные WebSocket-запросы и продолжайте контролировать, что они поступают из одного и того же источника, для чего

проверяйте оригинальный заголовок и передавайте в обе стороны токен (как в случае межсайтовой подделки запросов, CSRF), чтобы избежать межсайтового перехвата веб-сокета¹²⁸.

- Не разрешайте туннелирование через ваш веб-сокет к другим службам, например к базе данных. В таком случае если ваш веб-сокет окажется уязвим к межсайтовому скриптингу (XSS) или другим похожим атакам, то все туннели потенциально будут скомпрометированы. Обращения к службам должны производиться в обход вашего веб-сокета (например, по HTTPS)¹²⁹.
- По аналогии с ограничением темпа ограничивайте размер полезной нагрузки, чтобы избежать потенциальных атак разных типов. Мы должны приблизительно знать максимальный размер рабочей нагрузки и сбрасывать подключение, когда она превышена¹³⁰.
- Определитесь с форматом (протоколом) своих сообщений, чтобы можно было проверять данные, полученные от сервера или клиента. Если вы остановитесь, скажем, на JSON, то будете точно знать, что данные всегда должны быть в этом формате. В противном случае имеет место атака, и ее можно безопасно блокировать или игнорировать¹³⁰.

Рекомендации по бессерверным вычислениям

Бессерверные вычисления – это модель выполнения облачных вычислений, в которой поставщик облачных услуг выделяет машинные ресурсы по запросу и берет на себя заботу о серверах от имени своих клиентов. Он предоставляет серверные службы на основе фактического использования, что позволяет разработчикам не думать об инфраструктуре или серверах, на которых размещено приложение. Когда бессерверное приложение вызывается, оно поднимает контейнер, запускает приложение, дорабатывает до конца, а затем самоуничтожается, возвращая все ресурсы в облако.

Рекомендации

- * Как и для любого другого приложения, предоставляйте бессерверным приложениям только те права, которые нужны им для выполнения работы (принцип наименьших привилегий), и внимательно следите за тем, чтобы управление идентификацией и доступом было реализовано осмотрительно и тщательно протестировано¹³¹.
- Для создания бессерверных приложений выбирайте современную и актуальную среду или язык.
- Размещайте его за шлюзом API и конфигурируйте шлюз, так чтобы он выполнял аутентификацию, авторизацию, ограничение темпа и квотирование ресурсов – как минимум. В более современных шлюзах можно

также включать контроль входных данных. Да, вы можете использовать бессерверные приложения с сочетанием со шлюзами API³⁷.

- Обеспечивайте изолированные параметры функций (выполняйте аутентификацию, авторизацию и проверку данных, входящих в бессерверное приложение и получаемых от него; не пренебрегайте надлежащей гигиеной безопасности только потому, что находитесь за брандмауэром)³⁷.
- Не делайте слишком много в одном бессерверном приложении. Такие приложения не должны делать 27 вещей и работать почти непрерывно – и не только потому, что это было бы чрезвычайно дорого по сравнению с традиционным хостингом серверов. Они должны сделать что-то одно, а потом исчезнуть; в этом и состоит назначение бессерверного приложения. Если вам нужно делать много разного, то либо заведите много бессерверных приложений, либо обычное веб-приложение или API³⁷.
- * Применяйте управление доступом на уровне функций (это относится также к API), проверяя права доступа для каждого запроса. Это может показаться утомительным, но злоумышленники ищут ошибки на этом уровне¹³².
- * Задавайте для функций тайм-ауты. Каждой функции должно быть отведено максимальное время работы (как у ответа может быть максимальный размер). Задавайте тайм-аут, чтобы снимать транзакции, работающие дольше установленного времени, поскольку это может указывать на то, что имеет место атака. Отправьте уведомление¹³³.
- Применяйте сетевую архитектуру безопасности к способу, которым вы развертываете бессерверные приложения. По умолчанию лямбда-функции AWS работают в изолированном виртуальном частном облаке (Virtual Private Cloud – VPC), которым владеет служба Lambda, но если используется другое облако, то поместите их в список управления доступом (ACL). Для них нужно организовать мониторинг, протоколирование и уведомление; должны существовать защищающие их брандмауэры (подсети); вывод должен быть ограничен, если в нем нет необходимости, и т. д. – все так же, как для всех прочих приложений¹³⁴.

Рекомендации по безопасности IoT

IoT означает *Internet of Things* (интернет вещей), т. е. физические устройства, подключенные к сети, как, например, умные холодильники, кофеварки или часы. Традиционно такие устройства не могли похвастаться выдающейся конфиденциальностью и безопасностью. Иногда они управляют очень личными и важными системами, например замками в вашем доме или системой видеонаблюдения за младенцем, а это значит, что безопасность имеет точно такой же высокий приоритет, как и в системах любого другого типа.

Вспомним шутку:

- Буква «б» в выражении «интернет вещей» означает «безопасность».
- Но в этом выражении нет буквы «б»...
- Вот именно!

В этом разделе мы поговорим о том, как это исправить!

Рекомендации для тех, кто создает IoT-устройства

- Обеспечьте физическую защиту IoT-устройств. Если человек не может добраться до них или залезть в них, то битва уже наполовину выиграна. Это может быть дополнительная дверь, запирающая ваш умный аквариум, или другие устройства физической безопасности, которые могут пригодиться вашим пользователям.
- Защитите сеть, к которой подключены устройства. Часто IoT-устройства подключены к домашним сетям, которые оставлены без всякой защиты, с паролями по умолчанию (или вовсе без паролей), не обновляются и ужасно сконфигурированы¹³⁵. Позаботьтесь о том, чтобы обезопасить устройство в домашней сети пользователя было настолько просто, насколько возможно.
- Заранее разработайте план регулярного обновления прошивки и программного обеспечения устройств. Если вы создаете бытовые устройства, то постарайтесь организовать автоматическое обновление (если возможно). Ожидайте от пользователей худшего (извините, ребята) и старайтесь «навязывать» им безопасность при каждом удобном случае. Пример – задание безопасных параметров по умолчанию¹³⁵.
- Позаботьтесь о безопасности мобильного приложения (или того, к чему подключено IoT-устройство). Зачастую IoT-устройства поставляются без мобильных приложений¹³⁵.
- Развертывайте свои IoT-устройства в отдельной сети или сегменте сети, если есть такая возможность. И советуйте пользователям поступать так же. Представьте, что вы окружили свой замок рвом, чтобы не выпускать зло из сети наружу и не впускать его внутрь¹³⁵.
- Составьте план отслеживания и мониторинга IoT-устройств в своих рабочих сетях. Не позволяйте *никому ничего* подключать в вашей драгоценной сети. Научите пользователей делать то же самое¹³⁶.
- Предлагайте конечным пользователям многофакторную аутентификацию, если это возможно. Если она предлагается вам, как конечному пользователю, соглашайтесь¹³⁶.
- По возможности устанавливайте защиту конечных точек на IoT-устройстве¹³⁷.
- Используйте IoT-шлюз, аналогичный шлюзу API, чтобы добавить дополнительный уровень безопасности. Включайте ограничение темпа, квотирование ресурсов и дросселирование. Добавьте средства, позволяющие сделать это вашим пользователям, а в идеале создайте еще и мастер, который максимально упростит эту задачу.

- Если вы являетесь конечным пользователем IoT-устройства, а не его конструктором, то максимально ограничьте к нему доступ. Включите все возможные средства безопасности. Еще до покупки, активации и подключения к своей сети прочитайте заявление о конфиденциальности и решите, устраивает ли оно вас и вашу организацию. Следуйте тем же рекомендациям, которым следовали бы при подключении к офисной сети любого другого устройства.

Если вы изготовитель IoT-устройства, например тостера, то хорошенько подумайте, а действительно ли надо превращать его в IoT-устройство. При этом добавится что-то ценное? Если да, то стоит ли оно дополнительного риска? Какие данные вы намереваетесь собирать? Вам придется защищать данные в процессе передачи и хранения. У вас есть план, как это сделать? Превращая обычное устройство в IoT-устройство, вы существенно расширяете функциональность изделия и должны быть уверены, что она стоит дополнительных денег. Не срезайте углы и не думайте, что интернет вещей не нуждается в таких же строгих мерах предосторожности, как любой другой цифровой продукт; ваши клиенты рассчитывают на вас.

Мы дошли до конца главы, можете похлопать себя по плечу за то, что так углубились в вопросы безопасности, связанные с конкретными технологиями! В этой главе мы узнали о том, как обеспечить безопасность API, мобильных приложений, веб-сокетов, бессерверных приложений и IoT-устройств, – немало информации. Помните, что вы, будучи разработчиками, обладаете могуществом создавать удивительные приложения, а с большой властью приходит большая ответственность. Попробуйте ответить на следующие вопросы, чтобы проверить свою память и усвоение материала этой главы.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Назовите самую главную угрозу API в интернете.
2. Можно ли использовать шлюзы API не только для API?
3. Должны ли мы применять обычные приемы безопасного кодирования к IoT-устройствам?
4. Мы должны включать исходную сгенерированную системой информацию об ошибке в свои сообщения, которые показываем пользователям. Это так или не так?
5. Нужно ли тестировать API так же придирчиво, как веб-приложения?
6. Почему необходимо выводить из эксплуатации старые, уже не используемые версии API?
7. Почему следует обфусцировать или минимизировать код мобильных приложений? Почему мы не делаем этого для API и веб-приложений?

8. Назовите одно различие защиты конфиденциальной информации на телефоне и в обычном веб-приложении.
9. Почему мы запрашиваем только нужные нам разрешения для мобильного приложения (или приложения любого другого вида)?
10. Почему в Android желательно вручную перезаписать память, несмотря на то что данные уже стерты?
11. Почему мы *не* хотим, чтобы данные мобильного приложения были частью резервной копии телефона?
12. Является ли IOS безопасной относительно памяти?
13. Протокол веб-сокетов односторонний или двусторонний?
14. В веб-сокетах используется протокол SSL (Secure Sockets Layer)?
15. Почему следует избегать туннелирования через веб-сокеты?
16. Почему следует задавать тайм-аут для бессерверного приложения?
17. Почему следует применять управление доступом на уровне функций в бессерверных приложениях или API?
18. Зачем устанавливать средства обеспечения безопасности конечных точек на IoT-устройствах?
19. Зачем нужно сегментировать сеть или ставить брандмауэры вокруг IoT-устройств в нашей сети?

Популярные языки программирования

В этой главе мы углубимся в безопасность популярных языков программирования, дадим рекомендации и расскажем, каких подводных камней избегать. Языки программирования сами по себе не являются безопасными или небезопасными по умолчанию, но способы, которыми мы их используем, могут порождать уязвимости и другие проблемы безопасности. Это ошибки или дефекты реализации (в отличие от просчетов проектирования). Кроме того, в языках программирования есть такие части, как, например, определенные функции, отсутствие гарантий безопасного доступа к памяти, сериализация и т. д., которые несут в себе высокие риски, поэтому их следует избегать или использовать с сугубой осторожностью. Мы поговорим о важных частях следующих языков: JavaScript, HTML/CSS, Python, SQL, Node.js (не язык, а среда исполнения JavaScript), Java, TypeScript, C#, PHP и C/C++.

К каждому из этих языков программирования применяйте все то, что уже успели узнать из этой книги, а также специфические рекомендации для конкретного языка.

JavaScript

JavaScript – самый популярный в мире язык программирования^{*}; разработчики *обожают* JavaScript¹³⁸. Это мощный язык, позволяющий создавать интерактивный и динамический контент на веб-сайтах¹³⁹. Часто его используют в сочетании с HTML и CSS, чтобы сделать работу пользователя приятнее и удобнее, а заодно расширить функциональность веб-приложений. Язык TypeScript, который мы рассмотрим далее в этой главе, транпилируется (это форма компиляции) на JavaScript¹⁴⁰.

JavaScript может реагировать на действия пользователя в режиме реального времени, в режиме онлайн, что *изменило правила игры*, когда он появился

* Согласно рейтингу TIOBE за 2024 год, JavaScript занимает шестое место по популярности – после Python, C++, C, Java и C#. Есть и другие рейтинги, но ни в одном JavaScript не находится на первом месте. – *Прим. перев.*

в 1995 году¹⁴¹. Он превратил статические страницы в динамические приложения. Хотя JavaScript может работать как на стороне клиента, так и на стороне сервера, чаще он применяется для организации фронтального интерфейса.

На стороне клиента JavaScript выполняется непосредственно в браузере пользователя и позволяет динамически обновлять страницу, не перезагружая ее целиком (тем самым экономится трафик). Он также дает доступ в объектной модели документа (DOM), которая представляет структуру и содержание веб-страницы и позволяет JavaScript взаимодействовать с ней. На стороне сервера JavaScript выполняется с помощью таких сред, как Node.js, что позволяет создавать масштабируемые и высокопроизводительные приложения¹³⁹.

JavaScript обладает собственной экосистемой библиотек и каркасов (React, Vue, Angular и т. д.), которые упрощают решение типичных задач и позволяют разрабатывать сложные приложения более эффективно. Некоторые из них предоставляют фантастические средства безопасности!

Рекомендации по использованию JavaScript

- Используйте каркас JavaScript, который автоматически кодирует весь вывод, снимая с вас эту обязанность. Примерами могут служить React, Angular и Vue.js. Предварительно проверьте, имеются ли в каркасе опасные функции, и старайтесь избегать их. Например, в React такой является `dangerouslySetInnerHTML`, в Angular – семейство функций `bypassSecurityTrustAs*`, а во Vue.js – директива `v-html`¹⁴⁴.
- Проверяйте входные данные, кодируйте вывод и используйте заголовок политики безопасности содержимого (Content Security Policy – CSP), чтобы избежать межсайтового скриптинга (XSS).
- Избегайте встраиваемых скриптов. Каким бы соблазнительным ни было включение JavaScript непосредственно в HTML, оно открывает возможность для XSS. Лучше хранить JavaScript в отдельных внешних файлах, создавая тем самым дополнительный уровень безопасности¹⁴². Можно определять скрипты в заголовке CSP. Это похоже на встраивание SQL-кода, когда мы включаем в него полученные от пользователя данные и передаем для исполнения непосредственно базе данных. Но вместо этого используйте заковыченные переменные, например `src="..."`¹⁴³.
- Всегда рассматривайте данные, полученные от пользователя, как нечто, что можно отобразить, но нельзя использовать для организации логики в приложении (не используйте их для принятия бизнес-решений). Если бы все мы так всегда и поступали, то XSS исчез бы. Старайтесь применять это правило или рекомендацию при любой возможности, а когда это невозможно (по причинам, диктуемым бизнесом, а не вашими личными вкусами), действуйте предельно осторожно. Помните: пользовательскими данными является все, что пользователь *может* изменить, а не только информация в текстовых полях или других элементах ввода.

- Используйте `textContent` или `createElement` вместо `innerHTML` и `outerHTML`, чтобы предотвратить XSS¹⁴³.
- Используйте `encodeURIComponent` и `decodeURIComponent` вместо `escape()` и `unescape()`¹⁴³.
- Используйте `createElement` и `appendChild` вместо `document.write()`¹⁴³.
- Используйте платформенные альтернативы, например `he` или `lodash`, вместо `unescapeHTML()`¹⁴³.
- Используйте `encodeURIComponent()` и `decodeURIComponent()` вместо `encodeURI()` и `decodeURI()` для выделения частей URL и их проверки¹⁴³.
- Кодировать URL-адрес, перед тем как передавать его методом CSS `url()`¹⁴³.
- Находясь в контексте выполнения (DOM), выполняйте сначала экранирование URL, а *затем* экранирование JavaScript. Да, это нужно делать дважды, если вы собираетесь помещать полученные от пользователя данные в атрибут URL¹⁴³.
- Находясь в контексте выполнения (между тегами `<script>` и `</script>`), задавайте содержимое строки или текста путем присваивания свойству `textContent`, чтобы лишить его всех суперсил (гарантировать, что оно не будет интерпретировано). Например: `yourelement.textContent = TextFromTheUser;`¹⁴³.
- Указывайте, что текст – это текст, когда помещаете что-то полученное от пользователя в элемент данных для использования в JavaScript или CSS. Например, вместо `innerHTML` используйте `innerText` или `textContent`. Всегда недвусмысленно указывайте, что это текст; не позволяйте DOM решать за вас, потому что иногда она ошибается¹⁴³.
- С помощью метода `element.setAttribute` задавайте только безопасные статические атрибуты (`align`, `alink`, `alt`, `bgcolor`, `border`, `cellpadding`, `cellspacing`, `class`, `color`, `cols`, `colspan`, `coords`, `dir`, `face`, `height`, `hspace`, `ismap`, `lang`, `marginheight`, `marginwidth`, `multiple`, `noreferrer`, `noresize`, `noshade`, `nowrap`, `ref`, `rel`, `rev`, `rows`, `rowspan`, `scrolling`, `shape`, `span`, `summary`, `tabindex`, `title`, `usemap`, `valign`, `value`, `vlink`, `vspace`, `width`). Не используйте небезопасные динамические атрибуты (например, `onclick` или `onblur`)¹⁴³.
- Если вы используете встраиваемый JavaScript (чего, как было сказано выше, делать не рекомендуется) и JSON, то проверяйте, что заголовок `Content-Type` содержит `application/json`, а не `text/html`, чтобы предотвратить XSS¹⁴³.
- Переменные (пользовательские данные) следует помещать только в свойства CSS, но не в контексты. `<style> selector { property : $var-Unsafe; } </style>`¹⁴⁴.
- JavaScript-код следует исполнять в режиме `strict`¹⁴⁵.
- Используйте модуль `hpp` внутри Express, чтобы избежать уязвимости из-за загрязнения HTTP-параметров (HPP) (когда злоумышленник по-

сылает в `req.query` или `req.body` несколько параметров с одним и тем же именем и вредоносными значениями)¹⁷³.

- Подумайте о возможности использовать инструмент Dom Purify (<https://github.com/cure53/DOMPurify>), который несколько сайтов рекомендуют в качестве средства против XSS на основе DOM¹⁴⁶.
- Загружайте внешние ресурсы (в том числе скрипты!) безопасно, применяя проверку целостности подресурсов, когда включаете их в свой код. Даже если ресурс берется из сети доставки содержимого (CDN), все равно проверяйте его. Проверка целостности подресурсов (Subresource Integrity – SRI) – встроенное в современные браузеры средство, использующее криптографический хеш для проверки целостности внешнего скрипта¹⁴⁸.
- Минимизируйте и упаковывайте свои скрипты, а также добавляйте одноразовое случайное число, чтобы дополнительно обезопасить их. Используйте для этой цели программу Webpack или что-то подобное¹⁴⁸.
- Выполняя в своих приложениях транзакции (например, покупку), передавайте токен в обе стороны *вне cookie*, чтобы избежать межсайтовой подделки запросов (cross-site request forgery – CSRF). Это может быть скрытое поле; чтобы исполнять свою работу в качестве защитной меры, оно обязательно должно быть хорошо упрятано. Идея в том, чтобы защититься от фишинговых атак, когда пользователь переходит по ссылке, ведущей на инициализацию транзакции, в предположении, что пользователь уже вошел на сайт, являющийся целью атаки (например, Aliexpress.com или Amazon.com). Даже если пользователь уже находится на сайте в момент перехода по вредоносной ссылке, эта ссылка не будет включать противодействующий CSRF токен, и значит, атака потерпит неудачу.
- Используйте линтер и инструмент статического анализа (SAST) для проверки своего JavaScript-кода на наличие потенциальных уязвимостей и их исправление.
- Подвергайте аудиту зависимости, используя менеджер пакетов, например yarn или npm.
- Вовремя обновляйте пакеты.
- Будьте предельно осторожны, когда используете менеджеры пакетов, и проверяйте имена пакетов, чтобы избежать вредоносных пакетов с очень похожими именами.
- Используйте `let` вместо `var`. Для объявлений `let` областью видимости является блок или функция.
- Используйте `const`, если переменную не предполагается изменять.
- По возможности используйте строгое равенство (`===` и `!==`), а не `==`.

Чего избегать в коде на JavaScript

- Избегайте следующих функций: `eval()`, `innerHTML`, `outerHTML`, `Function()`, `escape()`, `unescape()`, `document.write()`, `unescapeHTML()`, `decodeURI()` и `encodeURI()`¹⁴³.

- Не используйте предложение `with()`¹⁴³.
- Не используйте конструкторы функций со строковым аргументом¹⁴³.
- Не конкатенируйте данные из ненадежного источника с JavaScript-кодом для создания элементов DOM и не используйте `document.write()` с динамически сгенерированным содержимым¹⁹⁷. Вместо этого рассмотрите следующие альтернативы:
 - используйте `createElement()`, а затем присвойте значение свойству, например `node.textContent` или `node.innerHTML`;
 - используйте `document.createTextNode()` и вставьте узел в подходящее место DOM;
 - используйте `element.setAttribute()` или `element[attribute]`.
- Используйте функции `setTimeout()` и `setInterval()` с осторожностью; очень тщательно проверяйте полученные от пользователя данные, прежде чем использовать их в аргументах этих функций. А еще лучше никогда не используйте такие данные в качестве аргументов¹⁴³.
- Избегайте JavaScript-методов, которые принимают код в виде строки, например `setAttribute()`, `setTimeout`, `setInterval` и `new Function`. Вместо этого присваивайте значение атрибуту напрямую¹⁴³.
- Не передавайте ненадежные данные методам отрисовки HTML, потому что они могут интерпретироваться (возможна XSS-атака). Примерами являются `innerHTML`, `outerHTML`, `document.write`, `document.writeln` и т. д.
- Поскольку язык JavaScript основан на прототипах, необходимо избегать внедрения прототипов (да-да, еще один тип внедрения)¹⁴⁷. Этого можно добиться следующими способами: (1) заморозить объект по умолчанию, которому все остальные наследуют, `Object.freeze(Object.prototype)`; (2) явно создавать объекты, которые не пользуются механизмом прототипов, `Object.create(null)`; (3) использовать `Map` вместо `Object`; (4) вообще избегать рекурсивных функций слияния.
- Не используйте одно и то же значение `document.domain` для разделения информации между страницами; это сводит на нет защиту, предоставляемую политикой одного источника. Вместо этого используйте `window.postMessage()`¹⁴⁷. Ни в коем случае не указывайте `*` в `window.postMessage()`; обязательно проверяйте источник и содержимое сообщения, перед тем как его использовать.
- Не ограничивайтесь проверкой полученных от пользователя данных на стороне клиента, это необходимо делать также на стороне сервера. Это относится к любому языку, но фронтальный интерфейс обычно пишется на JavaScript, поэтому есть смысл упомянуть об этом в данном разделе.
- По возможности избегайте глобальных переменных, а используйте вместо них закрытые переменные, доступ к которым со стороны внешних скриптов менее вероятен¹⁴⁹.

Примечание С помощью веб-прокси, например Burp Suite или Zaproxy, любой может обойти фронтальный JavaScript-код веб-приложения и отправлять запросы непосредственно веб-серверу. Поэтому проверки, санирования и экранирования на стороне клиента никогда недостаточно с точки зрения безопасности. Окончательное решение остается за сервером.

Короче говоря, JavaScript – чрезвычайно мощный язык и часть фронтального интерфейса практически всех сайтов на планете, а это значит, что обращаться с ним нужно очень осторожно. Далее мы поговорим о HTML и CSS, определяющих внешний вид сайта.

HTML и CSS

Не торопитесь со словами «HTML – это *не* язык программирования, Таня!». Я знаю. HTML (Hypertext Markup Language) – это язык *разметки*, который говорит браузеру, как отображать текст, изображения и другие формы мультимедиа на странице¹⁵⁰. Но он все равно несет в себе риски безопасности, поэтому нужно провести более глубокий анализ.

В этом разделе мы также рассмотрим каскадные таблицы стилей (Cascading Style Sheets – CSS), которые описывают представление HTML или XML. Вы, возможно, думаете, что уж CSS-то не несет никаких рисков, но увы! Будь то авторы, раскрывающие конфиденциальную информацию (разные роли в приложении или какие поля содержат секреты), использование CSS из ненадежных источников или неудовлетворительное управление доступом, злоумышленники знают, что CSS можно использовать для разведки или, хуже того, для атаки на веб-приложения¹⁵¹. Примечание: риски, связанные с CSS, очень малы по сравнению с любым языком программирования, да и риск HTML тоже невелик.

Рекомендации по работе с HTML и CSS

- Для предотвращения XSS необходимо проверять, что полученные данные (источник) безопасны, прежде чем мы начнем их использовать (сток). Поэтому все, что получено от пользователя или потенциально могло быть им модифицировано, должно подвергаться HTML-кодированию¹⁴⁴. Проверяйте входные данные и отвергайте недопустимые. Если вы вынуждены принимать специальные, потенциально опасные символы, проверяйте их и saniруйте данные перед использованием.
- Обязательно заключайте переменные в одиночные или двойные кавычки, чтобы лишить их сверхспособностей и показать, что это текст, а не код¹⁴⁴. Закавычивание значения затрудняют изменение контекста, в котором используется переменная, а значит, предотвращают XSS. Закавычивание также значительно сокращает множество символов, которые необходимо кодировать, в результате чего приложение становится

более надежным, а реализация кодирования упрощается. Кодирование в нескольких контекстах и так уже запутаннее, чем следовало бы, так давайте упростим, что можем.

- Создавайте разные таблицы стилей для разных ролей в приложении, а затем разрешайте только этим ролям обращаться к соответствующим CSS-файлам¹⁵¹. Для каждой страницы, которая пытается загрузить CSS-файл, сначала проверяйте, что роль данного пользователя в системе позволяет ему видеть данный CSS-файл (и страницу, запись данных и т. д.). Мы всегда должны сначала проверить наличие разрешения, а только потом разрешать.
- В качестве альтернативы предыдущему пункту можете удалить всю идентифицирующую информацию из таблиц стилей, так что у злоумышленника не будет возможности почерпнуть какую-нибудь информацию о ролях или еще о чем-то внутреннем, читая CSS-файлы. К сожалению, это мешает также и программистам, поэтому такой вариант менее привлекателен¹⁵¹.
- Если вы знаете, что ваш CSS-файл содержит конфиденциальную информацию, то подумайте насчет обфускации CSS¹⁵², чтобы скрыть ее от любопытных глаз.
- Думайте, прежде чем поместить что-то в скрытое поле; это все равно что повесить указатель для злоумышленников: «Ищи здесь!»¹⁵³
- Используйте все применимые заголовки безопасности (например, CSP). В конце концов, это не что иное, как HTML.
- Очень внимательно относитесь к следующим элементам и атрибутам, которые могут стать мишенью атаки с внедрением кода: обработчики событий, autofocus/on focus, form и formaction¹⁵⁴.

Предупреждение В отличие от кода, размещенного на сервере, JavaScript-код может увидеть любой, кто имеет браузер и выполнит команду «Просмотреть исходный код страницы». Он увидит все ваши комментарии, логику и все предпринятые вами меры безопасности! Будьте осторожны!

Чего избегать в HTML и CSS

- Не используйте CSS, полученный из ненадежного источника. В качестве альтернативы можете сделать копию, внимательно изучить ее и использовать нужные вам части. Или же ваша организация может принять риск использования стороннего CSS, если вы думаете, что можете доверять источнику¹⁵¹.
- Не позволяйте пользователям модифицировать ваш CSS-файл, потому что кто-нибудь может попробовать атаковать его (внедрить собственный код, как правило, JavaScript)¹⁵².
- Не помещайте полученные от пользователя данные в CSS или HTML. И вообще в любой код, который пишете. Пользователи должны вводить только данные, ни в коем случае не часть вашего кода.

- Не помещайте ничего важного или конфиденциального в комментарии, поскольку злоумышленник легко увидит их, выбрав из меню команду «Просмотреть исходный код страницы»¹⁵³.

Конкретно о HTML5

HTML 5 – самая последняя и теоретически окончательная версия¹⁵⁵. В ней появились новые функции, облегчающие создание веб-приложения, в том числе геолокация (определение местоположения пользователя), перетаскивание (элементов на веб-страницы), локальное хранилище (для хранения больших объемов данных в браузере), кеши приложений (поддерживают офлайновый просмотр), веб-исполнители, события, отправляемые сервером (Server-Sent Events – SSR), и многое другое. Легко понять, что новые возможности сопровождаются новыми рисками.

Отдельное спасибо ребятам из проекта OWASP HTML5 Security Cheat Sheet за помощь в написании этого раздела!

Вопросы безопасности

- Веб-хранилище и онлайн-хранилище (локальный кеш): не храните здесь конфиденциальные данные; используйте вместо этого безопасный cookie¹⁵⁶.
- При использовании механизма веб-сообщений (не веб-сокетов, которые рассматривались в главе 5)¹⁵⁶:
 - явно указывайте ожидаемый источник во втором аргументе `postMessage`; не используйте `*`;
 - при получении сообщений проверяйте источник и все данные (атрибут `data`);
 - не используйте данные, пересылаемые туда и обратно, для принятия бизнес-решений; пользователь легко может изменить их (P.S. И в DOM тоже их не храните.);
 - как уже было отмечено, избегайте функций типа `innerHTML` для помещения данных на страницу, используйте лучше более безопасный метод `textContent`;
 - если вы *вынуждены* использовать полученный от пользователя скрипт или иной внешний контент, отнеситесь к нему с величайшей осторожностью, внимательно изучите его, а затем тщательно протестируйте.
- Осторожнее с совместным использованием ресурсов между разными источниками (CORS)¹⁵⁶:
 - если вы собираетесь использовать метод `XMLHttpRequest.open`, проверьте URL по списку разрешенных. Это опасная территория;
 - для URL-адресов, разрешенных для совместного использования заголовком `Access-Control-Allow-Origin`, убедитесь в отсутствии конфиденциальных данных. Не используйте их на всем сайте, а только

- там, где необходимо. Думайте, прежде чем что-то использовать совместно¹⁵⁶;
- если вы используете механизм предварительной загрузки, то это прекрасно, но помните, что не все будут поступать так, кто-то будет по-прежнему использовать GET и POST. Проверьте, что у вас есть доступ ко *всем* глаголам (методам) HTTP¹⁵⁶;
 - не принимайте никаких запросов по протоколу HTTP. Всегда используйте только HTTPS¹⁵⁶;
 - проверяйте не только заголовок `origin`, принимая решения об управлении доступом. Проверяйте также сеансовый `cookie` и токен противодействия CSRF, если он применяется¹⁵⁶.
- Для событий, отправляемых сервером, проверяйте атрибут `event.origin`, чтобы быть уверенным, что событие пришло оттуда, откуда вы думаете¹⁵⁶.
- При работе с локальным хранилищем (оно же онлайнное или веб-хранилище) имейте в виду следующее¹⁵⁶:
- любой пользователь, имеющий локальные привилегии на машине, может получить доступ к этому хранилищу, так что ничего конфиденциального там быть не должно;
 - используйте `SessionStorage`, если данные нужны только для одного сеанса, и `LocalStorage`, если они должны храниться дольше. По возможности делайте так, чтобы данные по умолчанию уничтожались (`SessionStorage`);
 - не доверяйте никаким данным, сохраненным в любом режиме, поскольку они могли быть скомпрометированы, если приложение уязвимо для XSS (хранилище доступно для JavaScript-кода и его нельзя заблокировать, как `cookie`). Если, конечно, вы не шифруете данные перед сохранением и не расшифровываете перед использованием или не применяете еще какие-то меры для обеспечения их секретности и целостности. Если вы не можете использовать шифрование или еще какую-то систему, гарантирующую целостность, то должны проверять данные и экранировать либо санировать их перед использованием;
 - не храните идентификатор сеанса ни в одном из хранилищ, для этого существуют `cookie`;
 - не размещайте несколько приложений в одном источнике, так как все они будут использовать один и тот же объект `localStorage`. Используйте лучше разные поддомены.
- Избегайте уязвимости «tabnapping»* (которая больше не считается критической, поскольку современные браузеры по умолчанию устраняют ее), для чего используйте `rel="noreferrer"` или `rel="noopener"` для HTML-ссылок и добавляйте те же атрибуты в вызов `window.open`. Для пущей

* В переводе «забыл про вкладку». – Прим. перев.

уверенности можете добавить во все HTTP-ответы заголовок `Referer-Policy: no-referrer`. В настоящее время это делается в основном для обратной совместимости¹⁵⁶.

- Если вы вынуждены загружать ненадежный контент, то используйте атрибут `sandbox` тега `iFrame` (загружайте в `iFrame`). Существуют параметры, которые дополнительно укрепляют безопасность. Но лучше вообще не загружать ненадежный контент¹⁵⁶.

Честно говоря, начиная писать эту часть главы, я не предвидела никаких трудностей; я понятия не имела, как много рисков веб-приложению таят в себе HTML и CSS! Надеюсь, что материал был вам полезен и не напугал вас до полусмерти!

Python

Написание хорошего кода на Python – это не только о функциональности и производительности, это еще и о защите приложения и пользователей от потенциальных уязвимостей и атак. А учитывая, насколько Python популярен, разговор об обеспечении безопасности написанных на нем программ приобретает особую важность.

НЕОЖИДАННОЕ ЗАДАНИЕ ДЛЯ ТАНИ

Во время собеседования при устройстве на работу в компанию Semgrep мне предложили написать код на Python. Это было неожиданно: я не думала, что иду на собеседование, предполагалось, что это будет разговор о «секретном соусе» их продукта. Я вообще никогда не программировала на Python. К тому же в экзаменационном задании была рекурсия и матричные операции! Понятия не имею, как я его сдала, но, очевидно, как-то сдала, а остальное уже история!

Совет Если вы найдете ошибку безопасности в PyPi, CPython или Pip, сообщите о ней по адресу security@python.org¹⁵⁷. Но только в том случае, когда вы *уверены*, что обнаружили уязвимость; там работают занятые люди. P.S. Сообщать об ошибках в любом каркасе или языке не только ваше право, но и обязанность!

Что надо делать

- Использовать версию Python 3, а не версию 2, которая больше не поддерживается¹⁶¹.
- Регулярно обновлять среду Python. Это касается всех каркасов и самого языка; старайтесь не отставать.
- Использовать виртуальную среду и Pip Freeze, чтобы в состав поставки входили именно те модули, которые вы ожидаете¹⁵⁹. Вы же не хотите, чтобы локальные модули отличались от используемых в проекте. Разумеется, по возможности нужно проверять, что используемые мо-

дули не содержат известных уязвимостей, что они относительно актуальны и что если в них *имеется* уязвимость, то она недостижима из вашего кода.

- Всегда импортировать пакеты по *абсолютным* или *явным относительным* путям, чтобы получить именно то, что ожидается¹⁵⁹.
- Принимая от пользователя строку, которую вы собираетесь использовать в коде, пользуйтесь для манипулирования строками классом `Template` из модуля `string` и никакими другими функциями¹⁵⁹. Не используйте f-строки и функцию `str.format()` для обработки полученных от пользователя данных (или чего-то, что пользователь мог изменить).
- Помните, что опасности десериализации свойственны не только Java; Python также может пасть жертвой злоумышленников, отправляющих опасные сериализованные объекты.
- Используйте инструменты типа `pipenv check` и `safety` для проверки зависимостей на наличие потенциальных уязвимостей и управления зависимостями¹⁶⁰.
- Сканируйте свой код на уязвимости программой `Bandit` (свободной и с открытым исходным кодом). Она заточена под Python¹⁶².
- Следите за актуальностью своей версии пакета «certifi». Он используется библиотекой `requests` для проверки надежности сертификатов¹⁶¹.
- Храните секреты либо в переменных окружения (на эту тему в сети есть много пособий), либо в инструменте управления секретами¹⁵⁹.
- Подумайте о применении к своему коду линтера (в дополнение к статическому инструменту проверки безопасности), например `per8`, `pylint` или `flake8`. Это повысит качество и удобочитаемость кода, а стало быть, его безопасность и удобство сопровождения¹⁶³.
- Используйте последнюю версию Python, а не предзагруженную по умолчанию версию POSIX¹⁶³, так как последние часто сильно отстают.

Совет Вы, наверное, обратили внимание, что я предложила использовать `Pip Freeze`, а затем сказала, что фиксировать версии библиотек не нужно. Как можно *одновременно* заморозить библиотеки и продолжать их обновлять? Когда у вас появляется шанс про-извести обновление кода, вы стараетесь обновить столько библиотек, сколько можно, чтобы избежать накопления технического долга. Если библиотеки зафиксированы, то они не обновятся. Но при переходе из одной среды в другую (разработка -> контроль качества -> подготовка к развертыванию) версии кода не должны изменяться, потому что иначе будет тестироваться неизвестно что. Когда будете готовы выйти за рамки среды разработки, замораживайте, а до того обновляйте, обновляйте и обновляйте (особенно библиотеки!).

Чего делать *не* надо

- В Python 2 используйте не `input()`, а `raw_input()`. В Python 3 против использования `input()` нет возражений¹⁵⁸. P.S. Вообще больше не используйте Python 2.

- При использовании PyYAML для загрузки YAML загрузчик, применяемый по умолчанию, `yaml.load()`, не безопасен. Не используйте его¹⁵⁸.
- Не используйте подпроцесс с `shell=True` с последующей загрузкой в него Python-кода, полученного от пользователя. Это очень дурная идея.
- При импорте пакетов никогда не используйте «неявный» путь, только «явный» (в Python 3 неявные пути запрещены), чтобы не загрузить случайно не тот пакет¹⁵⁹.
- При использовании библиотек не следует фиксировать версии в файле `requirements.txt`, потому что так можно установить устаревшую и уязвимую версию модуля. Не фиксируйте версии, продолжайте обновлять¹⁵⁹.
- Не используйте следующие функции, если на вход подается текст, полученный от пользователя, или такой, который пользователь мог изменить: `eval()`, `exec()`, `compile()` и все прочие функции, которые выполняют системные действия или код в зависимости от заданных вами параметров¹⁶⁰.

Переходим к SQL!

SQL

Соберитесь вокруг (воображаемого) костра, чтобы послушать щекочущие нервы и ужасающие рассказы о SQL! Конечно, мы поговорим в этом разделе о печально известных атаках с внедрением SQL-кода (и о том, как их избежать), но затронем и другие вопросы, о которых вы, возможно, не слыхали. И в очередной раз я буду заклинать вас укреплять и регулярно применять исправления к своим SQL-серверам, если вы не хотите подвергнуть данные опасности. А ведь вы, разумеется, желаете, чтобы данные были максимально защищены, так что приступим к учебе!

Рекомендации по безопасности для языка SQL

- Используйте систему ORM (объектно-реляционное отображение), надстроенную поверх SQL. Среди прочего она поможет избежать внедрения SQL-кода!
- Шифруйте данные при хранении и в процессе передачи в базу данных и из нее¹⁶⁴.
- Используйте хранимые процедуры, а не прямые запросы к данным, когда хотите предоставить доступ к базе из приложения или из другой системы.

Рекомендации по укреплению базы данных и сервера SQL

- Укрепляйте SQL-сервер и операционную систему, в которой он работает¹⁶⁵. Просканируйте его инструментом оценки базы данных и (или) сканером уязвимостей (VA) и выполните все рекомендации – исправьте

все найденные проблемы. Затем получите руководство по укреплению от поставщика и выполните все описанные в нем рекомендации тоже¹⁶⁴.

- Программное обеспечение сервера базы данных должно быть актуальным. Регулярно сканируйте и применяйте обновления.
- Обращайтесь к базе данных из приложения только от имени учетной записи службы, ни в коем случае не человека¹⁶⁴.
- Не давайте административный доступ к БД никому, кроме администраторов базы данных. Разработчикам обычно не нужно администрировать базу данных¹⁶⁴.
- Не используйте для своего приложения учетные записи с привилегиями владельца базы данных (DBO). Крайне маловероятно, что вам *действительно нужны* привилегии DBO, гораздо вероятнее, что вы *предпочли бы* иметь их. Но, скорее всего, вам нужен только доступ для чтения или операций CRUD. Используйте минимальный объем привилегий в учетных записях приложений, чтобы уменьшить ущерб в случае инцидента¹⁶⁴.
- Классифицируйте (конфиденциальны или нет и если да, то в какой степени) и помечайте свои данные (в буквальном смысле – оставляйте где-то сведения о грифе, даже если данные открыты и не конфиденциальны)¹⁶⁴. Это действие покажет всем, как следует обращаться с данными. Делать это можно на уровне базы данных, таблицы или даже поля.
- Физически защищайте сервер. Никто, кроме сотрудников группы эксплуатации или поставщика облачных услуг, не должен иметь физический доступ к серверам.
- Включайте протоколирование, чтобы было что расследовать в случае инцидента¹⁶⁴. Имеются в виду журналы доступа (кто обращался к базе данных и когда), журналы запросов (какие запросы выполнялись) и, если есть возможность, метажурналы от поставщика облачных услуг (например, они могут сказать, когда в БД была добавлена новая политика безопасности).
- Позаботьтесь о наличии хорошей стратегии резервного копирования журналов и о том, чтобы к ним имели доступ только люди, которым это необходимо. Ваши журналы содержат в высшей степени конфиденциальную информацию.
- Включите мониторинг, чтобы вы знали, когда база данных выходит из строя¹⁶⁴. Крах базы данных может быть проблемой безопасности, хотя это и необязательно.
- Включайте многофакторную аутентификацию (МФА) для всех пользователей базы данных.
- Установите на сервер средства мониторинга целостности файлов и управления приложениями, чтобы защититься от вредоносного программного обеспечения.

- Для особо конфиденциальных приложений и рабочих нагрузок в облаке подумайте о создании виртуальных машин «конфиденциальных вычислений», специальном классе ВМ с дополнительными мерами защиты от атак по побочным каналам и др.¹⁶⁵.
- Разработайте всеобъемлющий и хорошо продуманный план резервного копирования и потренируйте откаты. В идеале резервные копии должны быть территориально распределены (а не находиться в одном здании, городе или даже стране). Должно быть предусмотрено отключение в случае атаки программ-вымогателей (вы не поверите, сколько людей оставляют свой диск с резервными копиями подключенным к машине, так что во время атаки он шифруется вместе со всем остальным в сети).
- Не называйте учетную запись локального администратора «admin» или «administrator». Выберите что-то менее очевидное¹⁶⁵.
- Используйте для доступа к базе данных сложные, длинные, уникальные пароли, в идеале сгенерированные менеджером паролей¹⁶⁵.
- Используйте для администрирования и резервного копирования разные учетные записи¹⁶⁵.
- Не устанавливайте ничего постороннего на сервер баз данных. Он должен выполнять только одну, чрезвычайно важную функцию¹⁶⁵.
- Выполняйте для сервера рекомендации по сетевой безопасности, например размещайте его за брандмауэром и закрывайте ненужные порты¹⁶⁵.
- Настройте систему так, чтобы после увольнения работника вы знали о том, что нужно удалить все его учетные записи. Это относится к любой системе, от электронной почты до отдела кадров¹⁶⁶.
- Проводите аудит доступа к серверу баз данных. Подключаются ли к нему люди, которые не должны? Почему? Разберитесь¹⁶⁶.
- Отключите службу просмотра SQL-серверов, поскольку она выдает важную информацию неаутентифицированным пользователям¹⁶⁵.

Вопросы безопасности для SQL

- Внедрение SQL-кода – самая известная уязвимость, с которой сталкивается SQL. Она имеет место, когда злоумышленник обманом заставляет приложение дописать свой код в конец вполне добропорядочного кода, превратив его тем самым во вредоносный, а затем отправляет его в виде команды SQL-серверу для выполнения (вместо использования хранимой процедуры). Обычно атакующий код вводится в поле, которое *должно* обрабатываться как данные, например в поле поискового запроса.
 - Чтобы избежать внедрения SQL-кода, используйте хранимые процедуры (вместо того чтобы отправлять команды серверу для выполнения напрямую) и проверяйте входные данные. Кроме того, тестируйте приложение на наличие уязвимостей с помощью теста

на проникновение или чего-то вроде SQLMap, используйте инструмент SAST для проверки кода и применяйте принцип наименьших привилегий к учетным записям базы данных, чтобы уменьшить ущерб, если, несмотря на все усилия, вы все-таки станете жертвой атаки. Последствия внедрения SQL-кода *ужасающие*, поэтому абсолютно необходимо предпринимать *все* эти усилия, чтобы избежать их¹⁶⁴.

- Не помещайте производственные данные в непроектованные среды. Это огромный риск безопасности и, безусловно, нереконструируемая практика (хотя и очень широко распространенная). Существует много продуктов и плагинов, умеющих создавать тестовые данные, например NuGet-пакет Bogus для .NET/C#. Если вам никак не удастся добыть пригодные для работы тестовые данные (данные, *не являющиеся* копией производственных), сделайте копию производственных данных и замаскируйте их (поменяйте местами поля или целые записи, чтобы было невозможно определить, какие данные к кому относятся). Для этого имеются специальные инструменты; не пытайтесь сделать это самостоятельно, потому что данные легко будет дерандомизировать. Благодаря этому приему конфиденциальные данные заказчиков останутся конфиденциальными¹⁶⁴.
- Следите за тем, чтобы учетные записи, используемые для доступа к базе данных в непроектованных средах, отличались от производственных. Одна и та же учетная запись службы не должна использоваться для редактирования тестовых и производственных данных.
- Не используйте один и тот же секрет (особенно пароль в строке подключения) для подключения к производственной и еще какой-то среде.
- Любой риск, относящийся к серверу вообще, относится и к SQL-серверам (вредоносные программы, кража, физические атаки, скачки напряжения, перегрев и т. д.).
- Проанализируйте весь SQL-код, в котором встречаются команды EXECUTE, EXEC, sp_executesql и любые хранимые процедуры из расширенных каталогов: xp_*¹⁶⁵.
- Не показывайте ошибки сервера пользователям, используйте только свои собственные сообщения (которые не выдают никакой системной информации, а особенно о структуре исполняемых команд)¹⁶⁵.
- При создании белых списков для проверки входных данных не включайте следующие символы и комбинации символов в качестве «допустимых»: ;, ", --, /*, */¹⁶⁵.

В этой области можно еще много чего сделать на сетевом и операционном уровне, но это не книга по администрированию сетей, и я не хотела бы слишком углубляться в эту тему, поскольку разработчики ПО обычно не имеют доступа к соответствующим средствам. Но так или иначе, делайте все, что в ваших силах, для защиты данных; пользователи скажут вам спасибо.

Node.js

Добро пожаловать в мир Node.js, где мощь JavaScript встречается с серверной стороной наших приложений. Будучи разработчиками, мы любим скорость, масштабируемость и гибкость, которые предлагает Node.js. Но от этих преимуществ неотделима и ответственность за обеспечение безопасности и надежности создаваемых на основе этой технологии приложений.

Node.js – серверная среда выполнения JavaScript с открытым исходным кодом, построенная на базе движка Chrome V8 JavaScript. С ее помощью мы можем выполнять JavaScript-код *вне браузера*, а значит, строить масштабируемые и высокопроизводительные приложения, не работающие в интернете (в отличие от веб-приложений, где тоже используется JavaScript)¹⁶⁷.

Одна из ключевых особенностей Node.js – событийная архитектура и модель неблокирующего ввода-вывода. Это означает, что Node.js может обрабатывать одновременные подключения эффективно, что делает его идеальным инструментом для создания приложений реального времени, таких как чаты, стриминговые платформы и инструменты совместной работы. По общему мнению, Node.js не подходит для трудоемких вычислений, например научных расчетов и моделирования (например, симулятора полетов)¹⁶⁷.

Вокруг Node.js сложилось большое сообщество, которое разрабатывает и сопровождает его обширную экосистему модулей и пакетов, доступную с помощью менеджера пакетов Node Package Manager (NPM). Node.js часто выбирают для построения веб-серверов и API за его способность обрабатывать высокий объем трафика и выполнять асинхронные операции, а также потому что разработчикам нужно изучать только один язык (а не JavaScript для фронтального интерфейса и какой-то другой язык для серверной части)¹⁶⁷.

Так чего же мы ждем? Поехали!

Рекомендации по работе с Node.js

- Не запускайте Node.js от имени root в производственной среде. Конечно, если вы учите на каком-то семинаре, то пожалуйста. Но, разрабатывая программы, предназначенные для людей, старайтесь ничего не запускать от имени root¹⁶⁸.
- Используйте командные инструменты NPM audit или Snyk NPM для проверки и актуализации зависимостей. Оба инструмента бесплатны¹⁶⁸. Есть также множество платных инструментов, которые могут помочь в этом деле.
- Используйте какой-нибудь линтер безопасности для Node.js (в дополнение, а не вместо SAST), например: ESLint, JSLint, JSHint, quick-lint-js или StandardJS¹⁶⁹.
- Не блокируйте цикл обработки событий, так как это может быть использовано для атаки типа «отказ в обслуживании» на ваше приложение. Все потоки должны быть доступны¹⁷¹.

- Пишите обратные вызовы, так чтобы они были несложными (количество шагов предсказуемо и единообразно) и выполнялись быстро. Не забывайте тестировать.
- Для более сложных задач ограничивайте объем входных данных и отвергайте слишком длинные. При выборе ограничения ориентируйтесь на худшее время – важно, чтобы поток не блокировался надолго. Оценка может потребовать выдвижения гипотез и тестирования.
- Избегайте использования синхронных API из модулей шифрования, сжатия, файловой системы и дочерних процессов. Они занимают много времени и могут заблокировать цикл обработки событий, поэтому выполняйте их асинхронно.
- Не позволяйте npm выполнять произвольные скрипты, для чего задавайте флаг `--ignore-scripts` и (или) запрещайте глобально с помощью команды `npm config set ignore-scripts true`¹⁷².
- Применяйте `npm ci` вместо `npm install`, если используете lock-файл для фиксации версий зависимостей. Тогда при наличии рассогласования между lock-файлом и файлом `package.json` будет выдана ошибка (вместо молчаливого игнорирования lock-файла в пользу `package.json`)¹⁷².
- При использовании `npm ci` добавляйте флаг `-production`, чтобы устанавливался только минимальный набор файлов, необходимых для эксплуатации (тесты и прочие не относящиеся к эксплуатации вещи будут опущены).
- Используйте вместо обратных вызовов `async-await` или хорошо зарекомендовавшую себя библиотеку обещаний для управления асинхронными ошибками¹⁷¹.
- Используйте встроенный объект ошибки, а не создавайте свой собственный. Заниматься обработкой строкового сообщения об ошибке тоже не надо¹⁷¹.
- Используйте инструмент управления процессами для перезапуска в случае неизвестной ошибки. *Pm2* и *forever* – примеры таких инструментов для Node.js¹⁷¹.
- Используйте полезные «заточенные» под NPM инструменты типа DOM-Purify и инструменты проверки форм, такие как *express-validator* или *XSS-filters*¹⁷¹.
- Для конфиденциальных операций пользуйтесь защитой на базе взаимодействия с пользователем¹⁷¹.
- Отдавайте предпочтение пользовательским заголовкам запроса, а не стандартным¹⁷¹.
- Используйте плоские цепочки Promise¹⁷³, гарантирующие, что ошибки и результаты попадут туда, откуда был вызван асинхронный обратный вызов, даже после нескольких шагов рекурсии.

- Используйте модули контроля входных данных, например `validator` и `express-mongo-sanitize`¹⁷³.
- Используйте модуль `toobusy-js` (или что-то подобное) для мониторинга цикла обработки событий на предмет потенциальных DOS-атак или иных ситуаций, чреватых крахом¹⁷³.
- Используйте такие модули, как `Express-bouncer`, `express-brute` и `rate-limiter`, чтобы снизить темп (дресселировать) запросов входа в систему, если есть подозрение на атаку полным перебором¹⁷³.
- Удаляйте неиспользуемые маршруты API (куда пользователь может отправить запрос и API ответит, хотя программа этого не предполагает). Это особенно важно в таких каркасах, как `Sails` и `Feathers`, которые «помогают» восполнять пробелы, самостоятельно генерируя «отсутствующие» конечные точки API. Это плохо, поскольку вы не контролируете эти ответы, убейте их¹⁷³.
- Если ваши объекты не предполагают редактирования, удаления или использования в циклах `FOR`, задайте соответствующие значения свойств `writable`, `configurable` и `enumerable`¹⁷³.
- Прослушивайте события ошибок, если используете объекты `EventEmitter`; они могут появиться в любом месте цепочки событий. Первым аргументом асинхронных вызовов должен быть объект `Error`; в противном случае ошибки не будут обработаны в маршруте `express`¹⁷³.
- При использовании регулярных выражений избегайте атаки `ReDOS` (DOS-атака на регулярные выражения). Для этого с помощью модуля `vuln-regex-detector` проверяйте, нет ли чего-то опасного в строке, прежде чем применить к ней регулярное выражение¹⁷³.
- Используйте режим `strict`, о котором было сказано в разделе, посвященном JavaScript¹⁷³.

Вопросы безопасности для Node.js

- Самыми значительными атаками на экосистему NPM были атаки на цепочки поставок, когда во внешние NPM-библиотеки внедрялся вредоносный код. Поэтому еще раз подчеркиваю, как важно поддерживать зависимости в актуальном состоянии, использовать только те библиотеки, которые вам действительно нужны, брать библиотеки только из надежного источника и использовать инструменты типа `npm audit`, бесплатную командную версию `Snyk` или еще какое-то средство анализа состава программы.
- DOS-атака на запросы к HTTP-серверу¹⁷⁰. Поскольку Node спроектирована как серверная программа, запрос, который выглядит безобидно, все равно можно использовать для атаки на сервер, если он разбивается на части, которые потребляют все имеющиеся ресурсы (CWE-400). Противодействие:
 - используйте обратный прокси-сервер (или брандмауэр веб-приложений [WAF], или балансировщик нагрузки, если WAF недоступен)

- и включайте все возможные средства безопасности для защиты приложения от такого вида атак. Особое внимание обращайте на задание максимального размера ответа и всего, что связано с ограничением темпа¹⁷⁰;
- тщательно конфигурируйте тайм-ауты сервера, чтобы не было простаивающих потоков или (подозрительно) медленно поступающих запросов;
- ограничивайте количество открытых сокетов (общее и для каждого хоста), чтобы не захлебнуться под лавиной запросов¹⁷⁰.
- Ограничивайте размер запроса (чтобы избежать DOS-атаки), задавая параметр `limit` в пакете `body-parser`, так чтобы принималась лишь сравнительно небольшая полезная нагрузка. Также при использовании обратных прокси или ПО промежуточного уровня `express` задавайте ограничения на размер некоторых типов содержимого¹⁷¹. Из того, что длина запроса *может* достигать 5 Мб, еще не следует, что она такой и *должна* быть.
- Избегайте атак с перепривязкой (`rebinding`) DNS (CWE-346). Для этого отключите инспектор на сигнале `SIGUSR1` и не запускайте протокол инспектора в производственной среде. Как и отладка, этот протокол не предназначен для режима эксплуатации¹⁷⁰.
- Не публикуйте потенциально конфиденциальную информацию из своих файлов и папок, находящихся в текущем каталоге, когда собираетесь опубликовать пакет¹⁷⁰.
 - Используйте `npm publish --dry-run`, чтобы вывести список всех файлов, предполагаемых к публикации, и внимательно проверьте его.
 - Используйте файлы `.gitignore` и `.npmignore`, чтобы перечислить файлы, не подлежащие публикации.
- Не используйте следующие опасные функции: `eval()` и `child_process.exec()`. Следующие модули используйте очень осторожно: `fs`, `vm`¹⁷³.

Java

Я изучала Java в колледже после Pascal, C и C++. Он казался таким *современным* со своим безопасным доступом к памяти, сборкой мусора и причудливой виртуальной машиной. Он мог работать *везде*! А приложения были *огромными*. На том курсе также читались основы объектно ориентированного программирования (ООП) – концепции, совершенно новой и удивительной для моего 20-летнего мозга*.

Сегодня, когда мне говорят, что приложение написано на Java, мне представляется древний монолит. Я думаю о том, что буду искать какие-то уяз-

* Вот интересно, как автор умудрилась изучать C++ и пройти мимо концепции объектно ориентированного программирования. – *Прим. перев.*

вимости и смогу использовать свои старые инструменты, например DAST (динамическое тестирование безопасности приложения), которые не так хорошо работают с более современными архитектурами. Я не хочу этим сказать, что написанное на Java приложение не может быть современным, надежным и безопасным! Когда почти половина (49 %) разработчиков говорят, что пишут на Java¹⁷⁴, мы должны признать, что это невероятно важный язык, и нужно понимать, как кодировать на нем безопасно.

Рекомендации по безопасности Java

- Следуйте рекомендациям Oracle по безопасному кодированию: <https://www.oracle.com/java/technologies/javase/seccodeguide.html>¹⁷⁵.
- Также изучите стандарт Oracle по безопасному кодированию на Java, написанный Фредом Лонгом под патронажем CERT¹⁷⁶.
- Вовремя обновляйте Java SDK¹⁷⁵.
- Следите за безопасностью виртуальной машины Java¹⁷⁵:
 - при задании параметров безопасности JVM ограничивайте разрешения, руководствуясь принципом минимальных привилегий;
 - пользуйтесь менеджерами безопасности, чтобы навязать политики безопасности и предотвратить несанкционированные действия.
- Сведите к минимуму количество проверок разрешений для объектов, потому что чем больше проверок, тем легче допустить ошибку¹⁷⁵.
- Инкапсулируйте свои объекты. Любой интерфейс метода, класса, пакета и модуля должен содержать набор взаимосвязанных поведений и ничего более. Не создавайте черных ходов и дополнительных интерфейсов, чтобы потом было легко получить какую-то информацию. Либо встраивайте официально, либо не делайте вовсе¹⁷⁵.
- Хотя язык Java безопасен относительно памяти, против него все равно возможны DOS-атаки. Старайтесь ограничивать входные данные разумными пределами, определенными лицами, отвечающими за продукт¹⁷⁵. Также применяйте ограничение темпа и остерегайтесь операций, потребляющих непропорционально большое количество ресурсов, например очень больших графических объектов, zip-бомб и т. д.
- Не забывайте освобождать ресурсы после использования, в том числе блокировки, открытые файлы и вручную выделенную память¹⁷⁵.
- Проверяйте границы целых чисел, несмотря на то что Java безопасен относительно памяти. Эти гарантии не стопроцентны¹⁷⁵.
- В Java исключение можно либо *обработать*, либо *распространить*. Очень часто исключения нельзя обработать в месте возникновения, поэтому распространение – употребительная практика. Сделайте ее частью проекта, а не импровизируйте по ходу дела¹⁷⁵.
 - Под *обработкой* понимается, что вы перехватываете исключение, что-то делаете с ним и затем продолжаете работу. То есть вы защитились от исключения.

- Под *распространением* понимается, что вы поручаете обработку исключения стороне, которая вызвала вашу функцию, код или что там еще. В идеале оно будет обработано там, откуда код был вызван.
- Не используйте определения типа XML-документа (Document Type Definition – DTD); если без этого никак не обойтись, действуйте с величайшей осторожностью¹⁷⁵.
- Избегайте BMP-файлов или ограничьтесь чтением локальных файлов, определенных стандартом Международного консорциума по цвету (International Color Consortium – ICC)¹⁷⁵.
- Для чисел с плавающей точкой в Java возможны эксплойты с использованием NaN (не число) или бесконечности. Строго проверяйте числа, полученные из ненадежных источников, перед использованием их в приложении¹⁷⁵.
- Делайте все классы, интерфейсы, члены классов и конструкторы закрытыми (private) или с доступом в пределах пакета (package-private), если только нет причин делать их открытыми (например, потому что это часть открытого API). По умолчанию закрывайте доступ. Документируйте все случаи использования «public» с объяснением причин¹⁷⁵.
- Модули в Java обеспечивают строгую инкапсуляцию (защиту) классов и интерфейсов в пределах пакетов, где они определены. Используйте модуль, чтобы скрыть содержимое пакета. Экспортируйте что-то из пакета, только если на то имеются веские причины, и документируйте их¹⁷⁵.
- Ограничивайте видимость экземпляров ClassLoader, потому что они многое умеют и содержат операции, которые вы, возможно, не хотите разделять¹⁷⁵.
- Ограничивайте расширяемость классов и методов, объявляя их final. Думайте, нужно ли им наследовать, или они должны быть «финальными». Не оставляйте их нефинальными, но никем не наследуемыми; вы должны принять какое-то определенное решение и кодировать в соответствии с ним¹⁷⁵.
- Если предполагается, что объект будет возвращать вызывающей стороне значение, полученное из ненадежного источника, то сначала проверьте его. Проверяйте любые входные данные перед использованием. Пример: объекты Class, возвращенные загрузчиками ClassLoader¹⁷⁵.
- Хотя код на чистом Java не подвержен традиционным атакам с переполнением буфера, о платформенных методах этого не скажешь¹⁷⁵. Чтобы защитить платформенный код, не объявляйте его открытым, а сделайте закрытым и раскрывайте функциональность через открытый метод-обертку, написанный на Java.
- По возможности используйте неизменяемые типы данных, классы, выходные типы и значения и т. д.¹⁷⁵. Если необходимо изменить или передать значение, создайте безопасную копию и передавайте ее.

- Делайте открытые статические поля финальными. Кроме того, делайте их константными¹⁷⁵.
- Не раскрывайте изменяемые статические переменные или модифицируемые коллекции. Раскрывайте вместо этого копию, доступную только для чтения¹⁷⁵.
- Предотвращайте несанкционированное конструирование конфиденциальных классов, делая конструкторы закрытыми. Вместо открытых конструкторов определяйте статические фабричные методы и выполняйте проверку с помощью `SecurityManager`¹⁷⁵.
- Если при выполнении конструктора возникла ошибка, не разрешайте использовать объект. Уничтожьте его и повторите вызов, если было исключение. Гарантируйте, что объект нельзя использовать, если обрабатываете или распространяете возникшее исключение¹⁷⁵.
- Не допускайте вызова из конструкторов, а также методов `clone`, `readObject` и `readObjectNoData`, допускающих переопределение, чтобы помешать злоумышленнику получить доступ к `this` и использовать его в собственных целях. В идеале все методы, допускающие переопределение, следует пометить, чтобы эту задачу было проще решить¹⁷⁵.
- Не разрешайте клонирование нефинальных классов, поскольку это может вызывать разного рода проблемы из-за наличия двух копий (одна из которых вами не контролируется). В идеале все нефинальные классы следует пометить, чтобы эту задачу было проще решить¹⁷⁵.
- Ограничивайте использование методов обратного вызова. Если собираетесь их использовать, то позаботьтесь о передаче контекста и строго относитесь к управлению доступом¹⁷⁵.
- Будьте осторожны и ограничивайте привилегии при вызове метода `java.security.AccessController.doPrivileged`. Он очень мощный и позволяет коду задавать собственные разрешения при выполнении операций, контролируемых `SecurityManager`¹⁷⁵.
- Не кешируйте результат привилегированных операций с последующей передачей в контекст, не имеющий таких же разрешений. Хотя это довольно необычно, Oracle включила эту рекомендацию в руководство по кодированию, а значит, кто-то это делал, и последствия были печальными¹⁷⁵.
- `SecurityManager` можно обойти с помощью нескольких методов в базовых библиотеках времени выполнения Java. Почему? Понятия не имею, почему кто-то счел эту идею здоровой. Вместо проверки по всей цепочке метод проверяет только разрешения непосредственно вызвавшего метода и дает права соответственно. Это называется *чувствительностью к вызывающей стороне* (*caller sensitive*). Если вы думаете, что ограничение, поставленное на уровне верхнего класса, будет распространяться по всей цепочке наследования... то это не всегда так¹⁷⁵.

Вот список методов, для которых разрешения следует проверять очень-очень тщательно:

- `java.io.ObjectStreamField.getType;`
- `java.io.ObjectStreamClass.forClass;`
- `java.lang.Class.newInstance` (объявлен нерекомендуемым);
- `java.lang.Class.getClassLoader;`
- `java.lang.Class.getClasses;`
- `java.lang.Class.getField(s);`
- `java.lang.Class.getMethod(s);`
- `java.lang.Class.getConstructor(s);`
- `java.lang.Class.getDeclaredClasses;`
- `java.lang.Class.getDeclaredField(s);`
- `java.lang.Class.getDeclaredMethod(s);`
- `java.lang.Class.getDeclaredConstructor(s);`
- `java.lang.Class.getDeclaringClass;`
- `java.lang.Class.getEnclosingMethod;`
- `java.lang.Class.getEnclosingClass;`
- `java.lang.Class.getEnclosingConstructor;`
- `java.lang.Class.getNestHost;`
- `java.lang.Class.getNestMembers;`
- `java.lang.ClassLoader.getParent;`
- `java.lang.ClassLoader.getPlatformClassLoader;`
- `java.lang.ClassLoader.getSystemClassLoader;`
- `java.lang.StackWalker.forEach;`
- `java.lang.StackWalker.getCallerClass;`
- `java.lang.StackWalker.walk;`
- `java.lang.foreign.SymbolLookup.loaderLookup;`
- `java.lang.invoke.MethodHandleProxies.asInterfaceInstance;`
- `java.lang.reflect.Proxy.getInvocationHandler;`
- `java.lang.reflect.Proxy.getProxyClass` (объявлен нерекомендуемым);
- `java.lang.reflect.Proxy.newProxyInstance;`
- `java.lang.Thread.getContextClassLoader;`
- `javax.sql.rowset.serial.SerialJavaObject.getFields.`

Сериализация в Java

Десериализация объектов, полученных из ненадежных источников, в Java опасна, ее следует избегать¹⁷⁵. Ниже приведено несколько советов, как работать с сериализацией безопасно.

- Не сериализуйте классы, чувствительные к безопасности.
- Защищайте конфиденциальные данные во время сериализации путем¹⁷⁵:
 - объявления конфиденциальных полей `transient`;
 - надлежащего определения поля-массива `serialPersistentFields`;

- избирательной реализации метода `writeObject` и использования `ObjectOutputStream.putField`;
 - реализации метода `writeReplace` для замены экземпляра сериальным прокси;
 - реализации интерфейса `Externalizable`.
- Проектируйте десериализацию, чтобы она работала как конструктор (в конце концов, это *и есть* конструирование объекта). Используйте `ObjectInputStream.readFields` вместо `ObjectInputStream.defaultReadObject`. Выполняйте проверки входных данных для метода `readObject`. Создавайте копии десериализованных изменяемых объектов, прежде чем присвоить их внутренним полям в реализации метода `readObject`¹⁷⁵.
 - Выполняйте одинаковые относящиеся к безопасности проверки в классе во время сериализации и десериализации¹⁷⁵.
 - Тщательно проверяйте разрешения для `SecurityManager` в той мере, в какой они применимы к десериализации¹⁷⁵.
 - Используйте средства фильтрации во время сериализации¹⁷⁵.
 - Сверяйтесь со шпаргалкой OWASP по десериализации, она *очень* полезна¹⁷⁷.

Чего следует избегать в Java

- Избегайте метода `Method.invoke()`¹⁷⁵.
- Избегайте использовать чувствительные к вызывающей стороне методы в интерфейсных классах¹⁷⁵.
- Доступ к `InvocationHandlers` в общем случае должен быть закрыт (ограничивайте его)¹⁷⁵.
- По возможности избегайте использовать Java Native Interface (JNI). Риски высоки¹⁷⁵.
- Избегайте внедрения кода при использовании Java Persistence Query Language, для чего применяйте параметризованные запросы и проверяйте входные данные¹⁷⁹.
- Избегайте внедрения XPath-кода, для чего используйте `XPath Variable Resolver`¹⁷⁹.
- Избегайте использования криптографической архитектуры Java (Java Cryptography Architecture – JCA) и классов криптографических расширений Java (Java Cryptography Extensions – JCE) для шифрования; используйте вместо них сторонние компоненты, которые проще реализовать безопасно (поскольку за вас уже сделано больше работы)¹⁷⁹.

Хотя на тему Java можно было бы написать еще очень много, размер этой книги ограничен (так говорит издатель). Я настоятельно рекомендую прочитать стандарт SEI CERT Oracle кодирования для Java в дополнение к моим рекомендациям¹⁸⁰.

TypeScript

TypeScript – новый модный язык, созданный Microsoft¹⁸¹; он является надмножеством JavaScript. В него введена статическая типизация и дополнительные средства, позволяющие разработчикам писать более чистый и удобный для сопровождения код благодаря отлавливанию потенциальных ошибок и предоставлению лучших инструментальных средств на этапе разработки. TypeScript-код *транспируется* в обычный JavaScript, поэтому он совместим со всеми движками JavaScript и браузерами. Он предлагает такие продвинутые средства, как классы, интерфейсы, модули и обобщенные типы, что упрощает создание крупномасштабных приложений. Статическая типизация позволяет обнаруживать ошибки еще до начала выполнения, так что код получается более надежным. Кроме того, поддерживаются современные средства JavaScript и предоставляется отличная поддержка со стороны редактора, в том числе автодополнение кода, средства рефакторинга и отладки. TypeScript широко распространен и поддерживается основными каркасами и библиотеками, что делает его идеальным выбором для построения масштабируемых и удобных для сопровождения веб-приложений¹⁸².

Что рекомендуется делать в коде на TypeScript

- Включайте режим `strict` (как в JavaScript)¹⁸².
- Не используйте тип данных `any`; определяйте конкретный тип данных¹⁸³. Оставляя что-то неизвестным или неоднозначным, мы открываем путь уязвимостям.
- Используйте `let` вместо `var`, чтобы у переменных была очевидная область видимости, которую невозможно скрытно переопределить в другом месте¹⁸³.
- Объявляйте константы как `const`. Тем самым гарантируется, что они не будут случайно переопределены где-то еще¹⁸³.
- Используйте кортежи для определения типизированных массивов фиксированной длины (это настоящий массив, который может содержать точно определенное число элементов, иначе возникнет синтаксическая ошибка)¹⁸³.
- Будьте осторожны, решая, какие свойства и методы ваших классов должны быть открытыми, закрытыми и защищенными, особенно, когда вовлечены конфиденциальные данные. Помните, что в TypeScript все классы открытые¹⁸³.
- Будьте осторожны при использовании `DefinitelyTyped` и других специальных типов для пакетов, с которыми не поставляется собственное определение типа. Проверьте точное совпадение с выводом¹⁸⁴.
- И в TypeScript, и в JavaScript используйте `===` и `!==`, а не `==` и `!=`. Причина в том, что при сравнении разных типов возможна неоднозначность¹⁸⁵.

- Задавайте значения по умолчанию при инициализации параметров функций. В противном случае они останутся неопределенными, и если по какой-то причине вы попытаетесь использовать их в вычислениях, не присвоив предварительно значения, то код окажется в неизвестном состоянии¹⁸⁵. По той же причине инициализируйте переменные.

Совет Объявление переменных как `const` относится не только к TypeScript, но и к JavaScript и Node.js! Давайте следить за тем, чтобы наши константы нельзя было переопределить ни в каком языке.

Чего не рекомендуется делать в коде на TypeScript

- Избегайте публикации карт исходного кода в производственной среде, поскольку они могут раскрыть оригинальный TypeScript-код¹⁸⁴.

Вы, наверное, заметили, что этот раздел невелик. Поскольку в TypeScript уже встроены некоторые средства безопасности (так как он строго типизирован), то если вы будете применять все, что узнали о JavaScript, а также не зависящие от языка рекомендации из части I и к тому же следовать безопасному жизненному циклу разработки системы, то все должно быть в порядке!

C#

Когда я работала разработчиком программного обеспечения в государственном учреждении, мне приходилось сначала писать на ASP, потом на VB.NET и, наконец, на C# – перед тем, как я перешла в Microsoft, центр вселенной .NET! К тому времени я уже несколько лет занималась безопасностью, и мои навыки программирования немного запылились. Хотя я была очень разочарована тем, что из Visual Studio убрали «визуальное» перетаскивание (видимо, никому, кроме меня, оно не нравилось?), зато была в восторге от каркаса и языка, спроектированного и сопровождаемого с учетом безопасности.

C# – статически типизированный язык со строгой проверкой типов, который обнаруживает ошибки на ранней стадии (на этапе компиляции, а не выполнения)¹⁸⁶. Он безопасен относительно памяти, является объектно ориентированным и компилируется в байт-код¹⁸⁸.

Что рекомендуется делать в коде на C#

- Старайтесь поддерживать актуальность каркаса и зависимостей. Удаляйте неиспользуемые зависимости перед развертыванием. Это просто лишний, никому не нужный багаж. В VS Code и Visual Studio есть возможность автоматического удаления неиспользуемых зависимостей.
- При работе с SQL Server для пущей безопасности используйте механизм объектно-реляционного отображения (ORM) Entity Framework.
- Используйте AntiXssEncoder или HTML-кодирование, встроенное в каркас для кодирования вывода¹⁸⁷.

- Подумайте об использовании таких библиотек, как AntiXSS, помогающих в *санитровании* входных данных (после проверки)¹⁸⁷.
- Используйте механизм идентификации в ASP.NET для управления сессиями и идентификацией¹⁸⁷.
- Используйте средство ASP.NET Request Validation для проверки и санитрования входных данных¹⁸⁷.
- Используйте встроенную в ASP.NET MVC поддержку антиподложных токенов (для противодействия CSRF)¹⁸⁷. Для этого добавьте вызов метода `@Html.AntiForgeryToken()` в файл `_Layout.cshtml`.
- Используйте атрибут `cookie SameSite` в файле `Startup.cs` для предотвращения CSRF: `options.Cookie.SameSite = SameSiteMode.Lax;`¹⁸⁷.
- Используйте стойкий алгоритм хеширования, например `bcrypt` или `PBKDF2`¹⁸⁷.
- При создании белого списка для проверки входных данных используйте либо атрибут `RegularExpression`, либо объектную модель регулярных выражений¹⁸⁸.
- Используйте движок MVC Razor, потому что он автоматически кодирует все входные данные, помогая избежать XSS¹⁸⁸.
- Во избежание XSS применяйте URL-кодирование к строкам запросов, пользуясь NuGet-библиотеками `System.Net.WebUtility.UrlEncode` и `System.Net.WebUtility.UrlDecode`¹⁸⁸.
- Для обработки ошибок либо используйте атрибут `ExceptionHandlerAttribute` (возможно, понадобится переопределить метод `OnException` и написать собственный обработчик исключений), либо сконфигурируйте `UseExceptionHandler` для работы в производственной среде (в средах более низкого уровня он показывает стандартное системное сообщение, а в производственной – пользовательское сообщение)¹⁸⁸.
- Удаляйте информацию о версии системы из ответов¹⁸⁸:
 - удалите заголовок `Server`, добавив строку `UseKestrel(c => c.AddServerHeader = false)` в метод `CreateWebHostBuilder` в файле `Program.cs`;
 - удалите заголовок `X-Powered-By`, добавив строку `<remove name="X-Powered-By"/>` в конфигурационный файл веб-приложения (если его нет, создайте). Эта строка должна находиться в разделе `<customHeaders>`. Ее можно удалить из настроек Internet Information Services (ISS), если вы считаете, что это проще;
 - удалите заголовок `X-AspNet-Version`, добавив строку `<httpRuntime enableVersionHeader="false" />` в конфигурационный файл веб-приложения;
 - удалите заголовок `X-AspNetMvc-Version`, добавив строку `MvcHandler.DisableMvcResponseHeader = true;` в начало файла `Global.aspx`.
- При аутентификации пользователей рассмотрите возможность использования платформенных возможностей Windows Authentication для

Windows-приложений и использования механизма Form Authentication для веб-приложений¹⁸⁹. Или пользуйтесь отраслевыми стандартами типа OAuth и OpenID Connect либо сторонними службами аутентификации, например X (прежний Twitter) или Google, с помощью службы IAuthentication¹⁹⁸.

- Для авторизации доступа к страницам и файлам рассмотрите возможность использования платформенной авторизации (например, списков управления доступом) и URL-авторизации (к каким URL-адресам разрешен доступ пользователям)¹⁸⁹. Можете также использовать компоненты авторизации, в том числе атрибуты `AuthorizeAttribute` и `AllowAnonymousAttribute`, которые находятся в пространстве имен `Microsoft.AspNetCore.Authorization`¹⁹³.
- Используйте класс поставщика членства из каркаса Membership для управления пользователями приложения, а также встроенные элементы управления для входа в веб-приложение¹⁸⁹.
- Используйте для протоколирования ILogger API или TradeSource (из .NET SDK) либо стороннюю библиотеку, например Log4Net, Nlog или serilog^{190, 191}.
- Пользуйтесь механизмом безопасности доступа к коду (Code Access Security – CAS) в .NET, чтобы определить разрешения для выполнения кода. Он может гарантировать, что ваш код не сможет обратиться к системным файлам и делать другие нежелательные вещи. Считайте, что это применение к вашему коду принципа наименьших привилегий; если код будет скомпрометирован, то ущерб окажется значительно меньше¹⁹².
- Используйте пространство имен `System.Security.Cryptography` для хеширования, формирования цифровых подписей, генерирования криптостойких случайных чисел и других вещей¹⁹².
- Используйте ASP.NET Identity для управления идентификацией, если не хотите использовать сторонний продукт или что-то предоставленное поставщиком облачных услуг¹⁹².
- Используйте анализатор Microsoft Roslyn для поиска уязвимостей в коде (это бесплатно) или в качестве инструмента статического анализа (это платно)¹⁹².
- Используйте пакеты `Microsoft.AspNetCore.DataProtection` для управления ключами, выполнения криптографических операций и других вещей, если работаете с .NET Core¹⁹³.
- В ASP.NET Core используйте средства противодействия подлогу в подсистеме ASP.NET Core Data Protection, чтобы предотвратить CSRF, если страница создана не Razor и, стало быть, уже содержит токены, препятствующие CSRF¹⁹⁶.
- По возможности используйте Razor, потому что он автоматически кодирует вывод всей информации, взятой из переменных¹⁹⁷.

- Используйте кодировщики, предоставленные каркасом в пространстве имен `System.Text.Encodings.Web`. Можете даже настраивать их, если хотите¹⁹⁷.
- Используйте объекты `System.Security.Permissions.PrincipalPermission` для реализации в своих приложениях ролевого управления доступом (RBAC), являющегося частью каркаса .NET¹⁹⁸.
- Используйте NuGet для поддержания пакетов в актуальном состоянии.
- Если для разбора XML-файлов вы используете `XmlTextReader`, то обязательно присваивайте свойству `DtdProcessing` значение `Prohibit` или `Ignore`, чтобы избежать уязвимости внешних сущностей (XML External Entities – XXE)¹⁹⁹.
- Используйте `System.Text.RegularExpressions` для создания регулярных выражений с целью проверки входных данных²⁰⁰.
- Используйте `System.Security.Cryptography` для криптографических операций, в частности хеширования и шифрования²⁰⁰.

Чего не рекомендуется делать в коде на C#

- При работе с .NET Core не используйте атрибут `RequireHttpsAttribute` для веб-API, которые принимают конфиденциальную информацию, потому что клиенты API могут не понимать или игнорировать перенаправления с HTTP на HTTPS. Либо игнорируйте (не прослушивайте) HTTP-запросы, либо возвращайте ответ с кодом 400¹⁹⁴.
- Не используйте инструмент Secret Manager в производственной среде, потому что он не шифрует сохраняемые секреты и не должен рассматриваться как надежное хранилище¹⁹⁵.
- Используйте `@` только в HTML-контексте, а не при попытке вставить ненадежные входные данные прямо в JavaScript. Razor автоматически обрабатывает только HTML-контекст¹⁹⁷.
- Не помещайте ненадежные данные в класс `HtmlString`, потому что он не производит кодирования автоматически¹⁹⁷.
- .NET компилирует код на промежуточный язык Microsoft Intermediate Language (MSIL), в который встроены проверки того, что он используется только в определенных адресах памяти. Не отменяйте эту защиту путем определения шага верификации¹⁹⁸.
- Не используйте функцию `eval()` с аргументом, полученным от пользователя, ни в каком языке¹⁹⁸.
- Не пишите собственные функции обеспечения безопасности взамен предоставленных каркасом. Лица, сопровождающие .NET, помешаны на безопасности и встраивают соответствующие средства в каркас специально. Пользуйтесь этим!

Вы, наверное, обратили внимание, что многое из того, что «рекомендуется делать» в C#, относится к средствам безопасности, встроенным в каркас .NET, – и это хорошо! Используйте все ценное, что вам дают. Сверх того,

Microsoft предлагает Visual Studio Code, великолепную и безумно популярную IDE, бесплатно. Пользуйтесь и ей тоже! И раз уж зашла об этом речь, добавляйте столько плагинов безопасности, сколько сможете; в экосистеме множество средств, которые помогут вам писать лучший код, чем когда-либо прежде.

PHP

PHP (это переставленный акроним от Hypertext Preprocessor) – очень популярный универсальный скриптовый язык с открытым исходным кодом, который прекрасно подходит для веб-разработки (и когда-то был самым распространенным выбором для создания интернет-магазинов)²⁰¹. На PHP написано множество унаследованных сайтов и систем, и зачастую, оказывая консультационные услуги по безопасности приложений и сталкиваясь с PHP-приложениями, я нахожу кучу уязвимостей – не потому, что PHP – плохой язык, а потому что чем старше приложение, тем вероятнее, что при его написании безопасность не стояла на первом месте.

PHP-код выполняется на сервере (не в браузере, как JavaScript), который генерирует HTML-разметку, отправляемую клиенту (браузеру). Это значит, что конечные пользователи видят не написанный вами код, а только сгенерированный, что создает дополнительный уровень абстракции и обфускации²⁰¹.

Люди, сопровождающие PHP, составили руководство по безопасности и распространяют его бесплатно, оно доступно по адресу <https://www.php.net/manual/en/security.php>. Однако предупреждаю – это чтение не для слабых духом. Отведите половину дня, чтобы прочитать и понять его.

Что рекомендуется делать в коде на PHP

- Если PHP устанавливается на сервер в виде двоичного файла, играющего роль интерфейса общего шлюза (Common Gateway Interface – CGI), то²⁰²:
 - используйте конфигурационные директивы `cgi.force_redirect`, `doc_root` и `user_dir`, чтобы предотвратить доступ к своему каталогу / `secret/doc.html`;
 - используйте конфигурационную директиву `cgi.force_redirect`, чтобы пользователи не могли вызывать ваш PHP-сервер напрямую. Все запросы в этом случае будут подвергаться контролируемому вам перенаправлению;
 - задайте корневой каталог с PHP-скриптами с помощью директивы `doc_root` в конфигурационном файле или переменной окружения `PHP_DOCUMENT_ROOT`, чтобы нельзя было выполнить никакие скрипты, кроме тех, что для этого предназначены;
 - подумайте о том, чтобы поместить двоичный PHP-парсер вне доступного из веба дерева файлов – для пущей безопасности. Сделайте его исполняемым и соответственно защитите.

- Если PHP устанавливается в качестве модуля Apache, то²⁰²:
 - используйте средства авторизации Apache или спроектируйте свои собственные на основе протокола LDAP (Lightweight Directory Access Protocol – LDAP) (на совсем уж крайний случай);
 - не давайте своему экземпляру Apache права root;
 - вместо прав root используйте `open_basedir` для управления доступом к каталогам (заодно это поможет избежать атак с выходом за пределы назначенного каталога).
- Для обеспечения безопасности сеанса нельзя рассчитывать только на модуль PHP `session`. Необходимо также включить `session.use_only_cookies`, заголовок безопасности HTTP Strict Transport Security (HSTS) и режим `session.use_strict_mode`; используйте управление сеансами на основе временных меток и регулярно регенерируйте идентификаторы сеансов с помощью `session_regenerate_id()`²⁰².
- Если используются определенные пользователем обработчики сохранения, то обязательно проверяйте все идентификаторы сеансов²⁰².
- Используйте `session_destroy()` для уничтожения старых сеансов, поскольку автоматически они не уничтожаются²⁰². Дайте ставшему неактивным сеансу несколько секунд, прежде чем уничтожить его; для этого задайте короткий срок действия (временную метку) в переменной `$_SESSION` и запрещайте доступ к сеансу в течение этого времени.
- Используйте сеансы, допускающие только чтение, когда сеансовые данные не нуждаются в обновлении (это значит, что и блокировать тоже нет нужды, что предотвращает DDOS-атаки против блокировки)²⁰².
- Текущий модуль `session` не обнаруживает изменения `$_SESSION`, когда сеанс неактивен. Не изменяйте `$_SESSION`, когда сеанс неактивен²⁰².
- Используйте `output_add_rewrite_var()` для защиты от CSRF²⁰².
- Укрепите параметры сеансов в INI-файле (стандартном конфигурационном файле)²⁰²:
 - задавайте `session.cookie_lifetime=0`, чтобы удалять cookie при закрытии браузера, а не оставлять их надолго;
 - задавайте `session.use_only_cookies=0n`, чтобы не использовать идентификатор сеанса по умолчанию, устанавливаемый методами GET и POST»;
 - используйте настройку `session.cookie_httponly=0n`, чтобы избежать доступа к cookie из JavaScript;
 - чтобы cookie посылались только по HTTPS: установите режим `session.cookie_secure=0n`;
 - установите режим `session.cookie_samesite="Lax"` или `session.cookie_samesite="Strict"` – strict предпочтительнее;
 - производите сборку мусора с помощью `session_gc()` и с помощью комбинации `session.gc_probability` и `session.gc_divisor` гарантируйте, что устаревшие данные сеансов удаляются. Нельзя полагаться на то, что сборщик мусора сделает это автоматически;

- выключите прозрачные идентификаторы сеансов, потому что этот механизм содержит уязвимости и иногда дает утечку идентификатора: `session.use_trans_sid=Off`;
- используйте `session.cache_limiter=nocache`, чтобы избежать кеширования конфиденциальных данных;
- используйте `session.hash_function="sha512"`, чтобы выбирался стойкий алгоритм хеширования;
- задавая `session.save_path`, следите за тем, чтобы каталог был недоступен для чтения всеми; это поможет избежать небезопасных прямых ссылок на объекты (insecure direct object references – IDOR).

Совет Возможно, что параметр `session.cookie_httponly=0n` показался вам странным. Если так, то вы не одиноки! Этот параметр не дает JavaScript получить доступ к значениям внутри cookie, что защищает конфиденциальные данные от потенциальных XSS-атак.

- Предоставляйте ограниченные права доступа к двоичному файлу PHP²⁰². Переменные, отправляемые в вызовах двоичного файла, проверяйте по белому списку. Также во время проверки данных обращайте внимания на нулевые байты и отвергайте содержащие их значения.
- Обновляйтесь чаще. Обновления берите непосредственно с сайта PHP.
- Не пренебрегайте «безопасностью через неясность», для чего установите параметр `expose_php` равным `off` в файле `php.ini`, чтобы скрыть свой PHP-код²⁰². Можете также изменить расширения файлов (дополнительная неясность), воспользовавшись директивой `.htaccess` или в самом конфигурационном файле Apache. Почему бы не заменить все расширения на «.ру»?
- Пользуйтесь предоставленными функциями работы с паролями: `password_hash()` для хеширования пароля и `password_verify()` для сравнения хешированного значения с хранимым²⁰².
- Используйте `htmlspecialchars()` или `htmlentities()` для экранирования специальных символов, которые вы вынуждены принимать (а не удалять в процесс санирования). `htmlentities()` экранирует все символы, а `htmlspecialchars()` – только «специальные»²⁰³.
- Используйте `filter_var()` для фильтрации полученных от пользователя данных при проверке почтовых адресов, URL-адресов и IP-адресов²⁰³.
- Используйте `strip_tags()` для удаления HTML-тегов из пользовательских данных, так чтобы остался только текст²⁰³.
- Подумайте об использовании бесплатных SAST для PHP²⁰⁴, например PHPStan, OWASP ASST (Automated Software Security Toolkit), PHP_CodeSniffer, Semgrep Community, progpilot, Psalm, VisualCodeGrepper и horusec.
- Подумайте об использовании модуля Snuffleupagus, который предотвращает целые классы ошибок и предлагает виртуальные заплатки (это реально помогает в случае дефектов «нулевых дней», эксплуатируемых втемную, как, например log4j)²⁰⁵.

- При использовании в производственной среде файла `php.ini-recommended` (на первом месте) или `php.ini` (на втором месте) не забывайте²⁰⁶:
 - изменить параметр `display_errors` на `off` (по умолчанию равен `on`);
 - изменить `display_startup_errors` на `off` (это значит, что вы должны проверять журналы);
 - установить новый путь к временному каталогу по умолчанию. Для этого измените параметр `session.save_path = /tmp/` на что-то другое;
 - установить `log_errors = on`, чтобы включить протоколирование;
 - установить `allow_url_fopen` в `false` (по умолчанию `open`);
 - установить `ignore_repeated_errors` в `off` (на случай атаки);
 - подумайте о компиляции своего PHP-приложения для работы в производственной среде. Это означает, что человек, имеющий доступ к серверу, не сможет ни прочитать, ни изменить исходный код приложения²⁰⁶.
- Следующие общие укрепленные параметры PHP предложены OWASP; используйте их²⁰⁷:
 - `doc_root = /path/DocumentRoot/PHP-scripts/;`
 - `open_basedir = /path/DocumentRoot/PHP-scripts/;`
 - `include_path = /path/PHP-pear/;`
 - `extension_dir = /path/PHP-extensions/;`
 - `mime_magic.magicfile = /path/PHP-magic.mime;`
 - `allow_url_fopen = Off;`
 - `allow_url_include = Off;`
 - `variables_order = "GPCS";`
 - `allow_webdav_methods = Off;`
 - `session.gc_maxlifetime = 600.`
- Если выполняется загрузка файлов на сервер, то используйте рекомендуемые ниже параметры. Если нет, то установите параметр `file_uploads` в `off`²⁰⁷:
 - `file_uploads = On;`
 - `upload_tmp_dir = /path/PHP-uploads/;`
 - `upload_max_filesize = 2М` (или иное разумное значение);
 - `max_file_uploads = 2` (или иное разумное значение).
- Подумайте о том, чтобы запретить опасные функции и потом не думать о них²⁰⁷:
 - `enable_dl = Off;`
 - `disable_functions = system, exec, shell_exec, passthru, phpinfo, show_source, highlight_file, popen, proc_open, fopen_with_path, dbmopen, dbase_open, putenv, move_uploaded_file, chdir, mkdir, rmdir, chmod, rename, filepro, filepro_rowcount, filepro_retrieve, posix_mkfifo.`
- Есть и другие вещи, позволяющие еще сильнее укрепить PHP. Следующие рекомендации взяты из шпаргалки OWASP²⁰⁷:
 - `session.referer_check = /application/path;`
 - `memory_limit = 50М;`

- `post_max_size = 20M;`
 - `max_execution_time = 60;`
 - `report_memleaks = 0n;`
 - `track_errors = 0ff;`
 - `html_errors = 0ff.`
- По возможности используйте строгое сравнение на равенство (`===` и `!==`) вместо нестрогого (`==` и `!=`).

Чего не рекомендуется делать в коде на PHP

- Не помещайте никаких интерпретаторов, включая интерпретатор PHP, в каталог `cgi-bin`. Если вы настаиваете на этом, то вызывайте его как двоичный файл CGI, чтобы усилить защиту²⁰².
- Не полагайтесь на `session.gc_maxlifetime` как на средство завершения действия идентификатора сеанса. Используйте истечение срока действия на основе временной метки²⁰².
- Не вызывайте `session_regenerate_id(true)` и `session_destroy()` в активном сеансе. Это противоречивое действие, которое уничтожит сеанс пользователя²⁰².
- Никогда не используйте конфиденциальные данные в качестве идентификатора. Часто разработчики используют префикс пользователя в сочетании с идентификатором сеанса в качестве весьма полезного GUID. Если это конфиденциальные данные и если они используются повсеместно, то долго ждать беды не придется²⁰².
- Разработчики не должны перезаписывать ненужные HTML-теги. Это лишнее усложнение, поэтому не делайте так, если нет острой необходимости²⁰².
- Используйте `session.trans_sid_hosts=[list of hosts]`, чтобы указать, какие серверы могут перезаписывать `trans_sid`. Не помещайте в список `*`²⁰².
- Чтобы проверить сеанс вручную и защититься от атак с внедрением идентификатора сеанса, отправьте ссылающуюся страницу, а затем проверьте ее при получении запроса: `session.referer_check=[отправивший URL]`²⁰².
- Не передавайте полученные от пользователей данные следующим функциям, которые исполняют свои аргументы: `fwrite()`, `system()`, `exec()`²⁰².
- Не используйте функции `show_source()`, `highlight_string()` и `highlight_file()` на реальном сайте; они предназначены только для отладки и содержат конфиденциальную информацию, которая может помочь злоумышленнику²⁰².
- Не включайте подразумеваемый по умолчанию режим сообщений об ошибках в коде, работающем в производственной среде. Вместо этого используйте предоставляемые PHP функции обработки ошибок для создания собственных обработчиков. Те, что подразумеваются по умол-

чанию, выдают конфиденциальные системные данные²⁰². Подумайте об использовании функции `error_reporting()` с последующей оценкой значений `E_ALL` в среде разработки для тестирования и отладки, а также чтобы убедиться в отсутствии конфиденциальных данных. Выключите ее перед передачей в эксплуатацию.

- Не передавайте полученные от пользователей данные следующим уязвимым функциям: `fopen()`, `fsockopen()`, `popen()`, `include()`, `include_once()`, `require()`, `require_once()`, `file_get_contents()`, `imagecreatefromXXX()`, `mkdir()`, `unlink()` и `rmdir()`²⁰⁶.

Как видите, чтобы сделать PHP-код безопасным, нужно немало потрудиться, в отличие от некоторых других языков и каркасов, в которые безопасность встроена изначально. Однако если вы выбрали PHP, то обеспечение безопасности – часть этого выбора. Удачи вам!

C/C++

Язык программирования C был разработан в 1972 году Деннисом Ритчи²⁰⁸. Язык C++, почти такого же возраста, как автор этой книги, был разработан в 1979 году в компании Bell Laboratories Бьярном Страуструпом, но публично стал доступен только в 1985 году²⁰⁹. C++ был создан, чтобы добавить объектно ориентированные средства в C²⁰⁹. Оба языка весьма низкого уровня и часто используются для программирования систем, которые должны быть чрезвычайно быстродействующими, нуждаются в ручном управлении ресурсами и должны взаимодействовать с операционной системой. Ни тот, ни другой язык не являются безопасными относительно памяти, а значит, уязвимы для разного рода переполнений (строки, целого, буфера и т. д.). Также вы должны самостоятельно заниматься управлением динамической памятью, сбором мусора, форматированием строк, безопасным чтением файлов, не забывать о возвратно-ориентированном программировании (ROP) и состояниях гонки, работать с указателями и т. д. и т. п. Я училась программировать на C, когда мне было 16 лет, а поскольку тогда еще была школьницей, то писала приложения для забавы, не думая о безопасности. Честно говоря, я полюбила программировать с самого начала. Я чувствовала себя невероятно могущественной, создавая что-то из ничего, и была простой как правда.

Снимаю шляпу перед всеми, кто заботится о наших стареющих приложениях на C/C++, помогая сделать их более безопасными; большая часть серверного программного обеспечения по всей планете по-прежнему написана на этих языках. Спасибо вам за то, что делаете наш мир более безопасным.

Совет Если вы пишете с нуля программу, которой нужна такая обработка и гибкость, какую могут предложить C и C++, то я настоятельно рекомендую посмотреть вместо них в сторону Rust, который обеспечивает безопасность работы с памятью. Это избавит вас от многих проблем.

Что следует делать в коде на C/C++

- Купите книгу Robert Seacord «Secure Coding in C and C++» и прочитайте ее.
- Прочитайте стандарт кодирования «SEI Cert C++» и рассмотрите возможность принять его в качестве стандарта в вашей организации. Эта книга проще, чем книга Роберта (по моему скромному мнению), но все равно достаточно подробно.
- Если есть возможность, приобретите инструмент статического анализа, чтобы обеспечить безопасность своего кода. Внимательно изучите вопрос о том, насколько полно поставщик поддерживает именно этот язык, потому что по большей части они ограничиваются поверхностными проверками. Для такого рода инструментов этот язык гораздо труднее прочих. На момент написания книги инструменты статического анализа первого поколения для C/C++ работают намного лучше инструментов второго поколения.
- Решите, кто будет выделять, а кто освобождать память (вызывающая или вызываемая сторона), и сделайте это частью стандарта для каждого проекта²¹⁰. Не отклоняйтесь от принятого решения, потому что это может закончиться утечками памяти. Позаботьтесь о том, чтобы по завершении использования память обязательно освобождалась.
- Не используйте тип `void*` или приведение типов для обхода строгой проверки типов. Строгая проверка типов – ваш друг²¹¹.
- Проверяйте, что значения переменной представимы ее типом. Это значит, что целая переменная должна быть не больше +2 147 483 647 и не меньше –2 147 483 648. То же относится к значениям с плавающей точкой (от 1.2E-38 до 3.4E+38) и значениям других типов. В C и C++ не предусмотрено никаких страховочных сетей, и программа беспощадно «упадет», если вы допустите ошибку в вычислениях²¹¹.
- Всегда инициализируйте переменные. Не оставляйте места случайности.
- Избегайте «ошибок на единицу». Они случаются в C/C++ очень часто. Могут засвидетельствовать на личном опыте.
- Будьте очень-очень осторожны при работе со следующими функциями. В них не встроены никакие меры предосторожности, поэтому дважды перепроверьте себя: `strcpy()`, `strcat()`, `scanf()`, `gets()`²¹².
- Компилируя написанный на C или C++ код такими компиляторами, как GCC или clang, задавайте следующие параметры, чтобы заставить компилятор уделять внимание уязвимостям и ошибкам и включать меры защиты на этапе выполнения²¹³:
 - `-O2 -Wall -Wformat -Wformat = 2 -Wconversion -Wimplicit-fallthrough \`
 - `-Werror=format-security \`
 - `-U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=3 \`
 - `-D_GLIBCXX_ASSERTIONS \`

- `-fstrict-flex-arrays=3 \`
 - `-fstack-clash-protection -fstack-protector-strong \`
 - `-Wl, -z, nodlopen -Wl, -z, noexecstack \`
 - `-Wl, -z, relro -Wl, -z, now`
- Всегда проверяйте границы любых данных, полученных от пользователя. Это в дополнение к проверке правильной инициализации (возможно, придется добавить в конец нулевой байт) и надлежащему уничтожению (освобождение памяти и сборка мусора) по завершении использования²¹².
 - Используйте идиому RAII (захват ресурса является инициализацией), чтобы автоматически захватывать ресурс в момент создания объекта и автоматически освобождать его в момент уничтожения объекта. Это помогает избежать утечек и ошибок²¹⁴.
 - Используйте умные указатели для управления выделением и освобождением²¹⁴.
 - При вызове функции не забывайте об обработке ошибок. Проверка параметров сюда тоже относится²¹⁴.
 - Используйте *безопасные* стандартные библиотеки, функции и контейнеры, такие как `std::vector`, `std::string` и `std::unordered_map`, вместо более старых *стандартных* библиотек²¹².
 - При составлении выражений (комбинаций литералов, переменных, операторов и вызовов функций, порождающих значение при вычислении) следите за корректностью записи и не позволяйте им возвращать `null` или другое непредусмотренное значение. Согласно стандарту CERT C++, этот вопрос нуждается в особом внимании²¹².
 - При использовании контейнеров (векторов и массивов) остерегайтесь таких распространенных ловушек, как «ошибка на 1», пренебрежение перехватом ошибок и проверкой границ и т. д.²¹²
 - Никогда не читайте неинициализированную память²¹².
 - Освобождайте активные блокировки при возникновении исключения²¹².

Чего не следует делать в коде на C/C++

- Не передавайте полученные от пользователя данные таким функциям, как `execl()`, `execpc()` и `CreateProcess()`, которые исполняют свой аргумент²¹². Если все-таки пользуетесь ими, будьте предельно осторожны.
- Не пользуйтесь функцией `rand()`: генерируемые ей числа недостаточно случайны²¹². Используйте вместо нее `srand()`, передавая ей текущее время, и `random()` (доступна, начиная с версии C++ 11)*, или что-то менее

* Тут у автора полная путаница. Начиная с версии C++11, доступен заголовок `<random>`, но в нем по-прежнему определены функции `srand()` и `rand()`. В стандарте POSIX для C действительно определена пара функций `srandom()` и `random()` в дополнение к `srand()` и `rand()`. – Прим. перев.

традиционное, например: `ntheory`, `Math::Prime::Util::GMP`, `Crypt::PRNG`, `Math::Random::Secure` или `Math::Random::ISAAC`.

- Не используйте `printf()`; вместо нее используйте более безопасные `snprintf()` или `fprintf_s()`²¹².
- Не оставляйте файловый поток открытым. Закрывайте файлы, закончив чтение из них²¹².
- Не вызывайте `setjmp()` и `longjmp()`*; используйте вместо них `throw` и `catch`²¹².

Есть несколько книг, специально посвященных вопросу о написании безопасного кода на С и С++, в этом разделе мы не можем охватить все. Материал этого раздела – лишь верхушка айсберга. Я рекомендую пройти какой-нибудь курс по этой теме и купить одну или несколько книг, прежде чем считать себя готовым к написанию безопасного кода на С. Я также рекомендую использовать инструмент статического анализа, а затем провести тестирование на проникновение, прежде чем выпускать свой код в мир.

Заключение

Заканчивая эту главу о безопасном кодировании на 10 самых популярных языках программирования, я надеюсь, что убедила вас в том, что ответственность за написание безопасного кода лежит на каждом разработчике ПО. Лучшие и худшие практики, которые мы рассмотрели для таких языков, как Python, JavaScript, Java и С++, вооружат вас знаниями, необходимыми для выявления и предотвращения типичных уязвимостей в своем коде. Я рассчитываю, что вы возьмете на себя инициативу писать более качественный и безопасный код. Будь то забота о том, какие функции использовать, принятие частей этой главы в качестве стандартов безопасного кодирования на своем рабочем месте или использование более безопасных библиотек – на каждом из нас лежит обязанность придавать важное значение безопасности кодирования.

Я призываю каждого читателя подумать о дальнейшем обучении в этой области. Я понимаю, что мои слова могут показаться корыстными, потому что я сама преподаю безопасное кодирование, но я занимаюсь этой работой, потому что она *необходима*. После того как персонал компании проходит обучение, вся ее культура изменяется к лучшему. Дискуссии в команде во время и после обучения ведут к более безопасным результатам для всех.

Наконец, я хочу попросить (в качестве личного одолжения): поделитесь хотя бы одной вещью, вынесенной из этой главы, со своими друзьями или коллегами. Когда человек узнает что-то от знакомого, а не от «Тани из группы безопасности», он воспринимает это иначе и относится к этому более серьезно. Будьте тем, кто помогает коллегам стать лучше.

* Эта рекомендация относится только к С++. – Прим. перев.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Поделитесь хотя бы одной вещью, вынесенной из этой главы, со своими друзьями или коллегами. Постарайтесь объяснить им, почему это важно и чем они рискуют, отказываясь делать то, что вы им показали.
2. Если вы пользуетесь одним из этих языков программирования на работе, то, когда появится время, возьмите приложение, за которое отвечаете, и проанализируйте его на предмет описанных ошибок. Попробуйте найти и исправить хотя бы одну!
3. Если вы пользуетесь одним из этих языков программирования на работе и еще не следуете какой-то инструкции или стандарту безопасного кодирования, примите части этой главы в качестве руководства для своей организации. Да, я разрешаю вам скопировать часть моей работы, чтобы вы и ваши коллеги могли писать более качественный код. Никто вас не накажет, обещаю.

Популярные каркасы

Каркасы программирования предоставляют базовую функциональность, что позволяет нам писать программы быстрее и создавать лучшие приложения. Считайте каркас уровнем абстракции, предлагающим инструменты, которыми мы можем пользоваться в процессе кодирования. В состав предлагаемой функциональности входят и средства безопасности, например кодирование вывода; это может оказаться весьма полезным, поскольку снимает с программиста бремя каждый раз помнить о том, как это делать правильно. Вместе с тем некоторые имеющиеся в каркасах средства уже устарели, содержат известные уязвимости или работают не *совсем* так, как описано. В этой главе мы на примере нескольких популярных каркасов программирования поговорим об элементах безопасности, которыми вам не грех пользоваться, а также о вещах, которых лучше избегать.

Совет При работе с любым каркасом старайтесь поддерживать его в актуальном состоянии. В обновления включены исправления безопасности, новые средства безопасности и другие чудесные штучки. По крайней мере, не доводите до того, что ваша версия уже не поддерживается.

Веб и JavaScript

Как нетрудно догадаться, веб-каркасы вращаются вокруг JavaScript. Они помогают сделать интерфейс веб-приложения красивым, интерактивным и быстрым. В большинстве из них имеются средства, предотвращающие межсайтовый скриптинг (XSS) – проблему безопасности номер один, которой при работе с JavaScript следует избегать любой ценой.

Express

Express.js (обычно просто Express) – минималистский и гибкий каркас для Node.js, предоставляющий средства для разработки веб-приложений, мо-

бильных приложений и API²¹⁵. Хотя сам Express весит немного, его можно дополнить модулями промежуточного уровня²¹⁵.

Совет Не помещайте полученные от пользователя данные в URL-адрес: `href="userProvidedUrl"`. Это относится к любому JavaScript-каркасу.

Совет Официальное руководство по безопасности в Express имеется по адресу <https://expressjs.com/en/advanced/best-practice-security.html>.

Какие элементы безопасности использовать

- Используйте модуль Helmet для реализации заголовков безопасности и с целью предотвратить XSS: `$ npm install --save helmet`²¹⁶.
- Не позволяйте злоумышленникам узнать, какую версию Express вы используете (чтобы избежать снятия цифрового отпечатка). Для этого удаляйте сообщение «powered by»: `app.disable('x-powered-by')`²¹⁶.
- Добавьте собственный обработчик ошибки «not found» (не найдено), чтобы конфиденциальная информация об ошибке приложения не просочилась наружу²¹⁶.
- Для cookie используйте модуль `express-session` поверх модуля `cookie-session`, поскольку данные cookie видны клиенту (не безопасно). Можете также задать все параметры cookie, в частности флаги `secure` и `httponly`²¹⁶.
- Можете использовать пакет `rate-limiter-flexible` для предотвращения атак полным перебором. Он поможет установить лимиты²¹⁶.
- Используйте аудит пакетов `node` для анализа дерева зависимостей: `$ npm audit`¹⁸⁷.
- Используйте бесплатную командную программу Snyk для поиска зависимостей, содержащих известные уязвимости¹⁸⁷:

```
$ npm install -g snyk
$ snyk test
```

- Подпишитесь на бюллетени по безопасности, например на GitHub по адресу <https://github.com/advisories>¹⁸⁷.
- Избегайте регулярных выражений, уязвимых для атак типа «отказ в обслуживании» (DOS), для чего проверяйте их на сайте <https://github.com/davisjam/vuln-regex-detector>²¹⁷.
- Подумайте об использовании Hydra (<https://github.com/vanhauser-thc/thc-hydra>) для тестирования атак полным перебором на свои страницы входа в систему. Используйте программу только для тестирования собственных приложений, а не во зло²¹⁸. Также присмотритесь к модулям `express-rate-limit` или `rate-limiter-flexible`, чтобы дополнительно защититься от атак полным перебором.
- Используйте модуль `express-validator` для проверки и санирования входных данных²¹⁹.

- Если вы пользуетесь веб-токенами JSON (JWT), то используйте модуль `Express-jwt-permissions` в сочетании с `express-jwt`, чтобы случайно не предоставить доступ, которого быть не должно²²⁰.
- Подумайте об использовании модуля `sanitize-html` для санации входных данных после их проверки²²⁰.

Чего избегать при работе с Express

- Не используйте устаревших или уязвимых версий Express, в частности 2.x и 3.x; на момент написания книги актуальными считались версии 4.x и выше²¹⁶.
- Не используйте модуль `cookie-session` для работы с сеансом или еще чем-то конфиденциальным²¹⁶.
- Не используйте имя `cookie`, подразумеваемое по умолчанию; вы можете переопределить его с помощью модуля `express-session`²¹⁶.
- Не используйте пакет `ratelimiter`; используйте вместо него пакет `rate-limiter-flexible`²¹⁶.
- Не верьте, что встроенного кодирования вывода «достаточно» для предотвращения XSS. Чтобы чувствовать себя в безопасности, обязательно применяйте статический и динамический анализ.

GRC СПЕШИТ НА ПОМОЩЬ

В прошлом году Боб работал в компании, которую приобрела гораздо более крупная компания, и он очень нервничал по этому поводу. Но, как ни удивительно, группа GRC (управление рисками и комплаенсом) пришла на помощь! Они помогли распределить разные продукты между разными командами, чтобы все познакомились с новыми коллегами, и предоставили актуальную документацию по всем новым процессам, которые необходимо было выполнять, включающую контактную информацию для каждой команды и службы, чтобы никто никого не ждал. Как будто собрались заботливые дядюшки и тетушки, помогающие всем освоиться на новом месте. Боб никогда не работал с такой группой GRC раньше и стал ее большим поклонником⁹¹.

React.js

React – очень популярный каркас разработки интерфейсов на JavaScript, предлагающий мощные встроенные средства безопасности, например реализацию виртуальной DOM, уникальный синтаксис JavaScript XML (JSX) и компонентную архитектуру, которая инкапсулирует функциональность внутри компонентов, чтобы ограничить область их выполнения²²¹. В этом разделе мы обсудим, какие элементы безопасности следует использовать, а также средства, которых лучше избегать или использовать очень осторожно.

Какие элементы безопасности использовать

- В React имеется функция `dangerouslySetInnerHTML`, которая названа так, чтобы разработчики знали, что собираются сделать нечто опасное.

В идеале использовать это средство (оно отрисовывает предоставленные пользователем данные) не стоит, но если выхода нет, то сначала прогоните пользовательские данные через DOMpurify, проверьте их, экранируйте (это в любом случае следует делать) и только потом передавайте этой опасной функции²²¹.

- Подумайте об использовании пакета Redux JS для единообразного управления состоянием²²².
- Синтаксис JSX: используйте привязку к данным с помощью фигурных скобок {}, и React автоматически экранирует значения для защиты от XSS-атак. Однако это работает только для отрисовки текстового содержимого, но не HTML-атрибутов. Используйте только для текста²²³! Если вам нужно включить пользовательские данные в HTML-атрибуты (идея, кстати, неудачная), то напишите собственный код проверки с последующим экранированием или санацией.
- Следите, чтобы гиперссылки, принимаемые от пользователей, начинались с HTTPS, чтобы избежать внедрения скрипта на базе URL²²³. Но в идеале вы должны сверять полученные от пользователей ссылки с белым списком, а затем использовать элемент из белого списка вместо того, что прислал пользователь. Кроме того, используйте платформенную функцию разбора URL для проверки URL-адресов²²³.
- React предлагает автоматическое экранирование содержимого при использовании функций отрисовки на стороне сервера, например `ReactDOMServer.renderToString()` и `ReactDOMServer.renderToStaticMarkup()`²²³. Пользуйтесь этим.
- Экранируйте символ `<`, чтобы избежать атак с внедрением кода. Выглядит это так: `</>`²²³.
- При отправке JSON-данных вместе с отрисованными на стороне сервера страницами React не забывайте экранировать символ `<!`. Выглядит это так: `JSON.stringify(preloadedState).replace(/</g, '\\u003c')`²²³. Спасибо, Рон Перрис и Лиран Тал, за этот код.
- Используйте инструмент ESLint для конфигурирования безопасности React; это бесплатный линтер²²³.
- Используйте синтаксис JSX вместо конкатенации строк в JavaScript²²¹.
- В следующих библиотеках используются лучшие рекомендации по безопасности, поэтому, возможно, вы захотите присмотреться к ним: Reactjs Router для маршрутизации, Reactjs Hook Form для валидации форм, Reactjs Redux для управления состоянием, Reactjs Helmet для управления заголовком документа²²⁴.
- Возможно, вы захотите использовать React вместе с Express, чтобы обезопасить серверную часть приложения (React отвечает только за фронтальную часть, но Express может заняться серверной).
- Подумайте об использовании библиотеки `secure-json-parse`; она предлагает проверки безопасности, дополняющие стандартный процесс разбора JSON²²⁴.

- В React предложена новая для меня концепция *области безопасности*. По сути дела, это белый список всех авторизованных пользователей, и, после того как кто-то входит в систему, вы проверяете его по этому списку. Это дополнительный уровень безопасности²²⁵.
- Вы можете задать атрибут `disabled` для любого элемента HTML, чтобы сделать его неизменяемым. Это значит, что он не сможет ни получить фокус, ни быть отправленным в составе формы. Вам придется добавить для такого элемента проверку, чтобы сбросить этот атрибут и получить возможность снова взаимодействовать с элементом, например передать его функции `user-input-validation`²²⁵.

Чего следует избегать

- Не используйте устаревших или уязвимых версий React.js.
- Не используйте сгенерированные пользователем свойства с функцией `createElement`²²¹.
- Не используйте `dangerouslySetInnerHTML`. Просто не используйте – и все. Это самое простое решение.
- React позволяет обращаться к элементам DOM напрямую, используя аварийные люки. Никогда – никогда! – не помещайте полученные от пользователя данные (или данные, которые могли быть изменены пользователем) в аварийный люк. Примерами аварийных люков являются `findDOMNode` и `createRef`²²¹.
- Не полагайтесь на автоматическое экранирование JSX нигде, кроме как в текстовых элементах. Все, что может интерпретировано (например, любой HTML-элемент, не являющийся текстом), нуждается в ручном экранировании (т. е. с помощью кода, написанного вами, а не JSX)²²³.
- Не конкатенируйте полученные от пользователя строки с выводом функции `renderToStaticMarkup()` перед отправкой строк клиенту для «увлажнения» (отрисовки на стороне сервера, а не клиента)²²³.
- Не передавайте полученные от пользователя или еще почему-либо ненадежные данные функции `renderToStaticMarkup()`²²³.
- Программисты часто устанавливают начальное состояние приложения с помощью данных в формате JSON. Функция `JSON.stringify()` преобразует данные в строку, не исключая при этом внедренных вредоносных JS-объектов. Не используйте `JSON.stringify()` с данными, полученными от пользователя²²³.

ЗОЛОТЫЕ ОБРАЗЫ ДЛЯ АЛИСЫ

К новому проекту Алисы были применимы самые разные законодательные акты, а также внутренние политики, описывающие шифрование, сетевые правила, облачную безопасность и всякие другие технические детали, в которых она не разбиралась. Алиса начала уже было волноваться, а потом поняла, что делать: поговорить со своей подругой Фэйт из команды GRC. Фэйт объяснила, что у них есть золотые образы со всеми применимыми

каркасами и тестами производительности, гарантирующие соответствие всем нормативным актам и интернет-политикам, которым проект Алисы должен соответствовать. Она объяснила, что тестирование уже было проведено, что образы прекрасно работали с другими системами и что Алиса может быть уверена, что ее приложения будут создаваться в безопасной инфраструктуре. Алиса расслабилась⁹¹...

Angular

Angular – популярный фронтальный каркас, разработанный и поддерживаемый Google. Он предлагает такие замечательные вещи, как привязку к данным, внедрение зависимостей, модульная структура и органичную интеграцию с TypeScript. Angular предлагает обширную библиотеку готовых компонентов, специализированных утилит для тестирования и умение создавать выразительные и динамичные пользовательские интерфейсы. Также имеется несколько встроенных средств безопасности, которые мы изучим в этом разделе.

Благодарю всех разработчиков Angular за прекрасно написанное руководство по безопасности, которое я буду не раз цитировать в этом разделе. Оно размещено по адресу <https://v17.angular.io/guide/security>.

Примечание В криптографии словом *nonce* (одноразовое случайное число) называется случайно сгенерированное число, служащее для защиты конфиденциального взаимодействия и гарантирующее подлинность транзакции.

Какие элементы безопасности использовать

- Angular принимает участие в программе вознаграждения за обнаружение уязвимостей в ПО с открытым исходным кодом, спонсируемой Google. Обнаружив уязвимости в Angular, отправьте отчет, заполнив форму по адресу <https://bughunters.google.com>²²⁶. Пожалуйста, сообщайте, только если вы абсолютно уверены, что нашли ошибку, люди там занятые. Проверяйте свои находки, прежде чем сообщать о них.
- Не вносите изменения в свою копию Angular. Доработанные версии обычно отстают от текущей и не включают исправлений безопасности и других улучшений. Вместо этого станьте членом сообщества и поделитесь своими изменениями со всеми, отправив запрос на включение изменения²²⁶.
- В Angular все значения по умолчанию считаются ненадежными. При вставке значения в DOM в результате привязки к шаблону или интерполяции Angular производит санирование и экранирование ненадежных значений. Если значение уже проверено и санировано вне каркаса Angular (и потому считается надежным), то вы должны сообщить об этом Angular, пометив его как надежное²²⁶. Но будьте осторожны – не стоит бездумно обходить это средство безопасности.

- Шаблоны Angular по умолчанию считаются надежными и должны рассматриваться как исполняемый код, а не как данные. Никогда не включайте в код шаблонов данные, полученные от пользователей, поскольку это прямой путь к внедрению уязвимостей²²⁶.
- Всегда используйте статический (ahead-of-time – AOT) компилятор шаблонов (а не своевременный [JIT]-компилятор) в производственной среде, чтобы избежать внедрения в шаблон²²⁶.
- Поскольку Angular – каркас для JavaScript и JavaScript – единственный язык, к которому применимы XSS-атаки, вы должны крайне настороженно относиться к этому конкретному виду уязвимостей. Для дополнительной защиты от XSS настройте заголовок безопасности Content Security Policy, включив механизм доверенных типов (Trusted Types), который гарантирует, что данные остаются данными и не будут случайно интерпретированы как код (предотвращая тем самым XSS)²²⁶. Примечание: это относится ко всем JavaScript-каркасам.
- В Angular определено четыре контекста безопасности²²⁶:
 - HTML: используется при интерпретации значения как HTML, например при привязке к `innerHTML`;
 - стиль: используется при привязке CSS к свойству стиля;
 - URL: используется для свойств URL, например `<a href>`;
 - URL ресурса: URL, который загружается и выполняется как код, например в теге `<script src>`.

Angular санирует ненадежные значения для HTML, стилей и URL, но не для URL ресурсов. Санитировать URL ресурсов невозможно, потому что они содержат произвольный код. В режиме разработки Angular печатает сообщение на консоль, когда должен изменить значение в процессе санации, чтобы вы знали о происходящем²²⁶.

- Если механизм доверенных типов не включен, то встроенный в браузер DOM API не защищает вас автоматически от XSS²²⁶.
- Если вы собираетесь манипулировать DOM непосредственно, используйте встроенную в Angular функцию санирования `DomSanitizer.sanitize`²²⁶.
- Если по причинам, диктуемым бизнесом, вы вынуждены использовать в своем приложении исполняемый код, написанный не вами, и если вы уверены, что аккуратно проверили и санировали эти входные данные, то пометьте их как надежные, чтобы Angular не отменил все сделанную вами трудную работу. Чтобы пометить значение как надежное, вставьте `DomSanitizer` и вызовите один из следующих методов²²⁶:
 - `bypassSecurityTrustHtml`;
 - `bypassSecurityTrustScript`;
 - `bypassSecurityTrustStyle`;
 - `bypassSecurityTrustUrl`;
 - `bypassSecurityTrustResourceUrl`.

- Будьте внимательны и используйте подходящий каркас, оказавшись в этой ситуации: HTML, стиль, URL или URL ресурса.
- Каркас Angular отлично сочетается с заголовком Content Security Policy (CSP) (примечание: как и любой другой веб-каркас!); ниже приведена минимальная рекомендованная Angular реализация CSP²²⁶:
 - `default-src 'self'; style-src 'self' 'nonce-randomNonceGoesHere';`
 - `script-src 'self' 'nonce-randomNonceGoesHere';`
- Одноразовое случайное число для CSP можно задать двумя способами²²⁶:
 - установить атрибут `ngCspNonce` для корневого элемента приложения: `<app ngCspNonce="randomNonceGoesHere"></app>`. Используйте этот подход, если имеете доступ к серверным шаблонам, так что можете установить одноразовое число в заголовке и в `index.html` при построении ответа;
 - задать одноразовое число с помощью внедряемого токена `CSP_NONCE`. Используйте этот подход, если имеете доступ к одноразовому числу во время выполнения и хотите кешировать `index.html`;
 - помните, что одноразовое число генерируется случайно для гарантии подлинности.
- Включение механизма доверенных типов совместно с CSP рекомендуется разработчиками Angular и автором этой книги²²⁶. Доверенные типы – веб-платформа, которая позволяет приложениям защищать мощные API, разрешая принимать только недоступные для подделки типизированные значения вместо строки, чтобы предотвратить уязвимости, возникающие из-за использования этих API в сочетании с входными данными, контролируемые пользователем²²⁷. Они помогают останавливать XSS-атаку, гарантируя, что созданные вами типы не могут быть изменены и даже не могут трактоваться как код. Чтобы включить механизм доверенных типов в своем приложении, вы должны сконфигурировать веб-сервер, так чтобы он генерировал HTTP-заголовки, задав одну из следующих политик Angular²²⁶:
 - `angular`: эта политика используется в проанализированном на безопасность внутреннем коде Angular и необходима, чтобы Angular мог работать при включенном механизме доверенных типов. Все встроенные в шаблоны значения и содержимое, санированное Angular, рассматриваются этой политикой как безопасные;
 - `angular#unsafe-bypass`: эта политика предназначена для приложений, которые применяют любой из методов `DomSanitizer`, для обхода мер безопасности, например `bypassSecurityTrustHtml`. Любое приложение, в котором используются эти методы, должно активировать эту политику;
 - `angular#unsafe-jit`: эта политика используется JIT-компилятором. Вы должны активировать ее, если приложение взаимодействует непосредственно с JIT-компилятором или работает в режиме JIT-компиляции, используя платформенные средства браузера;

- `angular#bundler`: эта политика используется командным сборщиком пакетов Angular при создании ленивых компонентных файлов.
- Вы должны включить механизм доверенных типов в производственной среде, во всех менее ответственных средах, в частности в среде разработки (с помощью Angular CLI) и Karma для автономного тестирования. Если вы включите его только в производственной среде, то не сможете правильно протестировать, и может получиться не очень хорошо²²⁶.
- В Angular встроены средства, предотвращающие две распространенные уязвимости HTTP: межсайтовую подделку запросов (CSRF) и межсайтовое включение скрипта (XSSI)²²⁶:
 - CSRF: Angular устанавливает HTTP-заголовок X-XSRF-TOKEN и отправляет его вместе с каждым допускающим изменение запросом (POST, PUT, вообще все, кроме GET), адресованным относительным URL-адресам. Но на стороне сервера его нужно проверять;
 - XSSI (или уязвимость JSON): библиотека Angular `HttpClient` автоматически убирает строку `)]}' , \n`, которую серверы добавляют, чтобы воспрепятствовать исполнению. Убедитесь, что ваш сервер добавляет эту строку, прежде чем возвращать своему JavaScript значения в формате JSON.
- Обязательно проанализируйте все случаи использования `ElementRef`, считающиеся риском безопасности²²⁶. Это единственный элемент во всем Angular, который авторы сочли необходимым пометить как риск в руководстве по безопасности. Будьте с ним осторожны.
- Используйте интерполяцию, чтобы безопасно закодировать потенциально опасные символы и экранировать ненадежные HTML- или CSS-выражения в шаблоне²²⁸.
- Если совместно с HTML-компонентом используются данные, полученные от пользователя (чего вообще-то делать не нужно, но если необходимо...), привязывайте его генерирование к `innerHTML`. Тем самым гарантируется, что данные будут интерпретированы как HTML в правильном контексте, а весь ввод будет санирован²²⁸.
- Пользуйтесь последней стабильной версией Angular. Не используйте версии меньше 2, потому что они заведомо небезопасны²²⁶.
- Angular предоставляет так называемые *защитники маршрутов* (`route guard`), или *маршрутизаторы*, которые защищают маршруты от несанкционированного доступа. Их можно использовать, чтобы гарантировать, что только аутентифицированные и авторизованные пользователи могут обращаться к определенным частям вашего приложения. Пользуйтесь защитниками маршрутов для управления доступом, аутентификации и обеспечения безопасности данных. Существует несколько классов защитников маршрутов, например `AuthGuard` и `RoleGuard`²²⁹.

Чего следует избегать

- Не используйте устаревшие и уязвимые версии Angular.
- Избегайте прямого взаимодействия с DOM, используйте по возможности шаблоны Angular. Если по-другому не получается, то хотя бы используйте встроенные в Angular функции санитизации (метод `DomSanitizer.sanitize` с подходящим `SecurityContext`)²²⁶.
- Не создавайте шаблонов Angular на стороне сервера, используя язык шаблонов; это сопряжено с высоким риском уязвимостей, связанных с внедрением в шаблон. Если вы вынуждены использовать язык шаблонов, то выбирайте такой, который экранирует значения для предотвращения уязвимостей XSS на сервере²²⁶.
- Не забывайте о своей половине защиты от CSRF: проверяйте, что HTTP-заголовок `X-XSRF-TOKEN` присутствует во всех запросах, кроме GET)²²⁶.
- Никогда не используйте «родной» DOM API для взаимодействия с HTML-элементами на странице. Например:
 - не вызывайте метод `node.appendChild()`;
 - не используйте методы объекта `document` для взаимодействия со страницей;
 - не пользуйтесь jQuery API.
- Не используйте сторонние (не Angular) движки шаблонов (например, такие движки для Node.js, как Pug и Handlebars), чтобы добавить данные или создать шаблоны Angular. Это все равно что создавать шаблон Angular на основе данных, полученных от пользователя: очень плохо с точки зрения безопасности, потому что так можно занести вредоносный код²²⁸.

ПОРЯДОК SBOM

Боб читал, что есть распоряжение правительства США включать SBOM (software bill of materials – список зависимостей программного обеспечения) во все новые продукты, поставляемые правительству США, если поставщик хочет сохранить свой статус в Федеральной программе управления рисками и авторизацией (Federal Risk and Authorization Management Program (FEDRAMP)). После поглощения Боб точно знал, к кому обращаться: к опекающей всех группе GRC. Они предоставили ему список требований к цепочке поставок, требований SBOM, требований из новых разделов FEDRAMP и новое «семейство» цепочек поставок, выпущенное Национальным институтом стандартов и технологий (NIST). Они собрали все важное в один обозримый список, чтобы работать с ним было проще. Благодаря этим благодетелям времени ушло *намного* меньше! В знак благодарности Боб купил им пончиков, он знал, где ему всегда помогут⁹¹.

jQuery

jQuery – быстрая и облегченная JavaScript-библиотека, используемая разработчиками по всему миру. Располагая интуитивно понятным синтаксисом

и обширной экосистемой плагинов, jQuery позволяет без усилий работать с DOM, обрабатывать события, развертывать анимации, выполнять AJAX-запросы и многое другое. Но эта гибкость – палка о двух концах, от разработчика требуется осторожность при работе с этим каркасом (библиотека часто используется как каркас, поэтому так я ее и буду в дальнейшем называть). Этот каркас существует уже давно и сейчас используется не так часто, как во времена его активной разработки; кроме того, он содержит очень мало средств обеспечения безопасности и не является «безопасным по умолчанию», как React, Angular и Vue.js²³⁴. В этом разделе мы поговорим о том, как использовать этот старый, но мощный каркас безопасно.

Какие элементы безопасности использовать

- Всегда используйте последнюю версию jQuery. Переходите с версий, содержащих известные уязвимости, на более поздние.
- Используйте метод `text()` вместо `html()` при вставке текстового содержимого. Этот метод гарантирует, что данные рассматриваются как простой текст, а не HTML или скрипт, так что всякая возможность XSS исключается²³⁶. Это единственное средство безопасности, которое данный каркас предлагает для борьбы с XSS.
- Проверяйте все данные, полученные в ответ на AJAX-запросы, а затем проводите санирование и кодирование вывода. Не помещайте такие данные в DOM, считайте их ненадежными²³⁶.
- Тщательно проверяйте весь ввод, затем санируйте и кодируйте вывод. Напишите собственные функции, поскольку в каркасе таковых нет.
- Тщательно проверяйте сторонние плагины jQuery перед использованием. В этом может помочь инструмент анализа состава программы (SCA).

Чего следует избегать

- Не используйте устаревшие и уязвимые версии jQuery.
- Хотя jQuery позволяет без труда преобразовать HTML-код в элементы, не делайте этого; вместо этого следует устанавливать атрибуты HTML. Вручную манипулировать элементами HTML или DOM непосредственно – всегда дурная идея²³⁵.
- В jQuery нет функций для экранирования и кодирования вывода; вам придется писать их самостоятельно²³⁵. Можно также использовать движок шаблонов, например Handlebars, который выполняет кодирование автоматически²³⁵.

Произведя многочисленные изыскания по этой теме, автор в настоящее время считает, что лучший совет – избегать jQuery. Если вы пользуетесь им ради обратной совместимости, то обновитесь до последней версии, выполняйте тщательное тестирование (статическое тестирование безопасности приложений [SAST], динамическое тестирование безопасности приложений [DAST], SCA, линтинг) и следуйте тем немногочисленным советам, которые приведены в этом разделе. Не используйте jQuery для новых разработок.

К ЧЕРТУ ЗАМЕНУ ПАРОЛЯ ЧЕРЕЗ КАЖДЫЕ 90 ДНЕЙ

Алиса как-то работала в небольшой компании, где сотрудники обязаны были менять пароли через каждые 90 дней. Как же это раздражало! «Безопасники» совершали обходы рабочих мест и находили приклеенные бумажки с паролями, после чего оставляли замечания, получать которые было... не слишком приятно. В один прекрасный день компания приняла на работу первого специалиста по GRC. Она тут же распорядилась прекратить эту практику. Очевидно, NIST требовал, что учетные записи служб (которые используются самими компьютерами) нужно менять каждые 90 дней, но пароли пользователей – только если подозрение о взломе, если изменилась политика паролей или если пароль очень слабый. Девушка сказала: «Да вам этого никогда и не нужно было делать!» Она сняла огромный груз с плеч Алисиной группы, да и с ее памяти тоже!

Vue.js

Vue.js известен своей прогрессивностью и доступностью, что облегчает создание динамичных пользовательских интерфейсов и одностраничных приложений (SPA). Он также ориентирован на производительность и простоту и предлагает компонентную архитектуру (в отличие от React и Angular), чрезвычайно подробную документацию (опять же, в отличие от многих других фронтальных каркасов), реактивную двустороннюю привязку данных, декларативную отрисовку и многое другое²³⁰. Неудивительно, что он так популярен. Вместе с тем это палка о двух концах, так как SPA-приложения очень сложны (на грани возможного) для оценки традиционными динамическими сканерами; это исключает один из трех видов тестирования, которые мы обычно проводим (статическое, динамическое и зависимостей), а значит, при написании кода необходима еще большая осторожность, чем обычно. В этом разделе мы углубимся в Vue.js.

Какие элементы безопасности использовать

- Если вы нашли уязвимость в каркасе Vue.js, сообщите о ней по адресу security@vuejs.org²³¹.
- Самое главное правило безопасности при работе с Vue – *никогда не использовать ненадежное содержимое в качестве шаблона компонента*. Это прямая цитата разработчиков Vue.js, поэтому отнеситесь к ней серьезно. Добавление ненадежных данных в шаблон компонента – прямая дорога к выполнению произвольного кода в вашем приложении, что с точки зрения безопасности приложения – самое худшее, что может случиться с JavaScript. Еще хуже, если код выполняется в процессе отрисовки на стороне сервера, поскольку это может привести к перехвату управления сервером, краже данных и еще многим катастрофическим исходам²³¹.
- HTML-содержимое автоматически экранируется путем заключения в двойные фигурные скобки: `{{ тут-ваше-содержимое }}`²³¹. Такая конструкция называется *v-текстом*. Динамические привязки к атрибутам

также автоматически экранируются с помощью платформенных API браузера (например, `setAttribute`)²³¹.

- Службы типа CodePen и JSFiddle позволяют выполнять предоставленное пользователями содержимое, но в таком контексте, где оно ожидается и выполняется в песочнице (до некоторой степени) внутри `iFrame`. Будьте осторожны, когда передаете полученные от пользователя данные таким службам, и тщательно тестируйте²³¹.
- Во Vue.js нет встроенной защиты от CSRF/XSRF и XSS, поэтому вы отвечаете за это сами²³¹. С этой целью можно передавать токен в каждом запросе и ответе, а затем проверять, что токен является частью запроса в каждой операции, выполняемой приложением.
- Представление может отрисовывать в клиентском SPA или отрисовать те же HTML-компоненты на сервере (последнее называется *отрисовкой на стороне сервера* [*server-side rendering* – SSR]), затем отправить их браузеру и там уже «увлажнить» статическую разметку, превратив ее в полностью интерактивное приложение на стороне клиента. Если делать так, то традиционный динамический сканер может проанализировать код (в отличие от варианта со SPA, когда большинство динамических сканеров бессильны), и с точки зрения безопасности это преимущество²³². Вместе с тем любой квалифицированный пентестер способен весьма подробно протестировать SPA.
- Используйте `v-if` (удаляет элемент из DOM, если `false`), а не `v-show` (скрывает элемент, оставляя его в DOM; это менее безопасно)²³³.
- Используйте `sanitize-url`, `vue-sanitize` или `vue-3-sanitize` для санирования URL-адресов, вручную делать этого не надо²³³.
- Используйте директиву `:key` внутри директивы `v-for`, чтобы отслеживать и идентифицировать изменения элементов в процессе отрисовки. Это поможет предотвратить атаки с изменением свойств (манипулирование состоянием компонента Vue.js путем изменения ключей элементов в цикле `v-for`) путем манипулирования DOM, а также некоторые атаки типа «отказ в обслуживании» (DoS) (основанные на чрезмерном потреблении ресурсов отрисовки)²³³.
- Подумайте об использовании JS-библиотеки `zxcvbn` для контроля сложности пароля вместо придумывания собственного алгоритма.

Чего следует избегать

- Не используйте устаревшие и уязвимые версии Vue.js. Я уже напоминаю заевшую пластинку, да?
- Лучше всего вообще не использовать полученные от пользователя данные в процессе явной отрисовки HTML-содержимого директивой `v-html`. Но если без этого *никак не обойтись*, то проверяйте, санируйте, экранируйте и кодируйте данные, тщательно тестируйте, документируйте причины принятия такого решения и получите добро от группы

безопасности. Это можно сделать с помощью шаблона, функции отрисовки или функции отрисовки с JSX²³¹. И все же полностью избегать пользовательских данных было бы всего лучше.

- Не помещайте пользовательские данные в URL-адреса: `href="userProvidedUrl"`. Это относится к любому JavaScript-каркасу, а не только к Vue.js²³¹.
- Не помещайте пользовательские данные в стили, например `style="userProvidedStyles"`²³¹. Если без этого не обойтись, то используйте `{{ }}` для экранирования значения, проверяйте и санируйте URL-адрес, откуда его получаете, и делайте это только в iFrame-песочнице²³¹.
- Не отрисовывайте теги `<script>` во Vue. Просто не делайте этого – и все. Здоровее будете²³¹.
- Никогда не устанавливайте Vue в узлы, которые могут содержать отрисованное сервером и предоставленное пользователями содержимое²³¹. Vue не может сказать, какое содержимое отрисовано сервером, а значит, не может защитить вас.

СЛОЖНОСТЬ С СЕРТИФИКАТАМИ

Бобу когда-то довелось руководить командой, которая занималась управлением сертификатами для их собственных приложений. Каждая группа должна была управлять своими сертификатами, и иногда случались ошибки. Сертификаты нужно было менять каждый год, потому что так повелел NIST. Как-то раз группа GRC сказала Бобу, что можно менять сертификат раз в два года, потому что NIST разрешил, а кроме того, предложила заполнять за Боба запросы на продление, чтобы сэкономить время. Группа GRC поняла, что может упростить процедуры, организовав централизованную поддержку всех групп, нуждающихся в сертификатах, и заодно сэкономить время и деньги. Да и аудиторам это понравилось. Боб был очень доволен, что ему больше не нужно думать об этом⁹¹.

Другие каркасы и библиотеки

Есть много разных каркасов, да и код для веба пишется не только на JavaScript! В этом разделе мы в основном будем рассматривать каркасы, ориентированные не только на веб и (или) JavaScript.

.NET (Core)

Во время работы в Федеральной государственной службе Канады я *очень* много писала в .NET в основном на языке VB.NET, но и на C# тоже. Когда я впервые просканировала свой код на наличие уязвимостей (сейчас этот класс инструментов называется DAST и применяется для поиска ошибок в веб-приложениях), сканер нашел XSS. В то время (2011 год) я еще никогда не слыхала о XSS, а поиск в сети, помнится, нашел всего три сайта, на которых

эта проблема упоминалась, и одним из них был OWASP. Мне понадобилось еще две попытки, прежде чем сканирование прошло успешно, и я помню, что мы в группе даже устроили собрание, чтобы обсудить XSS и что нам с этим делать. В то время я работала с очень талантливыми ребятами, и мы написали динамически компокуемую библиотеку (DLL), которая осуществляла проверку входных данных и кодирование вывода. В те времена каркас .NET еще не мог похвастаться таким изобилием замечательных средств защиты, как сейчас. Мы также создали шаблон для всех новых приложений, который включал нашу DLL, тщательно протестированную страницу входа и другие средства безопасности. Мы поделились всем, что узнали, написали отличную документацию, и, откровенно говоря, это была одна из лучших команд разработки, в которых мне довелось работать. Но не все команды таковы, и не каждая организация так хорошо организована. В этом разделе мы поговорим о том, как вы и ваша команда можете воспользоваться этим мощным и разносторонним каркасом, уделяющим много внимания безопасности.

Примечание: прочитайте раздел главы 6, посвященный C#, потому что многое из того, что можно сказать о каркасе .NET, неотделимо от рекомендаций по C#.

Какие элементы безопасности использовать

- Получайте извещения о безопасности, нажав кнопку Watch в следующих репозиториях на GitHub²⁴⁰:
 - объявления о безопасности в .NET Core;
 - объявления о безопасности в ASP.NET Core и Entity Framework Core.
- По возможности используйте встроенные в .NET средства безопасности, такие как аутентификация и управление сеансами, а не пишите свои собственные.
- Для каждой страницы и ресурса (например, объекта или элемента данных) проверяйте, есть ли у запрашивающего доступ. Используйте для этой цели функцию `System.Web.Security.Roles.IsUserInRole(userName, roleName)` или свою собственную. В идеале лучше пользоваться встроенными средствами²⁴⁰.
- Используйте `SecurityEssentials/Core/HttpHeaders.cs` для реализации заголовков безопасности, если работаете с каркасом .NET Core (объявлено нерекондуемым)²⁴⁰.
- Подумайте об использовании `dionach/StripHeaders` с GitHub для удаления информации о безопасности из заголовков ответов. Можно также внести изменения в `web.config` или `Startup.cs`, чтобы выдавать еще меньше информации²⁴⁰.
- Используйте Windows Data Protection API (DPAPI) для безопасного локального хранения конфиденциальных данных²⁴⁰.
- Используйте Entity Framework в качестве ORM (объектно-реляционного отображения) для своей базы данных. Кроме того, используйте

для взаимодействия с базой данных хранимые процедуры (параметризованные запросы в MS SQL), а не статические или динамические SQL-запросы²⁴⁰.

- .NET 5+ поддерживает свойство `ProcessStartInfo.ArgumentList`, которое выполняет кое-какое экранирование символов, но оно далеко от совершенства. Пользуйтесь им, но в дополнение к проверке и санированию²⁴⁰.
- В производственном режиме отладка и трассировка должны быть выключены. Для этого можно воспользоваться преобразованиями в `web.config`²⁴⁰:
 - `<compilation xdt:Transform="RemoveAttributes(debug)" />;`
 - `<trace enabled="false" xdt:Transform="Replace"/>.`
- .NET предлагает токены для предотвращения подделок в атаках типа CSRF; обязательно используйте тип `IAntiforgeryAdditionalDataProvider` в .NET Core. Есть также предложения для AJAX, Windows и т. д.²⁴⁰
- Используйте механизм ASP.NET (Core) Identity для управления идентификацией²⁴⁰.
- Обязательно подписывайте сборки и исполняемые файлы и используйте средства подписания NuGet-пакетов²⁴⁰.
- Подписывайте и (или) шифруйте сериализованные объекты, чтобы гарантировать их целостность. Проверяйте подписи и (или) выполняйте проверку целостности зашифрованных сериализованных объектов перед десериализацией. Не принимайте сериализованные объекты из неизвестных источников ни под каким видом²⁴⁰.
- Для обработки данных используйте один из следующих безопасных процессоров²⁴⁰:
 - `XmlSerializer` и `DataContractSerializer` для сериализации графов объектов в формате XML;
 - `BinaryReader` и `BinaryWriter` для XML и JSON;
 - `API System.Text.Json` для сериализации графов объектов в формате JSON.
- .NET предоставляет функцию протоколирования, которая может вам пригодиться, потому что вы, конечно же, протоколируете исход каждого обращения к элементам безопасности: `Microsoft.Extensions.Logging`²⁴⁰.
- Подумайте об использовании Application Insights для мониторинга²⁴⁰.
- Используйте `IPAddress.TryParse()` и `Uri.CheckHostName()` для проверки правильности IP-адресов и доменных имен²⁴⁰.
- Используйте библиотеку Microsoft AntiXSS при работе со старыми (4.5–4.9x) версиями .NET: `Install-Package AntiXSS`²⁴⁰. Для версий, начиная с 5, она недоступна.
- Используйте пользовательские страницы ошибок и не показывайте пользователю системную дату: `<customErrors mode="On" defaultRedirect="ErrorPage.html" />`²⁴⁰.

- Приложения могут пользоваться интерфейсом `IAuthentication` для доступа к сторонним службам аутентификации, в частности X (Twitter), Google и реализованным внутри организации службам OAuth²⁴¹.
- .NET-компиляторы транслируют код на промежуточный язык Microsoft Intermediate Language (MSIL), обеспечивающий *управляемое выполнение*, при котором возможны только обращения к разрешенной памяти. Это гарантирует изоляцию модулей и помогает среде исполнения соблюдать ограничения безопасности. Microsoft называет это «типобезопасностью» (type safety), хотя концепция не имеет ни малейшего отношения к тому, что все остальные отраслевые игроки обозначают этим термином²⁴¹. Пожалуйста, не отключайте этот механизм и не пытайтесь обойти его.
- Если вы вынуждены иметь дело с ненадежным кодом, используйте одну из следующих мер безопасности: виртуализацию, AppContainer'ы, контейнеры Hyper-V и (или) работайте от имени пользователей ОС с ограниченными правами²³⁷. Всегда проверяйте, а затем экранируйте или санируйте ненадежный код, в идеале сопоставляя его с белым списком.
- Защищайте данные, пока они находятся в памяти, объявляя их закрытыми (private) или внутренними (internal) (и ограничивайте область видимости текущей сборкой)²³⁷.
- Используйте встроенный в .NET механизм ролевой безопасности для реализации авторизации внутри приложений²³⁸.
- .NET поддерживает три вида принципов в предлагаемых средствах ролевой безопасности: общие (не имеют ничего общего с Windows), Windows (все, что связано с пользователями и ролями Windows) и пользовательские (определяйте как сочтете нужным)²³⁸. Чтобы выбрать нужный, используйте класс `System.Security.Permissions.PrincipalPermission`, а затем с помощью выбранного принципа проверяйте разрешения (управление доступом и авторизация) во всем приложении²³⁸.
- Что касается криптографических функций²³⁹:
 - используйте класс `RandomNumberGenerator`;
 - используйте *статические** .NET API для хеширования и имитовставок на основе хешей (hash-based message authentication code – HMAC), например `SHA256.HashData`, – по крайней мере, до тех пор пока бизнес-требования не вынудят вас написать свои реализации, – и делайте это только в случае острой необходимости;
 - если необходимо придумать или реализовать алгоритмы шифрования самостоятельно, наследуйте одному из классов `SymmetricAlgorithm`, `AsymmetricAlgorithm` или `HashAlgorithm`;
 - в большинстве случаев нет необходимости напрямую ссылаться на класс реализации алгоритма, например `AesCryptoServiceProvider`.

* One-shot API, т. е. API, состоящие из вызова одного статического метода без передачи состояния между вызовами. – Прим. перев.

Методы и свойства, которые обычно необходимы, объявлены в базовом классе алгоритма, например AES. Если возможно, не изобретайте свои собственные алгоритмы. Microsoft рекомендует следующие классы алгоритмов²³⁹:

- конфиденциальность данных: AES;
 - целостность данных: HMAC-SHA256, HMAC-SHA512;
 - цифровая подпись: ECDSA, RSA;
 - обмен ключами: ECDH (Elliptic-Curve Diffie-Hellman), RSA;
 - генерирование случайных чисел: `RandomNumberGenerator.GetBytes`, `RandomNumberGenerator.Fill`;
 - генерирование ключа по паролю: `Rfc2898DeriveBytes.Pbkdf2`.
- Шифруйте конфиденциальные части файла `web.config` с помощью программы `aspnet_regiis -pe`²⁴⁰.
 - Удаляйте заголовок версии, добавив следующую строку в файл `Machine.config`: `<http Runtime enableVersionHeader="false" />`²⁴⁰.
 - Удаляйте заголовок `Server`, воспользовавшись классом `HttpContext:HttpContext.Current.Response.Headers.Remove("Server")`; ²⁴⁰.
 - Проверяйте URI-адреса с помощью метода `Uri.IsWellFormedUriString`²⁴⁰.
 - Сравнивайте любое указанное пользователем имя с `User.Identity.Name`. Они должны совпадать²⁴⁰.
 - Проверяйте роль с помощью метода `User.Identity.IsInRole`, прежде чем давать доступ к чему-либо²⁴⁰.
 - В случае привязки к модели убедитесь, что либо (1) вы намереваетесь инициализировать все без исключения значения из формы, либо (2) вы инициализируете только те значения, что собирались. Это нужно, чтобы избежать атак с присваиванием непредусмотренных полей (когда вы собирались обновить только часть полей объекта, а злоумышленник добавил дополнительные значения, и в результате приложение, само того не подозревая, дало ему возможность присвоить значения полям, которые вы изменять не хотели)²⁴². Это можно сделать, явно перечислив все свойства, которые вы хотите обновить; пометив свойство, которое вы обновлять не хотите, как доступное только для чтения при первой инициализации значений; или воспользовавшись методом `TryUpdateModel` при обновлении модели.

В этом каркасе, конечно же, есть еще много средств безопасности, но надеюсь, этот список направит вас на путь истинный.

Чего следует избегать

- Не используйте механизм Code Access Security (CAS)²³⁷.
- Не используйте частично доверенный код²³⁷.
- Не используйте атрибут `AllowPartiallyTrustedCaller` (APTCA)²³⁷.
- Не используйте технологию .NET Remoting²³⁷.

- Не используйте технологию Distributed Component Object Model (DCOM)²³⁷.
- Не используйте двоичные форматы²³⁷.
- Не используйте тип `BinaryFormatter`, он опасен и не рекомендуется²⁴⁰.
- Избегайте или очень осторожно относитесь к атрибуту `[AllowHTML]` и вспомогательному классу `@Html.Raw`²⁴⁰. Не используйте их с данными, которым не доверяете.
- Не используйте класс `SqlCommand` с данными, полученными от пользователя или не заслуживающими доверия²⁴⁰.
- При использовании веб-сервера Internet Information Services (ISS) включайте трассировку в производственной среде²⁴⁰.
- Не используйте `Eval()`. Нигде и никогда²⁴¹.
- Избегайте атак с присваиванием непредусмотренным полям²⁴², для чего пользуйтесь атрибутом `BIND` при первой инициализации полей или методом `TryUpdateModel` для обновления модели.

ТАНЦЫ С СОГЛАСОВАНИЯМИ

Алисе нужно было получить 10 согласований, прежде чем передать новое приложение в эксплуатацию. Она чувствовала усталость от одного взгляда на этот список. И решила наудачу наведаться в группу GRC. Она спросила, действительно ли все эти люди должны согласовать выпуск, или, быть может, кого-то достаточно было просто *проинформировать*. Фэйт снова пришла к ней на помощь. Она сказала, что этот долгий процесс согласования уже вывел из равновесия несколько групп и, похоже, является пустой тратой времени и ресурсов компании, не принося *никакой* пользы. Месяц спустя Фэйт встретила Алису и познакомилась с новым процессом: список из пяти лиц, которые должны согласовывать крупные изменения (новые приложения или переписанные версии старых), и еще пять лиц, которых нужно ставить в известность. Если изменение было небольшим (исправление ошибки или всего одна новая функция) и если оно прошло все проверки в конвейере непрерывной интеграции и непрерывной доставки/развертывания (CI/CD) (имеется в виду SAST, SCA и линтинг), то необходимо было только одно ручное согласование, три автоматических (успешные тесты) и девять уведомлений. Фэйт снова сэкономила ей время! Алиса задумала как-нибудь пригласить ее на обед, она очень хотела познакомиться с ней поближе.

Ruby on Rails

Ruby on Rails, или просто Rails, – каркас полного цикла на базе мощного языка программирования Ruby. Rails исповедует принципы примата соглашения над конфигурацией и «не повторяйся» (*don't repeat yourself – DRY*), чтобы упростить разработку и позволить разработчикам сосредоточиться на существенных вещах, а не на стереотипном коде. Со своей богатой экосистемой пакетов `rubygem` (библиотек), встроенной поддержкой тестирования и упором на лучшие практики в области безопасности Rails остается любимым инструментом как для стартапов, так и для компаний, которые ставят на первое место скорость разработки и масштабируемость веб-приложений. В этом разделе мы поговорим о том, что делать и чего не делать при работе с Rails.

Какие элементы безопасности использовать

- Rails предоставляет объект сеанса для каждого пользователя приложения. Идентификаторы сеансов передаются в безопасных cookie и хранятся в одном из следующих мест²⁴³:
 - `ActionDispatch::Session::CookieStore`: все хранится на стороне клиента;
 - `ActionDispatch::Session::CacheStore`: данные хранятся в кеше Rails;
 - `ActionDispatch::Session::MemCacheStore`: данные хранятся в кластере memcached (это унаследованная реализация, рассмотрите вместо нее `CacheStore`);
 - `ActionDispatch::Session::ActiveRecordStore`: данные хранятся в базе данных с применением Active Record (требуется пакет `activerecord-session_store`);
 - пользовательское или стороннее хранилище.
- В Rails по умолчанию используется в качестве хранилища сеансов `ActionDispatch::Session::CookieStore`. `CookieStore` – это зашифрованный контейнер cookie, предлагающий безопасное место для хранения данных сеансов. В `CookieStore` можно сохранить 4 Кб данных – немного, но обычно достаточно²⁴³.
- Используйте команду `bin/rails secret` для получения новых длинных, случайных и уникальных секретов²⁴³.
- Для создания криптографической соли в тестовой среде и в среде разработки можно использовать `secret_key_base`, выведенный из имени приложения. В других средах следует использовать случайный ключ в файле `config/credentials.yml.enc`²⁴³.
- Не забывайте создавать новый идентификатор сеанса и делать недействительным старый при выходе пользователя из приложения или истечении тайм-аута: `reset_session`²⁴³.
- Подумайте об использовании пакета `Devise` или `AuthLogic` для аутентификации. Если хотите, чтобы пользователи могли входить с помощью сторонних систем аутентификации, например LinkedIn, Facebook, Google и т. д., то можете взять пакет `omniauth`²⁴⁴. Пакет `Devise` также умеет управлять пользователями и реализует другие средства безопасности²⁴³. Можете также пользоваться встроенным методом `has_secure_password`, который поддерживает безопасное хеширование пароля, механизмы подтверждения и восстановления²⁴³.
- Токен противодействия CSRF передается автоматически по умолчанию. Но все же проверьте, не был ли он случайно отключен²⁴³: `config.action_controller.default_protect_from_forgery = true`.
- По умолчанию Rails протоколирует все запросы, отправленные веб-приложению, в том числе конфиденциальные. С помощью фильтра запретите протоколирование всего, что не должно попадать в журнал: `config.filter_parameters << :name-of-your-secret`²⁴³.

- Регулярные выражения в Ruby работают не совсем обычно. Чтобы избежать XSS и других атак с внедрением кода, используйте `\A` (начало) и `\z` (конец) вместо `^` и `$` соответственно²⁴³.
- Проверяйте авторизацию при получении записей, вызывая функцию `current_user`. Она гарантирует, что пользователю разрешено видеть запрошенный ресурс:

```
@project = @current_user.projects.find(params[:id])243.
```

- Разрешите `<strong>` вместо удаления `<script>` для защиты от XSS²⁴³. В Ruby on Rails имеется встроенный фильтр специальных символов SQL, который экранирует `'`, `"`, символ NULL и разрывы строк. При использовании `Model.find(id)` или `Model.find_by_something(something)` эта защитная мера применяется автоматически. Но во фрагментах SQL, особенно в условиях (`where("...")`) и методах `connection.execute()` и `Model.find_by_sql()`, его необходимо применять вручную. Средство Rails, используемое вместо параметризованных запросов, называется *позиционным обработчиком*. `Model.where("zip_code = ? AND quantity >= ?", entered_zip_code, entered_quantity).first243`. Примечание: это работает только с экземплярами моделей.
- Подумайте об использовании ORM ActiveRecord; она поможет избежать внедрения SQL-кода²⁴⁴.
- Если вы не работаете с экземплярами моделей (см. совет выше), то используйте `sanitize_sql` для санирования входных данных перед отправкой их в базу²⁴³.
- Для санирования HTML-кода, полученного от пользователя, применяйте метод `sanitize()`²⁴³.
- Для кодирования вывода используйте метод `html_escape()` (или его псевдоним `h()`), который заменяет символы `&`, `"`, `<` и `>` соответствующими неинтерпретируемыми HTML-сущностями (`&`, `"`, `<` и `>`)²⁴³.
- Для обфускации и кодирования строк используйте Hackvertor²⁴³.
- При использовании команды `system` передавайте параметры, так будет безопасно: `system(команда, параметры)`²⁴³.
- Используйте ПО промежуточного уровня `ActionDispatch::HostAuthorization`, чтобы защититься от перепривязки DNS и других атак на заголовок Host. Оно активируется по умолчанию в среде разработки, но в производственной и прочих средах это нужно делать вручную, задав список разрешенных хостов²⁴³.
- Следующие заголовки безопасности сконфигурированы в Rails по умолчанию²⁴³:
 - `'X-Frame-Options' => 'SAMEORIGIN';`
 - `'X-XSS-Protection' => '0';`
 - `'X-Content-Type-Options' => 'nosniff';`

- 'X-Permitted-Cross-Domain-Policies' => 'none';
 - 'Referrer-Policy' => 'strict-origin-when-cross-origin'.
- Вы можете (и должны) добавить дополнительные заголовки безопасности в файле `config/application.rb`²⁴³.
 - В Rails секреты хранятся в зашифрованном виде в файле `config/credentials.yml.enc`. Rails использует ключ, хранящийся в файле `config/master.key` или в переменной окружения `ENV["RAILS_MASTER_KEY"]`, для шифрования файла учетных данных. Этот файл содержит ключ приложения `secret_key_base`. Его можно хранить к системе управления версиями, но главный ключ должен храниться еще где-то и в безопасности²⁴³. Или же можете использовать инструмент управления секретами.
 - Секреты, хранящиеся в файле учетных данных, доступны с помощью объекта `Rails.application.credentials`²⁴³.
 - Используйте команду `bundle update --conservative rubygem_name` для безопасного обновления уязвимых зависимостей²⁴³.
 - Используйте Brakeman, свободный инструмент статического анализа, созданный специально для Ruby, который ищет проблемы безопасности в коде²⁴⁴. Он не идеален, но для начала очень хорош, особенно учитывая, что бесплатен!
 - По умолчанию cookie в Ruby создаются навечно. Вряд ли вам это нужно для сеансовых cookie или еще чего-то конфиденциального. Задавайте разумный тайм-аут, по истечении которого cookies самоуничтожаются²⁴⁴.
 - Для создания гарантированно сложных паролей пользуйтесь пакетом `zxcvbn`²⁴⁴.
 - Пути в Ruby интуитивно понятны и легко угадываются. Чтобы защитить пользователей от попыток получить доступ к их записям, необходимо реализовать управление доступом. Подумайте об использовании для этой цели библиотек управления доступом на основе ресурсов, например `cancan` и `pundit`²⁴⁴.
 - Изучите поверхность атаки на свое приложение (части кода, доступные атакующему), для чего загляните в файл `/config/routes.rb`. Там записано, какие URL-адреса должны быть доступны и какими контроллерами они обрабатываются. Вы должны убедиться, что доступны только те URL-адреса, которые вы хотели сделать общедоступными. В этом же файле вы можете посмотреть на перекрестные ссылки между контроллерами и URL-адресами и сравнить их со своими предположениями о том, что должно быть доступно пользователям²⁴⁴. Кроме того, удаляйте все маршруты и действия API, которые не хотите раскрывать. Автогенерирование в Rails невероятно полезно, но иногда при этом создается излишняя (неиспользуемая) поверхность, которую лучше убрать.
 - Используя ORM Active Record для баз данных, будьте осторожны при вызове следующих методов, которые не санируют входные данные для

защиты от внедрения SQL-кода: Calculation, Delete All, Destroy All, Exists?, Joins, Order и некоторых других²⁴⁵.

- Для кодирования строк (URL-адресов) вручную используйте `url_encode(s)` или `u(s)`²⁴⁵.
- Подумайте об использовании пакета `secure_headers`, созданного группой безопасности Twitter для реализации дополнительных заголовков безопасности²⁴⁵.
- Подписывайте cookie, содержащие идентификатор сеанса, с помощью `cookies.signed [: secure_session]`. Подписание добавляет уровень безопасности, гарантирующий целостность cookie²⁴⁵.
- Включайте TLS, устанавливайте флаг `secure` для cookie и переадресуйте на протокол HTTPS – все это делается путем добавления директивы `config.force_ssl = true` в файл `config/application.rb`²⁴⁵.

Чего следует избегать

- Не отключайте встроенные по умолчанию средства безопасности без основательной причины. Если все-таки решили отключить, то документируйте свое решение и тщательно тестируйте его. Пример:

```
config.action_controller.default_protect_from_forgery = false.
```

- Не используйте функцию `html_safe` бездумно. Она означает, что вы уже надлежащим образом проверили, санировали, экранировали и закодировали информацию, помещаемую в поле. И теперь вы говорите Ruby, что эту информацию кодировать *не* нужно. То есть вы *отменяете* встроенную по умолчанию безопасность. Используйте эту функцию только тогда, когда *абсолютно уверены* в том, что делаете²⁴⁴.
- Обработывайте мультимедийные файлы асинхронно, чтобы избежать DoS-атак²⁴³.
- Если вы применяете `RedCloth`, то должны использовать белый список в качестве входного фильтра, потому что кое-какие проблемы безопасности там еще остались²⁴³.
- Если пользуетесь плагином `in_place_editor` или действиями, которые возвращают строку, а не отрисовывают представление, то должны экранировать возвращенное действием значение методом `h()`²⁴³.
- Будьте предельно осторожны при работе со следующими функциями, а еще лучше – вообще избегайте их: `system(command)`, `exec(command)`, `spawn(command)` и `command`²⁴³.
- Не используйте `Kernel#open`. Вместо нее используйте `File.open`, `IO.open` или `URI#open`²⁴³.
- Не конструируйте заголовки ответов на основе полученных от пользователей данных. Если без этого не обойтись, то проверяйте, санируйте или экранируйте вход, а затем кодируйте вывод²⁴³.

- Не отключайте режим `deep_munge`, потому что это может привести к неожиданному поведению базы данных. По умолчанию этот параметр включен: `config.action_dispatch.perform_deep_munge = true`²⁴³.
- Никогда не подавайте полученные от пользователя данные на вход следующих команд. Если вынуждены это делать, то проверяйте данные, санируйте или экранируйте их, затем кодируйте, а после всего этого тщательно тестируйте. Но лучше вообще держаться от них подальше: `Eval()`, `System()`, `ls -al /`, `exec()`, `spawn()`, `open()`, `Process.exec()`, `Process.spawn()`, `IO.binread()`, `IO.binwrite()`, `IO.foreach()`, `IO.popen()`, `IO.read()`, `IO.readlines()`, `IO.write()`²⁴⁴.
- Поля, для которых используются функции `raw`, `html_safe`, `content_tag` и им подобные, – потенциальные мишени XSS. Проверяйте, санируйте, экранируйте, кодируйте, если помещаете в них ненадежные данные. А по возможности вообще избегайте²⁴⁴.
- Не отправляйте транзакции в GET-запросах ни в каком языке, потому что для них нет таких встроенных мер защиты, как для POST-запросов. Например, Rails не предлагает никакой защиты от CSRF в GET-запросах²⁴⁴.
- Будьте очень осторожны при работе с методом `gender` и никогда не передавайте ему ненадежных данных²⁴⁴.
- Не помещайте следующие файлы в систему управления версиями, потому что велика вероятность, что они содержат конфиденциальные данные. Или же шифруйте их перед сохранением: `/config/database.yml`, `/config/initializers/secret_token.rb`, `/db/seeds.rb`, `/db/development.sqlite3`²⁴⁴.
- Подумайте об использовании пакета `Devise`, в котором для хеширования пароля используется функция `bcrypt`, а не стандартное шифрование ОС²⁴⁴.
- Из других свободных инструментов, которые можно использовать для тестирования Ruby-кода, упомянем `Semgrep OSS`, `Bearer`, `Gauntlet` и пакет `dawnscanner`.

КОМПЛАЕНС != БЕЗОПАСНОСТЬ

Раньше Боб думал, что комплаенс – это глупость. По его мнению, *комплаенс != безопасность*. Из-за необходимости соответствовать требованиям регуляторов его группа, оказавшись в новой компании, тратила массу времени на составление документации и создание шаблонов, и поначалу он думал, что это время потрачено зря. Но в один прекрасный день им пришло сообщение о крайне неприятной ошибке в рамках программы вознаграждения за найденные ошибки. Благодаря документации они за 5 мин выяснили, где этот код используется во всех приложениях. Они достали шаблон, исправили в нем ошибку, протестировали, а затем распространили исправление по всем приложениям. Это было *восхитительно*! Боб поверить не мог, как все оказалось легко и эффективно. В его прежней компании это могло бы стать проектом на 6 месяцев для трех человек, а здесь все разрешилось за один, пусть и долгий, день. Хотя с тех пор Боб не стал первейшим агитатором за комплаенс, но его отношение к нему точно улучшилось.

Spring и Spring Boot

Spring, написанный на Java каркас, вытеснил чрезвычайно небезопасный каркас Java Struts. Spring Boot, лидирующий Java-каркас, является расширением оригинального каркаса Spring²⁴⁶. Spring Boot органично интегрируется с экосистемой Spring и предлагает надежные функции, такие как автоматическая конфигурация и автономное развертывание, что позволяет проще и эффективнее создавать масштабируемые приложения производственного уровня, готовые к работе на предприятии. Он поддерживает архитектуру микросервисов, предлагает встроенные средства безопасности и нередко используется для быстрого создания прототипов. В этом разделе мы рассмотрим средства безопасности Spring и Spring Boot, а также подводные камни и свободные инструменты.

ПОЧЕМУ Я ПЛОХО ОТЗЫВАЮСЬ О STRUTS

Java Struts попортил мне немало крови в 2016 и 2017 годах из-за уязвимости в технологии удаленного выполнения кода (RCE), обнаруженной в нескольких версиях каркаса, в то время как мне было поручено запустить программу безопасности приложений в компании, где эксплуатировались сотни, если не тысячи приложений, написанных с применением Struts. Итог – напряженные дни и бессонные ночи. Кроме того, в Struts меньше средств безопасности, чем в Spring и в Spring Boot, и они слабее. Поэтому я настоятельно рекомендую использовать Spring или Spring Boot, а не Struts.

LOG4J

Существует несколько версий Java-библиотеки протоколирования Log4j, где имеются очень серьезные уязвимости (типа RCE), которые можно эксплуатировать, просто добавив текст в конец URL в адресной строке браузера. Злоумышленнику не нужно входить в систему. Не надо писать кучу сложного кода, атакующая строка доступна из любого места в интернете. Атака поразительно проста и смертоносна. Если вы пользуетесь Log4j, то проследите, чтобы версия была не ниже 2.18, а лучше еще выше (на момент написания книги текущей была версия 2.24). Компании по всему миру все еще страдают от этой уязвимости, найденной аж в 2021 году. Не окажитесь среди них!

Какие элементы безопасности использовать

- Используйте каркас Spring Security для аутентификации и управления доступом²⁴⁷.
- Используйте JDBC в качестве системы ORM для более безопасного доступа к базе данных.
- Используйте Spring Session для управления сессиями²⁴⁷.
- Если вы являетесь клиентом Hashicorp Vault, то можете использовать для доступа к нему Spring Vault без какой-либо дополнительной настройки²⁴⁷.
- В состав Spring по умолчанию включена защита от CSRF. Если вы используете какой-нибудь каркас JavaScript для организации интерфейса

и Spring на стороне сервера, то нужно будет настроить `CookieCsrfTokenRepository`, чтобы JavaScript мог читать cookie. Если вы не пользуетесь каркасом JavaScript, то тег `<form:form>` из Spring MVC или аннотация `@EnableWebSecurity` из Thymeleaf автоматически добавляют токен CSRF в скрытое поле²⁴⁸.

- Spring Security также предоставляет ряд заголовков безопасности по умолчанию: `Cache-Control`, `Pragma: no-cache`, `Expires`, `X-Content-Type-Options`, `Strict-Transport-Security`, `X-Frame-Options` и `X-XSS-Protection`²⁴⁸. Вместе с тем вы должны добавить все применимые заголовки, рассмотренные ранее в этой книге.
- Используйте модуль `PasswordEncoder` в сочетании со Spring Security для хеширования паролей²⁴⁸.
- Каркас Spring Security также интегрируется с облачными платформами, например Kubernetes и Cloud Foundry, предоставляя дополнительные средства безопасности, такие как взаимная TLS-аутентификация и взаимная аутентификация служб²⁴⁹.
- Используйте модуль `Spring Boot Actuator` для мониторинга микросервисов и протоколирования событий безопасности²⁴⁹.
- Аннотация `@Secured`, предоставляемая Spring Security, позволяет разработчикам определять роли, необходимые для доступа к конкретному методу²⁴⁹.
- Аннотации `@PreAuthorize` и `@PostAuthorize` позволяют разработчикам определять, какие роли необходимы для доступа к конкретному методу, до и после выполнения метода. `@PreAuthorize` и `@PostAuthorize` можно использовать вместе, чтобы обеспечить безопасность до и после метода²⁴⁹.
- Spring Security предлагает фильтры безопасности, которые можно настраивать (добавить еще более строгие проверки) путем реализации интерфейса `javax.servlet.Filter`²⁵⁰.
- В Spring Security поставщики безопасности – это подключаемые механизмы, которые отвечают за аутентификацию пользователей, управление учетными данными пользователей и организацию аутентифицированного сеанса. Имеются следующие поставщики²⁵⁰:
 - `InMemoryUserDetailsManager`: хранит учетные данные и роли пользователя в памяти (плохо масштабируется);
 - `JdbcUserDetailsManager`: читает учетные данные и роли пользователя из таблицы базы данных;
 - поставщик аутентификации LDAP: аутентифицирует пользователей с помощью LDAP-сервера;
 - поставщик аутентификации OAuth2: аутентифицирует пользователей по протоколу OAuth2, позволяя входить через свои социальные сети или учетную запись в Google;
 - поставщик аутентификации OpenID Connect: аутентифицирует пользователей по протоколу OpenID Connect, слою аутентификации поверх OAuth2;

- анонимный поставщик аутентификации: позволяет неаутентифицированным пользователям получать доступ к определенным ресурсам, не входя в систему.

Чего следует избегать

- Не отключайте средства безопасности, например `.csrf().disable()`; это ошибка²⁴⁹.
- Не используйте аннотацию `@PermitAll`. Так можно раскрыть конечные точки или данные неавторизованному пользователю.

НЕВОЗМОЖНЫЕ ПОЛИТИКИ

Алиса была раздосадована. Она обнаружила политику, показавшуюся ей невозможной. Она была основана на FEDRAMP^{*} и NIST с уровнем гарантий 3. Требовалось, чтобы ее компания могла доказать, что владеет ключом для OpenID Connect, но инженеры сказали ей, что такого ключа у них быть не может, потому что технология работает по-другому. Она даже заставила их прочитать запрос на изменение (RFC), в котором сказано, что сторона не может управлять ключами самостоятельно. Алиса отправилась в группу GRC с твердым намерением спустить на них всех собак. Но сначала она решила прогуляться вокруг здания, чтобы успокоиться и не орать, как ей очень хотелось, а говорить терпеливо и профессионально. Когда она объяснила ситуацию ребятам из GRC, те сказали, что это «требование на перспективу», рассчитанное на то, что в будущем системы будут ему отвечать. Алиса оказалась в затруднении: «Так что же нам делать? Я не могу удовлетворить это требование. Оно *невозможно*». Ей ответили: «Мы изменим эту политику, чтобы она соответствовала возможностям. Не волнуйся, мы все исправим. Спасибо, что сказала». На этом все и закончилось. Алиса готовилась к битве... а оказалось, что нужна всего лишь логика⁹¹!

Flask

Flask – простой, гибкий и легкий микрокаркас на Python – весьма популярный выбор для веб-разработки. А если добавить к нему пакет `flask-security`, то Flask становится еще мощнее! Благодаря своему облегченному и минималистскому дизайну Flask предлагает баланс между простотой и функциональностью, так что разработчик может сосредоточиться на построении приложения, а не на борьбе со сложностью. Обширная экосистема расширений в сочетании с эффективными функциями безопасности делают комбинацию Flask + Flask-Security отличным каркасом для разработки безопасных и масштабируемых веб-приложений. В этом разделе мы рассмотрим его функции безопасности.

Какие элементы безопасности использовать

- Flask конфигурирует Jinja2, так чтобы все значения экранировались автоматически, если явно не указано противное. Но он не может защитить от XSS путем внедрения атрибута. Не помещайте ненадежные данные

^{*} Федеральная программа управления рисками и авторизацией США.

в HTML-атрибуты и обязательно заключайте в двойные кавычки значения, которые все-таки помещаете: `<input value="{{value }}">`²⁵¹.

- Пользуйтесь защитой от CSRF: расширением Flask-WTF с ПО промежуточного уровня CSRFProtect²⁵¹.
- Подумайте об использовании библиотек аутентификации Flask, таких как Flask-Login и Flask-JWT²⁵¹.
- Flask предлагает собственный инструмент управления секретами, Flask-Secrets²⁵¹.
- Как всегда, следует использовать параметризованные запросы при обращении к базе данных, но в Flask можно также использовать метод курсора `execute` для безопасной привязки параметров. Правда, все равно нужно проверять и экранировать или санировать значения перед их вставкой.
- Подумайте об использовании каких-нибудь из следующих библиотек Flask для обеспечения безопасности: Flask-Login (управление пользовательскими сессиями), Flask-Principal (управление идентификацией), Flask-WTF (интеграция Flask и WTForms, включающая CSRF, загрузку файлов и reCAPTCHA), itsdangerous (подписывает данные для проверки целостности), passlib (хеширование пароля), webauthn (упрощает использование WebAuthn) и authlib (для построения серверов OAuth и OpenID Connect: JWS, JWK, JWA, JWT включены)²⁵².
- Двухфакторную аутентификацию можно включить, используя алгоритм `totp`, одноразовый пароль на основе времени²⁵².
- WebAuthn соединяет аутентификаторы (например, YubiKey и мобильную биометрию) с сайтами. Flask-Security поддерживает использование ключей WebAuthn в качестве основного или вспомогательного аутентификатора²⁵².
- Flask-Security также предлагает подтверждение электронной почты, восстановление пароля, проверку электронной почты, регистрацию пользователя, изменение пароля и отслеживание входов в систему – все «из коробки»²⁵².
- Flask-Security предоставляет декораторы аутентификации:
 - `auth_required()`;
 - `http_auth_required()`;
 - `auth_token_required()`.
- Также Flask-Security предоставляет декораторы авторизации:
 - `roles_required()`;
 - `roles_accepted()`;
 - `permissions_required()`;
 - `permissions_accepted()`.
- Декораторы авторизации и аутентификации простые и вряд ли удовлетворят всем вашим требованиям; по-видимому, вам придется использо-

вать специальный класс исключения, которое можно будет возбудить в ответ на ошибки авторизации Flask-Security или в своем собственном коде²⁵².

- По умолчанию конечные точки Flask-Security, требующие, чтобы вызывающая сторона была аутентифицирована, *не* поддерживают Basic Auth²⁵².
- Flask-Security поддерживает запрос «свежей», или недавней, аутентификации посредством метода `auth_required()` (его параметры определяют промежуток времени, в течение которого аутентификация считается «свежей»), метода `Security.reauthn_handler()` (который вызывается, когда запрос не прошел проверку на «свежесть» аутентификации) и конечных точек, которые запрашивают повторную аутентификацию пользователя (`SECURITY_VERIFY_URL`, `SECURITY_US_VERIFY_URL`, `SECURITY_WAN_VERIFY_URL`). Некоторые конечные точки Flask по умолчанию (`.wan_register`, `.wan_delete`, `.tf_setup`, `.us_setup` и `.mf_recovery_codes`) также можно снабдить проверкой на «свежесть» с помощью конфигурационных переменных `SECURITY_FRESHNESS` и `SECURITY_FRESHNESS_GRACE_PERIOD`²⁵².
- Flask-Security имеет валидатор пароля по умолчанию, который²⁵²:
 - проверяет минимальную и максимальную длину (минимальная задается с помощью конфигурационной переменной `SECURITY_PASSWORD_LENGTH_MIN`);
 - если переменная `SECURITY_PASSWORD_CHECK_BREACHED` установлена, то используется API для проверки нахождения пароля в списке взломанных. Конфигурационную переменную `SECURITY_PASSWORD_BREACHED_COUNT` можно использовать для задания минимального числа «взломов». Спасибо Трою Ханту и его команде, которые подарили нам эту возможность.
- Проверка паролей и средства Юникод-нормализации предлагает класс `PasswordUtil`²⁵².
- Flask-Security по умолчанию устанавливает конфигурационную переменную `SECURITY_RETURN_GENERIC_RESPONSES` в `True`, это гарантирует, что отправляются обобщенные ответы, не позволяющие перечислить пользователей²⁵².
- Установите `SECURITY_CONFIRMABLE` в `True`, чтобы избежать перечисления адресов электронной почты²⁵².
- Используйте библиотеку `WTForms` для создания объектов форм с определенными полями и правилами проверки, так чтобы проверка входных данных была встроена прямо в форму²⁵².
- Для санирования HTML-вывода в приложении Flask можно воспользоваться библиотекой `Bleach`²⁵³.

- Используйте `flask.escape()` для экранирования опасных символов²⁵⁴.
- Расширение `Flask-SSLify` делает HTTPS обязательным²⁵⁵.
- Подумайте об использовании расширения `Flask-JWT-Extended` для аутентификации на основе токенов²⁵⁵.
- Подумайте об использовании `Flask-Limiter` для ограничения темпа, чтобы защититься от злонамеренного использования, DDoS-атак и чрезмерного использования API²⁵⁵.
- Если вы разрешаете загружать файлы на сервер, то, как и в любом другом языке или каркасе, не используйте имя файла, заданное пользователем, а выбирайте свое собственное. Если по какой-то причине вы вынуждены использовать имя, заданное пользователем, то присмотритесь к функции `secure_filename()` из библиотеки `Werkzeug`, которая старается предотвратить уязвимость типа «обход каталогов».

Чего следует избегать

- Не используйте аннотацию `@login_required`, если планируете использовать `Flask-Security`; они конфликтуют²⁵².
- Не помещайте полученные от пользователя данные в параметр `next`, потому что это может привести к уязвимости типа «открытое перенаправление» (`open redirect`) для следующих конечных точек: `.login`, `.logout`, `.register`, `.verify`, `.two_factor_token_validation`, `.wan_verify_response`, `.wan_signin_response`, `.us_signin` и `us_verify`²⁵².
- Если в вашем приложении используется `WebAuthn`, то установите переменную `SECURITY_WAN_ALLOW_USER_HINTS` в `False`, чтобы избежать перечисления имен пользователей и адресов электронной почты²⁵².
- При использовании `Jinja2` не используйте фильтр `|safe` или класс `Markup` без острой необходимости, потому что это может сделать приложение уязвимым к XSS вследствие пометки строки как безопасного HTML-кода²⁵⁵.
- Не генерируйте HTML-шаблоны в обход `Jinja2`.

НЕ ПЕРЕСТАВАЙТЕ УЧИТЬСЯ

Боб довольно рано в своей карьере понял, что, если хочешь достичь совершенства в любой технологии, нужно полюбить учиться; никакого обходного пути нет. И члены группы GRC, и его замечательные программисты, и ребята из группы безопасности приложений, и системные администраторы – все они, похоже, должны были учиться постоянно. Он решил ежеквартально проводить сверку часов со своей командой, чтобы определить цели обучения, и сообщил об этом своему начальнику. Каждый квартал они изучали бы что-то новое, будь то технические предметы или коммуникативные навыки. Всегда есть способ стать лучше, и Боб хотел, чтобы его команда стала самой лучшей.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Если вы используете один из описанных в этой главе каркасов на работе, то подготовьте презентацию, краткое изложение или руководство по его использованию и поделитесь им со своими коллегами. Они будут вам благодарны!
2. Попробуйте использовать в повседневной работе одну или несколько новых для вас функций, методов, библиотек или расширений. Возможно, с помощью одного из этих новых ресурсов вы найдете более элегантное решение.
3. Используйте эту главу в качестве справочного материала, когда в следующий раз будете программировать с использованием одного из этих каркасов. И продолжайте так поступать до тех пор, пока вам больше не понадобится заглядывать в книгу, потому что вы уже и так знаете.

Категории уязвимостей

Первой уязвимостью, о которой я узнала во всех подробностях, было внедрение SQL-кода. Я была убита наповал идеей о том, что кто-то может заставить мое приложение делать то, что он хочет, а не то, что я ему приказываю. Как разработчик программного обеспечения, я чувствовала себя очень могущественной. Я думала, что могу решить проблемы практически любого человека, если проявлю достаточно изобретательности. Узнав, что кто-то другой может воспользоваться благами моего кода и, как следствие, мной и моей командой, я призадумалась.

С тех пор я начала изучать «хаки»: ситуации, когда случался инцидент безопасности, или кража данных, или еще что-то плохое с точки зрения безопасности. Каждый случай казался уникальным моему необученному и нетренированному уму. Но со временем я поняла, что уязвимости можно классифицировать по типам и что можно защищаться от целых категорий уязвимостей, что гораздо эффективнее. Создавать свою меру защиты от каждой уязвимости – все равно что сражаться с гидрой: все головы все равно не отрубишь. Но если разворачивать средства защиты от целых категорий, то можно одним махом предотвратить много конкретных уязвимостей.

Часто люди упоминают в разговоре широко известный документ «The OWASP Top 10», посвященный главным рискам веб-приложений. Многие думают, что эти 10 уязвимостей – единственное, о чем стоит беспокоиться, а потому планируют оборону не самым лучшим образом.

Поскольку большинство людей уже слышали о Топ-10 OWASP и я рассказывала об этом в своей предыдущей книге, я решила, что на этот раз расскажу об уязвимостях совершенно по-новому. Я хотела подойти нестандартно и помочь вам поступить так же. Вам не нужно запоминать каждое название или тип уязвимости, нужно лишь составить общее представление о принципах их работы, о рисках для ваших систем и данных и о мерах по смягчению последствий. Не нужно запоминать номера CVE или CWE (это идентификаторы уязвимостей); как разработчик программного обеспечения, вы должны понимать основные опасности, с которыми сталкиваются системы, и как максимально эффективно защитить от них свои приложения.

CVE И CWE

CVE означает Common Vulnerability Enumerator (Единый классификатор уязвимостей), а CWE – Common Weakness Enumerator (Единый классификатор слабостей). CVE – это дефекты безопасности в операционных системах, пакетах обновлений, библиотеках и программах – как коммерческих, например продуктах Microsoft или Oracle, так и с открытым исходным кодом, например ядре Linux, ОС FreeBSD OS и OpenSSH. CVE описывает дефекты в стандартных программных продуктах, а CWE – типы уязвимостей, встречающихся в *заказном программном обеспечении* или оборудовании, которое обычно создается внутри организации. Например, CVE-2021-44832 – один из номеров CVE, присвоенных уязвимости Log4j 2021 года, которая встречалась в нескольких версиях библиотеки протоколирования Log4j для Java²⁶⁸. С другой стороны, внедрение SQL-кода – категория уязвимостей, имеющая код CWE-89: «Неправильная нейтрализация специальных элементов в SQL-команде» («Внедрение SQL-кода»)²⁶⁹. Здесь я не следую обозначениям CWE, потому что за этим вы можете просто заглянуть на сайт, если захотите, но взяла на вооружение идею объяснения классов (как в CWE), а не конкретных примеров уязвимостей (как в CVE).

Примечание CWE – общие классы уязвимостей, а не конкретные примеры уязвимостей, как CVE.

Просчеты проектирования (логические просчеты)

Дефект (bug) – ошибка в способе реализации решения; это проблема в вашем *коде*. *Просчет* (flaw) – это ошибка проектирования, проблема в проекте системы. Если в вашем проекте имеется логический просчет, то, даже если код написан идеально, конечный продукт все равно будет несовершенным. И обойти это никак нельзя.

Когда специалисты по безопасности говорят об уязвимостях бизнес-логики, логических просчетах или просчетах проектирования, имеется в виду проблема в проекте системы с точки зрения безопасности, из-за которой возможно непредусмотренное поведение пользователей. В общем случае непредусмотренное поведение потенциально опасно, а иначе группе безопасности не о чем было бы беспокоиться.

Приведем несколько примеров логических просчетов.

- Система позволяет одному пользователю манипулировать учетной записью другого пользователя, например загружать данные, удалять записи или изменять настройки, к которым он не должен иметь доступа.
- Система позволяет пользователю создать учетную запись с чужим адресом электронной почты (к которому он не имеет доступа). Именно поэтому мы всегда подтверждаем, что пользователь, указавший некоторый почтовый адрес, действительно владеет им.

- Система позволяет пользователю пропустить части транзакции, например купить что-то, не заплатив, или купить несколько предметов, а заплатить за один.
- Система позволяет пользователю просматривать конфиденциальную информацию, которую он видеть не должен, например данные других пользователей, системные данные или внутренние данные компании.
- Система позволяет пользователю «пожирать» ресурсы, так что другие пользователи не могут получить доступ к системе; например, задавать такие критерии поиска, что будет найдено слишком много ресурсов или система заикнется.
- Система некорректно протоколирует действия пользователя или позволяет пользователю удалять сведения о своих прошлых действиях, так что вы не сможете узнать, что пользователь делал, находясь в системе.

Как такое возможно?

Логические просчеты случаются, когда разработчики и проектировщики делают неверные предположения о том, как пользователи будут взаимодействовать с системой²⁵⁶. Эти ошибки ведут к неадекватной проверке данных, введенных пользователем, и принятых пользователем решений. Без достаточной тщательной проверки наши пользователи могут сойти с предписанного им пути и выполнить в системе непредвиденные действия.

Логические просчеты также возможны, когда мы забываем поставить элементы безопасности туда, где они должны быть. Если в нужном месте отсутствует проверка ввода, кодирование вывода, аутентификация, управление доступом, шифрование и т. д., то, скорее всего, имеет место какой-то просчет.

Риск

Логические просчеты могут приводить как к малозначительным недостаткам, так и к критическим уязвимостям. Вы, наверное, слышали шутку: «Это не ошибка, а недокументированная особенность»; так говорят, когда хотят отвести проблему, найденную тестировщиком, обычно потому, что не понимают, какой вред из нее может проистечь. Я предложила бы исправлять все логические просчеты, даже если вы не видите, как их можно эксплуатировать. Из того, что вашего воображения не хватает, чтобы найти способ использовать просчет во зло, вовсе не следует, что и никто другой на это не способен.

Иногда риски логических просчетов очень велики. Если просчет связан с элементом безопасности, например аутентификацией, и в результате аутентификация может быть использована неправильно, то последствия могут быть ужасающими²⁵⁶.

Прошу вас, предполагайте, что серьезность логических просчетов в большинстве случаев лежит в диапазоне от высокой до критической. В конкретном случае можете ее понизить, если сочтете необходимым.

Предотвращение

Не существует автоматизированных инструментов, способных надежно выявлять просчеты в бизнес-логике. Иногда что-то они находят, но полагаться ни на один известный мне инструмент я бы не стала. Это значит, что нужна творческая ручная работа, которая, как правило, доставляет удовольствие.

Существует несколько способов поиска и предотвращения логических просчетов: выполнить моделирование угроз, создать пользовательские истории о безопасности, произвести критический анализ проекта вместе со специалистом по безопасности и убедиться, что все элементы безопасности на месте, и, наконец, в процессе проектирования без особенного старания делать предложения о том, как пользователи будут себя вести (все равно они почти всегда удивят вас). Старайтесь применять столько описанных в этой книге концепций безопасного проектирования, сколько сможете, ко всем своим проектам, что заодно поможет избежать просчетов в приложениях. Наконец, наймите опытного пентестера, покажите ему, как должно работать приложение, а затем посмотрите, что он найдет. Пентестеры могут быть чертовски изобретательны (следуя путями зла).

Примечание Принимая решения о крупных покупках, хорошо бы прислушаться к совету, только совет должен быть беспристрастным. Если вы захотите, скажем, купить аналитические материалы у Гартнера, то имейте в виду, что многие поставщики платят за то, чтобы их продукты подвергли анализу и включили в списки и отчеты Гартнера, например в их знаменитый магический квадрант. Когда на кону сотни тысяч, а то и миллионы долларов, без пристрастности не обходится. Компании типа IANS Research не принимают платежей от поставщиков за упоминание в их материалах или рекомендациях и считают частью своей миссии предлагать беспристрастные технические рекомендации. Вместе с тем хочу признаться, что я *сама* работала (на полставки) в IANS, поэтому пристрастна! Так или иначе, старайтесь по возможности получать советы из нескольких источников при принятии важных решений, таких как покупка инструмента, определение стратегии и прием на работу. Кроме того, если советчик может быть пристрастным, постарайтесь учесть это в процедуре принятия решения. Это относится и к моим советам!

Дефекты в коде (ошибки реализации)

Дефект – это ошибка, допущенная при написании кода, например орфографическая или грамматическая. Каждый пишущий код время от времени допускает дефекты. Не нарочно, но бывает! Дефект безопасности – это тоже дефект, только имеющий последствия для безопасности. Обычно это озна-

чает негативные последствия для доступности, конфиденциальности или целостности системы либо ее данных. Иными словами, уязвимость.

А быть может, программист забыл поставить проверку туда, где ей место, или неправильно написал код проверки: например, составил черный список недопустимых символов и попытался заблокировать их, вместо того чтобы составить белый список допустимых символов и разрешать только их. Злоумышленники часто обходят такие меры, что ведет к дефекту безопасности.

Как такое возможно?

Люди допускают ошибки. Знаю, я и сама грешна. Вот так дефекты и появляются. Мы бываем забывчивы, устаем, торопимся или просто не понимаем, что посадили ошибку. «Человеку свойственно ошибаться» – отсюда и дефекты. Часто дефекты возникают, если мы не применяем для их поиска несколько видов тестирования. Перед отправкой почтового сообщения мы подвергаем его орфографической и грамматической проверке – так и свою работу должны проверять дважды и трижды.

Риск

Риск, связанный с дефектом безопасности, может быть как очень мал, так и очень велик. Примерами могут служить внедрение кода, переполнения, небезопасная прямая ссылка на объект (*insecure direct object reference* – *IDOR*), межсайтовая подделка запросов (*CSRF*) и т. д.

Не все дефекты безопасности отличаются высоким риском; некоторые даже исправлять не стоит. Обсуждайте риски с группой безопасности, пока не наберетесь достаточно опыта, чтобы выносить собственное суждение.

Предотвращение

Для поиска дефектов мы должны тестировать, а в этой области автоматизированное тестирование не имеет равных. Применение инструментов статического анализа (*SAST*) поможет найти дефекты в коде, написанном вашей командой, а инструмент анализа состава программы (*SCA*) – в коде, который писали не вы (сторонних зависимостях). Я рекомендую сканировать весь код инструментами обоих типов, а также инструментом динамического анализа (*DAST*) или интерактивным (*IAST*), если у вас такой имеется, а после этого исправить все, что группа безопасности сочтет важным, прежде чем передавать код в эксплуатацию. Настраивайте регулярное автоматизированное сканирование, если лицензия позволяет.

Ручной анализ кода экспертом также может оказаться огромным подспорьем, как и тест на проникновение. Но если у вас нет для этого ресурсов (экспертов или денег, чтобы нанять экспертов), то автоматизированное тестиро-

вание может проделать очень большую работу, особенно если использовать несколько инструментов. Ваша группа безопасности поможет решить, какой уверенности «достаточно» в каждом конкретном случае.

Совет В главах 6 и 7 я рассказывала о нескольких свободных статических инструментах, которые вы можете «натравить» на свой код. Если ваш бюджет не позволяет покупать программы для сканирования кода, то все равно к вашим услугам есть много инструментов (да еще и разных типов!).

Совет Помните, что, даже если автоматический инструмент что-то нашел, исправлять это не всегда обязательно. Автоматические инструменты могут найти сотни и даже тысячи дефектов, и если ваша компания только и будет заниматься, что их исправлением (вместо создания новых функций и продуктов), то, скорее всего, обанкротится. Мы исправляем ошибки, которые влекут за собой неприемлемый риск для бизнеса, – чтобы все оставались в безопасности. Пообщайтесь с группой безопасности, и пусть они решают, на что направить силы, а какие риски принять.

Переполнения и другие проблемы при работе с памятью

Если ваша программа написана на языке программирования низкого уровня, например С или С++, то вы вынуждены работать с памятью напрямую. Вы должны следить за тем, чтобы выделить достаточно памяти и освободить ее по завершении использования, иначе вас поджидают большие неприятности. Вы должны быть уверены, что не перезаписали какую-то часть программы, стек, кучу и много другое; управление памятью подразумевает множество вещей. Хотя утечки памяти могут вызвать крах программы, а значит, ее недоступность, наиболее широко известные проблемы безопасности, связанные с памятью, – *переполнения*.

Рассмотрим несколько типов проблем при работе с памятью и начнем с переполнений.

Переполнения

Переполнение буфера возникает, когда программа пытается записать в буфер больше данных, чем помещается в выделенную для него память, т. е. когда данные записываются за концом выделенной памяти²⁵⁷. Обычно это приводит к беде. Представьте, что для строки было отведено 20 символов, пользователь ввел 25, а программа не проверила длину входных данных – вот вам и переполнение. Запись за границей блока выделенной памяти может привести к повреждению данных, краху программы или выполнению вредоносного кода²⁵⁷.

Примечание Под «выполнением вредоносного кода» понимается тот факт, что программа сошла с предначертанного пути и оказалась в области неопределенного поведения – поведения, которое злоумышленник пытается взять под контроль. Вредоносный код внедряется в приложение злоумышленником. С точки зрения безопасности приложения это конец света.

Если злоумышленник знает, что есть возможность перезаписать буфер, то он может экспериментально отыскать способ перезаписать указатель стека (который говорит компьютеру, какую команду исполнять следующей). А перезаписав указатель стека, он может заполнить буфер собственным шелл-кодом (эксплойт, который обычно пишется на машинном языке и несет полезную нагрузку, составляющую часть атакующего кода). Если вы подумали: «Какой ужас!» – значит, начинаете понимать, почему безопасность так много значит для нас, пекущихся о безопасности приложений. Переполнение может случиться с буферами, строками, целыми числами и т. д. Если вы вручную выделяете память для чего-то, то теоретически возможно переполнение.

Чтение за границей буфера

Чтение невыделенной памяти может привести к раскрытию конфиденциальных данных. Такое случается, когда программист не проверил входной запрос или ответ на него, из-за манипуляций с указателями или вследствие других ошибок управления памятью²⁵⁸. В результате злоумышленник может использовать эту информацию для обхода рандомизации распределения адресного пространства – лучшей из всех мер защиты от переполнений на уровне операционной системы²⁵⁸.

Страничные отказы

Адресное пространство – это область адресов, соответствующая сетевому хосту, периферийному устройству, сектору диска, ячейке памяти или еще какому-то компьютерному объекту²⁵⁹. Если ваша программа пытается перейти по адресу, не входящему в состав адресного пространства, значит в памяти нет соответствующей этому адресу страницы и, следовательно, программе некуда идти. Это называется *страничным отказом*²⁵⁸. Это один из многих способов, которыми приложение может попасть в неопределенное состояние, – чего вы точно не хотите.

Когда такое происходит, операционная система обычно доставляет сигнал «ошибка сегментации» проблемному процессу, что приводит к его аварийному завершению²⁵⁸. Операционную систему можно настроить так, что попытки чтения или записи памяти, адресуемой нулевым указателем, приводят к страничному отказу, чтобы избежать чтения недоступной процессу памяти,

но это с большой вероятностью все равно приведет к краху, возврату кода ошибки или еще какому-то нежелательному результату. Это уменьшение ущерба, а не решение*.

Использование после освобождения

Удалив объект, вы, по сути дела, затерли область памяти, которую он занимал²⁵⁸. Если впоследствии вы попытаетесь обратиться по адресу этой области, чтобы получить доступ к уже удаленному объекту, то возможны проблемы. Эта ситуация называется *использованием после освобождения*. В результате переменной может быть присвоен неправильно сконструированный объект, что приведет к краху приложения или еще какому-нибудь печальному исходу.

Неинициализированные переменные

В языках, небезопасных относительно памяти (например, С и С++), вы должны инициализировать переменные (присваивать им значение) перед использованием. Попытка прибавить 2 к неинициализированному целому числу ведет к хаосу.

Попытка перейти по неинициализированному адресу может привести к краху программы или неопределенному поведению. Это называется *разыменованием нулевого указателя*[†] (попытка перехода по несуществующему адресу)²⁵⁸.

Неинициализированный указатель (которому не было присвоено никакого значения), называется *недействительным*²⁵⁸. *Висячим* называется указатель, направленный на уже освобожденную область памяти (например, объект). Такой указатель указывает в никуда. Обе эти ситуации могут стать причиной недоступности, краха и других потенциальных проблем.

Утечки памяти

Утечки памяти происходят, когда использование памяти отслеживается некорректно или не отслеживается вовсе²⁵⁸. Итогом может стать исчерпание стека или кучи, двойное освобождение (т. е. освобождение уже освобожденной памяти), некорректное освобождение (передача функции освобождения некорректного адреса, что ведет к повреждению кучи), несогласованное ос-

* Это не ошибка перевода, автор действительно таким странным образом пытается описать различие между ошибкой адресации и механизмом виртуальной памяти. – *Прим. перев.*

† Только в случае, когда «неинициализированный адрес» на самом деле нулевой. Такие попытки перехватываются операционной системой. Попытка перехода по неинициализированному адресу, содержащему произвольное значение, может окончиться куда хуже. С терминологией (а возможно, и с пониманием сути явления) у автора просто беда. – *Прим. перев.*

вобождение (использование для освобождения памяти функции, не соответствующей функции, которая выделила эту память) или нежелательное наложение имен (выделение одной и той же памяти дважды, что может привести к путанице или недопустимым значениям)²⁵⁸. Проще говоря, любая утечка памяти может привести к краху системы, неправильным результатам, переходу приложения в неизвестное состояние – и все это ужасно с точки зрения качества, надежности и безопасности.

По возможности мы хотим избежать всех этих проблем.

ЯВЛЯЮТСЯ ЛИ СОСТОЯНИЯ ГОНКИ ПРОБЛЕМАМИ ПАМЯТИ?

Некоторые считают, что состояние гонки также следует относить к проблемам памяти²⁵⁸. Я же отвела им отдельный раздел. Даже если их считать проблемой памяти, решение у этой проблемы другое, а значит, и рассматривать ее следует отдельно.

Как такое возможно?

Проблемы при работе с памятью возникают, если мы используем язык, небезопасный относительно памяти, и управляем памятью неправильно. Использование языка, небезопасного относительно памяти, означает, что нужно делать больше работы. Ручной работы. И если делать ее неаккуратно, а тестировать небрежно, то возможны ошибки. Это встречается сплошь и рядом.

Риск

Атакующие нацелены на поиск проблем при работе с памятью, а мы, защитники, должны относиться к ним очень серьезно. Начиная с вышедшей в 1996 году статьи «Smashing the Stack for Fun and Profit» за авторством Aleph One, в которой было описано, как использовать переполнение буфера для атаки на программу, злоумышленники неизменно атакуют память. Помимо кражи потенциально конфиденциальных данных, краха системы или перехода в неизвестное состояние (что может не иметь никаких последствий, а может иметь критические), переполнение несет угрозу удаленного выполнения кода (remote code execution – RCE), а это критический риск для любой программы. RCE – самый страшный риск, когда речь заходит о безопасности приложений, поскольку удаленное управление вашими системами злоумышленником является наиболее опасным из возможных последствий. Относитесь к уязвимостям памяти очень серьезно, устраняйте их быстро.

Предотвращение

Лучшее решение проблемы – использовать безопасный относительно памяти язык. Если у вас появится возможность перевести проект с C/C++ или лю-

бого другого небезопасного языка на Rust или еще какой-нибудь безопасный язык, не пренебрегайте ей. Приложите все усилия и не стесняйтесь цитировать эту книгу, если потребуется. Вы избавите себя от огромного количества рисков и работы.

Совет Безопасные относительно памяти языки – это чудо! Они автоматически занимаются сборкой мусора; автоматически выделяют и освобождают память для переменных, объектов и вообще всего, что вы создаете; делают множество полезных вещей, чтобы почти наверняка избежать переполнений (редкие исключения бывают, и они обычно попадают в новостные ленты). Им всегда следует отдавать предпочтение перед языками, небезопасными относительно памяти, если есть выбор.

Но если вы не можете убедить властей предрежащих в необходимости перейти на язык, безопасный относительно памяти, надежда для вас и ваших приложений все же не потеряна.

- Убедитесь, что на всех серверах включена рандомизация распределения адресного пространства (ASLR), защита от выполнения данных (DEP) и защита от перезаписи при структурной обработке исключений (Structured Exception Handling Overwrite Protection – SEHOP)²⁶⁰; эти меры защищают от атак на переполнения.
- Мониторьте использование памяти, это поможет обнаружить утечки. Настройте уведомление, если потребление памяти резко увеличилось или объем занятой памяти медленно, но неуклонно растет.
- Используйте более безопасные функции (например, `strncpy()` вместо `strcpy()`, `strncat()` вместо `strcat()`), некоторые из них описаны в разделе главы 5, посвященном C/C++.
- Используйте библиотеки для повышения безопасности кода, например `Safe C String`, `SafeStr` и `Safestringlib`²⁶¹.
- Запишитесь на курсы по безопасному кодированию, ориентированные на проблемы при работе с памятью и (или) выбранный вами язык.
- Выполняйте анализ кода вручную и с помощью инструмента статического анализа (SAST).
- Используйте средства компилятора, например `StackShield`, `StackGuard` и `Libsafe`²⁶¹.
- Организуйте тест на проникновение или просканируйте приложение динамическим фаззером.
- Рассматривайте предупреждения компилятора как ошибки. Исправьте их. Не игнорируйте только потому, что код компилируется. Предупреждения – это тоже плохо²⁶¹.
- Вовремя применяйте исправления к серверам и регулярно укрепляйте их.
- Выполняйте прочие рекомендации, приведенные в этой книге.

ВНЕДРЕНИЕ КОДА ОБХОДИТСЯ ДОРОГО

Алиса изучала бюджет своего отдела, потому что замещала директора по информационным технологиям на время его отпуска. По одной статье – инцидент безопасности № 67 – затраты были *огромны*. Алиса сразу же пошла к начальнику отдела информационной безопасности, чтобы узнать, что случилось. Тот объяснил ей, что имел место инцидент с внедрением кода, приведший к краже данных; с компании потребовали выкуп (она ничего не заплатила, но на юристов *пришлось* потратиться...), затем она была вынуждена заплатить гигантский штраф и как раз сейчас готовит сообщение для рассылки пользователям. Алиса была *шокирована*. Затраты уже превысили стоимость ее дома (а у Алисы был хороший дом). И инцидент еще не был исчерпан! А еще репутационный ущерб, потеря клиентов и многое другое. Ничего себе, подумала Алиса, надо нам лучше работать над предотвращением таких вещей.

Внедрение кода: проблемы интерпретатора и компилятора

Внедрение – это когда злоумышленник обманом заставляет систему выполнить свой код, *как будто он является частью системы*. Система думает, что введенные им в поле данных значения – не данные, а код. Ваша система обращается с данными иначе, чем с кодом, но злоумышленник обвел ваши приложения вокруг пальца, так что меры защиты не сработали. Затем система интерпретирует или компилирует вредоносный код вместе с вашим и делает то, что он ей велит. Атакуя ваши системы и данные!

Как такое возможно?

Внедрение кода оказывается возможным, потому что система ошибочно принимает полученные от пользователя данные за часть своего кода. Так происходит, потому что мы халатно отнеслись к проверке, санированию или экранированию входных данных и не воспользовались слоями абстрагирования (ORM или параметризованными запросами для защиты базы данных, WAF или RASP для защиты веб-приложения и т. д.), чтобы защитить свою систему. Мы не выучили уроки безопасного кодирования, представленные ранее в этой книге, и недостаточно проанализировали и протестировали свой код. Короче говоря, сработали плохо.

Риск

Внедрение кода стояло на первом месте в списке 10 главных уязвимостей OWASP на протяжении 20 лет⁷², т. е. было самой главной угрозой веб-приложениям, по мнению экспертов. И так продолжалось до 2021 года, когда

оно опустилось на третье место. Если злоумышленнику удастся заставить наше приложение принять его код за часть приложения, а затем откомпилировать или интерпретировать его, то он может нанести ущерб нашим пользователям, базам данных, операционным системам, серверам и т. д. Все системы, к которым наши приложения имеют доступ, становятся потенциальными жертвами, а вредоносный код получает такие же права, какие есть у нашего приложения. Это ужасно, и результаты такой атаки могут быть ошеломляющими.

Внедрение может привести к удаленному выполнению кода, утечке конфиденциальных данных, нарушению целостности данных, недоступности системы, нанесению репутационного урона организации, отказу от обслуживания и т. д. Атака ограничена только воображением злоумышленника и правами, имеющимися у приложения. Это пример того, каким полезным для системы может оказаться применение принципа наименьших привилегий.

Предотвращение

Если вы будете следовать всем рекомендациям, приведенным в этой книге, то беспокоиться о внедрении кода вам никогда не придется. Но жизнь не идеальна; в нашей работе есть много вещей, которые мы не контролируем, а иногда мы совершаем ошибки. Давайте еще раз поговорим о том, как избежать внедрения кода, потому что держаться от него подальше крайне важно.

1. Сопоставляйте все поступающие в систему данные с белым списком. Проверку производите на надежной системе. Отклоняйте все, что не проходит проверку.
 - a. Если в системе используется несколько кодировок, приведите их все к одной (UTF8). Приводите значения к канонической форме. Делайте это до проверки и всех остальных шагов.
2. Санируйте или экранируйте специальные символы, которые не хотите принимать.
3. Используйте слои абстрагирования, чтобы еще ограничить возможность внедрения.
 - b. Используйте систему ORM для защиты базы данных.
 - c. Используйте параметризованные запросы для защиты базы данных.
 - d. Используйте фразу LIMIT и другие средства SQL, чтобы предотвратить массовое раскрытие записей⁷².
 - e. Установите перед приложением или API брандмауэр веб-приложений (WAF) или систему самозащиты приложения во время выполнения (RASP) для дополнительной защиты.
 - f. Используйте шлюз API для защиты своих API.
4. Выполняйте анализ кода (это может быть статический анализ).
5. Тестируйте (тестирование на проникновение, фаззинг и (или) автоматизированный динамический анализ).

Обнаружив возможность внедрения кода в свои системы, незамедлительно исправьте ошибку. Не стоит недооценивать эту опасность.

СЛЕПОЕ ВНЕДРЕНИЕ SQL-КОДА

У Боба когда-то было приложение с уязвимостью, которая называется «слепым внедрением SQL-кода». Он не сразу понял, что пентестер пытается втолковать ему и его команде. Поначалу Боб видел только, что приложение выдает кучу ошибок. Ну и что? Когда пентестер объяснил, что использует атаку на SQL для задания вопросов, требующих ответа «да» или «нет», и что ошибка означает «нет», а нормальный ответ – «да», команда поняла, что дело неладно. А затем пентестер добавил, что спрашивал у базы данных: «Верно ли, что первый символ 'а'? Нет? А верно ли, что это 'b'?» и т. д., пока наконец не получил имя таблицы, имя первого поля и его содержимое. Этого оказалось более чем достаточно, чтобы Боб понял, что должен исправить ошибку немедленно!

Проблемы с входными данными

Если бы все поступающие в систему входные данные идеально проверялись, потенциально опасные символы санировались или экранировались, и все вредоносные данные отвергались, то держу пари, что примерно половина всех программных уязвимостей исчезла бы в одночасье. Исчезла бы значительная часть уязвимостей, перечисленных в единых классификаторах слабостей (CWE).

Когда пользователь – случайно или намеренно – вводит неожиданные данные в систему, она иногда начинает вести себя не так, как задумано. Это может означать отказ от обслуживания, нарушение целостности данных, раскрытие конфиденциальных данных, вандализм, непредусмотренный доступ, крах системы, проблемы с законодательством или требованиями регуляторов и прочие последствия перехода в неизвестное состояние.

Как такое возможно?

Разработчики иногда неправильно понимают, что является входными данными для системы, забывают проверять данные или делают это неправильно. Параметры в URL-адресах, скрытые поля, данные из базы, которые могут быть изменены пользователем, данные от API или из другой системы, которую вы не контролируете, и т. д. – все это входные данные, которые иногда не проверяются. Любая проверка входных данных должна производиться в надежной системе, т. е. на сервере, а не во фронтальном JavaScript-коде. Проверять следует по белому списку (а не черному списку плохих символов). К сожалению, разработчики по всему миру снова и снова делают эти ошибки, и именно поэтому у меня до сих пор есть работа.

Риск

Проблемы с входными данными – причина или первый шаг некоторых особо опасных уязвимостей, включая переполнение буфера, внедрение кода, межсайтовый скриптинг (XSS) и др. Одни проблемы безвредны, другие представляют критические риски.

Предотвращение

Идентифицируйте и документируйте все типы входных данных, а затем попросите коллег проверить, не упустили ли вы чего-то. Пройдитесь с ними по всей системе, если есть такая возможность. Составив список, проверьте все пункты – выполняете ли вы проверку, санирование или экранирование данных. Сделав это, попросите коллегу перепроверить вашу работу или выполните статический анализ (а лучше то и другое!). Затем запустите автоматизированный фаззер (обычно является частью веб-прокси или комплекта инструментов DAST) для каждого вида входных данных, к которому можно применить фаззинг, и исправьте все найденные им проблемы. Если бюджет и время позволяют, проведите тестирование на проникновение.

Проблемы аутентификации и идентификации

Под *сетевым идентификатором* понимается уникальный идентификатор или учетные данные, ассоциированные с устройством, пользователем или программой в сети²⁶². Аутентификация – это процесс, с помощью которого компьютеры идентифицируют устройства и пользователей.

Если аутентификация или идентификация содержат ошибки, то пользователи могут получить доступ к данным или системам, к которым у них не должно быть доступа.

Как такое возможно?

Уверена, что, прочитав предыдущие главы, вы понимаете, каким сложным может быть проектирование и кодирование высококачественной системы аутентификации или идентификации. Ко всему прочему, эти системы ежеминутно подвергаются злоумышленниками испытаниям на прочность и устойчивость к атакам полным перебором. Для людей с дурными намерениями эта цель очень привлекательна, ибо является ключом к нашему цифровому

королевству. Проблемы такого рода возникают, потому что системы плохо спроектированы, реализованы и (или) протестированы, так как невероятно сложны и запутаны.

Риск

Доступ со стороны злоумышленника к конфиденциальным данным, системам, привилегиям и т. д. может создать критические риски – в зависимости от системы. В общем случае такой риск для безопасности всегда следует считать высоким.

Предотвращение

Прежде всего если имеется возможность, то покупайте систему, которая уже выполняет все эти функции, а не пишите свои собственные. Если это невозможно, то следуйте советам, приведенным в главе 2 этой книги. Выполняйте анализ и тестирование кода, кода покупаете систему или пишете свою собственную.

ОПРЕДЕЛЕНИЕ «ПОКУПКИ»

В выражении «Buy, Borrow, Build» (купи, заимствуй, создай) часть «buy» (купи) означает «приобрести систему, которая выполняет данную функцию». Вам нет нужды платить деньги за систему, если имеется бесплатный продукт (обычно с открытым исходным кодом), который уже реализует нужную вам функцию на приемлемом для вас уровне. Обычно платить приходится, но не всегда. Проведите исследование рынка, и, возможно, вам удастся сэкономить организации деньги.

ВЫХОД ЗА РАМКИ БЮДЖЕТА

Алиса недавно ходила на обед с подругами из сообщества женщин-руководителей, и одна из них рассказала, что вместо сокращений штата ИТ-департамент решил сократить бюджет на закупку инструментальных средств. Это означало, что ее подруге запретили продлевать контракты на SAST, DAST, SCA и другие средства обеспечения безопасности приложений. Она, конечно, была рада, что удалось сохранить штат, но понятия не имела, как делать свою работу без этих инструментов. На следующий день Алиса позвонила ей, они встретились и провели мозговой штурм. И нашли-таки очень неплохие бесплатные SAST и DAST! Найти бесплатный инструмент SCA, который им бы понравился, не удалось, но они придумали выход: взять бесплатный инструмент инвентаризации зависимостей (SBOM), а затем поискать в интернете, есть ли какие-нибудь серьезные проблемы с выявленными библиотеками. Решение не идеальное, но, когда Алиса в тот вечер пошла домой, ее подруга уже не считала положение с программой безопасности совсем уж безнадёжным. А Алиса была очень благодарна людям, создающим инструменты с открытым исходным кодом!

Проблемы авторизации и доступа

Если проблемы аутентификации и идентификации возникают только при первом контакте с системой, то вопросы авторизации и доступа приходится решать снова и снова уже после того, как пользователь вошел в систему. Каждую страницу, на которую пользователь заходит, каждую запись, которую он читает, и каждую привилегию, которую он получает, необходимо проверять *каждый раз*. У пользователя может быть доступ к основной системе, но не к ее административной части, к своим записям, но не к записям друзей. Авторизация и управление доступом должны гарантировать, что пользователь имеет доступ ко всему, что ему необходимо, и ни к чему сверх того.

Если имеются проблемы хотя бы в одном компоненте (аутентификация, авторизация, идентификация или управление доступом), то мы получаем непредусмотренный доступ: к программам, к данным, к сетям, к оборудованию и т. д.

Как такое возможно?

Дело не только в том, что эти системы сложны – не меньше, чем системы аутентификации и идентификации, – но и в том, что повторяющийся характер проверки каждого запроса доступа может оказаться весьма сложным и запутанным с программной точки зрения. Тестирование этих систем, вход от имени пользователей разных типов и проверка наличия или отсутствия доступа к данным, меню, изображениям, функциям и т. д. – в лучшем случае занятие скучное и монотонное. Это та часть тестирования на проникновение, которую я меньше всего люблю: требуется очень много внимания к деталям. Я благодарна своим друзьям-пентестерам, куда более терпеливым, чем я.

Риски и способы предотвращения этих проблем такие же, как для аутентификации и идентификации.

Проблемы конфигурации и реализации

Вам когда-нибудь приходилось, зайдя на маршрутизатор Wi-Fi, обнаружить, что пароль по умолчанию все еще активен? Тот самый, который известен всем, кто купил такое же устройство? Вы слышали о компаниях с «открытыми S3-контейнерами», у которых украли конфиденциальные данные? Это классические примеры ошибок конфигурации или реализации; тот, кто их допустил, не читал руководство по укреплению.

Если мы что-то сконфигурировали или настроили неправильно, то последствия могут быть разрушительными. Типичный пример, вошедший в OWASP Top Ten за 2021 год, – внешние сущности XML (XML external entities – XXE):

серверы с включенной поддержкой XML, которые принимают DTD-файлы, хотя не должны, создают уязвимость. Для устранения проблемы достаточно всего лишь отключить эту возможность, но, чтобы исправить, *нужно знать о существовании проблемы*. Вообще говоря, администраторы или программисты систем, в которых имеются ошибки конфигурации или реализации, не знают о проблеме, пока не станет слишком поздно.

Как такое возможно?

Большинство таких проблем вызваны одной из двух причин: игнорированием руководства по укреплению или незнанием того, что оно существует и его следует запросить, а потом выполнять. В большинстве организаций, где имеется уязвимость ХХЕ, не знают, что это такое, почему оно представляет угрозу и как это сказывается на них. Если бы знали, то при столь простом исправлении давно бы устранили.

Риск

Неправильная конфигурация безопасности может быть чревата как информационным/низким риском, так и критическим. Каждый случай следует оценивать отдельно.

Предотвращение

Быть может, вы думаете, что для предотвращения ошибок конфигурации нужно лишь волевое усилие, однако же имеется ряд шагов, которые нужно предпринять, чтобы никогда не попадаться в эту западню. И только на свою память рассчитывать не стоит.

1. Следуйте всем рекомендациям, приведенным в этой книге.
2. Настраивая систему, следуйте руководству по укреплению. Если оно не было поставлено вместе с системой, запросите.
3. Ежегодно выполняйте аудит всех систем, чтобы была уверенность в том, что предписания по укреплению по-прежнему выполняются. Проверьте, не вышла ли новая версия руководства.
4. Применяйте все исправления и обновления.
5. Разработайте всесторонний, быстрый и эффективный процесс применения исправлений и обновлений.
6. Разработайте процесс управления изменениями, предусматривающий документацию всех изменений. В идеале он должен быть автоматизированным и нетрудным.
7. Если есть возможность, проводите тестирование всех или хотя бы важных систем, особенно тех, что имеют решающее значение для выпол-

нения задач организации, интегрированы с такими системами или имеют дело с конфиденциальными данными.

Примечание После развертывания продукта, касающегося безопасности, спросите мнение тех, кого он затронул или вынужден его использовать. В одной из организаций, нанимавших меня на работу, был развернут инструмент, который, по идее, должен был отобрать административные права, но из-за неправильной настройки все лишились доступа к localhost. Разработчики жаловались, а «безопасники», которые никогда программистами не работали, понятия не имели, насколько важна возможность локальной компиляции и выполнения кода для программистов. Localhost незаменим! Меня привлекли к работе, мы все исправили, заново развернули, и программа стала работать как задумано (прав администратора больше ни у кого нет, но localhost снова функционирует). Программисты ликовали. Когда разработчик (или кто-то из пользователей) жалуется, прислушайтесь! Нам не нужен инструмент безопасности, которые делает несчастными всех остальных сотрудников ИТ-департамента.

Мошеннические транзакции

У меня по меньшей мере пять раз в жизни крали кредитную карту. Один раз ее вытащил из кошелька человек, с которым я работала; он много чего накупил в ближайшем винном магазине, а потом выкинул карту. Остальные четыре раза карту крали дистанционно, и вор совершил *несколько* мошеннических транзакций. К счастью, компании, выпустившие карты, не сняли с меня деньги за эти чужие покупки, а единственным неудобством стало ожидание доставки новой карты почтой. Но не все мошеннические транзакции так очевидны и безболезненны.

Мошенничеством с транзакциями называется любая обманная операция с намерением приобрести денежные средства, товары или услуги, не заплатив за них²⁶³. Этот риск свойствен не только операциям с денежными средствами, но и с покупкой подержанных книг в кредит, получением очков в видеоигре, продажей билетов на концерт и любой другой ситуацией, в которой имеет место обмен предмета или услуги на что-то еще.

Как такое возможно?

Люди крадут друг у друга с начала времен. Некоторые, увидев способ украсть что-то у другого человека или у безликой корпорации, тут же хватаются за эту возможность. Наша работа – защитить системы и данные от таких субъектов. Если мы не производим откат или закрытие после неудачной транзакции, то, возможно, создаем уязвимость. Если в нашем приложении имеется состояние гонки (подробнее об этом позже), непроверенные входные данные, логические ошибки и т. д., то открывается возможность для мошенничества.

Риск

Если кто-то шельмует в онлайнной игре, то вы, наверное, думаете, что риск невелик, но что, если на исход игры поставлены огромные деньги? Если кто-то придумал, как использовать один и тот же купон 300 раз вместо одного, то этого может оказаться достаточно, чтобы вышибить небольшую компанию из бизнеса. Постоянные мошеннические транзакции в финансовой отрасли – основная причина того, что мы платим такие большие комиссионные и проценты.

Только мошенничество с онлайнowymi платежами обошлось миру в 48 млрд долл. в 2023 году²⁶⁴. Учитывая, что в 2023 году на планете проживало 8.1 млрд человек, это получается по 6 долл. с носа только за один тип мошеннических операций. А таких типов несколько. Люди теряют свои пенсионные накопления, онлайнowe личности, дома и многое другое. Потенциальные риски на уровне индивидуума и организации катастрофические.

Предотвращение

Этот тип уязвимостей может быть вызван как ошибкой реализации (кодирования), так и логическим просчетом (проектирования). А значит, для их предотвращения мы должны выполнять моделирование угроз и анализ проекта, равно как и тщательный анализ кода всех транзакций в системе. Мы должны быть уверены, что *всегда откатываем ошибочную транзакцию и выполняем закрытие в случае ошибки* (как рекомендуется выше в этой книге). В случае, если ставки особенно высоки, я рекомендую еще организовать программу вознаграждения за найденные ошибки или привлечь нескольких серьезных исследователей для особо тщательного тестирования такой функциональности.

КРАЖА ЛИЧНОСТИ

Мои родители стали жертвой кражи данных в Канадском налоговом агентстве в 2021 году. В результате за их кредитной историей был установлен пожизненный надзор на случай, если кто-то попытается украсть их цифровые личности. Не знаю насчет вас, но мне, профессионалу в области информационной безопасности, стыдно, что наша отрасль не смогла защитить самых дорогих мне людей. И хотя я не могу вернуться назад во времени и предотвратить ту кражу, я надеюсь, что эта книга поможет моим читателям и мне самой приблизить гораздо более безопасное будущее для всех нас.

Атаки воспроизведением

Атака воспроизведением имеет место, когда злоумышленник перехватывает информацию о транзакции, аутентификации или управлении доступом в процессе ее передачи по сети, а затем передает повторно (воспроизводит)

в надежде получить несанкционированный доступ или совершить несанкционированную транзакцию²⁶⁵. Некто получает копию передаваемого по сети запроса и передает ее снова, надеясь получить те же самые результаты. Это может произойти с покупкой в бакалейной лавке, переводом денег с одного счета на другой или с любым другим видом цифровой транзакции.

Как такое возможно?

Если мы не озаботились созданием случайного значения (или еще какого-то токена) для каждой транзакции, одноразового пароля или временной метки, которые проверяются при получении запросов, и если мы не проверяем, что транзакция не является дубликатом, то возможна атака такого типа.

Риск

Риски атаки воспроизведением примерно такие же, как в случае онлайн-мошенничества: иногда они мизерны, а иногда огромны. Но для мошеннических транзакций больше возможных векторов атаки, чем у воспроизведения, поэтому они более распространены.

Предотвращение

Для предотвращения атаки воспроизведением можно предпринять следующие действия²⁶⁶.

1. Отправлять вместе с идентификатором сеанса одноразовое случайное число и проверять то и другое на сервере перед обработкой любых транзакций.
2. То же, но вместо случайного значения использовать одноразовый пароль.
3. То же, но в каждом запросе транзакции отправлять временную метку и проверять, что запрос поступил в течение определенного времени с момента этой метки. Это не устраняет угрозу полностью, но уменьшает временное окно.
4. То же, что 1, но отправлять временную метку. Запоминать временную метку вместе с каждым новым запросом транзакции. Сравнивать новые транзакции на предмет обнаружения дубликатов и, если временные метки совпадают, блокировать запрос.

Примечание: есть еще более сложные способы защититься от таких атак, например протоколы Kerberos, AODV и Challenge-Handshake Authentication Protocol, но мне кажется, что углубление в меры защиты на уровне сетевых протоколов выходит за рамки этой книги²⁶⁶.

Пересечение границ доверия

Когда приложению поступают данные из ненадежного источника, они считаются ненадежными, или «запачканными». Место, откуда мы их получили, называется *источником*, а первый случай использования их в приложении называется *стоком*. Мы всегда должны проверять, санировать и (или) экранировать значение, чтобы доказать, что ему можно доверять (или отклонить его), *прежде* чем допускать его к стоку. Пока не доказано, что значению можно доверять, оно считается «запачканным».

ПРИМЕР АНАЛИЗА ПОТОКА

Представьте, что вы пользуетесь средством поиска в какой-нибудь социальной сети, например X или BlueSky. Вы вводите в поисковое поле слово «#infosecgardening». В момент ввода это значение «запачкано», ему нельзя доверять. Затем приложение должно проверить ввод и, возможно, санировать или экранировать все специальные символы, которые обязано принимать. В предположении, что значение преодолело эти препятствия, оно больше не считается «запачканным». Приложение готово использовать его в первый раз, т. е. поместить в сток. Оно запрашивает у базы данных посты с таким хештегом. А потом показывает вам посты, содержащие красивые картинки достижений профессиональных специалистов по безопасности в области садоводства. Мир снова стал безопасным.

Выполняя статический анализ (SAST) или ручной анализ кода, мы документируем все входные данные как надежные или ненадежные, а затем прослеживаем значение до тех пор, пока оно либо не проходит проверку, либо не попадает в сток (впервые используется в коде). Если оно попало в сток *до* проверки, то мы имеем проблему безопасности. Это может быть внедрение кода, XSS или еще что-то, проблема определенно есть. В статическом анализе эта процедура называется *анализом потока*, и эта функциональность присутствует в инструментах SAST как первого, так и второго поколения.

Совет Одни инструменты SAST могут производить анализ потока сразу в нескольких файлах (внутрифайловый и межфайловый анализ), другие не могут, и на этот аспект следует обращать внимание при покупке инструмента. Если нет возможности межфайлового анализа (например, от фронтального JavaScript в серверный код), то можно пропустить некоторые источники или стоки.

Когда полученное от пользователя значение используется до установления его надежности, мы имеем *пересечение границ доверия*²⁶⁷. Это крайне нежелательное явление.

Как такое возможно?

Отсутствующая или неправильно реализованная проверка входных данных – проклятие для каждого специалиста по безопасности приложений. Почти всегда это происходит не намеренно, а в результате человеческой ошибки.

Риск

Риск пересечения границ доверия может быть любым: от низкого до критического. Примерами исходов могут служить как плохо отформатированные запросы и неправильные результаты в ответ (низкий риск), так и внедрение SQL-кода, ведущее к RCE (критический риск). Каждый случай следует оценивать отдельно, но я рекомендую исправлять все обнаруженные дефекты такого рода.

Предотвращение

Хотя и не хочется, чтобы меня воспринимали как заевшую пластинку, все же призываю вас последовать рекомендациям из главы 2 в разделе о проверке входных данных. К этому моменту вы уже должны быть достаточно подкованы, чтобы избежать описанной проблемы.

Проблемы при работе с файлами

Если ваше приложение обрабатывает файлы неправильно, то может иметь место крах приложения, атака на приложение или сервер или уничтожение данных, сохраненных в том же месте, что и файл. Код обработки файлов может иметь изъяны или упущения, он может не проверять тип или другие атрибуты файла или же не сканировать файл на присутствие вредоносного ПО²⁷⁰.

Файл может быть получен от пользователя, организации-партнера или от кого-то внутри своей компании, он может быть включен в ваши приложения, отредактирован вашими приложениями, загружен на сервер и сохранен в базе данных и т. д. Любая некорректная обработка файла может создать уязвимость, принадлежащую этой категории.

Предупреждение Разрешение неаутентифицированным пользователям загружать файлы в ваше приложение считается чрезвычайно рискованной операцией. По возможности избегайте ее.

Как такое возможно?

Некорректная обработка файла может быть вызвана дефектом или просчетом, т. е. ошибкой кодирования или проектирования. Часто мы не проверяем

заголовки файлов, чтобы удостовериться, что это действительно файл того типа, о котором говорит расширение имени. Проблему может составлять имя или путь к файлу, да и в самом файле может скрываться вредоносный код. Есть много способов испортить себе жизнь, если не проявить крайнюю осторожность.

Риск

Злоумышленник теоретически может получить доступ к конфиденциальным данным, выполнить команды операционной системы, распространить вирус в сети или в данных или «повалить» приложение. Атакующий может включить удаленные (RFI) или локальные (LFI) файлы в ваши приложения, скачать файлы или папки с ваших серверов, загрузить на ваши компьютеры опасные файлы или изменить файлы, к которым не должен иметь доступа. Эта ситуация также попадает в разряд некорректной проверки входных данных. Риск может быть любым – от среднего до критического, в зависимости от объема раскрытой информации.

Эта проблема может оказать негативное влияние на конфиденциальность, целостность и доступность системы, ее данных и всей вашей сети.

Предотвращение

Прочитайте раздел «Загрузка файлов» в главе 3.

Совет Подумайте об использовании свободного инструмента с открытым исходным кодом *AssemblyLine*, разработанного правительством Канады, для проверки файлов на присутствие вредоносного кода.

Проблемы при работе с объектами

Этот раздел я хочу начать с краткого обзора объектно ориентированного программирования (ООП). ООП обладает несколькими особенностями, отличающими его от декларативного (императивного) и функционального программирования, это может помочь или затруднить обеспечение надежности и безопасности приложения в зависимости от его дизайна и реализации. Рассмотрим их последовательно.

Прежде всего на удивление лаконичное определение ООП.

Инкапсулированная единица данных и поведения²⁷¹.

— Пользователь Reddit mcr1974

Отличительные особенности ООП

Наследование – способность создавать новый дочерний класс на основе родительского класса, дополняя его полями данных, переопределяя унаследованные методы для изменения их поведения или добавляя совсем новые методы. Это означает, что средства безопасности, встроенные в родительский класс, можно впоследствии как улучшить, так и свести на нет в дочерних классах с помощью наследования²⁷².

Класс включает структуры данных и методы. Отдельный экземпляр класса называется объектом. Класс -> Объект (экземпляр)²⁷³. Дочерний класс может наследовать характеристики родительского, а затем изменять, переопределять или добавлять новые, как гены ребенка могут отличаться от родительских. Дочерний класс, хоть и основан на родительском, является совершенно самостоятельной сущностью.

Представьте, что имеется класс транспортного средства. У любого транспортного средства есть колеса, руль, приборная панель и лобовое стекло. Дочерним классом может быть класс мотоцикла, у которого два или три колеса, а водитель находится не внутри транспортного средства. Другим дочерним классом может быть легковой автомобиль, у которого всегда четыре колеса, а водитель и пассажиры находятся внутри автомобиля (защищены). Все типы самодвижущихся экипажей (легковой автомобиль, грузовик, джип, танк, квадроцикл и т. д.) *наследуют* классу транспортного средства.

Вернемся к классу легкового автомобиля. Представьте, что я создала два его экземпляра: объект *Leaf* и объект *Fit*. В настоящее время я езжу на *Nissan Leaf*, а раньше ездила на *Honda Fit*. Хотя я ни в коем случае не собираюсь рекомендовать какой-то конкретный автомобиль другим, с этими двумя моделями я знакома лучше всего, поэтому и буду использовать их в качестве примеров.

Инкапсуляция скрывает внутренние детали объекта от остальной системы. Это значит, что только сам объект может читать и изменять свои данные, тогда как другие объекты или классы взаимодействуют с ним только посредством его открытых методов. Это позволяет гарантировать целостность данных, избежать непредусмотренных изменений и предотвратить несанкционированный доступ. Мы используем *модификаторы доступа* (закрытый, защищенный, открытый), чтобы определить, кто и как может обращаться к данным и методам членов²⁷².

Но вернемся к примеру с автомобилем. В прошлом году я впервые в жизни купила совершенно новый автомобиль, электрический *Nissan Leaf*. Раньше я ездила на подержанной *Honda Fit*. Я могла делать все что угодно со своей маленькой *Honda*. Могла подключить к ней сканер, купленный в интернете, узнать, что означает мигающая лампочка на приборной панели, и теоретически даже починить (хотя обычно предпочитаю, чтобы это делал кто-то другой). Моя *Honda* была как структурная программа: я могла получить доступ к любой ее части, хорошо это или плохо (я только один раз что-то сломала!). Но мой новый *Leaf* больше напоминает объектно ориентированную програм-

му, в которой большинство методов и переменных закрыты или защищены. Я не могу заменить жидкости, взломать операционную систему, и почти любое техобслуживание, кроме автоматического обновления ПО, влечет за собой потерю гарантии. Меня всего лишили. Внутренняя функциональность объекта автомобиля Leaf *инкапсулирована*.

Полиморфизм позволяет программе обращаться к объектам разных типов через один и тот же интерфейс. Происходит одно и то же, но в разных формах²⁷⁴. Лично мне это определение кажется совершенно непонятным, поэтому разберем его подробнее.

Примечание *Полиморфизм* – древнегреческое слово, означающее «много форм».

Допустим, что мы могли бы заставить метод или оператор вести себя по-разному в зависимости от контекста (вызывающего объекта). Часто встречающийся вариант полиморфизма возникает, когда объект дочернего класса вызывается по ссылке на родительский класс.

Это как если бы мама обратилась ко мне: «МИСС ЯНЦА, ВЫ НЕ ЗАБЫЛИ ПОЛОЖИТЬ ТАРЕЛКИ В ПОСУДОМОЙКУ?» Обычно-то она зовет меня Таня.

Приведем несколько примеров полиморфизма.

- *Перегрузка методов* – это создание в одном классе нескольких методов с одинаковым именем, но разными входами (параметрами) или выходами (возвращаемым значением). Хотя бы одна характеристика (вход или выход) должна отличаться, иначе это работать не будет.
- *Переопределение метода*, унаследованного от суперкласса (родительского класса): имя метода, его входы и выходы не меняются, но меняется поведение.
- *Перегрузка оператора* – переопределение встроенного оператора для работы с новыми типами.

Возвращаясь к своим любимым машинкам. У Fit был обычный двигатель внутреннего сгорания, работавший на бензине. У Leaf двигатель электрический, работающий совершенно иначе, но все равно доставляющий меня из одной точки в другую. Он по-прежнему называется двигателем, но принцип работы (методы) не имеет ничего общего с ДВС. Объект автомобиля Leaf работает от электрических батарей, т. е. у него другие входы, но один и тот же выход (доставить Таню из точки А в точку Б). Это пример полиморфизма, пусть и не очень формальный.

Под *абстракцией* понимается уменьшение сложности за счет сокрытия ненужных деталей объекта от пользователя.

Все автомобили что-то абстрагируют, одни больше, другие меньше. Чтобы вести машину, не нужно знать, как устроен двигатель, но вы, безусловно, должно отвезти ее в сервис, если загорелась лампочка «CHECK ENGINE» и вы не знаете, как самостоятельно проверить, что это означает. Я помню, как моя кузина показала мне «двигатель» своей Tesla, – просто багажник в передней части автомобиля. Она там хранит свою сумочку. Автомобиль едет, но двигатель полностью абстрагирован от пользователя.

А теперь вернемся к неправильной работе с объектами.

Десериализация и другие проблемы при работе с объектами

Неправильно работать с объектами можно по-разному, но два самых хорошо известных способа – небезопасная десериализация и нарушение авторизации на уровне объекта (broken object level authorization – BOLA). Принимая сериализованный объект из ненадежного источника и затем десериализуя его, мы идем на очень большой риск, а BOLA может стать причиной раскрытия конфиденциальных данных. Это может иметь печальные последствия для доступности, целостности и конфиденциальности системы или данных, а также привести к расширению привилегий.

Как такое возможно?

ООП – трудный предмет. Добиться правильного понимания абстрагирования, инкапсуляции и всего прочего нелегко. Неправильная работа с объектом может быть ошибкой реализации или проектирования, поскольку при использовании наследования или полиморфизма сделать ошибку особенно легко – ведь имена одинаковы, а действия совершенно различны. Если сложность велика, то вероятность ошибок возрастает, если в системе нет сдержек и противовесов.

Риск

Риск небезопасной десериализации велик, как и риск BOLA. Для других видов неправильной работы с объектами риск варьируется от среднего до высокого. Но он всегда выше низкого, поскольку программа может «грохнуться».

Предотвращение

Для уменьшения или устранения риска неправильной работы с объектами можно сделать многое. Ниже приведен список таких мер для начала.

- Применяйте принцип наименьших привилегий к объектам, т. е. внимательно относитесь к разрешениям, которые даете переменным и методам, а также способам использования и вызова объектов.
- Никогда не десериализуйте объект, полученный из ненадежного источника.
- Выпишите все разрешения, наследование, полиморфизм, абстракции и инкапсуляцию для объекта и попросите кого-нибудь проанализировать этот список. Пройдитесь по нему вместе, проговорите и, если нуж-

но, нарисуйте. Лично мне объяснение предмета кому-нибудь помогает найти изъяны практически в любом плане. Документируйте.

- Выполните моделирование угроз для своего проекта и включите объекты в обсуждение.
- Проведите анализ проекта вместе с группой безопасности. Можете совместить с моделированием угроз, если вам так проще.
- Выполните анализ кода, особенно в части реализации сложных объектов.
- Используйте отражение и интроспекцию редко и осторожно, ограничившись доступом к надежному и авторизованному коду²⁷².
- Для ООП используйте язык, безопасный относительно памяти. Если вынуждены использовать небезопасный язык, то следуйте рекомендациям по работе с памятью, приведенным ранее.
- Избегайте арифметических операций с указателями и умных указателей*. Проверяйте границы и действительность указателей²⁷².
- Инициализируйте объекты после создания в конструкторе²⁷².
- Уничтожайте объекты по завершении работы с ними с помощью деструктора. Частью этой практики должна быть сборка мусора.
- Подумайте об использовании паттернов проектирования, которые считаются полезными с точки зрения обеспечения безопасности, например: Одиночка, Фабрика, Заместитель, Стратегия и Декоратор, если они применимы к вашему проекту.
- Делайте все остальное, чего требует безопасный жизненный цикл разработки системы (SDLC), и пользуйтесь автоматизированными инструментами тестирования на безопасность. Гарантий не дает, но поможет.
- Подумайте о проведении теста на проникновение.

Проблемы управления секретами

Как уже обсуждалось выше, секреты – это скрытые от посторонних глаз значения, которые компьютеры используют для аутентификации друг друга, как человек использует пароли. Секретом может быть хеш, сертификат, ключ API, пароль и т. д. Говоря об ошибке при управлении секретом, мы обычно имеем в виду, что он попал в чужие руки (или мы подозреваем, что мог попасть). Он утек в интернет, им завладел злоумышленник, или он был еще как-то случайно раскрыт. Из того, что секрет утек, еще не следует, что кто-то обязательно использует этот факт против вас, но реагировать вы должны так, будто это произошло.

* Совет избегать умных указателей особенно подкупает. В авторе угадывается крупный специалист по ООП. Куда там Страуструпу. – *Прим. перев.*

Как такое возможно?

Раньше секреты хранили в коде. Теперь мы знаем, что это никуда не годится. Из-за этих уроков прошлого в коде можно найти уйму секретов, но есть и другая причина – люди допускают ошибки. Они забывают. Не имеют инструмента управления секретами и не уверены, куда поместить секрет. А быть может, они не понимают, что это секрет, например лицензионный ключ, поэтому относятся к его хранению небрежно. Иногда вы храните секреты в коде, но предполагали, что это будет только на этапе разработки, а потом забыли убрать перед передачей в эксплуатацию. По разным причинам это случается, но случается часто.

Риск

Помимо очевидного, – раскрытия конфиденциальных данных (сам секрет является такими данными), – такого рода проблемы обычно приводят к краже денег, данных и других ресурсов, расширению привилегий, ошибкам управления доступом и т. д. В худшем случае итогом может стать удаленное выполнение кода. К утечке действующего секрета следует относиться как к критическому риску, немедленно сменить его и инициировать процесс реагирования на инцидент, чтобы выяснить, была ли скомпрометирована ваша система.

Предотвращение

Это многошаговый процесс. Во-первых, с помощью специализированного сканера нужно выявить все секреты, которые небезопасно хранятся в вашем коде. Имеются платные и бесплатные инструменты, основное различие между ними в том, что бесплатные больше пропускают и дают больше ложноположительных результатов, так что на анализ отчета уходит больше времени. Платные инструменты дают более качественные результаты, и, стало быть, времени уходит меньше, но они стоят денег. Выбор за вами.

Выявив все секреты, вы должны решить, где хранить их безопасно. Лучше всего в инструменте управления секретами, но если у вас такого нет, то существуют и другие места, например в переменных окружения. Переменные окружения – не всегда безопасное место; если вы решите отказаться от инструмента управления секретами, то нужно будет провести изыскания. В таком случае я рекомендую обратиться к группе безопасности и (1) спросить, нет ли у них подходящего инструмента, и (2) спросить, где хранить секреты, если не в коде. Последуйте их совету.

Совет Позаботьтесь о том, чтобы максимально ограничить доступ к своему инструменту управления секретами! Он в буквальном смысле хранит ключи к вашему (цифровому) королевству. Применяйте самые строгие ограничения, выполняйте указания из руководства по укреплению и не забывайте о принципе наименьших привилегий.

Решив, где хранить выявленные секреты, выполните следующие действия.

1. Смените все секреты один за другим: пароли, ключи API и т. д. Поручите системе сгенерировать новые. Они должны быть случайными и уникальными. Никогда не используйте повторно старый секрет.
2. Поместите новый секрет туда, где он отныне будет храниться.
3. Измените приложение, чтобы оно брало секреты из нового места.
4. (факультативно) Настройте конвейер CI/CD, чтобы при сборке приложений он брал секреты из нового места.
5. Настройте в системе управления версиями точку подключения перед фиксацией, так чтобы перед каждой записью кода в систему запускался сканер секретов. При подозрении на присутствие секретов в коде он должен блокировать запись. Так вы избежите повторения всего процесса и предотвратите инциденты безопасности. Ну славно же!

Состояния гонки и проблемы хронометража

Мы уже рассматривали состояния гонки в книге «Alice and Bob Learn Application Security», но этот вопрос важен, поэтому вернемся к нему снова. Состояние гонки возникает, когда два процесса или потока в программе пытаются *одновременно обратиться к одному и тому же ресурсу*, и это приводит к проблеме (потенциально уязвимости)²⁷⁵. Состояния гонки чаще встречаются в многопоточных приложениях.

Проблема хронометража тесно связана с состояниями гонки. Хотя злоумышленник может запускать тесты и атаки, чтобы выяснить, сколько времени занимает некоторый процесс (это называется атакой с хронометражем), и использовать эту информацию для организации более серьезной атаки, здесь мы будем говорить о ситуации, когда хронометраж процесса или запроса сталкивается с каким-то другим процессом, и это приводит к состоянию гонки. Конечно, вы не хотите, чтобы злоумышленники хронометрировали все ваши запросы, чтобы понять, какой может причинить вам неприятности, но вообще проблема гораздо серьезнее и обычно влечет за собой печальные последствия.

Как такое возможно?

Любая ситуация, когда более одной системы пытаются одновременно использовать некий ресурс или процесс, может привести к состоянию гонки. Если бы вы попытались снять деньги с банковского счета одновременно с двух компьютеров, то могло бы возникнуть состояние гонки. Оба человека могли бы снять одну и ту же сумму. Или ваш баланс ушел бы в минус. Или вся система могла бы зависнуть, а потом «упасть». Любой из этих исходов сулит неприятности.

Представьте, что два процесса участвуют в гонке, стремясь получить ресурс первым. Именно так эта уязвимость получила свое название. Если мы не заблокируем ресурс перед его использованием, то может возникнуть состояние гонки. Блокировка ресурса на время использования с последующей разблокировкой предотвращает гонку.

Гонка возможна и в том случае, когда управление состоянием организовано неправильно и небезопасно²⁷⁶. Если одновременно работает несколько внешних процессов, которые вносят изменения в состояние, но эти изменения не проверяются должным образом или прерываются, либо проверки не считаются частью проектных спецификаций, то возможно состояние гонки. Если наши программы включают асинхронные операции и эти операции используются для управления разделяемыми ресурсами, то тоже возможно состояние гонки²⁷⁶.

Риск

Исход гонки непредсказуем. Может случиться что угодно – от отказа в обслуживании одного или многих пользователей до ошибок в бухгалтерском учете, краха системы, расширения привилегий, повреждения данных и даже кражи данных. Как правило, состояние гонки позволяет превысить некоторый лимит в бизнес-логике вашего приложения (так называемое *состояние гонки с превышением лимита*), и именно поэтому оно определяется как уязвимость, а не просто ошибка программирования²⁷⁷. Проблемы, вызванные состояниями гонки, не всегда очевидны, поэтому поначалу могут показаться нестрашными, но непредсказуемое поведение приложения никогда до добра не доводит. Вы *обязаны* устранить состояния гонки.

Предотвращение

Для предотвращения состояний гонки мы должны проделать несколько вещей, в основном на этапах тестирования и проектирования.

При проектировании системы соблюдайте следующие рекомендации.

- Всюду, где возможно, ограничивайте сложность. Сложность ведет к проблемам. Старайтесь быть проще.
- Если в проекте используются разделяемые ресурсы, убедитесь, что те части проекта, где к ним обращаются и изменяют их несколько потоков или процессов одновременно, проанализированы вместе с экспертом по безопасности, затем еще раз проанализируйте завершенный код и, наконец, очень тщательно протестируйте эту часть приложения²⁷⁶.
- Выполните те же шаги, что в предыдущем пункте, для любого кода, где производится управление состоянием и любые асинхронные операции. Это тоже части системы, где возможны состояния гонки²⁷⁶.

- Тщательно проектируйте блокировки для разделяемых ресурсов и процессов.
- Используйте механизмы обеспечения целостности и непротиворечивости хранилища, например: ограничения уникальности для столбцов, использование единственной хранимой процедуры (а не нескольких) для манипулирования одним системным значением (например, проверки финансовой операции перед ее выполнением) и применение механизмов конкурентности, имеющихся в базе данных²⁷⁷.
- Применяйте к транзакциям вторую (отдельную) проверку, прежде чем позволить им завершиться.
- Убедитесь, что ваш сеансовый каркас обрабатывает сеансы согласованно²⁷⁷. Если вы обновляете все сеансовые переменные пакетно, а не по одной, то состояние гонки для них возникнуть не может. По возможности стремитесь к согласованности, пусть даже в данном случае это чуть менее эффективно.
- Если вы решили управлять состоянием сеанса на стороне клиента, то обязательно шифруйте его и помните о сопряженных с этим рисках²⁷⁷.
- Подумайте, нельзя ли вообще отказаться от использования разделяемых состояний²⁷⁷.
 - Вместо этого потоки могли бы обмениваться сообщениями и хранить их в своей частной памяти, что исключило бы возникновение состояний гонки.
 - Можно было бы также использовать неизменяемые объекты, которые невозможно изменить после создания, что также исключило бы возможность гонок.

На этапе кодирования:

- используйте современный язык программирования и последнюю версию своего (современного!) каркаса программирования. В них имеются средства синхронизации²⁷⁸;
- используйте мьютексы, семафоры и мониторы для синхронизации потоков²⁷⁸;
- используйте потокобезопасные структуры данных и примитивы синхронизации, предоставляемые языком или каркасом²⁷⁶;
- используйте атомарные операции (которые с точки зрения программы производятся мгновенно), чтобы предотвратить вмешательство со стороны конкурентных потоков²⁷⁶. Пример – применение атомарной операции для изменения состояния программы;
- пишите код для проверки всех входных данных и любых изменений перед переходом приложения в новое состояние²⁷⁶;
- если в каркасе нет средств для выполнения предыдущей рекомендации, то найдите библиотеку, которая умеет это делать (но сначала проверьте ее на наличие уязвимостей!)²⁷⁶.

На этапе тестирования:

- выполняйте все обычные тесты, например статический и динамический анализ, а дополнительно нагрузочные тесты и тесты производительности;
- нанимая пентестера, сообщите ему, что в системе используется многопоточность, что управление состоянием нестандартное или что у вас есть сомнения по поводу механизмов блокировки. Попросите его обратить особое внимание на вопросы хронометража и потенциальные состояния гонки;
- проводите тестирование конкурентного выполнения. Если вы знаете, что используется многопоточность или что возможны состояния гонки, тестируйте эти аспекты специально. Этим может заняться ваша группа, группа безопасности или независимый консультант;
- наблюдайте за состояниями гонки посредством анализа журналов, сетевого трафика и мониторинга системной производительности. Ищите необычные закономерности, которые могут указывать на состояние гонки²⁷⁷.

Как видите, состояния гонки – непростой вопрос! Они могут причинить много вреда, а при небрежном отношении, да еще и в сложных системах, их легко пропустить. Будьте настороже и по возможности избегайте таких проблем.

Проблемы при работе с ресурсами

С точки зрения безопасности проблемой может являться неправильное освобождение ресурса, его останов или какая-то ошибка, сделавшая ресурс недоступным. Ваше приложение должно использовать ресурс, но не может, потому что ресурс занят кем-то еще.

Много лет назад я писала приложение, в котором присутствовала рекурсия, и открывала подключение к базе данных внутри рекурсивной функции. В производственной среде приложение падало, проработав примерно 10 мин. Я открывала одно подключение за другим, пока они не заканчивались у сервера базы, после чего приложение аварийно завершалось. Это была самая крупная программная ошибка в моей карьере (на мой взгляд), и мне было ужасно стыдно, когда я поняла, что наделала.

Примечание Состояния гонки также попадают в категорию проблем при работе с ресурсами, но мы их опустим, потому что посвятили им весь предыдущий раздел.

Как такое возможно?

Если мы не освобождаем каждый ресурс по завершении работы с ним, то возникает такая проблема. Если вы пользуетесь небезопасным относительно памяти языком, то попасть в эту ловушку проще простого, поскольку,

помимо управления сеансами, подключениями и всем прочим, вам нужно управлять еще и памятью.

Риск

Неосвобожденные ресурсы, как правило, приводят к недоступности и (или) ненадежности, но при некоторых условиях могут стать также причиной отказа в обслуживании или раскрытия конфиденциальных данных²⁷⁹. Это буква А в акрониме CIA (confidentially, integrity, availability – конфиденциальность, целостность, доступность). Поэтому обычно риск считается серьезным.

Предотвращение

Для предотвращения проблем, связанных с неосвобожденными ресурсами, начинать нужно с использования языка, безопасного относительно памяти, а также современного каркаса. Тем самым большая часть проблем будет решена.

Следите за всем, что открыли или выделили, будь то память, место в файловой системе, объекты, подключения и т. д. И проверяйте, все ли вы закрыли или освободили по завершении работы (включая все части объекта, т. е. его члены)²⁷⁹. Составьте список и вычеркивайте по одному. Так проще удостовериться, что все учтено.

Следует также пользоваться всеми возможностями каркаса, помогающими в этом отношении, например предложением `using` в VB.NET и C#, которое можно использовать в сочетании с открытием подключения к базе данных. Оно заставит вас закрыть подключение, иначе программа не откомпилируется. Сравните с обычной функцией `open()`, которой абсолютно безразлично, закрыли вы подключение или нет, что может привести к весьма печальному результату в производственной среде. Увы, я усвоила это на собственном горьком опыте.

Попадание в неизвестное состояние

Программа – это множество команд, исполняемых компьютером. Если компьютер не может найти следующую команду, если выбор следующей команды неоднозначен, если счетчик команд указывает неизвестно на что, а также во многих других ситуациях компьютер не знает, что делать дальше. Когда компьютер не знает, что делать дальше, он *попадает в неизвестное состояние*^{*}, а значит, ни программист, ни компьютер понятия не имеют, что делать. Результат может оказаться катастрофическим.

^{*} Очень своеобразное определение. Счетчик команд всегда на что-то указывает, поэтому компьютер-то знает, что делать дальше. Проблема в том, что программист не знает, что сделает компьютер. Состояние неизвестно программисту, а не компьютеру. – Прим. перев.

Примечание Некоторые употребляют термин *неопределенное состояние*, но я предпочла *неизвестное состояние*. Пользуйтесь тем, который вам ближе.

Многие уязвимости возникают, когда программа оказывается в неизвестном состоянии, например: логические дефекты, переполнения, состояния гонки, мошеннические транзакции, а также проблемы с памятью, ресурсами, хронометражем, аутентификацией и авторизацией. Список можно продолжить. Когда компьютер не знает, что делать дальше, может случиться буквально все, в том числе и ничего плохого.

Как такое возможно?

Попадание в неизвестное состояние имеет место, когда компьютер не знает, что делать дальше, и начинает гадать. Программисту совсем не нужно, чтобы компьютер самостоятельно решал, что делать дальше; мы хотим, чтобы он следовал нашим командам (написанному нами коду). Когда компьютер гадает, что делать дальше, может случиться почти все (что компьютер способен сделать).

Чем ниже уровень языка, тем проще создать такую ситуацию. Если мы пишем на C/C++ и используем указатели, то до хаоса очень недалеко. Если же используем существенно более абстрактный язык, например .NET C#, то сделать это труднее, но все же возможно.

Примечание Наша программа всегда должна знать, что делать дальше.

Риск

Следствием попадания в неизвестное состояние могут стать уязвимости почти всех категорий. В этой ситуации возможно почти все, но хорошего в этом нет никогда. Если ваша программа делает не то, что вы ожидаете, то это всегда плохо для бизнеса. Вы должны считать такую ситуацию критической и исправить ошибку незамедлительно.

Предотвращение

Я скажу, что из всех типов уязвимостей попадание в неизвестное состояние больше других заслуживает динамического анализа, тестирования на проникновения и ручного тестирования. Хотя такие вещи теоретически можно выявить в процессе моделирования угроз, статического анализа или ручного рецензирования кода, только непосредственное взаимодействие с приложением должно быть конечной проверкой отсутствия такого рода проблем.

Обязательно используйте `else`, `otherwise`, глобальный обработчик исключений, блоки `try/catch` и вообще любые механизмы, предоставляемые языком или каркасом для «отлавливания» неожиданностей, а затем указывайте

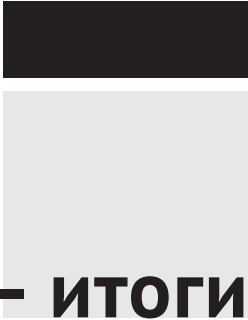
программе новое направление. Перехватить ошибку и ничего не сделать бесполезно, так проблему не решить. Вы должны сказать программе, что делать в случае ошибки.

Если вы будете следовать рекомендациям в этой книге, в том числе применять безопасный жизненный цикл разработки (SDLC) при создании ПО, то сделаете большой шаг на пути к предотвращению этой проблемы.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Из всех категорий уязвимостей, описанных в этой главе, выберите свою любимую.
 - a. Найдите в сети реальный пример такой уязвимости и прочитайте о нем.
 - b. Расскажите знакомому или коллеге, что вы узнали.
2. Своими словами объясните, что такое логический просчет и почему автоматизированные инструменты испытывают трудности в их обнаружении.
3. Приведите два примера мошеннических транзакций. Объясните, чем они грозят жертве. Примеры необязательно должны касаться финансового мошенничества.
4. Приведите два примера ошибок конфигурации или реализации (для любой технической системы по своему выбору). Объясните, чем каждая из них грозит системе. Каким может быть исход?
5. В чем разница между аутентификацией и авторизацией? Может ли одна существовать без другой? Если да, то почему? Или почему нет?
6. Почему секретами так трудно управлять? Если вы с этим не согласны, объясните почему. Если согласны – тоже.
7. Из всех категорий уязвимостей, перечисленных в этой главе, какая кажется вам наихудшей? Или самой страшной? Почему? Объясните своими словами.
8. Из всех рекомендаций, предложенных в этой главе для избегания описанных проблем, какие вы планируете внедрить? Назовите одну или две и объясните, почему поставили их на первое место.



Часть II — итоги

В этой части мы рассмотрели рекомендации для различных технологий, популярные языки, каркасы и уязвимости. Сейчас я попытаюсь подвести итоги этому огромному объему информации, чтобы вам было проще в нем разобраться.

Контрольный список рекомендаций по технологическим аспектам безопасного кодирования

Хотя API, мобильные приложения, веб-сокеты, бессерверные технологии и интернет вещей – различные технологии, есть некоторые элементы, общие для всех. Приведенный ниже контрольный список можно использовать как отправную точку при изучении любой новой технологии.

- Возьмите стандартное или напишите сами руководство по тому, как обезопасить технологию, используемую в вашей организации (в качестве отправной точки можно использовать сведения из этой книги), а затем ознакомьте с ним всех так или иначе затронутых сотрудников.
- Встраивайте в свои системы защиту от автоматизированных атак со стороны скриптов и ботов (или купите систему специально для этой цели).
- Всюду, где возможно, добавляйте уровни абстракции и эшелонированную оборону.
- Раз в год проводите инвентаризацию или, если есть возможность, купите соответствующий инструмент.
- Очень внимательно относитесь к авторизации (управлению доступом) в каждой системе на всех уровнях. Проверяйте вызов любой функции, запрос страницы, записи и т. д., прежде чем разрешить доступ.

- При любой возможности добавляйте ограничение темпа и другие лимиты. Никогда и ни для чего не разрешайте значение «не ограничено».
- Создаваемые и обслуживаемые вами системы должны пользоваться учетными записями служб (а не физических лиц) для работы в сети.
- Шифруйте данные в процессе передачи и при хранении (в том числе на локальном устройстве) и организуйте надлежащее управление секретами (не храните их в коде).
- Непременно выводите старые устройства и системы из эксплуатации.
- Применяйте принцип наименьших привилегий и другие принципы безопасного проектирования при малейшей возможности.
- Обфусцируйте код, который может попасть не в те руки.
- По возможности предлагайте пользователям многофакторную аутентификацию и применяйте стратегии, препятствующие подстановке учетных данных и атакам полным перебором.
- Помните про неприкосновенность частной жизни пользователей.
- Планируйте обновления, которые пользователи легко смогут применить самостоятельно, если такое поведение предполагается.
- Если необходимо, предоставьте конечному пользователю руководства по укреплению.
- Для параметров по умолчанию в вашей системе должны быть установлены безопасные значения. Чтобы отключить их, пользователь должен проделать дополнительную работу.
- Если ваша система физическая (устройство, оборудование), включите средства физической защиты (в этой книге не описываются). Защищать только программное обеспечение недостаточно.
- Следуйте рекомендациям, приведенным в этой книге, в том числе применяйте все средства безопасности, предоставляемые каркасом, протоколом или языком.
- Выполняйте безопасный жизненный цикл разработки системы, включающий протоколирование, мониторинг и уведомление в течение всего времени до вывода системы из эксплуатации.

Контрольный список рекомендаций по безопасному кодированию для языков и каркасов

Хотя все языки программирования и каркасы различны, в том, что касается безопасности, между ними есть кое-что общее. Используйте следующий высокоуровневый контрольный список в качестве отправной точки,

если вы не готовы реализовать углубленные рекомендации, предложенные в главах 6 и 7.

- Собираясь писать приложение с нуля, старайтесь выбирать язык и каркас исходя из количества доступных средств безопасности и современности. Чем новее каркас и язык, тем обычно он безопаснее по умолчанию. Используйте все предлагаемые средства безопасности.
- Всегда предпочитайте безопасный относительно памяти язык тому, который не обладает этим свойством по умолчанию.
- Используйте язык и каркас так, как это предусмотрено. Не отменяйте средства безопасности, предоставляемые «из коробки». В частности, не пишите встроенный JavaScript-код.
- Крайне осторожно относитесь ко всему, что выводится на экран, чтобы избежать межсайтового скриптинга (XSS). Значительная часть этих двух глав посвящена одной этой уязвимости.
- При любой возможности используйте зависящие от языка инструменты, в частности статические анализаторы (SAST) и линтеры, для улучшения своего кода.
- Обеспечивайте целостность скриптов, плагинов и всего того, что скачиваете и добавляете в свой код; пользуйтесь для этого подписанием кода или иными методами.
- Составьте список плохих функций (по типу приведенного в книге), которых следует избегать, а затем либо периодически сканируйте код с помощью утилиты `grep`, чтобы убедиться в их отсутствии, либо заведите правило в SAST-анализаторе или линтере, которое сообщает об ошибке в случае использования таких функций (так мы создаем «защитное ограждение»).
- Никогда не задавайте границу равной `*` в CORS в качестве источника или вообще всего, что предполагается ограничить.
- Не загружайте код из ненадежных источников. Имеются в виду таблицы стилей со случайного сайта, JavaScript-код, неизвестно откуда скачанный, или запуск скриптов обработки базы данных (даже если их прислал заказчик, к которому вы искренне расположены).
- Используйте последнюю версию языка и каркаса и регулярно обновляйте ее. Как минимум не используйте версий, поддержка которых прекращена.
- Обнаружив воспроизводимый дефект безопасности в языке или каркасе, сообщите о нем лицам, отвечающим за сопровождение, чтобы плодами исправления могли воспользоваться все.
- Помните, что небезопасная десериализация возможна в любом языке, допускающем сериализацию, а не только в Java. Не принимайте сериализованные объекты из ненадежных источников; если вы сами вынуждены послать такой объект по сети, то сначала зашифруйте его закры-

тым ключом, а затем расшифруйте на другом конце, чтобы убедиться в целостности, и только потом принимайтесь за десериализацию.

- Для SQL и любого другого языка запросов укрепите свой сервер и используйте параметризованные запросы.
- Используйте средства протоколирования, имеющиеся в каркасе, и защищайте свои журналы.
- Не используйте производственные данные в непроизводственных средах. Очень прошу!
- Не запускайте ничего от имени root в производственном режиме. Мы научены горьким опытом.
- Если авторы языка или каркаса предоставляют руководство по безопасности, внимательно изучите его.
- По возможности используйте неизменяемые типы данных, классы, выходные типы и значения и т. д.
- Если в языке имеется режим «strict», включите его.
- Используйте систему объектно-реляционного отображения (ORM) для вашей базы данных, если таковая имеется.
- Если каркас предлагает защиту от межсайтовой подделки запросов (CSRF), включите ее. Она не всегда включена по умолчанию, так что от вас могут потребоваться некоторые усилия.
- Если требуется вручную управлять памятью, то делайте это одинаково во всех программах, созданных или сопровождаемых вашей организацией. Будет гораздо проще искать ошибки при работе с памятью, если все делается единообразно.
- Всегда инициализируйте переменные. Не оставляйте ничего на волю случая.
- Рассматривайте предупреждения компилятора как ошибки и исправляйте их.

Сводка уязвимостей, на которые следует обращать внимание

Необязательно помнить наизусть все типы уязвимостей, чтобы быть хорошим программистом, но понимать риски и знать принципы работы уязвимостей полезно.

- Логические просчеты невозможно обнаружить автоматическими инструментами; их нужно искать, применяя моделирование угроз, пользовательские истории, касающиеся безопасности, и (или) ручное тестирование (на проникновение). В эту категорию попадают следующие

проблемы: аутентификация, авторизация, мошеннические транзакции, состояния гонки и атаки воспроизведением.

- Дефекты кодирования и реализации часто можно найти с помощью автоматических инструментов, например статических и динамических анализаторов, или ручных процессов, например анализа и рецензирования кода. Поскольку эти ошибки так легко ищутся, чрезвычайно важно исправлять их быстро. В эту категорию попадают внедрение кода, ошибки при работе с памятью, при проверке входных данных, ошибки конфигурирования и пересечение границ доверия.
- Некоторые проблемы могут быть отнесены как к просчетам проектирования, так и к дефектам реализации, в том числе неправильная конфигурация, ошибки при работе с файлами и с объектами, неправильное управление секретами, проблемы работы с ресурсами и переход в неизвестное состояние.
- Проблемы авторизации (управления доступом) сегодня более распространены, чем когда-либо прежде. Следует соблюдать предельную аккуратность при проверке и перепроверке прав доступа и по умолчанию всегда запрещать доступ во избежание проблем.
- Неправильное управление секретами встречается сплошь и рядом, и злоумышленники отлично подготовлены к эксплуатации любых упущений в этой области. Время, потраченное на вычищение секретов из кода и внедрение механизмов, предотвращающих их попадание туда, – это время, потраченное не зря.

Я надеюсь, что, прочитав эту сводку, вы согласитесь, что мы должны акцентировать внимание как на проектировании, так и на реализации систем – и только тогда сможем создать по-настоящему надежные и защищенные системы, которые хорошо послужат нашим заказчикам, – а это то, чего мы все хотим.

Теперь перейдем к третьей части книги, где будет описан безопасный жизненный цикл разработки систем (S-SDLC), соблюдая который вы сможете создавать отличное программное обеспечение.

Безопасный жизненный цикл разработки системы

Глава 9. Требования

Глава 10. Проектирование

Глава 11. Кодирование

Глава 12. Тестирование

Глава 13. Выпуск и развертывание

Глава 14. Сопровождение

Глава 15. Заключение

Требования

Какой бы проект вы ни начинали, вы должны знать, что собираетесь создать. Неважно, какую методологию разработки ПО – гибкую, водопада или DevOps – вы используете, все равно необходим четкий план (в письменном виде), показывающий, как должен выглядеть конечный продукт. В этой главе мы рассмотрим, что вы со своей командой можете сделать на этапе сбора требований к проекту, чтобы обеспечить безопасность создаваемой системы.

Начало проекта: общий обзор действий по обеспечению безопасности

Запуск в работу нового проекта обычно начинается с установочного совещания, на котором присутствуют все заинтересованные стороны, а целью является формулирование ожиданий от проекта. На этом совещании должен присутствовать представитель группы безопасности вашей организации. Он объяснит, где найти требования к безопасности, применимые к этому проекту, какие именно действия по обеспечению безопасности (тесты на проникновение, моделирование угроз и т. д.) предстоит выполнить в процессе работы над проектом и каковы критерии приемки проекта в эксплуатацию (с точки зрения безопасности). Если к этому проекту применимы какие-то политики, если имеются какие-то технические инструкции или документы, которые необходимо прочитать, или вообще что-то, что следует знать для успешного доведения проекта до конца с точки зрения безопасности, то это первая возможность довести эту информацию до вас и вашей команды.

В идеале на том же совещании представитель группы безопасности должен приступить к построению доверительных отношений с участниками проекта, если, конечно, они еще не знакомы. Группе безопасности предстоит работать с этой командой над проектом в течение многих месяцев, а то и лет, а хорошее начало полдела откачало. Если четко определить ожидания и заранее спланировать действия, так чтобы не было паники и ненужного напряжения, то и конечный продукт окажется лучше, и команда будет довольна.

БОБ СДВИГАЕТ ВЛЕВО

Боб помнит времена, когда ему еще не было 30 и приходилось очень много перерабатывать. Он помнит, как все программисты трудились долгими часами, иногда даже ночью, чтобы в последний момент перед сдачей проекта исправить ошибки. Поскольку Боб в итоге решил, что станет не разработчиком, а руководителем проекта, он поклялся себе, что будет делать все возможное, чтобы с его командами такого никогда не случилось. Если он заранее знал, что в конце будет тест на проникновение, то отводил время для исправления ошибок, которые обязательно будут найдены. Он также знал, что если «сдвинуть безопасность влево», т. е. начать думать о безопасности на ранних стадиях проекта, то в конце будет *ощутимо* меньше работы. Боб стал очень добросовестно относиться к предоставлению разработчикам инструментов, которые позволили бы им самостоятельно выявлять проблемы с безопасностью и устранять их раньше. Он также следил за тем, чтобы запланировать дополнительное время для исправления ошибок, найденных в процессе моделирования угроз, анализа кода на предмет безопасности или любых других связанных с безопасностью действий. На протяжении карьеры Боба неоднократно отмечали за отличную безопасность созданных под его руководством продуктов и за своевременную сдачу проектов. Боб старался объяснить всем, что нужно лишь «сдвинуть безопасность влево» и заранее отводить время для исправления найденных ошибок, а не предполагать, что разработчики будут работать вечерами и по выходным, но его редко слушали. Боб до сих пор не понимает, почему так мало людей руководит проектами так, как он. Быть может, нужно излагать свои мысли немножко лучше, чтобы в головах у людей что-то поменялось.

Составление графика и планирование проекта

Необходимо включать действия по обеспечению безопасности в официальный график проекта. Нам нужно запланировать достаточно времени на исправление найденных дефектов и просчетов. Недостаточно просто запланировать действие по обеспечению безопасности, нужно еще время, чтобы все исправить, потому что по ходу дела ошибки будут обязательно найдены.

Не знаю, как вы, но я предпочитаю вечерами и в выходные отдыхать, а не работать. И если вы спрашиваете у людей, включая и программистов, то уверена, почти все признают, что предпочли бы развлекаться, а не работать по 60–80 часов в неделю. Если мы не отведем достаточно времени для забот о безопасности, то результаты будут плачевны. Говоря «плачевные результаты», я имею в виду, что многие известные проблемы безопасности остались неисправленными, что все должны сильно перерабатывать, что проект сдается с опозданием, что затраты оказываются значительно больше запланированных и т. д. Есть очень много способов завалить проект, если не планировать все заранее.

Возможно, вы думаете: «Но я ведь программист, а не руководитель проекта. Я планы не составляю и вообще ничего не контролирую». Быть может, вы и не контролируете исполнение плана, но, безусловно, можете указывать на упущения и задавать вопросы. Это ваш шанс попытаться поправить дела,

прежде чем все пойдет слишком далеко. Если, глядя на график проекта, вы видите какое-то упущение, спросите: «Мне казалось, что в проекте предусмотрено тестирование на проникновение, но я не вижу его в графике. Почему? И нужно бы добавить время для исправления ошибок, которые мы обязательно найдем». Так вы, возможно, избавите себя от бесконечных вечеров на работе.

ДУЭЛЬ РУКОВОДИТЕЛЕЙ ПРОЕКТА

Когда-то Алиса работала над очень крупным проектом. Настолько крупным, что на него было выделено *два* руководителя, которые постоянно вламывались в кабинеты друг друга без предварительного согласования, требуя внесения изменений. Потом случился инцидент безопасности, и руководители совсем одурели. По несколько раз в день они заходили в кабинет расследователей инцидента, желая знать, когда будут устранены последствия. В конечном итоге терпение руководителя группы расследователей иссякло, и он громогласно объявил, что они тратят слишком много времени на объяснения в ущерб расследованию – и вообще, «убирайтесь из моего кабинета». Тогда Алиса думала, что все это весьма забавно, но совершенно очевидно, что снова и снова вмешиваться в работу расследователей абсолютно бесполезно. Но сейчас, повзрослев и вспоминая тот случай, Алиса думает: «Боже мой, неудивительно, что в этой отрасли так много эмоционально выгоревших».

Требования к безопасности

Если вы принимаете участие в техническом проекте, то должны иметь технические требования, чтобы *точно* знать, что вы собираетесь создать. В идеале в любом техническом проекте должны быть также технические требования к *безопасности*. Я имею в виду конкретные настройки безопасности, конфигурации, меры защиты, предосторожности и т. д. – все, что необходимо предпринять, чтобы соответствовать политикам безопасности, стандартам и инструкциям, применимым к вашей организации, и чтобы гарантировать, что уровень безопасности продукта не ниже чем требует организация. Чем детальнее эти требования, тем лучше!

Требования к проекту должны быть документированы и, как правило, согласованы. Если вы задокументируете требования надлежащим образом и впоследствии не получите согласование, то ваши решения и конечный продукт могут быть оспорены. Добейтесь, чтобы все стороны ознакомились с требованиями, дали свои отзывы, а затем согласовали окончательный вариант, перед тем как переходить к следующему этапу проекта.

Далее описано несколько потенциальных требований к безопасности для проектов разных типов. Списки далеко не полны, это только отправная точка. Обратитесь к главе 2 книги «Alice and Bob Learn Application Security» за списком требований к безопасности веб-приложений и для более глубокого погружения в тему.

Требования к веб-приложениям

- Весь входящий и исходящий трафик приложения должен быть зашифрован (передаваться по протоколу HTTPS), а шифрование должно отвечать принятой в организации политике (использовать безопасный протокол, например Transport Layer Security [TLS]), включая допустимые алгоритмы и длины ключей.
- Данные должны быть зашифрованы в месте хранения.
- Используйте последнюю стабильную версию языка программирования и каркаса.
- Следуйте принятым в вашей организации инструкциям или стандартам безопасного кодирования.
- Весь код должен быть просканирован статическим анализатором перед передачей в эксплуатацию и перед каждым циклом контроля качества, а все достижимые проблемы среднего уровня и выше должны быть решены. Недостижимые проблемы, начиная с высокого уровня, должны быть разрешены.
- Пользовательские сообщения об ошибках должны отвечать политике обработки ошибок, протоколирования и мониторинга. (Если возможно, ссылайтесь на другие документы, которые упоминаете.)
- Этот список неполон, добавьте требования, специфичные для своей организации.

Требования к веб-API: все требования к безопасности веб-приложений плюс следующие:

- любой веб-API должен быть доступен только через шлюз API. Как его конфигурировать, описано в документе, где приводятся подробные рекомендации касательно ограничения темпа, квотирования ресурсов, проверки входных данных, аутентификации и авторизации;
- все секреты, включая ключи API, должны храниться в инструменте управления секретами, доступ к ним должен быть только из программы. Секреты не должны храниться в коде. Весь код до передачи системы в эксплуатацию необходимо прогнать через сканер секретов;
- проверьте, что сообщения об ошибках не раскрывают слишком много информации, особенно системной. Помните: эти сообщения должны видеть только тестировщики;
- все API должны пользоваться сеткой служб. Ее конфигурация также должна быть описана в сопроводительной документации;
- динамический анализ API должен производиться инструментом тестирования, «заточенным» под API, силами группы безопасности до передачи в эксплуатацию. Все найденные проблемы безопасности среднего уровня серьезности и выше должны быть исправлены;
- этот список неполон, добавьте требования, специфичные для своей организации.

Требования к базе данных

- Все данные должны храниться в базе в зашифрованном виде.
- Все данные должны быть классифицированы по конфиденциальности, а затем помечены.
- Для всех учетных записей базы данных используйте стойкие, уникальные, случайные пароли.
- Включите МФА для всех учетных записей базы данных, которым разрешено не только читать данные.
- Сервер базы данных должен быть укреплен, а укрепленная конфигурация проверена до передачи в эксплуатацию, а затем не реже раза в год.
- Все учетные записи пользователей для приложений должны иметь только право на чтение или CRUD (создание, чтение, обновление, удаление). Права владельца базы данных (DBO) не разрешено выдавать без письменного разрешения группы безопасности.
- Все обращения к базе данных из API, веб-приложений и других систем должны производиться с помощью параметризованных запросов. Команды для непосредственного выполнения может посылать только администратор базы данных и только по специальным каналам – ни в коем случае не через API, бессерверную платформу или веб-приложение.
- К серверу базы данных следует регулярно применять исправления и обновления согласно внутренней политике.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Какие действия по обеспечению безопасности вы хотели бы реализовать на этапе сбора требований?
2. Назовите требования к безопасности, которые вы предъявили бы к веб-приложению, API или базе данных. По одному требованию для каждого!
3. Зачем нужны требования к безопасности проектов? Разве это не очевидно? Зачем их документировать?
4. Своими словами объясните, почему установочное совещание важно.
5. Что вы хотели бы узнать от группы безопасности на установочном совещании? Какой вопрос вы задали бы ей, если бы могли?
6. Своими словами объясните, почему следует включать действия по обеспечению безопасности в план-график проекта?

Проектирование

Мы уже выяснили, что для каждого проекта необходимо составить список требований. Но также необходимо спланировать, что именно мы собираемся создать, т. е. *спроектировать* программу. В методологии водопада для проектирования предусмотрен отдельный этап, тогда как в гибкой (agile) методологии и в DevOps мы часто проектируем по одной функции за раз, после того как основная часть системы уже создана. Кроме того, для гибкой методологии и DevOps мы в гораздо большей степени привлекаем к проектированию конечных пользователей, постоянно интересуясь у них, по правильному ли пути мы движемся. Технология DevOps – это вообще больше культурный сдвиг, чем просто методология. Она требует нового инструментария и перемены мышления, но я думаю, оно того стоит.

В этой главе мы поговорим о работах в области безопасности, которые можно выполнить на этапе проектирования программного обеспечения. Большая их часть применима также к оборудованию и другим типам проектируемых физических систем, но нас сейчас интересует только программное обеспечение. Не нужно выполнять каждое из этих действий для каждой создаваемой системы, но чем больше вы проделаете, чем безопаснее будет конечный продукт. Вы совместно с группой безопасности должны решить, какой уровень безопасности достаточен с учетом важности планируемой системы, терпимости организации к риску и т. д. Это не те вопросы, которые разработчик ПО может решить самостоятельно; нужен кто-то, обладающий для этого властными полномочиями, опытом и знаниями для принятия правильных решений от имени организации. Хотя, будучи разработчиком ПО, вы, возможно, не вправе решать, какие действия могут, а какие не могут иметь место в организации, вы все же можете сыграть очень важную роль добровольного и полного энтузиазма участника и знающего специалиста в предметной области.

Моделирование угроз

Моделирование угроз – это преимущественно ручной процесс выявления угроз системе. Это групповое мероприятие, в котором в идеале принимают участие технические представители от команды проекта, специалист по безопасности и представитель пользователя (которого часто называют *владельцем продукта*).

Моделирование угроз – это семейство операций, имеющих целью улучшить безопасность посредством выявления угроз и определения контрмер по их предотвращению или смягчению последствий для системы. Под угрозой понимается потенциальное или реальное нежелательное событие, вредоносное (например, DoS-атака) или случайное (сбой устройства хранения). Моделирование угроз – плановая деятельность по идентификации и оценке угроз приложению и его уязвимостей²⁸⁰.

– OWASP

Моделирование угроз может быть невероятно трудоемким процессом, требующим нескольких совещаний и написания многочисленных бумажных документов. А может быть всего одна встреча, выливающаяся в один документ. Обычно организации начинают с упрощенного процесса и со временем расширяют его. Гораздо важнее, чтобы используемый процесс соответствовал вашим потребностям и организация могла бы его поддерживать, чем идеальное соответствие какой-то определенной методологии. Хороший процесс, которому вы следуете всякий раз, намного лучше, чем идеальный процесс, который случается редко.

Вот четыре ключевых вопроса, которые должны быть рассмотрены согласно «Вопроснику по моделированию угроз» Адама Шостака²⁸¹:

- над чем вы работаете?
- что может пойти не так?
- что мы собираемся делать по этому поводу?
- хорошо ли мы справились с работой?

Первые три вопроса призваны выявить в ходе мозговых штурмов угрозы и способы их смягчения. Четвертый запрос должен заставить собравшихся подумать о том, были ли принятые решения адекватными. Возврат на шаг назад и взятие дополнительного времени на целостную оценку мер по смягчению последствий часто приводят к изменениям общего плана в лучшую сторону.

Существуют разные методологии моделирования угроз, в которых формально излагается расписание действий, но, чтобы начать движение вперед, проще всего начать с ответов на эти четыре вопроса. Из хорошо известных методологий моделирования угроз назовем STRIDE, PASTA, деревья атак, DREAM, Trike, VAST и т. д.

Если вас пригласят на совещание по моделированию угроз, я рекомендую взять с собой диаграмму потоков данных и всю имеющуюся проектную документацию. И собственное воображение! Один из моих любимых вопросов на таком совещании: «Если бы вы захотели взломать приложение, то как бы подошли к делу?» Подумайте над этим еще до совещания. Как бы вы стали взламывать собственное приложение? Что самое худшее, что с ним может произойти? Есть ли у вас какие-то тайные опасения? Как бы вы попытались злоупотребить своей системой? Существуют ли недокументированные обходные пути? Озвучить эти мысли на совещании было бы очень полезно, особенно если таким образом удастся найти реальную угрозу.

Во время совещания председательствующий от группы безопасности, скорее всего, задаст вам несколько вопросов о том, как все работает, где и как соединяются разные части, включены ли элементы безопасности в положенные им места и т. д. Не стоит особенно пугаться, когда вам будут описывать устрашающие сценарии атак; думать как злоумышленник – работа «безопасников». Помню, как во время своего первого совещания по моделированию угроз я думала: «Да как же такое может в голову кому-то прийти? Это же ужасно!» Но в конце концов моделирование угроз стало казаться мне весьма приятным и поучительным занятием, хотя чтобы прийти к такому выводу, понадобилось несколько лет.

Самая важная часть моделирования угроз – принятие мер по результатам обсуждения. Мало смысла в выявлении нескольких ужасных угроз, если потом не делать абсолютно ничего. Нужно их устранить (предотвратить) или смягчить их последствия (уменьшить ущерб). Возможно, придется включить в проект дополнительные уровни обороны, и уж точно необходимо уведомить об угрозах тестировщиков, чтобы они учли их в плане тестирования. Во время анализа и рецензирования кода вы тоже можете поискать угрозы у себя или у своих коллег. Ниже перечислены действия, следующие за моделированием угроз.

- Составьте перечень согласованных мер смягчения последствий и попросите всех участников совещания подписать его, выразив свое согласие. Сохраните этот документ вместе с остальной документацией по проекту или приложению.
- Составьте план-график или задайте крайний срок реализации каждой меры. Для каждого элемента списка установите даты проверки того, что исправления внесены, а не просто отложены в долгий ящик.
- Назначьте ответственного за анализ кода на предмет тех угроз, для которых было решено не предусматривать мер смягчения последствий; он должен проверить, возможны ли они. Если да, то, возможно, вы еще раз подумаете, нужно что-то исправлять или нет.
- Если в вашем распоряжении имеется пентестер, попросите его протестировать все выявленные угрозы.
- Если вы внесли крупные изменения в проект, – потому ли, что прошло много времени, или вследствие запросов со стороны владельцев про-

дукта, – повторите моделирование угроз, чтобы определить, не появилось ли дополнительных угроз, которые необходимо учитывать.

Спросите у «безопасников» о моделировании угроз – возможно, вы сможете поприсутствовать на следующем совещании. Обещаю, это будет интересный опыт!

Моделирование угроз рассматривается также в главе 2 этой книги, а также в книге «Alice and Bob Learn Application Security».

Паттерны и концепции безопасного проектирования

При проектировании систем мы можем решить, что будем следовать заведомо более безопасным паттернам и концепциям проектирования, описанным в разделе «Безопасное проектирование» главы 4. Поскольку эта часть книги посвящена безопасному жизненному циклу разработки системы, в этом разделе я хотела бы попросить вас применять полученные ранее знания к проектированию систем. Это значит, что при разработке проекта, составлении проектной документации или на совещаниях по проекту следует предлагать наиболее безопасный паттерн или концепцию, которые, по вашему мнению, могут соответствовать конкретной ситуации. Это также означает, что следует возражать, если кто-то пытается заставить вас не добавлять дополнительный уровень безопасности или не применять концепцию, которую вы считаете необходимой. Возможно, потребуется более четко акцентировать ценность безопасности и риски для организации. Хотя возможно, не вам принимать окончательное решение, внесение предложений о том, как ваша команда может добиться большего успеха, и противодействие срезанию углов могут очень сильно помочь вашей команде сделать правильный выбор.

Также вы можете поговорить с группой безопасности. Пригласите их посетить ваше совещание по проекту или проанализировать ваши диаграммы. Попросите высказать свое мнение. Спросите у руководителя команды или проекта, как вы можете посодействовать применению одного или нескольких из этих паттернов и концепций.

Часто некоторое действие, связанное с безопасностью, не выполняется не потому, что команда активно этому противится (из-за недостатка времени, бюджета или ресурсов), а потому что о нем никто не подумал. Поэтому вовремя предложенный паттерн или концепция могут встретить полное понимание, коль скоро имеют смысл для проекта.

Примечание Конкретные вопросы, относящиеся к этой теме, были подробно рассмотрены в главе 4 этой книги, а также в книге «Alice and Bob Learn Application Security». В этом разделе мы говорим о том, как устроить так, чтобы у вас был шанс применить полученные знания к рабочей ситуации.

Примечание Что делать, если в организации нет группы безопасности? Это ставит вас, разработчика программного обеспечения, в очень выгодное положение. Если вы работаете в совсем небольшой компании, то, вероятно, можете обратиться к руководителю отдела ИТ и рассказать ему о своих проблемах с безопасностью, планах или идеях. Если нет ни руководителя группы безопасности, ни самой группы, вся ответственность ложится на директора по информационным технологиям. Если компания настолько мала, что и такой должности в ней нет, то отвечать за это должен генеральный директор. Именно он принимает на себя все риски организации, большие и малые, поэтому должен знать о том, что происходит. Возможно, ему не понравится услышанное от вас известие о серьезном риске, уверяю вас, что знать ему об этом лучше, чем не знать. Или, хуже того, узнать из статьи в газете или из письма о выкупе...

Изображение архитектуры на доске

Цель архитектурного анализа – дать возможность нескольким людям взглянуть на архитектуру, которую вы запланировали для своего проекта, и посодействовать в ее улучшении. Изображение на доске – это процесс рисования проекта на доске, чтобы другие могли увидеть, что вы спроектировали. Доска позволяет легко исправлять любые проблемы, потому что вы можете просто стереть пальцем проблемный элемент и перерисовать его в соответствии с новой, улучшенной идеей.

В идеале рисование на доске должно сопровождаться объяснением. И все не обязательно быть художником (уверяю вас, я никуда не годный иллюстратор). Говорят же, что «картинка лучше тысячи слов», потому что большинству людей гораздо проще понять, что вы хотите сказать, если они видят иллюстрацию ваших слов.

Зачастую совещание с привлечением доски – неформальное мероприятие, сопровождающееся мозговым штурмом и решением задач. После того как все согласится, что нарисованная вами архитектура годится, вам нужно будет сфотографировать окончательную картинку на телефон, а потом уже можно приступать к написанию формального документа.

Завершив работу над архитектурным документом, вы, вероятно, организуете какую-то еще форму архитектурного анализа (например, совещание или рассылку документа с ожиданием отзывов), на этой раз *формальную*. Вы должны будете отстоять свои решения и ответить на вопросы. Кстати, если у вас уже имеются предложения от других команд, то есть все шансы согласовать документ с первой попытки.

Изучение потоков данных

На диаграмме потоков данных (ДПД) изображаются потоки информации для процесса или системы²⁸². Она, безусловно, понадобится вам для любой создаваемой программы; они не только удобны, но и помогают вам планировать свой проект, а другим – понять, как работает система.

С точки зрения безопасности проблемы могут возникнуть в местах сопряжения разных частей системы. Кроме того, вы знаете, что безопасность данных (их конфиденциальность, целостность и доступность) имеет высший приоритет. В совокупности эти два факта подводят нас к мысли о необходимости изучить, где и как данные перемещаются в системе.

Возьмите свою ДПД, изучите ее, стараясь найти потенциальные проблемы безопасности. Если вы уже проводили моделирование угроз, то, вероятно, искали угрозы на своей ДПД, а если нет, то включите эту операцию в свой процесс. В идеале хорошо бы пригласить человека из группы безопасности, чтобы он взглянул на диаграмму вместе с вами. Для каждой точки, где информация переходит из одного места в другое, вы должны задать себе некоторые вопросы. *Шифруются ли данные в процессе передачи?* Выполняются ли аутентификация и авторизация перед отправкой данных? Откуда вы знаете, кому отправляются данные? *Проверяются ли полученные данные*, чтобы была уверенность, что использовать полученное безопасно? Где хранятся данные? Шифруются ли данные при хранении? Были ли данные надлежащим образом классифицированы и помечены? Для любого места на диаграмме, где данные перемещаются или изменяются, необходимо задать себе как можно больше вопросов, чтобы не сомневаться в конфиденциальности, целостности и доступности данных. Если кажется, что какой-то элемент отсутствует, внесите его в проект или архитектуру.

Пользовательские истории о безопасности

Пользовательские истории вошли в обиход в 1989 году²⁸³ как часть методологий экстремального и гибкого программирования, а это значит, что они существуют уже давно, и наша отрасль должна быть хорошо с ними знакома. *Пользовательская история* – неформальный документ, который кратко объясняет программную систему²⁸³. Она пишется с точки зрения пользователя, и в ней максимально понятно описываются требования и ожидания для каждой функции системы. Обычно имеется одна история на каждую функцию, и она должна объяснять ценность этой функции и что она делает.

Пользовательские истории о безопасности рассказывают о функциях безопасности, имеющих в продукте.

Как и следовало ожидать, в OWASP имеются прекрасные примеры полезных пользовательских историй, они приведены на странице <https://github.com/OWASP/user-security-stories/blob/master/user-security-stories.md>²⁸⁴.

Приведем несколько примеров и мы.

- Как пекущийся о безопасности пользователь программы (ПБПП), я хочу быть уверен, что мои данные остаются в безопасности, когда я доверяю их этому продукту. Критерии приемки: данные должны шифроваться в процессе передачи и во время хранения. Классифицировать и пометить данные. Использовать параметризованные запросы при вза-

имодействии с базой данных. Использовать статический анализ для проверки реализации.

- Как ПБПП, я хочу чувствовать себя в безопасности при работе с этим приложением и не стать жертвой XSS-атак. Критерии приемки: используется JavaScript-каркас, который кодирует вывод. Сканировать код статическими и динамическими анализаторами, избегать встраиваемого HTML/JavaScript, анализировать код на предмет XSS и ненадлежащего кодирования данных.
- Как ПБПП, я хочу быть уверен, что никто не сможет воспользоваться моими учетными записями или перехватить управление ими. Критерии приемки: моделировать угрозы или выполнить анализ проекта, использовать уже имеющуюся систему аутентификации и авторизации, анализировать код на предмет присутствия этого элемента безопасности в надлежащем месте и провести тестирование на проникновение.
- Как ПБПП, я хочу быть уверен, что мой пароль для входа в эту систему хранится безопасно. Критерии приемки: анализировать код на наличие этой функции, убедиться, что в проектных требованиях отражено подсаливание и хеширование хранимых паролей и что статический анализатор обращает внимание на стойкость выбранных криптографических функций.

Для любого требования к безопасности и элемента безопасности, включенных в проект, необходимо создать пользовательскую историю и критерии приемки. Эти истории должны храниться вместе с прочими, и обращение со всеми должно быть одинаковым.

Упражнения

1. Создайте сами три пользовательские истории о безопасности. Первая должна быть о предотвращении хранения секретов в коде. Темы двух остальных придумайте сами. Не забывайте о критериях приемки!
2. Получите диаграмму потоков данных у себя на работе (или поищите что-нибудь простенькое по изображениям в Google), а затем внимательно проанализируйте ее на предмет проблем с безопасностью. Спросите себя: «Где тут нужно шифрование? Где бы я хотел иметь аутентификацию и авторизацию?» Обведите кружочками, добавьте примечания и составьте список того, что добавили бы *вы*, если бы это была ваша система.
3. Возьмите ту же диаграмму потоков данных и зайдите на сайт <https://shostack.org/games/elevation-of-privilege>. Это бесплатный сайт, на котором можно поиграть в карточную игру моделирования угроз, связанных с расширением привилегий, или купить ее. Попробуйте поиграть в режиме онлайн, используя найденное вами изображение (можете поменять изо-

бражение, если выбранное вами окажется слишком трудным). Если есть возможность, пригласите друзей – так играть будет интереснее.

4. Снова обратитесь к поиску Google по изображениям и поищите архитектурную диаграмму веб-приложения с API. Попробуйте такой запрос: «web app architecture diagram with API». Вы найдете несколько изображений; выберите то, что вам больше нравится, но в идеале хотелось бы, чтобы оно сопровождалось описанием назначения системы. Представьте, что вы пытаетесь смоделировать угрозы этой систем. Задайте себе четыре вопроса Адама Шостака, это должно помочь! Затем подумайте, как смягчить последствия этих угроз. Все тщательно записывайте, а не прокручивайте в голове. Запись поможет научиться и запомнить урок.
5. Нарисуйте примерную архитектуру для следующей системы, а затем попытайтесь изобразить ее на доске: веб-приложение, которое позволяет родителям планировать даты игр своих детей с друзьями. Потом можете добавить детей и их друзей, а также номера телефонов родителей друзей и их недельные расписания, чтобы система могла запланировать встречи детей в дни, когда их расписания совпадают. Каждый родитель заводит себе учетную запись в системе, а затем «связывается» с родителями детей, с которыми дружат его отпрыски. Какие риски могли бы быть в такой системе? Какие хранящиеся в ней данные конфиденциальны? Что, если какие-то дети не любят друг друга? Или у родителей нет машины? При желании этот пример можно развить довольно далеко, но можете предложить другую систему, если эта вас не устраивает. Идея в том, чтобы нарисовать систему, привыкнуть к изображению архитектуры и последующему нахождению в ней проблем и их исправлению.

Кодирование

При написании кода вы можете сделать несколько вещей, чтобы код стал безопаснее, именно этому и посвящена настоящая глава. Даже если в организации нет группы безопасности, готовой прийти вам на помощь, и нет денег, чтобы заплатить за мощные инструменты и обучение, все равно вы можете сделать больше, чем кажется. В вашей власти определить способ написания кода и выбрать некоторые используемые инструменты, и, даже если тестирование не входит в ваши должностные инструкции, вы все равно можете прогонять тесты своего кода. Уверена, вы слышали максимум «Знание – сила»; так вот, в этой главе я собираюсь вооружить вас знаниями, чтобы вы могли взять дело в свои руки.

Обучение

В бытность свою разработчиком я редко имела возможность получить доступ к обучению сверх засвидетельствованного моим дипломом по информатике, которое я оплатила самостоятельно, работая на двух работах на протяжении почти всего времени обучения. Решив сменить карьеру программиста на работу в области безопасности, я начала выступать на конференциях, чтобы не платить за участие. Поскольку на протяжении большей части моей взрослой жизни обменный курс канадского доллара по отношению к американскому был не в пользу первого, а обучение на английском языке в основном проводилось в Соединенных Штатах, получалось, что я не могла позволить себе почти никакого обучения. Помню, что подсчитала, что один курс в институте SANS, включающий одну попытку сдачи экзамена, обошелся бы примерно в 20 % моих доходов за год после вычета налогов. Это означало, что не будет никаких обедов. Никаких концертов. Никаких киношек. На протяжении всего года. На такую жертву я не решилась.

Я разработала целую стратегию, как получить достаточное образование и опыт, чтобы поменять работу, помимо выступлений на конференциях. Когда я только начинала карьеру в области безопасности приложений, фонд

OWASP резервировал бесплатные места на всех конференциях для одного руководителя команды и одного руководителя группы (chapter leader); я была тем и другим, поэтому таким образом получала бесплатное образование. Я также впитывала каждую каплю бесплатного обучения, которую могла найти в интернете, – от прослушивания подкастов и чтения блогов до посещения курсов различных онлайн-академий (качество которых сильно варьировалось). Я перечитала все книги со словом «кибер» в библиотеке, а также купила несколько учебников и проработала все упражнения в своей домашней лаборатории. Помимо всего этого, в процессе смены карьеры у меня было несколько профессиональных наставников, каждый из которых чему-то научил меня. Это был не самый эффективный способ начать новую карьеру (он занял несколько лет), и временами было нелегко, но в итоге все получилось, и мне удалось стать специалистом по безопасности, коим я являюсь и сегодня.

Работая программистом, я сделала все ошибки, о которых теперь рассказываю в этой книге. На курсе информатики в колледже нас учили сохранять секреты в коде, не проверять и не санитировать входные данные и проектировать системы, исходя из предположения о неявном доверии. Тогда мы просто не знали ничего лучшего. Но и сегодня большинство учебных заведений, присваивающих научные степени и раздающих дипломы, вообще не учат безопасности и чуть ли не каждый день выпускают новых разработчиков, почти или совсем не владеющих знаниями о безопасном кодировании.

Совет Если вы студент колледжа или университета, то спросите своего профессора, собирается ли он рассказывать о безопасности приложений или безопасном кодировании. Если он ответит отрицательно, спросите, почему нет. Попросите его добавить этот материал в программу курса. Скажите, что это важно для вас. Если таких просьб будет много, то, возможно, что-то скорее изменится к лучшему.

Я рассказываю все это не для того, чтобы вы пожалели меня, вынужденную так тяжело трудиться, а чтобы вы поняли, как *много работы* вам придется проделать самостоятельно (помимо основной работы) и что большинство программистов не готовы к этому. Они не собираются платить тысячи долларов из собственного кармана, учиться вечерами и использовать отпуска для посещения конференций. Я них есть личная жизнь, не ограничивающаяся работой, и, честно говоря, я считаю, что это нормально и так и должно быть. Баланс между работой и личной жизнью важен, и, конечно, я надеюсь, что вам не придется подражать мне в этом отношении, поскольку борюсь с этим изо всех сил.

Если вы хотите, чтобы программисты у вас на работе создавали и сопровождали безопасные приложения, то должны научить их, как это делать. Нельзя предполагать, что они с самого начала уже знают все это и задумываются об этом (и неважно, сколько им лет и где они учились программировать). Если вы надеетесь, что они будут учиться за свой счет, то вы будете очень и очень разочарованы. Платить за их обучение придется вашей организации.

Но есть и хорошие новости. Обучение может принимать разную форму, и сегодня вариантов гораздо больше, чем раньше, а цены значительно снизились за те десять лет, что я занимаюсь безопасностью приложений. Ниже описано несколько стратегий обучения безопасному кодированию и созданию безопасных программ, доступных вам и вашей организации.

Примечание Я преподаю безопасное кодирование. Вы можете нанять меня для обучения вашей команды всему, что написано в этой книге, и не только. Электронная почта для контактов: tanya@shehackspurple.ca.

Организации

Совершая групповую покупку, вы повышаете свою покупательскую способность. Это означает, что вы можете пригласить преподавателя для обучения, и в большинстве случаев это обойдется вам дешевле, чем отправлять своих сотрудников на обучение в частное учреждение или оплачивать обучение на конференции. Преподаватель также может подстроить курс под потребности вашей организации, и все сказанное вашей командой на занятиях будет подпадать под соглашение о неразглашении. Если вы работаете в государственном учреждении, то можете скооперироваться с другими подразделениями или партнерскими организациями, чтобы получить групповую скидку.

Некоторые организации приобретают лицензии на учебные платформы по кибербезопасности, которые содержат интерактивные уроки или уроки по запросу. Лично мне такой вариант очень нравится, потому что я могу планировать свое время и учиться понемногу каждый день, но не всем это важно. Вместе с тем я заплатила за много онлайн-курсов, на которых прослушала только пару уроков, а затем бросила вопреки самым лучшим намерениям. Если ваша организация выберет этот путь, то очень важно спланировать, как работники будут использовать систему, поскольку часто бывает, что человек заходит в нее только один раз, и лишь немногие проходят весь курс, если только их не принуждают к этому или не предлагают какого-то стимула. Кроме того, позаботьтесь о том, чтобы у работников было время для посещения уроков и выполнения домашних заданий; если вы думаете, что они будут тратить на это свои выходные, то готовьтесь к горькому разочарованию.

В некоторых организациях я наблюдала такую тактику: запустить программу обучения разработчиков или ответственных за безопасность, об обеих я не раз говорила на конференциях и писала. Программа обучения разработчиков подразумевает, что внутренний персонал составляет учебный план для всех разработчиков и может наполнять его содержанием самостоятельно или привлекая кого-то со стороны. Часто и регулярно проводятся небольшие тренинги, чтобы поддерживать интерес на высоком уровне. Программа для ответственных за безопасность похожа по способу обучения, но в ней участвует по одному представителю от каждой команды разработчи-

ков, который в итоге будет отвечать за безопасность создаваемых ей приложений; обычно объем обучения в этом случае гораздо больше. Я понимаю, что этого краткого описания недостаточно, чтобы воздать обеим темам должное, но вы без труда можете узнать больше, либо бесплатно посетив мой курс «Основы безопасности приложений, уровень 2», предлагаемый Semgrep Academy, либо поискав в интернете по запросу «SheHacksPurple» в сочетании с «developer education» или «security champion»; будет выдано много результатов, которые помогут вам составить прекрасный план для любой из двух программ.

Физические лица

Помню, как во время работы программистом я впервые просканировала одно из своих приложений динамическим анализатором безопасности (DAST). Я поискала в сети «cross-site scripting», чтобы узнать, как исправить найденные ошибки, и в тот момент на весь интернет оказалось всего три результата! Как же все изменилось за это время – к лучшему!

Бесплатных возможностей обучения сейчас больше, чем когда-либо прежде. Вы можете научиться безопасному кодированию, хакингу веб-приложений и многому другому, просто выполнив несколько поисков в интернете и потратив сколько-то времени. Разработчикам ПО я настоятельно рекомендую сосредоточиться на безопасном кодировании, безопасном проектировании и анализе кода на предмет безопасности. Если же вы планируете в будущем стать специалистом по безопасности приложений, то я рекомендую поучиться организации программы безопасности приложений, а затем более углубленно изучить свои любимые части этой программы (например, моделирование угроз и тестирование на проникновение). Узнайте, какие есть возможности, а *затем* продолжайте выбранный путь обучения.

Чтобы продолжать учиться, а не бросить это дело на полпути, вы должны выработать стратегию, подходящую лично для вас. Например, отвести для обучения какое-то время каждый день или несколько раз в неделю, привлечь напарника, перед которым будете отчитываться о проделанной работе, и (или) выбрать наиболее подходящие для вас форматы обучения (я, например, предпочитаю аудиокнигу печатным). Рекомендую почитать главу 10 книги «Alice and Bob Learn Application Security», где описано, как составить личный план непрерывного обучения.

Примечание Вы можете и дальше учиться у меня бесплатно! Я даю формальные уроки в Semgrep Academy, <https://academy.semgrep.dev>, а также выкладываю менее формальные видеоролики на своем канале в YouTube, <https://youtube.com/shehackspurple>. Чтобы не пропустить новых материалов или мероприятий, на которых я буду присутствовать, подпишитесь на мою рассылку, <https://newsletter.shehackspurple.ca>.

Анализ кода

Анализ кода на предмет безопасности подразумевает поиск уязвимостей в исходном коде приложения. Он может быть ручным или автоматизированным, но наилучшие результаты дает сочетание того и другого²⁸⁵. Иногда анализ кода называют тестированием методом *белого ящика*, но чаще *статическим анализом*. Для ручного анализа вам нужен только доступ к коду для чтения или возможность запустить инструмент статического тестирования приложения на безопасность (SAST).

Если вы собираетесь выполнять ручной анализ кода, то должны убедиться, что все предусмотренные проектом элементы безопасности находятся там, где должны быть, и правильно реализованы. Кроме того, поищите в коде смоделированные угрозы, проблемы бизнес-логики (просчеты) и реализации (дефекты).

Если вы собираетесь выполнять автоматизированный анализ кода, то должны будете воспользоваться инструментом SAST, который еще называют статическим анализатором. Согласно моей классификации, есть два типа таких инструментов: первого и второго поколения. Инструменты первого поколения выполняют комбинацию символического выполнения и анализа потока; это медленно и дает больше ложноположительных результатов, зато анализ чрезвычайно доскональный. Инструменты второго поколения гораздо быстрее и дают меньше ложноположительных результатов, но кое-что пропускают. Они производят поиск антипаттернов и анализ потока. В условиях, когда большинство организаций имеют сотни ждущих исправления ошибок и отдают приоритет быстрому выпуску новых функций, а не обеспечению безопасности, инструменты второго поколения быстро завоевали нашу отрасль. Чтобы узнать, относится ли инструмент SAST к первому или второму поколению, выясните, производит ли он символическое выполнение или поиск антипаттернов.

Статические анализаторы первого и второго поколения

Организация, в которой вы работаете, может предоставлять сотрудникам статический анализатор, а может и не предоставлять. Если предоставляет, пожалуйста, пользуйтесь им! В идеале он содержит плагин IDE, помогающий прямо в процессе кодирования. Если же нет, то в один прекрасный момент вам придет из группы безопасности отчет об ошибках. Если организация не предоставляет статический анализатор, то можете воспользоваться одним из нескольких бесплатных инструментов (например, Semgrep Community, Brakeman, Bandit, Norusec и т. д. – их *много*), анализирующих код на пред-

мет наличия уязвимостей. Вы должны использовать хотя бы один, а затем исправить все, что ему не нравится. Если не уверены, проконсультируйтесь с группой безопасности. Если у вас на работе нет группы безопасности, направьте вопрос в канал OWASP Slack; там много специалистов по безопасности приложений, готовых помочь.

Примечание Инструменты SAST первого поколения все еще занимают свое место в отрасли, пусть даже все компании дружно устремились в сторону DevOps, поскольку предпочитают «двигаться быстро и ломать все»*. Иногда вам нужно, чтобы приложение было настолько близко к идеалу, насколько это в человеческих силах, например, если занимаетесь борьбой с терроризмом, защищаете демократию в своей стране или работаете в области обороны. Бывает, что на защиту всего одного приложения не жалко потратить миллион долларов, например, когда на кону человеческие жизни или риски катастроф. Иногда разумно использовать инструменты *обоих* видов, чтобы обеспечить максимально тщательную проверку. Не отбрасывайте инструменты первого поколения только потому, что они отстают от нынешней сногшибательной скорости разработки ПО; всемо свое время и место, а они по-прежнему представляют ценность.

SAST первого поколения	SAST второго поколения
Fortify	Semgrep Code
Veracode	Snyk Code
Checkmarx	QuietAI

Совет Хотя есть много прекрасных инструментов, объявляющих себя «бесплатными», в некоторые из них встроены ограничения. Они бесплатны не вообще, а только на пробный период, что тоже неплохо, потому что позволяет оценить инструмент перед покупкой, но для длительного использования не годится. Прочитайте лицензию, прежде чем зайдете слишком далеко, чтобы (1) убедиться, что удовлетворяете ее условиям и (2) не оказаться отрезанным от помощи как раз тогда, когда она стала привычной.

Примечание Мой опыт показывает, что наиболее очевидное различие между платными и бесплатными инструментами SAST – качество результатов и способность к интеграции с другими вашими инструментами. Бесплатные инструменты редко предлагают API или интеграцию с конвейером CI/CD, репозиторием кода, средством отслеживания ошибок и другими инструментами разработки. Кроме того, бесплатные инструменты обычно дают *очень* много ложноположительных результатов. Если вы работаете в организации, у которой совсем нет бюджета, но полно времени, то сможете неплохо жить и с бесплатными SAST (OWASP поддерживает страницу, где все они перечислены: https://owasp.org/www-community/Source_Code_Analysis_Tools). Но если у вас много работы и мало свободного времени, то гораздо эффективнее приобрести платный инструмент SAST. То же относится к инструментам сканирования секретов: вы платите за то, чтобы получать меньше ложноположительных и ложноотрицательных результатов. Но для инструментов DAST это уже не так, два моих самых любимых (Burp Suite Pro и Zap) соответственно почти и совсем бесплатны.

* Девиз, который был популярен в Кремниевой долине с начала нулевых. Он стал известен благодаря Марку Цукербергу, призывающему начинающих бизнесменов творить в скоростном режиме и не обращать внимания на мелочи.

Совет Инструмент проверки качества кода, или линтер, не то же самое, что инструмент SAST. В идеале хорошо бы использовать *и* линтер, *и* инструмент SAST, но если вам по средствам купить только один, то я рекомендую остановиться на анализаторе безопасности (SAST).

Безопасные ограждения

Некоторые статические анализаторы позволяют создавать «безопасные ограждения», специальные правила, которые гарантируют выбор более безопасных приемов кодирования. Вы можете создать ограждения, помогающие соблюдать стандарты кодирования, например использовать одну функцию проверки вместо другой, соблюдать соглашения об именовании или пользоваться ВерблюжьейНотацией, а не змеиной_нотацией. Эти инструменты можно также использовать для поиска и предотвращения серьезных дефектов или просчетов безопасности и гарантировать, что больше они никогда не появятся. Ограждения можно реализовать с помощью инструмента SAST в виде плагина IDE, в момент помещения кода в систему управления версиями, в конвейере CI/CD или в любом другом месте – лишь бы они вернули вас и вашу команду на путь истинный, т. е. к написанию более качественного кода. Идея безопасного ограждения такая же, как физического ограждения на дороге: чтобы проинформировать вас о том, что вы сбились с пути и помочь вернуться на асфальтированную дорогу, где вам и надлежит находиться. Вы, конечно, можете и дальше ехать по грунтовке, т. е. не использовать безопасные умолчания и не соблюдать стандарты организации, но это значит, что вы поступаете так сознательно, имея на то веские основания. Создание и использование безопасных ограждений помогает избежать много распространенных ошибок, сэкономить время и деньги и в общем случае считается проактивным подходом к обеспечению безопасности. Если вы решите попробовать этот подход, проработайте его со своей группой безопасности для достижения наилучших результатов.

Плагины IDE и другие наставления

Ваша IDE – чрезвычайно мощный инструмент, достойный изумления. Когда я поступила в колледж, мы использовали Borland, и по сравнению с современными инструментами, например VS Code, это был ужас-ужас. Существует множество удивительных плагинов, которые могут стать частью IDE и помочь при решении многих задач. Есть плагины для статического анализа, для линтинга, для работы с вашими API и работающие в качестве прокси вашего веб-приложения. Выбор инструментов велик и способен ошеломить!

Я предлагаю использовать следующие плагины (от того поставщика, который вам больше нравится):

- для статического анализа: в идеале сомнительный код должен подчеркиваться красной извилистой линией;
- для сканирования секретов: который сообщает о присутствии секрета в вашем коде;
- для анализа состава программы: который сообщает, что вы используете заведомо небезопасные зависимости, а также номера других, более предпочтительных версий;
- линтер: повышение качества кода всегда приветствуется;
- сканер IaC: проверяет вашу облачную «инфраструктуру как код» на предмет неправильной конфигурации.

Запускайте их по меньшей мере один раз перед публикацией кода и исправляйте найденные ошибки. Я понимаю, что не всегда имеется возможность исправить все, что эти инструменты найдут, но исправляйте хотя бы некоторые изъяны.

Совет Не все плагины равны! Прежде чем скачать и установить плагин, проверяйте, что создавшая его компания заслуживает доверия. Маловероятно, что хорошо известная компания, работающая в области безопасности, намеренно создаст вредоносный плагин, но плагинов, созданных физическими лицами (если только они не являются всем известными), или независимых проектов с открытым исходным кодом от неизвестных авторов следует избегать (если только вы предварительно не подвергли их тщательно тестированию и не согласовали с группой безопасности).

Совет Раньше я рекомендовала прогонять автономные тесты для различных уязвимостей, например межсайтового скриптинга (XSS), но потом поняла, что эту идею можно более удачно масштабировать, если воспользоваться пользовательскими правилами SAST, а затем преобразовать их в безопасное ограждение – с тем же результатом. Вы можете создать по одному пользовательскому правилу для каждого языка и каждой уязвимости, а затем прогнать их для всей своей кодовой базы, тогда как в случае автономных тестов придется написать несколько тестов на каждую уязвимость и каждый тест повторить для всех блоков кода, в которых может проявиться эта уязвимость, – такой подход плохо масштабируется. Существуют также правила SAST, написанные сообществом, т. е. людьми вроде вас; их можно найти в интернете для всех популярных инструментов и языков.

Проверка безопасности зависимостей

Как уже было сказано выше, весь код приложения представляет риск, даже тот, который писали не вы. Если вы собираетесь использовать какой-нибудь каркас, пакет NuGet или Ruby или любой другой написанный кем-то код, то в идеале должны бы проверить, что использовать его безопасно. Самый простой и надежный способ сделать это – воспользоваться инструментом анализа состава программы (SCA), чтобы определить, в каких зависимостях имеются известные уязвимости, а затем решить, какие стоит обновить, от каких лучше вообще отказаться, а какие можно оставить.

Как решить, какие зависимости следует обновить или заменить?

Согласно посту Джонатана Юниса на сайте StackOverflow²⁸⁶, в общем случае обновлять зависимость следует, когда:

- 1) это необходимо;
- 2) это дает какое-то преимущество;
- 3) отказ от этого сулит неприятности.

Когда я была младшим разработчиком, я соглашалась с этим утверждением, и особенно с пунктом 1. Когда я продвинулась по карьерной лестнице и должна была вытягивать свою команду из пут накопившегося технического долга, я начала искать способы производить обновления понемногу, но постоянно, чтобы избежать неприятностей в будущем (пункт 3). Это было еще до того, как я столкнулась с трудностями в качестве специалиста по реагированию на инциденты из-за зависимостей, которые были настолько устаревшими, что мы просто не могли реагировать должным образом. Если требуется год или два, чтобы избавиться от крайне небезопасной зависимости, которая в настоящее время всю эксплуатируется, то такой результат абсолютно неприемлем с точки зрения безопасности. Подводя итог, можно сказать, что за прошедшие годы я изменила свое мнение и теперь считаю, что пункт 3 самый важный.

Сейчас я стараюсь убедить разработчиков в том, что зависимости нужно обновлять максимально быстро, так чтобы всегда использовать поддерживаемую версию. Кроме того, я применяю инструменты SCA к кодовой базе и прошу сообщать о зависимостях, которые имеют уязвимости среднего, высокого и критического уровня, достижимые или допускающие эксплуатацию. Если уязвимость критическая, я прошу обновить ее, даже если она не достижима, потому что это не что иное, как бомба с часовым механизмом, затаившаяся в программе.

Если вы понимаете, что перейти на новую версию зависимости будет очень трудно, долго или дорого, то поговорите с группой безопасности. Объясните, каковы затраты, и спросите совета, что с этим делать. Иногда оказывается, что есть другие решения, о которых стоит поразмыслить.

НЕСТАНДАРТНЫЙ ПОДХОД К БЕЗОПАСНОСТИ

Алиса помнит контракт на управление проектом, заключенный с крупной ИТ-компанией, которая обнаружила две серьезнейшие уязвимости в зависимостях. Приложения, в которых они были найдены, были очень старыми, и для того чтобы перейти на новые версии опасных зависимостей, требовалось полностью изменить архитектуру обоих приложений. Одно приложение обслуживало несколько сотен заказчиков, было размещено на их серверах и имело выход в интернет. У второго приложения было всего четыре заказчика, и планировалось вывести его из эксплуатации в течение года. После длительных обсуждений было решено перепроектировать первое приложение, а перед вторым поставить брандмауэр веб-приложений (WAF) у каждого пользователя до момента вывода

из эксплуатации. Команда Алисы решила, что это уменьшит риск до приемлемого уровня и обойдется дешевле, чем переписывание старого приложения, которое все равно дожидалось последние дни. Клиент был очень доволен нестандартным подходом Алисы.

Поиск секретов и управление ими

Под *секретами* понимаются конфиденциальные строки, которые используются компьютерами для аутентификации друг друга. Секретом может быть пароль, ключ API, сертификат и т. д. Любое длинное сгенерированное случайным образом значение, которое хранится в тайне, может использоваться в качестве секрета. Как я уже говорила, в колледже меня учили хранить секреты прямо в коде, но теперь это считается ошибкой. Имеется много задокументированных инцидентов безопасности, возникших из-за того, что злоумышленник сумел извлечь секреты из исходного кода, а затем использовать их для кражи конфиденциальных данных, перехвата управления системой и т. д. Поэтому наша отрасль пришла к идее о создании систем специально для защиты секретов, они называются *инструментами управления секретами*. Больше не считается правильным хранить секреты в системе управления версиями, вместо этого мы должны получить к ним доступ из программы, запрашивая у инструмента управления секретами.

Если в вашей организации нет единого плана управления секретами, и есть вероятность, что компания еще не проводила поиск секретов во всем репозитории кода, то я рекомендую начать именно с этого. Существует много сканеров секретов, как бесплатных, так и платных. Уже было сказано, что платные инструменты дают значительно более точные результаты, что экономит вам время, но они стоят денег, так что решайте сами с учетом своего бюджета.

Получив в свое распоряжение сканер секретов, просканируйте им всю кодовую базу. Вы будете удивлены, как мало времени потребуется для обнаружения первого секрета. Для всех находок нужно проверить, действительно ли это секрет, и если да, то активен ли он. Что-то секретом вообще не является; например, сканер может принять за секрет слово «password», хотя на самом деле это просто имя переменной или слово в предложении. Необходимо отбросить все ложноположительные результаты, а затем проверить то, что осталось, на активность. Секрет считается активным, если он используется в настоящее время, поэтому такие проблемы нужно устранять в первую очередь.

УТЕКШИЕ СЕКРЕТЫ НЕДОЛГО ОСТАЮТСЯ СЕКРЕТАМИ

Боб слышал от своих друзей в OWASP, что репозитории открытого исходного кода на GitHub сканируются *каждые 30 с* в поисках секретов! «Боже правый!» – подумал он. Он пометил себе на следующий же день установить точку подключения перед фиксацией, в которой будет запускаться недавно приобретенный сканер секретов.

Обнаружив секрет, вы должны немедленно сменить его новым. Если у вас на работе нет инструмента управления секретами, то смена секретов, по крайней мере, гарантирует, что вы испортите праздник тому, кто, возможно, уже украл секрет, и ему придется повторить попытку. Это уменьшает риск, но не сильно. Зато у вас появится шанс блеснуть перед группой безопасности, предъявив им список обнаруженных секретов, и попросить приобрести для вас инструмент управления секретами, чтобы работать с ними более безопасно. Есть надежда, что они согласятся.

Если у вас имеется инструмент управления секретами, то следует сменить все секреты и передать их под управление этого инструмента. Затем измените код, так чтобы секрет запрашивался у инструмента, а не хранился в коде, как раньше. Первое такое изменение потребует некоторых усилий, зато потом достаточно будет просто скопировать и вставить.

Почистив свою кодовую базу (избавив ее от секретов), вы должны будете предотвратить любые попытки снова сохранить секрет в коде. Первым делом сообщите всем о том, что вы сделали, и почему хранить секреты в коде плохо. Затем расскажите, как следует обращаться с секретами, начиная с этого момента (продемонстрируйте работу с инструментом). После того сконфигурируйте точку подключения перед фиксацией для своей системы управления версиями, где будет производиться сканирование кода на предмет наличия секретов и блокироваться любая попытка записать код, который, по мнению сканера, содержит секрет. Лучше перестраховаться: если что-то выглядит как секрет, не разрешайте помещать код в репозиторий. Сообщите всем, что будете блокировать запись кода, потенциально содержащего секреты, чтобы сотрудники, увидев сообщение, понимали, в чем проблема и как с ней бороться. Сделав все это, можете считать себя супергероем на ниве безопасности!

Удалив из кода все секреты и внедрив их программный запрос у инструмента управления секретами, вы значительно снизите риски для своей организации. Об этом можно даже упомянуть в резюме; лично я была бы очень рада узнать, что разработчик сделал это по собственному почину. Не забудьте сообщить об этом своей группе безопасности, они будут очень довольны вами.

Динамическое тестирование (DAST)

Когда я училась на пентестера, первым освоенным мной инструментом стал динамический анализатор Burp Suite Pro. Burp очень хорошо известен, безумно популярен и является весьма высококачественным инструментом. Он также может многое сломать, если вы плохо понимаете, что делаете, и, к сожалению, я сама и два моих клиента узнали об этом на собственном горьком опыте в процессе моего становления младшим пентестером. Поскольку в состав большинства динамических сканеров входит фаззер, они иногда могут случайно причинить вред приложениям и базе данных. Если у вас в штате нет человека, имеющего большой опыт работы с ними, то лучше не

позволять никому использовать такие инструменты против производственной системы. Вообще говоря, динамические сканеры должна использовать группа безопасности, а не разработчики, но однажды я работала в компании, которая создала песочницу с тестовыми данными, чтобы разработчики могли пользоваться динамическими сканерами в свое удовольствие. Так они нашли кучу ошибок.

Динамические сканеры – это программы, которые взаимодействуют с работающим веб-приложением или API и используются для поиска уязвимостей. Чтобы от сканера была польза, приложение должно быть запущено. Сканеру не нужен доступ к исходному коду, иногда этот подход называют *тестированием методом черного ящика*. DAST-сканеры обычно лучше работают с веб-приложениями, чем с API, если только вы не приобрели инструмент, специально «заточенный» для работы с API, или не пользуетесь более продвинутыми возможностями, чем просто активное сканирование.

Обычно у DAST-сканеров имеется несколько функций, но в основном они различаются по тому, способны ли выполнять только автоматическое сканирование или допускают использование для ручного тестирования. Последние часто применяются пентестерами и стоят дешевле полностью автоматических сканеров (нет, я не знаю, почему программа, обладающая более широкой функциональностью, стоит меньше). Те, что используются для ручного тестирования, способны нанести больший ущерб в руках неосторожного пользователя. К DAST-сканерам, допускающим ручное тестирование, относятся и два моих любимых: Zap и Burp.

Если вы решили научиться динамическому тестированию, то крайне важно получить письменное разрешение на тестирование от уполномоченного руководителя. И даже если такое разрешение получено, вы должны также написать документ, в котором точно описано, что вам разрешено, а чего не разрешено тестировать. За попытку атаковать веб-приложение в некоторых странах предусмотрено тюремное заключение. Имейте в виду, что если у вас нет разрешения на тестирование системы, а вы «натравите» на нее динамический сканер, то, в соответствии с законодательством большинства стран, вы *атакуете систему. Пожалуйста, не нарушайте закон*.

Если вы решили, что хотите начать использовать динамический сканер, и планируете больше узнать о динамическом сканировании, то я рекомендую начать с бесплатной программы Zap (Zed Attack Proxy) или Burp Suite Pro (которая имеет очень привлекательную цену по сравнению с другими инструментами DAST на рынке, примерно 2–3 % от стоимости корпоративного DAST). Определившись с инструментом, вы можете либо поэкспериментировать с ним в домашней лаборатории, как делала я, когда учителей, готовых поделиться информацией на эти темы, было не так много, или найти кого-то, кто вас обучит, – так получится гораздо быстрее. Я рекомендую материалы, публикуемые моей доброй подругой Раной Халиль (Rana Khalil), которая выкладывает множество бесплатных уроков на эту тему и прекрасно объясняет. Она научит вас, как создать домашнюю лабораторию, чтобы вы могли безопасно производить тестирование на собственном компьютере, разрешение на которое нужно получать только у самого себя.

Научившись сравнительно уверенно использовать динамический сканер, спросите у группы безопасности разрешения протестировать одно из своих приложений в среде разработки или в тестовой среде. Объясните, что вы многому научились и жаждете найти в своем приложении ошибки и исправить их, – посмотрите, сколько радости будет в их глазах. Но даже если они не засветятся от счастья, есть надежда, что все же позволят вам применить свои знания. Велики шансы, что они с неохотой разрешат вам использовать такой мощный инструмент на работе, поэтому если все же получите разрешение, то будьте чрезвычайно осторожны.

Совет Никогда не «натравливайте» динамический сканер на приложение, которое вам не разрешили тестировать. Это, безусловно, невежливо, а быть может, и незаконно!

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Разумно ли использовать динамический сканер (DAST) против любого приложения в интернете? Объясните свой ответ.
2. Разрешено ли использовать SAST-сканер (статистическое тестирование приложений на безопасность) против любого открытого кода, найденного в интернете? Объясните свой ответ.
3. Своими словами объясните, почему так важно следить за актуальностью зависимостей.
4. Какие плагины IDE уже имеются у вас? Какие вы хотели бы приобрести?
5. Что такое безопасное ограждение? Объясните своими словами и по возможности приведите пример.
6. Своими словами объясните, какова цель анализа кода? В чем его преимущество?
7. Потратьте несколько минут, чтобы составить список одной или нескольких тем, по которым вы хотели бы пройти обучение, включая возможный план достижения этой цели. Поговорите об этом со своим начальником или руководителем группы. Если вы сейчас без работы или работаете там, где обучиться этим навыкам нет возможности, то расскажите приятелю, что вы пытаетесь изучить, чтобы он держал вас в тонусе, помогая не растерять мотивацию.
8. Какие из упомянутых в этой главе инструментов есть у вас на работе: SAST, SCA, сканеры секретов, линтеры, DAST и т. д.? Выясните, если не знаете, а затем спросите у группы безопасности, можете ли вы воспользоваться каким-то из них. Включите свой опыт работы с ними в резюме, это ценится!

Тестирование

В зависимости от опыта работы вы, возможно, уже давно с нетерпением ждали этой главы о тестировании на безопасность или, наоборот, с ужасом ждали, когда до нее дойдет. Лично я предпочитаю думать о пентестерах так же, как о работниках службы контроля качества: они находят столько ошибок, сколько могут, чтобы у меня была возможность исправить их *до* того, как их увидит заказчик. Хотя их отчеты, полные информации об ошибках, добавляют мне работы, но они же помогают избежать неприятностей в будущем, если наши ошибки обнаружит не тот, кому мы доверяем и платим зарплату, а кто-то не столь этичный.

В этой главе будут рассмотрены следующие темы:

- тестовое покрытие и хронометраж;
- ручное тестирование;
- автоматизированное тестирование;
- фаззинг;
- интерактивное тестирование приложения на безопасность (IAST);
- программы вознаграждения за найденные ошибки;
- результаты тестирования.

Изучив эти вопросы, вы будете лучше подготовлены к этапу тестирования своих приложений на безопасность. Вы будете знать, чего ожидать, как готовиться и каковы возможные исходы.

Мы занимаемся тестированием на безопасность по нескольким причинам, и, возможно, вас удивит его ценность. Находя уязвимости на ранних стадиях жизненного цикла разработки системы (SDLC), мы и устранить их можем раньше, а это не только сэкономит нам время и деньги, но и позволит реализовать исправления более высокого качества. Мы тестируем на безопасность, чтобы уменьшить риски организации (что порадует инвесторов), ради комплаенса (удовлетворив аудиторов), для повышения качества (в ходе тестирования на безопасность могут обнаружиться не только уязвимости, но и, например, утечки памяти) и чтобы защитить своих заказчиков, работников и граждан в целом. Занимаясь тестированием на безопасность и устра-

нением найденных ошибок, мы помогаем своей организации поддерживать хорошую репутацию и не терять конкурентные преимущества; в наши дни заказчики задают вопросы о безопасности приобретаемых продуктов чаще, чем раньше. Никто не хочет быть клиентом «той компании, в которой произошла страшная утечка данных».

Тестирование на безопасность также оказывает благотворное влияние на культуру организации, в которой мы работаем. Если все знают, что их приложения будут тестироваться на безопасность, то усерднее работают над тем, чтобы сделать их безопасными. Мы также чаще беседуем на темы безопасности, и если повезет, то это поспособствует формированию более позитивного и предусмотрительного отношения к окружающим.

Регулярное тестирование на безопасность и работа над смягчением последствий ошибок также помогает повысить надежность создаваемого нами программного обеспечения, увеличив время безотказной работы и отказоустойчивость. Чем меньше ошибок, тем меньше крахов системы. Даже если мы решим не исправлять все без исключения найденные проблемы, их документирование поможет нам лучше подготовиться к потенциальному сбою и тем самым защитить всех причастных к делу.

Регулярное тестирование на безопасность – выигрышная стратегия для любой организации, создающей программное обеспечение.

Тестовое покрытие и хронометраж

Под *тестовым покрытием* в тестировании программного обеспечения понимается оценка и измерение степени протестированности ПО. Оно позволяет оценить, все ли критические компоненты, функции и возможности системы были подвергнуты достаточному тестированию, подтверждающему надлежащее функционирование и надежность. Тестовое покрытие важно для выявления протестированных частей и сведения к минимуму риска дефектов в программе.

Тестовое покрытие описывает, что протестировано, а что нет. Оно также говорит о том, как производилось тестирование: какими способами. Например, какие приложения регулярно проверяются автоматическими средствами, а какие подвергаются тестированию на проникновение. Но речь также может идти о том, с каких точек зрения тестируется система (статически, динамически, извне, изнутри, под нагрузкой и т. д.). Важно тестировать с разных точек зрения, а также систему в целом, чтобы обеспечить достаточную безопасность и надежность²⁸⁷.

Если вы собираетесь тестировать систему, то по возможности тестируйте ее целиком – от первого доступа к ней клиента (например, экрана входа в систему) до места хранения данных (обычно это база данных, но может быть и мейнфрейм), а также все системы, с которыми она интегрирована (API, другие приложения, сторонние базы данных и т. д.). Нужно предельно

точно документировать, что именно вы собираетесь тестировать, потому что если в результате ваших действий выйдет из строя что-то не описанное в документе, то вам не поздоровится. Очевидно, что вы не хотите причинить вред организации, но такое случается, и тогда нужно обезопасить себя от юридических последствий.

Выбор между глубиной и широтой покрытия

При определении покрытия вы должны решить, хотите ли углубиться в некоторые аспекты функциональности, уязвимости или части системы (глубина) или предпочитаете, чтобы была протестирована вся система (покрытие)²⁸⁸. Часто времени на то и другое не хватает, если тестированием занимается консультант по найму вроде меня. Я обычно предпочитаю широкое покрытие, а не углубление в какую-то одну часть, если только эта часть не несет в себе критический риск (например, в случае сбоя может умереть человек или раскрыта сверхсекретная информация, что нанесет ущерб государству).

Сканирование инфраструктуры

Принимая решение о тестовом покрытии своего приложения, вы должны определить, будет ли инфраструктура частью тестирования. Обычно контейнер, виртуальная машина или «платформа как услуга» (PaaS) исследуются сканером оценки уязвимостей (vulnerability assessment – VA), который проверяет правильность конфигурации и наличие всех исправлений, но тестирование сети на проникновение встречается реже. Я всегда рекомендую проверять инфраструктуру VA-сканером, даже если вы отвечаете только за безопасность своих приложений, потому что если злоумышленник сможет причинить ущерб организации посредством платформы, на которой размещено ваше приложение, то во всем обвинят все равно вас. По сути же это означает, что следует поручить сканирование кому-то из вашей команды или заняться этим самому (если у вас имеются необходимые права). И по возможности исправьте все проблемы, которые найдете.

Производственная и менее ответственные среды

Хотя самые точные результаты дает тестирование, проведенное в производственной среде, с ним связан риск причинения вреда производственной системе или ее данным. Но если вы следуете правильной практике разработки ПО, то предпроизводственная среда должна повторять производственную во всех отношениях, кроме производительности. Это значит, что тестирование веб-приложения или API должно давать одинаковые результаты (за исключением тестов производительности, нагрузочных тестов и тестов устойчивости к DoS-атакам). Если ваши среды не совпадают, то я рекомендую создать

проект, которые привел бы их в соответствие. Это достойная цель, имеющая большую ценность не только для безопасности.

При тестировании в производственной среде важно создать резервную копию сразу перед началом тестирования базы данных (всегда) и приложения (если только вы не практикуете DevOps и, следовательно, готовы произвести повторное развертывание идеально, быстро и уверенно). Вы должны точно знать, как произвести откат, если в процессе тестирования данные и приложение будут повреждены.

Охват

При описании охвата тестирования на безопасность (тестирования на проникновение или динамического сканирования) явно указывайте, какие в него включены страницы, URL-адреса и IP-адреса, а также все типы пользователей системы (администратор, гость и т. д.). Перечислите все API и другие конечные точки, а также какие еще системы включены в тестирование (например, интернет вещей [IoT] или мобильные приложения). Понадобится также список не охваченного тестами, например конкретных конечных точек, функций и некоторых типов атак (отказ в обслуживании [DoS] и перехват управления учетной записью почти всегда исключаются из сферы тестирования). Укажите, на какое время будет заключен контракт и сколько это будет стоить, если работу выполняет не сотрудник организации. Этот документ должен быть подписан лицом, уполномоченным одобрять такое тестирование, перед тем как оно начнется.

Совет При организации тестирования на проникновение не вычеркивайте то, что, по вашему мнению, может не пройти тестов, поскольку это сводит на нет саму идею тестирования. Такое часто случается, особенно в государственных учреждениях. Штамп, удостоверяющий безопасность, по своей ценности и рядом не лежал с полным пониманием угроз и рисков для ПО. «Оправданные возражения» больше не являются принимаемой судом причиной утечки данных.

Хронометраж

Вы должны выполнять все виды оценки безопасности (статическое тестирование приложения на безопасность [SAST], анализ состава программы [SCA], сканирование секретов, динамическое тестирование и т. д.) на протяжении всего SDLC и после передачи приложения в эксплуатацию, но в самом конце SDLC для совершенно нового приложения следует также провести тест на проникновение. Приложение должно быть максимально близко к выпускной версии, чтобы тестировалось то же, что будет эксплуатироваться, но при этом должно быть оставлено время, чтобы разработчики успели исправить все найденные проблемы.

Если приложение уже эксплуатируется и вы выполняете статическое (не опасное) тестирование (SAST, SCA, сканирование секретов, линтинг), то

можете протестировать (а затем повторить) последнюю версию, когда сочтете удобным.

Если вы выполняете динамические виды тестирования (динамическое тестирование приложения на безопасность [DAST], тестирование на проникновение, тестирование командой красных, фаззинг и т. д.) и приложение уже эксплуатируется, то нужно более внимательно и осторожно подходить к вопросу о том, когда и где проводить тестирование. Например, это могут быть нерабочие часы (часто посередь ночи) или в выходные. Или в середине рабочей недели, начиная с 9:00 утра на непроизводственной версии системы (в предпроизводственной или промежуточной среде), чтобы под рукой были люди, готовые исправить возникшие проблемы. Но при любом типе тестирования сначала нужно создать резервную копию приложения и данных.

Примечание Нанять хорошего и добросовестного пентестера может оказаться сложнее, чем кажется. Понять, обладает ли человек нужными вам навыками или только делает вид, трудно, если не обладаешь этими навыками сам. Но даже если такие навыки у него есть, трудно проверить, что он выполнил всю работу, которую обещал, потому что некоторые прогоняют несколько автоматизированных тестов и ставят в контрактах числа с потолка, думая, что вы не заметите разницы. Увы, это встречается гораздо чаще, чем вы можете предположить, и я призываю вас интересоваться рекомендациями или отзывами о компаниях и индивидуальных пентестерах, прежде чем расставаться с трудом заработанными денежками.

ЭТИЧЕСКИЕ ПРИНЦИПЫ АЛИСЫ

Пару лет назад Алиса встречалась с пентестером. После нескольких бокалов вина, когда они немного узнали друг друга, он гордо заявил, что работает «быстрее» других тестировщиков. Алиса сказала, что вряд ли это можно считать преимуществом, потому что платят ему посуточно, а раз контракт заканчивается быстро, то, наверное, и заплатят ему меньше? Так зачем же работать быстрее? Он объяснил, что ему в любом случае платят за полные 10 дней, а на деле уходит 3 или 4 дня. Алиса была в шоке! «Но это же нечестно», – сказала она, когда ее кавалер попытался пойти на попятную. С тех пор они больше не встречались.

Совет В случае тестирования, основанного на покрытии, тестировщик исследует как можно больше способов эксплуатировать вашу систему. В случае же тестирования, основанного на глубине, он сосредотачивается на одной части системы, но зато ее исследует исчерпывающим образом. Проконсультируйтесь с группой безопасности и выберите тот подход, который кажется вам полезнее. Тестирование на основе покрытия встречается значительно чаще.

Совет Если вы ищете, кого бы нанять для тестирования на безопасность, и вам предлагают очень низкую цену, насторожитесь. При найме квалифицированных специалистов из стран со слабыми валютами результат все же может оказаться хорошим, но может быть и так, что тестировщик просто выполнит автоматическое сканирование и скажет, что выполнил ручное тестирование. Спросите, как он планирует тестировать бизнес-логику приложения и включено ли в контракт моделирование угроз. Поинтересуйтесь, какие методы он планирует использовать и отвечает ли процедура стандарту OWASP ASVS (стандарт проверки безопасности приложения), инструкции OWASP по тестированию

мобильных приложений на безопасность (Mobile Application Security Testing Guide – MASTG) или еще какому-то наставлению. Узнайте, какие инструменты он собирается использовать и как будет проверять результаты тестов. Запросите пример отчета, чтобы оценить, сможете ли вы понять конечный результат и методологии, использованные в процессе тестирования. Если ответы тестировщика на эти вопросы не убедили вас в том, что он хорошо справится с работой, то продолжайте поиски.

Совет Согласуйте все в письменном виде, если собираетесь выполнить потенциально деструктивный тест (примечание: тесты на проникновение, нагрузочные тесты, тесты производительности и тесты на устойчивость к DoS-атакам являются потенциально деструктивными). Обычно я стараюсь не превосходить ситуации, но, хотя на меня никогда раньше не подавали в суд, уверена, что мне бы это не понравилось, и подозреваю, что и вам тоже. Давайте будем избегать этой возможности, тщательно документируя охват тестирования, типы тестирования, инструменты, которыми собираемся пользоваться, временные рамки тестирования, IP- и URL-адреса, и обязательно подписывать документ у того, кто облечен полномочиями одобрять тестирование. По завершении тестирования храните подписанный контракт несколько лет – просто на всякий случай.

Ручное тестирование

Работа опытного пентестера отличается от прогона автоматического сканера, как день от ночи. Хотя автоматические инструменты динамического сканирования способны найти самые разные проблемы, лежащие на поверхности, квалифицированный пентестер найдет все это и много чего еще. Он будет тестировать согласно вашим моделям угроз. Он протестирует бизнес-логику вашего приложения. Он проверит то, что могут и не могут делать пользователи разных типов во всех частях системы. Все это достигается путем ручного тестирования.

Ручное тестирование обычно подразумевает веб-браузер, веб-прокси (например, Burp Suite или Zed Attack Proxy, сокращенно Zap), а если имеются API, то также инструмент для отправки запросов и получения ответов от API (например, Postman, HURL, http-tanker и т. д.). В идеале веб-прокси должен обладать функциональностью для повтора запросов (чтоб тестировщик мог изменять запросы, пока не получит желаемый результат), а также возможностью запускать автоматические скрипты против приложения или API (для атак полным перебором, тестирования проверки входных данных, фаззинга и т. д.).

Приступая к ручному тестированию, опытный пентестер (в отличие от любителя) имеет план. У него есть схема, которой он следует, конкретные тесты, которые всегда выполняются для всех веб-приложений и API, а также тесты, зависящие от языка программирования или каркаса, использованного для разработки приложения. Он попросит вас рассказать о приложении, о моделях угроз (если таковые имеются), об архитектуре, а также показать документацию. Затем он составит подробный план тестирования вашего приложения настолько тщательно, насколько способен или насколько по-

зволюют временные рамки контракта. Часто он использует автоматические инструменты для тех частей, где они могут быть полезны, и ручное тестирование для всего остального. Он напишет отчет, в котором будет объяснено, что было найдено, какие риски влечет каждая обнаруженная проблема и приведены рекомендации по исправлению ошибок.

Любитель обычно использует только автоматические инструменты. Если он и прибегает к ручному тестированию, то зачастую проводит его хаотически, небрежно, создавая риски для приложения и (или) среды, в которой оно размещено. Если он находит уязвимость, то не всегда может объяснить ее или посоветовать, как ее устранить.

Пентестер-любитель, работающий без присмотра со стороны кого-то более опытного, может причинить серьезный ущерб вашим системам. Я это знаю по собственному опыту, потому что, к сожалению, сильно повредила приложения своего клиента во время краткого (полтора года) периода работы пентестером. Помню, как я волновалась, выполняя свой первый тест на проникновение. Мой первый профессиональный наставник по информационной безопасности заставил меня прочитывать половину «The Shell Coders Handbook» и половину «The Web App Hacker's Handbook» и велел за выходные изучить Burp Suite по видео на YouTube. На этом мое «обучение» закончилось. Затем он отдал мне на растерзание живую производственную систему клиента без какого-либо руководства (он обещал, что будет присматривать за мной, но обещания не выполнил). Каким-то образом мне удалось найти подделку запроса на стороне сервера (server-side request forgery – SSRF) и чисто случайно навести шороху в базе данных и на веб-сервере. В свое оправдание скажу, что, продолжая тестировать, я понятия не имела, что перезаписываю файлы на живом сервере и что за мной никто не наблюдает. Я не знала, что предпринятые мной атаки «сработали», пока не стало очень, очень поздно. И хотя клиенту удалось восстановиться с резервной копии, он был крайне недоволен.

Примечание Вы, наверное, обратили внимание, что я не назвала имя моего первого профессионального наставника по информационной безопасности. Это потому, что преподанные им уроки в основном относились к категории «чего не надо делать». Он многому научил меня в сфере этики, только способ избрал сомнительный. Мы общались недолго, и с тех пор я нашла других замечательных наставников, которые рассказали мне о законах, этике и о том, как ничего не сломать даже случайно. Спасибо вам, Адриен (RIP), Шериф, Николь и другие, за то, что открыли мне глаза.

Все это я говорю для того, чтобы вы были предельно осторожны при выборе того, кого нанимаете. Его клиент нанимал не меня, а его, а он взял меня на субподряд. В то время я думала, что клиент знает, что работу выполняю я, но после того, как на меня наорали, стало понятно, что о моем участии клиент понятия не имел, хотя мне было сказано иное. Это означало, что не только мой наставник нарушил контракт, но и что мы оба нарушили закон (я – потому что тестировала систему, не имея на то законного разрешения, а он – потому что предоставил мне доступ). И хотя он подписал со мной контракт,

я все равно могла попасть в беду. Мне никогда не приходило в голову, что он лжет, тем более в письменном виде. Вся моя карьера могла бы сложиться совсем по-другому, если бы клиент подал на него в суд или вызвал полицию.

К счастью, клиент восстановился с резервной копии и ничего не потерял. Данные не были ни уничтожены, ни украдены, просто у кого-то был испорчен день и потеряно время.

Поэтому прошу вас запрашивать отзывы и рекомендации при найме пентестера. Люди, знающие, что делают, имеют клиентов, готовых возносить им хвалу. А если вы не можете найти подходящего человека или не уверены в нем, напишите мне, и я порекомендую того, кому доверяю: Tanya@SheHacksPurple.ca.

ЭТИЧЕСКИЕ ПРИНЦИПЫ БОБА

Находясь на жизненном этапе «Я собираюсь стать пентестером», Боб встречался со многими потрясающими хакерами, некоторые из которых стали его друзьями. Он восхищался людьми, участвовавшими в соревнованиях «Захвати флаг» (CTF) и на последующих встречах, а особенно теми, кто получил контракты вместо него. Со временем он сблизился с человеком, хорошо известным в отрасли. Однажды Боб отправился к нему домой, чтобы потусоваться и заняться хакерством. На крючке у входной двери висело множество именных табличек. Боб спросил: «Разве по окончании контракта табличку не забирают? Я вижу, ты их коллекционируешь». Хозяин гордо пояснил, что все эти 35 табличек относятся к действующим контрактам. Боб был поражен! Как же этот парень ухитряется справляться с таким объемом работы? Тот рассказал, что он приходит, запускает свой сканер, а затем отправляется в следующее место. За день он обходит несколько мест, а потом собирает результаты и поручает помощнику составление отчетов – благо всего-то и надо, что копировать данные в документы Word. С дьявольской ухмылкой он объяснил, что выставляет счет за полный рабочий день, когда запускает сканирование, еще один за копирование результатов сканирования и еще один за все прочие действия. Он рассмеялся и сказал, что может «выставлять счета за 8 рабочих дней в неделю, если правильно разыграет свое карты». А потом насмеялся над своими клиентами и называл их идиотами. Бобу стало так противно! Он тут же решил, что найдет более порядочных людей в качестве учителей, пусть даже этот парень был самой популярной личностью в своем городе. Должен же быть кто-то лучше.

Автоматизированное тестирование

Автоматизированное тестирование на безопасность, пожалуй, изменилось с тех пор, как я начала заниматься безопасностью приложений, но все равно я получаю от него удовольствие. Когда я запускаю автоматический сканер типа DAST или динамический сканер API и вижу, как он что-то находит, во мне по-прежнему вспыхивает радость. Сканеры DAST особенно хороши для монолитных приложений и для быстрого выявления проблем, лежащих на поверхности, но уже не так хороши при работе с API и совсем никуда не годятся для анализа одностраничных приложений (SPA), которые целиком написаны на JavaScript. Динамические сканеры, «заточенные» под API, лучше

находят дефекты в API. И я до сих пор не встретила автоматический сканер, который бы хорошо работал со SPA.

В основе работы динамических сканеров лежит проксирование вашего веб-приложения или API и последующее наблюдение за тем, как вы используете приложение. Они записывают запросы и ответы, а затем отправляют собственные запросы, модифицируя их на основе полученных ответов, в попытке найти уязвимости. Как уже отмечалось, они прекрасно справляются с обнаружением уязвимостей, лежащих на поверхности, хуже – с более сложными проблемами и совсем плохо – с отысканием просчетов в бизнес-логике. Конечно, это мое личное профессиональное мнение на момент написания книги. Уверена, что любая компания, продающая DAST, скажет вам совсем другое. Как бы то ни было, я считаю, что запуск DAST-сканирования и исправление найденных ошибок до передачи в эксплуатацию – стоящее действие. Вместе с тем если вы собираетесь прогонять динамический сканер, то я рекомендую поручить это дело профессионалу или заниматься этим в непроизводственной среде с непроизводственными данными, потому что иногда такие инструменты способны причинить вред. В любом случае предварительно получите разрешение у группы безопасности, поскольку это считается «хакерством», и никто не хочет случайно получить инцидент безопасности.

Совет Нынче ИИ меняется чуть ли не каждый день, и я надеюсь, что поставщики DAST сумеют добавить ИИ в свои инструменты, чтобы сделать их поистине блистательными. Я была бы счастлива увидеть автоматический динамический сканер, который надежно находит логические просчеты так, как это делают пентестеры. Когда-нибудь!

Будучи разработчиком ПО (т. е. весьма квалифицированным и технически подкованным специалистом), вы, безусловно, справитесь с запуском DAST против своих приложений, это просто! Вам понадобится разрешение от группы безопасности (письменное) и безопасное место (например, виртуальная песочница или собственный сервер разработки). После этого нужно будет выбрать инструмент (Zap бесплатен, так что стоит начать с него), указать ему на свое веб-приложение и ждать результатов. Сделав это в первый раз, я нашла кучу ошибок и почувствовала себя очень могущественной! Попробуйте сами и убедитесь.

БОБ ОТКРЫВАЕТ ДЛЯ СЕБЯ ОБРАЗЕЦ ПОВЕДЕНИЯ

Несколько месяцев пообщавшись с людьми на собраниях, Боб встретил удивительного человека, называвшего себя Шериф. Хотя он не шумел и не хвастался, как некоторые другие пентестеры, Боб сразу понял, что этот парень знает свое дело. Не сразу, но Шериф взял Боба под свое крыло, давал советы по построению карьеры, сообщал о возможных чему-то научиться и рассказывал, как зарабатывать деньги, оставаясь порядочным. Боб чувствовал, что этот хакер действительно заботится о том, чтобы его клиенты поступали правильно и чтобы мир стал более безопасным. Боб решил, что нашел для себя подходящий образец.

Фаззинг

Фаззинг – функция, часто встречающаяся в DAST или веб-прокси. Она тестирует проверку входных данных посредством ввода в поле заведомо неправильных данных и анализа реакции системы. Помню, я наблюдала, как Burp Suite начал с ввода одной буквы «А» в поле, затем 50 букв А, а затем 5000. Он продолжал вводить самые разные данные, пока приложение не ответило немного иначе. Это насторожило меня, я начала копаться и поняла, что в алгоритме проверки была небольшая ошибка. Вот так я нашла уязвимость!

В идеале, как вы уже знаете, входные данные должны проверяться по белому, а не по черному списку. Фаззер может разобраться с черным списком за две секунды! Методом проб и ошибок он определит, каких символов нет в черном списке, и тогда вы в беде. Делается это быстро и автоматически. А затем опытный пентестер придумает, как использовать этот список символов, чтобы обойти вашу защиту.

Но фаззеры – наши друзья, потому что показывают, какие ошибки мы допустили при проверке входных данных. Они включены в большинство инструментов динамического сканирования, и я рекомендую пользоваться ими для тестирования всех полей ввода.

РАССТАВАТЬСЯ ТРУДНО

Проводя свой первый самостоятельный тест на проникновение, Боб решил честно отработать все часы, прописанные в контракте, и сделать все по первому разряду. Он собрался подражать Шерифу – быть скрупулезным и добросовестным. Он не только на славу протестировал безопасность, но и ухитрился построить модель угроз, помог клиенту изменить архитектуру приложения, сделав ее более безопасной, и даже составил для него мини-план, как повысить безопасность приложений в дальнейшем. В конце контракта Бобу было трудно расстаться с работой. Он чувствовал, что не все еще сделано, хотя сделал все, что мог в отведенное время.

Интерактивное тестирование приложений на безопасность (IAST)

IAST – сравнительно новая технология (ей около десяти лет), которая тестирует приложение во время работы изнутри. Она устанавливается внутрь приложения в виде двоичного кода; этот процесс называется *оснащением*. Инструмент IAST должен быть совместим с языком и каркасом, т. е. если вы используете что-то неизвестное, то, скорее всего, он работать не будет. Инструменты IAST работают только во время использования приложения, т. е. если вы установите такой на сервер разработки и будете взаимодействовать с приложением в час по чайной ложке, то вряд ли найдете много дефектов.

Зато если вы установите его в производственной среде и запустите во время выполнения теста на проникновение, то с большой вероятностью найдете целый ряд интересных ошибок.

Хотя лично мне никогда не доводилось использовать инструмент IAST в производственной среде, с годами у меня собралось много отзывов клиентов о продуктах такого типа, потому что мне очень интересно знать, чего я лишилась.

И вот какие я получала за эти годы отзывы об инструментах IAST:

- они стоят очень дорого по сравнению с другими инструментами тестирования приложений на безопасность;
- они дают высокий процент истинно положительных результатов и низкий процент ложноположительных;
- они порождают задержку, но очень небольшую;
- для их установки нужно потратить немало времени – которое можно было бы посвятить исправлению ошибок. Если у вас накопилось очень много отложенных проблем, то с IAST вряд ли стоит возиться;
- их ценность проявляется на более поздних этапах SDLC, потому что нельзя рассчитывать на полезные результаты, если приложение используется мало. И сдвинуть применение инструмента IAST влево не получится;
- почти все мои клиенты, использовавшие инструмент IAST, были банками или еще какими-то финансовыми учреждениями;
- обычно такие инструменты применяются в очень зрелых программах безопасности приложений и только для самых ответственных приложений.

Очень немногие из моих клиентов пользуются инструментами IAST (порядка 1 %), но те, что пользуются, похоже, довольны. Судите сами, и надеюсь, что информация, собранная мной за несколько лет, окажется вам полезна.

Программы вознаграждения за найденные ошибки

Еще один способ протестировать приложение – объявить программу вознаграждения за найденные ошибки. К тому же таким образом вы передаете часть тестирования на сторону, исследователям безопасности. Организация, объявившая программу вознаграждения, приглашает исследователей безопасности (известных также как *этические хакеры* или *охотники за вознаграждением*) потестировать копию одной или нескольких своих систем, безопасную относительно тестирования. Охотникам за вознаграждением платят за каждую описанную ошибку, которая (1) не выходит за рамки объявленной программы, (2) допускает эксплуатацию (не просто результат сканера, а де-

факт, создающий реальный риск для бизнеса) и (3) ранее не была описана и не является «известной проблемой». Компания должна заранее четко определить рамки программы, как сделала бы для теста на проникновение, и объявить размер выплат за каждый тип ошибок, порядок предоставления отчета, порядок регистрации в программе и другие детали.

Если вы решитесь объявить о такой программе, готовьтесь к получению *уймы* отчетов. Не все отчеты будут интересны, может быть много шума (ложноположительных результатов, бессмысленных отчетов, плохо написанных отчетов, отчетов, содержащих недостаточно информации, отчетов об ошибках, выходящих за рамки программы, и т. д.). Но вместе с тем вас, возможно, впечатлит, сколько удивительных вещей находят охотники за вознаграждением. Иногда они проявляют исключительно изобретательный и творческий подход к поиску ошибок.

Совет Иногда неопытные или незетичные охотники за вознаграждением атакуют ваших заказчиков, перехватывают управление их учетными записями, а затем пытаются получить выкуп с вас (требуя вознаграждение за возврат учетной записи владельцу) – и все это под предлогом, что они являются «законопослушными охотниками за вознаграждением». Но *перехват управления учетной записью* никогда не входит в рамки программы вознаграждения, и ни одна организация *никогда* не потакает охотнику, атакующему ее клиентов. Кроме того, активные учетные записи клиентов обычно оказываются за рамками программы вознаграждения. Поэтому четко и недвусмысленно укажите, что перехват управления учетной записью выходит за рамки программы, и любой охотник, атаковавший ваших клиентов (любым способом), исключается из программы, а информация о нем передается властям. Это поможет обереечь вас и ваших клиентов от неприятностей.

Я считаю, что, прежде чем задумываться о программе вознаграждения за найденные ошибки, вы должны иметь сильную программу безопасности приложений. Не думаю, что разумно платить 500 долл. за нахождение очевидной ошибки межсайтового скриптинга (XSS), которую вы сами могли бы найти за 10 мин с помощью Zap или Burp Suite. Вы уже должны были просканировать свои приложения разнообразными инструментами и исправить все, чем они недовольны, – до того, как приглашать охотников за вознаграждением. Вы также должны позаботиться о доступности ресурсов, необходимых для проверки присланных результатов (отчетов будет море, и как-то надо их фильтровать), а также выделить разработчикам время для исправления найденных ошибок. Если все это не подготовлено заранее, то максимального эффекта от программы вы не получите.

Некоторые поставщики могут попытаться убедить вас в необходимости начать свою новую программу безопасности приложений с программы вознаграждения за найденные ошибки, называя это «отличным первым шагом». Категорически не согласна. Если вы не уверены, что готовы запустить программу, наймите профессионального консультанта, который поможет вам оценить степень готовности, спросите совета у наставника, которому доверяете, или у людей из сообщества по информационной безопасности. Спрос

не грех, а эта тема так интересна, что любой коллега будет рад обсудить ее с вами – точно вам говорю!

Примечание Я питаю слабость к программам вознаграждения за найденные ошибки. На мой взгляд, если участвуют высококвалифицированные исследователи, а управление программой организовано на высоте, то будет проведено самое интенсивное и замечательное тестирование, какое только можно себе представить. Когда я работала в Microsoft, некоторые ошибки, найденные по программе вознаграждения, были просто невероятными. Но я также считаю, что если вы позволите кому угодно сообщать об ошибках, то будете погребены под завалами никуда не годных отчетов, которые ничего не дают ни вам, ни вашей команде. Я также полагаю, что если программа дурно организована, то вы, возможно, заплатите куда больше, чем необходимо, а желаемых результатов не получите. А напортачить здесь очень легко, поэтому нужна немалая подготовительная работа, чтобы получить эффект от программы. Если у ваших сотрудников нет опыта в проведении такой программы, то я настоятельно советую до ее запуска привлечь консультанта со стороны. Лично я рекомендую компанию Luta Security, но есть и другие хорошо известные в нашей отрасли компании, например HackerOne, Bug Crowd, Synack и др.

Результаты тестирования

Все группы безопасности различаются, а значит, предоставляют результаты тестирования разными способами и в разных форматах. Надеюсь, что ваша группа безопасности рада помочь, готова тратить на вас свое время и предоставляет результаты в информативной форме. Но может оказаться и не так.

В идеале результаты тестирования на безопасность должны приходить туда же, куда приходят все остальные ошибки: в систему отслеживания ошибок. Каждый пункт должен содержать краткую и четкую информацию о том, как воспроизвести ошибку, понятное описание риска, советы по ее исправлению (часто их называют рекомендациями по смягчению последствий) и, быть может, даже примеры кода. Если вам особенно повезло, то вы даже можете получить запрос на внесение изменений, содержащий предлагаемое исправление (а еще лучше, если исправление действительно качественное и не «ломает» ваше прекрасное приложение). Хорошо бы, чтобы была включена информация, которая поможет вам определить срочность исправления, и чтобы все пункты были упорядочены по приоритету. И наконец, в этом идеальном мире кто-то (или *что-то*, например ИИ) уже рассортировал результаты еще до того, как вы их увидели, и проверил, что все результаты настоящие, а не являются ложноположительными, чтобы вам не пришлось тратить время на охоту за призраками, разбираясь с тем, что и проблемой-то не является. Надеюсь, что именно так обстоит дело у вас на работе.

Увы, во многих организациях информация об ошибках безопасности предоставляется иначе. Результаты тестирования могут быть получены следующими способами:

- в ходе формальной презентации, когда пентестер демонстрирует слайды на собрании, где предположительно присутствует много руководителей;
- в виде PDF-файла, присланного вам по почте, без какой-либо дополнительной информации;
- ваш сервер непрерывной интеграции (CI) прерывает сборку, после чего вы просматриваете журналы и находите нечто, что инструмент проверки на безопасность счел ошибкой, и, пока вы ее не исправите (или не подавите проверку), программа в эксплуатацию не пойдет;
- вас пригласили на встречу или вы получили письмо, в котором вас просят зайти на инструментальную панель центра безопасности, а затем среди кучи чужих ошибок найти свои;
- результаты представлены в форме электронной таблицы;
- еще какой-то из мириады неудобных способов, которые никак не стыкуются с вашими обычными процессами разработки.

Если вы получаете ошибки от группы безопасности способом, далеким от идеального, то я рекомендую сообщить им, как вы обычно получаете отчеты об ошибках, и попросить автоматизировать свои процессы, чтобы доставлять их так же.

Действия по результатам тестирования

Предположим, что вы получили список проблем безопасности в одном из приложений, которое разрабатываете или сопровождаете. Хочется надеяться, что в проекте или вашим руководителем зарезервировано время для реагирования на эти проблемы (если нет, попросите зарезервировать). И в пределах этого времени вы хотите решить, что исправлять первым делом. Если повезет, то ошибки уже будут отсортированы по приоритету! А если нет, то можете посоветоваться с кем-то или прочитать описания и самостоятельно решить, какие ошибки выглядят наиболее ужасающе, и начать с них.

Прочитайте рекомендации по смягчению последствий и оцените их разумность! Если вы с ними согласны, отлично! Приступайте к исправлению. Если нет, то можете посмотреть, упомянута ли ошибка в этой книге, открыть соответствующий раздел и начать оттуда. Если этот фокус не проходит, то поищите в сети подходящую шпаргалку OWASP (Cheat Sheet). Если таковой не имеется, то просто задайте запрос, содержащий слово OWASP и название уязвимости. На сайте OWASP почти всегда есть хороший совет.

Написав исправление, которое ничего не «поломало», запустите инструмент, обнаруживший эту ошибку (если у вас есть к нему доступ), и посмотрите, считает ли он ее исправленной. Если да, прекрасно. Объедините исправление с ветвью, над которой работаете. Если нет, повторяйте, пока не получится.

Если у вас нет доступа к инструменту, выдававшему результаты, то поместите исправление в систему управления версиями, как делаете обычно, скрестите пальцы и надейтесь, что все будет хорошо. Лучше, если вы можете протестировать исправление самостоятельно, но не всегда такая возможность имеется.

Я понимаю, что существенно упрощаю ситуацию. Возможно, вам придется долго отлаживать, особенно если проблема расплзлась по нескольким файлам (*межфайловая*). Исправление требует времени, терпения, зачастую кофе, а иногда и чертыханья. Но вы справитесь. И результат стоит потраченных усилий! Спасибо, что вы это сделали. Разработчики, исправляющие ошибки, – это невоспетые герои.

Примечание В индустрии безопасности широко распространено мнение, что разработчики ПО вообще не думают о безопасности, а причина – большое число ошибок в журнале незавершенных задач. Считается, что разработчики *не хотят* исправлять дефекты безопасности. По мнению этих людей, разработчики исправляют их недостаточно быстро, или исправляют недостаточно много, или вообще не исправляют. Лично я не верю, что разработчикам плевать на безопасность. А верю, что у программистов много разных приоритетов и вещей, о которых нужно думать, и безопасность – одна из них (но не *единственная*). Я считаю, что часто им дают недостаточно, а то и вовсе не дают времени на исправление дефектов безопасности. Более того, во многих организациях у них нет никакого стимула исправлять дефекты безопасности, потому что премии, продвижение по службе и повышение зарплаты обусловлены созданием новых функций, а не исправлением дефектов безопасности (я называю это «порочным стимулированием»). В результате группы безопасности часто недовольны разработчиками, а мне кажется, что это недовольство следовало бы адресовать руководству. Но это я отвлеклась.

Предупреждение Не подавляйте диагностику ошибок и не помечайте их как ложноположительные, если на самом деле они таковыми не являются. Если вы не собираетесь что-то исправлять по причине недостатка времени, других приоритетов или еще почему-то, то так и напишите. *Помечая правильный результат как ложноположительный, вы причиняете вред.* Во-первых, вы обучаете инструмент неправильному (поэтому он, возможно, будет пропускать и другие ошибки в будущем, особенно если включает компонент ИИ). Во-вторых, вы искажаете метрики, из-за чего инструмент выглядит непрезентабельно и группа безопасности может подумать, что он плохо работает. В-третьих, и это самое главное, организация не понимает, каков текущий уровень риска приложения. Если вы подавите 20 ошибок, потому что знаете, что не можете их исправить, то все будут думать, что приложение безопасно, хотя на деле оно таковым не является. Лучше знать о небезопасности, чем скрывать это от организации. Группа безопасности также не сможет претендовать на выделение дополнительных ресурсов, если все выглядит так, будто ошибки исправляются. Я предпочитаю иметь точную информацию, даже если она «выглядит плохо», чем иметь неправильное представление об уровне безопасности, которую я должна обеспечивать. Уж сделайте мне одолжение, не помечайте реальные ошибки как ложноположительные результаты. Спасибо.

Заключительные мысли

Если мы не будем тестировать свои системы, то не будем знать, какие риски они представляют для организации, что оставляет нас безоружными. Без тестирования мы не можем быть уверены, что наши меры смягчения последствий, исправления ошибок и защитные механизмы функционируют правильно и что мы действительно защищены. Кем бы вы ни были – человеком, который проектирует или пишет программы, тестировщиком, инженером по безопасности или еще кем-то, – вы обязаны позаботиться о том, чтобы создаваемые или сопровождаемые вами системы были безопасны для пользователей, заказчиков, коллег, граждан и дорогих вам людей. Тестирование – это не просто нахождение новых и новых ошибок, его смысл в том, чтобы гарантировать высокое качество систем, а не может быть высококачественной система, которая небезопасна.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Своими словами объясните, почему охват – важная часть любого контракта на тестирование безопасности.
2. Почему необходимо иметь подписанный документ до начала тестирования на проникновение?
3. Своими словами объясните, чем тестирование на проникновение отличается от запуска автоматического динамического сканирования.
4. Динамическое сканирование потенциально опасно. Это верно или нет?
5. Какой способ тестирования дает самые высококачественные результаты: только автоматическое тестирование или комбинация ручного и автоматического тестирования?
6. Задача фаззинга – убедиться, что мы используем параметризованные запросы. Это верно или нет?
7. Своими словами объясните, почему важно, чтобы пентестеры имели высокие моральные принципы.
8. Своими словами объясните, почему важно, чтобы вы, как ИТ-профессионал (неважно какой специальности), имели высокие моральные принципы.

Выпуск и развертывание

Во время развертывания мы можем продолжать обеспечивать безопасность приложений благодаря изобретению систем непрерывной интеграции и непрерывной доставки (CI/CD), которые автоматизируют процесс выпуска ПО. В этой главе мы обсудим, как добавить гарантии безопасности в эти системы, а также как обеспечить безопасность самой системы CI/CD.

Вне зависимости от того, присутствует ли в конвейере CI/CD ручная проверка и одобрение, все равно это гигантское улучшение по сравнению с ручным процессом развертывания.

Совет Если даже вы не используете CI/CD или сервер сборки для выпуска приложений, не все еще потеряно! Вы по-прежнему можете многое сделать, чтобы обеспечить безопасность собираемого ПО. Вы можете выполнять тестирование до (статическое тестирование приложения на безопасность [SAST], анализ состава программы [SCA], сканирование секретов, линтинг) и после (динамическое тестирование или интерактивное тестирование приложения на безопасность [IAST]) компиляции и развертывания. Затем вы можете вручную сгенерировать контрольную сумму откомпилированного артефакта и сохранить ее вместе с библиотекой кода на случай, если придется доказывать целостность сборки впоследствии. Далее в этой главе мы поговорим о том, как укрепить и мониторить систему CI и приложения (о них в следующей главе), чтобы предотвратить попытки подменить приложение модифицированной версией.

КОМПЬЮТЕРНАЯ ГИГИЕНА СПЕШИТ НА ПОМОЩЬ

Как-то Боб работал с командой, которая развертывала все вручную. Он был потрясен тем, что команде из 18 человек потребовалось *2 недели*, чтобы развернуть новую версию своего приложения. И они делали этого каждые три месяца, как того требовал их подход «мини-водопада» к разработке ПО. Каждый раз при развертывании возникали проблемы. Версия, проверенная отделом контроля качества, не совпадала с развертываемой в производственной среде, промежуточная среда падала, к базе данных пользовательского приемочного тестирования (UAT) не удавалось подключиться – все было безумно «напряжно». В конце концов Боб выяснил, что администратор базы данных (DBA) вручную писал скрипты для каждой среды и что заранее они не тестировались. Он также обнаружил, что никаких скриптов отката не было и в помине, поэтому если DBA допускал ошибку, то он же и должен был с ней «разбираться», что часто являлось причиной задержек.

В течение следующих 90 дней Боб внедрял те же самые формальные процессы для изменений базы данных, что и для остального кода, – с применением системы управления версиями, тестирования и т. д., – не обращая внимания на постоянные жалобы, слова о том, что он «принимает слишком близко к сердцу», и обвинения в перфекционизме. Следующий мини-водопадный выпуск занял 3 дня, без серьезных ошибок. Во всех последующих выпусках на протяжении всего проекта Боб обращал особое внимание на точность, тестирование и воспроизводимость и никому не позволял ничего делать вручную. Каждый следующий выпуск занимал меньше времени, чем предыдущий, и проблем было все меньше. Быть может, Боб и не снискал особой популярности в команде в начале проекта, но при нем проект перешел из состояния «безнадежно отстаем от графика» в «поставлено с большим запасом» всего за один год. Боб очень гордился своей командой и переменами, которые они совершили вместе.

События безопасности в системе CI/CD

Предполагается, что конвейер CI/CD должен работать *быстро*. То есть речь идет не о часах, а о минутах. Добавляя тестирование на безопасность в такую систему, очень важно проследить за тем, чтобы оно было быстрым, точным и надежным. Это значит, что если ваш инструментарий известен выдачей ложноположительных результатов, то эту работу лучше выполнять вне автоматизированного процесса вывода. Кроме того, если тестирование окажется слишком медленным, то, возможно, стоит его автоматизировать, например выполнять ночью, а не в составе конвейера. Мне не раз встречались команды, которые с самыми лучшими намерениями включали сканер SAST первого поколения в систему CI/CD только затем, чтобы стать свидетелями бесчисленных уведомлений и прерываний сборки из-за ложноположительных результатов; естественно, команду это крайне раздражало. Это создает помехи, непроизводительную трату времени и денег и часто заканчивается отключением тестов безопасности. Учитесь на прошлых ошибках и тщательно проверяйте инструмент, прежде чем добавить его в CI. Затем покажите его нескольким разработчикам, послушайте их отзывы и внесите соответствующие корректировки.

В конвейер CI можно включить практически любой тип автоматизированного тестирования, но не все из них рекомендуются. Я предлагаю исключить или ограничить следующие типы тестов.

- Динамические автоматические сканеры (DAST) обычно занимают слишком много времени, особенно если нужно распространить их на каждую ссылку, к которой обращается приложение. Вместо этого запускайте сканер по расписанию вне конвейера и выполняйте только пассивное сканирование (анализ запросов и ответов без фаззинга и отправки любых запросов) или включайте лишь особо важные тесты, например аутентификации, защиты от внедрения кода и т. д., а все остальные отключите.

- Фаззинг может работать очень медленно, а иногда вносит неразбериху. Применять его нужно только против тестовых данных (или при наличии резервной копии), и лучше бы выполнять его вне конвейера.
- Статические анализаторы первого поколения, производящие символическое выполнение кода, могут работать долго и порождать много ложноположительных результатов. Подумайте об их запуске вне конвейера или только для некоторых типов тестов, например защиты от внедрения кода, XSS и других критических уязвимостей. Сканеры SAST второго поколения (поиск антипаттернов) обычно работают в составе конвейера без нареканий.
- Нагрузочные тесты и тесты производительности всегда должны запускаться вне конвейера. Точка. На время их выполнения вам нужны все доступные ресурсы, чтобы знать, на что способно приложение. Дополнительные сведения о нагрузочном тестировании и тестировании производительности см. в главе о тестировании книги «Alice and Bob Learn Application Security».

Прерывание сборки

Я работала со многими группами безопасности, которые с нетерпением ждали «прерывания сборки», надеясь, что это изменит поведение разработчиков, заставит их уделять больше внимание безопасности приложений и вообще решит кучу проблем. К сожалению, прерывание сборки не является чудодейственным средством.

Под *прерыванием сборки* понимается остановка конвейера CI/CD, занятого выпуском версии (все равно, на каком этапе) после непрошедшего теста. Обычно это делается, чтобы избежать случайного попадания очень серьезной уязвимости или дефекта в производственную среду. Мы не прерываем сборку по незначительным причинам или из-за какой-то старой уязвимости. Мы делаем это не для того, чтобы кого-то наказать или пристыдить. Это аварийная мера, похожая на дерганье шнура Андон на физической сборочной линии. Если вы прерываете сборку много раз в день, то *поступаете неправильно*.

В идеале ваш жизненный цикл разработки системы (SDLC) должен включать несколько связанных с безопасностью действий, которые выполняются еще до начала CI/CD, поэтому код, который вы пытаетесь выпустить, находится в хорошем состоянии, прежде чем будет запущена CI. Проверки в конвейере CI/CD оставлены на случай, если вы что-то пропустили раньше. Это не первый ваш подход к безопасности, а зачастую последняя проверка перед сдачей в эксплуатацию.

Совет Включая новый инструмент в свой конвейер, вы всегда должны начинать с режима уведомления и мониторинга. Когда-нибудь вы дойдете и до режима блокировки, но не сразу. Так вы сможете понаблюдать за инструментом в реальной ситуации, не нанося ущерба остальным тщательно настроенным инструментам, уже присутствующим в конвейере.

Совет Если вы постоянно прерываете сборку, имейте в виду, что это не только не эффективно, но и не способствует приобретению друзей. Попробуйте «сдвинуть безопасность влево», т. е. начинать тестировать на более ранних стадиях SDLC. Инструменты безопасности есть и в руках разработчиков, а не только группы безопасности.

Сканирование секретов

Сканеры секретов ищут секреты в вашем коде. В идеале в качестве первого шага вы должны просканировать всю кодовую базу и убрать из нее секреты, а обращаться к ним только из программы, запрашивая у инструмента управления секретами. После этого нужно создать точку подключения перед фиксацией, в которой сканирование кода производится перед записью его в репозиторий; запись должна быть запрещена, если секрет найден. Таким образом, вы сможете избежать инцидента безопасности (сохранение секрета в систему управления версиями означает его утечку, и в этот момент возвращаться уже поздно – придется менять секрет и начинать все с начала). Если точка подключения перед фиксацией имеется, то нет нужды снова сканировать секреты в конвейере CI/CD – в предположении, что CI всегда извлекает код непосредственно из репозитория, и вы не делаете ничего странного, например копирования кода откуда-то из другого места, чтобы прогнать его через CI.

Статический анализ

Инструменты SAST второго поколения производят поиск антипаттернов, поэтому работают очень быстро (от нескольких секунд до нескольких минут, а не от нескольких часов до нескольких дней). Они предназначены специально для работы в среде CI или IDE, и обычно все, что они находят, правильно. Но иногда случаются и ложноотрицательные результаты (они что-то пропускают), поэтому не следует ограничиваться только одним типом тестирования. На момент написания этой книги не существует одного типа инструмента безопасности, который делал бы все, что вам нужно.

Динамический анализ

Я уже говорила, что не являюсь горячим сторонником включения инструмента DAST в конвейер CI/CD, потому что он выполняет *очень много* работы. Вы можете заранее каталогизировать свой сайт* через HTML-архив (HAR) или уменьшить охват (например, тестировать только на 10 типов уязвимостей или только очень малую часть приложения), но все равно это будет медленно, а результаты окажутся не очень полезны. Кроме того, традици-

* Под каталогизацией сайта понимается сбор данных о взаимодействии между браузером и сайтом. – Прим. перев.

онные инструменты DAST склонны к генерированию большого количества ложноположительных и ложноотрицательных результатов, а значит, если конвейер работает в режиме блокировки, а не уведомления, то сборка будет часто прерываться. Я все же хочу использовать инструменты DAST, но обычно, находясь в CI, делаю это только в пассивном режиме (смотрю на запросы и ответы, но не посылаю собственных запросов), а чаще вообще не включаю их в конвейер. Я предпочитаю использовать DAST вне конвейера либо вручную (когда нужно скрупулезное тестирование, включающее ручное), либо в автоматическом режиме – ночью или в нерабочие часы запускается «полный прогон» (тестируется все подряд, хотя это занимает много времени). Результаты я изучаю позже, на досуге.

Анализ состава программы

SCA отлично подходит для CI/CD, поскольку работает очень быстро. Я уже говорила, что предпочитаю инструменты SCA, которые могут сообщить, является ли уязвимость в библиотеке достижимой или допускающей эксплуатацию, и надеюсь, что скоро это будут делать все инструменты SCA, а не только некоторые. Я бы всегда включала инструмент SCA в конвейер CI в предположении, что имею лицензию.

Линтинг

Хотя, строго говоря, линтинг больше относится к качеству, а не к безопасности кода, я все же считаю такие инструменты очень полезными. Я предпочитаю запускать линтинг из IDE, потому что, будучи фанатиком контроля, хочу исправить все недочеты, прежде чем кто-то другой получит шанс взглянуть на мою работу. Я отношусь так к любому инструменту, даже когда это не слишком разумно. Линтер – прекрасный кандидат для включения в CI, но такая проверка может быть реализована и в точке подключения к системе управления версиями перед фиксацией или объединением. Решайте, что вам больше нравится.

Сканеры инфраструктуры как кода

Хотя технически мы отвечаем только за приложения, мы хотим также быть уверенными, что инфраструктура, обслуживающая наши приложения, находится в достойном состоянии. Если для ее создания мы используем технологию инфраструктуры как кода (IaC), то можем использовать сканер IaC для проверки безопасности всех принятых решений. Они работают быстро, обычно дают точные результаты (по сравнению с SAST и DAST) и предлагают полезные советы (исправить здесь, изменить конфигурацию там и т. д.).

Совет

Примерами IaC служат Terraform, шаблоны Azure ARM и CloudFormation.

В отличие от сканера оценки уязвимостей (vulnerability assessment – VA) сканеру IaC не нужны для анализа работающий контейнер или виртуальная машина; он может читать любой язык, используемый в вашем файле описания IaC (часто YAML).

P.S. Обращайтесь с файлами описания IaC так же, как с кодом: помещайте в систему управления версиями, тщательно тестируйте и т. д.

Обеспечение безопасности самого конвейера CI/CD

В этом году OWASP выпустил дополнительные списки «Главных десяти», и один из них называется «The CI/CD Top Ten». Это список главных 10 рисков системам CI/CD, и о некоторых из них я расскажу в этом разделе. Список большой, вы можете скачать его по адресу <https://owasp.org/www-project-top-10-ci-cd-security-risks/>.

Как уже было сказано ранее в этой книге и во многих книгах по DevOps, система CI/CD имеет невероятные возможности и привилегии; попав не в те руки, она может причинить неопиcуемый вред. Критически важно, чтобы доступ к таким системам, их конфиденциальность, целостность и доступность строго контролировались и охранялись.

Первое, о чем нужно позаботиться при работе с CI/CD, – управление доступом: кто что может изменять и как отслеживать изменения (по-другому это называется аутентификацией, авторизацией, протоколированием и аудитом; OWASP #1, #2 и #10)²⁸⁹. Вы же не хотите, чтобы необученный простак мог вносить изменения и выпускать код, ни с кем не согласовав, как не хотите, чтобы это делал человек с дурными намерениями; оба могут создать огромные проблемы вам, вашей организации и вашим приложениям. Ограничивайте доступ, включайте аудит и мониторинг и часто проверяйте, что доступ имеют только те, кому он необходим по должности. При увольнении сотрудника не забывайте отзывать его права. Позаботьтесь о том, чтобы в систему в качестве технических мер предосторожности было встроено наличие согласований, необходимых в соответствии с политиками организации (принудительно останавливайте сборку, пока не будет получено одобрение). Кроме того, не разрешайте сотрудникам использовать свои личные адреса электронной почты для доступа к рабочим системам, в том числе и CI²⁸⁹.

Второе, на что следует обращать внимание при работе с CI/CD, – целостность используемых пакетов и зависимостей (OWASP #3). Часто мы пользуемся загруженными из разных мест сторонними компонентами для сборки своего приложения, но при этом не проверяем контрольные суммы или подписи или по ошибке скачиваем не те версии. Важность проверки правильности версий всех компонентов становится все более очевидной с каждой новой успешной атакой на цепочки поставок. Кроме того, не следует брать

образы или пакеты из ненадежных источников, таких как Docker Hub, куда любой желающий может поместить все что угодно для всеобщего пользования. Получайте компоненты из надежных источников. Если ваш поставщик облачных услуг предлагает автоматизированное сканирование образов как часть пакета, воспользуйтесь этим! Подвергайте аудиту полный путь прохождения пакетов, образов и всего прочего, что включаете в свой проект, – должна проверяться целостность каждого элемента.

Совет Docker Hub, GitHub, Stack Overflow и многие другие онлайн-платформы коллективного пользования – прекрасные места для обмена идеями и обучения. Но надо понимать, что любая платформа, позволяющая любому желающему загружать код, изображения, библиотеки и прочее, подразумевает, что найденные там материалы потенциально опасны и их следует рассматривать как не заслуживающие доверия, пока не доказано противное. Скачанное с таких сайтов используйте в своей рабочей среде с особой осторожностью и применяйте все действующие в организации политики.

Все тесты, скрипты, make-файлы и все, что выполняется как часть конвейера, должны быть доступны только лицам, облеченным доверием, перед запуском должны проходить проверку целостности и храниться в системе управления версиями²⁸⁹. В идеале эти файлы должны храниться отдельно от остального кода проекта, поскольку велика вероятность, что всей команде редактировать их нет нужды. В таком случае имеет смысл завести подкаталог с другими правилами идентификации и управления доступом.

Отделите права доступа к самому конвейеру от прочих сред. Часто разработчики запрашивают больше прав, чем необходимо, – на всякий случай, а затем забывают отказаться от лишних прав. Если ваш конвейер работает от имени учетной записи службы, которой были даны чрезвычайно широкие права, то злоумышленник, перехвативший управление этой записью, сможет использовать ее во зло. Он получит и доступ к секретам! Давайте каждой части своего конвейера CI только те права, которые ей действительно необходимы. Например, можно сделать так, чтобы вся команда разработчиков могла одобрять выпуск версий в среде разработки и контроля качества, но ничего сверх того, или чтобы руководитель команды мог вносить изменения в любую часть конвейера, а остальные члены делать этого не могли. Раз в год или чаще проводите аудит прав доступа²⁸⁹.

Если конвейер локальный, а не облачный, то продумайте надежный план резервного копирования. Я видела, как люди теряют свои конвейеры, а на повторное конфигурирование уходят недели. Сердце разрывается от перспективы потерять код, а конвейер и есть код!

При работе с секретами важно не только защитить их от попадания в исходный код, но и то, как организованы доступ к ним и их использование в системе CI/CD. Проверьте, что они не утекают (например, не сохраняются в журналах CI и не показываются в сообщениях об ошибках), не используются неправильно и не находятся в открытом доступе. Если вы видите (или хотя бы подозреваете), что секреты утекли, смените их немедленно²⁸⁹.

Одним из пунктов в любом списке «Десяти главных» OWASP неизменно является неправильная конфигурация безопасности, и к CI/CD это относится в той же мере, что к любой другой системе²⁸⁹. Инвентаризация системы, ее укрепление, ежегодный аудит, применение принципа наименьших привилегий, тестирование на проникновение и т. д. – все это должно применяться не только к создаваемым вами приложениям, но и к конвейеру CI/CD. Эта фантастически мощные системы, поэтому их необходимо держать под замком (буквально). Следуйте инструкции по укреплению.

Большинство организаций подключают свои конвейеры CI/CD к сторонним системам, зачастую продуктам типа «Программное обеспечение как услуга» (SaaS). Эти продукты нужно не только запирать на замок, но и документировать, и у вас должен быть доступ к управлению ими. Хотя подключить полезный инструмент к своему конвейеру DevOps соблазнительно, вы должны убедиться в том, что он правильно сконфигурирован, развернут всюду, где должен, что у вас имеется нужное количество лицензий, что он запускается всякий раз, как вы думаете, что он должен быть запущен, что для инструмента установлены политики, соответствующие принятым в вашей организации, и т. д. Даже самый замечательный инструмент можно использовать ненадлежащим образом. Кроме того, должны существовать процесс согласования для добавления новых инструментов, система, которая делает все инструменты наблюдаемыми (в идеале вы еще и метрики должны собирать), политики и соответствующий ей процесс вывода из эксплуатации и мониторинг использования (для того чтобы быть уверенным, что вы получаете то, за что заплатили, а также чтобы случайно не нарушить лицензионное соглашение)²⁸⁹.

Вы можете подписывать не только свой код, но и все артефакты, порожденные конвейером (отчеты, журналы, изображения). Тогда проверка подписей (или контрольных сумм) на каждом шаге конвейера поможет обеспечить дополнительную защиту от атак против системы CI²⁸⁹.

Последний момент, который я хочу предложить вашему вниманию, не входит в список «Главных десяти угроз CI/CD» OWASP, а касается неконтролируемого потребления ресурсов. Например, кто-то случайно запустил создание 3000 изображений вместо 30 или решил прогнать конвейер тысячи раз, хотя это никак не обосновано. В технических системах устанавливать лимиты необходимо при любой возможности. Очень легко задать лимиты неоднозначно или не задать вовсе, а потом получить кошмарный счет от поставщика облачных услуг.

2365 ПРОГОНОВ КОНВЕЙЕРА В НЕДЕЛЮ!

Алиса заключила контракт с большой компанией, которая потратила два года, чтобы внедрить у себя DevOps. Кто-то показал ей суперские новенькие конвейеры, которые они создали. Они позволили Алисе пощелкать мышью и посмотреть, как они работают. Алиса задержала курсор над именем конвейера, и всплыло имя; Алиса спросила, что оно значит. Гид сказал, что данный конвейер принадлежит этому человеку. Тогда Алиса щелкнула по имени, и появился длинный список, содержащий много конвейеров. Мелькнула надпись

«Страница 1 из 133». Алиса сказала: «Но это значит, что этому человеку принадлежат тысячи конвейеров. Зачем столько? Сколько вообще должно быть конвейеров у человека? Вам не кажется, что это *чересчур*?» Гид скорчил гримасу. После многочасового расследования оказалось, что Алиса случайно ткнула на сотрудника, который создавал конвейеры для забавы и занимался этим больше года – за счет компании. Все его 2365 конвейеров запускались каждую неделю, что обходилось компании во многие тысячи долларов. Сотрудника уволили, а компания с того дня ввела строгий аудит своих систем.

Поддержание целостности сборки

Я понимаю, что в предыдущем разделе было немало сказано на эту тему, но хочу немного углубиться в эту конкретную часть SDLC. Какую бы методику вы ни применяли – DevOps, гибкую разработку, водопад или еще что-то, – способность доказать целостность выпущенного вами ПО поможет заслужить доверие заказчиков и собственной группы безопасности. А заодно предотвратит инциденты безопасности! Ниже перечислено несколько вещей, которые помогут вам (1) гарантировать, что выпущено именно то, что вы хотели выпустить, и (2) доказать это.

- Подписывайте своей цифровой подписью версии ПО и (или) пакетов данных. Особенно это относится к сериализованным данным²⁹⁰.
- Берите библиотеки и сторонние зависимости только из надежных репозиториев. Подумайте о размещении внутреннего репозитория «заведомо хороших» (проверенных) компонентов²⁹⁰.
- Используйте инструмент анализа состава программы (SCA), который проверяет, что компоненты не содержат известных уязвимостей, а если такие уязвимости есть, то они не допускают эксплуатации в вашем приложении. Если это не так, переходите на исправленную версию. Относитесь к этому так же, как к уязвимостям в собственном коде: исправляйте²⁹⁰.
- Выполняйте анализ и рецензирование кода. Ищите вредоносный код и уязвимости²⁹⁰.
- Заведите процесс анализа изменений конфигурации IDE, репозиториев кода, песочниц, конвейеров и вообще всего, что используется для создания вашего ПО²⁹⁰.
- Следуйте всем рекомендациям, приведенным в этой книге ранее.
- Не отправляйте и не принимайте сериализованные данные из ненадежного источника. Если все-таки получаете данные из ненадежного источника или отправляете ему, то обязательно шифруйте и подписывайте их. Проверяйте подпись перед десериализацией²⁹⁰.

БЕЗУМНЫЕ СЧЕТА ОТ ПОСТАВЩИКА ОБЛАЧНЫХ УСЛУГ

Коллега Алисы в прошлом семестре нанял студента-стажера, который был настоящим вундеркиндом. Его так впечатлили таланты нового сотрудника, что он дал ему полную свободу действий в отношении конвейера CI/CD и спустя всего две недели позволил ему выпускать приложение в эксплуатацию без предварительного одобрения. Через несколько недель Алиса зашла в кафетерий и увидела коллегу, обхватившего голову руками. «Что стряслось?» – спросила Алиса. Он объяснил, что вундеркинд случайно добавил два лишних нуля перед развертыванием на платформе Kubernetes, и он только что получил сюрприз – счет за облачные услуги на 30 000 долл.! Он не знал, как сказать об этом начальнику, их бюджет и так уже был в минусе. Нет, только не это!

Одобрение выпуска группой безопасности

Часто группа безопасности хочет ознакомиться с версией до ее передачи в эксплуатацию. Иногда это выливается в несколько шагов, написание бумаг и проведение встреч, а иногда достаточно краткого обсуждения или несколько писем по электронной почте. Все зависит от политик группы безопасности, склонности вашей организации к риску, законодательства и требований комплаенса, а также от количества сотрудников группы безопасности (иногда они и предпочли бы более длительный или углубленный процесс, но для этого просто не хватает персонала). Если вы хотите передать код в эксплуатацию как можно раньше, то я рекомендую изучить, какие действия необходимо предпринять для получения одобрения, и приступить к ним на возможно более ранних этапах SDLC. Если необходимо моделирование угроз, то запланируйте его. Если требуется статический анализ, узнайте, можно ли получить доступ к инструменту, чтобы вы могли сделать это заранее и исправить все найденные недочеты. Тогда вы без труда и с честью пройдете проверку группой безопасности. Если нужно заполнить анкету или составить документ по архитектуре безопасности, то приступайте к работе прямо сейчас. Хотя вряд ли это самая любимая вами часть проекта, но если вы раньше начнете, это не затормозит работу в дальнейшем.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Почему нужно обеспечивать безопасность конвейера CI/CD? В чем цель? Что плохого может случиться?
2. Зачем нужно подписывать код цифровой подписью? Что этим достигается?

3. Зачем нужно проверять целостность сторонней библиотеки? Что плохого может случиться, если мы скачаем не то, что ожидаем, а затем опубликуем в составе нашей программы?
4. Почему так важно управлять доступом к конвейеру CI/CD?
5. Назовите два вида тестирования, которые следует включить в конвейер CI/CD.
6. Назовите один вид тестирования, который не следует включать в конвейер CI/CD.
7. Какой ваш любимый вид тестирования на безопасность? Почему именно он?

Сопровождение

Очень часто, работая с уже давно присутствующими на рынке компаниями, я обнаруживаю, что основное внимание группы безопасности приковано к новым приложениям в ущерб унаследованным или ранее написанным. Мне приходится тратить много времени, объясняя, почему нужно выделять время, ресурсы и деньги на обеспечение безопасности унаследованных приложений. И это нелепо, потому что если взглянуть на уровень безопасности программного обеспечения в любой зрелой (старше 5 лет) организации, то именно унаследованные, а вовсе не новые приложения являются основной причиной риска для бизнеса. Я не хочу сказать, что унаследованных приложений больше, чем новых, и, следовательно, в них больше ошибок. Я имею в виду, что старение программного обеспечения происходит некрасиво с точки зрения безопасности. С каждым годом наша отрасль относится к безопасности все серьезнее, поэтому мы создаем более безопасные приложения, большинство языков и каркасов становятся безопаснее с выходом каждой новой версии, народ постепенно просвещается в вопросах безопасности по мере новых взломов. Но маловероятно, что старые приложения подвергались такому тщательному тестированию, как новые. Кроме того, чем дольше существует программа, тем больше времени было у злоумышленников для нахождения проблем с ее безопасностью, причем это относится не только к написанным нами продуктам, но и к библиотекам и прочим компонентам, которые в них используются. Это значит, что даже если когда-то давно мы все сделали идеально, компоненты со временем могли деградировать. С учетом всех этих факторов старое программное обеспечение обычно менее безопасно, чем новое, если только организация не отдает приоритет безопасности унаследованных систем, но такое мне встречалось редко.

Работая в области безопасности приложений, я всегда старалась уделять половину времени обеспечению безопасности унаследованных систем, несмотря на неодобрение такой позиции в некоторых организациях, где я работала. Я настоятельно рекомендую вам распределять свое время таким же образом, если есть возможность. Я понимаю, что не всегда вы сами решаете, над чем работать, но если у вас есть пространство для маневра, то уверяю вас, что самые старые приложения могут оказаться тем местом, где положительный эффект от вашей работы будет максимальным.

В этой главе мы поговорим о том, как сопровождать систему, чтобы она старела *красиво*. Мы хотим, чтобы наша программа продолжала служить целям бизнеса в долгосрочной перспективе безопасно и стабильно. Но чтобы так и было, нам придется поработать!

Примечание В этой книге под *унаследованной системой* я понимаю программу, которая эксплуатируется год или более. Обычно компании проводят тестирование на проникновение и другие действия по обеспечению безопасности перед передачей программы в эксплуатацию, но потом уделяют этому гораздо меньше внимания. Это значит, что со временем программа, не получающая достаточного сопровождения, становится менее безопасной. Хотя срок «год после выпуска» выбран мной совершенно произвольно, я считаю, что после годичной эксплуатации абсолютно необходимо подвергать системы повторному тестированию.

КОНФУЗ С ПРИОБРЕТЕНИЕМ

Компания Алисы в прошлом году приобрела небольшую компанию-конкурента. Сделано это было, чтобы заполучить клиентов конкурента, но также и потому что одно веб-приложение, написанное конкурентом, было популярнее аналогичного приложения компании Алисы. И компания Алисы очень хотела иметь функциональность, которая так нравилась заказчикам. Когда Алиса приняла компанию под свое крыло, группа безопасности настояла на сканировании всех ее приложений на наличие уязвимостей. Через неделю они пришли к ней взбешенные. Они сказали, что статический анализатор нашел в главном приложении, ради которого Алиса и купила компанию, 43 000 уязвимостей!!! Алиса не понимала, как такое возможно. Она высказала предположение, что инструмент мог соврать. Группа безопасности признала, что использовала статический анализатор первого поколения, поэтому ложноположительных результатов могло быть много, но они начали проверять их вручную и уже нашли 10 возможностей внедрения кода. Они сказали, что исправление приложения обойдется в сотни тысяч долларов или же придется переписывать его с нуля. Они назвали его «катастрофическим риском для бизнеса» и что ей надо было бы предварительно предоставить им возможность оценить его в рамках процедуры оценки актива. Алиса изменила процедуры таким образом, чтобы всегда включать тщательное тестирование безопасности важных приложений компаний, которые она будет приобретать в будущем.

Мониторинг, уведомление и наблюдаемость

Под мониторингом программы понимается реализация систем, позволяющих наблюдать и проследивать операции и действия пользователей, приложений, инфраструктуры, сетевых служб и всех прочих систем, взаимодействующих с нашей программой²⁹¹. Это может относиться к размещенным в облаке веб-приложениям, которыми пользуются миллионы, или к Windows API на сервере, используемом только внутри компании. Мы мониторим программу, чтобы собирать метрики и таким образом удостовериться, что все работает, как ожидалось, а также обнаруживать проблемы, в том числе с безопасностью²⁹².

Уведомление безопасности – это извещение или предупреждение, отправляемое, если в компьютерной системе или сети обнаружена предполагаемая или явная угроза²⁹³. Мы хотим, чтобы наши программы отправляли уведомления, обнаружив проблему, чтобы мы могли как можно быстрее разобраться или отреагировать на нее для ограничения ущерба. Мы также хотим, чтобы журналы, создаваемые нашей программой, отправлялись в информационно-аналитическую систему безопасности (SIEM), если таковая имеется в организации для мониторинга и уведомлений сверх уже имеющихся в наших системах.

Существуют системы мониторинга, которые можно купить и установить, и, откровенно говоря, они чертовски полезны (и не только с точки зрения безопасности!). Они помогут увеличить время безотказной работы приложения, их быстродействие и обеспечат визуализацию данных о производительности в режиме реального времени. Они помогут понять, насколько приложение удобно пользователям, и оценить его текущее состояние. Благодаря им вы узнаете, имеет ли место инцидент безопасности или обычный ИТ-инцидент. Примечание: они не умеют распознавать все типы инцидентов, но все равно очень полезны.

Иногда продукты для мониторинга называются средствами обеспечения *наблюдаемости*, но между ними есть различие, что бы там ни говорили маркетологи. Под *мониторингом* понимается сбор данных и генерирование отчетов о различных метриках, определяющих состояние системы. Задача обеспечения *наблюдаемости* – исследование взаимодействия систем и собранных в результате мониторинга данных с целью найти истинную причину проблем²⁹⁴. Если вы можете получить доступ к инструменту обеспечения наблюдаемости в дополнение к средствам мониторинга и уведомления, то это будет весьма полезно. Спросите у группы безопасности, есть ли у них такой инструмент, и если да, то можете ли вы «понаблюдать» за порученными вам приложениями. Возможно, вы будете удивлены тем, что увидите, количеством угроз, с которыми ваши приложения сталкиваются ежедневно.

Наличие всех этих систем выделит вас среди большинства других компаний-разработчиков ПО как ребят, которые превыше всего заботятся о доступности, безопасности и качестве.

Говоря о стратегии уведомления, наблюдаемости и мониторинга, мы должны уделить первостепенное внимание трем важным аспектам. Это:

- *журналы*: подробные сведения о событиях и действиях вашей системы для прослеживания истории и порядка происходящего в системе;
- *метрики*: результаты измерения производительности системы, например количество пользовательских транзакций, обновлений базы данных, потребления процессора и памяти и т. д.;
- *трассировка*: распределенная трассировка, которая прослеживает прохождение запроса или транзакции через различные компоненты и службы внутри приложения (включая вызовы API, бессерверные системы и т. д.).

Без протоколирования мониторинг толком невозможен. Без протоколирования и мониторинга невозможны точная трассировка и наблюдаемость. Каждый слой основывается на предыдущем, чтобы мы точно знали, что происходит в наших приложениях.

МОНИТОРИНГ – ПУТЬ К ПОБЕДЕ

Когда Боб жил своей недолгой хакерской мечтой и постоянно посещал собрания, он как-то отправился на семинар по внедрению SQL-кода. У двух парней, которые проводили семинар, имелось заведомо уязвимое приложение, и они продолжали атаковать его с помощью внедрений, но уведомление, которое они ожидали получить от поставщика облачных услуг, все не появлялось. Боб спросил: «А протоколирование и мониторинг у вас включены?» – и они ответили, что нет. Он объяснил, что поставщик «не видит», включены ли эти два режима. Они включили их и повторили демонстрацию, и на этот раз долгожданное всплывающее окно появилось. Боб почувствовал себя очень умным! Когда позже Боб спросил у них, почему они не включили протоколирование и мониторинг, они ответили, что это «слишком дорого». Боб рассказал им о ценности этих служб. Он также выяснил, как можно точно настроить такие параметры, как объем и срок хранения, чтобы в будущем сократить расходы.

Блокировка и экранирование

Есть возможность надстроить над вашими приложениями уровень абстракции, через который будут проходить все запросы (до получения приложением) и ответы (после отправки приложением). Идея в том, что этот дополнительный уровень улучшит защиту, поскольку будет действовать как экран для ваших систем. Такие уровни могут блокировать атаки на ваши приложения или другие потенциальные проблемы. Примерами могут служить брандмауэры веб-приложений (WAF), системы самозащиты приложений на этапе выполнения (RASP), сети доставки содержимого (CDN), шлюзы API, сетки служб, каркасы объектно-реляционных отображений (ORM), параметризованные запросы (элемент безопасности внутри кода технически не уровень абстракции, но все-таки экран) и даже что-то доморощенное. На рис. 14.1 показана архитектура с несколькими уровнями защитных абстракций, или экранов.

Примечание Под *виртуальным латанием* понимается добавление регулярного выражения к звену сети (например, к WAF, RASP или CDN), которое сопоставляется с образцом тревожащей вас атаки, чтобы такие атаки блокировать. Если некоторая уязвимость, например Log4j, эксплуатируется во многих системах по всему миру и ваша организация обеспокоена этим, то применение виртуальной заплатки даст вам дополнительное время для реагирования, обновления или исправления своих систем. Она буквально может стать спасательным кругом.

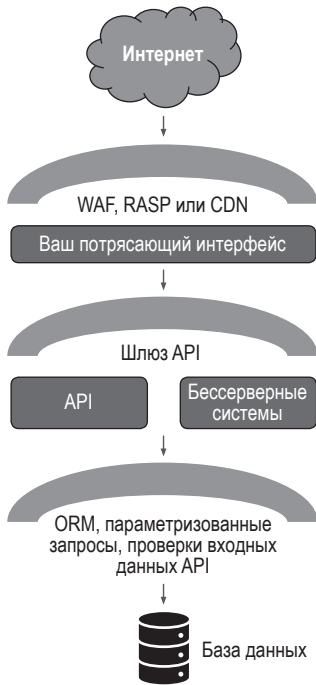


Рис. 14.1. Потенциальные уровни абстракции

Брандмауэры веб-приложений (WAF)

WAF создает экран между веб-приложением и интернетом; это помогает смягчить последствия многих распространенных атак²⁹⁵. Обычно WAF устанавливается на балансировщик нагрузки или какую-то другую конечную точку (но не на веб-сервер). Он добавляет небольшую задержку и анализирует запросы и ответы веб-приложения. WAF может работать в режиме уведомления или блокировки, т. е. соответственно сообщать о том, что видит что-то подозрительное, или останавливать прохождение запросов и ответов. Можно устанавливать на WAF более или менее строгие правила, но чем они строже, тем вероятнее возможность заблокировать добропорядочные запросы, так что необходимо соблюдать баланс. WAF может блокировать не только плохие запросы приложению, но и плохие ответы (например, если вы собрались было вернуть все содержимое своей базы данных, то WAF может признать такой ответ неприемлемым и заблокировать его). Имеется также возможность ставить виртуальные заплатки.

Сети доставки содержимого (CDN)

CDN – это территориально распределенная группа серверов, которая кеширует содержимое, размещая его рядом с конечными пользователями, чтобы ускорить его доставку; это могут быть HTML-страницы, JavaScript-файлы,

изображения и видео²⁹⁶. CDN также могут помочь в фильтрации (блокировке) вредоносного трафика, смягчении последствий DoS-атак, защите от других киберугроз, выполняя роль прокси безопасности между пользователями и сервером²⁹⁷. Хотя безопасность не основное их назначение, CDN предлагают дополнительный эшелон обороны. Иногда они предлагают также виртуальное латание, хотя обычно только для высокоприоритетных уязвимостей, активно эксплуатируемых на практике.

Самозащита приложения на этапе выполнения (RASP)

RASP – технология обеспечения безопасности приложений, в которой для обнаружения и блокировки компьютерных атак используется оснащение инструментальными средствами, работающими во время выполнения, т. е. задействуется информация, полученная *внутри* работающей программы²⁹⁸. RASP устанавливается внутрь приложения (этот процесс называется *оснащением*), поэтому работает в контексте, сильно отличающемся от большинства других систем экранирования. Такие системы приводят к небольшой задержке, защищают приложение изнутри и зависят от технологии (они должны быть совместимы с языком и каркасом). Их установка сложна и подчас занимает много времени, но результаты отличаются высоким качеством. Кроме того, они допускают виртуальное латание.

Виртуальное латание

Виртуальное латание (его еще называют *экранированием уязвимостей*) – мера защиты от известных уязвимостей, активно эксплуатируемых в сети. Принцип работы основан на реализации уровней политик и правил безопасности, которые предотвращают и перехватывают эксплойт, не давая ему добраться по сети до уязвимости. Проще говоря, виртуальная заплатка – это одно или несколько регулярных выражений (REGEX), используемых как черный список, отфильтровывающий некоторые входные данные. Он блокирует потенциальные эксплойты, запрещая все, что выглядит как атакующий код. Виртуальное латание применяется, если имеется высокий риск, но еще нет официального исправления от поставщика или применение этого исправления занимает много времени. Виртуальная заплатка может служить вам дни, недели, месяцы или даже годы, если она достаточно хороша, а риск достаточно велик.

Шлюзы API

Шлюз API – это программный элемент или служба, предоставляемая поставщиком облачных услуг, которая предназначена для публикации, сопровождения, мониторинга и обеспечения безопасности API на основе REST,

HTTP и веб-сокеты²⁹⁹. Шлюз предлагает ограничение темпа, дросселирование, квотирование ресурсов, аутентификацию, авторизацию, а иногда даже проверку входных данных. Лично я никогда не открываю API в интернет, не поставив перед ним шлюза; среди прочего они останавливают ботов и атаки полным перебором. Они упрощают управление сложной архитектурой и делают ее безопаснее. Многие шлюзы API можно использовать и с бессерверными приложениями в качестве дополнительного эшелона обороны.

Примечание Хотя мы уже обсуждали каркасы ORM для баз данных и параметризованных запросов, хочу заметить, что это примеры уровней абстракции, защищающих наши программы и данные от угроз.

Каждый дополнительный уровень усиливает защиту, т. е. мы имеем *эшелонированную оборону* в действии. Хотя ни один уровень абстракции не считается идеальным, все они уменьшают риск. Но при этом увеличивают сложность, время и затраты. Спросите у группы безопасности приложений, какие уровни имеет смысл реализовывать для каждой системы. Примечание: проверка входных данных и использование хранимых процедур не являются факультативными уровнями, они необходимы всегда!

Замечание специально для исследователей данных

Если вам необходимо регулярно запрашивать производственную базу данных или озеро данных, то можете поставить перед базой API или шлюз API, включающий проверку входных данных, ORM и хранимую процедуру. Это добавит управление доступом, поможет применить принцип наименьших привилегий, добавит протоколирование и аудитопригодность, предотвратит внедрение кода и т. д. Если вам кажется, что добавить все это перед базой данных непросто, то вы правы. Но уверяю вас, что серьезный инцидент безопасности, случайное повреждение ценных данных или публичная утечка данных окажутся во много раз болезненнее. Я проводила обучение в компаниях, занимающихся исследованием данных, и обнаружила, что они часто не следуют лучшим практикам разработки ПО. Призываю вас принять все уроки, приведенные в этой книге, близко к сердцу, а особенно этот раздел об уровнях абстракции и экранировании – он станет отличной отправной точкой, если вы со своей командой прямо сейчас запрашиваете производственные данные без страховочной сетки.

Совет Есть много статей о том, как обойти защиту со стороны брандмауэра веб-приложений, хранимых процедур и т. д., но эти меры все равно существенно уменьшают количество атак, которые видит ваше приложение, сводя их на нет на более ранних этапах. Они повышают вашу безопасность, хотя ни одна из них не дает абсолютной защиты. Вместе с тем *никогда* не следует рассматривать экран или уровень абстракции как замену безопасному кодированию; гарантии безопасности и надежности приложения всегда должны иметь высший приоритет. Все остальное, включая экранирование, не более чем подспорье.

CDN СПЕШИТ НА ПОМОЩЬ!

В 2021 году во время Рождества на мир обрушилась кошмарная уязвимость Log4j. К счастью, компания, где тогда работала Алиса, находилась за CDN, которая очень быстро приняла действия и развернула виртуальную заплату для всех своих клиентов. У Алисы гора с плеч свалилась! Она слышала, что ее знакомые в других компаниях запаниковали, не зная, что делать, тогда как CDN, обсуживавшая ее компанию, обо всех позаботилась. Алиса пометила себе, что нужно будет возобновить контракт, и спокойно отправилась праздновать Рождество!

Непрерывное тестирование

Ответственность за приложение мы несем и после его выпуска. Мы должны и дальше гарантировать его безопасность, надежность и стабильность. Для этого нужно регулярно тестировать его, потому что, как мы знаем, программное обеспечение, старея, теряет гордую осанку.

Говоря о *непрерывном тестировании*, я не имею в виду, что программу нужно тестировать ежеминутно, это вовсе не обязательно. Но это должно происходить значительно чаще, чем раз в год. Насколько чаще, зависит от риска, с которым готова смириться организация. Если речь идет об особо ответственной системе, которая сталкивается с известными вам серьезными угрозами и содержит чрезвычайно конфиденциальную информацию, то тестировать ее стоит очень часто. Если же система не особенно важна, не содержит конфиденциальных данных и вы не знаете о каких-то существенных угрозах для нее, то можно тестировать и реже.

Тестирование не обязательно означает тест на проникновение. Это может быть еженедельное сканирование инструментом анализа состава программы (SCA), ежемесячный прогон динамического анализатора (DAST) и статического анализатора (SAST) с последующим устранением всех найденных проблем. После крупных изменений (например, кардинального рефакторинга или добавления новой функциональности) можно провести тестирование на проникновение и моделирование угроз. Настроить прогон инструментов SAST и SCA для еженедельного сканирования всего репозитория кода очень просто. Сложнее выделить время для анализа результатов и решения о том, что имеет смысл исправлять, поскольку это может оказаться весьма трудоемким занятием. Но в этом сможет помочь ваша группа безопасности приложений.

Смысл непрерывного сканирования (тестирования) в том, чтобы всегда иметь точное представление о степени безопасности своих систем. Никто не может позволить себе еженедельно проводить тестирование на проникновение для всех систем. И, откровенно говоря, я думаю, что это было бы пустой тратой денег. Но вы *можете* запускать автоматические инструменты очень часто, что позволит получить представление о текущем состоянии.

Я рекомендую как минимум прогонять SAST и SCA для репозитория кода каждую неделю. Эти виды сканирования ничего не могут повредить, а взглянув на приложения под двумя разными углами, вы сможете найти, а затем и исправить разнообразные проблемы с безопасностью.

ТРИУМФ ИНТЕРОПЕРАБЕЛЬНОСТИ

Все знают, что Боб *обожает* автоматизацию и умные устройства. Дома у Боба каждое утро автоматически запускается кофе-машина, у него установлены автоматические замки с видеокамерой, чтобы он знал, «кто стучится в дверь ко мне», а на работе он автоматизирует все, что может. Покупая инструмент разработки или обеспечения безопасности, Боб всегда проверяет, может ли он работать с IDE компании, средствами непрерывной интеграции и доставки (CI/CD), программой отслеживания ошибок и репозиторием кода – это минимум. Если инструмент несовместим, то Боб его не купит. Если вы не можете что-то автоматизировать, то зачем оно вам? Это девиз Боба.

Инциденты безопасности

ИТ-инцидент безопасности – это любая удавшаяся или неудавшаяся попытка получения несанкционированного доступа, использования, раскрытия, модификации или уничтожения цифровой информации или информационных систем. Сюда входит вмешательство в работу информационных систем и нарушение политик организации³⁰⁰. Примеры инцидентов безопасности: кража данных, доступ злоумышленника к чужим учетным записям или данным, а также преднамеренное лишение доступа к системе (отказ в обслуживании). Это может быть также простое удаление важного файла пользователем или случайное выкладывание конфиденциальных данных в общий доступ. Действие необязательно должно быть совершено с умыслом причинить вред, чтобы считаться инцидентом безопасности.

В ходе расследования инцидента группа реагирования (incident response – IR) может попросить у вас помощи. Обычно это связано с тем, что вы являетесь экспертом по атакованной системе, но иногда вас могут попросить написать скрипт, получить для них какие-то данные или предоставить справочную информацию.

При взаимодействии с группой реагирования соблюдайте как минимум следующие правила.

- Все ее просьбы рассматривайте как имеющие высший приоритет. Это аварийная ситуация, так к ней и относитесь. Если ваш начальник хочет, чтобы вы занялись чем-то другим, попросите его связаться с руководителем расследования инцидента, пусть они договариваются между собой.
- Следуйте политике «кому нужно, тот знает». Не обсуждайте ничего связанного с инцидентом ни с кем, кроме группы расследования, если не

получали явного разрешения. Если начальник будет расспрашивать вас о деталях, отсылайте к руководителю расследования. Если информация не нужна человеку срочно для выполнения своей работы, то ничего ему не сообщайте. Пусть руководитель расследования решает, кому и что нужно знать. Простого *желания* знать недостаточно, даже если это ваш начальник.

- Если заметите что-то подозрительное, сообщите группе безопасности. Это правило действует всегда, а не только во время расследования инцидента. Если что-то увидели, сообщайте, даже если не вполне уверены. Группа безопасности не станет ругать вас за ложную тревогу, но будет безмерно благодарна, если вы действительно обнаружили что-то вызывающее тревогу.
- Если вас включили в группу расследования инцидента, сообщайте о том, что вам нужно отлучиться или покинуть рабочее место на ночь. Оставьте номер телефона, по которому с вами можно связаться. Не исчезайте просто так, вы можете понадобиться.
- Не «геройствуйте», пытайтесь расследовать инцидент самостоятельно. Всегда сообщите о нем группе безопасности и действуйте под их руководством. Они имеют необходимую подготовку и опыт, поэтому справятся с ситуацией лучше кого-либо другого. Они знают, как обращаться с уликами, так чтобы ничего не испортить и не сделать их непригодными для судебного разбирательства (акт приема-передачи улики). Они также давали согласие на определенные риски, когда подписывали контракт при приеме на работу, чего вы не делали. Необученный человек, берущийся за расследование инцидента, может причинить огромный вред; не делайте этого.

Если вам повезет, то вас никогда не позовут принять участие в расследовании инцидента безопасности, а ваша компания не станет жертвой серьезного взлома. Но если такое все же случится, то следование этим правилам очень поможет группе реагирования в ситуации, когда они особенно нуждаются в вас!

ЛЮБОВЬ КАНАДЦЕВ К ФИГУРНОМУ КАТАНИЮ

Алиса подружилась с Фэйт из группы управления рисками и комплаенсом, и однажды вечером, когда они сидели в баре, узнала, что та когда-то занималась расследованием инцидента. Расследователи обычно не склонны рассказывать о своей работе, но одним случаем Фэйт все же поделилась с Алисой. Она расследовала инцидент безопасности много лет назад, когда сотрудник службы поддержки попытался разрешить проблему самостоятельно. Ее позвали в большой зал, где собралось все высшее руководство компании, охваченное паникой, и там она узнала, что в одном из корпусов обнаружена вредоносная программа. Фэйт знала, что этот корпус не подключен к интернету и не оборудован умными устройствами, поэтому вредоносу туда попасть было неоткуда. Сотрудник службы поддержки сообщил руководству, что в здании пропал исходящий интернет, потому что там обнаружена вредоносная программа, что все они в опасности и что «он знает, о чем говорит». Ему поверили, хотя это не имело ни малейшего смысла. Группа Фэйт запустила Wireshark, и уже через 10 мин стало ясно, что все запросы идут на один сайт, транслирующий борьбу

за золотые медали по фигурному катанию на Олимпийских играх, в которой Канада принимала участие. Алиса заметила: «Ну мы же знаем, как канадцы любят фигурное катание». Фэйт заблокировала сайт, вывела событие на большой экран в переговорной, чтобы всем было видно, а затем попросила всех «прекратить трансляцию чертовой Олимпиады и забивать интернет-канал». Чтобы со всем разобраться, понадобилось примерно 20 мин. Но вот что касается парня из поддержки... Фэйт еще много недель пришлось снова и снова объяснять всей компании: «Нет, не было никакого вредоноса», «Да, я уверена» и «Само собой, Канада должна была получить золото»; она тоже не могла поверить в то, что Канада завоевала только серебро. К концу рассказа Алиса и Фэйт покатались со смеху.

Непрерывность бизнеса и планирование аварийного восстановления

Непрерывность бизнеса – это способность организации продолжать доставку товаров или услуг на заранее определенном приемлемом уровне после аварийного инцидента. Планирование непрерывности бизнеса (ВСП) означает создание систем предотвращения и восстановления для устранения потенциальных угроз компании с последующим их документированием. Помимо предотвращения, целью является обеспечение непрерывности операций до и во время аварийно-восстановительных работ. Непрерывность бизнеса является предполагаемым результатом надлежащего выполнения как планирования обеспечения непрерывности бизнеса, так и аварийного восстановления.

— «Википедия»³⁰¹

Определение Тани: ВСП помогает организации вернуться к нормальному функционированию бизнеса после аварии. ВСП распространяется на весь бизнес, включая персонал, места его работы и инструменты, необходимые ему для выполнения работы. Например, в ВСП может рассматриваться вопрос о том, как быстро доставить новые компьютеры персоналу, если несколько машин вышло из строя из-за перепада напряжения, как настроить временную программу работы из дома и какие существуют альтернативные каналы получения платежей от клиентов, если система процессинга банковских карт долгое время не работает.

Аварийное восстановление (Disaster Recovery – DR) – это способность организации восстановить доступ к ИТ-инфраструктуре и ее функционирование после аварии, природной или вызванной действиями человека (включая ошибку и вредоносные действия). DR считается подмножеством обеспечения непрерывности бизнеса, ориентированным на приведение ИТ-систем, поддерживающих критические функции бизнеса, в работоспособное состояние как можно скорее после аварийной ситуации.

— Google³⁰²

Определение Тани: в плане аварийного восстановления подробно описано, как привести ИТ-системы в работоспособное состояние (как минимум) после аварии. Например, может быть описано, как будут работать люди (в цифровой сфере), если обычное место работы (и, стало быть, сеть ИТ) недоступно из-за наводнения, или как они будут публиковать написанный код, если поставщик облачных услуг обанкротился.

Ваши приложения должны быть частью планов DR и BCP. Создаваемые вами системы очень важны, и (обычно) без них бизнес работать не может. Они необходимы для предоставления услуг и продуктов и даже для функционирования самой компании. Без них большинство организаций не может работать нормально, поэтому приложения должны быть охвачены обоими планами на случай аварийной ситуации.

В планах должны быть учтены не только сами приложения, но и системы, которые используются для сборки и сопровождения этих приложений. Серверы CI/CD, репозиторий кода, все песочницы или тестовые среды – вообще все, что необходимо вам для работы, должно быть включено в план восстановления, чтобы вы могли вернуться к работе в максимально сжатые сроки.

Совет Советы по DR и BCP: подумайте о каждом инструменте, который используете в повседневной работе, – чем вы могли бы его заменить, если бы его не стало? Например, вы, возможно, используете для общения с коллегами Slack или Teams. Было бы неплохо, если бы все вы установили Signal или другое приложение со сквозным шифрованием, а затем обменялись контактами, чтобы можно было продолжать общение, если обычный ваш мессенджер окажется недоступным. Быть может, стоило бы сохранить все артефакты развертывания в резервном месте (например, в другом облаке или на другом сервере на территории предприятия), чтобы при необходимости можно было быстро произвести повторное развертывание. Планируйте альтернативные способы выполнения всех наиболее важных функций своей команды – в итоге получится хороший план.

Если вы старший разработчик, менеджер или даже руководитель группы, то, возможно, вам интересно место вашей команды в планах BCP и DR. Вы хотите быть уверены, что ваши активы в них учтены, чтобы команда могла как можно скорее приступить к работе после аварии.

Примеры аварий: попадание молнии в здание, наводнение, отключение электроэнергии, программа-вымогатель, пожар или наводнение у поставщика облачных услуг, в результате чего все серверы (и ваш в том числе!) отключены на целую неделю, эпидемия или война. Все, что может нарушить или нанести ущерб основам ведения бизнеса, является основанием для приведения в действие планов BCP и DR.

Совет Вы, возможно, заметили, что в начале эпидемии COVID-19 в 2020 году некоторые компании не сбавляли темпов, в то время как другим потребовалось много времени, чтобы вернуться к нормальной работе. Успешными оказались те, кто лучше спланировал обеспечение непрерывности бизнеса. Снимаю шляпу перед всеми, кто умеет хорошо планировать.

МОЛНИЯ БЬЕТ ДВАЖДЫ

Боб прослушал подкаст о том, как центры обработки данных Azure пострадали сначала от молнии, потом от пожара, потом от наводнения, а напоследок в них снова ударила молния – и все в один день. Это была мощнейшая буря. Никто не пострадал, а гость подкаста подробно описал, как они почти мгновенно перенаправили весь трафик в другие ЦОДы, а затем меньше чем за неделю вернули поврежденный ЦОД к жизни. Боб был поражен! Он всегда думал, что Microsoft снаружи выглядит довольно впечатляюще, но это уже походило на чудо. Оказывается, что Microsoft подходит к DR и BCP очень серьезно и не ограничивается планированием; она еще и проводит практическую проверку планов. И хорошо, что так делает! В тот день Боб шел на работу, охваченный желанием как можно скорее довести свой BCP до конца.

Упражнения

Ответы на эти вопросы вы найдете в приложении В «Ответы к упражнениям».

1. Почему ваше ПО должно быть включено в план аварийного восстановления (DR)? Чем вы рискуете, если это не будет сделано?
2. В чем разница между аварийным восстановлением и непрерывностью бизнеса?
3. Объясните смысл фразы «кому нужно, тот знает» своими словами. Также объясните, какое отношение она имеет к реагированию на инциденты безопасности.
4. Назовите одну или несколько причин, по которым мы должны продолжать тестирование своего ПО даже после развертывания.
5. Зачем нашим приложениям один или несколько уровней абстракции или защиты? Почему имеет смысл иметь больше одного уровня?
6. Должны ли исследователи данных следовать лучшим практикам безопасного кодирования? Объясните свой ответ.

Заключение

Хотя это и заключительная глава, мы продолжим изучать в ней новые вещи. Я хочу напоследок поделиться несколькими мыслями, привычками и идеями, чтобы подтолкнуть вас в том направлении, которое пыталась задать на протяжении всей книги, – в сторону становления разработчика, ответственно относящегося к безопасности. Вам не нужно переходить в группу безопасности (хотя они, конечно, могли бы воспользоваться вашей помощью), чтобы внести большой положительный вклад в работу организации. Просто одолжите эту книгу коллеге, когда прочитаете до конца, – и уже это может изменить чей-то подход к программированию. Давайте окунемся в мир безопасного кодирования вместе.

Хорошие привычки

В своей первой книге «Alice and Bob Learn Application Security» я рассказала о нескольких хороших привычках, которые любой технический специалист, на мой взгляд, должен сделать частью своей жизни. В этом разделе я повторю их и добавлю несколько новых, специально для разработчиков.

- Всякий раз, как вы ищете в сети, как сделать что-то в своей программе, ищите самый безопасный способ сделать это. Возможно, это потребует немного больше времени, но ваш код станет гораздо безопаснее. «Решение», за которое голосует большинство на многих онлайн-форумах, обычно оказывается наименее безопасным. Отказ от копирования в свою программу кода, получившего большинство голосов, но при этом ни разу не безопасного, сильно повысит качество вашего кода.
- Включайте многофакторную аутентификацию (МФА) для всех учетных записей, представляющих для вас ценность. Банки, государственные услуги, медицинские учетные записи, электронная почта, учетные записи, в которых хранятся номера ваших банковских карт и все, что необходимо для работы, относится к этой категории – включайте МФА. Это самое важное, что вы, как конечный пользователь, можете сделать для

защиты от подстановки учетных данных и кражи данных. Пожалуйста, сделайте это немедленно.

- Установите менеджер паролей, а затем смените пароли ко всем важным учетным записям на случайные и уникальные значения, сгенерированные менеджером. Начиная с этого момента, используйте его для многофакторной аутентификации, чтобы упростить себе жизнь. Наконец, установите плагин менеджера паролей во все свои браузеры. Настройте автоматическое резервное копирование хранилища паролей; после его заполнения эта возможность окажется очень полезной.
 - Примечание: наверное, не стоит устанавливать менеджер паролей на свой телефон, если вы храните в нем вопросы МФА, потому что если телефон будет скомпрометирован, то злоумышленник получит все. Быть может, в менеджере паролей на телефоне имеет смысл хранить только самые часто используемые пароли, а не все.
 - Возможно, имеет смысл завести второй менеджер паролей для хранения вопросов МФА, защищенный другим паролем.
 - Два предыдущих пункта – дополнительные меры предосторожности. Лично я не использую ни ту ни другую. Вы поступайте, как знаете.
- Зарегистрируйте все свои почтовые адреса на сайте HaveIBeenPwned.com (если ваш менеджер паролей уже не предоставляет аналогичную функциональность). Он уведомит вас, если узнает, что ваши данные были украдены. Подчас уведомления приходят поздно (сайт может сообщить, только когда узнает, а иногда между кражей и получением сведений о ней проходят годы), а то и не приходят вовсе (не все украденные данные сливаются в сеть). И все же это очень полезная служба. Если вы получили уведомление от нее, то смените пароль немедленно и проверьте, не было ли несанкционированного доступа к вашей учетной записи. Снимаю шляпу перед Троем Хантом, Шарлоттой и остальными членами команды HaveIBeenPwned.
- Используйте парольную фразу или ключ доступа (см. следующий пункт) для доступа к менеджеру паролей. Парольная фраза – это фраза, известная только вам. Не используйте хорошо известные фразы, например слова из песен или знаменитые цитаты. У меня в качестве парольной фразы фигурирует шутка целиком (вопрос и ответ). Она очень длинная, но я теперь могу печатать быстро и никогда ее не забуду. Из-за длины фразы компьютеру понадобится очень, очень много времени, чтобы подобрать ее. Не используйте что-то простенькое для входа в свой менеджер паролей – чем длиннее, тем лучше. Это в буквальном смысле ключ к вашему онлайн-королевству.
- Подумайте об использовании ключей доступа. Они не требуют ввода пароля и просты в использовании. Вы можете хранить их в менеджере паролей. Все, что облегчает обеспечение вашей безопасности, – благо.

- Не используйте пароли повторно. Это очень простое решение, и преступники о нем знают. Расскажите людям, с которыми вы общаетесь, о менеджерах паролей и о том, почему использовать пароли повторно не следует. Так вы, возможно, спасете кого-то, кто вам дорог, от ужасной судьбы.
- Получите доступ к инструментам безопасности и запускайте их для своего кода, даже если группа безопасности их не предоставляет. Надеюсь, впрочем, что в вашей организации группа безопасности прекрасная – предоставляет поддержку, инструменты, лицензии и т. д. Если же нет, то берите дело в свои руки. Есть немало бесплатных инструментов, которые помогут вам писать лучший код (и пройти очередное тестирование на проникновение).
- Подумайте о вступлении в сообщества, занимающиеся безопасностью, например OWASP, «Let's Talk Software Security» и др., готовые принять под свое крыло разработчиков ПО и специалистов по безопасности. Мало того что вы узнаете много нового, так еще и встречаете удивительных людей. Со многими из своих ближайших друзей я познакомилась в различных сообществах и горячо рекомендую вступить в них.
- По возможности избегайте технического долга. Почти всегда он является и долгом безопасности, а это проигрышная игра. Всеми силами старайтесь вовремя устанавливать обновления, переходить на новые версии, управлять зависимостями и т. д. Знаю, что это скучно, но очень важно и ценно.
- Следите, чтобы ваша команда вела точный и своевременный учет всех веб-активов, за которые вы отвечаете. В идеале хорошо бы иметь перечень всего, что есть в организации, но, скорее всего, вы можете контролировать только свою команду. Заносите в список, где можно найти тот или иной код, работающие приложения, различные среды со своими базами данных, контактные данные и т. д. – все, что, по вашему мнению, может оказаться полезным. Этот документ будет полезен не только разработчикам из вашей команды, но и группе безопасности.
- Вот совсем уж очевидный совет: не оставляйте машину разблокированной, когда отлучаетесь, не устанавливайте на свой рабочий компьютер программы-вымогатели и изображения, скачанные из интернета, не заходите на сомнительные сайты с предупреждениями и не используйте свой рабочий компьютер в неблагоприятных целях. Подозреваю, вы все это и так знаете, но не могу не упомянуть.
- Думайте о том, какую информацию о себе вы оставляете в сети. Не пости- те фотографий своего рабочего пропуска, свою дату рождения, свой домашний адрес и т. д. Не делитесь ни с кем компрометирующими цифровыми изображениями, если не готовы в один прекрасный день встретить их в интернете. Будьте особенно осторожны с фотографиями и видео себя, которые размещаете на приложениях для знакомств или отправляете людям, с которыми только собираетесь познакомиться. Если они могут быть когда-то в будущем использованы против вас, например для

шантажа, то подумайте еще раз (а лучше не раз), прежде чем делиться ими с людьми, которым не полностью доверяете (в том числе о наличии у них технических знаний и возможностей для защиты этих данных).

- Помните, что вы суперпользователь, а потому желанная мишень для злоумышленников. Разработчики ПО *могущественны*. Каждый день с утра до вечера вы творите нечто из ничего. Человек, имеющий зуб на вашу компанию, был бы рад получить доступ к вашим учетным записям, поэтому предельно серьезно относитесь к защите своих рабочих устройств, учетных записей, данных и всего прочего.
- Если не уверены, проконсультируйтесь с группой безопасности. Спрос не грех, а заботиться о том, чтобы вы выполняли свою работу безопасно, – их прямая обязанность. Как человек, испытывающий трудности при обращении за помощью, говорю вам – поступайте так, потому что знаю, что многие думают «Я и сам справлюсь с этим» или «Не хочется их беспокоить». Вот не надо так думать в этом случае. Они специалисты, которых наняли специально, чтобы помогать вам, так заставьте их отработать свою зарплату.
- Относитесь с подозрением ко всему, что кажется слишком хорошим, чтобы быть правдой. Это может быть пришедшее по электронной почте сообщение, в котором вам предлагают бесплатную поддержку библиотеки с открытым исходным кодом, или еще что-то. Если это происходит ни с того ни с сего, попробуйте проверить, воспользовавшись другим источником.

БОБ ПОЛЮБИЛ СВОЙ МЕНЕДЖЕР ПАРОЛЕЙ

Боб раньше практиковал «повторное использование паролей», но на собственном горьком опыте узнал, что этого делать не следует, и много лет назад перешел на менеджер паролей. Удивительно, но он несколько раз выручал его: от сообщения о краже его учетных данных до сохранения информации его кредитной карты и даже до предупреждения о том, что срок действия его кредитной карты подходит к концу. Однажды он перешел по фишинговой ссылке: в тот день он сильно устал, а эта ссылка выглядела *точно* так же, как адрес электронной почты его банка. Когда он зашел на сайт и захотел было залогиниться, менеджер паролей не захотел работать. Он наотрез отказался помогать ему войти. Тогда он внимательно пригляделся к адресной строке и понял, что адрес банковского сайта был набран неправильно! Ужас какой! Он чуть было не выдал пароль к своей учетной записи в банке! Любовь Боба к своему менеджеру паролей еще возросла.

Ваша обязанность

Вы обязаны делать свою работу настолько безопасно, насколько можете, а также соблюдать все политики, стандарты и инструкции своей организации. Это означает, что проектировать и писать приложения следует безопасно, что вы должны исправлять ошибки, которые вас просят исправить, и обяза-

ны использовать предоставленные вам инструменты. Все сверх того – дополнительно. То есть если вы решили в свое личное время прослушать курс по безопасности, прочитать эту книгу и изучить моделирование угроз, то это уже выходит за рамки ваших должностных обязанностей (и делает вас *потрясающим*).

Вы также обязаны уведомлять своего начальника и (или) группу безопасности, если увидите что-то, на ваш взгляд, неправильное. Например, вы должны сообщить, что команда собирается проектировать свой собственный механизм шифрования, аутентификации, авторизации, идентификации, управления сессиями или еще какой-то элемент безопасности, вместо того воспользоваться одобренным модулем, подходом или продуктом. А также о том, что ваш API демонстрирует очень странное поведение, которое команда не в силах объяснить. Если вы знаете о дефектах безопасности, которые нигде не отслеживаются, считайте своей обязанностью их документировать. Быть может, вы встретили в интернете какие-то сведения о своей компании, которые вас насторожили. Сообщите об этом. Поверьте, они хотят об этом знать.

НЕ ХРАНИТЕ В СЕКРЕТЕ ТО, ЧТО СКРЫВАТЬ НЕ СЛЕДУЕТ

В прошлом году Алиса наняла пентестера, а после кто-то из разработчиков сказал, что пентестер действительно хорош в своем деле. Алиса и так это знала; она посмотрела отзывы о нем и убедилась, что его высоко оценивают. Но ей стало интересно, что имел в виду разработчик. «Да? И как это проявилось?» – спросила она. Разработчик сказал, что у него был список известных ему уязвимостей, и два предыдущих пентестера их не нашли, а этот нашел все и, стало быть, заслужил свою репутацию. Алиса пришла в бешенство! Разработчик знал об уязвимостях и не сообщил о них?! Она решила, что нужно обдумать эту информацию, а не принимать поспешные решения в гневе.

Сколько будет достаточно?

Вопрос, который мучил меня в начале карьеры в области кибербезопасности: «Как узнать, что я сделала *достаточно*?» Я все время переживала, что еще не довела свою организацию до приемлемого уровня безопасности. Поднимаясь по ступеням карьерной лестницы, я осознала, что в некоторых организациях, где я работала, не было того уровня уверенности, который требовался; большинство из них паниковали, как паникует человек, который тонет, пытаясь потушить пожар. Мне это ощущение не нравилось. Я всегда чувствовала, что работаю недостаточно и что моя работа никак не кончается.

После того как опыта и полномочий у меня прибавилось, я начала ставить конкретные цели. Мы рассматривали всю программу и решали, куда направить максимум усилий и какой уровень безопасности считать «достаточно хорошим», после чего я направляла команду к этой точке настолько быстро, насколько могла. Имея измеримые и достижимые цели, я могу спокойно

идти домой вечером, чувствуя, что работа (запланированная на этот день) сделана. Для меня это важно и помогает не испытывать такой сильный стресс на работе.

Если вы работаете в организации, где нет конкретных целей, стандартов или соглашений об уровне обслуживания (service-level agreement – SLA), и создается впечатление, что никто не ждет от вас ничего конкретного в связи с безопасностью, задайте вопрос. Я подозреваю, что, помимо очевидного, – исправлять дефекты безопасности, которые повесили на вас, и не нарушить политики, – группа безопасности все же ожидает от вас каких-то действий, даже если явно об этом не было сказано. Спросите у них, какие шаги предполагаются частью жизненного цикла разработки системы (SDLC), не нужна ли им какая-то помощь с вашей стороны, не планируют ли они отправить вас на курсы безопасного кодирования и т. д. Все это очень естественные вопросы. Если вам ответят, что от вас ничего не нужно, то и хорошо. Однако такой ответ может оказаться для вас не очень утешительным.

Но я все же думаю, что есть масса вещей, к работе над которыми они хотели бы привлечь вас. Они могут предложить вам стать ответственным за безопасность, или поучаствовать в моделировании угроз, или попросить проанализировать некоторые их идеи и поделиться своим мнением, чтобы в итоге сделать их более удобными для разработчиков. Вы можете стать их лучшим другом навсегда – разработчиком, на которого можно положиться.

АМБИЦИОЗНЫЕ ЦЕЛИ

Ставить цели очень важно как в личной, так и в профессиональной жизни. Меня часто поражает, как много организаций, с которыми я имею дело, не имеют никаких целей в своих программах безопасности приложений и как много людей не ставят для себя никаких карьерных целей, кроме как «не быть уволенным». Если поставленные вами цели допускают измерение, так что совершенно ясно, когда они достигнуты, то и шансов достичь их гораздо больше. Если ваша цель – «нарастить мышечную массу на 5 фунтов в течение следующих 6 месяцев», «изучить французский достаточно хорошо, чтобы 2 недели путешествовать по Франции, не переходя на английский» или «не иметь серьезных уязвимостей к следующему тесту на проникновение», то легко понять, достигнуты они или нет. Необходимо планировать шаги по достижению этих целей и отводить время для каждого шага. Например, поднимать штангу каждый вторник и четверг и выпивать три протеиновых коктейля в неделю, посещать курсы разговорного французского в вечерней школе в течение 6 месяцев или выполнить для своего кода статическое тестирование безопасности приложения (SAST) и анализ состава программы (SCA) и исправить все найденные недочеты до прихода пентестера. Я понимаю, что эти цели несколько упрощенные, но идея, надеюсь, понятна. Чем подробнее и конкретнее вы прописываете цели и чем большему числу людей о них рассказываете (чтобы было кому контролировать), тем вероятнее, что цель будет достигнута. Если вы нацелились на что-то *большое*, то должны действовать очень обдуманно и целенаправленно. Никто не пишет походя книгу, не основывает новую компанию и не учит второй язык; нужно трудиться как проклятый. Я хочу, чтобы *вы* вершили великие дела. Подумайте о постановке одной или нескольких важных для себя целей. Поверьте мне, испытываешь потрясающее чувство, когда удастся осуществить то, о чем всегда мечтал.

Примечание Не перестарайтесь, поставив слишком много профессиональных целей. Помните, что свободное время, право на счастье, забота о себе и проведение времени с теми, кого любишь, тоже должны быть частью ваших целей и повседневной жизни!

Безопасное использование искусственного интеллекта

С тех пор как ChatGPT стал доступен широкой публике, я видела много организаций, использующих искусственный интеллект небезопасно. Они дают ему слишком много воли и позволяют принимать решения без надзора. Они не проверяют полученные результаты, прежде чем их использовать. Они предоставляют ему конфиденциальные данные, которые не должны были бы раскрывать. Они тестируют недостаточно тщательно. Они пренебрегают моделированием угроз, и неудивительно, что проблемы случаются вновь и вновь.

Есть книги, в которых эта тема рассматривается более глубоко, например «Uncontrollable» моего друга Даррена Макки, которую я рекомендую прочитать всем, кто беспокоится по поводу ИИ вообще. В этом разделе мы поговорим о защите на верхнем уровне в связи с опасениями, относящимися к разработке ПО.

- Не позволяйте ИИ принимать решения от имени приложений без надзора. Каждое решение, принятое ИИ, должно проверяться в какой-то другой части кода (не контролируемой тем же ИИ). ИИ некогда не следует доверять принятию важных решений в одиночку. Примеры: выплатить возмещение клиенту, изменить дату билета на самолет или удалить учетную запись безобразничающего пользователя. Если ИИ принимает такого рода решения, то должен существовать другой процесс, который их проверяет, прежде чем привести в действие.
- Никогда не давайте ИИ власть, позволяющую влиять на ваши системы, организацию или данные. ИИ не должен контролировать ни себя, ни других. В настоящее время исследователи не скрывают, что не полностью понимают, как ИИ строит модели, а потому не могут объяснить, почему в моделях по-прежнему присутствуют галлюцинации. Не понимая, почему модели работают так, а не иначе, мы, конечно же, не должны отдавать им контроль. Когда вы смотрите научно-фантастический фильм о том, как ИИ захватил власть над миром, причиной всегда является передача ему слишком большого контроля. Я не пытаюсь вас запугать, просто предостерегаю от очевидных и глупых ошибок. Доверять системе, которую мы не вполне понимаем, принятие важных решений от нашего имени – значит напрашиваться на глупые ошибки. Мы видели это не раз.

- Не делитесь конфиденциальной или личной информацией с ИИ, если не являетесь ее владельцем и не получили разрешения от группы безопасности. Я уже попадала в эту ловушку. Я хотела, чтобы ИИ помог мне написать маркетинговые материалы, когда готовилась к поглощению своей компании, и сообщила ему о готовящемся поглощении. Мы собирались объявить о сделке всего через два-три дня, но информация все еще считалась технически конфиденциальной. Что, если бы ИИ выдал эту информацию в ответах кому-то другому, и этот кто-то узнал о сделке до официального объявления? В этом примере информация было хоть и конфиденциальна, но не очень критична. Несколько человек уже догадались из социальных сетей, что Semgter собирается купить We Hack Purple. Ну а если бы речь шла о чем-то более серьезном? Или о гораздо более крупной сделке? Что, если бы вы спросили у ИИ об очень конфиденциальной медицинской информации? ИИ узнает, кто вы, когда вы используете ее, и, задавая вопрос, вы, по сути дела, говорите ему: «Меня очень интересует состояние здоровья XYZ». Это также, вероятно, означает, что вы или кто-то очень близкий вам находитесь в таком состоянии (если вы не студент-медик и не врач). Вы хотите сообщить всему миру о своем состоянии здоровья? Нет? Тогда подумайте дважды, прежде чем делиться этой информацией с ИИ. Это касается и рабочей информации!
- Если вы попросите ИИ написать за вас код, то должны обязательно просмотреть его, прежде чем включать в производственную систему. Или помещать его в репозиторий. Вы должны прогнать его через инструменты SAST и SCA, чтобы лишний раз убедиться, что ничего не опущено. Точно так же, как вы не станете брать случайное изображение из Docker Hub и включать его в производственную систему без проверки на безопасность, так не следует бездумно брать и код, сгенерированный ИИ. Я знаю, что так проще, но сделать плохо всегда проще. Вы же хотите сделать хорошо, а это значит, что код нужно проанализировать, прежде чем использовать. Если вам непонятно, как он работает, спросите у приятеля или у коллеги. Но если вы не понимаете полученный от ИИ код, то я рекомендую воздержаться от его использования.
- Следите за тем, чтобы в материалах, созданных для вас ИИ, не были нарушены ничьи авторские права. Это случается на удивление часто, а получить повестку в суд – удовольствие ниже среднего. Проверить, что вам дали, можно, заключив ответ в кавычки и поискав его в интернете – для начала. Если это изображение, можете выполнить поиск по изображениям. Кроме того, убедитесь, что вы имеете право использовать то, что создал ИИ для вашей работы (что этот материал не защищен авторским правом, в отличие от того, чтобы было создано для вас лично). Если нужна дополнительная информация, проконсультируйтесь с юристами.
- Всегда перепроверяйте все порожденное ИИ. Даже если вы устали и в предыдущие пять раз все было нормально. Поверьте мне.

Ниже описано, как использовать ИИ к собственной выгоде.

- Используйте ИИ для написания пользовательских историй, документации и всего, что пишется для работы. Проверьте, прежде чем кому-то показывать. Помните, что первый вариант, скорее всего, будет... несовершенным.
- Используйте ИИ как помощника при написании кода, только обязательно проверяйте его и не предоставляйте ИИ конфиденциальных данных.
- Используйте ИИ для помощи при поиске уязвимостей! Если вам разрешено делиться своим кодом (например, потому что он открытый), то загрузите его в ИИ и попросите найти уязвимости. Возможно, он справится не так хорошо, как инструмент SAST, но если ИИ у вас есть, а SAST нет, то пользуйтесь тем, что доступно!
- Попросите ИИ смоделировать угрозы. Скорее всего, он выдвинет кучу нелепых идей, но обычно среди них я нахожу одну-две полезных.
- Попросите ИИ помочь с проектированием: опишите, что вы хотите, и посмотрите, что он предложит. По моему опыту, спрашивать ИИ – все равно что спрашивать младшего сотрудника: идеи, возможно, сумасбродные, зато вы получаете кучу результатов очень быстро, и некоторые из них годятся для работы.
- Попросите ИИ добавить комментарии в ваш код. Возможно, он сможет добавить больше, чем у вас хватило бы терпения. Но, пожалуйста, будьте кратки. Никто не хочет читать роман в процессе кодирования. Нужен разумный баланс.
- Попросите ИИ дать предложения по исправлению ошибок. Среди них могут попасться отличные идеи!
- Проявите изобретательность – меня всегда поражает, какие интересные вещи умеет делать ИИ.

Непрерывное обучение

Решив посвятить себя техническим дисциплинам, вы соглашаетесь всю жизнь учиться. В этой области невозможно добиться успеха, если не изучать новое. Ниже перечислены некоторые стратегии, которые оказались полезны мне; берите на вооружение все, что может вам пригодиться!

- Если ваша работа – написание кода, рекомендую изучить безопасное кодирование.
- Если вы занимаетесь проектированием программного обеспечения или являетесь системным архитектором, изучайте безопасное проектирование и моделирование угроз.
- Рекомендую понять, какой способ обучения оптимален именно для вас. Все люди разные, и важно, как именно вы, а не кто-то другой может

добиться наилучших результатов. Лично я люблю читать или рассматривать что-то, попробуйте сначала сами, а потом покажите кому-то другому. Именно так я начала преподавать и вести семинары; учить кого-то или писать для кого-то помогает мне лучше запомнить материал и разложить по полочкам. Попробуйте найти лучшие для себя способы обучения и пользуйтесь ими.

- Попробуйте найти оптимальные для себя форматы обучения. Хотя я люблю живой разговор или обучение на курсах, такой способ получения знаний стоит мне слишком дорого, но, как выяснилось, онлайн-курсы по требованию тоже неплохо работают. Я люблю формат «по требованию», потому что он позволяет мне пересмотреть видео несколько раз при необходимости, особенно если я отвлекаюсь и начинаю думать о чем-то другом (а так бывает очень часто, так что принимайте это в расчет, если это свойственно и вам тоже). Но вообще-то гораздо проще сосредоточиться, когда общаешься с живым преподавателем, особенно хорошим. Возможно, для вас читать бумажную книгу проще, чем слушать аудио, а выполнять практические упражнения лучше, чем принимать участие в обсуждении. Определите, какие форматы вам больше всего подходят, и попытайтесь найти курс, включающий их.
- Попробуйте определить, какие способы обучения вы можете себе позволить. Когда я только начинала, я искала бесплатные материалы в интернете, а книги брала в библиотеке. Бесплатно-то бесплатно, но это отнимало много времени, а сами материалы были очень низкого качества. В итоге я открыла, что могу посещать конференции бесплатно, если буду на них выступать, а поскольку я очень ценю живое общение, для меня это оказалось отличным выходом. Я также стараюсь найти возможность пройти обучение, за которое платит работодатель (включая наблюдение за работой специалистов и закрепление наставника в рабочее время). Из других вариантов упомяну подкасты, новостные рассылки, очные встречи, кружки по интересам, книги, аудиокниги, приглашение платного инструктора в офис, посещение учебных курсов перед конференцией, запуск собственного проекта с открытым исходным кодом или работа на полставки. Есть масса способов чему-то научиться.
- Попросите на работе, чтобы вам позволили понаблюдать за работой специалистов. Найдите профессионального наставника. Попросите включить вас в проект другой группы для получения опыта.
- Расскажите своему начальнику или наставнику о своих карьерных целях и попросите совета, как действовать. У них могут найтись удивительные идеи, о которых вы даже и подумать не могли.
- Отведите время для проверки себя и своих целей, чтобы понять, насколько вы продвинулись. Если есть возможность, найдите партнера, который будет наблюдать за вами и контролировать, чтобы вы не сбились с пути.

Совет В прошлом году я училась у Адама Шостака, а он великолепный учитель. Были предусмотрены интерактивные занятия, лекции и обсуждения, и *мне* учиться у него было легко. Но так бывает не всегда; не все преподаватели одинаковы, и манера преподавания у всех разная. Если вы встретите человека, которого сочтете хорошим учителем, то я рекомендую почерпнуть от него столько знаний, сколько будете способны поглотить, а в идеале все 100 % материалов, которые вас интересуют, потому что учиться у него будет проще, чем у кого-то другого. Заодно это побудит вас продолжать обучение.

Станьте ответственным за безопасность

Ответственный за безопасность – это человек, который играет роль связующего звена между своей командой и группой безопасности. Обычно он проходит специальное обучение, и на него возлагаются дополнительные обязанности. Эта строка будет отлично смотреться в вашем резюме, вы узнаете, как стать разработчиком, относящимся к безопасности со всей серьезностью, а если вы планируете когда-нибудь сменить специальность и заняться информационной безопасностью, то это станет большим шагом в правильном направлении.

Стать ответственным за безопасность – значит добровольно побуждать команду делать все, что рекомендует группа безопасности. В частности, это может быть проверка результатов всех автоматических инструментов с обязательным устранением важных недочетов, выполнение всех относящихся к безопасности частей SDLC с информированием группы безопасности, регулярные встречи с группой безопасности для отчета о текущем состоянии дел, соблюдение командой каких-то SLA или еще что-то. Все программы различны, как и ожидания от них.

Вы, наверное, думаете: «Ну а мне-то что с того?» Во-первых, став ответственным, вы будете творить добро. Когда я говорю «добро», я имею в виду добро, которое творил Супермен: защищать других (коллег, инвесторов, общество и заказчиков). Вы также станете более заметны в организации, а когда приходит время для повышения, выбор обычно падает на самого выделяющегося. Вы научитесь многим полезным вещам, в том числе как писать более качественный код и лучше проектировать приложения, узнаете, с какими типами рисков и угрозами ваши системы сталкиваются повседневно. Я понимаю, что необъективна, но думаю, что эти вещи очень увлекательны. Участие в программе ответственности за безопасность часто подразумевает различные льготы, например возможность взять отгул после обеда, бесплатное питание или такие подарки, как бесплатные книги или наградные монеты. Вы также освоите навыки, которые помогут защитить себя, своих близких и свои персональные устройства на протяжении всей оставшейся жизни. Отличная сделка!

Я горячо рекомендую записаться в ответственные за безопасность, если представится такая возможность, пусть даже всего на несколько месяцев. Скажите, что вы прочли эту книгу, и ваши шансы повысятся.

ВЗРАЩЕННЫЙ МНОЙ ОТВЕТСТВЕННЫЙ ЗА БЕЗОПАСНОСТЬ

С самой первой своей программы безопасности приложений я вводила программы ответственности за безопасность, формальные или неформальные. Это помогает мне выстраивать доверие в команде. Честно скажу, я люблю людей, а программа ответственных за безопасность позволяет мне встречаться с очень крутыми разработчиками, а они относятся к типу людей, которые мне особенно нравятся. Помню, в школе мне больше всех нравились те, с кем я сталкивалась на уроках программирования, и, если бы мне пришлось выбирать, с кем работать всю жизнь, я выбрала бы их. Недавно один из ответственных за безопасность в моей первой программе сказал, что он запустил *свою* первую программу! Я еще несколько лет назад знала, что он перейдет на работу в области безопасности, но то, что он запустил такую программу в своей организации, очень порадовало! Лиам, ты молодец!

Все на борт

Когда «Таня из группы безопасности» говорит разработчику, что делать, это не то же самое, что сказать своему коллеге: «Слушай сюда, отныне мы должны делать вот так». Вы для него равный, тогда как на группу безопасности смотрят как на чужаков или в лучшем случае как на посторонних. Полагаю, ваше влияние на свою команду гораздо сильнее, чем вы думаете, и, если вы предложите другим соблюдать инструкцию по безопасному кодированию или новую политику безопасности, посетить совещание по моделированию угроз и т. д., то коллеги, скорее всего, вас послушают. Особенно если вы пойдете вместе с ними. Когда приглашаешь людей пойти вместе, мероприятие кажется им не таким пугающим и более интересным. Все в выигрыше!

Возьмите за правило делать замечание, когда видите, как кто-то делает то, чего делать не должен, или не делает то, что должен. Если вы нашли ошибку или изящное решение, подумайте о том, чтобы обсудить это с командой и документировать, если оно того заслуживает. Говорите о безопасности. Говорите о проблемах. Спрашивайте мнение коллег. Чем больше внимания вы уделяете этой теме, тем вероятнее, что и другие последуют вашему примеру. Медленно, но верно культура вашей команды будет меняться к лучшему.

Совет Если вы увидели, что кто-то делает что-то неправильно, привлекайте к этому внимание мягко. Подумайте, как бы сами хотели услышать дурные новости от другого, особенно если ошибка действительно грубая. Проявите уважение и сочувствие. Никому не нравится, когда его дитя называют уродом; пожалуйста, помните об этом, сообщая о чем-то далеком от идеала.

Переход в группу безопасности

Дочитав до этого места, вы, возможно, подумали, что хотели бы когда-нибудь работать в группе безопасности. Шансы получить такое место будут гораздо выше, если вы воплотили все прочитанное в этой книге на прак-

тике. Читая книгу и применяя свои знания, обязательно расскажите группе безопасности, на что рассчитываете. Не забудьте упомянуть, что вы пытались моделировать угрозы в своей команде. Скажите, что вы написали инструкцию по безопасному кодированию, и попросите оценить ее. Выполнение все большего числа относящихся к безопасности вещей на своем рабочем месте и уведомление об этом группы безопасности покажет им, насколько вы заинтересованы и серьезно настроены.

Вы можете также поинтересоваться, не нужна ли группе безопасности ваша помощь. Спросите, нет ли проекта, над которым вы могли бы поработать. Спросите, нет ли работы, которую они планируют провести в вашей команде и в которой вы могли бы принять участие. Но прежде узнайте, не возражает ли ваш начальник – я вовсе не подначиваю вас ссориться с начальством. Добровольная помощь группе безопасности откроет перед вами возможность учиться, стать более заметным и даст шанс продемонстрировать свои новые умения.

Самый простой способ перейти из отдела разработки ПО в отдел информационной безопасности – сделать это там, где вы уже работаете. Вы знаете обо всех корпоративных системах, что имеет огромное значение для группы безопасности. Кроме того, не будет периода раскочки; новым работникам нужно много времени, чтобы войти в курс дела, а вы уже готовы. И связываться с агентством по подбору кадров тоже не нужно, а это экономия времени и средств. Наем способного сотрудника из другого отдела всегда является удачной сделкой с точки зрения руководства. Это также означает, что у вас есть возможность заранее «пройти собеседование» с командой безопасности, показав им проекты, над которыми вы уже работали, что они смогут переговорить с вашим руководителем, а вы сможете добровольно помочь им в текущей работе. Все это в совокупности и делает такой путь перехода отдел информационной безопасности самым простым.

Совет Если вы хотите перейти в другой отдел, скажите группе безопасности, что мечтаете работать в области безопасности. Скажите об этом на собраниях, скажите своему начальнику, скажите всем. Если вы будете хранить это в секрете, то, скорее всего, ничего не получится. Сообщите всему миру о том, чего вы хотите.

Устройство на работу в области безопасности в другой организации

Хотя переход на другую работу в пределах одной организации – самый простой способ заняться информационной безопасностью, так получается не всегда. Возможно, ваш начальник не захочет вас отпустить. Возможно, в группе безопасности уже заполнен штат, и на ближайшие год-два бюджет на расширение штата не предусмотрен. Возможно, из-за размера организации группы безопасности в ней нет вовсе. Возможно, вам кажется, что здешняя группа безопасности – банда придурков, и именно с ними вы работать не

хотите, хотя твердо уверены, что безопасность – как раз то, что вам нужно. Давайте поговорим, как обратиться по вопросу трудоустройства в другую организацию.

Печальный факт об отрасли информационной безопасности, о котором вам стоит знать, – то, что вакансии предлагаются в основном на позиции старших специалистов. Часто, говоря о позиции начального уровня, ожидают пяти-семилетнего опыта работы, что, мягко говоря, является, противоречием. Поэтому я рекомендую подавать заявление, даже если у вас нет достаточного опыта. Если речь идет о работе начального уровня и вы чувствуете себя готовым к переходу в сферу безопасности, подавайте.

Но я рекомендую в заявлении описать весь свой относящийся к делу опыт, пусть даже не вполне профессиональный. Например, перечислите книги, которые вы прочли, включая эту, чтобы показать, что вы учились в свое свободное время. Если вы принимали участие в каком-нибудь проекте с открытым исходным кодом, если вы являетесь членом какого-то сообщества, связанного с безопасностью, если вы бывали на конференциях или посещали курсы, отметьте все это. Существует много онлайн-академий, который выдают сертификаты о посещении курсов; это не то же самое, что формальная сертификация, но вы можете получить парочку для своего резюме – будет не лишним. Следует также упомянуть, были ли вы ответственным за безопасность, а также обо всех связанных с безопасностью работах, которые вы выполняли на своем рабочем месте, и изложить все это в сопроводительном письме, чтобы показать, насколько вы полны решимости поменять сферу деятельности.

Совет Начиная с весны 2025 года, я буду транслировать бесплатные, живые, виртуальные уроки по каждой главе книги, как то было в случае с моей первой книгой. Позже они будут сохранены на YouTube в виде плейлиста на случай, если вы что-то пропустили. Подпишитесь на мою рассылку, чтобы получать уведомления: <https://newsletter.shehackspurple.ca>.

Переходя из разработки ПО в безопасность, я проявила немало упорства. В то время я занимала высокую техническую должность (уровня CS-03) в одном из государственных органов Канады, занимала ее почти десять лет и не хотела идти на понижение при переходе. Я работала в области разработки ПО уже 17 лет, имела за плечами кучу успешных проектов и полагала, что, хотя еще не являюсь специалистом в области безопасности, все же смогу отлично справиться с любой порученной мне работой. Подавая заявление о приеме на работу, я выполнила бесплатный тест на проникновение, а затем смягчила последствия всех найденных уязвимостей, чтобы доказать, что могу работать по обе стороны баррикад. Я также предъявила отличные характеристики с текущего и предыдущего места работы, и у меня был профессиональный наставник, хорошо известный в отрасли, который также охарактеризовал меня как человека, способного быстро обучаться и чрезвычайно целеустремленного. Вы должны позаботиться о наличии характеристик, пусть даже просто от членов сообщества, которые могут засвидетельствовать тот факт, что вы

посещали все ежемесячные собрания, действуете профессионально и очень стараетесь учиться и начать работать в отрасли.

Другой способ перейти в сферу безопасности – найти профессионального наставника. На протяжении карьеры у меня было несколько наставников, с одними у нас сложились формальные отношения, с другими неформальные, но все они мне помогли. Они давали советы, вводили в курс дела и помогали избежать ошибок. У меня были наставники по безопасности, по практике ведения бизнеса, по публичным выступлениям, по всему, чего я пыталась достичь, особенно если это было что-то новое. Я нахожу человека, который, по моему мнению, прекрасно владеет каким-то навыком, и перенимаю его опыт. Пытаясь найти наставника, важно помнить, что он делает вам величайшее одолжение бесплатно. Так что цените его время.

У меня есть несколько рекомендаций по работе с профессиональным наставником.

- Если наставник назначает вам встречу, не уклоняйтесь от нее. Всегда отвечайте на его электронные письма и сообщения в мессенджерах. Ни в коем случае не тратьте его время попусту. Его время – дар вам, относитесь к нему соответственно.
- Всегда пробуйте его советы на практике. Если вы не думаете, что совет абсолютно ужасающий, то стоит его попробовать. Даже если придется как следует поработать. Я помню, что несколько моих профессиональных наставников говорили мне, что им нравится меня учить, потому что я всегда стараюсь делать, что они говорят, и сообщаю о результатах. Если наставник дал вам совет, но вы ему не последовали, не получили желаемых результатов, а потом еще и пожаловались, то наставник может всерьез обидеться. Если вы не слушаете совета, то как он может вам помочь? Если вы думаете, что совет плохой, то вы выбрали не того наставника, и пора двигаться дальше. Нет ничего плохого в том, чтобы расстаться с наставником (вежливо), если он вам не подходит.
- Всегда заканчивайте начатое. Если наставник дает совет, знакомит вас с кем-то или предоставляет какую-то другую возможность, расскажите ему, чем все закончилось. Обязательно сообщайте о результате. Если он попросил вас прочитать какую-то книгу, то, даже если вы одолели только половину, потому что книга оказалась слишком сложной, все равно объясните ему это. Мой первый наставник поручил мне прочитать «The Shellcoder's Handbook», я дошла до главы 10, выполнила все упражнения и решила, что не хочу писать шелл-код. Мне не понравилось писать эксплойты на ассемблере и заниматься обратным проектированием, но, именно прочитав эту книгу, я поняла, что это не для меня. О чем и сказала наставнику. Я многому научилась, и это оказалось полезно, так как теперь я знала, что этот путь в мире кибербезопасности мне не подходит.
- Спросите, можете ли вы чем-то помочь наставнику. Если можете, обязательно помогите. Вариантов много: представить кому-то, помочь ре-

бенку или младшему сотруднику, да даже просто выслушать накопившееся на душе. Он делает вам огромное одолжение, и, если вы можете чем-то отплатить взамен, сделайте это. Я побывала на многих встречах, от которых предпочла бы отказаться только потому, что об этом попросил наставник. Я перед ними в долгу!

- Благодарите своих наставников. Я всегда высказываю им благодарность. В своих книгах, блогах, давая интервью, а особенно лично. Я регулярно говорю им о своей признательности. Я часто сообщаю собеседникам, что у меня есть наставники, и отдаю им должное. Я знаю, что работу сделала я сама, но тот факт, что мне указали направление, дали ощутить поддержку и открыли новые возможности, для меня бесценен. Я хочу, чтобы все, и они в том числе, знали, как высоко я ценю их.
- Не ожидайте, что наставник уделит вам очень много времени. Я обычно встречаюсь со своими раз в месяц или раз в квартал на 30 мин. У меня к этому моменту уже накопились вопросы, и обычно я предварительно сообщаю о том, какой эффект дали их советы в прошлом месяце или квартале. Если я могу уделить им свое время, то делаю это.
- Договоритесь с наставником о том, все ваши разговоры конфиденциальны. Очень важно, чтобы вы были уверены в скромности своего наставника. Явное подтверждение стоит тех неловких 30 с, которые потребуются на высказывание этой просьбы.
- Проверьте, что наставнику можно доверять и что не существует никакого конфликта интересов. Дважды мои наставники очень настойчиво подталкивали меня к сделке, в которой они выступали бы в роли бенефициаров; я была вынуждена порвать отношения с обоими. Очень важно, чтобы наставник помогал вам по доброте душевной, а не потому, что надеется втянуть вас в какие-то выгодные для него делишки. Совершенно нормально помогать своему наставнику, делать ему одолжение или работать в его интересах. Но если в советах, которые он дает, скрыт потенциальный конфликт интересов, то лучше поставить отношения на паузу, пока ситуация не прояснится, или вообще прервать их. Если наставник выглядит раздосадованным, когда вы заговорили о конфликте интересов, значит, это не тот человек, которого стоило выбирать в наставники.
- Неплохо иметь кого-то, кто больше похож на советчика, чем на наставника. Быть может, вы изредка задаете ему конкретные вопросы по какой-то одной теме и не считаете его наставником. Тоже нормально! Пусть это будет советчик – человек, дающий отличные советы, которого вы цените и уважаете. Я знаю нескольких людей, которые считают меня советчиком, и это прекрасно работает.
- Еще стоит подумать о контролирующем партнере. Это и не наставник, и не советчик. Он не дает вам сбиться с пути, контролирует вас и, когда необходимо, дает пинка. Если вы испытываете трудности с заверше-

нием проектов или не можете сосредоточиться, то контролирующий партнер может оказаться тем, кто поднимет вашу карьеру или бизнес на следующий уровень. Он поможет вам достичь своих самых амбициозных целей.

- Какие вопросы можно задать своему наставнику:
 - какие книги или статьи вы рекомендуете прочитать по теме X?
 - не стоит ли мне с кем-то встретиться?
 - не нужно ли мне пройти какие-то курсы или просмотреть какие-то видео?
 - не следует ли мне вступить в какие-то сообщества и посетить какие-то мероприятия?
 - не стоит ли подписаться на какие-то новостные рассылки или блоги?
 - как получить опыт работы в интересующей меня области?
 - как я мог бы лучше справиться с ситуацией X? Что мне сделать по-другому в следующий раз?
 - у меня намечается встреча по вопросу X. Есть ли у вас предложения о том, как донести до Y информацию: более четко, более убедительно, более профессионально, более технически грамотно и т. д.?

Если у вас есть профессиональный наставник, то нужно будет немного заняться самоанализом. Когда у меня впервые появился наставник, я хотела изучить кое-какие технические вещи, но очень быстро поняла, что нужно также поработать над навыками общения. Лично я с трудом устанавливаю границы и говорю «нет», и над этим мне следует поработать, а не браться постоянно за неоплачиваемую работу, потому что я знаю, что сделать ее нужно, а никто другой этим заниматься не будет. У меня от этого на душе неспокойно.

Прямо сейчас у меня есть наставник (привет, Илона!), который помогает мне научиться говорить «нет» – вежливо, но твердо. Довольно часто люди настолько раздражаются, когда кто-то нарушает их личные границы, что выходят из себя, чего нам, конечно, не следует допускать в профессиональной обстановке. Я всегда хочу быть милой, и надо было научиться говорить «нет», оставаясь милой. Я считаю бесценным иметь поблизости человека, который всегда предельно откровенен со мной. «Таня, на этот раз проблема в тебе. Ты согласилась на слишком многое. Давай-ка пробежимся по списку и кое-что удалим. А затем посмотрим, как в следующий раз говорить “нет” вместо “да”». Сначала вы, вероятно, будете обращать больше внимания на технические умения, но если будете продолжать учиться и слушать наставника, то усовершенствуетесь также в навыках общения, выстраивании круга общения, деловых навыках и многом другом. Нет предела совершенству, когда тобой руководит удивительный человек, на которого ты равняешься. И на этом позвольте мне поблагодарить еще нескольких людей, которых я чту своими наставниками, советчиками и контролирующими партнерами: Илону, Кима, Ченси, Дагана, Клинта, Зейна, Дэна, Троя и Адама. Спасибо за то, что учили меня, руководили мной и поддерживали меня.

Совет Каждый понедельник в X (Twitter), LinkedIn, BlueSky и Mastadon я веду программу подбора наставников под названием #CyberMentoringMonday. Не стесняйтесь использовать этот хештег и мой тег (@SheHacksPurple) или просто заходите на мои еженедельные посиделки в поисках наставника. Я сделаю ретвит или репост, если увижу это (иногда X подавляет уведомления, но я делаю что могу). Объясните, чему вы хотите научиться, какие усилия уже приложили, чтобы этому научиться, и каковы ваши цели (например, понять, в какой именно области кибербезопасности вы хотели бы работать, или стоит ли вам вообще заниматься этим делом). Начиная с 2018 года, сотни (если не тысячи – я не веду счет) людей нашли друг друга с помощью этого еженедельного мероприятия и хештега. Приглашаю и вас воспользоваться, если вы считаете, что это может принести вам пользу.

Заключение

В этой книге мы много говорили о том, как писать безопасный код вне зависимости от языка, а также дали конкретные советы для нескольких языков программирования и каркасов. Мы рассмотрели, как более безопасно программировать разные типы приложений, в том числе мобильные, интернет вещей (IoT), веб-сокеты и API. Мы обсудили разные типы уязвимостей, а также много разных операций, которые можно добавить в SDLC, чтобы превратить его в *безопасный* SDLC. По пути нам попадались упражнения, рассказы об Алисе, Бобе и их друзьях и, хочется надеяться, идеи, которые вас вдохновили. Все эти рекомендации должны сделать из вас *безопасного* кодировщика, а если повезет, еще и энтузиаста безопасности.

Если хотите узнать больше, то есть несколько возможностей.

- Наймите меня для обучения безопасному кодированию, посещайте мои публичные живые тренинги или запишитесь на один из моих онлайн-курсов.
- Прочитайте мою первую книгу. Читайте другие книги по интересующим вас вопросам безопасности. Попробуйте найти автора, который вам понравится, и прочитайте все им написанное.
- Подпишитесь на мой бесплатный информационный бюллетень и читайте мой блог по адресу shehackspurple.ca.
- Подпишитесь на еженедельный бесплатный информационный бюллетень TL;DR, в котором дается краткая сводка технических блогов и других учебных материалов (tldrsec.com).
- Запишитесь на бесплатные онлайн-курсы в таких местах, как Semgrep Academy (создано мной), OWASP Secure Coding Dojo (создано моим другом Полом Ионеску), API Sec University и т. д. В сети есть много замечательных ресурсов.
- Смотрите мои бесплатные видео на YouTube (youtube.com/shehackspurple). Также смотрите видео на YouTube-канале OWASP; они изумительны.

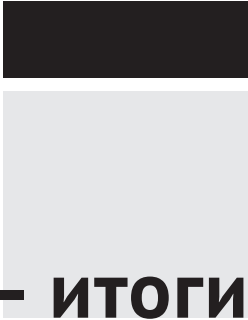
- Вступайте в интересные вам онлайн-сообщества. Например, я люблю сообщество OWASP и участвую в их конференциях и собраниях местных отделений. Выберите то, что вам подходит. Потом обзаведитесь друзьями.
- Поищите другие информационные бюллетени, влиятельных лиц, YouTube-каналы, которые вам нравятся, и подпишитесь на них.
- Найдите интересные вам подкасты и подпишитесь на них. Слушать можно, когда гуляете, ведете машину, занимаетесь уборкой или физкультурой. Я *очень многому* научилась во время стирки.
- Если найдете человека, создающего контент, который вам определенно по душе, читайте все написанное им. Я так часто делаю. Нахожу автора, который мне нравится, и читаю все его книги. Авторы часто создают свои произведения в разных форматах, например: подкасты, блоги, онлайн-курсы, живые выступления на конференциях и т. д. Если вы читаете чей-то блог и обнаруживаете, что автор создает и что-то другое, можете познакомиться и с этим тоже!
- Подумайте о том, чтобы завести контролирующего партнера или наставника. Или обоих сразу! У меня не хватает слов, чтобы описать ценность таких отношений.
- Дайте себе слово никогда не прекращать учиться. Пообещайте себе – письменно – ставить обучение на первое место в своей карьере. Скажите об этом другу, своей второй половине или наставнику. Расскажите им о своих целях или объявите о них публично в социальных сетях, как когда-то поступила я, сообщив в Twitter, что собираюсь написать эту книгу. Если о чем-то объявить во всеуслышание или в письменном виде, то гораздо больше шансов, что это свершится.

Прочитав эту книгу, вы должны чувствовать гораздо большую уверенность в том, что создаете более безопасные приложения. Вы должны знать, где искать ответы на свои вопросы и где продолжить обучение. Я думаю, что группа безопасности тоже заметит вас и те усилия, которые вы стали прикладывать. Ваши коллеги, вероятно, будут просить у вас совета, а другие команды станут спрашивать ваше мнение. Возможно, у вас будут просить почитать эту книгу или поделиться тем, что вы узнали, с другими членами команды. Вполне может статься, что вы получите известность как «безопасный разработчик», и завоюете репутацию человека, который печется о безопасности и у которого точно можно получить хороший совет. Ваша карьера может пойти по новому, в большей степени ориентированному на безопасность пути.

Я заканчиваю эту книгу призывом к действию. Я уже много раз говорила об этом в книге; я хотела бы, чтобы вы щедро делились почерпнутой из книги информацией с другими людьми. Пожалуйста, делитесь сведениями о безопасности, которые считаете ценными, неважно, происходят они из этой книги или из других источников. Одна из целей моей карьеры – способствовать движению нашей отрасли вперед, особенно в части построения

более безопасного программного обеспечения. Я не хочу и дальше читать о крупных инцидентах безопасности и кражах данных. Заполняя на каком-то сайте очередную форму, я не хочу думать: «Боже, надеюсь у них не украдут мои данные». Увы, сейчас (как это ни печально) я вынуждена предполагать, что это только вопрос времени. Каждый работающий в нашей отрасли человек несет ответственность за максимально безопасное выполнение своей работы, но большинство из них не думает, что разделение информации с другими – часть этой ответственности. Я надеюсь, что вы и я вместе сможем изменить такое умонастроение. Прошу вас, дайте почитать эту книгу коллеге или приятелю, поделитесь мыслями, которые показались вам интересными, и внедряйте в практику все, что представляется вам ценным. Помогите мне в моей миссии делиться знаниями с другими людьми при малейшей возможности.

Спасибо.



Часть III – итоги

В этой части мы рассмотрели безопасный жизненный цикл разработки системы (SDLC): те действия, которые необходимо включить в SDLC, чтобы гарантировать надежный выпуск безопасного программного обеспечения.

Контрольный список действий, связанных с безопасностью, на каждом этапе SDLC

Какого бы SDLC вы ни придерживались, необходимо собрать требования, разработать проект, написать код, протестировать его, выпустить, а затем сопровождать. Хотя я использовала названия этих задач, заимствованные из методологии водопада, все мы обязаны выполнять их независимо от применяемой методологии. Старайтесь включать одно или несколько описанных ниже действий, связанных с безопасностью, в каждый этап своего SDLC.

Примечание На каждом этапе SDLC исправляйте проблемы, которые влекут риски для организации.

Этап сбора требований

- Сформулируйте требования к безопасности и добавьте их в проект.
- Пригласите представителя группы безопасности на установочное совещание по проекту.
- Включите действия, связанные с безопасностью, в план-график проекта.

Этап проектирования

- Выполните моделирование угроз своему приложению (взглянув на него с точки зрения злоумышленника).
- При любой возможности применяйте безопасные паттерны проектирования и концепции.

- Создавайте пользовательские истории, относящиеся к безопасности, если в вашем проекте вообще практикуются пользовательские истории.
- Выполняйте анализ безопасности архитектуры или моделирование на доске с приглашением представителя группы безопасности. Цель – поиск потенциальных просчетов проектирования.
- Анализируйте все потоки данных на предмет потенциальных проблем безопасности.

Этап кодирования

- Устанавливайте и используйте плагины IDE, касающиеся безопасности.
- Сканируйте код статическим анализатором (SAST), анализатором состава программы (SCA) и сканером секретов.
- Выполняйте ручной анализ кода самостоятельно или с привлечением коллег.
- Включайте автономные тесты для проверки функций безопасности, а также тестируйте на безопасность функции, напрямую не связанные с безопасностью.
- Запущенное приложение просканируйте динамическим анализатором, если получили разрешение от группы безопасности. Если нет, попросите, чтобы они просканировали приложение сами.

Этап тестирования

- Запрашивайте как можно больше тестов, а если у вас есть право выбора, то выбирайте их в порядке приоритета: тестирование на проникновение, автоматическое тестирование, IAST.
- Если раньше у вас не было права доступа к инструментам SAST, SCA и сканирования секретов, попросите дать его сейчас.
- Чем раньше вы начнете прогонять тесты, тем больше времени останется на исправление дефектов. Если вы можете запускать автоматические инструменты на этапе кодирования, не дожидаясь начала этапа тестирования, тем лучше; старайтесь начинать тестирование как можно раньше.
- В идеале хорошо бы исправлять проблемы и требовать повторного тестирования многократно на протяжении этого этапа.

Этап выпуска и развертывания

- Если вы пользуетесь конвейером CI/CD, то в него следует добавить инструменты сканирования и тестирования на безопасность, но будьте осторожны и избегайте инструментов, выдающих много ложноположительных результатов.
- Ограничьте доступ к процессу выпуска и инструментарию, особенно к указанным выше системам, и регулярно создавайте резервные копии.

- Потратить время и силы на то, чтобы группа безопасности давала согласие быстрее и эффективнее, а по возможности осуществить переход разработчиков на самообслуживание – выигрыш для всех заинтересованных сторон.

Этап сопровождения

- Все системы, вне зависимости от размера, нуждаются в протоколировании, мониторинге и уведомлении.
- Добавление блокировки и (или) экранирования в общедоступные приложения облегчит вашу жизнь.
- Эксплуатируемые приложения необходимо и дальше тестировать, даже если кажется, что в них ничего не изменилось.
- Не только создаваемые нами приложения, но и системы, которыми мы пользуемся для их создания, должны быть учтены в планах обеспечения непрерывности бизнеса и аварийного восстановления.
- Нас могут привлечь к расследованию инцидента безопасности; в таком случае скрупулезно следуйте политике «кому нужно, тот знает» и всем инструкциям, полученным от группы расследования.
- Если вы видите что-то странное, что не можете объяснить, или что-то похожее на потенциальную проблему кибербезопасности, немедленно вызывайте группу безопасности и опишите, что вас насторожило.

Спасибо, что прочли эту книгу, вступили в битву со злом вместе со мной и многими другими, пекущимися о цифровой безопасности. Надеюсь увидеть вас на живых трансляциях, на моем канале YouTube, в моих социальных сетях, на моих занятиях или на мероприятиях, устраиваемых сообществом по всему миру. Я очень дружелюбна; если вы встретите меня на каком-нибудь мероприятии, поздоровайтесь – я буду рада с вами познакомиться.

Ценю ваше стремление сделать наш мир безопаснее.

Tanya Janca,
SheHacksPurple

Ресурсы

Глава 1. Введение в основы безопасности

1. О доверии особям: <https://hbr.org/2009/06/rethinking-trust>.
2. Персонажи: https://en.wikipedia.org/wiki/Alice_and_Bob.
3. Врезка с определениями: <https://languages.oup.com/google-dictionary-en>.
4. Авария в компании Rogers: <https://www.cp24.com/news/rogers-service-starting-to-recover-after-canada-wide-outage-1.5979474>.
5. Принцип наименьших привилегий: <https://www.cyberark.com/what-is/least-privilege>.
6. В Telegram по умолчанию нет входного шифрования: <https://www.trustedreviews.com/news/is-telegram-safe-4130553>.
7. Различные нормативные акты, относящиеся к кибербезопасности: https://en.wikipedia.org/wiki/Cyber-security_regulation.
8. Перечень некоторых аспектов комплаенса: <https://www.shiftright.io/blog/getting-to-know-compliance-in-software-development>.
9. <https://nordlayer.com/blog/cybersecurity-compliance-everything-you-need-to-know>.
10. Каркас знаний по безопасности: <https://owasp.org/www-project-security-knowledge-framework>.
11. OWASP SAMM: <https://owasp.org/www-project-samm>.
12. NIST 800-53: <https://cyberexperts.com/application-security-framework>.
13. <https://www.cisecurity.org/controls>.
14. ISO 27001 и 27002: <https://www.bitsight.com/blog/7-cybersecurity-frameworks-to-reduce-cyber-risk>.
15. SOC 2: <https://www.bitsight.com/blog/7-cybersecurity-frameworks-to-reduce-cyber-risk>.

Глава 2. В начале пути

16. Пример кода проверки входных данных: <https://docs.microsoft.com/en-us/office/vba/language/reference/user-interface-help/isdate-function>.
17. Синтаксис C#: <https://www.tutorialsteacher.com/csharp/csharp-datetime>.
18. Синтаксис вычитания года из даты в C#: <https://stackoverflow.com/questions/2992451/how-to-subtract-a-year-from-the-datetime>.
19. Синтаксис TryParse: <https://www.inforbiro.com/blog/c-check-ifstring-is-datetime>.
20. W3schools, проверка входных данных в PHP: https://www.w3schools.com/php/php_form_validation.asp.
21. Шпаргалка по предотвращению XSS: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.
22. Кодировщики HTML в .NET и PHP: <https://www.web2generators.com/html-based-tools/online-html-entities-encoder-and-decoder>.
23. Примеры кодирования: <https://portswigger.net/web-security/cross-site-scripting/preventing>.
24. Логические реализации методов управления доступом: <https://delinea.com/blog/access-control-models-methods>.
25. Схема разделения секрета Шамира: https://en.wikipedia.org/wiki/Shamir%27s_secret_sharing.
26. Криптографическое уничтожение: <https://en.wikipedia.org/wiki/Crypto-shredding>.
27. Определение криптографической соли: [https://en.wikipedia.org/wiki/Salt_\(cryptography\)](https://en.wikipedia.org/wiki/Salt_(cryptography)).
28. Определение криптографического перца: [https://en.wikipedia.org/wiki/Pepper_\(cryptography\)](https://en.wikipedia.org/wiki/Pepper_(cryptography)).
29. Заголовки Sec-Fetch: <https://w3c.github.io/webappsec-fetch-metadata>.
30. CSP Strict-Dynamic: <https://speakerdeck.com/lweichselbaum/o-19-securing-web-apps-with-modern-platform-features?slide=23>.
31. Подробнее об использовании CSP: <https://csp.withgoogle.com>.
32. Как проверить, что ваша CSP реализована корректно: <https://csp-evaluator.withgoogle.com>.
33. О доверенных типах: <https://github.com/w3c/trusted-types>.
34. Не используйте X-XSS-Protection и фиксацию открытого ключа: <https://www.invicti.com/blog/web-security/http-security-headers>.
35. Политика одного источника: <https://portswigger.net/web-security/cors/same-origin-policy>.
36. Использование CORS для обхода политики одного источника: https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy#:~:text=The%20same%2Dorigin%20policy%20is,documents%2C%20reducing%20possible%20attack%20vectors.
37. Заголовки безопасности HTTP и безопасные cookie: «Alice and Bob Learn Application Security», глава 2.

Глава 3. Улучшение

38. Определение базы данных: <https://en.wikipedia.org/wiki/Database>.
39. Как протестировать и укрепить свой сервер баз данных: «Alice and Bob Learn Application Security», глава 6.
40. Загрузка файлов: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html.
41. Инженерия конфиденциальности: https://en.wikipedia.org/wiki/Privacy_engineering, <https://www.nist.gov/itl/applied-cybersecurity/privacy-engineering/>, <https://arxiv.org/pdf/2007.08613.pdf>.
42. Советы по защите файлов: <https://www.passportalmsp.com/blog/11-best-practices-for-document-management-security>.
43. Надеем Доуба провел семинар в Оттавском отделении OWASP, на котором я научилась делать все описанное в этом абзаце, зима 2014.
44. Избегайте плохих функций: <https://www.freecodecamp.org/news/buffer-overflow-attacks>.
45. Безопасные C-библиотеки: <https://snyk.io/blog/buffer-overflow-attacks-in-c>.
46. ASLR и DEP: <https://www.synopsys.com/blogs/software-security/detect-prevent-and-mitigate-buffer-overflow-attacks>.
47. Сериализация и ее применения: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/serialization>.
48. «Alice and Bob Learn Application Security», глава 5.
49. Шпаргалка OWASP по небезопасной десериализации: https://cheatsheet-series.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html.
50. Определение конфиденциальности: <https://www.oxfordlearnersdictionaries.com/definition/english/privacy>.
51. Цитата: «Уединение – это о самоопределении и достоинстве. Не только о данных». Abigail Dubiniecki «Digital Self-Defence Post-Roe», BSides Ottawa, November 2022.
52. Состояние системы: <https://www.igi-global.com/dictionary/state-of-a-system/50412>.
53. Состояние системы, определение 2: [https://en.wikipedia.org/wiki/State_\(computer_science\)](https://en.wikipedia.org/wiki/State_(computer_science)).
54. Анализируйте код одной системы, которая доставляет и обрабатывает все ошибки: https://owasp.org/www-community/Improper_Error_Handling.
55. Внедрение в журнал: https://owasp.org/www-community/attacks/Log_Injection.
56. Мой начальник, Билл Коллс, давным-давно научил меня блокировке записей. Я приблизительно воспроизвела этот разговор, состоявшийся более 20 лет назад. Билл, если ты это читаешь, спасибо тебе. Ты был моим профессиональным наставником, и я не сомневаюсь, что твоя вера в меня на ранних этапах моей карьеры помогла мне стать такой, какой я стала. Огромное тебе спасибо за это.
57. Криптографическое хранение – шпаргалка OWASP: https://cheatsheet-series.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html.

58. Криптографические стандарты и наставления: <https://www.nist.gov/programs-projects/cryptographic-standards-and-guidelines>.
59. Строго типизированные языки: <https://www.techtarget.com/whatis/definition/strongly-typed>.
60. Что такое предметно-ориентированное проектирование? Статья по безопасности: <https://securityintelligence.com/posts/secure-coding-domain-driven-design>.
61. Sec4dev 2022, «Typed Security Preventing Vulnerabilities By Design», Michael Koppmann: <https://www.youtube.com/watch?v=TCc6gbTtK8o>.
62. Общая ссылка на книгу Эванса «Domain Driven Design».

Глава 4. Достижение цели

63. Ссылка на книгу «Drift Into Failure»: <https://www.routledge.com/Drift-in-to-Failure-From-Hunting-Broken-Components-to-Understanding-Complex/Dekker/p/book/9781409422211>.
64. Предметно-ориентированное проектирование, из эпизода 68 подкаста «We Hack Purple» с участием Гаяндипа Сингха Джунеджи: <https://wehackpurple.com/podcast/episode-68-with-gagandeep-singh>.
65. Вопросник Адама Шостака по моделированию угроз: <https://github.com/adamshostack/4QuestionFrame>.
66. «Securing Open Source Dependencies: It's Not Just Your Code That You Need to Secure», с участием Раны Халиль: <https://www.youtube.com/watch?v=oD0sDOvnwlc>.
67. Идея вымощенной дороги, популяризация Netflix: <https://snyk.io/blog/three-considerations-building-security-program>.
68. «Разработчики ПО как вода»: я говорила это сотни раз. Буквально.
69. Вымощенная дорога: <https://www.hella-secure.com/post/create-your-paved-roads>.
70. Не забывайте о лицензировании: <https://snyk.io/learn/code-security-audit>.
71. Надежная система: https://wiki.owasp.org/images/a/ac/OWASP_SCP_Quick_Reference_Guide_v2.doc.
72. OWASP, десять главных ошибок при конфигурации безопасности: <https://owasp.org/www-project-top-ten>.
73. Шпаргалка OWASP по уклонению от фильтра XSS: https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html.
74. Тестирование проверки конечных точек, «Руководство OWASP по тестированию мобильных приложений»: <https://mas.owasp.org/MASTG/tests/android/MASVS-NETWORK/MASTG-TEST-0021/static-analysis>.
75. Повторно используемый код и почему это хорошо: <https://www.browserstack.com/guide/importance-of-code-reusability>.
76. Заблаговременная забота о повторном использовании кода: <https://data-myte.com/code-reuse>.

77. Определение системного вызова: <https://www.geeksforgeeks.org/introduction-of-system-call>.
78. Плюсы и минусы комментирования кода: <https://www.quora.com/What-are-the-benefits-and-drawbacks-of-code-documentation-versus-commenting-within-the-code-itself>.
79. Согласие обязательно: <https://auth0.com/docs/secure/data-privacy-and-compliance/gdpr/gdpr-conditions-for-consent#:~:text=According%20to%20Article%207%20of,withdraw%20consent%20at%20any%20time>.
80. Групповой гражданский иск против CRA в связи с кражей данных: <https://rhelaw.com/class-action/cra-privacy-breach-class-action-2>.
81. OWASP, десять главных ошибок, связанных с целостностью программ и данных: https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures.
82. Не доверяйте никому, даже своей маме: «Alice and Bob Learn Application Security».
83. OWASP, десять главных проблем целостности программ, как избежать: https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures.
84. Определение неизменяемости, сайт Oxford Languages.
85. Еще о неизменяемости: <https://jamstack.org/glossary/immutable/#:~:text=While%20a%20%22mutable%22%20item%20can,is%20routed%20to%20them%20accordingly>.
86. Определение полного перебора: <https://www.merriam-webster.com/dictionary/brute-force#:~:text=%3A%20relying%20on%20or%20achieved%20through,planned%2C%20or%20precisely%20directed%20methods>.
87. Блокирование атак полным перебором: https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks.
88. Определение элемента безопасности от NIST: https://csrc.nist.gov/glossary/term/security_control#:~:text=Definitions%3A,set%20of%20defined%20security%20requirements.
89. Элементы безопасности F5: <https://www.f5.com/labs/learning-center/what-are-security-controls#:~:text=Control%20Functions,-Preventative%20controls%20describe&text=activity%20from%20occurring-,Examples%20include%20physical%20controls%20such%20as%20fences%2C%20locks%2C%20and%20alarm,%2C%20data%20classification%2C%20and%20auditing>.
90. «Technical debt is security debt»: Dave Lewis, Cisco.
91. Духоподъемная история о GRC: Крис Карпентер.
92. «Protecting Against Unexpected System Calls»: https://www.usenix.org/legacy/event/sec05/tech/full_papers/linn/linn.pdf.

Глава 5. Рекомендации, зависящие от технологий

93. SOAP теряет популярность: <https://www.postman.com/state-of-api/api-technologies>.

94. GraphQL больше не популярен: https://wundergraph.com/blog/graphql_rest_openapi_trend_analysis_2023.
95. Атаки на API: <https://salt.security/blog/understanding-api-attacks-why-are-they-different-and-how-can-you-stop-them>.
96. OWASP, десять главных проблем безопасности API: <https://owasp.org/www-project-api-security>.
97. Инвентаризация API: <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management>.
98. Размер WAF: <https://curity.io/resources/learn/api-security-best-practices>.
99. Аутентификация: <https://stackoverflow.blog/2021/10/06/best-practices-for-authentication-and-authorization-for-rest-apis>.
100. Дополнительные рекомендации по безопасности API: <https://blog.hubspot.com/website/api-security#best-practices>.
101. Внутреннее руководство Google, несколько извлечений: <https://developers.google.com/maps/api-security-best-practices>.
102. Изменения в «OWASP десять главных проблем безопасности»: <https://nonamesecurity.com/blog/changes-to-the-owasp-api-security-top-ten-2019-to-2023>.
103. Управление ключами API: <https://stripe.com/docs/keys>.
104. Все о безопасности мобильных приложений, «NowSecure Mobile App Book»: <https://books.nowsecure.com/secure-mobile-development/en>.
105. Используйте сертификаты для подписания кода: <https://www.nowsecure.com/blog/2023/02/24/13-mobile-app-security-best-practices>.
106. В Android память на самом деле не стирается: <https://books.nowsecure.com/secure-mobile-development/en/coding-practices/understand-secure-deletion-of-data.html>.
107. Избегайте UUID: <https://books.nowsecure.com/secure-mobile-development/en/sensitive-data/limit-use-of-uuid.html>.
108. Сохраняйте настройки в коде: <https://books.nowsecure.com/secure-mobile-development/en/sensitive-data/protect-application-settings.html>.
109. Не включайте конфиденциальные данные приложений в резервные копии телефона: <https://books.nowsecure.com/secure-mobile-development/en/sensitive-data/avoid-storing-app-data-in-backups.html>.
110. Отключайте автокоррекцию для любой конфиденциальной информации, а не только для полей паролей: <https://books.nowsecure.com/secure-mobile-development/en/caching-logging/be-aware-of-the-keyboard-cache.html>.
111. Избегайте WebView: <https://books.nowsecure.com/secure-mobile-development/en/webviews/prevent-framing-and-clickjacking.html>.
112. Осторожно выбирайте класс защиты: <https://books.nowsecure.com/secure-mobile-development/en/ios/use-the-keychain-carefully.html>.
113. Блокируйте кеширование моментальных снимков приложения: <https://books.nowsecure.com/secure-mobile-development/en/ios/avoid-cached-application-snapshots.html>.

114. Средства безопасности памяти в IOS: <https://books.nowsecure.com/secure-mobile-development/en/ios/implement-protections-against-buffer-overflow-attacks.html>.
115. Явно отключайте кеширование: <https://books.nowsecure.com/secure-mobile-development/en/ios/avoid-caching-https-requests-responses.html>.
116. Книга по безопасной разработке мобильных приложений: <https://books.nowsecure.com/secure-mobile-development/en/ios/implement-app-transport-security.html>.
117. Избегайте файлов, доступных для чтения всем, в Android: <https://books.nowsecure.com/secure-mobile-development/en/android/implement-file-permissions-carefully.html>.
118. Используйте намерения с осторожностью: <https://books.nowsecure.com/secure-mobile-development/en/android/implement-intents-carefully.html>.
119. Поставщики контента: <https://books.nowsecure.com/secure-mobile-development/en/android/implement-content-providers-carefully.html>.
120. Защищайте службы приложений от внешних вызовов: <https://books.nowsecure.com/secure-mobile-development/en/android/protect-application-services.html>.
121. Избегайте сохранения кешированных фотографий, снятых камерой: <https://books.nowsecure.com/secure-mobile-development/en/android/avoid-storing-cached-camera-images.html>.
122. Избегайте кеширования объектов GUI: <https://books.nowsecure.com/secure-mobile-development/en/android/avoid-gui-objects-caching.html>.
123. Подписывайте свой APK в Android: <https://books.nowsecure.com/secure-mobile-development/en/android/android-apk-signing.html>.
124. Определение веб-сокета: <https://en.wikipedia.org/wiki/WebSocket>.
125. Сравнение веб-сокетов с HTTP: <https://ably.com/topic/websockets-vs-http>.
126. Веб-сокеты: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>.
127. Еще о веб-сокетах: <https://datatracker.ietf.org/doc/html/rfc6455>.
128. Межсайтовый перехват веб-сокетов: <https://ably.com/topic/websocket-security>.
129. Избегайте туннелирования: <https://devcenter.heroku.com/articles/websocket-security>.
130. Максимальный размер полезной нагрузки и формат сообщений: <https://www.freecodecamp.org/news/how-to-secure-your-websocket-connections-d0be0996c556>.
131. Модель безопасности бессерверных приложений в AWS: <https://aws.amazon.com/blogs/architecture/architecting-secure-serverless-applications>.
132. Безопасность на уровне функций: <https://www.checkpoint.com/cyber-hub/cloud-security/what-is-serverless-security/5-serverless-security-tips-and-best-practices>.
133. Тайм-ауты функций: <https://protectonce.com/the-10-best-practices-for-serverless-security>.
134. ACL и VPC: <https://medium.com/@saurabhmarketer23/serverless-security-best-practices-ensuring-robustness-with-aws-lambda-311643eef268>.

- 135. IOT от OWASP: https://wiki.owasp.org/index.php/OWASP_Internet_of_Things_Project#tab=IoT_Top_10.
- 136. Отслеживайте устройства интернета вещей в сети: <https://securityscorecard.com/blog/best-practices-for-securing-internet-of-things>.
- 137. Защита конечных точек: <https://www.fortinet.com/resources/cyberglossary/iot-best-practices>.

Глава 6. Популярные языки программирования

- 138. Разработчики любят JavaScript: <https://radixweb.com/blog/top-javascript-usage-statistics>.
- 139. Что такое JavaScript: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/First_steps/What_is_JavaScript.
- 140. Typescript транслируется на JavaScript: <https://www.toptal.com/typescript/typescript-vs-javascript-guide>.
- 141. Изобретение JavaScript: https://www.w3schools.com/js/js_history.asp.
- 142. Избегайте встроенных скриптов: <https://sugarclub.sugarcrm.com/dev-club/b/dev-blog/posts/csp>.
- 143. Избегайте DOM XSS: https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html.
- 144. Предотвращение XSS: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.
- 145. Используйте режим strict: https://www.w3schools.com/js/js_strict.asp.
- 146. DOM Purify: <https://github.com/cure53/DOMPurify#dompurify>.
- 147. Внедрение прототипа: <https://www.synopsys.com/blogs/software-security/javascript-security-best-practices.html>.
- 148. Проверка целостности подресурсов: <https://raygun.com/blog/js-security-vulnerabilities-best-practices>.
- 149. Избегайте глобальных переменных: <https://aptori.dev/blog/javascript-security-best-practices-for-building-secure-applications>.
- 150. Что такое CSS: <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- 151. Конфиденциальная информация в CSS: https://cheatsheetseries.owasp.org/cheatsheets/Securing_Cascading_Style_Sheets_Cheat_Sheet.html.
- 152. Внедрение в CSS: <https://www.geeksforgeeks.org/how-to-secure-cascading-style-sheets>.
- 153. Избегайте комментариев в HTML: https://community.pega.com/sites/pdn.pega.com/files/help_v731/security/best-practices/sec-security-guidelines-custom-HTML-ref.htm.
- 154. Опасные атрибуты и элементы в HTML: <https://dev.to/ieeecsvitc/security-vulnerabilities-and-prevention-in-html5-p0m>.
- 155. Что такое HTML 5: <https://www.ericom.com/glossary/what-is-html5>.

156. Шпаргалка OWASP по безопасности HTML 5: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html.
157. Как сообщать об ошибках в PyPi: <https://www.python.org/dev/security>.
158. Рекомендации по программированию на Python: <https://avatao.com/blog-common-issues-and-best-practices-in-python>.
159. Использование виртуальной среды: <https://www.securecoding.com/blog/python-security-practices-you-should-maintain>.
160. Избегайте eval() и других подобных функций: <https://aptori.dev/blog/python-security-cheat-sheet-for-developers>.
161. Используйте Python 3, а не 2 и т. д.: https://res.cloudinary.com/snyk/image/upload/v1551385381/Python_Security_Best_Practices_Cheat_Sheet_.pdf.
162. Bandit: <https://bandit.readthedocs.io/en/latest>.
163. Об использовании линтера и других рекомендациях для Python: <https://snyk.io/blog/python-security-best-practices-cheat-sheet>.
164. «Alice and Bob Learn Application Security», раздел о базах данных.
165. Рекомендации по безопасной работе с SQL от Microsoft: <https://learn.microsoft.com/en-us/sql/relational-databases/security/sql-server-security-best-practices?view=sql-server-ver16>.
166. Еще о безопасности SQL: https://www.netwrix.com/sql_server_security_best_practices.html.
167. Факты о Node: <https://nodejs.org>.
168. Не запускайте Node от имени root: <https://semaphoreci.com/blog/securing-nodejs>.
169. Линтеры для Node.JS: <https://www.makeuseof.com/node-linters-top>.
170. Уязвимости Node: <https://nodejs.org/en/guides/security>.
171. Рекомендации по работе с Node, Snyk: <https://snyk.io/learn/nodejs-security-best-practice>.
172. Рекомендации по безопасности в Node: <https://nodejs.org/en/learn/getting-started/security-best-practices>.
173. Шпаргалка OWASP по Node: https://cheatsheetseries.owasp.org/cheatsheets/Nodejs_Security_Cheat_Sheet.html.
174. Использование Java: <https://thenewstack.io/java-usage-keeps-climbing-according-to-new-survey>.
175. Наставление по безопасному кодированию для Oracle: <https://www.oracle.com/java/technologies/javase/seccodeguide.html>.
176. Fred Long «The CERT Oracle Secure Coding Standard for Java».
177. Шпаргалка OWASP по десериализации: https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html.
178. Документация по безопасности в Oracle Java SE: <https://docs.oracle.com/en/java/javase/14/security/index.html>.
179. Шпаргалка OWASP по безопасности в Java: https://cheatsheetseries.owasp.org/cheatsheets/Java_Security_Cheat_Sheet.html.
180. Стандарт кодирования SEI CERT Oracle для Java: <https://wiki.sei.cmu.edu/confluence/display/java/SEI+CERT+Oracle+Coding+Standard+for+Java>.

181. Основы Typescript: <https://cloudblogs.microsoft.com/opensource/2018/03/05/typescript-overview>.
182. Статья в Snyk о TypeScript: <https://snyk.io/blog/best-practices-react-typescript-security>.
183. Рекомендации по программированию на TypeScript 2021: <https://medium.com/@warkiringoda/typescript-best-practices-2021-a58aee199661>.
184. Aptori о безопасном кодировании на TypeScript: <https://aptori.dev/blog/secure-coding-in-typescript-best-practices-to-build-secure-applications>.
185. Рекомендации по TypeScript: <https://github.com/andresousa/typescript-best-practices>.
186. Различия между C#, C++ и C: <https://csharp-station.com/understanding-the-differences-between-c-c-and-c>.
187. Используйте средства .NET: <https://clickysoft.com/dot-net-security-best-practices>.
188. Атрибуты регулярных выражений и прочее: <https://www.syncfusion.com/blogs/post/10-practices-secure-asp-net-core-mvc-app>.
189. Вся специфика C#: <https://learn.microsoft.com/en-us/aspnet/web-forms/overview/older-versions-security/introduction/security-basics-and-asp-net-support-cs>.
190. Протоколирование в C#: <https://learn.microsoft.com/en-us/dotnet/core/extensions/logging?tabs=command-line>.
191. Протоколирование в .NET: <https://www.papertrail.com/solution/tips/7-best-practices-for-c-logging-with-examples>.
192. Средства безопасности в C#: <https://clouddevs.com/c-sharp/cybersecurity>.
193. Пространство имен auth в NET: <https://learn.microsoft.com/en-us/aspnet/core/security/authorization/introduction?view=aspnetcore-8.0>.
194. HTTP-запросы: <https://learn.microsoft.com/en-us/aspnet/core/security/enforcing-ssl?view=aspnetcore-8.0&tabs=visual-studio%2Clinux-ubuntu>.
195. Не используйте инструмент управления секретами: <https://learn.microsoft.com/en-us/aspnet/core/security/app-secrets?view=aspnetcore-8.0&tabs=windows>.
196. Защита от CSRF: <https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery?view=aspnetcore-8.0>.
197. Razor кодирует автоматически: <https://learn.microsoft.com/en-us/aspnet/core/security/cross-site-scripting?view=aspnetcore-8.0>.
198. Другие типы аутентификации: <https://www.stackhawk.com/blog/guide-to-security-in-net>.
199. Избегайте XXE: <https://www.syncfusion.com/blogs/post/10-practices-secure-asp-net-core-mvc-app>.
200. Угол C#: <https://www.c-sharpcorner.com/article/secure-coding-in-c-sharp-protecting-applications-and-user-data>.
201. Что такое PHP: <https://www.php.net/manual/en/intro-what-is.php>.
202. Руководство по безопасности для PHP: <https://www.php.net/manual/en/security.php>.

203. Экранирование в PHP: <https://medium.com/@london.lingo.01/advanced-php-security-techniques-protecting-your-code-from-attacks-7d22ac5e2f74>.
204. Список бесплатных инструментов SAST для PHP: https://owasp.org/www-community/Source_Code_Analysis_Tools.
205. Модуль Snuffleupagus: <https://snuffleupagus.readthedocs.io>.
206. Настройки PHP для университета Queens: https://www.queensu.ca/its/sites/itswww/files/uploaded_files/Governance%20Instruments/queens-php-development-handbook.pdf.
207. Руководство OWASP по укреплению PHP: https://cheatsheetseries.owasp.org/cheatsheets/PHP_Configuration_Cheat_Sheet.html.
208. История C: <https://www.linkedin.com/pulse/brief-history-c-evangelos-pat-sourakos>.
209. История C++: <https://www.oreilly.com/library/view/object-oriented-programming/9789332503663/xhtml/head-0045.xhtml>.
210. Robert Seacord «Secure Coding in C and C++»: https://insights.sei.cmu.edu/documents/1312/2005_009_001_52710.pdf.
211. Избегайте использования void* и приведения типов: <https://www.incredibuild.com/blog/top-10-secure-c-coding-practices>.
212. SEI Cert C++: <https://resources.sei.cmu.edu/downloads/secure-coding/assets/sei-cert-cpp-coding-standard-2016-v01.pdf>.
213. Флаги компилятора: <https://best.openssf.org/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.html>.
214. RAI: <https://blog.bytehacker.in/secure-coding-in-cpp>.

Глава 7. Популярные каркасы

215. Что такое Express? <https://expressjs.com>.
216. Рекомендации по безопасному использованию Express.js, из официального источника: <https://expressjs.com/en/advanced/best-practice-security.html>.
217. Избегайте уязвимых регулярных выражений: <https://github.com/davis-jam/vuln-regex-detector>.
218. Hydra: <https://github.com/vanhauser-thc/thc-hydra>.
219. Модуль express validator: <https://express-validator.github.io/docs>.
220. Проверка JWT: <https://www.freecodecamp.org/news/express-js-security-tips>.
221. Рекомендации по безопасности при работе с React: <https://relevant.software/blog/react-js-security-guide>.
222. Пакет Redux.js: <https://redux.js.org>.
223. Рекомендации по безопасности при работе с React.js: <https://snyk.io/blog/10-react-security-best-practices>. (Примечание: эта статья была переписана плагиаторами, а в большинстве случаев просто скопирована во многих других блогах, но этот был первым, поэтому я ссылаюсь на него.)

224. Хорошие библиотеки для React: <https://www.turing.com/kb/reactjs-security-best-practices#reactjs-security-best-practices>.
225. Security-Realm: <https://www.freecodecamp.org/news/best-practices-for-security-of-your-react-js-application>.
226. Официальная страница по безопасности в Angular: <https://angular.dev/best-practices/security>.
227. Определение доверенных типов в W3C school: <https://w3c.github.io/trusted-types/dist/спec>.
228. Рекомендации Snyk по Angular: <https://snyk.io/blog/angular-security-best-practices>.
229. Защитники маршрутов: <https://www.scholarhat.com/tutorial/angular/tips-to-secure-your-angular-applications>.
230. Преимущества Vue.js: <https://www.monocubed.com/blog/advantages-of-vue-js>.
231. Официальные рекомендации по безопасности во Vue.js: <https://vuejs.org/guide/best-practices/security>.
232. <https://vuejs.org/guide/scaling-up/ssr.html>.
233. Vue.js: <https://medium.com/@wasantosfi/some-security-practices-to-vue-js-developments-e7512d07536d>.
234. jQuery устарел и вышел из моды: <https://blog.jquery.com/2024>.
235. «PSA jQuery is bad for security»: <https://palant.info/2020/03/02/psa-jquery-is-bad-for-the-security-of-your-project>.
236. Написание безопасного кода на jQuery: <https://reintech.io/blog/securing-jquery-code-xss-vulnerabilities>.
237. Наставления Microsoft по безопасному кодированию в .NET: <https://learn.microsoft.com/en-us/dotnet/standard/security/secure-coding-guidelines>.
238. Идентификация и авторизация в .NET: <https://learn.microsoft.com/en-us/dotnet/standard/security/key-security-concepts>.
239. Криптографическая модель Microsoft: <https://learn.microsoft.com/en-us/dotnet/standard/security/cryptography-model>.
240. Шпаргалка OWASP по .NET: https://cheatsheetseries.owasp.org/cheatsheets/DotNet_Security_Cheat_Sheet.html.
241. Руководство по безопасности в .NET от Stackhawk: <https://www.stackhawk.com/blog/guide-to-security-in-net>.
242. Присваивание непредусмотренных полей: <https://andrewlock.net/preventing-mass-assignment-or-over-posting-in-asp-net-core>.
243. Официальное руководство по безопасности в Ruby on Rails: https://guides.rubyonrails.org/action_controller_overview.html#session.
244. Шпаргалка OWASP по Ruby on Rails: https://cheatsheetseries.owasp.org/cheatsheets/Ruby_on_Rails_Cheat_Sheet.html.
245. Рекомендации по безопасности в Ruby on Rails: <https://www.codementor.io/ruby-on-rails/tutorial/ruby-on-rails-security-best-practices>.

- 246. Spring Boot заменил Struts: <https://www.orientsoftware.com/blog/spring-vs-spring-boot>.
- 247. Основы Spring: <https://spring.io>.
- 248. Безопасность в Spring Boot: <https://developer.okta.com/blog/2018/07/30/10-ways-to-secure-spring-boot>.
- 249. Интеграция безопасности Spring с облаком: <https://www.geeksforgeeks.org/best-practices-to-secure-microservices-with-spring-security>.
- 250. Статья в Geeks for Geeks о безопасности в Spring: <https://www.geeksforgeeks.org/spring-security-secure-your-web-application>.
- 251. Cloud devs: <https://clouddevs.com/flask/best-practices-and-techniques>.
- 252. Read-The-Docs о безопасности в Flask: <https://flask-security-too.readthedocs.io/en/stable>.
- 253. Рекомендации по безопасности в Flask: <https://python.plainenglish.io/the-best-security-practice-of-your-flask-application-api-691a21e37c27>.
- 254. Рекомендации по безопасности в Flask: <https://medium.com/@sarah.schmidtdean/safeguarding-your-database-in-flask-e1a1302e3e33>.
- 255. Рекомендации по защите Flask-приложений: <https://escape.tech/blog/best-practices-protect-flask-applications/#owasp-top-10>.

Глава 8. Категории уязвимостей

- 256. Логические просчеты: <https://portswigger.net/web-security/logic-flaws>.
- 257. Определение переполнения буфера по версии OWASP: https://owasp.org/www-community/vulnerabilities/Buffer_Overflow.
- 258. Статья в википедии о безопасности доступа к памяти: https://en.wikipedia.org/wiki/Memory_safety.
- 259. Определение адресного пространства: https://en.wikipedia.org/wiki/Address_space.
- 260. ASLR и другие определения: <https://www.fortinet.com/resources/cyberglossary/buffer-overflow>.
- 261. Использование GCC и другие рекомендации по предотвращению проблем доступа к памяти: <https://anchore.com/blog/breaking-down-nist-ssdf-compilers-and-interpreter-security>.
- 262. Определение идентификации: <https://www.devx.com/terms/network-identity>.
- 263. Определение мошеннической транзакции: <https://www.datavisor.com/wiki/transaction-fraud>.
- 264. Убытки от мошенничества в сфере интернет-торговли: <https://www.statista.com/statistics/1273177/ecommerce-payment-fraud-losses-globally>.
- 265. Определение атаки воспроизведением: https://csrc.nist.gov/glossary/term/replay_attack.

- 266. Предотвращение атак воспроизведением: https://en.wikipedia.org/wiki/Replay_attack.
- 267. Определение пересечения границ доверия: <https://cwe.mitre.org/data/definitions/501.html>.
- 268. Номер CVE для атаки на Log4j: <https://nvd.nist.gov/vuln/detail/CVE-2021-44832>.
- 269. CWE для SQLi: <https://cwe.mitre.org/data/definitions/89.html>.
- 270. Определение некорректной обработки файла, риски и смягчение последствий: <https://securityforeveryone.com/scan/parent/improper-file-process>.
- 271. Определение ООП от Reddit: https://www.reddit.com/r/computerscience/comments/1an2ssn/strictly_speaking_what_is_an_object_in_programming.
- 272. Объектно ориентированное программирование и безопасность: https://www.adacore.com/uploads/techPapers/Safety_Security_OOP.pdf.
- 273. Различие между классом и объектом: https://www.codecademy.com/forum_questions/558cd3fc76b8fe06280002ce.
- 274. Определение полиморфизма: <https://stackify.com/oop-concept-polymorphism/#:~:text=Polymorphism%20is%20one%20of%20the,types%20through%20the%20same%20interface>.
- 275. Определение состояния гонки: <https://www.techtarget.com/searchstorage/definition/race-condition>.
- 276. Причины гонок: <https://www.indusface.com/learning/what-is-a-race-condition-vulnerability>.
- 277. Состояния гонки позволяют превысить логические лимиты: <https://portswigger.net/web-security/race-conditions>.
- 278. Еще о предотвращении гонок: <https://www.imperva.com/learn/application-security/race-condition>.
- 279. Неосвобожденные ресурсы: <https://cwe.mitre.org/data/definitions/404.html>.

Глава 10. Проектирование

- 280. Определение моделирования угроз от OWASP: https://owasp.org/www-community/Threat_Modeling.
- 281. Система 4 вопросов Адама Шостака по моделированию угроз: <https://github.com/adamshostack/4QuestionFrame>.
- 282. Определение диаграммы потоков данных: <https://www.lucidchart.com/pages/data-flow-diagram>.
- 283. Появление пользовательских историй: <https://www.knowledgehut.com/blog/project-management/3-c-of-user-stories>.
- 284. OWASP о пользовательских историях по безопасности: <https://github.com/OWASP/user-security-stories/blob/master/user-security-stories.md>.

Глава 11. Кодирование

- 285. Определение анализа кода: <https://www.synopsys.com/glossary/what-is-code-review.html>.
- 286. Когда обновлять зависимости: <https://softwareengineering.stackexchange.com/questions/340705/when-should-dependencies-be-updated>.

Глава 12. Тестирование

- 287. Определение тестового покрытия: <https://testsigma.medium.com/test-coverage-in-software-testing-a-comprehensive-guide-8b3d0386a0a7>.
- 288. Глубина и покрытие: <https://www.packetlabs.net/posts/coverage-based-penetration-testing>.

Глава 13. Выпуск и развертывание

- 289. OWASP, десять главных рисков CI/CD: <https://owasp.org/www-project-top-10-ci-cd-security-risks>.
- 290. OWASP, десять главных ошибок, связанных с целостностью программ и данных, A08:2021: https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures.

Глава 14. Сопровождение

- 291. Определение мониторинга программного обеспечения: <https://www.techopedia.com/definition/4313/monitoring-software>.
- 292. Зачем мониторить программное обеспечение: <https://www.techtarget.com/searchitoperations/definition/IT-monitoring>.
- 293. Определение уведомления безопасности: <https://cyberpedia.reasonlabs.com/EN/security%20alerts.html>.
- 294. Различие между мониторингом и наблюдаемостью: <https://aws.amazon.com/compare/the-difference-between-monitoring-and-observability>.
- 295. Определение WAF: <https://www.cloudflare.com/en-ca/learning/ddos/glossary/web-application-firewall-waf>.
- 296. Определение сети доставки контента: <https://www.cloudflare.com/en-ca/learning/cdn/what-is-a-cdn>.
- 297. Безопасность CDN: <https://www.sentinelone.com/cybersecurity-101/what-is-a-cdn-content-delivery-network>.

298. Определение RASP: https://en.wikipedia.org/wiki/Runtime_application_self-protection.
299. Определение шлюза API: <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>.
300. Определение инцидента безопасности: https://csrc.nist.gov/glossary/term/security_incident.
301. Определение непрерывности бизнеса: https://en.wikipedia.org/wiki/Business_continuity_planning.
302. Определение аварийного восстановления: <https://cloud.google.com/learn/what-is-disaster-recovery>.

Ответы к упражнениям

Глава 1. Введение в основы безопасности

1. Мы заново тестируем код после исправления ошибки, чтобы убедиться, что случайно не создали новую проблему.
2. Мы поддерживаем каркасы в актуальном состоянии, чтобы всегда работать с поддерживаемой версией, чтобы впоследствии было проще перейти на новую версию, чтобы получать новую функциональность (в том числе относящуюся к безопасности) и исправления безопасности.
3. Мне нравится новый каркас OWASP DSOMM. Я беру на вооружение идеи, которые мне больше по душе, и применяю их, а остальные оставляю без внимания. Предлагаемые возможности весьма разносторонние; похоже, они подходят большинству современных SDLC.
4. Моя модель угроз в чем-то похожа на поведение всех женщин: мы должны помнить о рисках, когда проводим время с мужчинами, которых не знаем и которым не доверяем. Кроме того, как человек, имеющий много поклонников в интернете, я сталкиваюсь с дополнительными угрозами – в моей жизни уже появлялись сталкеры, и не один. Поэтому я должна оберегать адрес своего дома (от незваных визитеров), свою профессиональную репутацию (от клеветы и самозванства) и даже свой домен (есть и такие «поклонники», которые раз за разом провозглашали старый домен моей компании, `wehackringle.com`, вредоносным, что привело к потере бизнеса). Как человек, зарабатывающий на жизнь прежде всего своим интеллектуальным трудом, я должна защищаться от плагиата, а это невероятно трудная битва. Для публичного человека список угроз (к сожалению) очень длинный.
5. Когда я нажимаю «спасибо, нет» в ответ на предложение получать cookie от сайта какой-то компании, я предполагаю, что она с уважением отнесется к моему выбору. То есть что их маркетологи не будут следить за моими похождениями в интернете без моего согласия. Другой пример подразумеваемого доверия – подача заявки на участие в конференции. Я посылаю

свою работу оргкомитету и предполагаю, что ее прочтут и решат, стоит ли докладывать ее на конференции. Но что, если кто-то из увидевших мою работу решить немного изменить ее, а затем подать на другую конференцию под своим именем? Как я узнаю об этом? Как это доказать? Я не знаю ни об одном таком случае, но тем не менее это угроза, о которой мало кто думает. Большинство из нас доверяют по умолчанию много раз на дню.

6. Я узнала о нескольких новых способах работы с группами GRC (управление рисками и комплаенсом), когда писала эту книгу, и планирую использовать эти идеи (разделение информации) в последующей работе. Особенно когда испытываю трудности при работе с такими группами.
7. Настраивая менеджер паролей для всей своей компании (We Hack Purple), я создала разные сейфы для разных команд. Один набор паролей был нужен отделу маркетингу, например, для социальных сетей. Другой – для администрации, т. е. для меня и моего заместителя. Это сильно упростило смену учетных данных (переустановку всех паролей) при увольнении сотрудника, потому что я точно знала, к чему он имел доступ, а к чему не имел. Если бы в компании не применялся принцип наименьших привилегий, то сотрудник мог бы получить доступ, которого у него не должно быть. Например, заместитель администратора мог бы твитнуть в аккаунте компании «А я вообще не люблю фиолетовый цвет!» – понятно, что это было бы не хорошо. Кроме того, мне пришлось бы менять все без исключения пароли при увольнении любого сотрудника, а это было бы пустой и вовсе не интересной тратой времени.
8. Библиотека, содержащая функции для управления безопасностью (проверка входных данных, кодирование вывода, шифрование и т. д.) была бы отличным выбором по умолчанию. «Пользуйтесь библиотекой для реализации всех элементов безопасности, не пишите собственные». Тогда все сильно упрощается!
9. Когда компания выставляет в интернет API без шлюза и думает, что поэтому его никто не увидит, это попытка безопасности через неясность. Да, вы не объявили, как их можно найти, но очень скоро кто-нибудь их обнаружит. Еще один пример безопасности через неясность – не публиковать много данных о себе в сети, например о своем адресе, о составе семьи, о том, как выглядит ваш рабочий пропуск, о дате своего рождения и т. д. Чем больше информации вы сообщаете, тем проще украсть вашу личность, выдать себя за вас или ответить на контрольные вопросы на любом сайте, где вы оставили правильные ответы.

Примечание: не оставляйте правильные ответы на контрольные вопросы. Лучше введите какой-то длинное, случайное, уникальное значение – как при вводе пароля. А затем сохраните его вместе с паролем в менеджере паролей. Это гораздо безопаснее, чем (1) делиться своими персональными данными со случайным сайтом, на котором вы создали учетную запись, и (2) указывать что-то, о чем можно догадаться, потратив 30 с на изучение вашей страницы в Facebook.

10. Если в вашем приложении есть корзина для покупок, то в случае ошибки при оформлении заказа обязательно нужно откатиться и произвести закрытие. Что, если вы списали со счета деньги, но не отправили купленное? Или наоборот? Или пользователь придумал, как воспользоваться ошибкой в процедуре оформления заказа, чтобы, заплатив один раз, получить несколько товаров? При попытке продолжить работу после системной ошибки с транзакцией может случиться много чего неприятного, поэтому не думайте, что так проще, – это заблуждение.

Глава 2. В начале пути

1. Ложь. Все входные данные нужно проверять. Что, если этот API вызывается из какого-то другого места? Что, если ваше приложение было скомпрометировано? Что, если приложение плохо проверяет данные и что-то пропускает?
2. Один раз. Если вы все сделали правильно.
3. Сначала аутентификация, потом авторизация. Это важно, потому что вы должны знать, кого собираетесь впустить в систему. Мы не хотим, чтобы человек даже на один-единственный шаг проник в систему, пока не опознали его.
4. Ключи API, строки подключения, пароли, сертификаты, хеш, секреты API, лицензионные ключи.
5. Правда.
6. Ложь.
7. Злоумышленники могут использовать эту информацию для организации более результативных атак. Это может сбивать с толку пользователей, выставлять нас непрофессионалами и плохо отражаться на нашем приложении.
8. Злоумышленник может использовать ошибки, чтобы перевести наше приложение в неизвестное состояние, а затем атаковать его. Это может сбивать с толку пользователей, выставлять нас непрофессионалами и плохо отражаться на нашем приложении, а может привести к совершенно неожиданному результату – плохому или хорошему.

Глава 3. Улучшение

1. Да, конечно! Часто переписывать все приложение нерентабельно. Или же оно используется в закрытой сети, поэтому вероятность появления чего-то вредоносного в сети настолько мала, что риск присутствия в сети постороннего больше, чем риск уязвимости переполнения (потому что

в этот момент у вас есть удаленный доступ к серверам). Не всегда обязательно использовать языки, безопасные относительно памяти, но если нет основательных противопоказаний, то лучше использовать.

2. Не так! Ошибки возможны и в строго и в слабо типизированных языках программирования!
3. В обоих случаях!
4. Так! Не блокируйте вставку в поле пароля из буфера обмена.
5. Так. Выступайте в защиту конфиденциальности ваших пользователей, сограждан и вообще всех на свете при малейшей возможности.
6. Так! Мы обязаны защищать сервер, программное обеспечение базы данных, сеть и данные, хранящиеся в базе.
7. Не так. Глупость какая!
8. Это вопрос с подвохом! Да, это интеллектуальная собственность, и да, вы должны защищать целостность и доступность кода, но не всегда обязаны защищать его конфиденциальность. Если вы работаете над проектом с открытым исходным кодом, то приняли осознанное решение сделать его общедоступным. Это значит, что вы сохраняете авторские права и можете установить любые условия лицензии (по согласованию с другими участниками проекта). Но, сделав исходный код открытым, вы явно согласились не защищать его конфиденциальность. Он по-прежнему имеет ценность, но вы готовы поделиться им с обществом. Тем не менее вы все же хотели бы, чтобы (1) никто не мог удалить его (это было бы ужасно) и (2) никто не покушался на его целостность в дурных целях (например, не добавлял специально код, содержащий уязвимости, с целью эксплуатировать их впоследствии). Но ответ все равно «так», открыли вы свой исходный код или нет. Заботьтесь о своем коде!
9. Так-то так, но есть нюансы. Вы можете использовать сериализованные объекты, которые сами же и создали (я имею в виду, что они были сериализованы системами, которые вы или ваша организация контролирует, и вы точно знаете, что им можно доверять, и при этом нет никаких шансов, что сериализованный объект был изменен или подменен злоумышленником). Главное, что нужно запомнить: не десериализовывать объект из ненадежного источника, а таким источником может быть и ваша собственная система, если она оставляет возможность изменения сериализованных объектов третьей стороной. Если сомневаетесь, откажитесь от десериализации!
10. Не так. Когда мне говорят «Я доверяю своим сотрудникам» и выставляют это причиной, по которой можно и пренебречь гигиеной безопасности, я думаю, что собеседник не вполне понимает человеческую природу. Как насчет того, чтобы снять с банковского счета миллионы долларов, принадлежащих вашей компании, зайти в кафетерий, бросить их на ближайший столик и уйти домой? Вы так сделаете? Нет? А почему? Но... а я-то думала, что вы доверяете своим сотрудникам! Понимаете теперь,

почему это неправильный ответ? Производственные данные должны оставаться только в производственной системе, а их замаскированная версия (или искусственно сгенерированные тестовые данные) должна храниться и использоваться в менее ответственных средах.

Глава 4. Достижение цели

1. Пример технического долга – использование устаревших версий библиотек или каркасов. Их обновление – способ оплатить этот технический долг.
2. Защищайте свои рабочие учетные записи, используя длинные уникальные пароли, менеджер паролей и МФА. Мы делаем это, чтобы не потерять работу, не подвергнуть опасности свою организацию и потому что суперпользователи – лакомая мишень для злоумышленников.
3. Проверка входных данных, кодирование вывода, сканирование загруженных файлов на вирусы, аутентификация, авторизация, политики, видеокамеры, сторожевые собаки и т. д.
4. Атаки полным перебором обычно используются для взлома систем (пробивая путь сквозь аутентификацию и авторизацию), а это значит, что злоумышленник попадает в систему, где не должен находиться, – кошмар с любой точки зрения. Защититься от таких атак трудно по нескольким причинам: пользователи любят использовать пароли повторно и выбирают слабые пароли, украденные учетные данные дешево (и нелегально) продаются в интернете, в мире работает множество автоматических скриптов и ботов, готовых атаковать ваше приложение сразу по получении приказа.
5. Если целостность программы скомпрометирована, то любое ее действие может оказаться некорректным, в том числе она может выдать неверный итог при оформлении заказа или продать вам лекарство не в той дозировке или не того наименования. Результаты могут быть в буквальном смысле смертельными.
6. Пользователи – главные судьи вашей работы. Если она им не нравится, то они не будут ей пользоваться (за исключением случаев, когда это требуется по закону или когда работодатель принуждает их к этому). Если им не подходят службы, функции или условия, предлагаемые вами, то они уйдут к конкурентам. Это одна причина. А другая заключается в том, что в некоторых случаях закон обязывает нас прислушиваться к желаниям пользователей и получать их согласие. Примером может служить Общий регламент по защите данных в Европе.
7. Наличие документации означает, что мы знаем, что делает приложение и как оно устроено, что помогает нам реагировать на инциденты безопасности и обновлять его.

8. Комментарии говорят следующему разработчику, который будет сопровождать программу (а это можете быть и вы сами), что делает каждая часть кода. Это позволяет быстрее исправлять ошибки и реагировать на проблемы, связанные с безопасностью.
9. Мы должны защищать системные вызовы от внедрения кода (и прочих атак), чтобы помешать злоумышленнику перехватить контроль над нашими системами. Мы не можем позволить, чтобы контроль попал не в те руки.
10. Преимущество: экономия времени и денег. Если код уже был протестирован на безопасность, то мы знаем, что использовать его безопасно и что в нем нет дефектов безопасности. Недостаток: если код не был протестирован и в нем имеется дефект безопасности, а вы решили использовать его повторно, то теперь вы имеете два или более дефектов безопасности.
11. Если мы не будем укреплять наши системы (выбирать безопасные конфигурации), то их можно будет успешно атаковать. Популярные коммерческие системы (например, Jira или or Confluence) могут иметь уязвимости, если сконфигурированы неправильно, а это значит, что даже не особо продвинутый злоумышленник сможет успешно их эксплуатировать. Примером укрепления системы может служить изменение имени администратора, замена пароля по умолчанию на более длинный, сложный и уникальный, включение МФА, активация протоколирования и мониторинга.
12. Белым списком называется список символов (буквы, цифры, специальные символы), которые могут встречаться во входных данных, копируемых в переменную или параметр. Напротив, черный список – это список символов, которые вы считаете плохими и запрещаете. Очень трудно построить исчерпывающий черный список, тогда как идеальный белый список составить довольно легко.
13. Надежной считается система, которую мы (ИТ-департамент или владелец системы) контролируем и которая не может быть никак изменена внешним пользователем. Обычно это сервер, а не фронтальный JavaScript-интерфейс, которым можно манипулировать в веб-браузере или с помощью веб-прокси (иногда входит в состав DAST). Поскольку запросами от браузера и ответами ему можно манипулировать, доверять им не следует, иначе легко стать жертвой атаки. Мы должны выполнять все важные функции в надежной системе, чтобы гарантировать их целостность.
14. Если ваш код неудобочитаем, то его трудно анализировать (на присутствие любых проблем, в том числе проблем безопасности), трудно исправлять ошибки и добавлять новые функции. Неудобочитаемый код замедляет все действия, поэтому работа с ним обходится дороже.

15. Под безопасным умолчанием понимается задание некоторого безопасного значения по умолчанию вместо позволяющего больше, но менее безопасного.

Вымощенная дорога – это набор настроек безопасности, которые заранее заданы для пользователей или разработчиков, чтобы им было проще решать свои задачи самым безопасным способом. Это путь, которому, по мнению группы безопасности, вы должны следовать при использовании системы или выполнении некоторого действия. Он проектируется так, чтобы для решения задачи менее безопасным способом нужно было приложить дополнительные усилия.

16. Безопасное умолчание: включение МФА для новых учетных записей по умолчанию, вместо того чтобы заставлять пользователей заходить в подменю для включения этого режима уже после создания учетной записи.

Вымощенная дорога: создание на вашем любимом языке программирования шаблона для всех новых приложений, в который уже включены проверка входных данных, кодирование вывода и другие средства безопасности. Проще использовать уже готовые средства безопасности, чем писать свои с нуля.

17. Да, если вы используете конвейер CI/CD для сборки своей программы, то это часть вашей цепочки поставок. Все, что используется для сборки, – часть цепочки поставок.
18. В идеальном мире мы никогда не стали бы использовать зависимости, содержащие уязвимости, а пользовались бы только стопроцентно безопасными библиотеками. Но наш мир не идеален, и такой подход был бы чрезмерно дорогим, трудоемким и ограничивающим. Поэтому я рекомендую обновлять или вовсе удалять только зависимости, содержащие достижимые или допускающие эксплуатацию уязвимости. Если же вы не знаете точно, достижима ли уязвимость, то предполагайте худшее и применяйте к ней те же правила – первоочередное обновление или удаление. Информация о достижимости может уменьшить количество подозрительных уязвимостей на 90 % и более – огромная экономия времени.
19. В идеале мы должны были бы моделировать угрозы еще на этапе проектирования, но я с радостью соглашусь на более позднее моделирование, чем на его полное отсутствие. Но хочу подчеркнуть, что на этапе проектирования у нас достаточно информации, чтобы провести отличный сеанс моделирования угроз, но поскольку мы еще ничего не написали, то вольны вносить любые изменения в проект – и это нам ничего (или почти ничего) не будет стоить. Это также самый удобный момент для внесения крупных изменений в проект, чтобы не впихивать в него безопасность позже, что часто оказывается не лучшим решением.

Глава 5. Рекомендации, зависящие от технологии

1. Боты и автоматические атаки любого другого вида, возможно, выполняемые скриптами.
2. Да, вы можете использовать их также для бессерверных приложений.
3. Конечно, должны! Обычные практики безопасного кодирования применимы к любой технологии, в которой есть код.
4. Не так! Эту информацию мог бы использовать злоумышленник для организации синтаксически корректной атаки.
5. Да. И, быть может, еще придирчивее, если ваши веб-приложения не содержат никакой бизнес-логики. Ваши API – мозг приложения, который каждую секунду принимает бизнес-решения; очень важно тестировать их максимально придирчиво.
6. Потому что злоумышленники охотятся за ними! О старых приложениях и API часто заботятся с прохладцей и недостаточно тестируют на безопасность, поэтому они могут быть полны уязвимостей. Если нет бизнес-требования поддерживать старую версию, то отключайте неиспользуемые и не поддерживаемые API (да и любые другие системы, отвечающие этому описанию!).
7. Мобильные приложения декомпилируются злоумышленниками значительно чаще, чем приложения других типов. API и веб-приложения (по большей части) находятся на сервере, так что злоумышленник не может их получить и декомпилировать. Другое дело мобильные приложения, которые находятся прямо в устройстве, на котором работают: в телефоне!
8. Телефоны оснащены флеш-памятью типа NAND, которая постоянно пытается кешировать информацию. Добавьте сюда связку ключей, автокоррекцию, автоматическое резервное копирование приложений и пр. – все они могут сохранять конфиденциальные данные там, где им не место. Чтобы обезопасить конфиденциальные данные в телефоне, необходимо принимать дополнительные меры предосторожности.
9. Это пример применения принципов наименьших привилегий и предполагаемой возможности взлома. Мы хотим уменьшить объем запрашиваемых разрешений на случай неприятностей. Если наше приложение успешно атаковали, но ему мало что разрешено, то ущерб, который может причинить злоумышленник, существенно уменьшается, так что последствия атаки не столь катастрофичны.
10. В Android бывает, что указатель удаляется, а сами данные ничем не заменяются в памяти. В результате злоумышленник может взять телефон. Пройтись по нераспределенной памяти и найти ваши конфиденциальные данные. Нет, только не это!

11. Если наше приложение содержит конфиденциальные данные, то мы должны внимательно следить за тем, чтобы они не попали в резервную копию телефона, которая по умолчанию не защищает конфиденциальные данные должным образом.
12. Нет, не является! И Android тоже! Это значит, что мы должны принимать много дополнительных предосторожностей, чтобы избежать уязвимостей, вызванных переполнением. Будьте очень осторожны – это серьезная ошибка!
13. Двусторонний (двунаправленный). Он даже позволяет одновременно отправлять сообщения в обе стороны.
14. Нет, используется протокол WSS (Secure Web Socket Protocol), версия шифрования в процессе передачи, специфичная для WebSocket.
15. Это пример применения принципа наименьших привилегий; если мы используем туннелирование и падем жертвой XSS (что встречается весьма часто), то наши туннели станут доступны злоумышленнику! Отказ от использования туннелей значительно уменьшает поверхность атаки.
16. Транзакции должны занимать определенное время. Если какая-то транзакция длится значительно дольше обычного, то иногда это указывает на атаку. Задание тайм-аута может остановить начатую атаку, ограничить темп и сохранить деньги.
17. Из десяти главных проблем безопасности API по версии OWASP (2023) четыре связаны с управлением доступом. Злоумышленники атакуют различные уровни плохо реализованного управления доступом и добиваются успеха чаще, чем когда-либо прежде. Мы должны проверять вызов каждой функции.
18. К сожалению, злоумышленники создают вредоносные программы для атак на IoT-устройства, а продукты, обеспечивающие безопасность конечных точек, могут защитить нас от некоторых таких атак.
19. Сегментация сети – пример применения принципа предполагаемой возможности взлома, и в случае IoT нет никакой разницы: мы должны проектировать систему и планировать свои действия, считая, что в один прекрасный день нас взломают. IoT-устройства – мишени в наших сетях, и если выделить их в отдельный сегмент, то даже в случае успешной атаки злоумышленник получит доступ только к небольшой части сети.

Глава 8. Категории уязвимостей

2. Логическим просчетом называется ситуация, когда программа делает что-то неожиданное, а не то, что должна делать. Поскольку это просчет, а не дефект, ошибка кроется в логике принятия решения приложением, а не в способе кодирования. Сканеры кода могут найти ошибки кодиро-

вания, но не понимают решений, которые пытается принять программа. Они не обладают телепатическими способностями, чтобы понять, что программа сбилась с пути, который вы ей предназначали.

3. Некто использует купон на получения 50-процентной скидки при покупке брюк. Код на купоне «october2025», а сейчас на дворе октябрь. В следующем месяце он покупает еще брюки, используя код «november2025», и это снова работает. Человек злоупотребил закономерностью присваивания скидочных кодов и нанес ущерб бизнесу, который ожидал, что код будет использоваться только специальными клиентами, а не любым человеком с улицы.

Некто играет в онлайн-игру и планирует купить в ней виртуальное оружие. Он открывает игру на двух разных компьютерах, входит в нее на обеих машинах, а затем выполняет одну и ту же транзакцию в одно и то же время. Каким-то образом по завершении транзакции он получает две единицы оружия. Пострадавшей стороной является игровая компания, которая упустила дополнительную возможность получить прибыль от внутриигровых покупок, а также другие игроки, поскольку злоумышленник получил несправедливое преимущество в игре.

4. Как-то я проводила тестирование на проникновение для SharePoint и обнаружила XSS повсеместно. Я смогла сохранить XSS в именах страниц, так что если бы впоследствии на страницу зашел кто-то другой, то страница атаковала бы его. Если бы владельцы последовали советам в руководстве по укреплению, то проблема была бы решена немедленно.
5. Аутентификация отвечает на вопрос «кто вы?», а авторизация – на вопрос «разрешено ли вам здесь находиться?». Я собиралась написать, что одно невозможно без другого, но это я мыслю исключительно в категориях безопасности. С точки зрения безопасности нужно то и другое. Но предположим, что вы маркетолог, который хочет следить за людьми и преследовать их перенацеленной рекламой по всему интернету. Он мог бы аутентифицировать посетителей (выяснить, кто они такие), а затем следить за ними, не предоставляя доступа ни к чему. Поэтому я считаю, что можно использовать одно без другого, если доступ к некоторому ресурсу разрешен любому человеку.
6. Я думаю, что управлять секретами трудно по нескольким причинам. 1) Инструменты управления секретами стоят очень дорого, а значит, не каждый может ими воспользоваться. 2) Большинство из нас учили сохранять секреты в коде, а теперь мы пытаемся забыть этот урок и делать по-другому. 3) Ошибки всегда возможны.
7. Меня особенно раздражает попадание в неизвестное состояние, потому что я не знаю, что произойдет. В случае гонки или внедрения кода я знаю, что это ужасно, но могу предположить, что впоследствии. При попадании в неизвестное состояние возможно что угодно.
8. Я собираюсь использовать язык, безопасный относительно памяти, и постараюсь избегать сложности!

Глава 9. Требования

1. Я бы хотела добавить как минимум следующие требования: моделировать угрозы, использовать инструменты статического сканирования, сканирования секретов и динамического сканирования. Я бы также хотела встретиться с представителем группы безопасности на установочном совещании и задать ему ряд вопросов.
2. Веб-приложение: все данные шифруются во время передачи и при хранении. API: доступно только через шлюз API, который поддерживает следующие возможности: ограничение темпа, аутентификация, авторизация и проверка входных данных. База данных: никакие системные учетные записи не имеют привилегий владельца базы данных, а обращаться к базе данных из приложения можно только с помощью параметризованных запросов.
3. Все требования к любому проекту должны быть документированы и согласованы, если вы хотите быть уверены, что они войдут в конечный продукт. Если вас это не волнует, то можете не согласовывать.
Требования к безопасности большинству работающих в сфере ИТ не очевидны, потому что их не проходят в большинстве учебных заведений и курсов обучения, если только программа не нацелена специально на безопасность (например, курс безопасного кодирования). На общих курсах управления проектами и разработки ПО безопасность не затрагивается вовсе или затрагивается в минимальной степени.
4. На установочных совещаниях знакомятся все заинтересованные в проекте стороны, формулируются ожидания, определяются сферы ответственности и официально объявляется о старте проекта. Каждая из этих вещей чрезвычайно важна для успеха проекта.
5. В бытность свою разработчиком ПО я задавала, в частности, такие вопросы:
 - будет ли в конце тестирование на проникновение? И будет ли оно оплачиваться из бюджета моего проекта? Я предпочла бы, чтобы оплата производилась из бюджета группы безопасности;
 - будете ли вы заниматься организацией тестирования на проникновение для меня? Или я сама должна кого-то найти и все организовать? Есть ли у вас действующий механизм закупки товаров и услуг?
 - собираетесь ли вы проверять код моих приложений какими-то автоматическими инструментами? Если да, могу ли я получить к ним доступ, чтобы предварительно просканировать код и исправить ошибки?
 - можно ли уже сейчас взглянуть на формы, которые я должна буду заполнить? Чтобы я знала, что от меня потребуется. Например, обзорный документ об архитектуре?
6. Если действия по обеспечению безопасности проекта не включены в план-график, то когда предполагается выполнять эту работу? По вече-

рам и в выходные! Мы должны предусмотреть время для каждого такого действия и дополнительное время для исправления найденных недочетов (например, по результатам тестирования), тогда мы сможем лучше распределить время между работой и личной жизнью, завершить проекты вовремя и не выходя за рамки бюджета и избежать эмоционального выгорания работников и текучки кадров.

Глава 11. Кодирование

1. «Натравив» автоматический динамический сканер с фаззером (обычно его называют DAST) на веб-приложение, вы, по сути дела, атакуете его. Нажимая кнопку «Сканировать», «Атаковать» или «Активное сканирование», вы атакуете то, на что указали. В большинстве стран это незаконно, и я не оправдываю киберпреступников. Вы легко можете взломать чей-то API или веб-приложение, уничтожить данные или организовать атаку типа отказа в обслуживании. Прошу вас, не тестируйте приложение с помощью DAST, не получив на это письменного разрешения.
2. Да. Вы вправе сканировать любой код, до которого можете дотянуться, с помощью инструмента SAST, не опасаясь что-то повредить. Это неразрушающие инструменты, они лишь читают код и анализируют его на наличие уязвимостей. Если вы хотите заслужить репутацию «чудо что за парня», отправьте отчет или заполните на сайте форму, описав ошибки, найденные в проекте с открытым исходным кодом.
3. Не следя за актуальностью зависимостей, мы накапливаем технический долг, не получаем самых последних замечательных возможностей, а также исправлений безопасности. Кроме того, если в зависимости будет обнаружена серьезная уязвимость, уже эксплуатируемая на практике, то быстро перейти на новую версию будет трудно.
4. В настоящее время у меня в VS Code установлены плагины Docker, Semgrep и 42Crunch.
5. Безопасное ограждение – это технический элемент, который побуждает воздерживаться от небезопасных действий и вести себя более безопасно. Дорожные ограждения помогают автомобилям оставаться на дороге (в отличие от съезда с дороги, что небезопасно). Хорошим примером безопасного ограждения может служить запуск сканера секретов в точке подключения «перед фиксацией» системы управления версиями; он отклоняет любой код, содержащий что-то похожее на секреты, и выдает предупреждение. Другой пример – взять список «функций, которых следует избегать» из главы 6 для своего языка, а затем создать для инструмента SAST правило, которое ищет такие функции и уведомляет вас. Еще один пример безопасного ограждения – прогон этих правил SAST в IDE, а затем еще и при записи кода в систему управления версиями.

6. Мы анализируем код, чтобы повысить его безопасность и качество, улучшить удобочитаемость, стиль и т. д. Ко всему прочему, мы еще и учимся писать более качественный код. Куда ни посмотри, выигрыш!

Глава 12. Тестирование

1. Если в процесс тестирования мы «сломали» что-то, не входящее в область охвата, то нас могут привлечь к суду. Если для проведения тестирования мы найдем кого-то и не пропишем четко область охвата, то тестирующий сможет атаковать то, что мы не хотели, и «повалить» систему, уничтожить наши данные, увидеть данные, которые не должен был видеть, и т. д.
2. Подписанный документ нужен для того, чтобы его можно было предъявить в суде. Пентестеры и исследователей безопасности то и дело привлекают к суду. Имея подписанный документ с четко очерченным охватом, очень просто узнать, действовал человек подобающим или неподобающим образом.
3. Тестирование на проникновение производится специалистом, тогда как запустить сканер может любой человек, обладающий минимальными техническими знаниями. Для тестирования на проникновение применяются многочисленные инструменты и ручные методы, а цель – найти по возможности все проблемы безопасности и проверить, допускают ли они эксплуатацию. Автоматические тесты не всегда способны по-настоящему проверить найденные ими же результаты и обычно находят гораздо меньше проблем, чем талантливый пентестер.
4. Верно. Они могут уничтожить ваши данные, повлечь отказ в обслуживании и даже «повалить» весь веб-сервер. Любой динамический сканер, посылающий запросы вашей системе (а не просто пассивно читающий ответы), может причинить вред.
Примечание: инструмент DAST можно запускать и в пассивном режиме, но тогда результаты будут малоценными.
5. Комбинация автоматического и ручного тестирования всегда дает наилучшие результаты. Видеть обе стороны картины всегда полезнее.
6. Неверно. Цель фаззинга – протестировать механизм проверки входных данных.
7. Хороший пентестер может очень многое, ему вынуждены доверять, поэтому он должен иметь высокие моральные принципы. Вы даете ему ключи (учетные данные) ко многим системам, позволяете настраивать опасные инструменты внутри вашей сети, а затем использовать их для тестирования пределов ваших систем. Кроме того, мы часто нанимаем пентестера, потому что сами не являемся специалистами в этой области, а значит, понятия не имеем, что именно он делает. Мы также обычно платим пентестерам довольно высокие гонорары и позволяем им работать удаленно.

Таким образом, мы оказываем незнакомцу очень большое доверие. Если пентестер не отличается высокими моральными принципами, то может навредить нашей организации множеством способов. Он может нарушить работу наших ИТ-систем, получить доступ к конфиденциальным данным или системам или совершить мошеннические действия под предлогом тестирования на проникновение, хотя в реальности запустил всего лишь простенький автоматический сканер. Абсолютно необходимо, чтобы пентестеры были «рыцарями без страха и упрека».

8. Как и пентестеры, ИТ-специалисты могут очень многое, особенно разработчики ПО, системные администраторы и сотрудники отдела технической поддержки. Мы имеем доступ к самым разным удивительным системам, данным и привилегиям. Если ИТ-специалист становится угрозой изнутри (штатный сотрудник или сотрудник, работающий по договору, который причиняет ущерб, обычно намеренно, организации, в которой работает), то зачастую он наиболее опасен, потому что имеет самый широкий доступ. Поэтому блюдите моральные принципы. Если у вас возникает искушение сделать что-то заведомо дурное, быть может, пора сменить место работы?

Глава 13. Выпуск и развертывание

1. Конвейер CI/CD – часть вашей цепочки поставок И весьма мощный инструмент. Мы обязаны обеспечить его безопасность для защиты от неподобающего использования (намеренного или случайного). Неавторизованные пользователи не должны иметь возможность выпускать код, удалять отчеты, развертывать контейнеры, удалять или деактивировать важные тесты и т. д.
2. Подписание кода гарантирует его целостность (что он не был изменен, после того как вы создали копию) и подлинность (что он получен от вашей команды/организации, а не от злоумышленника).
3. Начиная с 2020 года, известно об атаках, в ходе которых злоумышленники подменяли, вставляли посторонний код или иным способом модифицировали добропорядочные сторонние библиотеки, так что они становились опасными. Затем те же самые лица атаковали приложения, содержащие измененную библиотеку. Были также случаи, когда люди скачивали неправильную версию библиотеки по злому умыслу или по ошибке, что также имело печальные последствия. Из-за этих новых векторов атак стало понятно, что необходимо проверять целостность всех зависимостей, используемых в приложениях.
4. Смотрите ответ 1. Злоумышленник, имеющий доступ к вашему конвейеру CI/CD, мог бы выпустить вредоносный код, отключить тесты безопасности, резко увеличить счета за облачные услуги и бессмысленно потратить

огромный объем ресурсов – и это лишь малая часть ущерба, который он мог бы причинить.

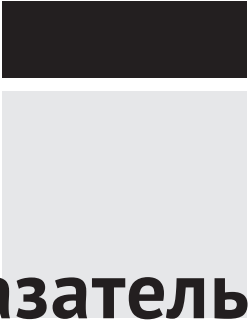
5. Я предпочла бы видеть в конвейере CI/CD инструменты SAST, SCA, IaC (инфраструктура как код) и сканирования секретов.
6. Я обычно не включаю DAST в конвейер CI/CD, за исключением случаев, когда пытаюсь доказать или продемонстрировать что-то. Мой опыт показывает, что это большая работа, дающая минимальный выигрыш. Кроме того, я предпочитаю не делать в CI ничего, что потребляет слишком много ресурсов (DoS, нагрузочное тестирование или тестирование производительности) или может причинить ущерб (например, Metasploit). Обычно статические инструменты (анализ кода) в конвейере CI лучше, чем динамические (взаимодействие с работающим приложением).
7. Сканирование секретов, потому что его легко выполнить, а обнаружение действующего секрета в коде, скорее всего, позволит избежать инцидента безопасности. Найдя действительно важный секрет, я ощущаю себя на седьмом небе; даже если всю работу проделал инструмент, я прожила день не зря.

Глава 14. Сопровождение

1. Если ваше ПО не включено в план аварийного восстановления, то в случае аварии ваша часть организации окажется не готовой, и весь бизнес может прекратить функционирование. Мне доводилось работать в местах, где случались аварии, и некоторые отделы продолжали работать, а у других не было вообще никакого плана. Аварии происходят чаще, чем вы думаете, и готовность к ним – один из способов представить себя в лучшем свете, когда будут рассматриваться кандидаты на повышение.
2. Аварийное восстановление охватывает в основном ИТ-департамент и системы, используемые для поддержки бизнеса, его цель – скорейшее восстановление функционирования бизнеса хотя бы на минимальном уровне. Если ваша компания продает вытребеньки, значит, она должна возобновить их продажи. Если она предоставляет услуги, значит, должна как можно скорее возобновить предоставление хотя бы некоторых услуг. План обеспечения непрерывности бизнеса (BCP) охватывает весь бизнес и возвращение к нормальному (полному) функционированию всего и вся (это занимает больше времени, чем аварийное восстановление). BCP распространяется на людей, рабочие места, процессы и ИТ.
3. Фраза «кому нужно, тот знает» означает, что информация должна быть известна только тем людям, которым она нужна по долгу службы, а не любому любопытствующему. Если человек принимает участие в расследовании инцидента, он, вероятно, должен знать о нем все. Если вы подслушали в лифте какую-то интересную сплетню, то вам знать не нужно.

Принцип «кому нужно, тот знает» важно применять к реагированию на инциденты безопасности, потому что 1) разглашение известной вам информации потенциально может нанести ущерб компании, клиентам, гражданам и работникам; 2) в инциденте мог участвовать инсайдер, и разглашение может предупредить его; 3) разглашение может повлечь за собой панику и напугать людей. Есть еще много причин не сплетничать на чувствительные темы, я назвала лишь малую часть.

4. Мы продолжаем тестировать, потому что инструменты тестирования регулярно обновляются: добавляются новые типы тестов, в библиотеках и каркасах постоянно обнаруживаются новые уязвимости, да и при предыдущем тестировании мы могли что-то упустить.
5. Несколько уровней обороны организуется на случай, если один не сработает или злоумышленник сумеет его обойти.
6. Если ваша работа состоит в написании кода, то вы должны позаботиться о его безопасности, для того вы и читаете эту книгу. Исследователи данных работают с нашими самыми ценными цифровыми ресурсами: данными! Мы должны защищать их всеми силами!



Предметный указатель

Символы

42Crunch, плагин для IDE, 157
.NET (Core), 221
 кодирование вывода, 50

A

AAA (аутентификация, авторизация, аудит, 54
ACL (списки управления доступом), 56
 групповые политики, 57
AD (Active Directory)
и идентификация, 54
Angular, 213
API (интерфейс прикладного программирования), 152
 охват, 309
 рекомендации, 152
 тестирование, 311
 шлюзы, 25
DPAPI (Data Protection API), 222
ARC (автоматический подсчет ссылок), 162
ASLR (рандомизация распределения адресного пространства), 162, 248
ASVS (стандарт проверки безопасности приложения), 36
ATS (транспортная безопасность приложения), 162

B

BCP (план обеспечения непрерывности бизнеса), 30
BOLA (нарушение авторизации на уровне объекта), 264

Burp Suite Pro, 304

C

C#, 194
C/C++, 203
CCPA (закон штата Калифорния о защите конфиденциальности потребителей), 35, 64
CDN (сеть доставки содержимого), 337
ChatGPT, 352
Checkmarx, 298
CIA триада (конфиденциальность, целостность, доступность, 25
CI/CD (непрерывная интеграция и непрерывная доставка), 322
 инструменты SAST первого поколения, 323
 линтинг, 326
 нагрузочные тесты, 324
 обеспечение безопасности конвейера, 327
 прерывание сборки, 324
 сканеры IaC, 326
 сканирование секретов, 325
 тесты производительности, 324
 управление доступом, 327
 фаззинг, 324
 целостность пакетов, 327
DAST (динамические автоматические сканеры), 323
SaaS (программное обеспечение как услуга), 329
SCA (анализ состава программы), 326
CIS (Центр интернет-безопасности), 37

COEP (Cross-Origin Embedder Policy), 70
Cookie, 73
COOP (Cross-Origin Opener Policy), 71
CORB (Cross-Origin Read Blocking), 72
CORP (Cross-Origin Resource Policy), 71
CORS (Cross-Origin Resource Sharing), 72, 163
COVID-19 планы на случай аварии, 30
CRUD (создание-чтение-обновление-удаление), 28
CSF (Cybersecurity Framework), 37
CSP (Content Security Policy) заголовок, 215
CVE (Единый классификатор уязвимостей), 240
CWE (Единый классификатор слабостей), 240

D

DAST (инструмент динамического тестирования безопасности приложений), 157, 221, 243, 303, 310, 313, 325
 и непрерывное тестирование, 340
 фаззинг, 315
 Burp Suite Pro, 304
 Zap (Zed Attack Proxy), 304
DBA (администратор базы данных), 28, 79
DBO (владелец базы данных), 28
DEP (защита от выполнения данных), 248
DevOps, и CI/CD, 329
DORA (закон о цифровой операционной устойчивости), 34
DoS-атаки, 135
DPAPI (Data Protection API), 222
DR (план аварийного восстановления), 30, 343
DREAM, 286

E

EFF (фонд электронных рубежей), 62
Express.js, 208

F

FEDRAMP (Федеральная программа управления рисками и авторизацией), 217
FISMA (Федеральный закон о модернизации информационной безопасности 2014 года), 34

Flask, 234
Fortify, 298

G

GDPR (Общий регламент ЕС по защите персональных данных), 34, 60, 64
GRC (управление рисками и комплаенсом), 106

H

HIPAA (закон о преемственности и подотчетности медицинского страхования), 34
HITECH (закон о применении медицинских информационных технологий в экономической деятельности и клинической практике), 35
HTML-архив (HAR), 325
HTML (Hypertext Markup Language), 174
 рекомендации, 174
 чего избегать, 175
 HTML5, 176
HTTPS, 62

I

IaC сканеры, 300
IAST (интерактивное тестирование безопасности приложений), 37, 243, 315
IDE, 117
 плагины, 299
IDOR (небезопасная прямая ссылка на объект), 243
IoT (интернет вещей)
 и охват, 309
 рекомендации, 165
IPS/IDS (системы предотвращения и обнаружения вторжений), 128
IR (группа реагирования на инциденты), 341

J

Java
 рекомендации, 188
 сериализация, 191
 чего избегать, 192
JavaScript
 рекомендации, 170
 чего избегать, 172
jQuery, 217
JSX синтаксис, 210

К

KISS (будь проще, глупышка), 41

L

Log4j, 110

N

NIST (Национальный институт стандартов и технологий), 217

специальные публикации, 35

Node.js, 184

вопросы безопасности, 186

рекомендации, 184

O

Okta, 54

ORM (объектно-реляционное отображение), 51

OWASP

канал Slack, 96

каркас знаний о безопасности, 36

определение моделирования

угроз, 286

открытый проект по безопасности

веб-приложений, 36

шпаргалка по предотвращению

XSS, 50

SAMM (модель зрелости контроля качества ПО), 36

P

PASTA, 286

PCI DSS (стандарт защиты информации в индустрии платежных карт), 35

Permissions-Policy заголовков, 69

PHP, 198

кодирование вывода, 50

PIPEDA (закон о защите персональной информации и электронных документах), 64

PWA (прогрессивные веб-приложения), 159

Python

рекомендации, 178

чего избегать, 179

Q

QuietAI, 298

R

RASP (самозащита приложения на этапе выполнения), 37, 338

RCE (удаленное выполнение кода), 86

React.js, 210

Redux JS, 211

Ruby on Rails, 226

S

SaaS (программное обеспечение как услуга), в системах CI/CD, 329

SAMM (модель оценки зрелости ПО), 36

SAST (статическая проверка приложения на безопасность), 117, 243, 298, 325

в системах CI/CD, 323

и непрерывное тестирование, 340

SBOM (список зависимостей программного обеспечения), 217

Semgrep Code, 298

SIEM (информационно-аналитическая система безопасности, 95

Snyk Code, 298

Spring, 232

Spring Boot, 232

SQL

встраиваемый код, 53

рекомендации по безопасности, 180

рекомендации по укреплению базы данных и сервера, 180

SSO (единая точка входа), 54

STRIDE, 286

T

TFS (Team Foundation Server), 85

TLS (Transport Layer Security), 99

Trike, 286

TypeScript, 193

V

VAST, 286

Veracode, 298

Vue.js, 219

W

WAF (брандмауэр веб-приложений), 337

WSS (WebSocket Protocol), 163

X

XSS (межсайтовый скриптинг), 46, 49

XXE (внешние сущности XML), 254

Z

Zap (Zed Attack Proxy), 304

A

Автоматизированное
тестирование, 313

Авторизация, 53

управление доступом, 55

Адама Шостака вопросник
по моделированию угроз, 286

Административные элементы
безопасности, 137

Адресное пространство, 245

Анализ

кода, 297

потока, 259

состава программы (SCA), 111, 243,
326

зависимости, 301

Атаки

воспроизведением, 257

полным перебором, 135

типа «отказ в обслуживании», 135

Аудитопригодный код, 118

Аутентификация, 53

двухфакторная, 80

Б

Базы данных, 78

восстановление, 79

маскирование данных, 80

минимальные привилегии, 80

мониторинг запросов, 80

откат, 81

пароли, 80

поставщики облачных услуг, 80

серверы, 78

сети, 79

требования к безопасности, 284

шифрование в процессе передачи, 79

Безопасность

связи, 62

через неясность, 32

Безопасные cookie, 73

Безопасные ограждения, 299

Безопасные умолчания, 29

для пользователей, 115

для разработчиков, 116

Безопасный жизненный цикл
разработки системы (SDLC), 44, 105

Белые списки, 123

Бессерверные вычисления,
рекомендации, 164

Блокировка, 336

брандмауэры веб-приложений
(WAF), 337

виртуальное латание, 338

исследователи данных, 339

ресурсов, 97

самозащита приложения на этапе
выполнения (RASP), 338

сети доставки содержимого
(CDN), 338

шлюзы API, 338

Буфер

переполнение, 244

чтение за границей, 245

В

Важные функции, 122

Веб-сокеты, рекомендации, 163

Виртуальная машина, 133

Виртуальное латание, 338

Владелец продукта, 286

Внедрение

кода, 249

SQL-кода, 52

слепое, 251

Вымощенные дороги, 29

Выпуск, 322

защита целостности, 330

одобрение группой безопасности, 331

Г

Групповые политики, 57

Д

Деревья атак, 286

Десериализация, 89

Дефекты (bugs), 106

исправление разработчиками, 320

ложноположительные, 320

повторное тестирование после
исправления, 40

предотвращение, 243

причины, 243

программы вознаграждения, 316

риск, 243

Дешифрование, 99
 Диаграмма потоков данных (ДПД), 287, 289
 Динамически типизированные языки, 101
 Динамическое тестирование, 310
 Доверенные типы, 215
 Доверие
 пересечение границ, 259
 подразумеваемое, 25
 Документация, 130

Е

ЕС закон о кибербезопасности, 34

З

Зависимости
 анализ состава программы (SCA), 111
 безопасность, 110
 проверка и обновление, 300
 Заголовки безопасности
 доверенные типы, 67
 заголовки запроса метаданных, 66
 политика одного источника, 70
 COEP (Cross-Origin Embedder Policy), 70
 Content-Security-Policy, 68
 COOP (Cross-Origin Opener Policy), 71
 CORB (Cross-Origin Read Blocking), 72
 CORP (Cross-Origin Resource Policy), 71
 CORS (Cross-Origin Resource Sharing), 72
 Expect-CT, 69
 HPKP (Public Key Pinning Extension), 69
 HSTS (HTTP Strict-Transport-Security), 68
 Permissions-Policy, 69
 Referrer-Policy, 69
 X-Content-Type-Options, 68
 X-Frame-Options, 68
 X-XSS-Protection, 69
 Загрузка файлов на сервер, 83
 исходный код, 84
 переполнения, 87
 управление памятью, 85
 Закон об уведомлении в случае нарушения безопасности, 34

Закрытие после отказа, 34
 Защита
 от перезаписи при структурной обработке исключений (SEHOP), 248
 от разбиения стека, 162
 Защитники маршрутов, 216
 Зонирование сети, 25

И

Идентификация, 53
 Изображение архитектуры на доске, 289
 ИИ (искусственный интеллект), безопасное использование, 352
 Имена хостов (проверка), 125
 Интероперабельность, 341
 Инциденты безопасности, 341
 Исправление, 39

К

Канадская налоговая служба (CRA), 132
 Каркасы, 44, 208
 безопасности, 36
 веб-каркасы (JavaScript)
 Angular, 213
 Express.js, 208
 jQuery, 217
 React.js, 210
 Vue.js, 219
 Flask, 234
 .NET (Core), 221
 Ruby on Rails, 226
 Spring, 232
 Spring Boot, 232
 Каскадные таблицы стилей (CSS), 174
 рекомендации, 174
 чего избегать, 175
 Квантовые вычисления, 99
 Ключи доступа, 61
 Код
 аудитопригодный, 118
 повторно используемый, 126
 удобочитаемый, 118, 119
 Кодирование, 47
 вывода, 48
 Комментарии, 128
 Комплаенс, 34
 Конфиденциальность, 89
 интернет-кеш, 91

персональная идентифицирующая информация (ПИ), 91
персональные медицинские данные (РНИ), 91

Корректирующий элемент безопасности, 137

Кража данных, 63

Криптография, 62, 98

криптографическое уничтожение, 60

одноразовое случайное число, 213

хеширование, 99

.NET, 224

Купи, заимствуй, создай, 253

Л

Линтер, 300, 326

Логические просчеты, 240

источники, 241

предотвращение, 242

риск, 241

М

Маршрутизаторы, 216

Маскирование данных, 80

Межсайтовая подделка запросов (CSRF), 164, 243

Метод водопада, 133

Метрики, 335

Минимизация данных, 63

Мобильные приложения
и охват, 309

рекомендации, 159

Моделирование угроз, 39

вопросник Адама Шостака, 286

диаграмма потоков данных (ДПД), 287

определение OWASP, 286

Модель партнерства, 44

Мониторинг, 95

журналы, 335

метрики, 335

трассировка, 335

уведомления безопасности, 335

Мошеннические транзакции, 256

Н

Наблюдаемость, 334

Наставники, 360

Небезопасная десериализация, 89

Неизвестное состояние, 271

Неизменяемые сборки, 132

Ненадежные системы, 121

Неопределенное состояние, 272

Непрерывное обучение, 354

Непрерывное тестирование, 340

Нулевое доверие, 31

Нулевой указатель, разыменован, 246

О

Область безопасности, 212

Обнаруживающий элемент безопасности, 137

Обновления, 39

Обработка ошибок, 74, 92

внедрение в журнал, 94

Обратная совместимость, 38

Объектно ориентированное программирование (ООП), 261

инкапсуляция, 262

наследование, 262

полиморфизм, 263

предметно-ориентированная разработка, 102

Одноразовое случайное число, 213

Одностраничные приложения (SPA), 219

Оснащение, 315

Ответственный за безопасность, 356

Откат, 81

Отлавливание ошибок, 92

Охват, 309

Охотники за вознаграждением, 316

П

Параметризованные запросы, 51

Пересечение границ доверия, 259

Перехват управления учетной записью, 317

Перчение, 61

Планирование действий

в чрезвычайных ситуациях, 30

Поверхность атаки, уменьшение, 33

Подписание кода, 133

Подсаливание, 61

Политика одного источника, 70

Пользовательские истории, 290

Превентивный элемент безопасности, 137

Предметно-ориентированная разработка (DDD), 102
 Предположение возможности взлома, 30
 Прерывание сборки, 324
 Принцип наименьших привилегий, 27
 безопасность базы данных, 80
 Проблемы
 аутентификации, 252
 доступа, 254
 идентификации, 252
 конфигурации, 254
 при работе с объектами
 десериализация, 264
 нарушение авторизации на уровне
 объекта (BOLA), 264
 при работе с памятью, 244
 при работе с ресурсами, 270
 при работе с файлами, 260
 реализации, 254
 сетевой идентификации, 252
 Проверка входных данных, 45
 Проектирование
 парадигмы, 107
 просчеты, 240
 Протоколирование, 95

Р

Разделение обязанностей, 134
 Расположение рамок проекта, 44
 Расширенные привилегии, 137
 Регламент по кибербезопасности
 департамента финансовых услуг города
 Нью-Йорка, 35

С

Санирование, 47
 Сериализация, 89
 Система управления уровнем
 безопасности (CSPM), 117
 Системные вызовы, 127
 Сканер оценки уязвимостей, 308
 Сканирование секретов, 325
 плагины для IDE, 300
 Слабо типизированные языки, 101
 Смена работы
 переход в группу безопасности, 358
 Согласие пользователя, 131
 Состояние, 92

Состояния гонки, 247, 267
 Статический анализ, 297
 безопасные ограждения, 299
 бесплатные инструменты, 298
 плагины для IDE, 300
 Статически типизированные
 языки, 101
 Страничный отказ, 245
 Строго типизированные языки, 100

Т

Тестирование
 автоматизированное, 313
 в производственной среде, 308
 глубина, 308
 динамическое (DAST), 303
 инфраструктура, 308
 методом белого ящика, 297
 методом черного ящика, 304
 на проникновение, 309
 непрерывное, 340
 основанное на глубине, 310
 основанное на покрытии, 310
 повторное тестирование
 исправлений, 40
 покрытие, 307
 причины, 306
 программа вознаграждения
 за найденные ошибки, 316
 результаты, 318
 ручное, 311
 сканирование секретов, 325
 фаззинг, 315
 хронометраж, 307
 этические хакеры, 316
 IAST (интерактивное тестирование
 безопасности приложений), 315
 Zap (Zed Attack Proxy), 304
 Технические элементы
 безопасности, 136
 Типобезопасность, 224
 Трассировка, 335

У

Уведомление, 95
 журналы, 335
 метрики, 335
 трассировка, 335
 Унаследованные системы, 334

Универсальный уникальный идентификатор (UUID), 161

Управление

доступом, 55

избирательное (DAC), 56

мандатное (MAC), 56

модели, 56

на основе правил, 56

ролевое (RBAC), 56

списки управления доступом (ACL), 56

управление сеансами, 57

управление секретами, 58

управление файлами, 82

CI/CD, 327

паролями, 60, 98

безопасные умолчания, 116

беспарольные системы, 60

перчение, 61

подсаливание, 61

смена через каждые 90 дней, 219

секретами, 58, 117

инструменты, 302

проблемы, 265

файлами, 81

система мониторинга целостности

файлов (FIM), 128

Управляемое выполнение, 224

Утечка

данных, 63

памяти, 246

Уязвимости, 239

внедрение кода, 249

дефекты в коде, 242

логические просчеты, 240

мошеннические транзакции, 256

неизвестное состояние, 271

пересечение границ доверия, 259

проблемы авторизации

и доступа, 254

проблемы аутентификации

и идентификации, 252

проблемы интерпретатора

и компилятора, 249

проблемы конфигурации

и реализации, 254

проблемы при работе

с объектами, 261

проблемы при работе с памятью, 244

проблемы при работе

с ресурсами, 270

проблемы при работе с файлами, 260

проблемы с входными данными, 251

проблемы управления секретами, 265

проблемы хронометража, 267

просчеты проектирования, 240

состояния гонки, 267

страничные отказы, 245

Ф

Фаззинг, 315

в конвейере CI/CD, 324

Физические элементы

безопасности, 136

Х

Хорошие привычки, 346

Хранимые процедуры, 53

Ц

Целеполагание, 351

Целостность

определение, 26

проверка, 132

цепочки поставок ПО, 133

Цепочки поставок, 111

безопасность, 32

Ч

Черные списки, 123

Ш

Шелл-код, 86

Шифрование, 99

данных в процессе передачи, 79

доморощенное, 62

Шлюзы API, и блокировка, 338

Э

Экранирование, 47

Эксплойты, 116

Элементы безопасности, 136

Эмулятор, 159

Этические хакеры, 316

Эшелонированная оборона, 31

Я

Языки

безопасные относительно

памяти, 103, 247

динамически типизированные, 101
слабо типизированные, 101
статически типизированные, 101
строго типизированные, 100
C#, 194
C/C++, 203
HTML и CSS, 174

Java, 187
JavaScript, 169
Node.js, 184
PHP, 198
Python, 178
SQL, 180
TypeScript, 193

Книги издательства «ДМК Пресс»
можно купить оптом и в розницу на складе издательства по адресу:
Москва, ул. Электродная, д. 2, стр. 12, офис 7, тел. +7 (499) 322-19-38,
а также заказать на сайте **www.dmkpress.com**
с доставкой в любой регион РФ

Таня Янца

Алиса и Боб учатся безопасному кодированию

Главный редактор	<i>Мовчан Д. А.</i>
Зам. главного редактора	<i>Яценков В. С.</i> editor@dmkpress.com
Перевод	<i>Слинкин А. А.</i>
Корректор	<i>Абросимова Л. А.</i>
Верстка	<i>Чаннова А. А.</i>
Дизайн обложки	<i>Мовчан А. Г.</i>

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 33,31. Тираж 100 экз.

Веб-сайт издательства: **www.dmkpress.com**

Раскройте секреты борьбы с уязвимостями, основанные на изначальной безопасности кода!

Книга предлагает принципиально новый подход к безопасной разработке кода. Истории из жизни вымышленных персонажей Алисы и Боба, примеры реальных ситуаций, технические объяснения и диаграммы легко объясняют запутанные идеи и понятия безопасности и позволяют сразу же начать применять их. Даны общие советы для любого технического стека, а также более конкретные рекомендации для десятка языков программирования и всех известных классов уязвимостей. Вы научитесь создавать безопасный код на таких популярных языках, как Python, Java, JavaScript и других, а также получите опыт защиты фреймворков Angular, .Net и React.

Краткое содержание:

- безопасное кодирование на Python, Java, Javascript, C/C++, SQL, C#, PHP и других языках;
- безопасность в Angular, Express, React, .Net и Spring;
- рекомендации по обеспечению безопасности API, мобильных приложений, веб-сокетов, бессерверных приложений, IoT-устройств и сетки служб;
- основные категории уязвимостей, как они возникают, их риски и способы их избежать;
- безопасный жизненный цикл разработки системы;
- моделирование угроз, тестирование и анализ кода;
- основы создания безопасного кода, не зависящие от технологии и применимые к любому языку и каркасу.

Издание предназначено для широкой аудитории, включающей разработчиков ПО всех уровней, инженеров по безопасности, архитекторов программ и специалистов в области безопасности приложений. Знакомство с практическими примерами и конкретными приложениями углубит ваше понимание важнейших принципов безопасности и позволит прочно усвоить их.



www.dmk.pf



ISBN 978-5-93700-388-1



9 785937 003881 >